NATIONAL POSTGRADUATE ENTRANCE EXAMINATION

408

计算机学科专业基础综合

系统复习讲义 · 章节训练 · 三套模拟卷

数据结构 · 计算机组成原理 · 操作系统 · 计算机网络

2026 REVIEW EDITION

luoyue

github.com/luoyueyuguang

目录

第一部分　数据结构 **1**

1　线性表 2
- 1.1　先把“排队”这件事说清楚 2
- 1.2　顺序表：让元素挨着放 2
- 1.3　单链表：用指针把散落的结点串起来 5
- 1.4　双链表与循环链表 8
- 1.5　静态链表：拿数组下标当指针 9
- 1.6　双指针：把两趟扫描压成一趟 10
- 1.7　顺序表与链表怎么选 12

2　习题 13
- 2.1　基础过关题 13
- 2.2　真题风格与综合训练 14

3　栈与队列 16
- 3.1　栈：把操作挤到一端 16
- 3.2　队列与循环队列 19
- 3.3　表达式：一棵树，三种遍历 23
- 3.4　双端队列 25
- 3.5　数组与特殊矩阵的压缩存储 26
- 3.6　栈与队列的典型应用 27

4　习题 30
- 4.1　基础过关题 30
- 4.2　真题风格与综合训练 31

5　串 33
- 5.1　串的存储与基本操作 33
- 5.2　朴素匹配：失配就从头再来 34
- 5.3　KMP：让主串指针只向前走 35
- 5.4　next 数组怎么算 36
- 5.5　前缀函数：同一个量的另一种记法 38
- 5.6　nextval：跳过注定失败的比较 38

6　习题 40
- 6.1　基础过关题 40
- 6.2　真题风格与综合训练 41

7　树与二叉树 43
- 7.1　树的形状与几条计数规律 43

7.2 二叉树：左右必须分清 44
7.3 二叉树的两种存储布局 45
7.4 遍历：三种深度优先与一种广度优先 46
7.5 由遍历序列还原二叉树 49
7.6 线索二叉树：把空指针用起来 50
7.7 树、森林与二叉树的互相表示 52
7.8 Huffman 树：让总码长最短 53
7.9 二叉排序树 54
7.10 AVL 树：把树高压住 56
7.11 并查集：只关心“谁和谁一伙” 57
7.12 几组容易混的说法 58
8 习题 59
8.1 基础过关题 59
8.2 真题风格与综合训练 60
9 图 62
9.1 图的基本概念 62
9.2 图的存储 63
9.3 图的遍历 64
9.4 最小生成树 66
9.5 最短路径 68
9.6 拓扑排序 70
9.7 AOE 网与关键路径 71
9.8 本章算法对照 73
10 习题 74
10.1 基础过关题 74
10.2 真题风格与综合训练 76
11 查找 79
11.1 从“比较几次”说起 79
11.2 顺序查找：哨兵只是省掉一次边界判断 80
11.3 折半查找：判定树把比较次数画出来 81
11.4 分块查找：索引表加块内顺序 82
11.5 二叉排序树 83
11.6 AVL 树：把树高控制在 $O(\log n)$ 85
11.7 B-树：为磁盘访问设计的多路查找树 86
11.8 B+ 树：把索引和数据分开 88
11.9 散列表 89
11.10 红黑树 91
11.11 几种查找结构放在一起看 92
12 习题 94
12.1 基础过关题 94
12.2 真题风格与综合训练 95
13 排序 97
13.1 插入排序 97
13.2 交换排序 101
13.3 选择排序 104
13.4 归并排序 106
13.5 基数排序 108

13.6 内部排序方法的比较 109
13.7 外部排序 111
14 习题 114
14.1 基础过关题 114
14.2 真题风格与综合训练 115
15 算法设计专题 117
15.1 分治：把问题对半切开 117
15.2 贪心：只保留一个候选 120
15.3 动态规划：把子问题的答案存起来 122
15.4 回溯：在解空间树上走到底再回头 124
15.5 复杂度：把账算清楚 126
15.6 四条路线怎么选 129
16 习题 130
16.1 基础过关题 130
16.2 真题风格与综合训练 131

第二部分 计算机组成原理 134
1 计算机系统概述 135
1.1 一条指令在机器里走一遍 135
1.2 硬件之上：软件与层次结构 136
1.3 性能：指令、周期与时间 137
1.4 Amdahl 定律：一部分加速能换来多少 139
1.5 存储层次：快、大、便宜不能同时要 140
1.6 容易混淆的几组概念 141
2 习题 143
2.1 基础过关题 143
2.2 真题风格与综合训练 144
3 数据的表示与运算 146
3.1 进位计数制与常见编码 146
3.2 校验码 147
3.3 补码：把减法变成加法 147
3.4 定点数与溢出 149
3.5 IEEE 754 浮点数 151
3.6 浮点加减运算 152
3.7 乘除运算 153
3.8 算术逻辑单元 155
4 习题 157
4.1 基础过关题 157
4.2 真题风格与综合训练 158
5 存储系统 160
5.1 存储器的分类 160
5.2 从一次循环看存储层次与局部性 160
5.3 主存芯片的扩展与多体并行 162
5.4 SRAM 与 DRAM 的存储单元 163
5.5 Cache 的块、地址划分与映射 165
5.6 Cache 的替换算法 167
5.7 Cache 的写策略与性能 168

5.8 虚拟存储器与页表 170
5.9 TLB 与 Cache 的协同 171
5.10 缺页异常与页面置换 173
5.11 段式与段页式 174
5.12 几组容易混淆的概念 175
5.13 拓展阅读 175
6 习题 176
6.1 基础过关题 176
6.2 真题风格与综合训练 177
7 指令系统 179
7.1 指令格式：字段怎么放 180
7.2 地址码数量与操作数来源 181
7.3 扩展操作码 181
7.4 数据寻址方式 182
7.5 指令寻址与转移 184
7.6 指令的几大类 185
7.7 CISC 与 RISC 185
7.8 拓展阅读 186
8 习题 187
8.1 基础过关题 187
8.2 真题风格与综合训练 188
9 中央处理器（CPU） 190
9.1 指令周期与定时 190
9.2 数据通路 191
9.3 控制器 193
9.4 流水线 195
9.5 数据冒险与转发 197
9.6 控制冒险 199
9.7 结构冒险与流水线控制 200
9.8 从流水线到多核 200
9.9 把这一章串起来 200
10 习题 201
10.1 基础过关题 201
10.2 真题风格与综合训练 202
11 总线 204
11.1 带宽：宽度、频率与每周期传送次数 205
11.2 突发传输：一次地址，多个数据 206
11.3 仲裁：多个主设备怎么排队 207
11.4 定时：按节拍走，还是按握手走 209
11.5 从共享并行到点对点串行 211
12 习题 212
12.1 基础过关题 212
12.2 真题风格与综合训练 213
13 输入输出系统 215
13.1 从一根导线开始：I/O 接口做什么 215
13.2 程序查询：CPU 守着设备 216

13.3 中断：让设备主动找 CPU 217
13.4 DMA：把搬数据交给硬件 219
13.5 磁盘：机械运动的代价 221
13.6 几种 I/O 控制方式放在一起看 223
14 习题 225
14.1 基础过关题 225
14.2 真题风格与综合训练 226

第三部分 操作系统 228
1 操作系统概述 229
1.1 一次运行里，操作系统做了什么 229
1.2 多道程序：让 CPU 不等 I/O 230
1.3 并发不是并行 231
1.4 共享、虚拟与异步 231
1.5 从批处理到实时系统 232
1.6 内核态与用户态 232
1.7 大内核与微内核 233
1.8 中断与异常 234
1.9 系统调用 236
1.10 操作系统的体系结构 237
1.11 容易混淆的几组概念 238
1.12 本章在 408 中的位置 239
2 习题 240
2.1 基础过关题 240
2.2 真题风格与综合训练 241
3 进程管理 243
3.1 程序与进程 243
3.2 进程状态：PCB 在队列间移动 244
3.3 线程 245
3.4 进程通信 245
3.5 处理机调度 246
3.6 同步与互斥 248
3.7 经典同步问题 250
3.8 死锁 252
3.9 几个容易混淆的判断 254
4 习题 255
4.1 基础过关题 255
4.2 真题风格与综合训练 256
5 内存管理 258
5.1 地址空间、重定位与内存保护 259
5.2 连续分配与碎片 260
5.3 分页：把地址切成页号和页内偏移 261
5.4 TLB：把常用的页表项放到手边 264
5.5 分段与段页式 265
5.6 虚拟内存与请求分页 266
5.7 页面置换算法 269
5.8 页面分配、工作集与抖动 271

5.9 做内存管理题的顺序 273
6 习题 274
6.1 基础过关题 274
6.2 真题风格与综合训练 275
7 文件管理 277
7.1 路径名、目录项与 inode 277
7.2 inode 与文件系统的磁盘布局 278
7.3 数据块怎么放：连续、链接与索引 279
7.4 空闲空间管理 282
7.5 磁盘调度 283
7.6 把这一章串起来 285
8 习题 286
8.1 基础过关题 286
8.2 真题风格与综合训练 287
9 I/O 管理 289
9.1 设备、控制器与总线 289
9.2 I/O 控制方式 290
9.3 I/O 软件层次 292
9.4 缓冲区管理 293
9.5 SPOOLing 296
9.6 设备分配 297
9.7 几组容易混淆的概念 299
10 习题 300
10.1 基础过关题 300
10.2 真题风格与综合训练 301
11 综合专题 303
11.1 时间线：CPU、磁盘和就绪队列 303
11.2 资源账本：谁占着，谁在等 305
11.3 一次访存：TLB、页表、Cache 与缺页 306
11.4 文件访问：从偏移量到磁头 308
11.5 把口径写进答案 310
12 习题 311
12.1 基础过关题 311
12.2 真题风格与综合训练 313

第四部分 计算机网络 317

1 计算机网络体系结构 318
1.1 分层：一次网页访问拆开来看 318
1.2 封装：首部是怎样一层层加上的 319
1.3 三种交换方式：电路、报文、分组 320
1.4 带宽、时延与“管道” 321
1.5 OSI 七层与 TCP/IP 四层 323
1.6 协议、服务与 SAP 324
1.7 本章在 408 中的位置 325
2 习题 326
2.1 基础过关题 326
2.2 真题风格与综合训练 327

3 物理层 329
3.1 数据通信的基本量 330
3.2 信道能传多快：奈奎斯特与香农 331
3.3 数字数据的编码 332
3.4 数字数据的调制 333
3.5 模拟信号的数字化：PCM 334
3.6 传输介质 334
3.7 信道复用 335
3.8 中继器与集线器 336
4 习题 338
4.1 基础过关题 338
4.2 真题风格与综合训练 339
5 数据链路层 341
5.1 帧定界：从连续的比特里切出帧 341
5.2 差错检测与纠正 342
5.3 可靠传输：确认、超时与滑动窗口 343
5.4 介质访问控制 345
5.5 CSMA/CD：共享以太网如何抢信道 346
5.6 以太网帧与 MAC 地址 347
5.7 交换机与自学习 348
5.8 VLAN 349
5.9 PPP 349
5.10 无线局域网为什么不用 CSMA/CD 350
5.11 408 高频考点速查 351
6 习题 352
6.1 基础过关题 352
6.2 真题风格与综合训练 353
7 网络层 355
7.1 网络层在整条路径上的位置 355
7.2 IPv4 地址与数据报 355
7.3 子网、掩码与 CIDR 357
7.4 ARP 与 ICMP 358
7.5 路由表、转发与路由协议 360
7.6 NAT 362
7.7 一次访问的完整路径 362
8 习题 363
8.1 基础过关题 363
8.2 真题风格与综合训练 364
9 传输层 367
9.1 端口、套接字与复用分用 367
9.2 UDP：只加一层薄薄的信封 367
9.3 可靠传输：从停等到滑动窗口 368
9.4 TCP 报文段与可靠传输 371
9.5 TCP 的流量控制 373
9.6 TCP 的拥塞控制 374
9.7 连接管理 375
9.8 把本章串起来 378

10 习题 378
10.1 基础过关题 378
10.2 真题风格与综合训练 380
11 应用层 382
11.1 DNS：把域名换成 IP 地址 382
11.2 HTTP：一次请求到底发生了什么 384
11.3 FTP：两条连接分工 387
11.4 电子邮件：SMTP 发，POP3/IMAP 收 388
11.5 DHCP：主机怎样拿到地址 391
11.6 本章小结：把协议放到一条时间线上 392
12 习题 394
12.1 基础过关题 394
12.2 真题风格与综合训练 395

第五部分 408 模拟试卷 **398**
1 模拟卷一 399
2 模拟卷二 412
3 模拟卷三 425

第六部分 附录 **438**

第一章 真题索引使用说明 **439**

PART I　第一部分

数据结构

1 线性表

1.1 先把“排队”这件事说清楚

把班里的成绩单按学号排成一行：第一行是 1 号，第二行是 2 号……这张表能做的操作很有限——取第 5 行、找学号为 2024003 的那一行、在第 3 行前插进一行、删掉第 7 行。注意这些操作只关心元素排成的**次序**，不关心每个元素具体是什么，也不关心它存在哪里。把“排成一队”这件事单独抽出来，就是线性表。

定义 1.1: 线性表

线性表是由 n $(n \geqslant 0)$ 个数据元素构成的有限序列 $a_1, a_2, \ldots, a_n$。除首元素外，每个元素有且只有一个直接前驱；除末元素外，每个元素有且只有一个直接后继。n 称为表长，$n = 0$ 时是空表。
元素在序列中的序号 $1, 2, \ldots, n$ 称为**位序**，它从 1 开始；C 数组的下标从 0 开始，两者相差 1，这是后面大量边界错误的来源。

线性表是**逻辑结构**。真正放进内存时，硬件只提供两样东西：地址，以及该地址上存的内容。要让逻辑上相邻的两个元素在物理上也相邻，就把它们塞进连续的存储单元，这是**顺序表**；不要求物理相邻，而是在每个元素旁边记下下一个元素在哪里，这是**链表**。同一条逻辑上的线性表，可以有两种存法，后面所有“谁更快”的比较，本质上都是这两种映射方式的比较。

线性表的抽象数据类型把“能对它做什么”固定下来，与存法无关：

线性表的 ADT（C 风格）

操作	语义
`InitList(L)`	构造一个空表
`Length(L)`	返回表长
`GetElem(L, i)`	取位序为 i 的元素（按位查找）
`LocateElem(L, e)`	找第一个值等于 e 的元素（按值查找）
`Insert(L, i, e)`	在第 i 位之前插入 e
`Delete(L, i)`	删除第 i 位元素
`Empty(L)`	判空

表里有两个极容易混的操作：`GetElem` 是“按位置取”，`LocateElem` 是“按值找”。在顺序表上，前者 $O(1)$、后者 $O(n)$；在链表上，两者都是 $O(n)$。408 的选择题很喜欢在这里设陷阱，比如“顺序表查找的时间复杂度是 $O(1)$”——只有按位查找才成立。

1.2 顺序表：让元素挨着放

先看一个具体的表：`int data[8] = {12, 7, 25, 18, 33}`，`length = 5`。五个有效元素占数组的前五个单元，每个 `int` 占 4 个字节，首地址是 2000。



前 5 个单元有效，后两个尚未使用

图里的关键信息是：每个单元的下标和地址都能直接读出，相邻单元的地址差固定为 4。因此第 i 个元素的地址不必从头找，算一下就行：

$$\mathrm{LOC}(a_i) = \mathrm{LOC}(a_1) + (i-1) \times s, \qquad 1 \leqslant i \leqslant n,$$

其中 s 是单个元素占的字节数。例如 a_4 的地址是 $2000 + (4-1) \times 4 = 2012$。这种“给定位置就能在常数时间内算出地址”的能力叫**随机存取**，是顺序表区别于链表的根本特征。

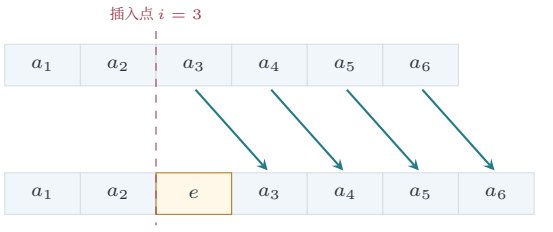
要点 1.1: 顺序表的类型定义

```c
#define MaxSize 50
typedef int ElemType;
typedef struct {
    ElemType data[MaxSize];   // 静态分配的存储区
    int length;               // 当前元素个数，不是最大下标
} SqList;
```

静态数组版本在编译时就定死了容量。若把 `data` 改成指向动态数组的指针，并另记一个容量 `MaxSize`，初始化时用 `malloc` 申请空间、表满时用 `realloc` 换更大的一块并把旧数据拷过去，就是动态顺序表。扩容那一次是 $O(n)$，但容量按倍数增长，连续多次插入平摊下来每次仍是 $O(1)$。408 只需要知道“动态顺序表可以扩容，扩容要整体搬移”。

插入：从后往前搬

在长度为 n 的表中，把 e 插到第 i 位之前（$1 \leqslant i \leqslant n+1$，$i = n+1$ 表示追加到表尾）。原来占着第 i 位及之后的元素都必须往后挪一格，给 e 腾位置。



第 3 个及其后的 4 个元素各后移一位，移动次数 $n - i + 1 = 4$

图中四条斜箭头就是四次赋值。它们必须**从右往左**执行：先把 a_6 放到第 7 格，再放 a_5……如果反过来从 a_3 开始往右搬，a_3 会先覆盖掉尚未搬走的 a_4，数据就丢了。代码里的循环变量从 `length` 递减到 `i`，正是这个道理。

```c
bool ListInsert(SqList *L, int i, ElemType e) {
    if (i < 1 || i > L->length + 1) return false;  // 位序越界
    if (L->length >= MaxSize)       return false;  // 表满
    for (int j = L->length; j >= i; j--)           // 从后往前搬
        L->data[j] = L->data[j - 1];
    L->data[i - 1] = e;                            // 位序 i 对应下标 i-1
```

```
    L->length++;
    return true;
}
```

插在第 i 位要移动 $n-i+1$ 个元素。若 $n+1$ 个合法位置等概率出现，平均移动次数为

$$\frac{1}{n+1}\sum_{i=1}^{n+1}(n-i+1)=\frac{n+(n-1)+\cdots+0}{n+1}=\frac{n(n+1)/2}{n+1}=\frac{n}{2}.$$

最好情况是插在表尾，移动 0 个，$O(1)$；最坏情况是插在表头，整表后移，$O(n)$。

删除：从前往后补

删除第 i 位（$1\leqslant i\leqslant n$）时方向相反：第 $i+1$ 到第 n 个元素依次往前补一格，把空洞填上。这时循环必须从 i 开始递增，否则前面的元素会覆盖还没读取的后继。

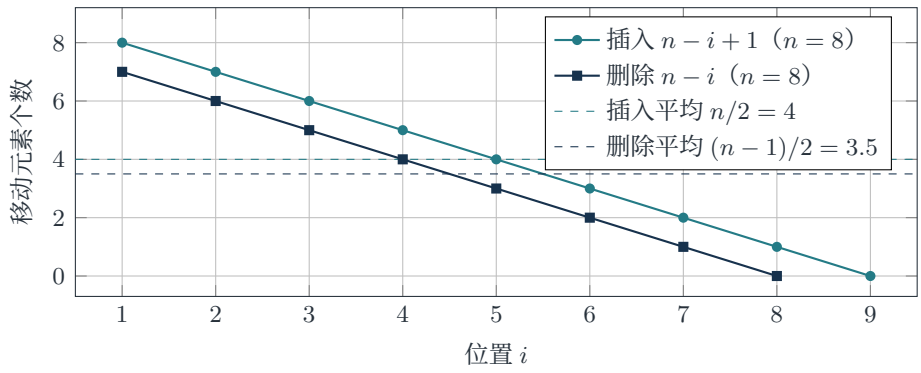
```
bool ListDelete(SqList *L, int i, ElemType *e) {
    if (i < 1 || i > L->length) return false;
    *e = L->data[i - 1];
    for (int j = i; j < L->length; j++)    // 从前往后补
        L->data[j - 1] = L->data[j];
    L->length--;
    return true;
}
```

删除第 i 位要移动 $n-i$ 个元素，平均值是

$$\frac{1}{n}\sum_{i=1}^{n}(n-i)=\frac{(n-1)+(n-2)+\cdots+0}{n}=\frac{n-1}{2}.$$

它比插入的平均值少 $1/2$，原因有两个：删除只有 n 个合法位置（没有“第 $n+1$ 位”），而且被删元素本身不用移动。

把两个公式放在同一张图里看，差异和趋势都很直观：



两条实线都是斜率为 -1 的直线，说明“位置越靠前，搬得越多”；虚线是各自在所有位置上的平均值，插入的平均线始终比删除高半个元素。表长为 n 时，这两条线只是把图中的 $n=8$ 换成对应数值，形状不变。

按值查找

按位取值是 $O(1)$，按值查找则要逐个比较：

```c
int LocateElem(SqList L, ElemType e) {
    for (int i = 0; i < L.length; i++)
        if (L.data[i] == e) return i + 1;   // 返回位序
    return 0;                               // 0 表示未找到
}
```

若每个位置被查到的概率相同，查找成功时平均比较次数为 $(1+2+\cdots+n)/n=(n+1)/2$；查找失败则要把 n 个元素全部比完。两种情况都是 $O(n)$。

注意

顺序表代码里的边界，几乎都出在“位序”和“下标”的换算上：

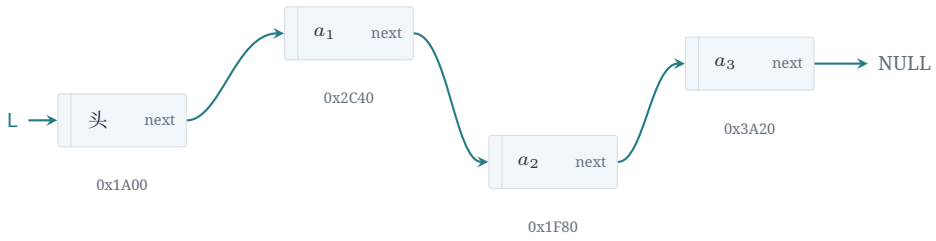
- `length` 是元素个数。表空是 `length == 0`，表满是 `length == MaxSize`，最后一个元素的下标是 `length - 1`。
- 插入的合法范围是 $1\leqslant i\leqslant n+1$，删除是 $1\leqslant i\leqslant n$。判断越界要用当前的 `length`，而不是 `MaxSize`。
- 插入时先判 i 再判满、删除时先判 i 再判空，顺序反了虽然结果不错，但会给出误导性的返回值。

1.3 单链表：用指针把散落的结点串起来

顺序表插入要搬半个表。如果表很长、又频繁在中间插队，搬移就成了主要开销。链表的想法很直接：元素不必挨着放，每个结点在存数据的同时，额外存一个指针，指向下一个结点在哪里。

定义 1.2: 单链表

每个结点含两个域：**数据域** `data` 存元素，**指针域** `next` 存直接后继结点的地址。最后一个结点的 `next` 为 `NULL`。结点在内存中的位置任意，逻辑次序完全由指针决定。



图中每个方框右边的 `next` 存的是下一个方框的地址：a_1 在 0x2C40，a_2 在 0x1F80，两者相距很远，但 a_1 的 `next` 恰好写着 0x1F80，于是逻辑上 a_2 紧跟 a_1。地址没有任何规律，所以第 i 个结点的位置算不出来，只能从头沿 `next` 一步一步走，这就是**顺序存取**，代价是 $O(n)$。

```c
typedef struct LNode {
    ElemType data;
    struct LNode *next;
} LNode, *LinkList;
```

`LinkList` 就是指向 `LNode` 的指针。它指向的可能是首元结点，也可能是一个不存数据的头结点，这两种约定决定了边界代码怎么写。

(a) 带头结点：L 指向头结点，头结点的 data 不用



(b) 不带头结点：L 直接指向首元结点



对比上下两组：带头结点时，即使表空，L 也始终指向一个真实结点，因此“在表头插入/删除”和“在中间插入/删除”可以用同一段代码；不带头结点时，空表是 `L == NULL`，在表头操作必须修改 L 本身（通常把 `LinkList *` 传进去，或让函数返回新的头指针）。题目没有明说时，按题设给出的类型和空表描述判断，不要默认某一套。

按位查找从**头结点**开始计数，头结点算第 0 个：

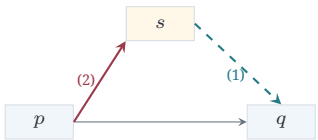
```
LNode *GetElem(LinkList L, int i) {
    if (i < 0) return NULL;
    LNode *p = L;                   // p 指向头结点（第 0 个）
    int j = 0;
    while (p != NULL && j < i) {  // 走 i 步后 p 指向第 i 个结点
        p = p->next;
        j++;
    }
    return p;                       // i 超过表长时返回 NULL
}
```

`GetElem(L, 0)` 返回头结点，`GetElem(L, 1)` 返回首元结点。循环每走一步只做一次指针赋值，走 $i-1$ 步，所以是 $O(i)$，最坏 $O(n)$。

插入与删除：改指针，不搬数据

要在第 i 位插入 e，先找到第 $i-1$ 个结点 p，让新结点 s 接住 p 原来的后继，再让 p 指向 s。

(a) 先接后继，再断前驱



(b) 先断前驱会丢链

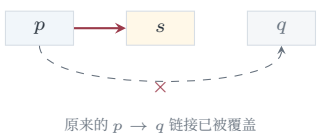


图 (a) 中 (1) 是 `s->next = p->next`，(2) 是 `p->next = s`。(1) 先从 p 那里读出 q 的地址存进 s，(2) 再把 p 的指针改成指向 s。顺序反过来，p 的 `next` 一旦被改写，q 的地址就无处可查，(b) 中那条虚线链接就断了，q 及其后的所有结点全部丢失。

```
bool ListInsert(LinkList L, int i, ElemType e) {
    if (i < 1) return false;
    LNode *p = GetElem(L, i - 1);      // 找第 i-1 个结点
    if (p == NULL) return false;
    LNode *s = (LNode *)malloc(sizeof(LNode));
    if (s == NULL) return false;
    s->data = e;
    s->next = p->next;                 // (1) 新结点接住原后继
    p->next = s;                       // (2) 前驱改指新结点
    return true;
}
```

带头结点时 $i=1$ 的 p 就是头结点，上面这段代码不用特判就能插在表头。找到 p 花 $O(n)$，改指针只有常数次，所以“按位插入”整体仍是 $O(n)$——常说的“链表插入 $O(1)$”有个前提：已经拿到前驱指针。

删除同理，先找到第 $i-1$ 个结点，让它的 next 绕过第 i 个结点，再释放后者：

```
bool ListDelete(LinkList L, int i, ElemType *e) {
    if (i < 1) return false;
    LNode *p = GetElem(L, i - 1), *q;
    if (p == NULL || p->next == NULL) return false;  // 无前驱或无第 i 个
    q = p->next;
    *e = q->data;
    p->next = q->next;
    free(q);
    return true;
}
```

注意

“删除指针 p 所指结点”是一个经典陷阱。如果只给 p、不给前驱，常规做法是把后继的数据复制到 p，再删掉后继：

```
q = p->next;
p->data = q->data;
p->next = q->next;
free(q);
```

它在 $O(1)$ 时间内完成，但有三个前提：p 不是尾结点（否则没有后继可供复制）；结点数据可以整体复制；外部没有别的指针还指着被删的那个后继结点。题目若问“能否在 $O(1)$ 时间内删除”，要连这些条件一起答。

建表：头插与尾插

从空表开始逐个插入 n 个元素，插的位置不同，得到的次序也不同。

头插法把每个新结点插到头结点之后：

```
LinkList List_HeadInsert(int n) {
    LinkList L = (LinkList)malloc(sizeof(LNode));
    L->next = NULL;                        // 头结点
    for (int i = 0; i < n; i++) {
        LNode *s = (LNode *)malloc(sizeof(LNode));
        scanf("%d", &s->data);
```

```
        s->next = L->next;                  // 新结点接住当前首元
        L->next = s;                        // 头结点改指新结点
    }
    return L;
}
```

每次插入都在表头，n 次共 $O(n)$。输入 $1,2,3$，得到的是 $3,2,1$——次序正好相反。这个性质可以直接用来做就地逆置：把头结点摘下，再把原链表的结点依次头插回去。

尾插法要保持输入次序，必须记住尾结点，否则每插一个都要从头走到尾，总代价变成 $O(n^2)$：

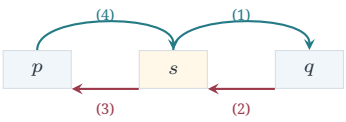
```
LinkList List_TailInsert(int n) {
    LinkList L = (LinkList)malloc(sizeof(LNode));
    LNode *r = L;                           // r 始终指向尾结点
    for (int i = 0; i < n; i++) {
        LNode *s = (LNode *)malloc(sizeof(LNode));
        scanf("%d", &s->data);
        r->next = s;
        r = s;                              // 尾指针后移
    }
    r->next = NULL;                         // 收尾
    return L;
}
```

1.4 双链表与循环链表

单链表只能单向走。要删除结点 p，必须知道它的前驱，而找前驱又得从头再扫一遍。双链表给每个结点再加一个指向前驱的指针，把这个回头路省掉。

```
typedef struct DNode {
    ElemType data;
    struct DNode *prior, *next;
} DNode, *DLinkList;
```

在 p 之后插入 s，一共要改四个指针：



删除 q：



`p->next = q->next;  q->next->prior = p;`

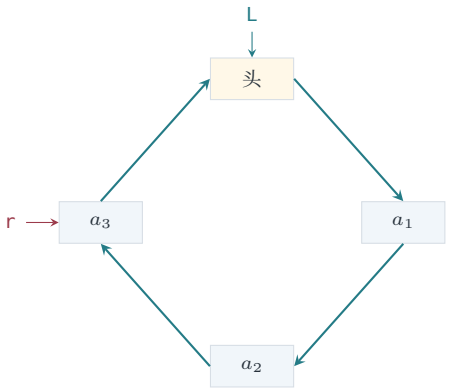
上半部分是插入，青线是 `next` 方向，红线是 `prior` 方向，编号对应四条语句：

```
s->next = p->next;                    // (1) 先接住 p 原来的后继 q
if (p->next != NULL) p->next->prior = s;  // (2) q 回指 s
s->prior = p;                         // (3) s 指回 p
p->next = s;                          // (4) p 才指向 s
```

重要的是 (1)(2)(3) 都必须在 (4) 之前完成：(4) 一执行，`p->next` 就变成了 s，再去读“p 原来的后继”就晚了，而 (2) 若拖到 (4) 之后，修改的会是 s 自己的 `prior`。其中 (2) 要判空，因为 p 可能是尾结点，此时 q 不存在。若链表是循环双链表，`p->next` 永远非空，这个判断可以省掉。

删除 q 只需两步：让 p 的 `next` 指向 r，再让 r 的 `prior` 指回 p，最后释放 q。图下半部分画的就是这两条新边；同样地，q 是尾结点时要判空。

循环链表把尾结点的 `next` 指回头结点，整条链首尾相接：



尾结点 a_3 的 next 指回头结点，不是 NULL

看图验证两个常用结论。第一，带头结点的循环单链表判空条件是 `L->next == L`：表空时头结点自己指向自己，环上只有头结点。第二，若只保存尾指针 `r`（指向 a_3），则 `r->next` 是头结点，`r->next->next` 是首元结点，所以在表头插入和在表尾插入都只要常数时间，不需要遍历。这正是“只设尾指针的循环单链表”常被考的原因。

循环双链表再多一条：头结点的 `prior` 指向尾结点，于是从表头可以 $O(1)$ 找到表尾。判空条件写成 `L->next == L`（此时也必有 `L->prior == L`）。由于任何结点的两个指针都非空，插入删除不需要判空分支。

1.5 静态链表：拿数组下标当指针

有些场合没有指针（早期语言），或者题目要求“不移动元素又要能插删”，这时可以用数组模拟链表：每个分量存数据和下一个分量的**下标**，用 -1 表示结束。

下标	0	1	2	3	4	5
data	—	A	B	C	D	空闲
next	1	3	4	2	-1	—



0 号分量当表头，它的 `next = 1` 指向 A；A 的 `next = 3` 指向下标 3 的 C；C 的 `next = 2` 指向 B；B 指向 D；D 的 `next = -1` 结束。注意逻辑次序是 A、C、B、D，与它们在数组里的物理次序 1、2、3、4 不同——`next` 里存的是列号（数组下标），不是地址，这正是“静态链表”与普通链表的唯一区别。

```
#define MaxSize 100
typedef struct {
    ElemType data;
```

```
    int next;               // 后继分量的数组下标，-1 表示结束
} SLinkList[MaxSize];
```

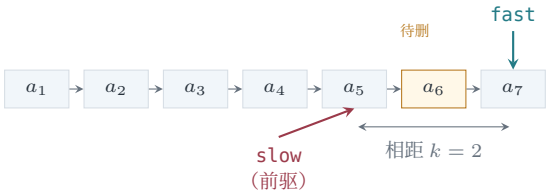
插入时从空闲分量中取一个，改两个 next；删除时把被删分量挂回空闲链。整个过程一个元素都不移动，这是它的优点。代价是容量固定、按位查找仍要沿 next 走 $O(n)$ 步，而且 next 存的是下标，写错一位不会报错，只会悄悄指到别处。

1.6 双指针：把两趟扫描压成一趟

双指针不是一种存储结构，而是一类扫描技巧：用两个指针在同一条序列上以不同速度、或从不同起点移动，让它们各自负责一件事，从而避免嵌套循环。下面四种用法覆盖了 408 的绝大多数考法。

快慢指针找倒数第 k 个

让 fast 先走 k 步，然后两个指针同步后移。由于间距始终是 k，当 fast 停在尾结点时，slow 恰好停在倒数第 k 个结点的前驱上。



图中 $n=7$、$k=2$：fast 先走 2 步到 a_2，同步移动 5 步后停在 a_7，slow 从第 0 个位置（头结点）走到 a_5，而待删的倒数第 2 个是 a_6，正好是 slow->next。两个指针的间距在整段移动中保持为 2，这就是“倒数第 k 个”与“前驱”之间的换算依据。

对应的代码只有两段循环：

```
bool DeleteKthFromEnd(LinkList L, int k, ElemType *e) {
    if (k < 1) return false;
    LNode *fast = L, *slow = L;
    for (int i = 0; i < k; i++) {
        if (fast == NULL) return false;   // k 超过表长
        fast = fast->next;
    }
    if (fast == NULL) return false;
    while (fast->next != NULL) {          // fast 停在尾结点
        fast = fast->next;
        slow = slow->next;
    }
    LNode *q = slow->next;
    *e = q->data;
    slow->next = q->next;
    free(q);
    return true;
}
```

两处边界决定成败：$k=n$ 时 fast 恰好停在尾结点，第二段循环一次也不执行，slow 仍在头结点，删的是首元结点；$k>n$ 时 fast 在走的过程中变成 NULL，必须提前返回。另外，起点

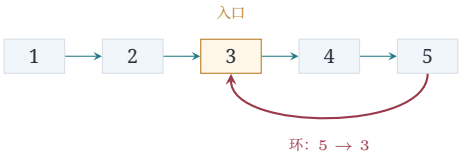
不同结论会平移一位：若从首元结点出发走 k 步，最终 slow 指的就是待删结点本身，而不是它的前驱。答题时要写清楚从头结点还是首元结点开始。

找中间结点

slow 每次走 1 步，fast 每次走 2 步。fast 走到 NULL（偶数个结点）或 fast->next == NULL（奇数个）时，slow 就在中间：n 为奇数时是第 $(n+1)/2$ 个；n 为偶数时是第 $n/2+1$ 个，即后半段的第一个。循环条件必须写成 fast != NULL && fast->next != NULL，因为循环体里要访问两次 next。若题目要求偶数时取前半段的最后一个，把循环体末尾的判断提前一步即可，结论整体左移一位——答题时先看清题目要哪一个中位数。

判环与找环入口

Floyd 判环仍然是一快一慢：slow 一步、fast 两步，有环必相遇，无环则 fast 先撞到 NULL。



图中 1、2 是环外的前缀，3、4、5 构成环，5 的 next 指回 3。两个指针进入环后，fast 每轮比 slow 多走一步，二者在环上的距离每轮缩小 1，所以最多绕一圈就会相遇。

相遇之后要找入口：把 slow 放回头结点，两个指针都改成每次走 1 步，再次相遇的位置就是环入口。设头到入口为 a 步，入口到相遇点为 b 步，环长为 L。相遇时 slow 走了 $a+b$，fast 走了 $a+b+kL$，而 fast 的路程是 slow 的两倍：

$$2(a+b) = a+b+kL \implies a = kL-b = (k-1)L+(L-b).$$

也就是说，从头走 a 步、从相遇点走 $L-b$ 步，会在同一时刻到达入口。

```
bool HasCycle(LinkList L) {
    LNode *slow = L, *fast = L;
    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;          // 一步
        fast = fast->next->next;    // 两步
        if (slow == fast) return true;
    }
    return false;
}
```

判环时两个指针从同一个结点出发即可（通常取表头）；进入环后它们的距离每轮缩小 1，所以有环一定会相遇。求环长则更简单：相遇后让一个指针不动、另一个每次走一步，再次回到相遇点所走的步数就是 L。

顺序表上的双指针

顺序表虽然支持随机存取，双指针仍然能把 $O(n^2)$ 降成 $O(n)$：

- **归并两个有序表**：i、j 各指一个表，每次把较小的那个放进结果表并移动对应指针。每比较一次至少确定一个元素，最坏比较 $m+n-1$ 次。

- **有序表去重**：读指针 i 扫描整个数组，写指针 k 指向已保留部分的末尾。只有当前元素与最后一个已保留元素不同时，才写入 `data[k]` 并让 k 前移。k 永远不超过 i，所以不会覆盖尚未读取的元素。
- **按值划分**：左右指针从两端向中间夹，把满足条件的元素换到左边、不满足的换到右边，只扫描一趟。

这类题的两个易错点：循环条件用 `i < j` 还是 `i <= j`（决定中间那个元素要不要处理），以及写指针与读指针重合时能否安全赋值（能，因为写指针从不超前于读指针）。

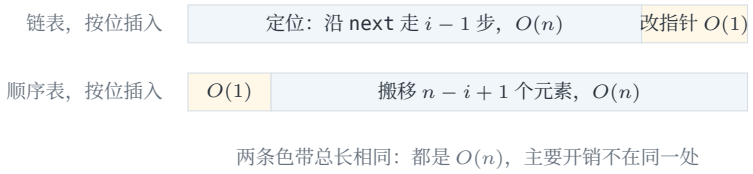
1.7 顺序表与链表怎么选

把两种存法的代价列在一起，比较才有意义：

顺序表与链表的操作代价

操作	顺序表	单链表
按位取第 i 个	$O(1)$（随机存取）	$O(n)$（顺序存取）
按值查找	$O(n)$，平均 $(n+1)/2$ 次比较	$O(n)$，平均 $(n+1)/2$ 次比较
在第 i 位插入	$O(n)$，平均移动 $n/2$ 个	定位 $O(n)$，改指针 $O(1)$
删除第 i 个	$O(n)$，平均移动 $(n-1)/2$ 个	定位 $O(n)$，改指针 $O(1)$
已知前驱指针后插入	仍需搬移，$O(n)$	$O(1)$
空间	预分配，可能留空位	每结点多一个指针域
扩容	需整体搬移，均摊 $O(1)$	单个结点申请，无整体搬移

“链表插入是 $O(1)$”这句话只有在**已经拿到插入位置的前驱指针**时才成立。若题目要求“在第 i 位插入”，链表仍要先花 $O(n)$ 定位，总时间与顺序表同阶；只是两边的钱花在不同地方：



图中蓝色部分一边是“沿指针找位置”，一边是“在连续内存里搬元素”。这也解释了为什么表很长时顺序表反而常常更快：搬移是顺序访问，硬件预取友好，而链表每走一步都可能是一次缓存缺失。反过来，若已经持有某个结点的指针、要在它后面反复插入，链表才是真正的 $O(1)$。

空间上，**存储密度**指数据本身占的字节数与整个结点占的字节数之比。单链表的结点是“数据 + 一个指针”：若 `ElemType` 为 4 字节、指针 4 字节，密度为 $4/8 = 0.5$；64 位系统指针 8 字节，则降到 $4/12 \approx 0.33$。双链表还要再乘一次折扣。顺序表除了末尾可能浪费的预留空间，密度接近 1，但表满时要么扩容、要么失败。

选择的原则并不复杂：需要频繁按位访问、表长变化不大、内存连续，用顺序表；需要频繁在已知位置插入删除、表长难以预估、单元素较大而指针开销可接受，用链表。考试里更常见的不是选型，而是判断某句“顺序表一定比链表快”或“链表插入一定是 $O(1)$”对不对——判断的依据就是上面这张表，以及“定位”是否已经完成。

2 习题

2.1 基础过关题

1. (填空题) 一个长度为 n 的顺序表，若在 $1,2,\ldots,n+1$ 的每个合法位置插入元素的概率相同，则插入一个元素平均需要向后移动 _____ 个元素；若在 $1,2,\ldots,n$ 的每个位置删除元素的概率相同，则删除一个元素平均需要向前移动 _____ 个元素。

2. (单项选择题) 下列关于顺序表和链表的叙述中，正确的是

   (A) 顺序表的插入和删除操作一定比链表慢

   (B) 单链表按位查找第 i 个结点需要 $O(i)$ 时间，找到该结点后在其后插入一个新结点只需 $O(1)$ 时间

   (C) 顺序表只能采用静态数组实现，表满后无法再扩充容量

   (D) 链表结点中含有指针域，所以链表的存储密度高于顺序表

3. (单项选择题) 在带头结点的双链表中，指针 `p` 指向某结点，`q = p->next` 指向其后继结点（`q` 可能为尾结点）。要删除 `q` 并释放其空间，下列语句序列正确的是

   (A) 先执行 `p->next = q->next`；若 `q->next` 非空，再执行 `q->next->prior = p`；最后 `free(q)`

   (B) 先执行 `q->next->prior = p`；再执行 `p->next = q->next`；最后 `free(q)`

   (C) 先执行 `p->next = q->next`；再执行 `q->next->prior = p`；最后 `free(q)`

   (D) 先执行 `p->prior = q->next`；再执行 `q->next->prior = p`；最后 `free(q)`

4. (单项选择题) 带头结点的单链表 `L` 中至少有 k 个数据结点。指针 `p`、`q` 初始都指向首元结点，先让 `p` 沿 `next` 走 $k-1$ 步，再让 `p`、`q` 同步后移，使 `q` 最终指向倒数第 k 个结点。同步后移时循环继续的条件应是

   (A) `p != NULL`　　(B) `p->next != NULL`

   (C) `q->next != NULL`　　(D) `p != q`

5. (**填空题**) 顺序表 L 的长度为 n，其中值等于 x 的元素共有 m 个。现要删除所有值为 x 的元素，并保持剩余元素的相对次序。采用读写指针法：令 $k=0$，依次扫描下标 $i=0,1,\ldots,n-1$，若 `L.data[i] != x`，就执行 `L.data[k++] = L.data[i]`，最后令 `L.length = k`。该算法的时间复杂度为 ____，辅助空间复杂度为 ____，结束后表长为 ____。

6. (**单项选择题**) 不带头结点的单链表，头指针为 L（L 可能为 NULL）。指针 s 指向一个待插入的新结点。要把 s 插入为新的首元结点，正确的语句序列是

(A) `s->next = L; L = s;`　　(B) `L->next = s; s->next = L;`

(C) `s->next = L->next; L->next = s;`　　(D) `L = s; s->next = L;`

2.2 真题风格与综合训练

7. (**单项选择题**) 两个长度分别为 m 和 n 的升序顺序表 A、B（$m,n\geqslant 1$），采用双指针归并：i、j 分别指向 A、B 当前待比较元素，每次比较 $A[i]$ 与 $B[j]$，把较小者放入结果表并移动相应指针；当其中一个表扫描完后，把另一个表的剩余元素直接依次放入结果表。最坏情况下，关键字比较次数为

(A) $m+n$　　(B) $m+n-1$

(C) $\min(m,n)$　　(D) $\max(m,n)$

8. (**单项选择题**) 用头插法将带头结点的单链表就地逆置（不申请新的数据结点）。下列说法正确的是

(A) 需要额外申请 n 个结点暂存数据，空间复杂度为 $O(n)$

(B) 逆置完成后，原首元结点成为新链表的首元结点

(C) 修改当前结点 p 的 next 之前，必须先保存 `p->next`，否则无法继续访问剩余结点

(D) 逆置过程需要两层嵌套循环，时间复杂度为 $O(n^2)$

9. （真题风格・算法设计题）设带头结点的单链表 L 的结点结构为 `[data | next]`，数据结点个数为 n（$n \geqslant 0$）。设计算法将链表就地逆置，要求不新建数据结点、不使用递归，辅助空间为 $O(1)$。写出 C 语言实现，并说明算法为什么正确，分析时间复杂度与空间复杂度。

10. （真题风格・算法设计题）设带头结点的单链表 L 中有 n 个数据结点，结点结构为 `[data | next]`。给定整数 k，设计一个尽可能高效的算法，删除链表中倒数第 k 个结点。要求：只允许从头到尾扫描一趟；$k < 1$ 或 $k > n$ 时返回 `false`，且链表保持不变；删除成功时返回 `true`，并用 `*e` 带回被删结点的数据。分析算法的时间复杂度与空间复杂度。

3 栈与队列

在文本编辑器里连按几次撤销，撤掉的内容正好按输入的逆序消失；打印机则相反，先提交的文档先输出。前一条规则是“后进先出”，后一条是“先进先出”，它们分别对应本章的两个主角——栈和队列。两者都是线性表，只是把插入、删除的位置限制住了：栈只允许在一端动手，队列规定一端进、另一端出。限制减少了可用的操作，却让“记住最近的若干件事”和“按到达顺序处理”变得非常直接。

这一章先看两种结构在顺序存储和链式存储下如何落地，重点是指针的约定和边界条件；然后把循环队列的下标运算讲清楚；接着用表达式求值说明栈的典型用法；最后讨论两端都能进出的双端队列。

3.1 栈：把操作挤到一端

栈只允许在表的一端插入和删除。允许操作的一端叫栈顶（top），另一端叫栈底（bottom）；插入叫入栈（push），删除叫出栈（pop）。

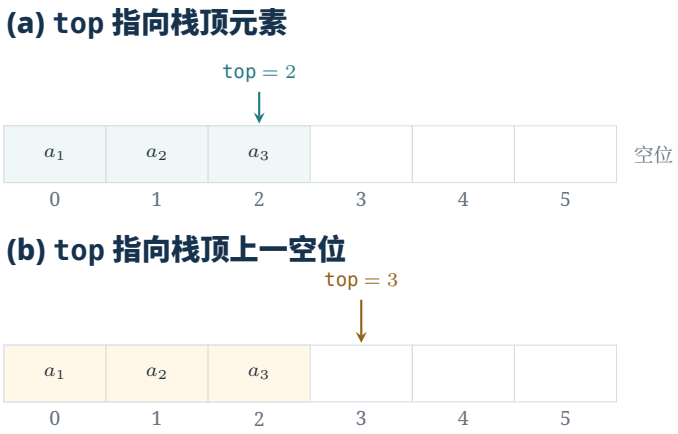
定义 3.1: 栈

栈是只允许在表的一端进行插入和删除的线性表。插入和删除都在栈顶进行，元素按后进先出（LIFO, Last In First Out）的次序离开。

这个定义有个直接推论：栈底元素只有在它上面的元素全部弹出后才可能被访问，所以栈不能像顺序表那样按下标随机存取。反过来说，判空、取栈顶、入栈、出栈都只碰栈顶，时间都是 $O(1)$。

顺序栈：top 到底指哪里

最直接的做法是用一段连续空间 `data[0..MaxSize-1]` 存放元素，再用一个整数 `top` 记录栈顶位置。数组本身没有方向，代码怎么写取决于 `top` 指向哪里。408 常见两种约定：让 `top` 指向栈顶元素，或者让它指向栈顶上面的那个空位。



图怎么看：两种约定的差别只在 `top` 停在哪里。上图 `top` 压在栈顶元素上，数值等于元素个数减一；下图 `top` 停在空位，数值正好等于元素个数。

两种 top 约定的对照

约定	入栈	出栈	判空	判满
指向栈顶元素（初值 -1）	`data[++top]=e`	`e=data[top--]`	`top==-1`	`top==MaxSize-1`
指向栈顶空位（初值 0）	`data[top++]=e`	`e=data[--top]`	`top==0`	`top==MaxSize`

两种写法没有优劣，但公式不能混用。做选择题时先看初始化语句：若 `top = -1`，就是第一种；若 `top = 0`，就是第二种。入栈和出栈的“先移指针”与“先读写”也要跟着变——`++top` 是先加再用，`top++` 是先用再加，恰好对应“top 指栈顶”和“top 指空位”两种语义。

按第一种约定，顺序栈的常用操作可以写成这样。

```c
#define MaxSize 50
typedef int ElemType;
typedef struct {
    ElemType data[MaxSize];
    int top;                // 栈顶元素下标，空栈为 -1
} SqStack;

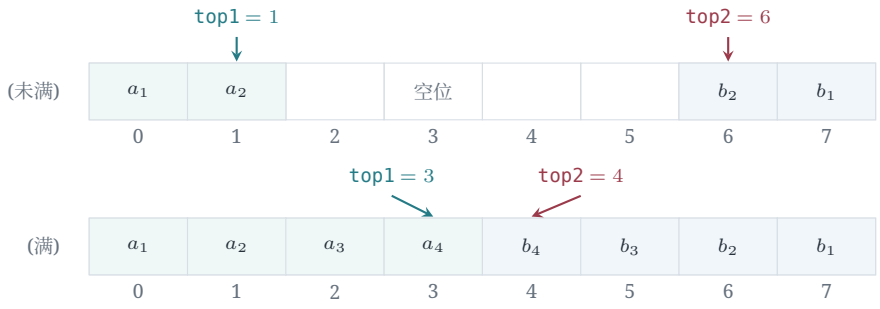
void InitStack(SqStack *S) { S->top = -1; }
bool StackEmpty(SqStack S) { return S.top == -1; }

bool Push(SqStack *S, ElemType e) {
    if (S->top == MaxSize - 1) return false;   // 栈满
    S->data[++S->top] = e;                     // 先移指针，再写入
    return true;
}

bool Pop(SqStack *S, ElemType *e) {
    if (S->top == -1) return false;            // 栈空
    *e = S->data[S->top--];                    // 先读出，再移指针
    return true;
}
```

共享栈

两个栈各自按最大需求开数组，往往有一边长期空着。让它们共用一段空间就能互相调剂：栈 1 从下标 0 向右长，栈 2 从下标 MaxSize − 1 向左长，各自的栈顶指针仍然指向自己的栈顶元素。



图怎么看：上面一行两栈之间还有空位；下面一行 `top1` 与 `top2` 已经相邻，中间再没有任何格子，此时 `top1`+1 =`top2`，整个数组被占满。

栈 1 的 top1 初始为 -1，栈 2 的 top2 初始为 MaxSize，两者都指向自己栈顶的元素。入栈分别是 data[++top1] = e 和 data[--top2] = e。判满条件为

$$\texttt{top1} + 1 = \texttt{top2}.$$

这里不能写成 top1 == top2：数组的一个位置只能属于一个栈，两个指针相邻时已经没有空位；若允许相等，那一格归谁就说不清了。

```c
typedef struct {
    ElemType data[MaxSize];
    int top1;    // 栈 1 栈顶，初值 -1
    int top2;    // 栈 2 栈顶，初值 MaxSize
} SharedStack;

bool PushShared(SharedStack *S, int which, ElemType e) {
    if (S->top1 + 1 == S->top2) return false;   // 已满
    if (which == 1) S->data[++S->top1] = e;
    else            S->data[--S->top2] = e;
    return true;
}
```

链栈

栈顶放在链表头部最省事：入栈就是头插，出栈就是删除首结点，两步都是常数时间。链栈不需要头结点，头指针本身就是栈顶指针，空栈时头指针为 NULL。



图怎么看：链表头就是栈顶，top 直接指向存放 a_3 的结点。入栈时新结点的指针域指向原栈顶，再让 top 指向新结点；出栈时把 top 后移一格并释放原结点。

```c
typedef struct SNode {
    ElemType data;
    struct SNode *next;
} SNode, *LinkStack;

void InitStack(LinkStack *S) { *S = NULL; }

bool Push(LinkStack *S, ElemType e) {
    SNode *p = (SNode *)malloc(sizeof(SNode));
    if (p == NULL) return false;
    p->data = e;
    p->next = *S;      // 新结点接在原栈顶前面
    *S = p;
    return true;
}

bool Pop(LinkStack *S, ElemType *e) {
    if (*S == NULL) return false;
```

```
    SNode *p = *S;
    *e = p->data;
    *S = p->next;
    free(p);
    return true;
}
```

顺序栈容量固定，满了就只能报错；链栈不会因为容量满而失败，但每个元素多一个指针域，`malloc` 和 `free` 也有额外开销。两种实现的基本操作都是 $O(1)$。

关于出栈序列，有一个常用结论：n 个元素按固定顺序入栈，可能得到的不同出栈序列数为第 n 个 Catalan 数

$$C_n = \frac{1}{n+1}\binom{2n}{n},$$

即 $1, 2, 5, 14, 42, \ldots$。例如 $n = 3$ 时六种排列中有五种可以由栈产生。判断某个具体序列是否合法，直接按“只能弹出当前栈顶”模拟一遍最稳妥。

3.2 队列与循环队列

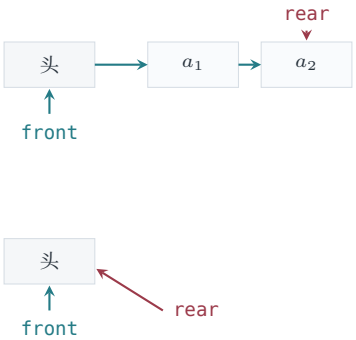
队列把插入限制在表尾、删除限制在表头：新元素只能从队尾进，离开的只能是队头元素。先到的先离开，称为先进先出（FIFO, First In First Out）。

定义 3.2: 队列

队列是只允许在表尾插入、在表头删除的线性表。插入端称为队尾（rear），删除端称为队头（front）；入队记作 enqueue，出队记作 dequeue。

链队列

链式队列用单链表实现，队头在链表头部，队尾在链表尾部。因为要在一头删除、另一头插入，必须同时保存 `front` 和 `rear` 两个指针。通常带头结点：`front` 指向头结点，`rear` 指向最后一个数据结点，空队时两者都指向头结点。头结点让空队和非空队的入队代码合并成同一段。



图怎么看：上面一行 `front` 指着头结点、`rear` 指着最后一个数据结点；下面一行是删掉唯一元素后的状态，两个指针都回到头结点。

```
typedef struct QNode {
    ElemType data;
    struct QNode *next;
} QNode;

```

```c
typedef struct {
    QNode *front;    // 指向头结点
    QNode *rear;     // 指向最后一个数据结点
} LinkQueue;

void InitQueue(LinkQueue *Q) {
    Q->front = Q->rear = (QNode *)malloc(sizeof(QNode));
    Q->front->next = NULL;
}

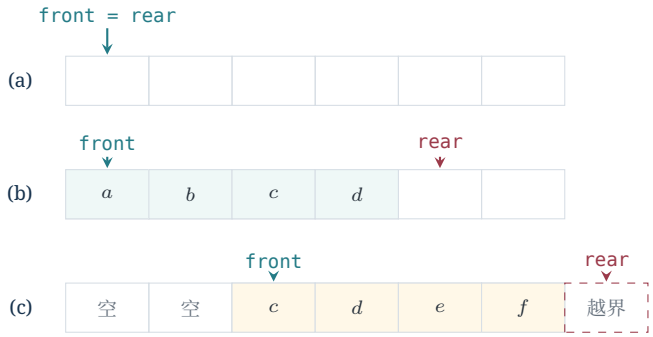
bool EnQueue(LinkQueue *Q, ElemType e) {
    QNode *p = (QNode *)malloc(sizeof(QNode));
    if (p == NULL) return false;
    p->data = e;
    p->next = NULL;
    Q->rear->next = p;   // 挂到队尾
    Q->rear = p;         // 更新队尾
    return true;
}

bool DeQueue(LinkQueue *Q, ElemType *e) {
    if (Q->front == Q->rear) return false;   // 空队
    QNode *p = Q->front->next;               // 队头数据结点
    *e = p->data;
    Q->front->next = p->next;
    if (Q->rear == p) Q->rear = Q->front;    // 删的是最后一个结点
    free(p);
    return true;
}
```

出队时那个 `if` 不能省。删除的若是队列中唯一的结点，`rear` 还停在它上面；结点释放后 `rear` 就成了悬挂指针，下一次入队会往已释放的内存里写。判断条件写成 `rear == p` 即可。

顺序队列与假溢出

顺序队列用数组 `Q[0..MaxSize-1]` 存元素，`front` 指向队头元素，`rear` 指向队尾元素的下一个位置。入队先写 `Q[rear] = e`，再 `rear++`；出队先取 `Q[front]`，再 `front++`。两个下标都只增不减，来看会发生什么。

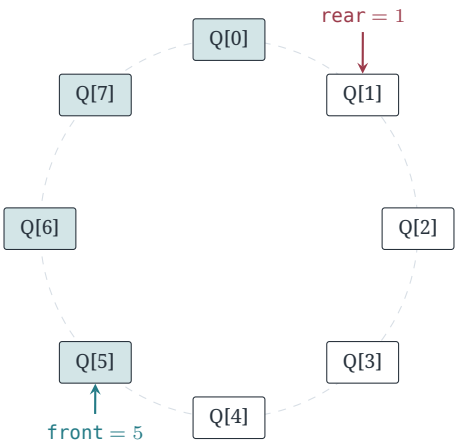


图怎么看：从 (a) 到 (b)，入队四个元素，`rear` 一路右移；从 (b) 到 (c)，出队两个元素后 `front` 也右移，再入队两个元素，`rear` 到了下标 6，越出了数组。此时 `Q[0]`、`Q[1]` 明明空着，却再也放不进新元素。

这种“数组前面有空位、rear 却已到末尾”的现象叫假溢出。根子在于下标只朝一个方向走，把数组当成了线段。把首尾接起来，让它变成环，下标就能绕回去——这就是循环队列。

循环队列的下标运算

循环队列仍然用 `Q[0..MaxSize-1]`，只是规定任何下标加一后都要对 `MaxSize` 取模：`rear` 走到 `MaxSize-1` 后再加一，回到 0。



图怎么看：环上的编号按顺时针排列，`front` 指向队头元素 `Q[5]`，`rear` 指向下一个可写的空位 `Q[1]`。从 `Q[5]` 走到 `Q[7]`，再绕回 `Q[0]`，这四格就是队内元素；`Q[1]` 本身不存元素。

设 `MaxSize` 为 M，采用“`front` 指向队头元素、`rear` 指向队尾元素的下一个位置”的约定，基本关系如下：

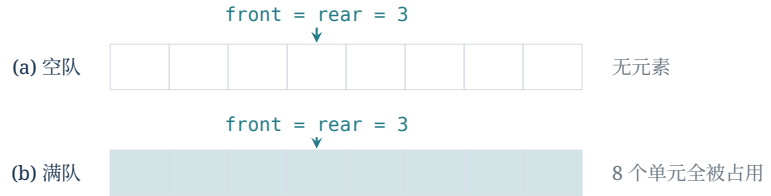
- 初始：`front = rear = 0`；
- 判空：`front == rear`；
- 入队：`Q[rear] = e`，然后 `rear = (rear + 1) % M`；
- 出队：`e = Q[front]`，然后 `front = (front + 1) % M`；
- 元素个数：$(\texttt{rear} - \texttt{front} + M) \bmod M$。

个数公式里加 M 是为了处理 `rear` 绕圈后小于 `front` 的情况：`rear-front` 可能是负数，加上 M 再取模，结果就落回 $0 \sim M-1$。例如上图 $M=8$，$(1-5+8) \bmod 8 = 4$。

空队时 `front` 与 `rear` 相等；如果允许数组装满，满队时两个指针同样相等。为了区分这两种情况，最常见的办法是牺牲一个单元——`rear` 永远不追上 `front`，当 `rear` 的下一个位置就是 `front` 时判满：

$$(\texttt{rear}+1) \bmod M = \texttt{front}.$$

代价是数组最多只存 $M-1$ 个元素。下图把空队和满队并排放在一起，两者的指针位置完全相同，这就是必须引入额外信息的原因。



图怎么看：两种情况下 `front` 和 `rear` 都等于 3，只看这两个指针无法判断队列是空还是满。

循环队列判空、判满的三种方案

方案	判空	判满	最大容量
牺牲一个单元	`front == rear`	`(rear+1)%M == front`	$M-1$
增设计数器 `size`	`size == 0`	`size == M`	M
增设标记 `tag`	`front==rear && tag==0`	`front==rear && tag==1`	M

`tag` 的维护很简单：每次入队置 1，出队置 0。只有当 `front == rear` 时才需要看它——相等且 `tag` 为 0 说明最后一次操作是出队，队空；相等且 `tag` 为 1 说明最后一次操作是入队，队满。

按牺牲一个单元的方案，循环队列的代码是：

```
typedef struct {
    ElemType data[MaxSize];
    int front;   // 队头元素下标
    int rear;    // 队尾元素的下一个位置
} SqQueue;

void InitQueue(SqQueue *Q) { Q->front = Q->rear = 0; }

bool EnQueue(SqQueue *Q, ElemType e) {
    if ((Q->rear + 1) % MaxSize == Q->front) return false;  // 满
    Q->data[Q->rear] = e;
    Q->rear = (Q->rear + 1) % MaxSize;
    return true;
}

bool DeQueue(SqQueue *Q, ElemType *e) {
    if (Q->front == Q->rear) return false;                  // 空
    *e = Q->data[Q->front];
    Q->front = (Q->front + 1) % MaxSize;
    return true;
}

int QueueLength(SqQueue Q) {
    return (Q.rear - Q.front + MaxSize) % MaxSize;
}
```

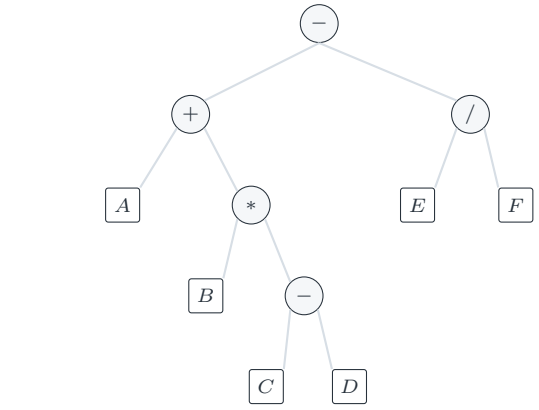
下标约定不止这一种。若题目把 `rear` 定义为“指向队尾元素”，常见初始化是 `front = 0`、`rear = M-1`；入队时先 `rear = (rear+1) % M` 再写入，出队仍然先取 `Q[front]` 再 `front = (front+1) % M`。这时元素个数变成

$$(\texttt{rear}-\texttt{front}+1+M)\bmod M,$$

判空是 $(\texttt{rear}+1)\bmod M=\texttt{front}$；若同样牺牲一个单元，判满则是 $(\texttt{rear}+2)\bmod M=\texttt{front}$。这些式子不必死背：画出环，数一数从 `front` 走到 `rear` 要经过几格，公式自然就出来了。

3.3 表达式：一棵树，三种遍历

计算 1+2*3 得到 7 而不是 9，因为乘法先算。运算符有优先级，同级还有结合方向，括号又能强行改变次序。把表达式写成树以后，这些规则都变成树的结构：内部结点是运算符，它的左右子树就是两个操作数；子树必须先算完，因为父结点需要它们的结果。



后序（左、右、根）：A B C D - * + E F / -
前序（根、左、右）：- + A * B - C D / E F
中序（左、根、右）：A + B * C - D - E / F

图怎么看：叶子是操作数，内部结点是运算符。对同一棵树做后序遍历，得到的就是后缀表达式；前序得到前缀；中序得到中缀，但遍历本身不输出括号，必须按树的结构补回括号才和原式等价。

三种记法只是同一棵树的三种遍历：

- 中缀表达式：运算符夹在两个操作数中间，符合人的阅读习惯，但需要优先级、结合性和括号来消除歧义；
- 后缀表达式（逆波兰表示）：运算符在操作数之后，运算次序由位置唯一确定，不需要括号，从左往右扫描即可求值；
- 前缀表达式（波兰表示）：运算符在操作数之前，从右往左扫描即可求值。

中缀转后缀

转换只用一个运算符栈。从左到右扫描中缀表达式，把操作数和运算符分别处理：

1. 遇操作数，直接输出；
2. 遇左括号，入栈；
3. 遇右括号，依次弹出栈顶运算符并输出，直到弹出左括号为止；左括号只出栈，不输出；
4. 遇运算符，若栈空、栈顶是左括号，或栈顶运算符优先级低于当前运算符，就把当前运算符入栈；否则反复弹出栈顶并输出，直到满足入栈条件；
5. 扫描结束后，把栈中剩余的运算符依次弹出输出。

优先级只涉及两级：*// 高于 +/-。左括号入栈后视为最低，只有右括号能让它出栈。同级运算符左结合，所以栈顶优先级“不低于”当前运算符时就要弹出——这样先出现的同级运算符会先输出。

以 $A+B*(C-D)-E/F$ 为例，逐步跟踪如下。

$A+B*(C-D)-E/F$ 的转换过程

步骤	读入	运算符栈（底 → 顶）	输出
1	A	空	A
2	+	+	A
3	B	+	AB
4	*	+ *	AB
5	(	+ * (	AB
6	C	+ * (	ABC
7	-	+ * (-	ABC
8	D	+ * (-	ABCD
9	)	+ *	ABCD-
10	-	-	ABCD-*+
11	E	-	ABCD-*+E
12	/	- /	ABCD-*+E
13	F	- /	ABCD-*+EF
14	结束	空	ABCD-*+EF/-

第 9 步的右括号把括号内的减号弹了出来；第 10 步读到的减号优先级低于栈顶的 * 和 +，于是两者依次弹出，减号才入栈。最后扫描结束时栈里还剩 / 和 -，先后弹出就补上了结果的末尾。注意输出的操作数顺序始终是 A, B, C, D, E, F，改变的只是运算符的位置，可以用这一点快速自查。

不想跟踪栈时，可以先把中缀表达式按优先级完全加括号，再把每个运算符移到它所在括号的右括号之后，最后去掉所有括号。$A+B*(C-D)-E/F$ 的全括号形式是

$$\big((A+(B*(C-D)))-(E/F)\big),$$

把运算符依次移到对应右括号后面，就得到 ABCD-*+EF/-。

后缀表达式求值

后缀表达式从左往右扫描，只用一个操作数栈：遇操作数入栈；遇运算符弹出两个操作数，先弹出的是右操作数，后弹出的是左操作数，计算完把结果压回栈。减法和除法不满足交换律，这个次序不能反。扫描结束时栈中剩下的唯一元素就是答案。

```c
int EvalPostfix(char *s) {          // 操作数为单个数字字符
    SqStack S; InitStack(&S);
    for (int i = 0; s[i] != '\0'; i++) {
        if (s[i] >= '0' && s[i] <= '9') {
            Push(&S, s[i] - '0');
        } else {
            int b, a;
            Pop(&S, &b);             // 先弹出的是右操作数
            Pop(&S, &a);             // 后弹出的是左操作数
            switch (s[i]) {
                case '+': Push(&S, a + b); break;
                case '-': Push(&S, a - b); break;
                case '*': Push(&S, a * b); break;
                case '/': Push(&S, a / b); break;
```

```
            }
        }
    }
    int v; Pop(&S, &v);
    return v;
}
```

以 `5 1 2 + 4 * + 3 -` 为例，栈的变化如下。

`5 1 2 + 4 * + 3 -` 的求值过程

读入	操作	栈（底 → 顶）
5	入栈	5
1	入栈	$5, 1$
2	入栈	$5, 1, 2$
+	弹出 $2, 1$，计算 $1 + 2 = 3$	$5, 3$
4	入栈	$5, 3, 4$
*	弹出 $4, 3$，计算 $3 \times 4 = 12$	$5, 12$
+	弹出 $12, 5$，计算 $5 + 12 = 17$	17
3	入栈	$17, 3$
-	弹出 $3, 17$，计算 $17 - 3 = 14$	14

最后一步若把次序弄反，会算成 $3 - 17 = -14$。记住“先弹出的是右操作数”，或者在草稿上把两个数按弹出顺序标成 b, a，再写 $a - b$。

前缀表达式扫描方向相反：从右往左，遇操作数入栈，遇运算符弹出两个，先弹出的是左操作数。两种扫描方向对应两种弹出次序，做题时不要凭感觉，按扫描方向推一遍。

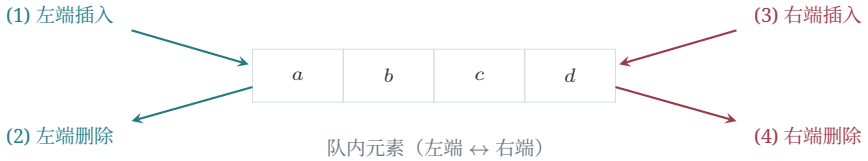
实际编译器更常用两个栈直接算中缀：一个操作数栈，一个运算符栈。遇操作数入操作数栈；遇运算符，先把运算符栈中所有优先级不低于它的运算符弹出并各做一次运算，再把它入栈；遇左括号直接入栈；遇右括号弹出运算符直到左括号；扫描结束后清空运算符栈。它和中缀转后缀的规则一一对应，区别只是运算符出栈时立刻计算，而不是写进输出。408 若问转换过程中运算符栈的最大深度，左括号也占一个位置，不要漏算。

3.4 双端队列

栈把操作锁在同一端，队列把进和出分到两端。双端队列（double-ended queue，deque）把两端的限制都放开：两端都可以插入，也都可以删除。

定义 3.3: 双端队列

双端队列是允许在表的两端进行插入和删除的线性表。若限定插入只在一端进行、删除仍可在两端进行，称为输入受限的双端队列；若限定删除只在一端进行、插入仍可在两端进行，称为输出受限的双端队列。



图怎么看：四个编号对应四种操作。输入受限的双端队列只保留 (1)、(3) 中的一个（插入端唯一），输出受限的只保留 (2)、(4) 中的一个（删除端唯一）。

三类双端队列允许的操作

变体	(1) 左端插入	(2) 左端删除	(3) 右端插入	(4) 右端删除
无限制	✓	✓	✓	✓
输入受限（只许左端插入）	✓	✓	×	✓
输出受限（只许左端删除）	✓	×	✓	✓

两种受限方式只是把“插入端”或“删除端”固定住，另一端的相应操作仍然保留。退化关系也很直接：插入和删除都限制在同一端，得到栈；固定一端只进、另一端只出，得到队列。

判断某个输出序列能否由双端队列产生，先抓住两条限制：任何时刻能取走的只有两端的元素，中间元素必须等两端被取空才轮得到；元素进队后相对次序不再改变。若插入还只允许在一端，则有一个更强的不变量——队列中的元素始终是输入序列里尚未输出的连续一段，删除只是在段的两端划掉元素，还没插入的部分只能接在插入端。

以“只在右端插入”为例，输入 a, b, c, d，看输出 a, d, b, c 是否能做到：

- 插入 a，从左端删除 a；
- 依次插入 b, c, d，此时队列为 b, c, d，从右端删除 d；
- 从左端删除 b，再删除 c。

得到 a, d, b, c。同样的序列用栈做不到：要第一个输出 a，只能压入 a 后立刻弹出；之后压入 b, c, d，弹出 d 时栈顶是 c，无法越过 c 去取 b。这说明双端队列的输出能力严格强于栈。

检查候选序列时可以逐项模拟：当前要输出的元素若还没入队，就把输入一直插入到它为止；插入完成后，它必须正好位于某一端，否则该序列不可能。这个模拟不需要回溯——只在一端插入时，多插入元素只会把目标挡在更里面，不会带来新的可能。

也正因为限制还在，双端队列并不能产生所有排列。$n = 3$ 时，栈能产生 5 种出栈序列，队列只有 1 种，而三种双端队列都能产生全部 $3! = 6$ 种；元素多起来以后就不行了：$n = 4$ 时，输入受限的双端队列能产生 22 种，输出受限的也是 22 种，只有无限制的双端队列才能产生全部 24 种。

3.5 数组与特殊矩阵的压缩存储

二维数组在内存里仍是一维排列，只是排列顺序有两种。设数组 A 有 m 行 n 列，每个元素占 L 字节，首元素 $A[0][0]$ 的地址为 $\mathrm{LOC}(0,0)$，则

$$\text{行优先：}\quad \mathrm{LOC}(i,j) = \mathrm{LOC}(0,0) + (i \cdot n + j)L,$$

$$\text{列优先：}\quad \mathrm{LOC}(i,j) = \mathrm{LOC}(0,0) + (j \cdot m + i)L.$$

行优先先把一整行存完再存下一行，所以 $A[i][j]$ 前面有 i 整行、每行 n 个元素，再加上本行的 j 个。列优先同理，只是行列互换。若题目下标从 1 开始，先把 i, j 各减 1；若某维有下界，也要先减去下界再套公式。

特殊矩阵的非零元素有规律，可以只存一部分。对称矩阵满足 $A[i][j] = A[j][i]$，存下三角（含主对角线）即可，共 $n(n+1)/2$ 个元素。按行存放、下标从 0 开始时，元素 $A[i][j]$（$i \geqslant j$）在一维数组中的下标为

$$k = \frac{i(i+1)}{2} + j.$$

这个式子来自计数：前 i 行共有 $1+2+\cdots+i=\frac{i(i+1)}{2}$ 个元素，第 i 行里 $A[i][j]$ 前面还有 j 个。访问上三角的 $A[i][j]$（$i<j$）时，利用对称性改取 $A[j][i]$，代入同一公式即可。

4 阶对称矩阵（只画下三角）

0	×	×	×
1	2	×	×
3	4	5	×
6	7	8	9

一维数组 $B[0..9]$（按行存放的下三角）

0	1	2	3	4	5	6	7	8	9

图怎么看：矩阵中每个下三角元素都标出了它在一维数组中的下标；按行读这些编号，正好是 $0,1,2,\ldots$，说明存储顺序就是逐行排列。上三角带 × 的元素不单独保存，访问时用对称性换成 $A[j][i]$。

三角矩阵与对称矩阵的区别在于，另一侧不是对称元素而是同一个常量 c，所以存储量是 $n(n+1)/2+1$：三角区照上面的方式存，另外留一个单元放 c。

三对角矩阵只有主对角线及其上下相邻的两条对角线可能非零，共 $3n-2$ 个位置。按行存放、下标从 0 开始时，$|i-j|\leqslant 1$ 的元素映射为

$$k=2i+j.$$

可以这样核对：第 0 行有 2 个元素，占 $k=0,1$；第 $i\geqslant 1$ 行的位置是 $k=3i-1,\ 3i,\ 3i+1$，所以行首下标依次是 $2,5,8,\ldots$。最后一行只有 2 个元素，总数正好是 $3n-2$。

稀疏矩阵的非零元素很少，用三元组 $(\text{行},\text{列},\text{值})$ 表或十字链表存储，空间与非零元素个数 t 成正比，而不是固定为 n^2。

3.6 栈与队列的典型应用

括号匹配

编译器检查括号是否配对，用的是栈。遇左括号入栈；遇右括号，若栈已空说明右括号多了，否则弹出栈顶检查类型是否相同。扫描结束后栈必须为空，否则说明还有左括号没配上。整个过程每个括号只进出栈一次，时间 $O(n)$，空间 $O(n)$。

```
bool BracketMatch(char *s) {
    SqStack S; InitStack(&S);
    for (int i = 0; s[i] != '\0'; i++) {
        char c = s[i];
        if (c == '(' || c == '[' || c == '{') {
            Push(&S, c);
        } else if (c == ')' || c == ']' || c == '}') {
            char top;
            if (!Pop(&S, &top)) return false;              // 右括号多
            if ((c == ')' && top != '(') ||
                (c == ']' && top != '[') ||
                (c == '}' && top != '{')) return false;     // 类型不配
```

```
        }
    }
    return StackEmpty(S);                                   // 左括号多则失败
}
```

递归与调用栈

函数调用时，系统把返回地址、参数和局部变量压入调用栈，函数返回时再弹出。递归函数每深入一层就多一个栈帧，递归深度就是同时存在的栈帧数的最大值。

```
int fact(int n) {
    if (n <= 1) return 1;
    return n * fact(n - 1);
}
```

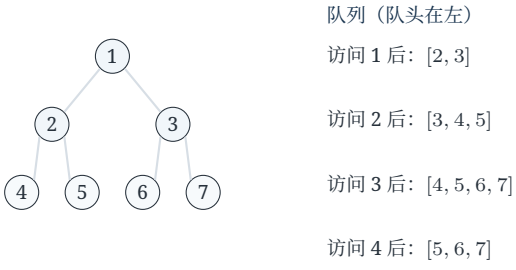


图怎么看：栈帧自下而上压入，`fact(1)` 在栈顶。返回时从栈顶开始弹出，每层的返回值乘上本层的 n，交给下一层。

任何递归都能用显式栈改写成非递归：把每层需要保存的状态自己压栈，就绕开了系统调用栈。如果递归调用是函数的最后一步（尾递归），还可以直接改成循环，连栈都不需要。

层次遍历

队列的 FIFO 性质保证按层访问：根结点先入队；每次取队头访问，并把它的孩子依次入队。同一层的结点一定排在前一层所有结点之后，所以出队顺序就是层序。



图怎么看：队列里始终是“已经发现、还没访问”的结点，并且按发现顺序排列。访问一个结点时把它的孩子接到队尾，于是同一层访问完才会轮到下一层。

```
void LevelOrder(BiTree T) {          // 队列元素类型设为 BiTree
    SqQueue Q; InitQueue(&Q);
    if (T != NULL) EnQueue(&Q, T);
    while (Q.front != Q.rear) {
        BiTree p;
```

```
6           DeQueue(&Q, &p);
7           visit(p);
8           if (p->lchild != NULL) EnQueue(&Q, p->lchild);
9           if (p->rchild != NULL) EnQueue(&Q, p->rchild);
10      }
11  }
```

打印任务队列、键盘缓冲区、消息队列和操作系统的就绪队列也都是队列。它们要的共同性质是“先到先处理”：后来的任务不能插到先来的任务前面，否则先到的任务可能被无限推后。

几处容易记混的地方

- `top` 的两种约定不能混用。看见 `top = -1` 就用“指向栈顶元素”那一列，看见 `top = 0` 就用“指向空位”那一列；入栈、出栈、判空、判满要一起换。
- 循环队列牺牲一个单元时，数组长度是 M，可存元素最多 $M-1$ 个。题目问“容量”还是“数组大小”，答案不同。
- 元素个数公式随 `rear` 的含义变化：`rear` 指向队尾下一空位时是 $(\texttt{rear}-\texttt{front}+M)\bmod M$；`rear` 指向队尾元素时是 $(\texttt{rear}-\texttt{front}+1+M)\bmod M$。
- 后缀表达式求值，先弹出的是右操作数。减法和除法必须写成“后弹出者减/除先弹出者”。
- 共享栈判满条件是 `top1+1=top2`，前提是 `top1`、`top2` 都指向各自栈顶元素。
- 链队列删除最后一个结点后，`rear` 要回指头结点；否则 `rear` 成为悬挂指针，下一次入队就会写已释放的内存。

拓展阅读：

- 邓俊辉《数据结构（C++ 语言版）》第 4 章「栈与队列」——表达式求值的完整实现与递归转非递归。
- Sedgewick《算法》第 1.3 节「背包、队列和栈」——用链表和数组实现这些抽象数据类型。

4 习题

4.1 基础过关题

1. （单项选择题）设栈的初始状态为空，入栈序列为 $1,2,3,4$，每个元素入栈后允许在任意时刻出栈（入栈顺序不能改变）。下列出栈序列中**不可能**出现的是

(A) $1,2,3,4$　(B) $4,3,2,1$

(C) $3,1,2,4$　(D) $2,3,4,1$

2. （填空题）某顺序栈的栈顶指针 `top` 指向栈顶元素，空栈时 `top`$=-1$，存储空间足够大。依次执行 `Push(A)`、`Push(B)`、`Pop`、`Push(C)`、`Push(D)`、`Pop`、`Pop` 后，`top` 的值为 ____，栈中元素个数为 ____，栈顶元素为 ____。

3. （填空题）循环队列存放在数组 $Q[0..9]$ 中，约定 `front` 指向队头元素，`rear` 指向队尾元素的下一个位置。某时刻 `front`$=6$、`rear`$=3$。此时队列中元素个数为 ____，队尾元素所在的下标为 ____；若判满采用“牺牲一个存储单元”的策略，此时最多还能入队 ____ 个元素。

4. （填空题）中缀表达式 $A*(B+C)-D/E$ 对应的后缀表达式为 _____；若把中缀表达式改成 $A*(B+C-D)/E$，对应的后缀表达式为 _____。

5. （单项选择题）后缀表达式（操作数均为整数，除法为整除）$3\ 10\ 4\ -\ -\ 6\ 2\ /\ +$ 的值为

(A) -3　(B) 0

(C) 3　(D) 9

6. （单项选择题）一个输入受限的双端队列：元素只允许从同一端（左端）插入，但允许从两端删除。现有 $1,2,3,4$ 依次从左端插入，插入过程中可随时从任一端删除。下列输出序列中**不可能**得到的是

(A) $1,2,3,4$　(B) $2,4,3,1$

(C) $3,1,4,2$　(D) $4,2,1,3$

4.2 真题风格与综合训练

7. （真题风格・单项选择题）用算符优先法把中缀表达式 $A-(B+C*D)/E+F$ 转换为后缀表达式，转换过程中运算符栈（栈中同时存放运算符和左括号）中元素个数的最大值是

(A) 3　(B) 4

(C) 5　(D) 6

8. （真题风格・单项选择题）循环队列 $Q[0..7]$ 采用“牺牲一个存储单元”的方式判满，`front` 指向队头元素，`rear` 指向队尾元素的下一个位置，初始时 `front`=`rear`= 0。依次执行：入队 a、b、c、d、e；出队 3 次；入队 f、g、h、i、j；出队 4 次；入队 k。最终 `front`、`rear` 的值和队列中元素个数分别是

(A) 7，3，4　(B) 7，3，3

(C) 3，7，4　(D) 7，2，4

9. （真题风格·综合应用题）括号匹配。给定由字母、运算符及三种括号 `()`、`[]`、`{}` 组成的字符串，设计算法判断其中的括号是否正确配对，要求在不匹配时指出错误类型与出错位置（字符下标从 1 开始）。回答：

   (1) 写出算法（说明栈中存放什么、何时入栈、何时出栈、扫描结束后如何判定）；

   (2) 对字符串 S_1 = `[(a+b)*{c-[d/(e+f)]}]`，求扫描过程中栈内元素个数的最大值；

   (3) 对字符串 S_2 = `[(a+b)*{c-[d/(e+f)]})`，判断是否匹配；若不匹配，说明出错位置和错误类型。

10. （真题风格·综合应用题）用两个栈 S1、S2 模拟一个队列（栈的 `top` 指向栈顶元素，初值 -1，容量足够）。约定：入队时元素压入 S1；出队时若 S2 非空，直接弹出 S2 栈顶；若 S2 为空，先把 S1 中元素全部弹出并压入 S2，再从 S2 弹出。回答：

   (1) 写出入队与出队算法；

   (2) 元素 a、b、c、d、e 依次入队，然后连续出队 2 次，再入队 f、g，最后连续出队 3 次。写出全部出队元素，并给出每一步之后 S1、S2 的内容（自底向顶）；

   (3) 说明 n 次入队与 n 次出队（任意交错）的总时间复杂度为什么是 $O(n)$。

5 串

在编辑器里按一次“查找”，在 DNA 序列里找一段碱基，在日志里筛一个错误码，做的都是同一件事：在一段长字符串里找一个短字符串第一次出现的位置。串就是为这类“按字符连续比较”准备的结构。它的逻辑结构和线性表一样，只是每个元素限定为字符；操作上则常以整段子串为单位，而不是逐个元素处理。

串是零个或多个字符组成的有限序列，记作

$$S = a_1 a_2 \cdots a_n \qquad (n \geq 0),$$

其中 n 是串长。$n = 0$ 时称为空串，记作 ε。

以 $T =$ abcababc 为例：abc、cab、ababc 都是它的子串，子串必须连续；aaa 只是子序列，不是子串。包含子串的串叫主串，被查找的串叫模式串。从第一个字符开始的一段叫前缀，以最后一个字符结尾的一段叫后缀，“真”前缀、真后缀指不包含整个串本身。既是真前缀又是真后缀的串叫 border，例如 abcababc 的 border 是 abc；而 abcab 的 border 是 ab。

按起始位置和长度区分，长度为 n 的串有 $n(n+1)/2$ 个非空子串，加上空串共 $n(n+1)/2+1$ 个；真子串（去掉串本身）有 $n(n+1)/2$ 个。这里按“位置”计数：aaa 中两个 a 算两个不同的子串。串相等要求长度相同且对应位置字符完全相同；按字典序比较时，从第一个不同的字符决定大小，若短串是长串的前缀，则短串更小。

> **注意**
>
> 空串 ε 的长度为 0；空格串由一个或多个空格组成，长度等于空格个数，至少为 1。空串是任何串的子串，也是任何非空串的真前缀和真后缀。题目里“空串”“空格串”“长度为 0”是三种不同说法。

5.1 串的存储与基本操作

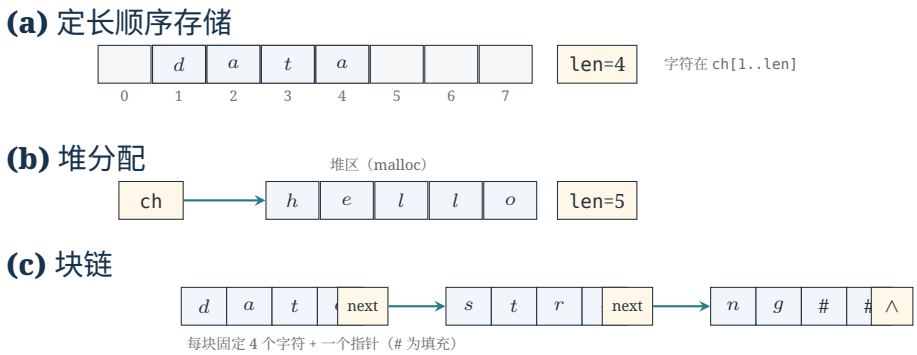
顺序存储最常见。408 的代码通常约定 `ch[1..length]` 保存字符，`ch[0]` 空着不用，串长单独放在 `length` 字段：

```
#define MAXLEN 255
typedef struct {
    char ch[MAXLEN + 1];   /* ch[1..length] 存字符，ch[0] 不用 */
    int  length;           /* 当前串长 */
} SString;
```

有的教材把长度塞进 `ch[0]`（值域只有 $0 \sim 255$），但做题时若题目给了 `length` 字段，就按上面的约定理解。堆分配存储把字符放在 `malloc` 得到的一段连续空间里，结构体只保存首地址和长度，适合串长变化较大的场合：

```c
typedef struct {
    char *ch;              /* 指向堆区，按实际串长分配 */
    int   length;
} HString;
```

块链存储把字符切成固定大小的块，块与块之间用指针连接。它的插入、删除比顺序存储灵活，但想取第 i 个字符得沿链找块，不能随机访问。



三种布局画的都是逻辑结构：顺序存储靠下标随机访问；堆分配多一次间接寻址，换来可变串长；块链把字符切块，找第 i 个字符要沿链定位。

串的 ADT 以整串为操作对象：赋值 `StrAssign`、复制 `StrCopy`、判空 `StrEmpty`、比较 `StrCompare`、求长 `StrLength`、取子串 `SubString`、连接 `Concat`、定位 `Index`、清空 `ClearString`。前几个操作基本都是一次 $O(n)$ 的搬运；本章的重点是

$$\texttt{Index}(S, T, pos),$$

即从主串 S 的第 pos 个字符起，找模式串 T 第一次出现的位置，找不到返回 0。下面的 BF 和 KMP 都是它的实现。

5.2 朴素匹配：失配就从头再来

BF（Brute Force，暴力匹配）的做法很直接：把 T 依次对齐到 S 的每个起点，逐字符比较；一旦失配，T 整体右移一位，T 的指针回到第一个字符，主串指针回到本轮起点的下一个位置。下面记主串长 $n = |S|$、模式串长 $m = |T|$，通常 $n \geq m$。

取 $S = \texttt{ababcabcacbab}$、$T = \texttt{abcac}$：

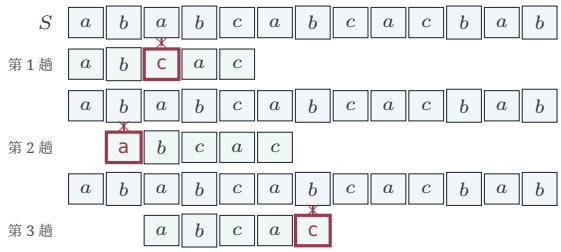
- 第 1 趟从 $S[1]$ 对齐：前两对 a、b 相同，第 3 对 $S[3] = \texttt{a}$ 与 $T[3] = \texttt{c}$ 失配。
- 第 2 趟从 $S[2]$ 对齐：$S[2] = \texttt{b}$ 与 $T[1] = \texttt{a}$ 立即失配。
- 第 3 趟从 $S[3]$ 对齐：abca 四对相同，第 5 对 $S[7] = \texttt{b}$ 与 $T[5] = \texttt{c}$ 失配。
- 第 4、5 趟分别从 $S[4]$、$S[5]$ 对齐，首字符就失配。
- 第 6 趟从 $S[6]$ 对齐：$S[6..10] = \texttt{abcac} = T$，匹配成功，返回 6。

代码里用 i 记主串当前比较位置、j 记模式串当前比较位置。失配时本趟起点是 $i - j + 1$，下一趟从它的下一位开始，所以 i 改成 $i - j + 2$：

```c
int Index_BF(SString S, SString T) {
    int i = 1, j = 1;
    while (i <= S.length && j <= T.length) {
        if (S.ch[i] == T.ch[j]) { ++i; ++j; }
```

```
        else { i = i - j + 2; j = 1; }   /* 回到本趟起点的下一位 */
    }
    if (j > T.length) return i - T.length;  /* 本趟起点 */
    return 0;
}
```

匹配成功时 $j = T.length + 1$，i 停在匹配段之后，所以起点是 $i - T.length$。



第 6 趟从 $S[6]$ 对齐时 $S[6..10]$ = abcac，匹配成功；第 4、5 趟均在首字符失配

图中每一块是一次对齐：上排是主串 S，下排是模式串 T，红框标出失配位置；第 1、2、3 趟的起点依次右移一位。

比较次数取决于失配发生得多早。最好情况每趟首字符就失配，比较 $n-m+1$ 次，$O(n)$；最坏情况每趟都比到 T 的末尾才失败，比较 $(n-m+1)m$ 次，$O(nm)$。典型触发串是 S = aaaa$\cdots$a、T = aaaab：每一趟先匹配若干 a，最后一个字符才失败。平均情况要看字符分布，不能不加条件地写成 $O(n+m)$；408 问最坏复杂度时答 $O(nm)$。

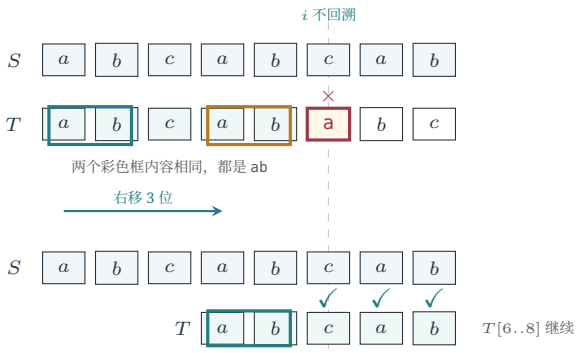
BF 的问题不在慢，而在重复。失配时前 $j-1$ 个字符已经确认相等，这些信息被丢掉了，下一趟又从 $T[1]$ 开始比。KMP 要做的就是把这段信息留下来。

5.3 KMP：让主串指针只向前走

还是用 T = abcababc。假设主串的一段是 S = abcabcab$\cdots$，T 从 $S[1]$ 对齐：

$$\begin{aligned} S&: \ \texttt{a}\ \texttt{b}\ \texttt{c}\ \texttt{a}\ \texttt{b}\ \texttt{c}\ \texttt{a}\ \texttt{b} \\ T&: \ \texttt{a}\ \texttt{b}\ \texttt{c}\ \texttt{a}\ \texttt{b}\ \texttt{a}\ \texttt{b}\ \texttt{c} \end{aligned}$$

前 5 个字符 abcab 相同，第 6 对 $S[6]$ = c 与 $T[6]$ = a 失配。注意已匹配的 abcab 有 border ab：它的前缀 ab 等于后缀 ab。把 T 右移，使 $T[1..2]$ 对准刚匹配过的最后两个字符 $S[4..5]$，这两个字符就不用再比；下一次直接比较 $T[3]$ = c 与 $S[6]$ = c。



图中上半部分是失配前的对齐，两个彩色框标出相等的前后缀；下半部分是右移后的对齐，绿色勾表示这几个字符不用重新比较，主串指针仍停在原来的失配字符处。

推广到一般情况：设失配发生在 $T[j]$，已匹配部分是 $P = T[1..j-1]$。若 P 的最长 border 长度为 k，把 T 右移 $(j-1)-k$ 位，就能让 $T[1..k]$ 对准 P 的后缀，下一次比较 $T[k+1]$ 与 $S[i]$，主串指针 i 不动。

为什么可以跳过中间的位移？假设某个更小的位移 $d < (j-1)-k$ 也能匹配，那么 P 就存在一个长度大于 k 的 border，和 k 是最长 border 矛盾。所以按最长 border 滑动不会漏掉任何可能的匹配。

把每个 j 对应的 $k+1$ 预先算出来，就是 **next** 数组（下标从 1 开始）：

$$\text{next}[j] = \begin{cases} 0, & j = 1, \\ 1 + \max\{\, k \mid 0 \le k < j-1,\ T[1..k] = T[j-k..j-1] \,\}, & j \ge 2. \end{cases} \tag{1}$$

$k = 0$ 时约定空串等于空串，所以 $j \ge 2$ 时 $\text{next}[j] \ge 1$。用文字说：**next[j] 是 $T[1..j-1]$ 的最长相等真前后缀长度加 1**。这里的“真”不能丢：若把整个 $T[1..j-1]$ 当前缀，失配后模式串不会移动，等于没跳。失配后的实际位移是

$$(j-1) - (\text{next}[j] - 1) = j - \text{next}[j].$$

408 常用下标从 1 开始的版本，$\text{next}[1] = 0$、$\text{next}[2] = 1$。有的教材用 $\text{next}[1] = -1$（所有值减 1），有的直接用 0 下标的前缀函数 π（存 border 长度本身）；做题前先看题干采用哪套约定。

匹配过程只改模式串指针：

```
int Index_KMP(SString S, SString T, int next[]) {
    int i = 1, j = 1;
    while (i <= S.length && j <= T.length) {
        if (j == 0 || S.ch[i] == T.ch[j]) { ++i; ++j; }
        else j = next[j];          /* i 不动，只退模式串 */
    }
    if (j > T.length) return i - T.length;
    return 0;
}
```

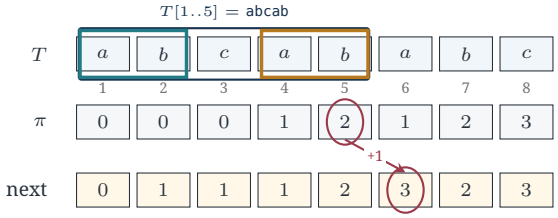
$j = 0$ 是 $\text{next}[1] = 0$ 带来的哨兵。首字符失配时 j 变成 0，下一轮走 `j==0` 分支，i 和 j 同时加一，相当于放弃当前起点、从主串下一个字符重新开始。C 的 `||` 从左到右短路，所以 `T.ch[0]` 不会被读。匹配成功时 $j = T.length + 1$，返回 $i - T.length$。

复杂度这样看：i 只增不减，最多增加 n 次；每次失配 $j = \text{next}[j]$ 都严格减小；j 只在 i 增加时增加，所以 j 减小的总次数不超过它增加的总次数，也不超过 n。因此 while 循环至多执行约 $2n$ 次，匹配阶段 $O(n)$；预处理 $O(m)$；总时间 $O(n+m)$。额外空间是 next 数组的 $O(m)$。

5.4 next 数组怎么算

先把 $T =$ `abcabaabc` 手算一遍。设 $\pi[j]$ 是 $T[1..j]$ 的最长 border 长度，则 $\text{next}[j] = \pi[j-1] + 1$（$j \ge 2$），$\text{next}[1] = 0$：

j	1	2	3	4	5	6	7	8
$T[j]$	a	b	c	a	b	a	b	c
$\pi[j]$	0	0	0	1	2	1	2	3
next$[j]$	0	1	1	1	2	3	2	3



图里 π 行存的是 border 长度本身，next 行把它右移一位再加 1；$j=6$ 时看的是 $\pi[5]=2$，所以 $\text{next}[6]=3$。$\text{next}[1]=0$ 不对应任何 border，只是首字符失配时的哨兵。

$j=6$：前 5 个字符 abcab 的最长 border 是 ab，长度 2，所以 $\text{next}[6]=3$。$j=4$：abc 没有非空 border，$\text{next}[4]=1$。当 $m\geq 2$ 时 $\text{next}[2]=1$ 恒成立，因为前 1 个字符没有真前缀，border 长度为 0。

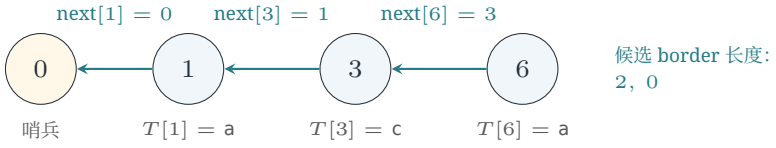
手算时每个 j 都重新找 border，最坏 $O(m^2)$。利用已经算出的值可以做到 $O(m)$。设当前在求 $\text{next}[i+1]$，k 表示“当前候选 border 长度加 1”：

- 若 $k=0$，没有更短的候选，直接让 k 变成 1，$\text{next}[i+1]=1$；
- 若 $T[i]=T[k]$，当前 border 可以延长一位，$k\leftarrow k+1$，$\text{next}[i+1]=k$；
- 否则当前长度不行，退到更短的 border。$T[1..k-1]$ 的最长 border 长度是 $\text{next}[k]-1$，所以令 $k\leftarrow\text{next}[k]$，再比较 $T[i]$ 与新的 $T[k]$。

```
void GetNext(SString T, int next[]) {
    int i = 1, k = 0;
    next[1] = 0;
    while (i < T.length) {
        if (k == 0 || T.ch[i] == T.ch[k]) {
            ++i; ++k;
            next[i] = k;
        } else {
            k = next[k];          /* 退到更短的 border */
        }
    }
}
```

当 $k=0$ 时，条件里的 T.ch[k] 因为短路不会被读；随后 k 变成 1，等价于“从空前缀重新开始配”。

拿 $T=$ abcababc 走一段：算到 $\text{next}[6]=3$ 后，$i=6,k=3$。$T[6]=$ a $\neq T[3]=$ c，退到 $k=\text{next}[3]=1$；再比 $T[6]=$ a $=T[1]=$ a，于是 $i=7,k=2$，$\text{next}[7]=2$。这段过程就是下面这条链从 6 退到 3、再退到 1。



链上每个值都是“下一个候选位置”。$j=6$ 失配时，先试长度 2 的 border（跳到 3），不行再试长度 0（跳到 1），最后到哨兵 0。next 链的每个后继都严格变小，所以这条链最多走 m 步。

预处理复杂度：i 从 1 增到 m，只增不减；k 每次失配严格减小，且只在 i 增加时增加，所以总回退次数不超过 m，整个 GetNext 是 $O(m)$。

5.5 前缀函数：同一个量的另一种记法

很多教材和代码不用带哨兵的 next，而直接定义前缀函数

$$\pi[i] = \max\{k \mid 0 \le k < i,\ T[1..k] = T[i-k+1..i]\},$$

即 $T[1..i]$ 的最长相等真前后缀长度，$\pi[1] = 0$。它和 next 只差一个平移：

$$\text{next}[j] = \pi[j-1] + 1 \quad (j \ge 2), \qquad \text{next}[1] = 0.$$

反过来 $\pi[i] = \text{next}[i+1] - 1$。两者含的信息完全相同，做题时不要混用：题目要 next 就给带 1 的版本，要 π 就给从 0 开始的长度。

0 下标实现里，模式串记作 $P[0..m-1]$，$\pi[0] = 0$，$\pi[i]$ 表示 $P[0..i]$ 的最长 border 长度。构造代码比 1 下标的 `GetNext` 更短：

```
void GetPi(const char *P, int m, int pi[]) {
    pi[0] = 0;
    for (int i = 1, k = 0; i < m; ++i) {
        while (k > 0 && P[i] != P[k])
            k = pi[k - 1];       /* 退到更短的 border */
        if (P[i] == P[k]) ++k;
        pi[i] = k;
    }
}
```

这里 k 直接就是 border 长度，不是 next 里的长度加 1，所以回退写 $\pi[k-1]$ 而不是 $\pi[k]$。这是两套写法最容易写错的地方。

用 π 匹配时，i 扫主串，j 是已匹配长度；失配就 $j = \pi[j-1]$，$j = m$ 时报告起点 $i - m + 1$，然后令 $j = \pi[j-1]$ 继续找下一次出现（允许重叠）。匹配阶段的摊还分析与 1 下标版本完全一样，仍是 $O(n)$。

5.6 nextval：跳过注定失败的比较

next 有一个小毛病：如果 $T[j] = T[\text{next}[j]]$，失配后跳到 next$[j]$，比较的还是同一个字符，必然再次失配。nextval 把这种连续相同的跳转一次走完。定义为

$$\text{nextval}[j] = \begin{cases} 0, & j = 1, \\ \text{next}[j], & T[j] \ne T[\text{next}[j]], \\ \text{nextval}[\text{next}[j]], & T[j] = T[\text{next}[j]]. \end{cases}$$

因为 next$[j] < j$，右边的 nextval 在从左到右计算时已经求出，不需要真的递归。

先算一个具体例子。对 $T =$ `abcabab`c：

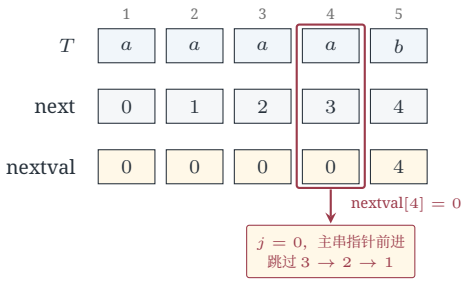
j	1	2	3	4	5	6	7	8
$T[j]$	a	b	c	a	b	a	b	c
next$[j]$	0	1	1	1	2	3	2	3
nextval$[j]$	0	1	1	0	1	3	1	1

$j = 4$：next$[4] = 1$，而 $T[4] = T[1] =$ a，跳到 1 仍会比较 a；nextval$[4] =$ nextval$[1] = 0$，直接让主串指针前进。$j = 8$ 同理：next$[8] = 3$，$T[8] = T[3] =$ c，所以 nextval$[8] =$ nextval$[3] = 1$。

再看一个重复字符更多的例子：$T=$ aaaab 时 next $=(0,1,2,3,4)$，nextval $=(0,0,0,0,4)$。若在 $j=4$ 失配（文本当前字符不是 a），按 next 会依次跳 3、2、1，每次都在比较 a，直到 $j=0$ 才前进主串；nextval 直接把 j 置 0。$j=5$ 不一样：$T[5]=$ b $\neq T[4]=$ a，所以 nextval$[5]=$ next$[5]=4$，这里没有可跳过的重复字符。

代码和 `GetNext` 的差别只有两处：匹配成功时根据 $T[i]$ 与 $T[k]$ 是否相同决定写 k 还是 nextval$[k]$；失配回退用 nextval$[k]$ 而不是 next$[k]$。

```
void GetNextVal(SString T, int nextval[]) {
    int i = 1, k = 0;
    nextval[1] = 0;
    while (i < T.length) {
        if (k == 0 || T.ch[i] == T.ch[k]) {
            ++i; ++k;
            nextval[i] = (T.ch[i] != T.ch[k]) ? k : nextval[k];
        } else k = nextval[k];     /* 沿 nextval 链回退 */
    }
}
```



图中第 4 列是关键：next$[4]=3$，而 nextval$[4]=0$；向下箭头表示失配后直接跳到 $j=0$，跳过 $3\to2\to1$ 这三次注定失败的比较。

nextval 只改变跳转目标，不改变匹配结果，也不改变 $O(n+m)$ 的最坏复杂度；它减少的是重复字符多的模式串上的常数。匹配代码把 `next` 换成 `nextval` 即可。

注意

408 选择题里最容易错的几处：

- next$[j]$ 看的是前 $j-1$ 个字符，不是前 j 个；next$[1]=0$ 是哨兵，不对应 border。
- 下标从 1 开始时 next$[1]=0$、next$[2]=1$（$m\geq2$）；若改用 0 下标并令 next$[0]=-1$，1 下标的值整体减 1；若直接问前缀函数 π，$\pi[i]$ 是 border 长度本身。
- 失配后的位移是 $j-$ next$[j]$，不是 next$[j]-1$。
- BF 最坏 $O(nm)$，KMP 是 $O(n+m)$；不要把 BF 的平均情况当成最坏情况。
- next 只由模式串决定，和主串无关；同一个 T 在不同 S 上 next 不变。

6 习题

6.1 基础过关题

1. （单项选择题）下列关于串的说法中，正确的是

(A) 空串是含有一个空格字符的串　　(B) 两个串相等当且仅当它们的长度相等

(C) 空格串的长度等于其中空格字符的个数　　(D) 串只能采用顺序存储，不能采用链式存储

2. （计算题）设主串 $S =$ "abababababc"，模式串 $T =$ "ababc"，下标均从 1 开始。用 BF 算法查找 T 在 S 中第一次出现的位置，写出每一趟的对齐位置、该趟比较次数和失配位置，并求总比较次数。

3. （填空题）串 $S =$ "abcde" 中，子串（按起始位置和长度区分，包含空串）共有 ____ 个；其中真子串（不包含 S 本身）有 ____ 个。

4. （填空题）模式串 $T =$ "ababaca"，下标从 1 开始，则 $\text{next}[4] =$ ____，$\text{next}[6] =$ ____，$\text{next}[7] =$ ____。

5. （单项选择题）已知模式串 $T =$ "aabaab" 的 next 数组为 $(0, 1, 2, 1, 2, 3)$（下标从 1 开始），则 nextval$[3]$ 与 nextval$[6]$ 分别为

(A) 2 和 2　(B) 0 和 2

(C) 2 和 3　(D) 0 和 3

6. (单项选择题) 用 BF 算法进行模式匹配时，下列情形中比较次数会达到最坏情况 $O(nm)$ 的是

(A) 主串和模式串的字符集都很大　(B) 主串为 a^n，模式串为 $a^{m-1}b$，且 $m \leqslant n/2$

(C) 模式串长度 $m = 1$　(D) 主串中不含模式串的首字符

6.2 真题风格与综合训练

7. (真题风格・单项选择题) 已知模式串 $T =$ "abcabcaa"，下标从 1 开始，则 next[6] 与 next[8] 分别为

(A) 3 和 4　(B) 3 和 5

(C) 4 和 5　(D) 4 和 4

8. (真题风格・单项选择题) 关于 nextval 数组，下列说法正确的是

(A) 对任意 $j \geqslant 2$，nextval[j] < next[j]　(B) 若 $T[j] \neq T[\text{next}[j]]$，则 nextval[$j$] = next[$j$]

(C) nextval 的计算依赖主串　(D) 改用 nextval 后比较次数一定减少

9. (真题风格・综合题) 设模式串 $T =$ "abacabab"，主串 $S =$ "abacabacabab"，下标均从 1 开始。

(1) 求 T 的 next 数组与 nextval 数组；

(2) 按 next 数组执行 KMP 匹配，指出每次失配后指针如何变化，列出全部字符比较并求首次匹配位置与总比较次数；

(3) 若改用 nextval 数组，比较次数是否改变？说明理由。

10. (真题风格·算法综合题)设主串 S 与模式串 T 均采用顺序存储,字符存放在 `ch[1..length]` 中,下标从 1 开始。设计算法统计 T 在 S 中出现的所有起始位置(允许重叠,例如 $S=$"aaaaa"、$T=$"aa" 时位置为 1、2、3、4)，要求总时间为 $O(n+m)$，其中 $n=|S|$、$m=|T|$。

(1) 说明算法假设与计数口径;

(2) 给出求 next 的预处理函数和统计函数（C 语言描述);

(3) 分析时间与空间复杂度;

(4) 写出 $S=$"aaaaa"、$T=$"aa" 时的输出。

拓展阅读:

- 邓俊辉《数据结构（C++ 语言版)》第 9 章「串」——KMP 算法的完整推导，包括 next 数组的改进（nextval)。
- Sedgewick《算法》第 5.3 节「子字符串查找」——KMP 的 DFA 视角理解。

7 树与二叉树

家族谱、文件目录、算术表达式，画出来都是“一层套一层”的结构：一个人有父母，一个目录里有子目录，一个运算符有两个运算对象。它们共同的特征是**分叉**——一个结点可以带多个后继，而不是像线性表那样只有一个。把这种结构抽象出来，就是树。

7.1 树的形状与几条计数规律

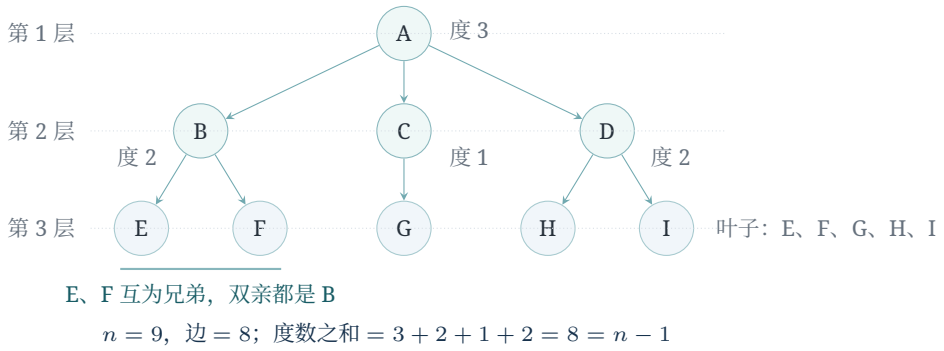
先看一个具体的组织图：总经理 A 下面有三位部门经理 B、C、D；B 手下有组长 E、F；C 手下有组长 G；D 手下有组长 H、I。把每个人画成一个结点，把“直接领导”画成一条有向边，就得到下图的形状。它没有环，任意两个结点之间有且只有一条简单路径，这正是树的特征。

定义 7.1: 树

树是 n $(n \geqslant 0)$ 个结点的有限集合。$n = 0$ 时称为空树；$n > 0$ 时满足：

1. 有且仅有一个特殊的结点称为**根**；
2. 其余结点可以分成 m $(m > 0)$ 个互不相交的集合 $T_1, \ldots, T_m$，每个集合本身又是一棵树，称为根的**子树**。

定义是递归的：树由根和若干棵子树组成，子树仍是树。正因为如此，后面几乎所有操作都写成递归形式。



图怎么看：每一层用虚线隔开，层号从 1 开始；结点旁标的是它的度（孩子个数）。根 A 有 3 个孩子，所以度是 3；E、F 没有孩子，度是 0，也就是叶子。

有了图，几个术语可以一次说清。结点的**度**是它的孩子个数，所有结点度的最大值称为**树的度**。根所在的层号为 1；从根到某结点的边数加 1 就是该结点的**深度**（层号），树中最大的层号称为**高度**。从根到某结点经过的边数称为**路径长度**。同一个双亲的孩子互为**兄弟**。如果结点的孩子区分左右次序，这棵树称为**有序树**，否则称为**无序树**；二叉树可以看成每个结点有两个固定位置（左、右）的有序树：位置为空也算一种状态，所以只有一个孩子时仍要区分左右。

从图中还能数出一个恒等式。除根以外，每个结点恰好有一条边连向它的双亲，所以 n 个结点共有 $n - 1$ 条边；换一个方向数，每条边都是某个结点连向它的一个孩子，所以边数又等于所有结点度数之和。设度为 i 的结点有 n_i 个，就有

$$n - 1 = \sum_{i \geqslant 1} i\, n_i, \qquad n = \sum_{i \geqslant 0} n_i.$$

两式相减，得到只含叶子数的式子

$$n_0 = 1 + \sum_{i \geqslant 2} (i-1)\, n_i.$$

它的用法很直接：题目给出各种非叶结点的个数，叶子数不用画图就能算出来。比如图中有 $n_3 = 1$、$n_2 = 2$、$n_1 = 1$，则 $n_0 = 1 + 2 \times 1 + 1 \times 2 = 5$，与图上的五个叶子一致。

注意

“度”在两套语言里含义不同。树里说结点的度，指它有几个孩子，$\sum \deg(v) = n - 1$；把树当无向图看时，图的度还要把连向双亲的那条边算上，于是 $\sum \deg_{\text{图}}(v) = 2(n-1)$。408 的选择题常用这一点做干扰项。

另外，深度从 1 还是从 0 开始，各教材约定不同。本章与 408 常用约定一致：根的层号（深度）为 1，树的高度等于最大层号。做题时以题干约定为准。

7.2 二叉树：左右必须分清

二叉树常被误解成“每个结点最多两个孩子”。真正的限制更强：**左右子树是有次序的**。一个结点只有一个孩子时，也必须说明它是左孩子还是右孩子。这个区别不是文字游戏——中序遍历的“左、根、右”，二叉排序树的“小在左、大在右”，Huffman 编码的 0 和 1，全都依赖左右之分。

定义 7.2: 二叉树

二叉树是 n $(n \geqslant 0)$ 个结点的有限集合，它或者为空，或者由一个根结点和两棵互不相交的、分别称为**左子树**和**右子树**的二叉树组成。

因此，二叉树不是“度为 2 的树”：度为 2 的树至少含有一个度为 2 的结点，而二叉树可以一个都没有（例如每个结点只有左孩子的单支树）。

满二叉树与完全二叉树

满二叉树要求每一层都排满；完全二叉树放宽了最后一层，但要求结点从左到右连续排列，中间不能空。

定义 7.3: 满二叉树与完全二叉树

高度为 h 的二叉树，若第 i 层恰有 2^{i-1} 个结点 $(1 \leqslant i \leqslant h)$，称为**满二叉树**，结点总数为 $2^h - 1$。若一棵 n 个结点的二叉树，按满二叉树的层序编号后，其结点恰好占据编号 $1 \sim n$，称为**完全二叉树**。等价地说：除最后一层外每层都满，最后一层的结点从左到右连续。

二叉树的基本性质

设二叉树中度为 0、1、2 的结点数分别为 n_0, n_1, n_2。

要点 7.1: 二叉树的计数性质

1. 第 i 层至多有 2^{i-1} 个结点（$i \geqslant 1$）；高度为 h 的二叉树至多有 $2^h - 1$ 个结点。
2. $n_0 = n_2 + 1$。
3. n 个结点的二叉树，高度至少为 $\lceil \log_2(n+1) \rceil$，至多为 n。

第一条按层数归纳即可：根是 2^0，每个结点至多带两个孩子，下一层至多翻倍。

第二条是 408 出现频率最高的公式，证明只用数边：结点总数 $n = n_0 + n_1 + n_2$；除根外每个结点有一条入边，故边数为 $n-1$；另一方面，度为 1 的结点贡献 1 条出边，度为 2 的贡献 2 条，所以

$$n - 1 = n_1 + 2n_2.$$

两式联立消去 n_1：

$$n_0 + n_1 + n_2 - 1 = n_1 + 2n_2 \implies n_0 = n_2 + 1.$$

这个等式与树的形状无关，只要求每个结点至多两个孩子。做题时若给出 n_0 和 n_2 中的一个，另一个立刻确定；若给出总结点数，还要用 n_1 的取值补足。

对完全二叉树，n_1 只能取 0 或 1（最后一层不会出现“有右孩子却没有左孩子”的结点）。由 $n = 2n_0 + n_1 - 1$ 可得

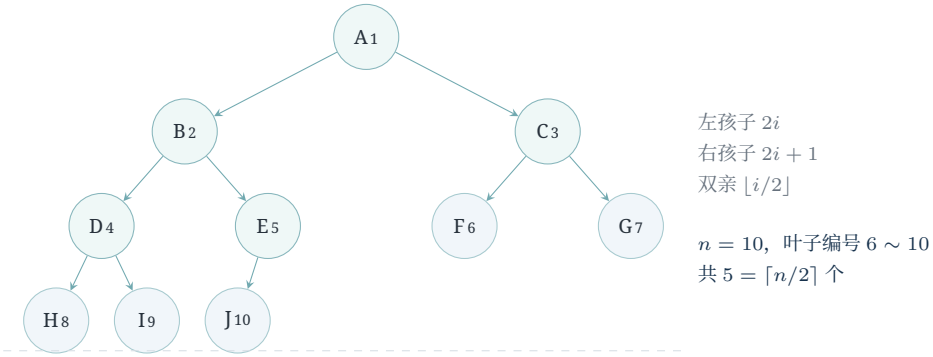
$$n_0 = \left\lceil \frac{n}{2} \right\rceil, \qquad n_1 = \begin{cases} 1, & n \text{ 为偶数}, \\ 0, & n \text{ 为奇数}. \end{cases}$$

高度则满足

$$h = \lceil \log_2(n+1) \rceil = \lfloor \log_2 n \rfloor + 1.$$

例如 $n = 1000$ 时，$n_1 = 1$，$n_0 = 500$，$h = \lceil \log_2 1001 \rceil = 10$。

完全二叉树的编号与数组下标一一对应，下图把编号规则画了出来。



第 4 层：8、9、10 从左到右连续

图怎么看：结点内的小字是层序编号。这张图把完全二叉树的编号规则画全了：$n = 10$ 时，前五个结点是分支结点，编号 $6 \sim 10$ 是叶子；编号为 i 的结点，孩子编号直接是 $2i$ 和 $2i+1$，双亲编号是 $\lfloor i/2 \rfloor$。因此完全二叉树可以不用指针，直接存进数组。

7.3　二叉树的两种存储布局

顺序存储

完全二叉树的编号已经给出了下标，直接把结点按编号放进一维数组即可：`T[i]` 存编号为 i 的结点，`T[0]` 闲置。下标关系就是上一节的三条公式。

非完全二叉树也能这样存，但必须按完全二叉树的编号留空。下图对比了两种情况：十个结点的完全树恰好占十个单元；而四个结点的右单支树，编号依次是 1、3、7、15，要占十五个单元，其中十一个是空位。



图怎么看：每个格子下方的数字是数组下标，格子里是结点编号对应的数据。完全树的下标连续；右单支树的数据只落在 $1, 3, 7, 15$ 四个位置，其余格子必须留空。

最坏情况就是右单支树：深度为 k 时最后一个结点的编号是 $2^k - 1$，而实际只有 k 个结点，空间从 $O(n)$ 退化到 $O(2^n)$。所以顺序存储只适合完全二叉树，或者结点位置固定的场合（如堆）。

二叉链表

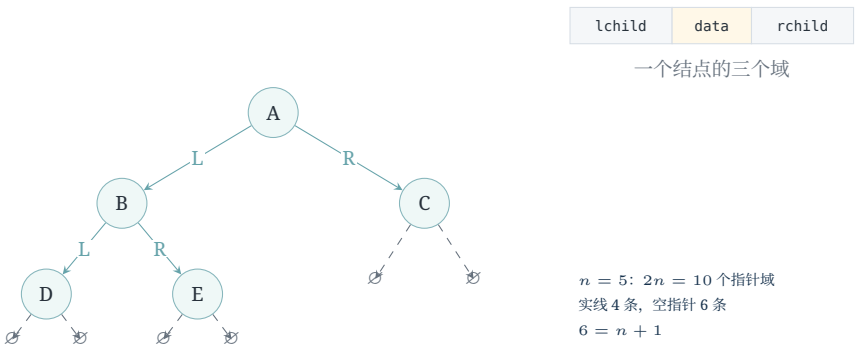
一般二叉树更常用链式存储。每个结点存数据域和两个指针域：

```c
typedef struct BiTNode {
    ElemType data;
    struct BiTNode *lchild, *rchild;
} BiTNode, *BiTree;
```

n 个结点一共有 $2n$ 个指针域，其中 $n-1$ 个用来指向孩子（每条边对应一个指针），剩下的

$$2n - (n-1) = n + 1$$

个是空指针。这个数字很关键：下一节的线索二叉树，就是把这 $n+1$ 个空指针利用起来。



图怎么看：实线是真正指向孩子的指针，旁边标 L/R 表示左或右；虚线指向 ∅，表示该指针域为空。五个结点共有十个指针域，四条实线之外还剩六个空域。

如果还需要频繁找双亲，可以在结点里再加一个 `parent` 指针，称为三叉链表。代价是每个结点多一个指针域。

7.4　遍历：三种深度优先与一种广度优先

遍历是把每个结点访问一次，且只访问一次。“访问”具体做什么不影响次序：可以打印、累加、求高度。三种深度优先遍历的差别只有一句话：**访问根结点的时机**。

要点 7.2: 先序、中序、后序

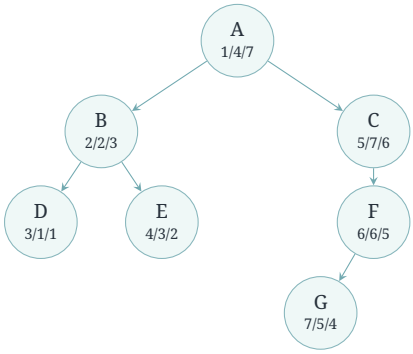
以根为参照：

- **先序**（根左右）：先访问根，再先序遍历左子树，最后先序遍历右子树；
- **中序**（左根右）：先中序遍历左子树，再访问根，最后中序遍历右子树；
- **后序**（左右根）：先后序遍历左子树，再后序遍历右子树，最后访问根。

把“访问根”这一行在递归代码中挪动位置，就得到三种算法：

```
void PreOrder(BiTree T) {
    if (T) {
        visit(T);               // 根
        PreOrder(T->lchild);    // 左
        PreOrder(T->rchild);    // 右
    }
}
void InOrder(BiTree T) {
    if (T) {
        InOrder(T->lchild);     // 左
        visit(T);               // 根
        InOrder(T->rchild);     // 右
    }
}
void PostOrder(BiTree T) {
    if (T) {
        PostOrder(T->lchild);   // 左
        PostOrder(T->rchild);   // 右
        visit(T);               // 根
    }
}
```

结点内：先序 / 中序 / 后序序号



先序：A B D E C F G
中序：D B E A G F C
后序：D E B G F C A

图怎么看：每个结点有三个序号，分别对应它在三种序列中的位置。以 A 为例：先序第一个访问它，中序在左子树 D、B、E 之后访问它，后序要等整棵树都访问完才轮到它，所以是 1/4/7。按第一分量从小到大读出结点就是先序，按第二、第三分量同理，得到下面的三行序列。

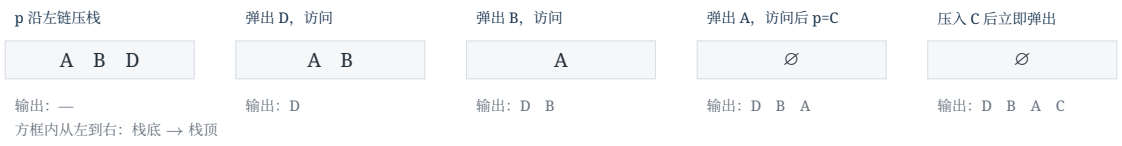
非递归遍历与栈

递归能工作，是因为系统栈替我们记住了“还没有做完的结点”。自己用栈模拟，就能把过程展开。以中序为例，反复做两件事：沿着左孩子一路压栈，直到空；然后弹栈访问，转向右子树。

```
void InOrderNR(BiTree T) {
    Stack S; InitStack(&S);
    BiTree p = T;
    while (p || !StackEmpty(S)) {
        if (p) {
            Push(&S, p);          // 沿左链走到底，边走边压
            p = p->lchild;
        } else {
            Pop(&S, &p);          // 左边走不动了，弹出栈顶
            visit(p);
            p = p->rchild;        // 转向右子树
        }
    }
}
```

先序只是把 `visit(p)` 挪到压栈之前；后序麻烦一些，因为弹出栈顶时可能只是“左子树刚做完”，右子树还没访问。解决办法是记一个 `lastVisited`：只有右孩子为空或已经访问过，才真正弹出并访问。

```
void PostOrderNR(BiTree T) {
    Stack S; InitStack(&S);
    BiTree p = T, last = NULL;
    while (p || !StackEmpty(S)) {
        if (p) {
            Push(&S, p);
            p = p->lchild;
        } else {
            GetTop(S, &p);
            if (p->rchild && p->rchild != last)
                p = p->rchild;        // 右子树还没做，先去右边
            else {
                Pop(&S, &p);
                visit(p);
                last = p;             // 记录刚访问的结点
                p = NULL;
            }
        }
    }
}
```

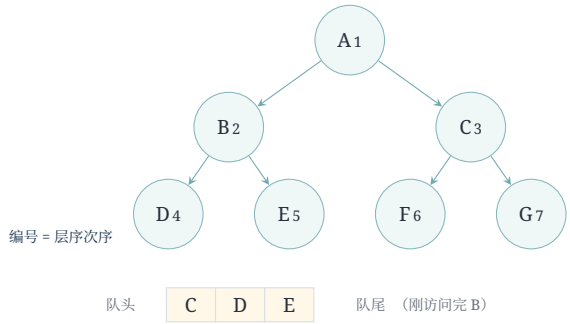


图怎么看：从左到右是时间顺序，方框里是当前的栈，右边是已经输出的序列。前四步走完左链并连续弹出，第五步才转到 A 的右子树 C。

层次遍历与队列

层次遍历按“从上到下、同层从左到右”访问。它不用栈，而用队列：根先入队；每次出队一个结点并访问，然后把它的左、右孩子依次入队。

```
void LevelOrder(BiTree T) {
    Queue Q; InitQueue(&Q);
    if (T) EnQueue(&Q, T);
    while (!QueueEmpty(Q)) {
        BiTree p;
        DeQueue(&Q, &p);
        visit(p);
        if (p->lchild) EnQueue(&Q, p->lchild);
        if (p->rchild) EnQueue(&Q, p->rchild);
    }
}
```



图怎么看：结点内的数字是层序访问的次序；下方队列画的是刚访问完 B、准备访问 C 的瞬间，队头是 C，后面依次是 D、E。

三个深度优先遍历的时间复杂度都是 $O(n)$：每个结点进一次、出一次。空间取决于栈的深度，即树高 h，最坏 $O(n)$（单支树），最好 $O(\log n)$（平衡树）。层次遍历的空间是队列的最大长度，也就是树的最大宽度 w，最坏同样是 $O(n)$。

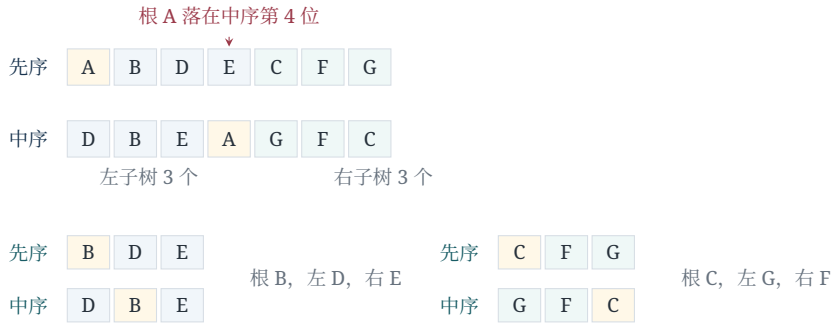
7.5 由遍历序列还原二叉树

结论 7.1: 哪些序列组合能唯一确定二叉树

在结点关键字互异、序列合法的前提下：

- 先序 + 中序：唯一确定；
- 后序 + 中序：唯一确定；
- 层次 + 中序：唯一确定；
- 先序 + 后序：不能唯一确定。

先序 + 中序的做法只有一句话：先序的第一个是根，拿这个根去中序里切出左右两段，再按段长回到先序切分，递归处理。



图怎么看：上面两行是整棵树的先序与中序，橙色格是根；蓝色段和青色段分别是根的左、右子树。把两段分别取出来，就得到下面两个规模更小的子问题，橙色格是子树的根。

写成代码，先序区间 `[pl,pr]`、中序区间 `[il,ir]`，左子树长度由根在中序中的位置决定：

```
BiTree Build(char pre[], char in[], int pl, int pr, int il, int ir) {
    if (pl > pr) return NULL;
    BiTree root = (BiTree)malloc(sizeof(BiTNode));
    root->data = pre[pl];
    int k = il;
    while (k <= ir && in[k] != pre[pl]) k++;
    if (k > ir) { free(root); return NULL; }   // 序列不匹配
    int leftLen = k - il;
    root->lchild = Build(pre, in, pl+1, pl+leftLen, il, k-1);
    root->rchild = Build(pre, in, pl+leftLen+1, pr, k+1, ir);
    return root;
}
```

后序 + 中序完全对称：后序的最后一个元素是根，其余步骤相同。

先序 + 后序为什么不够？下图给了最小反例：两棵树都只有两个结点，先序都是 AB，后序都是 BA，但一棵 B 在左，一棵 B 在右。先序告诉我们“A 是根、B 是孩子”，后序告诉我们“B 先于 A 被访问”，两者都无法确定 B 挂在哪一边。



同样的两个序列，对应两种不同树形

图怎么看：两棵树都只有 A、B 两个结点，先序都是先 A 后 B，后序都是先 B 后 A；差别只在 B 是左孩子还是右孩子，而这两个序列都没有记录这个信息。

7.6　线索二叉树：把空指针用起来

二叉链表有 $n+1$ 个空指针。遍历时我们常常需要“下一个是谁”“上一个是谁”，而递归或栈都在额外花空间。线索二叉树的做法是：把空着的左指针改为指向该遍历次序下的**前驱**，空着的右指针改为指向**后继**。指针现在有两种身份，必须加标记区分。

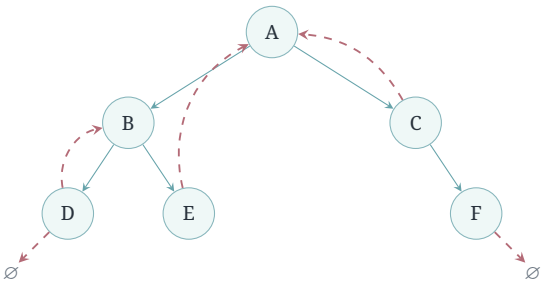
定义 7.4: 线索二叉树

结点增加两个标志位 ltag、rtag：

- ltag=0：lchild 指向左孩子；ltag=1：lchild 指向前驱线索。
- rtag=0：rchild 指向右孩子；rtag=1：rchild 指向后继线索。

按某种遍历次序建立的线索，分别称为先序、中序、后序线索二叉树。

```c
typedef struct ThreadNode {
    ElemType data;
    struct ThreadNode *lchild, *rchild;
    int ltag, rtag;          // 0: 孩子; 1: 线索
} ThreadNode, *ThreadTree;
```



实线：孩子指针；虚线：线索（中序前驱/后继）

中序序列：D B E A C F；空指针 $n+1=7$ 个，全部改为线索

图怎么看：实线仍是孩子指针，虚线是中序线索。E 指向 B、F 指向 C 的线索都与孩子边重合，没有重复画出。最值得注意的是 E 的右指针：E 没有右孩子，空出来的域直接指向 A，而 A 并不在 E 的子树里。这就是线索的价值——它把遍历的“下一步”存进了空域，遍历时不必再借助栈。

中序线索化的过程就是中序遍历，只是把“访问”换成“修改空指针”。需要用一个 pre 记住刚刚访问的结点，才能把前驱的后继线索接上：

```c
ThreadNode *pre = NULL;            // 全局变量，指向刚访问的结点

void InThread(ThreadTree T) {
    if (T) {
        InThread(T->lchild);
        if (T->lchild == NULL) {   // 左空，指向前驱
            T->lchild = pre;
            T->ltag = 1;
        }
        if (pre != NULL && pre->rchild == NULL) {  // 前驱的右空，指向后继
            pre->rchild = T;
            pre->rtag = 1;
        }
        pre = T;
        InThread(T->rchild);
    }
}
```

找前驱、后继分两种情况。以中序为例，结点 p 的后继：

- 若 `rtag=1`，右指针本身就是后继；
- 若 `rtag=0`，说明有右子树，中序下下一个访问的是右子树的**最左下**结点。

前驱完全对称：`ltag=1` 时左指针就是前驱；否则是左子树的**最右下**结点。

```
ThreadNode *InSucc(ThreadNode *p) {
    if (p->rtag == 1) return p->rchild;
    ThreadNode *q = p->rchild;
    while (q->ltag == 0) q = q->lchild;   // 右子树最左下
    return q;
}
ThreadNode *InPred(ThreadNode *p) {
    if (p->ltag == 1) return p->lchild;
    ThreadNode *q = p->lchild;
    while (q->rtag == 0) q = q->rchild;   // 左子树最右下
    return q;
}
```

注意

线索只对建立它的那种次序有效。三种次序下“就地找前驱/后继”的难易正好相反：

- 中序：前驱、后继都能就地找到，不需要双亲或栈；当线索指针不可用时，要沿子树走到最左下或最右下，最坏 $O(h)$；
- 先序：后继容易（有左孩子就是左孩子，否则是右孩子或线索），**前驱**可能在祖先方向，通常要借助双亲指针；
- 后序：前驱容易，**后继**可能在祖先方向，通常要借助双亲指针。

另外，判断 `lchild` 到底指孩子还是线索，必须看 `ltag`，不能看它是否为空。

7.7 树、森林与二叉树的互相表示

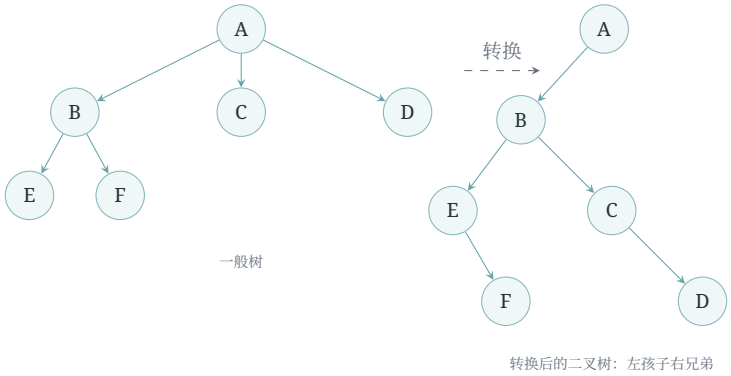
一般树每个结点可以有任意多个孩子，直接用“左孩子、右孩子”两个指针存不下。常见办法有三种。

双亲表示法用数组存每个结点和它的双亲下标，求双亲 $O(1)$，求孩子要扫描整个数组。**孩子表示法**把每个结点的孩子串成链表。408 最常用的是**孩子兄弟表示法**：每个结点两个指针，`firstchild` 指向第一个孩子，`nextsibling` 指向下一个兄弟。

```
typedef struct CSNode {
    ElemType data;
    struct CSNode *firstchild, *nextsibling;
} CSNode, *CSTree;
```

这两个指针恰好就是二叉树的左、右指针，于是任何树都能唯一地转成二叉树，规则叫“**左孩子右兄弟**”：

1. 每个结点只保留与第一个孩子的连线，作为左孩子；
2. 结点原来的兄弟，依次连成右孩子链；
3. 原先向下的边因此变成向左下或向右下的边。



图怎么看：A 没有兄弟，所以它的右子树为空；B 的兄弟 C、D 被串成 B 的右链；E 的兄弟 F 变成 E 的右孩子。森林转换只是多一步：第一棵树的根作总根，后面每棵树的根依次作为前一个根的右孩子。因此森林对应的二叉树，根一定有右子树（只要森林里不止一棵树）。

遍历的对应关系常考，而且很好记：

- 树的**先根遍历** = 对应二叉树的**先序**；
- 树的**后根遍历** = 对应二叉树的**中序**；
- 森林的**先序** = 对应二叉树的先序；森林的**中序** = 对应二叉树的中序。

树没有“中根遍历”，因为一个结点有多个孩子，“中间”没有定义。

7.8　Huffman 树：让总码长最短

设四个字符出现的次数是 $2, 4, 5, 7$，总共 18 个字符。若用等长编码，每个字符 2 位，共需 $18 \times 2 = 36$ 位。能不能更短？把出现多的字符编短一些、出现少的编长一些，总长度就会下降。Huffman 树给出的是一种最优的变长前缀编码，本例可以做到 35 位。

衡量一棵带权二叉树好坏的量是**带权路径长度**：

$$\mathrm{WPL} = \sum_{i=1}^{n} w_i \, l_i,$$

其中 w_i 是第 i 个叶子的权值，l_i 是它到根的边数（也就是深度减一）。WPL 越小，平均码长越短。

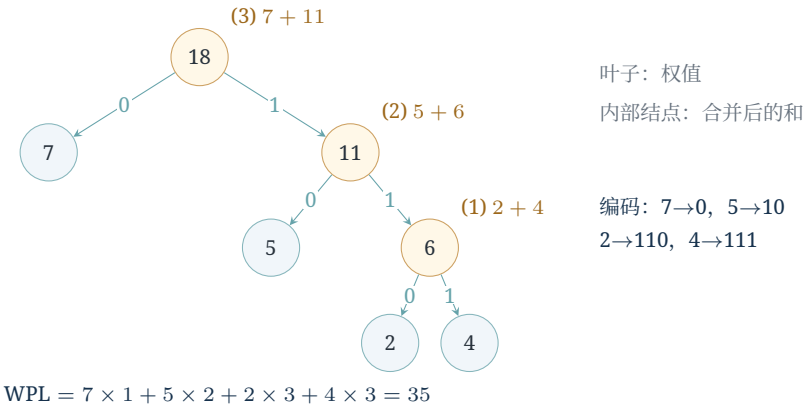
定义 7.5: Huffman 树

给定 n 个权值 $w_1, \ldots, w_n$ 作为 n 个叶子的权，若某棵二叉树的 WPL 达到最小，称它为**最优二叉树**或 **Huffman 树**。

构造方法是贪心的，每一步都合并当前最小的两个：

1. 把 n 个权值各自看成一棵单结点树，组成森林；
2. 取出根权值最小的两棵树，作为左右子树合并，新根的权值为两者之和；
3. 把新树放回森林，重复，直到只剩一棵树。

用最小堆实现，每次取两个最小值、压回一个和，复杂度 $O(n \log n)$。



图怎么看：圆圈里的数是权值，内部结点是合并出来的和，边上的 0/1 是编码。括号中的编号是合并的先后：先合并 2 和 4，再合并 5 与 6，最后合并 7 与 11。WPL 也可以直接由三次合并的和相加得到：$6+11+18=35$。

Huffman 树有几条常用性质，都可以从构造过程直接看出来：

1. 没有度为 1 的结点。每次合并都是“左、右各一棵”，不会产生单孩子结点。于是 $n_1=0$，由 $n_0=n_2+1$ 得总结点数 $n=2n_0-1$。
2. 权值大的叶子不会比权值小的叶子更深。否则交换这两个叶子的权值，树形不变而 WPL 下降，与最优性矛盾。
3. 树不唯一：左右子树可以交换，等权权值的合并次序也可以不同。但 **WPL 唯一**。
4. WPL 等于所有内部结点权值之和。每个叶子权值在它到根的每条边上被计一次，累加起来正好等于所有祖先内部结点的权值和。

把 Huffman 树的左分支记 0、右分支记 1，从根到叶子的 0/1 串就是该字符的编码。由于字符只放在叶子上，任何一个编码都不会是另一个编码的前缀——这就是**前缀编码**，解码时不必预先知道码长，从根出发逐位下行，到叶子就输出一个字符，再回到根继续。

注意

Huffman 编码是“前缀码中最优”的，不是“所有编码中最优的”。如果允许编码之间互为前缀，虽然平均长度可能更短，但无法唯一解码，实际不能用。
计算题里常见的口径要写清楚：平均码长 $=\mathrm{WPL}/\sum w_i$；等长编码位数 $=\lceil \log_2 n \rceil$；压缩率按“节省的位数 / 原位数”计算。此外，给定一组编码判断是否为前缀码，只需检查有没有一个码是另一个码的前缀。

7.9　二叉排序树

二叉排序树（BST）把“左小右大”写进结构里，借它可以在 $O(h)$ 时间内完成查找、插入、删除。h 是树高，理想情况 $O(\log n)$，退化成单支树时 $O(n)$。

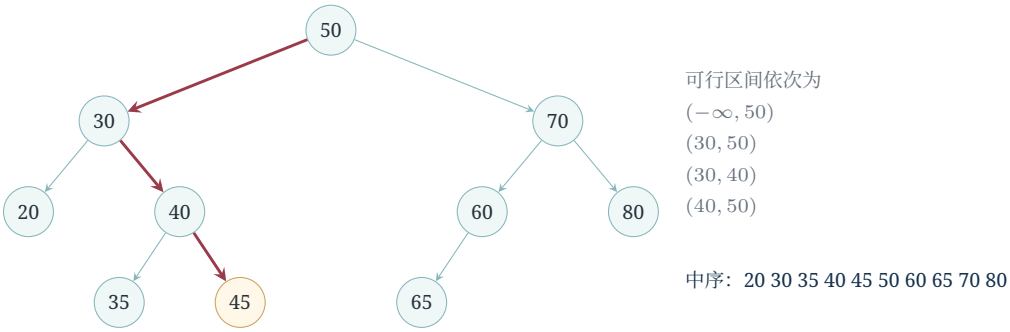
定义 7.6: 二叉排序树

一棵二叉树是二叉排序树，当且仅当：对任意结点，其左子树（若存在）中所有关键字都小于该结点的关键字，其右子树中所有关键字都大于该结点的关键字，且左右子树本身也是二叉排序树。

注意定义要求的是“整棵左子树都小”，不是“左孩子小”。因此判断一棵树是否为 BST，不能只比较父子，正确做法是中序遍历，检查序列是否严格递增；或者递归时把每个结点限制在一

个开区间 (lo, hi) 内。

查找的过程就是不断缩小可行区间：比当前结点小就往左，区间上界改为当前关键字；大就往右，下界改为当前关键字。



图怎么看：粗箭头是查找 45 走过的路径，每走一步都把可能的关键字限制在一个更窄的开区间里。最终 45 落在 $(40, 50)$ 内。

```c
BiTree BSTSearch(BiTree T, KeyType key) {
    while (T != NULL && T->data != key)
        T = (key < T->data) ? T->lchild : T->rchild;
    return T;                          // 找不到时返回 NULL
}

int BSTInsert(BiTree *T, KeyType key) {
    if (*T == NULL) {
        *T = (BiTree)malloc(sizeof(BiTNode));
        (*T)->data = key;
        (*T)->lchild = (*T)->rchild = NULL;
        return 1;
    }
    if (key == (*T)->data) return 0; // 已存在，不重复插入
    return (key < (*T)->data)
         ? BSTInsert(&((*T)->lchild), key)
         : BSTInsert(&((*T)->rchild), key);
}
```

插入的落点一定是某个空指针，新结点总是成为叶子，不需要移动已有结点。删除分三种情况：

1. 被删结点是叶子：直接摘掉，把双亲对应指针置空；
2. 只有一个孩子：用这个孩子顶替它的位置；
3. 有两个孩子：用中序前驱（左子树最右下的结点）或中序后继（右子树最左下的结点）顶替它的关键字，再删除那个替身。替身至多只有一个孩子，于是归入前两种情况。



图怎么看：左边虚线圆圈住的是待删的 50；沿它的左子树一直向右找到前驱 45，把 45 复制到根，再删掉原来的 45。右边是删除后的树，40 的右指针已经置空。

同一个关键字集合，插入次序不同会得到不同形状的 BST。按 $1,2,3,4,5$ 依次插入，树退化成右单支，查找变回 $O(n)$；按 $3,2,4,1,5$ 插入则接近平衡。这正是下一节要解决的问题。

7.10　AVL 树：把树高压住

AVL 树是最早的自平衡二叉排序树。它在每个结点上记录左右子树的高度差，一旦某个结点的平衡因子超出 $\{-1,0,1\}$，就通过旋转把局部结构调整回来。

定义 7.7: AVL 树

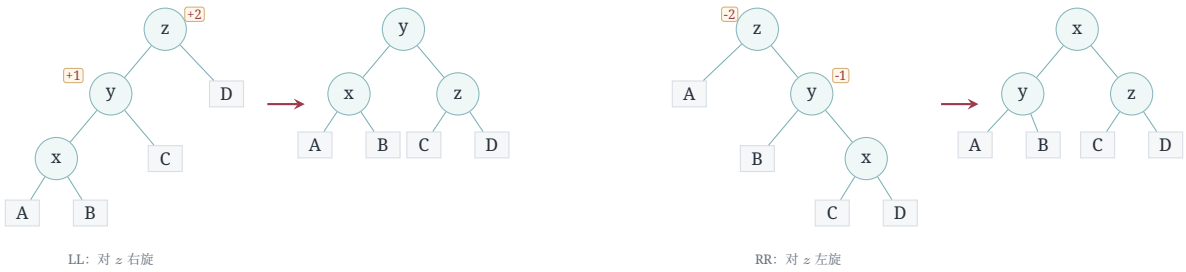
二叉排序树中，若每个结点的平衡因子

$$\mathrm{BF}(v) = h(v\text{ 的左子树}) - h(v\text{ 的右子树})$$

都满足 $|\mathrm{BF}(v)| \leqslant 1$，称为 **AVL 树**。这里叶子高度为 1，空树高度为 0，因此叶子 $\mathrm{BF} = 0$。

插入一个结点后，只有从插入点到根这条路径上的平衡因子会变。从插入点往上找，第一个失衡的结点记为 z；沿 z 到插入点的路径取下面两层的结点 y 和 x，根据 z, y, x 的相对位置分成四种情况：

- **LL**：x 在 z 的左子树的左子树，对 z 做一次**右旋**；
- **RR**：x 在 z 的右子树的右子树，对 z 做一次**左旋**；
- **LR**：x 在 z 的左子树的右子树，先对 y 左旋，再对 z 右旋；
- **RL**：x 在 z 的右子树的左子树，先对 y 右旋，再对 z 左旋。

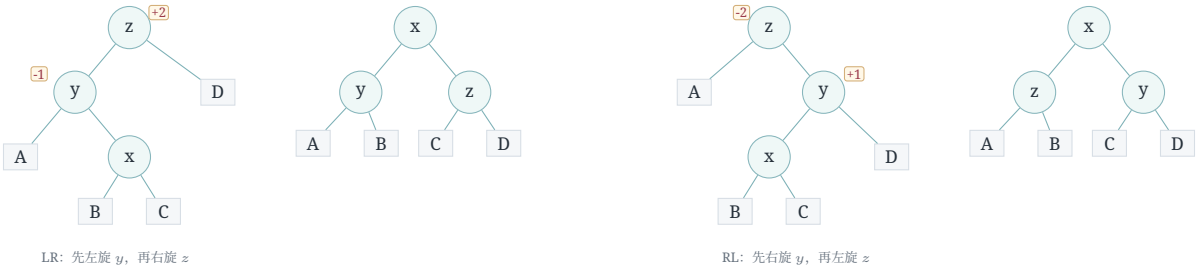


图怎么看：左边是旋转前，右边是旋转后，方框 A–D 表示高度为 h 或 $h+1$ 的子树。LL 和 RR 都是“中间结点 y 上位”，一次旋转即可；旋转后 x, y, z 的平衡因子都回到 0，子树高度也恢复。

单旋转的代码以 LL 为例：把 y 的右子树挂到 z 的左边，再让 z 成为 y 的右孩子。

```
void RotateRight(BiTree *z) {       // LL
    BiTree y = (*z)->lchild;
    (*z)->lchild = y->rchild;       // y 的右子树改挂到 z 左边
    y->rchild = *z;
    *z = y;                         // y 上位
}
void RotateLeft(BiTree *z) {        // RR
    BiTree y = (*z)->rchild;
    (*z)->rchild = y->lchild;
    y->lchild = *z;
    *z = y;
}
```

双旋转的情况多一步“先把孙子转成直链”：LR 先对 y 左旋，变成 LL，再对 z 右旋；RL 先对 y 右旋，变成 RR，再对 z 左旋。



图怎么看：LR 旋转前 z 的左子树偏高、y 的右子树偏高，两个方向相反，单旋解决不了；先把 x 转到 y 的位置变成 LL，再按 LL 处理。RL 完全对称。旋转后新根都是 x。

AVL 的插入最多做一次旋转（单旋或双旋）就能恢复整棵树；删除则可能沿路径一直往上，需要 $O(\log n)$ 次旋转。无论哪种，查找、插入、删除都是 $O(\log n)$。

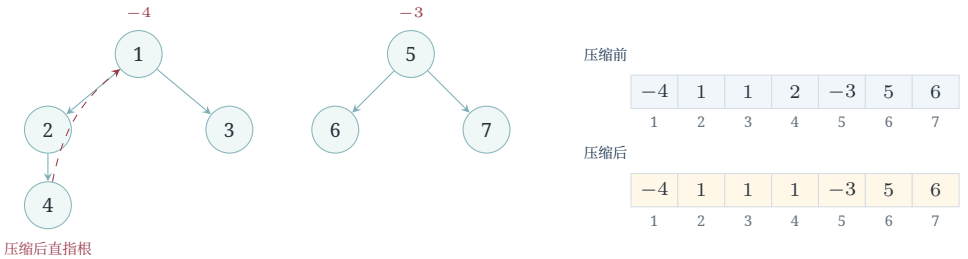
为什么高度能压到对数级？考虑高度为 h 的 AVL 树最少需要多少个结点，记为 N_h（叶子高度为 1，空树 $h=0$）。根的两棵子树高度最多差 1，为了让结点最少，一边取 $h-1$，另一边取 $h-2$：

$$N_h = N_{h-1} + N_{h-2} + 1, \qquad N_0 = 0,\ N_1 = 1.$$

数列是 $1, 2, 4, 7, 12, 20, \ldots$，恰好 $N_h = F_{h+2} - 1$（$F_1 = F_2 = 1$）。由 Fibonacci 数的增长可得 $h < 1.44 \log_2(n+2)$，所以 $h = O(\log n)$。

7.11 并查集：只关心“谁和谁一伙”

并查集管理若干个不相交的集合，只支持两种操作：查某个元素属于哪个集合（`Find`），把两个集合合并（`Union`）。它底层就是一片森林，用双亲表示法存：数组 `parent[i]` 记录 i 的双亲，根结点的 `parent` 存负数，绝对值表示集合大小。



图怎么看：箭头指向双亲，根没有双亲，所以它的格子里存负数，-4 表示这个集合有 4 个元素。执行 `Find(4)` 后，路径 $4 \to 2 \to 1$ 上的结点被直接挂到根 1 下，数组里只有 `parent[4]` 由 2 改成 1。

```
int parent[MaxSize];

int Find(int x) {                    // 路径压缩
    if (parent[x] < 0) return x;
    return parent[x] = Find(parent[x]);
}

void Union(int x, int y) {           // 按大小合并
    int rx = Find(x), ry = Find(y);
    if (rx == ry) return;
    if (parent[rx] > parent[ry]) {   // rx 集合更小（负数的绝对值小）
        parent[ry] += parent[rx];
        parent[rx] = ry;
    } else {
        parent[rx] += parent[ry];
        parent[ry] = rx;
    }
}
```

只按大小合并，树高是 $O(\log n)$；只做路径压缩，单次操作可能很慢但摊还也不差；两者一起用，摊还复杂度是 $O(\alpha(n))$，其中 α 是反 Ackermann 函数，在实际规模内不超过 4。408 里它常与 Kruskal 算法一起出现：判断加入一条边会不会成环，就是判断两个端点是否已经在同一个集合。

7.12 几组容易混的说法

概念对照

容易错的说法	正确的理解
二叉树就是度为 2 的树	二叉树允许没有度为 2 的结点，但必须区分左右；度为 2 的树至少有一个度为 2 的结点
满二叉树 = 完全二叉树	满是每层都满；完全是最后一层从左到右连续，可以缺右边的结点
树的度 = 结点的度	树的度是所有结点度的最大值；某个结点的度是它的孩子数
深度从 0 开始	408 常用根深度为 1；题目另有约定时按题目
先序 + 后序能定树	不能，必须再有中序（或层次）序列
线索树中 `lchild` 一定指孩子	要看 `ltag`：0 是孩子，1 是前驱
Huffman 树唯一	树形不唯一，WPL 唯一
AVL 插入后要一路旋转	插入最多一次单旋或双旋；删除才可能多次

8 习题

8.1 基础过关题

1. (单项选择题) 在一棵树中，有 5 个度为 5 的结点、4 个度为 4 的结点、3 个度为 3 的结点、2 个度为 2 的结点，其余结点都是叶结点。则该树的叶结点个数为

(A) 39 (B) 40

(C) 41 (D) 45

2. (填空题) 一棵有 1000 个结点的完全二叉树，其叶结点个数为 ____，高度为 ____（约定根的层次为 1）。

3. (单项选择题) 已知一棵二叉树的先序遍历序列为 ABDFCEG，中序遍历序列为 DBFAECG，则其后序遍历序列为

(A) DFBEGCA (B) DFBECGA

(C) FDBEGCA (D) DFBEGAC

4. (单项选择题) 下列关于线索二叉树的叙述中，正确的是

(A) 先序线索二叉树中，任一结点都可在不访问双亲的情况下直接找到其先序前驱

(B) 后序线索二叉树中，任一结点都可在不访问双亲的情况下直接找到其后序后继

(C) 中序线索二叉树中，任一结点都可在不访问双亲的情况下直接找到其中序前驱和中序后继

(D) 三种线索二叉树中，任一结点都可在不访问双亲的情况下直接找到其遍历后继

5. (单项选择题) 对权值集合 $\{3, 6, 8, 12, 15\}$ 构造 Huffman 树，其带权路径长度 WPL 为

(A) 95 (B) 96

(C) 97 (D) 100

6. （单项选择题）从空树开始，依次插入关键字 $30, 20, 10, 25, 28$ 构造 AVL 树，最终根结点与其右孩子的关键字分别为

(A) 20 和 25　(B) 20 和 28

(C) 28 和 30　(D) 30 和 25

8.2 真题风格与综合训练

7. （真题风格・单项选择题）在下列关键字序列中，不可能是同一棵二叉排序树中某次查找所经过的结点序列的是

(A) 50, 30, 70, 60, 55, 58

(B) 50, 70, 60, 65, 62

(C) 50, 30, 70, 60, 55, 65

(D) 50, 30, 70, 65, 62

8. （真题风格・单项选择题）设森林 F 按“左孩子右兄弟”规则转换为二叉树 B。已知 B 的先序遍历序列为 ABCDEF，中序遍历序列为 CBDAEF，则 F 中树的棵数与第一棵树的结点数分别为

(A) 3，4　(B) 4，4

(C) 3，3　(D) 2，4

9. （真题风格・综合题）七个字符 A~G 在电文中出现的次数依次为 $4, 7, 9, 11, 14, 20, 25$（单位：千次），总次数为 90 千次。

   (1) 按上述权值构造 Huffman 树，写出每一步的合并过程及合并后的森林；

   (2) 约定左分支为 0、右分支为 1，给出各字符的 Huffman 编码，并说明编码是否唯一；

   (3) 计算 WPL 和平均码长（单位：位），写出计算口径；

   (4) 若改用定长编码，至少需要多少位？与 Huffman 编码相比，传输这 90 千个字符可节省多少比特？

10. （真题风格・综合题）设二叉树以二叉链表存储，结点结构为 `lchild`、`data`、`rchild`，相关栈操作 `InitStack`、`Push`、`Pop`、`StackEmpty` 均已定义。

    (1) 写出中序遍历的非递归算法（用栈），并说明每个结点入栈、出栈各多少次；

    (2) 若把该树中序线索化（结点增加 `ltag`、`rtag`），写出不借助栈的中序遍历算法；

    (3) 比较两种方法的时间、空间复杂度，并说明后序线索二叉树为什么通常仍需栈才能遍历。

拓展阅读：

- 邓俊辉《数据结构（C++ 语言版）》第 5 章「二叉树」——遍历算法族、线索化与 Huffman 编码的贪心正确性；第 6 章「图」中并查集在 Kruskal 算法里的用法。
- Sedgewick《算法》（第 4 版）第 3.2–3.3 节「二叉查找树」与「平衡查找树」——从 BST 的查找、插入、删除过渡到 AVL 与红黑树的平衡思想。

9 图

课程之间的先修关系、城市之间的公路网、网络中数据包的转发路径，都可以抽象成同一类对象：用点表示事物，用线表示事物之间的联系。点之间的线有没有方向、带不带权值，决定了后续该用哪种算法。本章从这种抽象出发，先说明图怎么存，再讨论怎么遍历，最后处理最小生成树、最短路径、拓扑排序和关键路径这几类典型问题。

9.1 图的基本概念

一个图记作 $G=(V,E)$。V 是顶点集，E 是边集。$|V|=n$ 称为图的阶，$|E|=e$ 是边数。边没有方向时写作 (u,v)，称为无向边；边有方向时写作 $\langle u,v\rangle$，称为弧，u 是弧尾，v 是弧头。既没有自环、也没有重边的图称为简单图；本章默认讨论简单图。

若任意两个顶点之间都有边，称为完全图。无向完全图 K_n 的边数是

$$\binom{n}{2}=\frac{n(n-1)}{2},$$

有向完全图每对顶点之间有两个方向的弧，弧数为 $n(n-1)$。这两个数给出了简单图边数的上界。

度、路径与连通

无向图中，与顶点 v 关联的边数称为 v 的度，记作 $\deg(v)$。有向图把度拆成入度和出度：以 v 为弧头的弧数是入度 $\mathrm{id}(v)$，以 v 为弧尾的弧数是出度 $\mathrm{od}(v)$。每条无向边有两个端点，每条弧有一个弧尾和一个弧头，因此

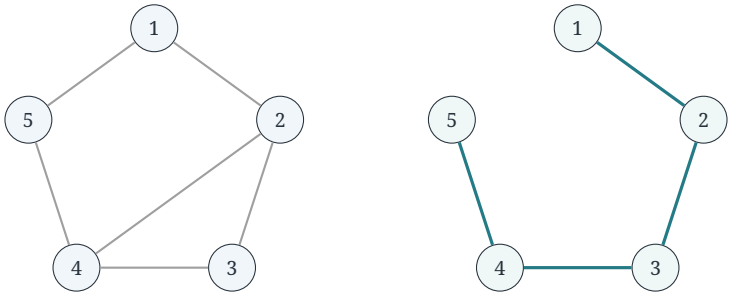
$$\sum_{v\in V}\deg(v)=2e,\qquad \sum_{v\in V}\mathrm{id}(v)=\sum_{v\in V}\mathrm{od}(v)=e.$$

第一条式子常称为握手定理。它有一个直接推论：度数为奇数的顶点个数一定是偶数。

顶点序列 $v_0,v_1,\dots,v_k$ 中，若相邻两项之间都有边，则称它为一条路径，路径上边的数目 k 是路径长度。顶点不重复的路径称为简单路径。起点和终点相同的路径称为回路；除起点终点外顶点不重复的回路称为简单回路。

无向图中，若 u 到 v 存在路径，称 u 与 v 连通。任意两个顶点都连通的图称为连通图。连通图的极大连通子图就是它本身；非连通图可以分成若干个连通分量。这里的“极大”指不能再加入顶点或边而保持连通。有向图对应的概念是强连通：任意两个顶点互相可达。强连通图的极大强连通子图称为强连通分量。

连通图的生成树，是包含全部 n 个顶点、恰好 $n-1$ 条边的连通子图。它连通且没有回路，去掉任意一条边都会变成非连通图，因此是“极小连通子图”。非连通图有 k 个连通分量时，每个分量各取一棵生成树，合起来称为生成森林，共 $n-k$ 条边。



原图 G　　生成树 T

左图有 6 条边；右图保留 4 条边仍然连通，再删一条就会断开。右图是左图的一棵生成树，边数正好是 $n-1=4$。

9.2 图的存储

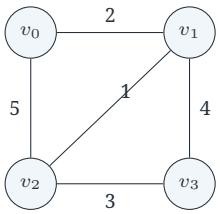
图有两种最常用的存法。邻接矩阵关心“任意两点之间有没有边”，邻接表关心“每个点连出去哪些边”。选用哪一种，取决于图是稠密还是稀疏，以及算法需要频繁做哪类查询。

邻接矩阵

设顶点编号为 $v_0, v_1, \ldots, v_{n-1}$。邻接矩阵 A 是 $n \times n$ 的矩阵。无权图中

$$A[i][j] = \begin{cases} 1, & (v_i, v_j) \in E \text{ 或} \langle v_i, v_j \rangle \in E, \\ 0, & \text{否则.} \end{cases}$$

带权图把 1 换成边权 w_{ij}；没有边时填 ∞，而不是 0。这一点在最短路径里很关键：0 是合法的边权，用它表示“没有边”会让松弛过程把不存在的边当成权值为 0 的边。



	v_0	v_1	v_2	v_3
v_0	0	2	5	∞
v_1	2	0	1	4
v_2	5	1	0	3
v_3	∞	4	3	0

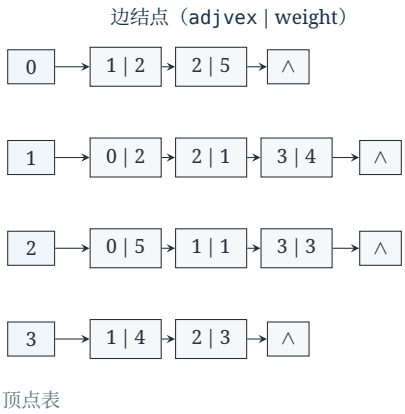
左：带权无向图。右：它的邻接矩阵。无向图的矩阵沿主对角线对称，例如 $A[1][2] = A[2][1] = 1$；对角线为 0，没有边的地方是 ∞。

邻接矩阵的优缺点很直接。空间是 $O(n^2)$，适合边数接近 n^2 的稠密图。判断 (v_i, v_j) 是否有边只需读一个单元，时间 $O(1)$。无权图中，无向图顶点 v_i 的度就是第 i 行非零元的个数；有向图的出度是行和，入度是列和。带权图不能直接对行求和来求度，要先数不等于 ∞ 的元素。代价是，遍历一个顶点的所有邻边要扫描整行，时间是 $O(n)$，与它实际连了多少条边无关。

对无权图，邻接矩阵还有一个性质：$A^k[i][j]$ 等于从 v_i 到 v_j、长度为 k 的走法数。这里的走法允许重复经过顶点和边，不能把它当成简单路径的条数。

邻接表

邻接表为每个顶点挂一条单链表，链表结点存邻接点下标和边权。无向图的每条边存两次，所以 e 条边需要 $2e$ 个边结点；有向图每条弧存一次，需要 e 个边结点。



同一个无向图的邻接表。每个边结点存“邻接点下标 | 权值”。无向图每条边存两次，共 $2e = 10$ 个边结点；表尾用 $\wedge$ 表示空指针。

邻接表的空间是 $O(n+e)$，适合稀疏图。求无向图中顶点 v 的度，只需数它链表的结点个数，时间是 $O(\deg(v))$。求有向图的出度同样方便；求入度则要扫描全部链表，或者另外建一张逆邻接表。判断 (u,v) 是否存在也要沿着 u 的链表找，时间是 $O(\deg(u))$。邻接表不唯一：同一条链表里，结点的先后顺序可以不同，所以同一张图可以写出多种邻接表。

两种存法的对比如下。

	邻接矩阵	邻接表
空间	$O(n^2)$	$O(n+e)$
判断 (u,v) 是否有边	$O(1)$	$O(\deg(u))$
遍历 u 的全部邻边	$O(n)$	$O(\deg(u))$
适合	稠密图	稀疏图
存储是否唯一	唯一	不唯一

后文的 `MGraph` 指邻接矩阵，`ALGraph` 指邻接表，`ArcNode` 是邻接表中的边结点。

此外还有两种改进结构。十字链表用于有向图：顶点结点存 `data`、`firstin`、`firstout`，弧结点存 `tailvex`、`headvex`、`weight`、`hlink`、`tlink`。一条弧同时挂进“同弧尾”和“同弧头”两条链，因此出度和入度都能顺着链求。邻接多重表用于无向图：每条边只建一个结点，存 `ivex`、`jvex`、`ilink`、`jlink`，两个端点各自把这条边挂进自己的链。它避免了邻接表中一条无向边存两次的问题，删除边也更方便。

9.3 图的遍历

遍历图是从某个顶点出发，沿边走到所有能到达的顶点，并且每个顶点只访问一次。和树的遍历不同，图里可能有回路，所以必须用一个 `visited` 数组记录已经访问过的顶点，否则会在回路里打转。

广度优先搜索

广度优先搜索（BFS）按“离起点的边数”分层访问。先访问起点，再访问所有与起点直接相邻的顶点，然后访问距离为 2 的顶点，依此类推。实现时用一个队列保存“已经发现、但还没有扩展其邻边”的顶点。

```c
void BFS(ALGraph *G, int s) {
    bool visited[MaxV] = {false};
    int queue[MaxV], front = 0, rear = 0;
    visited[s] = true;
    queue[rear++] = s;
    while (front < rear) {
        int u = queue[front++];
        visit(u);
        for (ArcNode *p = G->adjlist[u].first; p; p = p->next) {
            int v = p->adjvex;
            if (!visited[v]) {
                visited[v] = true;      // 入队时就标记，避免重复入队
                queue[rear++] = v;
            }
        }
    }
}
```

入队时立刻置 `visited[v]=true`，而不是等出队时再标记。否则同一个顶点可能被多个前驱重复放入队列。

BFS 有一个重要性质：在无权图中，BFS 首次访问顶点 v 时经过的边数，就是起点到 v 的最短路径长度。因为 BFS 严格按层扩展，距离为 k 的顶点全部处理完，才会处理距离为 $k+1$ 的顶点。把 BFS 过程中“首次发现顶点”的边留下来，就得到 BFS 生成树；非连通图从每个未访问顶点各做一次 BFS，得到 BFS 生成森林。

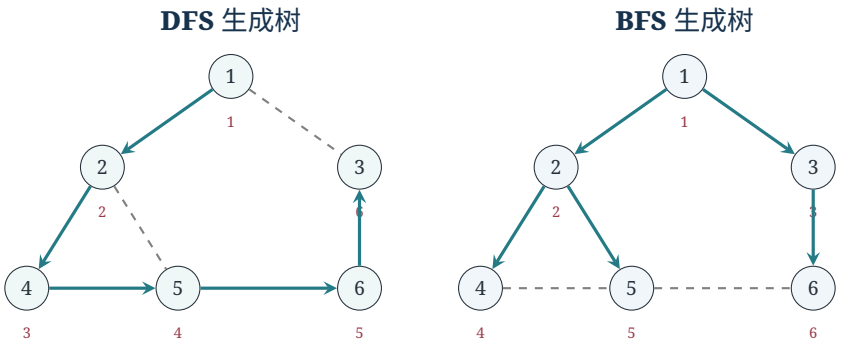
深度优先搜索

深度优先搜索（DFS）沿一条路一直走，走到不能再走时回溯，换一个分支继续。递归写法最自然。

```c
bool visited[MaxV];
void DFS(ALGraph *G, int v) {
    visited[v] = true;
    visit(v);
    for (ArcNode *p = G->adjlist[v].first; p; p = p->next)
        if (!visited[p->adjvex])
            DFS(G, p->adjvex);
}
```

DFS 的递归深度最多为 n，栈空间是 $O(n)$。和 BFS 一样，DFS 过程中首次到达某个顶点所用的边构成 DFS 生成树。指向祖先的非树边称为回边；在无向图中，非树边一定是回边；在有向图中还可能出现指向已完成顶点的前向边或横叉边。无向图中遇到回边说明存在回路；有向图中回边指向当前递归栈里的祖先，同样说明存在有向回路。

	BFS	DFS
辅助结构	队列	栈（或递归）
访问顺序	按层扩展	沿路径深入，走不通再回溯
生成树	根到顶点的路径边数最少	不保证边数最少
典型用途	无权最短路、分层	连通性、回路、拓扑排序
时间复杂度	$O(n+e)$ 或 $O(n^2)$	$O(n+e)$ 或 $O(n^2)$



同一张图，从顶点 1 出发。实线箭头是生成树边，灰色虚线是非树边；顶点下方的小号数字是访问次序。DFS 序列为 $1,2,4,5,6,3$，BFS 序列为 $1,2,3,4,5,6$。

两种遍历的时间复杂度相同。邻接表存储时，每个顶点访问一次，每条边扫描一次（无向图两次），总计 $O(n+e)$。邻接矩阵存储时，每访问一个顶点都要扫描它的一整行，总计 $O(n^2)$。

遍历序列是否唯一，取决于题目有没有规定起点和邻接点的扫描顺序。即使用邻接矩阵，若从不同顶点出发，序列也会不同；用邻接表时，链表里结点的顺序还会影响序列。考试中若要比较两个序列，先看清题目给出的起点和“按编号递增”之类的规则。

> **注意**
>
> BFS 能求无权图的最短路径，DFS 不能。DFS 找到的路径只保证是一条可行路径，不保证边数最少。反过来，DFS 更适合判断回路、求强连通分量、做拓扑排序；BFS 更适合分层和求最少边数。

9.4 最小生成树

在一个连通带权无向图中，生成树有很多棵。边权之和最小的那棵称为最小生成树（Minimum Spanning Tree，MST）。例如要在 n 个城市之间铺光缆，要求任意两城连通且总长度最短，就是在求 MST。

MST 可能不唯一。比如一个三角形的三条边权都是 1，任意去掉一条边都能得到一棵 MST。但所有 MST 的总权值相同。若图中所有边权互不相同，MST 唯一。

求 MST 的两种经典算法都基于同一个事实：对图的任意一个割（把顶点分成两个非空集合），横跨这个割的最小权边一定属于某棵 MST。这条性质称为割性质。Prim 算法每次选一个割，Kruskal 算法则从全局最小的边开始，用并查集判断会不会成环。

Prim 算法

Prim 算法维护一个已经加入树的顶点集合 S。初始时 S 只含起点。每一轮，在“一个端点在 S 内、另一个端点在 S 外”的边中选权值最小的一条，把外面的顶点加入 S。重复 $n-1$ 轮，就得到一棵生成树。

```
void Prim(MGraph G, int s) {
    int lowcost[MaxV], closest[MaxV];
    bool inTree[MaxV] = {false};
    for (int i = 0; i < G.n; i++) {
        lowcost[i] = G.edges[s][i];   // i 到当前树的最小边权
        closest[i] = s;               // 该边在树内的端点
```

```
    }
    inTree[s] = true;
    lowcost[s] = 0;
    for (int k = 1; k < G.n; k++) {
        int u = -1, min = INF;
        for (int i = 0; i < G.n; i++)
            if (!inTree[i] && lowcost[i] < min) {
                min = lowcost[i]; u = i;
            }
        if (u == -1) return;          // 图不连通
        inTree[u] = true;
        for (int i = 0; i < G.n; i++)
            if (!inTree[i] && G.edges[u][i] < lowcost[i]) {
                lowcost[i] = G.edges[u][i];
                closest[i] = u;
            }
    }
}
```

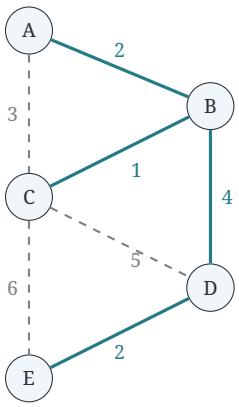
`lowcost[i]` 始终表示顶点 i 到当前树 S 的最小边权，`closest[i]` 记录这条最小边在树内的端点。选中最小的 `lowcost[u]` 后，把 u 加入树，再用 u 的出边更新其他顶点的 `lowcost`。

用邻接矩阵实现时，每轮找最小值和更新各需 $O(n)$，共 $n-1$ 轮，时间复杂度 $O(n^2)$。这个复杂度只和顶点数有关，和边数无关，所以 Prim 适合稠密图。用邻接表和堆优化可以做到 $O(e\log n)$，但 408 更常考 $O(n^2)$ 的数组版本。

Kruskal 算法

Kruskal 算法把所有边按权值从小到大排序，依次检查。若当前边的两个端点属于不同的连通分量，就加入这条边；否则加入会成环，直接舍弃。选够 $n-1$ 条边时停止。

判断两个端点是否属于同一分量，用并查集。初始时每个顶点各自成一个集合，加入一条边就合并两个集合。排序是主要开销，时间复杂度 $O(e\log e)$；并查集的操作近似常数。Kruskal 的时间主要花在边数上，适合稀疏图。



粗线是 Kruskal 依次选出的边：$BC(1)$、$AB(2)$、$DE(2)$、$BD(4)$，总权 9。虚线边 $AC(3)$、$CD(5)$、$CE(6)$ 加入后会成环，因此舍弃。若从 A 出发运行 Prim，选边次序是 AB, BC, BD, DE，得到的树相同。

9.5 最短路径

最短路径问题分两种：单源最短路径求一个起点到其余各点的最短路径；全源最短路径求任意两点之间的最短路径。边权可以为负，但本章默认不含负权回路；有负权回路时，绕回路可以无限减小路径长度，最短路径不存在。

无权图：BFS

无权图可以把每条边看成权值为 1。此时 BFS 按层扩展，首次到达某顶点经过的边数就是最短路径长度。BFS 生成树上从起点到顶点的路径就是一条最短路径。这个方法时间是 $O(n+e)$，比直接套用带权算法更简单。

Dijkstra 算法

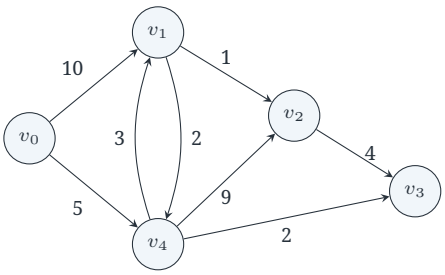
Dijkstra 算法处理非负边权的单源最短路径。它维护一个已经确定最短距离的顶点集合 S，以及数组 $d[i]$，表示目前找到的从源点到 i 的最短距离。每轮从未确定的顶点中选 d 最小的那个，把它加入 S，再用它的出边松弛其他顶点。

```
void Dijkstra(MGraph G, int s, int dist[], int path[]) {
    bool final[MaxV] = {false};
    for (int i = 0; i < G.n; i++) {
        dist[i] = G.edges[s][i];
        path[i] = (dist[i] < INF) ? s : -1;
    }
    dist[s] = 0; final[s] = true; path[s] = -1;
    for (int k = 1; k < G.n; k++) {
        int u = -1, min = INF;
        for (int i = 0; i < G.n; i++)
            if (!final[i] && dist[i] < min) {
                min = dist[i]; u = i;
            }
        if (u == -1) break;           // 剩余顶点不可达
        final[u] = true;
        for (int i = 0; i < G.n; i++)
            if (!final[i] && dist[u] + G.edges[u][i] < dist[i]) {
                dist[i] = dist[u] + G.edges[u][i];
                path[i] = u;
            }
    }
}
```

`path[i]` 记录最短路径上 i 的前驱。求完整路径时从终点沿 `path` 往回走，直到源点，再反过来输出。

Dijkstra 与 Prim 的代码结构很像，区别在数组含义。Prim 的 `lowcost[i]` 是 i 到当前树的一条边权；Dijkstra 的 `dist[i]` 是源点到 i 的整条路径长度。更新时，Prim 用 $w(u,i)$ 直接比较，Dijkstra 用 `dist[u]`+$w(u,i)$ 比较。

邻接矩阵实现的时间复杂度是 $O(n^2)$，适合稠密图。用邻接表加优先队列可以做到 $O((n+e)\log n)$。408 选择题常要求手算每一轮的 d 数组，下面用一个具体例子走一遍。



轮次	选出	d_0	d_1	d_2	d_3	d_4
初始	–	0	10	∞	∞	5
1	v_4	0	8	14	7	**5**
2	v_3	0	8	14	**7**	5
3	v_1	0	**8**	9	7	5
4	v_2	0	8	**9**	7	5

源点为 v_0。表中是每轮松弛后的 d 值，粗体是本轮选中的顶点。最终最短路为 $v_0 \to v_4 \to v_1 \to v_2$，长度 $5+3+1=9$。

为什么 Dijkstra 要求边权非负？看一个反例：$s \to a$ 的权是 2，$s \to b$ 的权是 5，$b \to a$ 的权是 -4。Dijkstra 第一轮会选中 a，把 $d[a]=2$ 定死；但实际最短路径是 $s \to b \to a$，长度为 $5-4=1$。负权边让“当前最小的 d”不再保证是最终答案，贪心选择失效。

Floyd 算法

Floyd 算法求任意两点之间的最短路径。它用动态规划，把“允许经过哪些顶点作为中转”作为阶段。定义 $D^{(k)}[i][j]$ 为：从 v_i 到 v_j，只允许把 $v_0, v_1, \ldots, v_k$ 当作中转点时的最短路径长度。这里的“中转点”不包含起点和终点本身。

初始时 $D^{(0)}$ 就是邻接矩阵：要么是直达边的权值，要么是 ∞。加入第 k 个中转点后，从 i 到 j 的路径只有两种可能：不经过 v_k，长度是 $D^{(k-1)}[i][j]$；经过 v_k，长度是 $D^{(k-1)}[i][k]+D^{(k-1)}[k][j]$。取较小者：

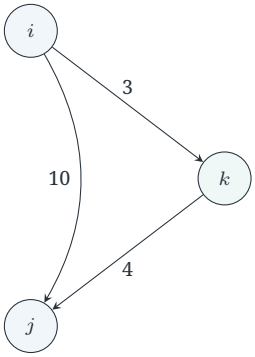
$$D^{(k)}[i][j]=\min\left(D^{(k-1)}[i][j],\ D^{(k-1)}[i][k]+D^{(k-1)}[k][j]\right).$$

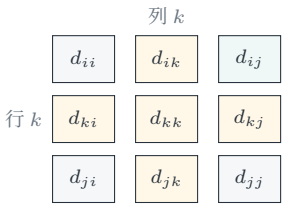
k 从 0 到 $n-1$ 循环，最后的 $D^{(n)}$ 就是全源最短路径长度矩阵。

```
void Floyd(MGraph G, int D[][MaxV], int path[][MaxV]) {
    for (int i = 0; i < G.n; i++)
        for (int j = 0; j < G.n; j++) {
            D[i][j] = G.edges[i][j];
            path[i][j] = (D[i][j] < INF && i != j) ? i : -1;
        }
    for (int k = 0; k < G.n; k++)
        for (int i = 0; i < G.n; i++)
            for (int j = 0; j < G.n; j++)
                if (D[i][k] + D[k][j] < D[i][j]) {
                    D[i][j] = D[i][k] + D[k][j];
                    path[i][j] = path[k][j];
                }
}
```

三个循环的顺序不能随意交换。k 必须在最外层：每一轮只允许加入一个新的中转点，三轮循环分别固定“阶段、起点、终点”。如果把 k 放到最内层，某些需要多个中转点的路径会在错误的阶段被更新，结果可能偏大。D 可以在原地更新，因为第 k 轮中 $D[i][k]$ 和 $D[k][j]$ 都不会改变：它们对应的路径不允许把 v_k 当中转点，加入 v_k 不会让它们更短（前提是没有负权回路）。

时间复杂度 $O(n^3)$，空间 $O(n^2)$。Floyd 允许负权边，但不能有负权回路。运行结束后若某个 $D[i][i]<0$，说明存在从 i 出发又回到 i 的负权回路，最短路径无意义。





第 k 轮把 v_k 作为中转点：比较直达的 $d_{ij}=10$ 和绕行的 $d_{ik}+d_{kj}=3+4=7$，更新为 7。行 k 与列 k 的旧值在本轮保持不变，它们提供 d_{ik} 和 d_{kj} 两个加数。

例如，四个顶点的有向图，边权如下：

$$D^{(0)}=\begin{pmatrix}0&4&11&\infty\\6&0&2&\infty\\3&\infty&0&4\\\infty&7&5&0\end{pmatrix},\qquad D^{(4)}=\begin{pmatrix}0&4&6&10\\5&0&2&6\\3&7&0&4\\8&7&5&0\end{pmatrix}.$$

计算时按 $k=0,1,2,3$ 逐轮更新。$k=0$ 时，$D[2][1]$ 由 ∞ 变为 $3+4=7$；$k=1$ 时，$D[0][2]$ 由 11 变为 $4+2=6$；$k=2$ 时，$D[0][3]$ 由 ∞ 变为 $6+4=10$，$D[1][0]$ 由 6 变为 $2+3=5$，$D[3][0]$ 由 ∞ 变为 $5+3=8$；$k=3$ 时没有更短的组合。最终矩阵中 $D[0][3]=10$ 对应路径 $0\to1\to2\to3$，长度 $4+2+4=10$。

9.6 拓扑排序

有向无环图（DAG）常用于表示“谁必须在谁之前完成”。例如课程 A 是课程 B 的先修课，就从 A 向 B 连一条弧。把这种图的所有顶点排成一个线性序列，使得每条弧的弧尾都排在弧头之前，这个序列称为拓扑序列，求序列的过程称为拓扑排序。

拓扑序列不唯一：算法在某一步可能同时有多个入度为 0 的顶点，先输出哪一个都可以。两个没有路径关系的顶点，在不同拓扑序列中的先后可以不同；但在同一个序列里任意交换两个不相邻的顶点，可能会破坏别的约束。一个 DAG 的拓扑序列唯一，当且仅当它在每一步都只有一个入度为 0 的顶点；等价地说，图中存在一条经过所有顶点的有向路径。

基于入度的 Kahn 算法

Kahn 算法反复做两件事：输出一个入度为 0 的顶点；删除它的所有出边，并把终点的入度减 1。删边后可能出现新的入度为 0 的顶点，把它加入队列。若最终输出的顶点数小于 n，说明剩下的顶点都有入边，图中存在有向回路。

```
bool TopoSort(ALGraph *G, int topo[]) {
    int indeg[MaxV] = {0}, cnt = 0;
    for (int u = 0; u < G->n; u++)
        for (ArcNode *p = G->adjlist[u].first; p; p = p->next)
            indeg[p->adjvex]++;
    int queue[MaxV], front = 0, rear = 0;
    for (int i = 0; i < G->n; i++)
        if (indeg[i] == 0) queue[rear++] = i;
```

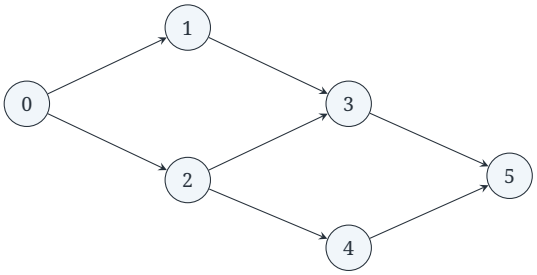
```
    while (front < rear) {
        int u = queue[front++];
        topo[cnt++] = u;
        for (ArcNode *p = G->adjlist[u].first; p; p = p->next)
            if (--indeg[p->adjvex] == 0)
                queue[rear++] = p->adjvex;
    }
    return cnt == G->n;       // false 表示有回路
}
```

用邻接表时，计算入度要扫描所有边，之后每个顶点入队、出队一次，每条边被处理一次，总时间 $O(n+e)$。用栈代替队列也能得到拓扑序列，只是输出次序不同。下一小节的 DFS 方法从另一个角度得到拓扑序：先递归到最深处，回溯时把顶点压栈，最后从栈顶依次弹出。

基于 DFS 的方法

对 DAG 做 DFS，记录每个顶点“完成”的次序。一个顶点完成时，它的所有后继都已经完成，所以按完成时间从晚到早排列，就得到拓扑序列。实现时把顶点在回溯阶段压入栈，最后依次弹出即可。

DFS 还能同时判断回路：如果从 u 出发的 DFS 遇到一个仍在递归栈中的顶点 v，说明存在树边路径 $v \to \cdots \to u$，再加上回边 $u \to v$，构成有向回路。用一个颜色数组区分“未访问、在栈中、已完成”三种状态即可。





图中下方一行是其中一个拓扑序列。各顶点入度：$v_0:0$，$v_1:1$，$v_2:1$，$v_3:2$，$v_4:1$，$v_5:2$。按“编号小的先出队”的规则，得到 $0,1,2,3,4,5$。若某一步同时有两个入度为 0 的顶点，交换它们的次序可以得到另一个合法序列。

在图中的 DAG 上再加一条弧 $v_5 \to v_1$，就出现回路 $v_1 \to v_3 \to v_5 \to v_1$。此时 Kahn 算法输出 v_0, v_2, v_4 后，剩余顶点的入度都不为 0，输出个数小于 n，从而检测到回路。

9.7 AOE 网与关键路径

AOV 网（Activity On Vertex）用顶点表示活动，用弧表示活动之间的先后关系，拓扑排序解决的是“活动按什么顺序做”。AOE 网（Activity On Edge）换一种表示：弧表示活动，弧上的权表示活动持续时间，顶点表示事件，即某些活动完成或开始的时刻。AOE 网通常只有一个源点（开始事件）和一个汇点（结束事件），源点入度为 0，汇点出度为 0。

关键路径是 AOE 网中从源点到汇点的最长路径，它的长度就是整个工程的最短工期。路径越长，说明这条链上的活动越不能拖延。要找出关键路径，需要计算每个事件的最早发生时间和最迟发生时间。

事件的最早、最迟发生时间

设 $ve(k)$ 是事件 v_k 的最早发生时间。源点的 ve 为 0。其他事件必须等所有前驱活动完成才能发生，所以

$$ve(k) = \max_{\langle v_j, v_k\rangle \in E} \{ve(j) + w(j,k)\}.$$

按拓扑序从左往右算，计算 v_k 时它的所有前驱都已经算好。

设 $vl(k)$ 是事件 v_k 的最迟发生时间，即在不推迟整个工期的前提下，事件最迟可以在什么时刻发生。汇点的 vl 等于它的 ve，也就是总工期。其他事件必须给所有后继活动留出足够时间，所以

$$vl(k) = \min_{\langle v_k, v_j\rangle \in E} \{vl(j) - w(k,j)\}.$$

按逆拓扑序从右往左算，计算 v_k 时它的所有后继都已经算好。

活动的最早、最迟开始时间

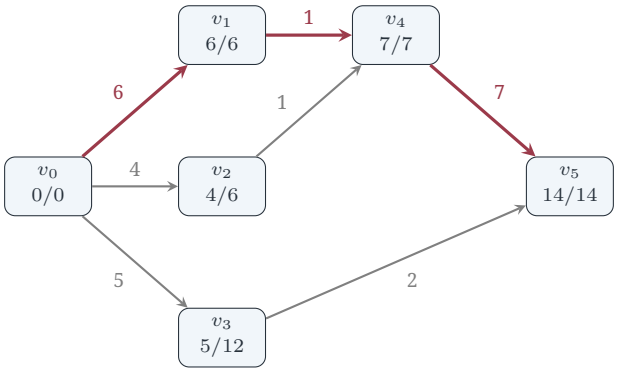
对活动 $a = \langle v_i, v_j\rangle$，它的最早开始时间等于弧尾事件的最早发生时间：

$$e(a) = ve(i).$$

它的最迟开始时间等于弧头事件的最迟发生时间减去活动持续时间：

$$l(a) = vl(j) - w(i,j).$$

两者之差 $l(a) - e(a)$ 是活动的时间余量，也叫松弛时间。若 $l(a) = e(a)$，说明这个活动没有任何余量，推迟一天就会推迟整个工程，称为关键活动。由关键活动首尾相接、从源点通到汇点的路径就是关键路径。



事件框内为 ve/vl；粗线是关键活动，关键路径 $v_0 \to v_1 \to v_4 \to v_5$，工期 $6+1+7=14$。

把各活动的 e、l 列成表，能更清楚地看出谁有余量。

活动	弧	持续时间	e	l	$l-e$
a_1	$v_0 \to v_1$	6	0	0	0
a_2	$v_0 \to v_2$	4	0	2	2
a_3	$v_0 \to v_3$	5	0	7	7
a_4	$v_1 \to v_4$	1	6	6	0
a_5	$v_2 \to v_4$	1	4	6	2
a_6	$v_3 \to v_5$	2	5	12	7
a_7	$v_4 \to v_5$	7	7	7	0

表中 $l-e=0$ 的活动 a_1, a_4, a_7 都是关键活动，它们首尾相接，构成关键路径。a_2 和 a_5 各有 2 个时间单位的余量；a_3 和 a_6 的余量是 7。

关于关键路径，有几个结论经常出现在选择题里。

- 关键路径是源点到汇点的最长路径，长度等于最短工期。
- 关键路径可能不唯一。只缩短其中一条路径上的非公共活动，另一条关键路径的长度不变，总工期也不变。
- 要缩短总工期，必须缩短所有关键路径的公共关键活动；若几条关键路径没有公共活动，就要同时缩短每条关键路径上的活动。
- 缩短非关键活动一般不能缩短工期；一旦缩短量超过它的时间余量，它自己也会变成关键活动，需要重新判断关键路径。

9.8 本章算法对照

把本章的算法放在一起比较，考试时更容易选对方法。

算法	解决的问题	适用条件	时间复杂度
BFS	无权图单源最短路、分层遍历	边权相同	$O(n+e)$（邻接表）
DFS	连通性、回路判定、拓扑排序	无特殊限制	$O(n+e)$（邻接表）
Prim	最小生成树	连通无向带权图	$O(n^2)$，适合稠密图
Kruskal	最小生成树	连通无向带权图	$O(e\log e)$，适合稀疏图
Dijkstra	单源最短路	边权非负	$O(n^2)$ 或 $O((n+e)\log n)$
Floyd	全源最短路	允许负权，无负权回路	$O(n^3)$
Kahn	拓扑排序	有向图，可判回路	$O(n+e)$
关键路径	工程最短工期	AOE 网，DAG	$O(n+e)$

还有几个容易混淆的计数结论：

- 连通无向图至少有 $n-1$ 条边；n 个顶点的树恰好有 $n-1$ 条边。
- 强连通有向图（$n \geqslant 2$）至少有 n 条弧，因为每个顶点至少要有一条出弧和一条入弧。
- 无向图中所有顶点度数之和为 $2e$，奇度顶点有偶数个。
- 邻接表存无向图有 $2e$ 个边结点，存有向图有 e 个边结点。
- 拓扑序列一般不唯一；当且仅当每一步只有一个入度为 0 的顶点时，序列唯一。

拓展阅读：

- 邓俊辉《数据结构（C++ 语言版）》第 6 章「图」——图的 ADT、BFS/DFS、Prim/Kruskal/Dijkstra 的完整实现，以及强连通分量的 Tarjan 算法。
- Sedgewick《算法》第 4 章「图」——无向图与有向图的遍历框架，最小生成树的 Prim 延时版本与即时版本，最短路径的 Dijkstra 与 Bellman-Ford。
- CLRS《算法导论》第 22–25 章——图的表示、广度优先搜索、拓扑排序、最小生成树和单源最短路径的严格分析。

10 习题

10.1 基础过关题

1. （单项选择题）下列叙述中正确的是

(A) 含 6 个顶点的简单无向图，其邻接矩阵中 1 的个数等于边数的 2 倍

(B) 用邻接表存储无向图时，边结点个数等于图的边数

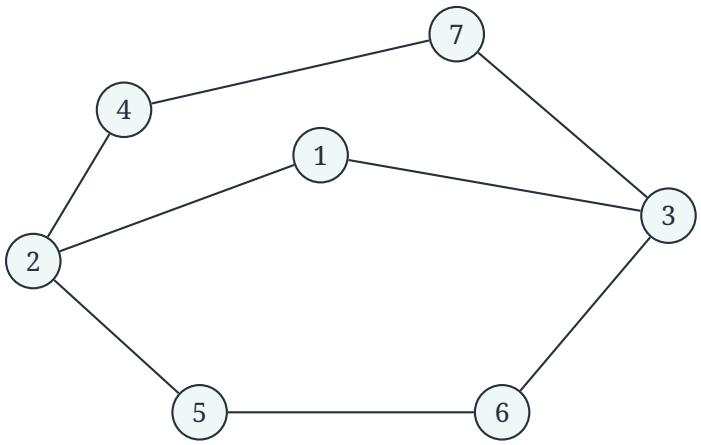
(C) 有向图邻接矩阵中第 i 行非零元素的个数等于顶点 i 的入度

(D) 对稀疏图而言，邻接矩阵比邻接表更省存储空间

2. （填空题）无向图 G 的 7 个顶点的邻接表如下（每个链表中邻接点按编号递增排列）：

$$1:2,3;\quad 2:1,4,5;\quad 3:1,6,7;\quad 4:2,7;\quad 5:2,6;\quad 6:3,5;\quad 7:3,4.$$

从顶点 1 出发分别进行 BFS 和 DFS，访问一个顶点后按邻接表次序考察其邻接点。写出 BFS 访问序列和 DFS 访问序列。



3. (单项选择题) 关于图的遍历，下列说法正确的是

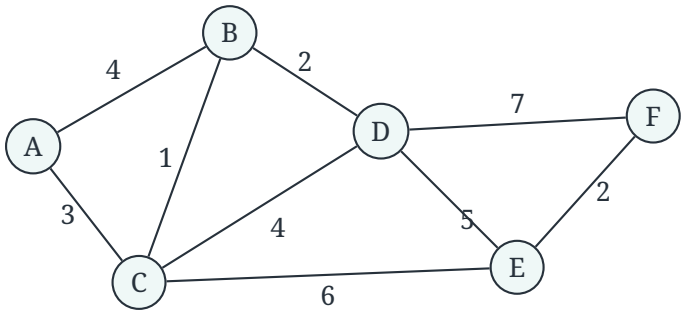
(A) 同一个图从同一顶点出发，进行 DFS 得到的访问序列是唯一的

(B) 对无向图进行 DFS 时，遇到的非树边一定连接某个顶点与它的祖先

(C) 对连通图进行 BFS, 所得 BFS 生成树中任意两顶点间的路径，就是原图中这两点间的最短路径

(D) 用邻接矩阵存储图时，BFS 的时间复杂度为 $O(n+e)$

4. (填空题) 带权无向图 G 如下图所示，边上的数字为权值。用 Prim 算法从顶点 A 开始构造最小生成树，每步选连接“树内”与“树外”的最小权边（本题每步的最小权边唯一）。写出依次加入的边、各步的权值以及最小生成树的总权值。

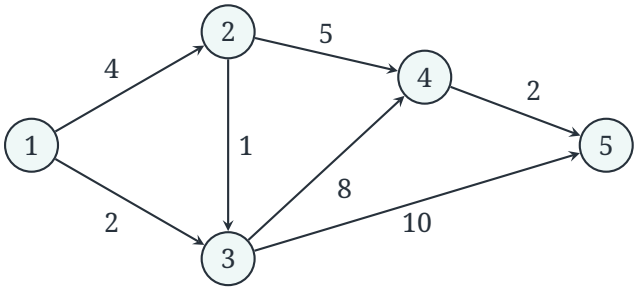


5. (填空题) 带权有向图 G 如下图所示，边上数字为权值，所有边权均为正。用 Dijkstra 算法求顶点 1 到其余各顶点的最短路径。

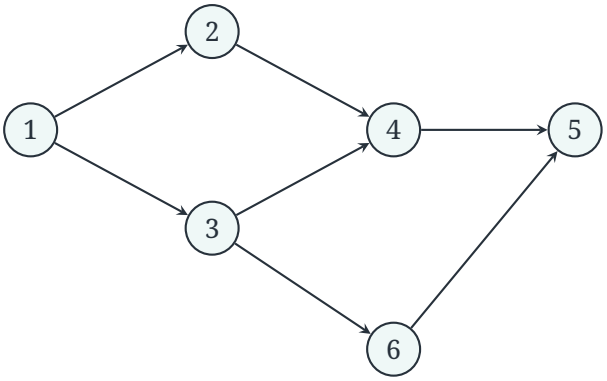
(1) 写出顶点被依次确定（加入集合 S）的次序；

(2) 写出算法结束时 d_2, d_3, d_4, d_5 的值；

(3) 写出从 1 到 5 的最短路径。



6. （单项选择题）有向图 G 如下图所示。下列序列中，不可能是 G 的拓扑序列的是

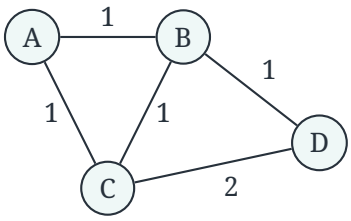


(A) $1,2,3,4,6,5$　(B) $1,3,2,6,4,5$

(C) $1,2,3,6,4,5$　(D) $1,3,2,4,5,6$

10.2 真题风格与综合训练

7. （真题风格・单项选择题）无向图 G 如下图所示，边上的数字为权值。关于 G 的最小生成树，下列说法正确的是



(A) 只有 1 棵，权值为 3　(B) 有 2 棵，权值均为 3

(C) 有 3 棵，权值均为 3　(D) 有 3 棵，权值不全相同

8. (真题风格・单项选择题) 下列关于最短路径算法的叙述中，正确的是

(A) 图中存在负权边时，Dijkstra 算法可能过早确定某个顶点，得到比真实最短距离大的结果

(B) 只要图中不存在负权回路，Dijkstra 算法求出的结果就一定正确

(C) Floyd 算法不能用于含有负权边的图

(D) 求每对顶点之间的最短路径，只能对每个顶点各调用一次 Dijkstra 算法

9. (综合应用题) 有向图采用邻接表存储，类型定义如下（MaxV 为最大顶点数）：

```
#define MaxV 100
typedef struct ArcNode {
    int adjvex;                 // 弧头顶点下标
    struct ArcNode *next;       // 下一条出边
} ArcNode;
typedef struct {
    char data;
    ArcNode *first;
} VNode;
typedef struct {
    VNode adjlist[MaxV];
    int n, e;
} ALGraph;
```

设计一个算法，判断 G 中是否存在回路。要求：

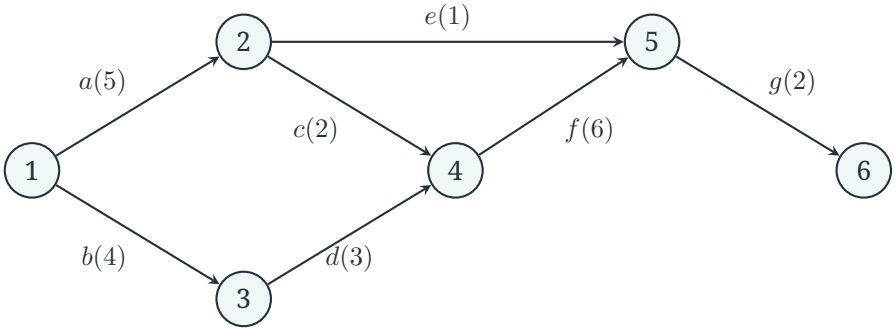
(1) 说明算法思路；

(2) 给出 C 语言实现；

(3) 分析时间复杂度；

(4) 说明若改用拓扑排序判定回路，复杂度是多少，两种做法在“能否给出一条具体回路”上有何差别。

10. （综合应用题）下图是一个 AOE 网，顶点 $1 \sim 6$ 表示事件，有向边表示活动，边上的字母为活动名、括号内为持续时间（单位：天）。设源点为 1，汇点为 6。



(1) 求各事件的最早发生时间 ve 和最迟发生时间 vl；

(2) 求各活动的最早开始时间 e、最迟开始时间 l 及时间余量 $l-e$，指出关键活动；

(3) 写出全部关键路径及工程工期；

(4) 若单独把活动 a 缩短 1 天，工期是否改变？若单独把活动 f 缩短 1 天呢？说明理由。

11 查找

在一百万条学籍记录里按学号找一个人，和在二十个人的通讯录里找一个名字，差别不只是快慢。前者要求你根本不能逐条翻看，后者随手扫一遍也能接受。查找这一章讨论的就是这件事：记录按什么方式摆放，给定一个关键字之后需要比较多少次才能定位；以及当记录还要频繁插入、删除时，这种摆放方式怎样维护。

11.1 从“比较几次”说起

先看一个最小的例子。把 $21, 35, 47, 58, 69$ 依次放进一个数组，从头逐个比较。找 47 要比较 3 次 $(21, 35, 47)$；找 60 要把五个元素都比一遍才能确认不在表中，比较 5 次。如果问“平均要比较几次”，就必须先说清楚对哪些查找对象平均：是对表中已有的五个元素等概率地找，还是把可能出现的任意关键字都算进来。前者叫查找成功，后者叫查找不成功，两个平均值通常不相等。

定义 11.1: 查找表、关键字与平均查找长度

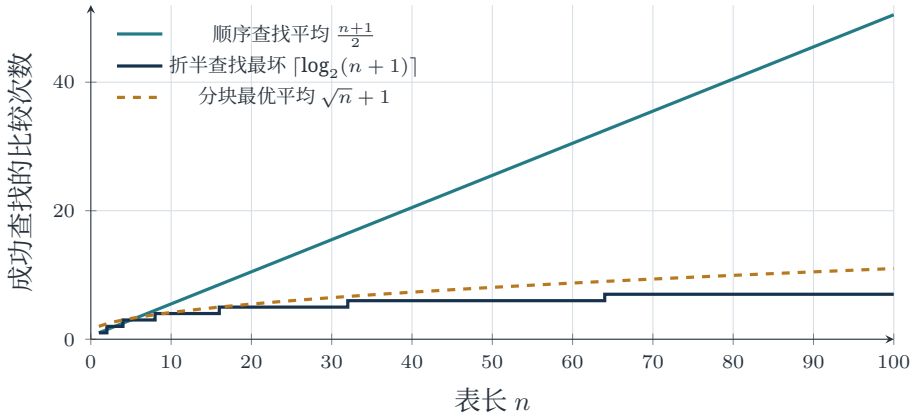
查找表是由同一类型记录构成的集合。只做查询、检索的称为静态查找表；还要支持插入、删除的称为动态查找表。记录中用来标识它的数据项叫关键字；能唯一标识记录的是主关键字，不能唯一标识的是次关键字。

平均查找长度（Average Search Length，ASL）是查找过程中关键字比较次数的期望：

$$\mathrm{ASL} = \sum_{i=1}^{n} P_i\, C_i,$$

其中 n 是查找对象的个数，P_i 是查找第 i 个对象的概率，C_i 是找到它所需的比较次数。

公式里的“查找对象”在成功和失败两种口径下不是同一批东西。等概率时，成功查找取 $P_i = 1/n$，分母是 n；失败查找要把所有可能落空的位置都算进去，分母往往不是 n，例如折半查找的判定树有 $n+1$ 个失败位置，散列表有 m 个散列地址。题目没有明说时，先看它给的是“成功”还是“不成功”，再确定分母。



图中横轴是表长，纵轴是成功查找的比较次数。顺序查找和分块查找按等概率下的平均值画，折半查找按最坏情况画（平均还要少约一次，但增长趋势相同）。顺序查找是一条直线，表长翻倍代价也翻倍；折半查找几乎贴着横轴，表长从 10 增到 100 也只多出三四次比较；分块查找取每块约 $\sqrt{n}$ 个元素时按 $\sqrt{n}$ 增长，正好夹在两者之间。三条曲线的形状，比记住三个具体数字更有用。

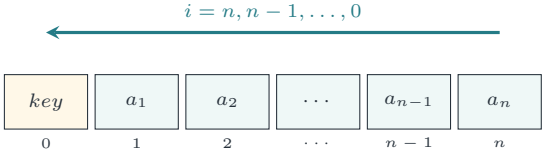
11.2 顺序查找：哨兵只是省掉一次边界判断

顺序查找没有任何前提：表可以无序，可以用数组，也可以用链表。从一端开始逐个比较，相等就返回位置，走完还没有相等就是失败。数组实现时，常把待查关键字先写进 0 号单元，让循环从 n 倒着走：

```c
typedef struct {
    ElemType *elem;      // elem[0] 留作哨兵
    int TableLen;
} SSTable;

int Search_Seq(SSTable ST, KeyType key) {
    ST.elem[0].key = key;                       // 哨兵
    int i;
    for (i = ST.TableLen; ST.elem[i].key != key; --i)
        ;                                       // 不必判断 i >= 1
    return i;                                   // 返回 0 表示失败
}
```

循环从 n 递减到 0，无论表里有没有这个关键字，$i = 0$ 时一定会与哨兵相等而停下。这样循环条件里少了一个下标越界的判断，但关键字比较的次数并没有减少，时间复杂度仍是 $O(n)$。



先写哨兵，再从尾部倒扫；
循环里只剩关键字比较。
返回 $i = 0$ 即查找失败。

图中 0 号单元存的是待查关键字本身，$1 \sim n$ 才是表内元素。扫描方向从右往左，最坏会一路走到 0 号单元；哨兵保证这个循环一定终止，因此不用再写 `i >= 1`。

成功查找时，第 i 个元素要比较 $n-i+1$ 次，等概率平均为

$$\mathrm{ASL}_{\text{成功}} = \frac{1}{n}\sum_{i=1}^{n}(n-i+1) = \frac{n+1}{2}.$$

失败查找时，从 a_n 一直比较到哨兵，共 $n+1$ 次；若不用哨兵，只比较 n 个元素就发现越界，代价是 n。两种写法只差一个常数，但考试常拿这个数字设选项，写答案时要看清代码里有没有哨兵。

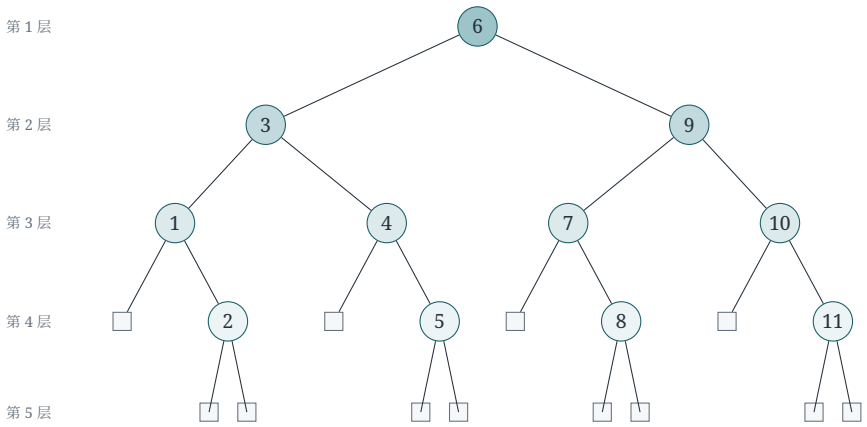
如果表已经按关键字有序，失败查找可以在第一个大于待查关键字的元素处提前停下。此时 $n+1$ 个失败区间的比较次数依次是 $1, 2, \ldots, n, n$，平均为 $\dfrac{n(n+3)}{2(n+1)}$；成功查找的平均次数仍是 $(n+1)/2$，因为每个元素的位置没有变。

11.3 折半查找：判定树把比较次数画出来

折半查找要求两条：表必须顺序存储，并且按关键字有序。顺序存储是为了 $O(1)$ 取到中间位置，链表找中点本身就要走 $O(n)$，所以链表不能直接折半。下面以递增表为例，递减表把两个分支对调即可。

```
int Binary_Search(SSTable ST, KeyType key) {
    int low = 1, high = ST.TableLen;
    while (low <= high) {
        int mid = low + (high - low) / 2;   // 等价于 (low+high)/2，且不溢出
        if (ST.elem[mid].key == key)
            return mid;
        else if (ST.elem[mid].key > key)
            high = mid - 1;
        else
            low = mid + 1;
    }
    return 0;                                   // low > high，查找失败
}
```

每比较一次，待查区间缩小一半。把每次选中的 `mid` 画成一个结点，就得到折半查找的判定树：圆形结点是表中真实存在的元素，方形结点是 `low > high` 时落入的空区间。n 个元素之间和两端一共有 $n+1$ 个空档，所以方形结点正好 $n+1$ 个。



$n=11$：成功结点 $1+2+4+4$，失败结点 $4+8=12$

图中圆形结点里的数字是数组下标，颜色越深层次越浅。第 $1,2,3,4$ 层分别有 $1,2,4,4$ 个成功结点；失败结点有 4 个挂在第 3 层结点下面、8 个挂在第 4 层结点下面。

成功查找的比较次数就是圆形结点所在层数，因此

$$\mathrm{ASL}_{\text{成功}}=\frac{1\cdot 1+2\cdot 2+4\cdot 3+4\cdot 4}{11}=\frac{33}{11}=3.$$

失败查找时，最后一个被比较的圆形结点是方形结点的父结点，按常见约定只数这些关键字比较，不再把空指针判断另算一次。于是上图中失败代价为 3 的有 4 个、为 4 的有 8 个：

$$\mathrm{ASL}_{\text{不成功}}=\frac{4\cdot 3+8\cdot 4}{4+8}=\frac{44}{12}=\frac{11}{3}\approx 3.67.$$

如果题目把“判断空指针”也算一次比较，就在每个失败代价上再加 1，得到 4 和 5，平均值变为 $56/12=14/3$。两种答案都有人用，关键是同一道题里口径要统一。

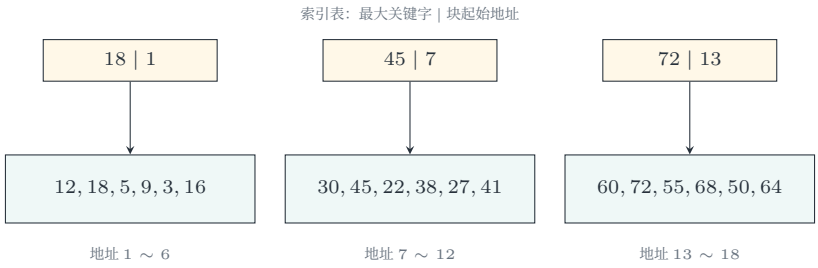
判定树的高度是 $\lceil\log_2(n+1)\rceil$，因为一棵高度为 h 的二叉树最多容纳 2^h-1 个内部结点。由此最坏情况比较 $\lceil\log_2(n+1)\rceil$ 次，平均约 $\log_2(n+1)-1$ 次（n 较大时），时间复杂度 $O(\log n)$。

注意

`mid` 的取整方式会改变判定树的形状。取 $\lfloor(\mathrm{low}+\mathrm{high})/2\rfloor$ 时，对偶数个元素，右子树结点数不少于左子树；取 $\lceil(\mathrm{low}+\mathrm{high})/2\rceil$ 时反过来。题目若给出了具体的比较过程或判定树，就按过程画，不要默认某一种取整；若只问“最坏比较次数”，两种取整得到的树高相同。

11.4　分块查找：索引表加块内顺序

折半查找快，但插入一个元素可能要移动半个表；顺序查找插入方便，查找又太慢。分块查找取中间路线：把记录按关键字分成若干块，块与块之间整体有序，块内可以无序；另外建一张索引表，每项记下该块的最大关键字和该块的起始地址。



块间有序：$18 < 22$，$45 < 50$；块内不要求有序

图中上方是索引表，每项存“该块最大关键字 + 块起始地址”；下方三个块内部是乱序的，但前一块的所有关键字都小于后一块。查 38 时先在索引表中找到 45 对应的第 2 块，再在块内顺序比较。

设表长 n，分成 b 块，每块 s 个元素，$n = bs$。索引表用顺序查找、块内也用顺序查找时，索引平均比较 $(b+1)/2$ 次，块内平均 $(s+1)/2$ 次，总代价

$$\mathrm{ASL} = \frac{b+1}{2} + \frac{s+1}{2} = \frac{1}{2}\left(\frac{n}{s} + s\right) + 1.$$

由均值不等式，$s = \sqrt{n}$ 时取最小值 $\sqrt{n}+1$，此时块数也约为 $\sqrt{n}$。若索引表改用折半查找，索引部分降为约 $\log_2(b+1) - 1$ 次，总代价约为

$$\mathrm{ASL} \approx \log_2(b+1) - 1 + \frac{s+1}{2}.$$

分块查找的插入、删除只影响一个块：块内重新排列，块太大就分裂，太小就与相邻块合并，索引表随之更新。它适合记录数较大、需要按块组织的索引顺序文件。

11.5 二叉排序树

折半查找每次把区间砍一半，但要求连续存储；如果记录还要频繁插入、删除，数组的搬移代价很高。二叉排序树（Binary Search Tree，BST）把“比一次、丢掉一半”的思路搬到链式结构上：左子树的所有关键字小于根，右子树的所有关键字大于根，左右子树本身也都是 BST。

408 默认 BST 中不出现重复关键字；若题设允许相等，通常约定统一放左子树或统一放右子树。例如依次插入 $45, 24, 53, 12, 37, 93$，先来的 45 做根；$24 < 45$ 往左，$53 > 45$ 往右；$12 < 24$ 挂在 24 左边，$37 > 24$ 挂在 24 右边，$93 > 53$ 挂在 53 右边。查找 37 的路径是 $45 \to 24 \to 37$，比较三次。

BST 有一个很实用的性质：对整棵树做中序遍历，得到的就是关键字的递增序列。判断一棵二叉树是不是 BST，最稳妥的办法就是检查中序序列是否严格递增；删除时能用前驱或后继替代，也是因为中序次序不能被打乱。

查找和插入可以写成同一套下降过程。查找走到空指针就失败；插入走到空指针就把新结点挂在那里，所以新结点一定是叶子。关键字已经存在时通常不重复插入。

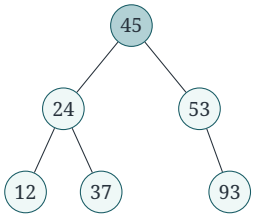
```
typedef struct BSTNode {
    KeyType key;
    struct BSTNode *lchild, *rchild;
} BSTNode, *BSTree;

BSTNode *BST_Search(BSTree T, KeyType key) {
    while (T != NULL && T->key != key)
        T = (key < T->key) ? T->lchild : T->rchild;
```

```
    return T;                          // NULL 表示失败
}

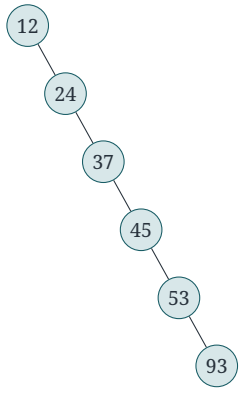
bool BST_Insert(BSTree *T, KeyType key) {
    if (*T == NULL) {
        *T = (BSTNode *)malloc(sizeof(BSTNode));
        (*T)->key = key;
        (*T)->lchild = (*T)->rchild = NULL;
        return true;
    }
    if (key == (*T)->key) return false;
    if (key < (*T)->key)
        return BST_Insert(&(*T)->lchild, key);
    return BST_Insert(&(*T)->rchild, key);
}
```

查找的代价是根到目标结点的路径长度。树越矮越快：最平衡时树高与折半判定树相同，为 $O(\log n)$；最坏时每个结点只有一个孩子，退化成单链表，$O(n)$。同样六个关键字，插入顺序不同，形态可以差很多。



插入 $45, 24, 53, 12, 37, 93$

$1+2+2+3+3+3=14$，$\text{ASL}=\dfrac{14}{6}=\dfrac{7}{3}$



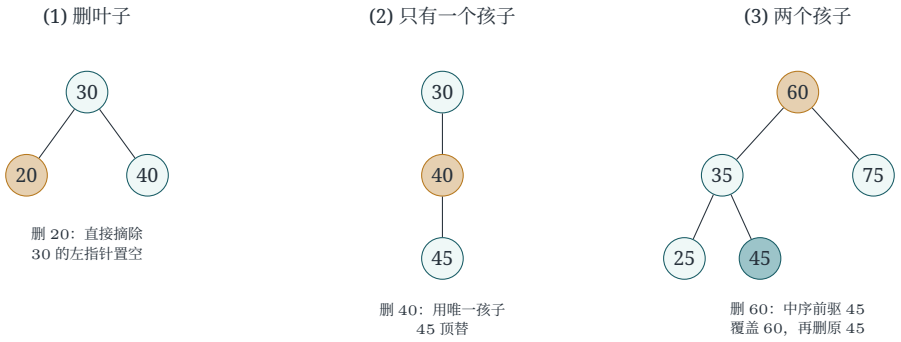
插入 $12, 24, 37, 45, 53, 93$

$1+2+3+4+5+6=21$，$\text{ASL}=\dfrac{21}{6}=\dfrac{7}{2}$

左边六个关键字分布得比较均匀，查找长度按层累加；右边按递增顺序插入，每个新结点都挂在最右下，整棵树退化成一条链。两棵树的结点完全相同，ASL 却从 $7/3$ 变成 $7/2$，最坏情况是 $O(n)$。

失败查找的比较次数是路径上实际比较过的内部结点个数，也就是对应外部失败结点的父结点层数，计数口径与折半查找相同；n 个结点一共有 $n+1$ 个失败位置。比如在左图里查一个小于 12 的数，会比较 $45, 24, 12$ 三次后落到 12 的左空指针，失败代价为 3。

删除要分三种情况。被删结点是叶子，直接摘掉；只有一个孩子，让孩子顶替它的位置；有两个孩子，则用中序前驱（左子树最右结点）或中序后继（右子树最左结点）的值覆盖它，再转而删除那个前驱或后继。前驱没有右孩子，后继没有左孩子，所以“有两个孩子”的删除一定退化成前两种情况。



三个面板对应删除的三种情况。第三种也可用中序后继 75 覆盖，得到的树形不同，但中序序列仍然是 $25, 35, 45, 75$，两种做法都合法，除非题目指定用前驱或后继。

11.6 AVL 树：把树高控制在 $O(\log n)$

BST 的查找代价完全取决于树高，而树高又取决于插入顺序。要让它稳定在 $O(\log n)$，就得在每次插入、删除后检查并修复失衡。AVL 树首先是一棵 BST，在此基础上再要求：任意结点的左右子树高度之差不超过 1。

平衡因子定义为

$$\mathrm{bf} = \text{左子树高度} - \text{右子树高度},$$

AVL 树中每个结点的 $\mathrm{bf} \in \{-1, 0, 1\}$。空树高度记 0，只有根的树高度记 1。

含 n 个结点的 AVL 树能有多高？反过来问更好算：高度为 h 的 AVL 树最少需要多少个结点。根占一个；为了让结点总数最少，左右子树高度取 $h-1$ 和 $h-2$（两者相差不超过 1），并且都取对应高度的最少结点形态，于是

$$N(h) = N(h-1) + N(h-2) + 1, \qquad N(0) = 0,\ N(1) = 1.$$

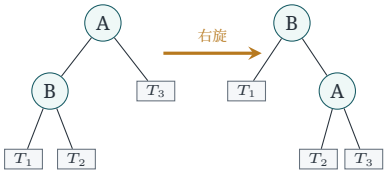
依次得到

h	1	2	3	4	5	6
$N(h)$	1	2	4	7	12	20

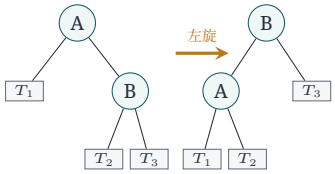
这个数列比斐波那契数列每项少 1（$N(h) = F_{h+2} - 1$），按指数增长，所以 $h = O(\log n)$，最坏约为 $1.44 \log_2(n+1)$。高度为 5 的 AVL 树至少要有 12 个结点，而不是满二叉树的 31 个。

插入后失衡只可能出现在从插入点往根走的路径上。从插入点向上找到第一个 $|\mathrm{bf}| = 2$ 的结点 A，看从 A 到新结点走的两步方向，一共四种情况：

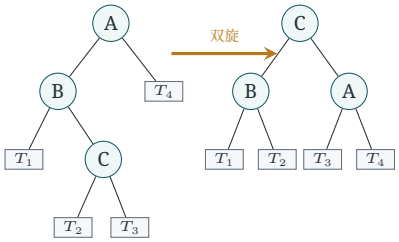
(a) LL：右单旋



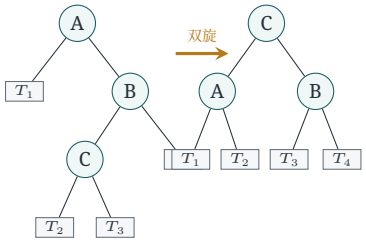
(b) RR：左单旋



(c) LR：先左旋 B，再右旋 A



(d) RL：先右旋 B，再左旋 A



每个面板左边是失衡时的局部形态，右边是调整后的结果。LL 和 RR 只用一次单旋，LR 和 RL 要两次：先转孩子，再转失衡结点。判断类型时从 A 沿插入方向走两步——“左左”是 LL，“左右”是 LR，依此类推。旋转不改变中序序列，只改父子关系。

以 LL 为例，右旋的代码只有三行指针操作：

```
AVLNode *RotateRight(AVLNode *p) {
    AVLNode *L = p->lchild;
    p->lchild = L->rchild;      // L 的右子树挂到 p 的左边
    L->rchild = p;
    return L;                   // L 成为新的子树根
}
```

左旋完全对称，把 `lchild` 与 `rchild` 对调即可。LR 先对 A 的左孩子左旋，再对 A 右旋；RL 先对 A 的右孩子右旋，再对 A 左旋。

插入最多做一次旋转（单旋或双旋）：调整后这棵子树的高度回到插入前的值，祖先的平衡因子不会继续变化。删除不一样，一次旋转可能让子树变矮，祖先可能接着失衡，所以删除最坏要沿路径做 $O(\log n)$ 次调整。

11.7 B-树：为磁盘访问设计的多路查找树

内存里的比较很快，磁盘的一次读写在毫秒级。二叉树每个结点只放一个关键字，10^6 条记录树高约 20，意味着二十次磁盘访问；如果把一个结点做成一个磁盘块，放几百个关键字、几百个孩子，树高会降到三四层。B-树就是这种“多路、平衡”的查找树。

定义 11.2: m 阶 B-树

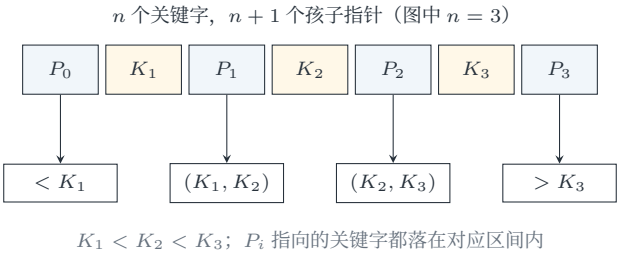
一棵 m 阶 B-树满足：

1. 每个结点最多有 m 棵子树、$m-1$ 个关键字；
2. 除根外，每个数据结点至少有 $\lceil m/2 \rceil - 1$ 个关键字；若它还有孩子，则至少有 $\lceil m/2 \rceil$ 棵子树；根若不是最底层，至少有 2 棵子树；

3. 结点内关键字递增排列，K_i 左侧子树的所有关键字小于 K_i，右侧子树的所有关键字大于 K_i；
4. 所有失败结点（最底层数据结点的空指针）都在同一层。

这里要先统一术语。有的教材把最底层存放关键字的结点叫“叶结点”，有的把它的空孩子叫“叶结点/失败结点”。本文把前者叫**数据结点**，后者叫**失败结点**；树高按数据结点的层数计，根为第 1 层，不计失败结点层。读题时先看它是否把失败结点也算一层，必要时换算。

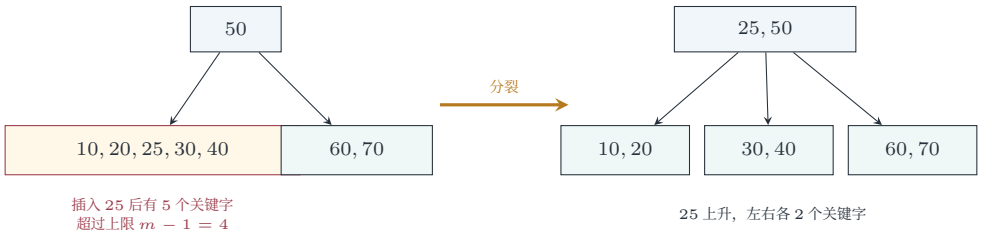
一个数据结点的内部布局是“指针、关键字、指针、关键字……指针”：n 个关键字把关键字空间切成 $n+1$ 段，正好对应 $n+1$ 个孩子指针。



图中关键字把整个关键字空间切成四段，每个指针负责一段。查找时先在结点内部确定落在哪一段（关键字有序，可以顺序比较，也可以折半），再沿对应指针下一层。结点内的比较在内存中完成，真正昂贵的是沿指针读下一层磁盘块。

查找。从根开始，在结点内找到第一个不小于待查关键字的位置：相等就成功；否则沿前一个指针下降。走到失败结点就说明不存在。磁盘访问次数等于树高，结点内比较次数不超过 m。

插入。新关键字总是先插入最底层数据结点。若插入后关键字数不超过 $m-1$，直接完成；若达到 m 个，就把结点分裂：取第 $\lceil m/2 \rceil$ 个关键字上升给父结点，左右两半各成一个结点（m 为偶数时取第 $m/2$ 或 $m/2+1$ 个都能保持合法，本文统一取前者，题目另行指定时按题目）。父结点可能因此也达到 m 个，继续分裂，最坏一直传到根；根分裂时产生新根，树高加 1。



左图是 $m=5$（最多 4 个关键字）时的一次溢出：插入 25 后左孩子有五个关键字。取第 $\lceil 5/2 \rceil = 3$ 个关键字 25 上升，左边留下 10, 20，右边留下 30, 40，父结点变成 25, 50。若父结点也满了，就对父结点重复同样的分裂。

删除。先找到关键字。若它不在最底层，用中序前驱或后继替换它，问题转成删除最底层的关键字。删除后若关键字数低于 $\lceil m/2 \rceil - 1$，就发生下溢：

- 兄弟结点够借（关键字数多于下限）：父结点中的分隔关键字下移，兄弟中相邻的关键字上移，相当于绕父结点转一下；
- 兄弟也只有下限个关键字：把当前结点、父结点中的分隔关键字、兄弟合并成一个结点。父结点少了一个关键字，可能继续下溢。

根结点只剩 0 个关键字时，删除根，让它的唯一孩子成为新根，树高减 1。

高度。n 个关键字对应 $n+1$ 个空指针（失败结点）。高度为 h 时，失败结点至少有 $2\lceil m/2\rceil^{h-1}$ 个，因此

$$n+1 \geq 2\left\lceil \frac{m}{2} \right\rceil^{h-1} \quad \Longrightarrow \quad h \leq 1+\log_{\lceil m/2 \rceil} \frac{n+1}{2}.$$

另一方面，高度为 h 的 m 阶 B-树最多有 m^h-1 个关键字，所以

$$h \geq \lceil \log_m(n+1) \rceil .$$

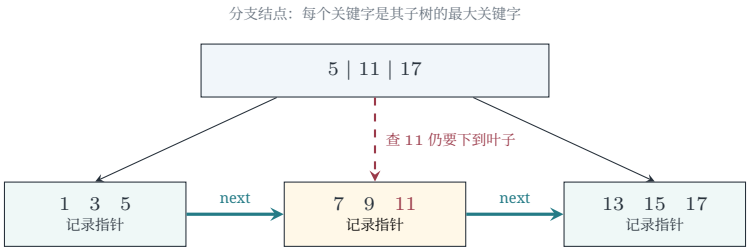
以 $m=3$、$n=7$ 为例，最矮可以是 2 层（最多 $3^2-1=8$ 个关键字），最高不超过 $1+\log_2 4=3$ 层。实际树高取决于插入、删除的历史，不一定取到这两个端点。

11.8 B+ 树：把索引和数据分开

B+ 树是 B-树为数据库和文件系统改造出来的变体。最大的变化是：分支结点只存索引，所有关键字和记录指针都放在叶结点；叶结点之间再用指针串成有序链表。

具体性质可以对照 B-树记：

- 每个分支结点最多有 m 棵子树；非叶根至少 2 棵，其他分支结点至少 $\lceil m/2 \rceil$ 棵；
- 分支结点的子树个数与关键字个数相等（B-树是关键字数比子树数少 1）；
- 分支结点中的关键字是各子树最大关键字的副本，只起分界作用；
- 所有关键字都出现在叶结点，叶结点存放记录指针（或记录本身），并按关键字顺序链接。



所有关键字都在叶结点；叶链支持范围扫描

图中分支结点的 $5, 11, 17$ 分别是三棵子树的最大关键字，它们只是索引副本，真正的记录指针在叶结点。查 11 时虽然根结点里已经出现 11，仍必须沿中间指针下到叶子才能拿到记录；查范围 $[9, 15]$ 时先定位到中间的叶子，再沿 `next` 指针扫到右边的叶子。

B+ 树和 B-树最容易混淆的几点，可以放在一起看：

B-树与 B+ 树的区别

	B-树	B+ 树
关键字与记录指针	每个结点都可能存记录指针	只有叶结点存记录指针
分支结点关键字数	子树数减 1	与子树数相等，是子树最大关键字的副本
查找结束位置	任意层命中即可结束	必须走到叶结点
叶结点	无链接	按关键字顺序链接
范围查询	需要中序遍历	定位左端点后沿叶链扫描

同样大的磁盘块，B+ 树的分支结点不存记录指针，能放下更多索引项，树更矮，磁盘 I/O 更少；叶链又让范围查询不需要回到上层。这就是数据库索引普遍采用 B+ 树的原因。B-树则在单点查找时可能在分支结点提前命中，结构更紧凑。

11.9 散列表

前面所有方法都靠比较关键字来缩小范围，散列则试图不比较：用一个函数把关键字直接算成地址。比如学号 20230117，令 $H(k) = k \bmod 1000$，地址就是 117。理想情况下一次就能定位，时间复杂度 $O(1)$。

问题是不同的关键字可能算出同一个地址，这叫冲突。例如 117 和 20231117 都会落到 117。散列技术要解决两件事：设计一个把关键字打散的散列函数；冲突发生时安排一个规则继续找位置。

散列函数。常见的有：

- 除留余数法：$H(k) = k \bmod p$，p 取不大于表长 m 的最大素数。它最简单，也最常考。不要把 p 取成 2 的幂，否则关键字的低位相同就会全部冲突。
- 直接定址法：$H(k) = ak + b$。关键字基本连续时不会冲突，但地址空间可能浪费很大。
- 平方取中法：把关键字平方，取中间几位。关键字的每一位都不够均匀时，平方后中间位通常更均匀。
- 数字分析法：关键字位数较多时，挑分布均匀的若干位作地址。
- 折叠法：把关键字分成等长几段，相加后取低位。

装填因子。记

$$\alpha = \frac{n}{m},$$

其中 n 是已存入的记录数，m 是散列表长度。α 越大，表越满，冲突越多。开放定址法必须 $\alpha < 1$；链地址法可以超过 1，但链会变长。

开放定址法

冲突时按一个探测序列找下一个空槽：

$$H_i = \big(H(key) + d_i\big) \bmod m, \qquad i = 0, 1, 2, \ldots$$

常用的 d_i 有三种：

- 线性探测：$d_i = i$，即依次看 $H, H+1, H+2, \ldots$。实现最简单，但会产生**聚集**：一段连续被占用后，任何落到这段里的关键字都要往后排，这段就越排越长。
- 二次探测：$d_i = 1^2, -1^2, 2^2, -2^2, \ldots$，即左右交替跳平方步。它能打散线性探测的连续聚集；若希望双向平方探测覆盖整张表，常取 m 为 $4k+3$ 的素数。只取单向 i^2 时通常覆盖不了全部位置，不能把“$4k+3$”无条件套到所有二次探测实现上。
- 双重散列：$d_i = i \cdot H_2(key)$，步长由第二个散列函数决定。$H_2(key)$ 要与 m 互质，否则探测序列可能只覆盖表的一部分。它打散效果最好。

插入 58：$H(58)=3$，探测 $3\to4\to5\to6\to7\to8\to9$



图中 $H(k)=k \bmod 11$，依次插入 $26,25,72,38,8,18,58$。前六个关键字各就各位，58 的本地址 3 已被占用，沿 $4,5,6,7,8,9$ 一路探测，直到 10 号槽为空才停下。上方数字是每个关键字的成功探测次数，下方数字是从该地址出发查找不存在关键字时的探测次数（含最后遇到的空槽）。$3\sim9$ 号槽连成一片，就是线性探测的聚集现象。

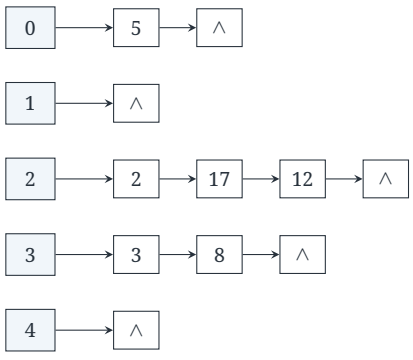
开放定址法有一个容易忽略的限制：删除不能直接把槽置空，否则会截断其他关键字的探测序列。比如把 26 从 4 号槽删掉，再查 58，探测到 4 号槽为空就会误判“不存在”，而 58 其实在 9 号槽。正确做法是留下删除标记（墓碑），查找时跳过它，插入时可以复用它。

链地址法

链地址法把所有散列到同一地址的记录挂成一条单链表，散列表的每个槽只存链表的头指针。

```c
typedef struct HashNode {
    KeyType key;
    struct HashNode *next;
} HashNode;

typedef struct {
    HashNode *table[MAXSIZE];
    int size;
} HashTable;
```



$H(k)=k \bmod 5$；新结点头插

图中 $5,12,3,8,17,2$ 依次插入 $m=5$ 的散列表。$12,17,2$ 都落在 2 号槽，按头插法形成 $2\to17\to12$；$3,8$ 形成 $3\to8$。空槽直接挂空指针。删除只需在链表里摘掉一个结点，不像开放定址法那样要留墓碑。

链地址法的优点是：不会产生开放定址法的“聚集”，删除简单，装填因子可以大于 1，表长也不必事先定得很准。缺点是结点需要额外的指针空间，且记录在内存中不连续，缓存局部性不如开放定址法。

ASL 的手算方法

散列表的 ASL 通常不套近似公式，而是对着表逐个数。

成功查找。对每个已存入的关键字，数它从本地址开始探测了几次（第一次散列也算一次），求和除以 n。前面线性探测的例子中，七个关键字的探测次数是

$$1, 1, 1, 1, 1, 1, 7,$$

所以

$$\mathrm{ASL}_{\text{成功}} = \frac{1+1+1+1+1+1+7}{7} = \frac{13}{7} \approx 1.86.$$

不成功查找。对每个可能的散列地址 j，假设要找一个散列到 j 但表中不存在的关键字，沿探测序列走到空槽为止，记下经过的槽数 c_j（包含最后那个空槽），再对 m 个地址求平均。该例中

$$c_0, \ldots, c_{10} = 1, 1, 1, 8, 7, 6, 5, 4, 3, 2, 1,$$

求和得 39，因此

$$\mathrm{ASL}_{\text{不成功}} = \frac{39}{11} \approx 3.55.$$

这里分母是表长 m，不是记录数 n。如果散列函数只可能产生其中 q 个地址（比如关键字都满足某个同余条件），分母才是 q。

链地址法的失败查找只需把对应链上的关键字全部比较一遍：空链比较 0 次，长度 L 的链比较 L 次。上例中五条链的长度为 $1, 0, 3, 2, 0$，故

$$\mathrm{ASL}_{\text{不成功}} = \frac{1+0+3+2+0}{5} = \frac{6}{5} = 1.2.$$

成功查找时，长度为 L 的链各位置贡献 $1+2+\cdots+L = L(L+1)/2$，与链内顺序无关，所以上例

$$\mathrm{ASL}_{\text{成功}} = \frac{1+6+3}{6} = \frac{5}{3} \approx 1.67.$$

如果题目把“走到空指针”也计一次比较，失败查找每个地址都要再加 1；先看清题干的计数口径。

在均匀散列假设下，还可以用装填因子估计平均代价：

$$\begin{aligned} &\text{线性探测：} && \mathrm{ASL}_{\text{成功}} \approx \frac{1}{2}\left(1+\frac{1}{1-\alpha}\right), && \mathrm{ASL}_{\text{不成功}} \approx \frac{1}{2}\left(1+\frac{1}{(1-\alpha)^2}\right), \\ &\text{链地址法：} && \mathrm{ASL}_{\text{成功}} \approx 1+\frac{\alpha}{2}, && \mathrm{ASL}_{\text{不成功}} \approx \alpha. \end{aligned}$$

这些是期望值，用来判断趋势；408 的计算题通常给出具体的表和探测规则，要求按表手算，不要直接把公式当答案。

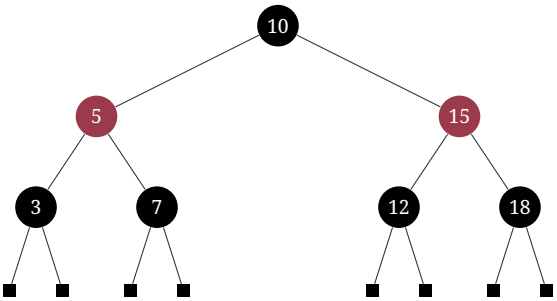
11.10 红黑树

AVL 树要求左右子树高度差不超过 1，条件很严，插入删除时旋转较多。红黑树换了一种约束：给结点染红或黑，用颜色规则保证“最长路径不超过最短路径的两倍”，平衡稍松，但维护代价更低。

一棵红黑树满足：

1. 每个结点是红色或黑色；
2. 根结点是黑色；
3. 每个叶结点（这里的叶结点指空指针 NIL）是黑色；
4. 红结点的两个孩子都是黑色，即不存在连续的红色结点；

5. 从任一结点到其所有后代 NIL 的路径上，黑色结点数目相同（称为黑高相等）。



根到任一 NIL 都经过 3 个黑结点（含 NIL）

图中黑结点为实心，红结点为深色。5 和 15 是红结点，它们的两个孩子都是黑结点；从根 10 出发到任意一个 NIL，黑色结点数都是 3。红色结点不能相邻，所以一条路径上红结点最多与黑结点一样多，最长路径不超过最短路径的两倍。

由这两条可以推出树高 $h \le 2\log_2(n+1)$。直观地说：黑高为 b 的子树至少有 2^b-1 个结点；红结点不能连续，所以根到叶子的路径上红结点数不超过黑结点数，树高不超过 $2b$。查找、插入、删除都是 $O(\log n)$。

插入时先把新结点染红，这样不会破坏黑高。若父结点是黑色，直接完成；若父结点是红色，看叔叔结点的颜色：

- 叔叔为红：父、叔变黑，祖父变红，把可能的问题上移到祖父继续检查（若祖父就是根，最后再把它染黑）；
- 叔叔为黑或不存在：按 LL/RR/LR/RL 做旋转，再重新染色。

一次插入最多旋转两次。删除的修复更繁琐，但同样是 $O(\log n)$。

和 AVL 树相比：AVL 更严格，树更矮，查找略快，但插入删除旋转更多；红黑树平衡稍松，旋转少，适合插入删除频繁的场景。C++ 的 `map/set` 和 Java 的 `TreeMap` 底层多用红黑树。

11.11 几种查找结构放在一起看

查找结构的横向比较

结构	前提/存储	成功查找代价	插入、删除与适用场景
顺序查找	任意表，顺序或链式	$\frac{n+1}{2}$，$O(n)$	插入 $O(1)$（表尾），适合小表、无序表
折半查找	有序，顺序存储	最坏 $\lceil \log_2(n+1) \rceil$，平均约 $\log_2(n+1)-1$	插入删除 $O(n)$，适合静态表
分块查找	块间有序、块内任意，另建索引	$\approx \log_2(b+1)-1+\frac{s+1}{2}$ 或 $\frac{b+s+2}{2}$	块内调整，适合索引顺序文件
二叉排序树	无额外要求	平均 $O(\log n)$，最坏 $O(n)$	插入删除 $O(h)$，形态依赖插入顺序
AVL 树	BST，$\lvert bf \rvert \le 1$	$O(\log n)$，最坏约 $1.44\log_2(n+1)$	插入至多一次旋转，删除可能多次；查多改少
B-树	外存，m 阶多路平衡	$O(\log_m n)$ 次结点访问	分裂、借/合；磁盘索引
B+ 树	外存，叶结点链接	$O(\log_m n)$，范围查询沿叶链	数据库、文件系统索引
散列表	选好散列函数与冲突处理	平均 $O(1+\alpha)$，最坏 $O(n)$	平均 $O(1)$；不支持顺序/范围查找

选择哪一种，先问三个问题：表是静态的还是频繁改动的；数据在内存还是磁盘；只需要等值查找，还是还要按关键字范围扫描。静态且有序，折半最省事；内存中频繁插入删除，用 AVL 或红黑树；数据在外存、要范围查询，用 B+ 树；只做等值查找且不要求顺序，散列表通常最快。

12 习题

12.1 基础过关题

1. （单项选择题）对含 12 个互异关键字的有序表进行折半查找，取 $\texttt{mid} = \lfloor(\texttt{low} + \texttt{high})/2\rfloor$。判定树中根所在层为第 1 层；查找失败时，失败结点的查找长度按其父结点所在层计算，不再额外计算空指针判断。则查找成功与查找不成功的平均查找长度分别为

(A) $\frac{37}{12}$，$\frac{49}{13}$　(B) $\frac{37}{12}$，$\frac{51}{13}$

(C) 3，4　(D) $\frac{39}{12}$，$\frac{49}{13}$

2. （单项选择题）长度为 100 的顺序表存放在下标 $1 \sim 100$ 中，下标 0 处存放待查关键字作为哨兵。查找每个元素的概率相等，且比较次数包含与哨兵的比较。则查找成功时的平均查找长度和查找失败时的比较次数分别为

(A) 50，101　(B) 50.5，101

(C) 50.5，100　(D) 50，100

3. （填空题）从空树开始，依次插入关键字

$$50,\ 30,\ 70,\ 20,\ 40,\ 60,\ 80,\ 35,\ 45,\ 65$$

构造一棵二叉排序树。等概率下查找成功的平均查找长度为 _____。

4. （单项选择题）从空 AVL 树开始，依次插入 $50, 20, 10, 30, 40$。插入 40 后，最小不平衡子树的根结点以及应做的旋转是

(A) 50，LR 型　(B) 20，LR 型

(C) 50，RL 型　(D) 20，RL 型

5. （单项选择题）从空树开始，向一棵 3 阶 B-树依次插入关键字 $1,2,3,4,5,6,7$。分裂时取 $\lceil m/2 \rceil$ 处的关键字上移。最终根结点中的关键字及树的高度（只计存放关键字的结点层数）为

(A) 根为 $[4]$，高度为 3　(B) 根为 $[2,4]$，高度为 3

(C) 根为 $[2,4,6]$，高度为 3　(D) 根为 $[4]$，高度为 2

6. （单项选择题）散列表长 $m=13$，散列函数 $H(k)=k \bmod 13$，采用线性探测法处理冲突（探测序列为 $d_i=0,1,2,\ldots$）。依次插入关键字 $25,41,18,32,56,70,83$，等概率下查找成功的平均查找长度为

(A) $\frac{12}{7}$　(B) $\frac{13}{7}$

(C) $\frac{15}{7}$　(D) 2

12.2 真题风格与综合训练

7. （真题风格・选择题）下列关于 B+ 树的叙述中，正确的是

(A) B+ 树的非叶结点既存放索引关键字，也存放指向记录的指针

(B) B+ 树的所有关键字都出现在叶结点中，叶结点按键值大小链接成有序链表

(C) B+ 树查找时，只要在非叶结点匹配到关键字就可以结束，不必到达叶结点

(D) B+ 树的每个非叶结点至少含 m 个关键字（m 为树的阶）

8. （真题风格・选择题）规定空树高度为 0，只有根结点的树高度为 1。一棵高度为 5 的 AVL 树最少含有多少个结点?

(A) 12　(B) 15

(C) 20　(D) 31

9. （真题风格・综合题）散列表长 $m = 11$，散列函数 $H(k) = k \bmod 11$，采用链地址法处理冲突，新结点插入到对应单链表的表头。依次插入关键字

$$19,\ 14,\ 23,\ 1,\ 68,\ 20,\ 84,\ 27,\ 55,\ 11.$$

(1) 画出最终的散列表（写出每条链上的次序）；

(2) 求等概率下查找成功的平均查找长度 $\text{ASL}_{\text{成功}}$；

(3) 求查找不成功的平均查找长度 $\text{ASL}_{\text{不成功}}$ 及装填因子 α。不成功查找只统计与表中关键字的比较次数，空链计 0 次。

10. （真题风格・综合题）设二叉树采用二叉链表存储，每个结点的关键字为整数且互不相同。设计算法判断该二叉树是否为二叉排序树。要求：

(1) 说明算法的基本设计思想；

(2) 用 C 语言实现算法；

(3) 分析时间复杂度和空间复杂度；

(4) 说明“只检查每个结点与它的左右孩子的大小关系”为什么不够，并给出一个反例。

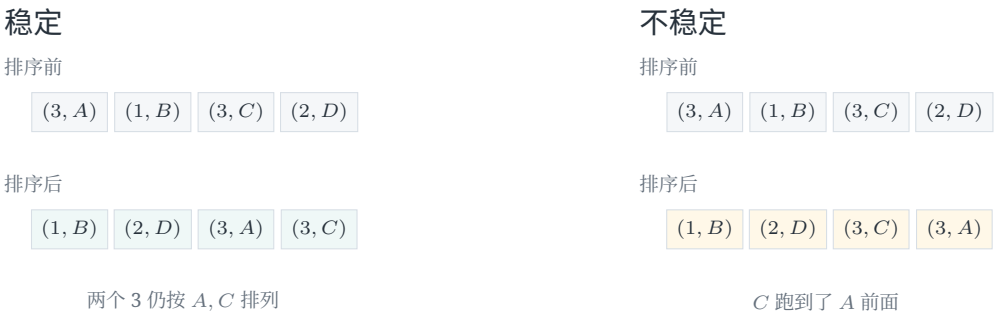
13 排序

把 n 个记录按关键字排成非递减序列，看起来只是比较和交换，实际要同时考虑三件事：比较多少次、搬动多少数据、关键字相同的记录会不会被拆散。以考生成绩表为例，总分相同的两个人原本按准考证号排列；如果排序后他们的先后次序变了，名次就会跟着变，这正是稳定性要解决的问题。

设待排序的记录为 $R_1, R_2, \ldots, R_n$，对应的关键字为 $k_1, k_2, \ldots, k_n$。排序就是确定一个排列 $p(1), p(2), \ldots, p(n)$，使

$$k_{p(1)} \leqslant k_{p(2)} \leqslant \cdots \leqslant k_{p(n)}.$$

如果 $k_i = k_j$ 且 $i < j$，排序后 R_i 仍然排在 R_j 前面，就称这个排序算法是**稳定的**；否则称它是**不稳定的**。注意稳定性只对关键字相等的记录有意义，关键字各不相同时无所谓稳定与否。



图里每个二元组是“关键字, 记录编号”。左右两边的输入完全一样，区别只在两个关键字为 3 的记录 A、C 排序后的相对次序：左边保持 A 在 C 前，右边把 C 放到了 A 前。实际应用中，若原序列本身按某个次要关键字有序，稳定排序可以保住这个次序，不必再排一次。

按数据是否全部放进内存，排序分为**内部排序**和**外部排序**。内部排序的全部记录都在内存中，本章前面的插入、交换、选择、归并、基数排序都属于这一类；外部排序处理的是内存装不下的大文件，排序过程中要反复读写外存，做法一般是“先在内存中生成若干有序的初始归并段，再反复归并”。评价一个内部排序算法，主要看时间复杂度（比较次数与移动次数之和）、辅助空间和稳定性；外部排序还要看读写外存的次数，因为磁盘 I/O 通常比内存中的比较慢几个数量级。

13.1 插入排序

插入排序维护一个已经有序的前缀，每次把后面的一个记录插进这个前缀。它和整理扑克牌一样：左手是排好的牌，右手摸到新牌后，从右往左找位置插进去，后面的牌依次让位。

这里有一个贯穿始终的不变式：进入第 i 趟时，前缀 $A[0..i-1]$ 已经有序；把 $A[i]$ 插进去后，前缀扩大到 $A[0..i]$，仍然有序。从 $i = 1$ 做到 $i = n - 1$，整个数组就排好了。插入排序是稳定的、原地的，在序列基本有序时非常快；代价是元素的后移，最坏情况下每趟都要搬动整个前缀。

直接插入排序

以 $\{49, 38, 65, 97, 76, 13, 27, 49'\}$ 为例，带撇的 $49'$ 表示关键字同为 49 的另一个记录。初始时把第一个元素 49 看作已排好的前缀，随后依次插入 38、65、97、76、13、27、49'：

$$
\begin{aligned}
& 49 \mid 38\ 65\ 97\ 76\ 13\ 27\ 49' \\
\rightarrow\ & 38\ 49 \mid 65\ 97\ 76\ 13\ 27\ 49' \\
\rightarrow\ & 38\ 49\ 65 \mid 97\ 76\ 13\ 27\ 49' \\
\rightarrow\ & 38\ 49\ 65\ 97 \mid 76\ 13\ 27\ 49' \\
\rightarrow\ & 38\ 49\ 65\ 76\ 97 \mid 13\ 27\ 49' \\
\rightarrow\ & 13\ 38\ 49\ 65\ 76\ 97 \mid 27\ 49' \\
\rightarrow\ & 13\ 27\ 38\ 49\ 65\ 76\ 97 \mid 49' \\
\rightarrow\ & 13\ 27\ 38\ 49\ 49'\ 65\ 76\ 97.
\end{aligned}
$$

竖线左边是已排序前缀。插入 76 时，97 后移一格；插入 13 时，前缀里的元素全部后移；插入 49' 时，它停在 49 的后面，因为循环条件用的是严格大于。

```
void InsertSort(int A[], int n) {
    for (int i = 1; i < n; i++) {
        int temp = A[i], j = i - 1;
        while (j >= 0 && A[j] > temp) {   // 严格大于才后移
            A[j + 1] = A[j];
            j--;
        }
        A[j + 1] = temp;
    }
}
```

初始	49	38	65	97	76	13	27	49′
$i=1$	38	49	65	97	76	13	27	49′
$i=2$	38	49	65	97	76	13	27	49′
$i=3$	38	49	65	97	76	13	27	49′
$i=4$	38	49	65	76	97	13	27	49′
$i=5$	13	38	49	65	76	97	27	49′
$i=6$	13	27	38	49	65	76	97	49′
$i=7$	13	27	38	49	49′	65	76	97

已排序前缀　本趟插入的记录

图里每一行是插入 $A[i]$ 之后的数组状态，浅色格子是已排序前缀，深色格子是刚插入的记录。可以看到插入排序只做“后移”和“填回”两种动作，前缀之外的元素从不被访问。

直接插入排序的比较次数和初始序列关系很大。最好情况是初始已经有序：每趟只比较一次就发现不用后移，共 $n-1$ 次比较，移动 0 次，时间 $O(n)$。最坏情况是初始逆序：第 i 趟要比较 i 次，总比较次数

$$
\sum_{i=1}^{n-1} i = \frac{n(n-1)}{2},
$$

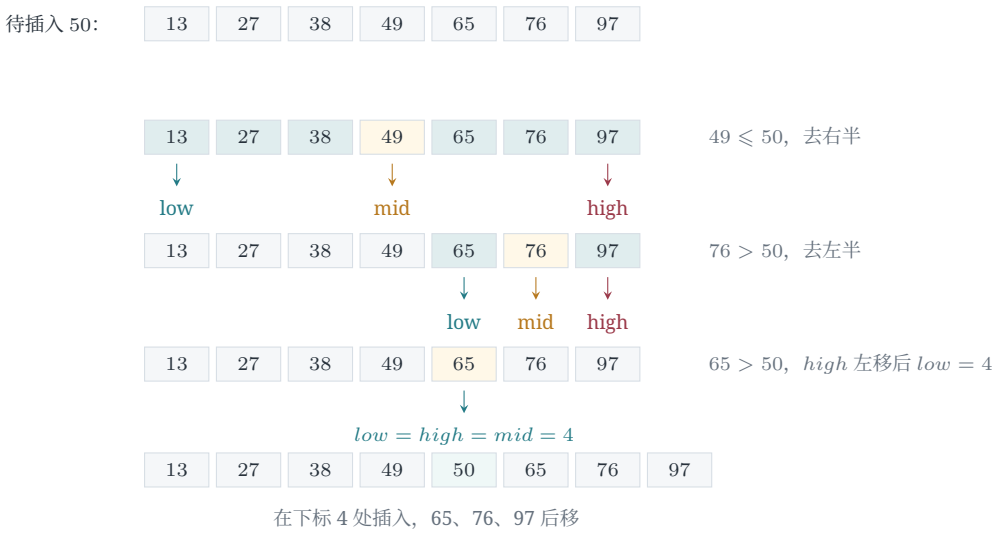
后移次数也是 $\frac{n(n-1)}{2}$；若把每次暂存 `temp` 和最后回填也算作移动，还要再加 $2(n-1)$ 次。平均情况下比较和后移都约为 $\frac{n^2}{4}$，时间复杂度仍是 $O(n^2)$。辅助空间只有一个 `temp`，为 $O(1)$。循环条件是 `A[j] > temp`，等于时不后移，所以直接插入排序稳定。

若数组从下标 1 开始，常见写法是把 $A[0]$ 当作**哨兵**：把待插入记录本身存进 $A[0]$，内层循环就可以省掉 `j >= 0` 的判断，因为比较到下标 0 时 `A[0] > temp` 为假，循环自然停止。408 题里看到 $A[0]$ 不参与排序结果，就是这种带哨兵的写法；此时元素存放在 $A[1..n]$，有效长度是 n。

折半插入排序

折半插入排序只改“找位置”这一步。既然前缀 $A[0..i-1]$ 已经有序，插入 $A[i]$ 时可以用二分查找确定它应该落在哪里，而不必从后往前逐个比较。

```
void BinInsertSort(int A[], int n) {
    for (int i = 1; i < n; i++) {
        int temp = A[i];
        int low = 0, high = i - 1;
        while (low <= high) {                // 在前缀中折半查找
            int mid = (low + high) / 2;
            if (A[mid] > temp) high = mid - 1;
            else               low  = mid + 1;  // 相等时插到后面
        }
        for (int j = i - 1; j >= low; j--)   // 统一后移
            A[j + 1] = A[j];
        A[low] = temp;
    }
}
```



在下标 4 处插入，65、76、97 后移

图里把一次折半查找拆成三步：区间从 $[0,6]$ 缩到 $[4,6]$，再缩到 $[4,4]$，最后 low 停在 4。待插入的 50 落在 49 和 65 之间，后面的 65、76、97 仍要逐个后移一格。每一行中浅蓝格子是当前查找区间，深色格子是这个区间的中点；比较完中点后，区间缩到左半或右半，下一行画的就是缩小后的区间。

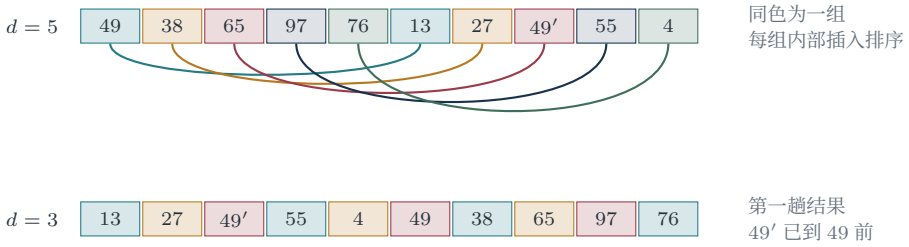
折半查找每趟最多比较 $\lceil \log_2(i+1) \rceil$ 次，n 趟累计的比较次数为 $\Theta(n \log n)$，比直接插入排序的 $\Theta(n^2)$ 少。但找到位置之后，后移的元素个数没有变：前缀中大于 `temp` 的元素仍然要全部后移。所以移动次数的最坏情况仍是 $\Theta(n^2)$，总时间复杂度还是 $O(n^2)$，只是常数中的比较部分小

了。折半插入排序的循环在 `A[mid] == temp` 时令 `low = mid + 1`，把相等记录插在后面，因此它也是稳定的。它适合比较操作很贵、移动相对便宜的场合；408 若问“折半插入减少了什么”，答案只是比较次数。

希尔排序

希尔排序（缩小增量排序）换了一个思路：先让相隔较远的元素大致归位，使序列变得“基本有序”，最后再做一次直接插入排序时，后移的距离就会短很多。

具体做法是取一个增量 d，把下标相差 d 的记录看成一组，例如 $A[i], A[i+d], A[i+2d], \ldots$ 为一组；对每组分别做直接插入排序。然后缩小 d，重复上述过程，直到 $d = 1$，此时整个序列做一次普通的直接插入排序。注意分组不是把数组切成连续的三段，而是按固定间隔“跳着”取元素。



图的上排是初始序列按 $d = 5$ 分组：下标 $0, 5$ 一组，$1, 6$ 一组，依此类推，弧线连接同一组的元素；下排是 $d = 5$ 这一趟排完后的序列，再按 $d = 3$ 重新分组。分组会随增量改变，同一个记录在不同趟可能落到不同的组里。

仍以 $\{49, 38, 65, 97, 76, 13, 27, 49', 55, 4\}$ 为例。$d = 5$ 时五组分别是 $(49, 13)$、$(38, 27)$、$(65, 49')$、$(97, 55)$、$(76, 4)$，组内排序后得到

$$\{13, 27, 49', 55, 4, 49, 38, 65, 97, 76\}.$$

接着取 $d = 3$，三组为 $(13, 55, 38, 76)$、$(27, 4, 65)$、$(49', 49, 97)$，排完得到

$$\{13, 4, 49', 38, 27, 49, 55, 65, 97, 76\}.$$

最后 $d = 1$ 做一次直接插入排序，得到

$$\{4, 13, 27, 38, 49', 49, 55, 65, 76, 97\}.$$

原来 49 排在 49’ 前面，最终 49’ 却跑到了 49 前面。原因是 $d = 5$ 时 49 和 49’ 被分进了不同的组，排序后它们的相对次序就变了；后面的趟都是稳定排序，无法把这个次序改回来。所以希尔排序**不稳定**。

```
void ShellSort(int A[], int n) {
    for (int d = n / 2; d >= 1; d /= 2) {   // 增量每次减半
        for (int i = d; i < n; i++) {
            int temp = A[i], j = i - d;
            while (j >= 0 && A[j] > temp) { // 组内直接插入
                A[j + d] = A[j];
                j -= d;
            }
            A[j + d] = temp;
        }
    }
```

```
}
```

希尔排序的时间复杂度取决于增量序列，不能用一个统一的式子写完。取初始增量 $d=\lfloor n/2\rfloor$、之后每次减半直到 1 时，最坏情况是 $O(n^2)$；改用 Hibbard 增量 $1,3,7,\dots,2^k-1$，最坏可降到 $O(n^{3/2})$；Sedgewick 增量还能更好。平均情况下它通常明显快于直接插入排序，经验上常被写成 $O(n^{1.3})$ 左右，但这不是对所有增量序列都成立的严格结论。408 一般只要求会按给定增量手工模拟，并记住“依赖增量、通常优于 $O(n^2)$、最坏仍可能到 $O(n^2)$、不稳定、空间 $O(1)$”。

13.2 交换排序

交换排序通过交换逆序的元素对来逐步消除逆序。冒泡排序只交换相邻元素，快速排序则把元素交换到离枢轴较远的位置，因此两者的常数和适用场景差别很大。

冒泡排序

冒泡排序每趟从前往后扫描，比较相邻的两个元素，逆序就交换。一趟下来，当前未排序部分的最大元素会被一路“冒”到最后。下一趟的扫描范围就可以缩短一格。

以 $\{6,2,8,4,9,1,7\}$ 为例：

$$
\begin{aligned}
&6\ 2\ 8\ 4\ 9\ 1\ 7\\
\to\ &2\ 6\ 4\ 8\ 1\ 7\ |\ 9\\
\to\ &2\ 4\ 6\ 1\ 7\ |\ 8\ 9\\
\to\ &2\ 4\ 1\ 6\ |\ 7\ 8\ 9\\
\to\ &2\ 1\ 4\ |\ 6\ 7\ 8\ 9\\
\to\ &1\ 2\ 4\ 6\ 7\ 8\ 9.
\end{aligned}
$$

第一趟把 9 送到末尾，第二趟把 8 送到倒数第二格，依此类推。第五趟结束后整个序列已经有序，第六趟扫描时没有发生任何交换，算法就可以提前结束。

```
void BubbleSort(int A[], int n) {
    for (int i = 0; i < n - 1; i++) {
        bool swapped = false;
        for (int j = 0; j < n - 1 - i; j++) {
            if (A[j] > A[j + 1]) {        // 严格大于才交换
                int t = A[j]; A[j] = A[j + 1]; A[j + 1] = t;
                swapped = true;
            }
        }
        if (!swapped) break;              // 本趟无交换，已有序
    }
}
```

初始	6	2	8	4	9	1	7
第 1 趟	2	6	4	8	1	7	9
第 2 趟	2	4	6	1	7	8	9
第 3 趟	2	4	1	6	7	8	9
第 4 趟	2	1	4	6	7	8	9
第 5 趟	1	2	4	6	7	8	9

9 就位

8 就位

7 就位

6 就位

第 6 趟无交换
提前结束

图里右侧深色格子是每趟结束后已经落到最终位置的元素。每趟只保证一个最大值就位，前面部分可能仍然乱；第五趟结束后整个序列已经有序，但算法还要再扫一趟确认没有交换才能停。

冒泡排序最好情况是初始有序：第一趟比较 $n-1$ 次、不交换，直接结束，时间 $O(n)$。最坏情况是初始逆序：比较次数为

$$\sum_{i=1}^{n-1}(n-i)=\frac{n(n-1)}{2},$$

每次比较都交换，一次交换要 3 次赋值，移动次数为 $\frac{3n(n-1)}{2}$，时间 $O(n^2)$。平均也是 $O(n^2)$，辅助空间 $O(1)$。

冒泡排序的稳定性由交换条件决定。写 `A[j] > A[j+1]` 时，相等元素不会交换，相对次序不变；若写成 `>=`，相等的两个元素也会交换，排序就不稳定了。这一点和直接插入排序的 `>` 是同一个道理。

快速排序

快速排序把排序拆成“划分”和“递归”两步。划分选一个枢轴（pivot），把序列重新排列成“左边都不大于枢轴，右边都不小于枢轴”，枢轴本身落到最终位置；然后对左右两段分别递归。

以 $\{49,38,65,97,76,13,27,49'\}$ 为例，取第一个元素 49 为枢轴。先从右往左找第一个小于 49 的 27，把它填到左端的坑里；再从左往右找第一个大于 49 的 65，填到右端的坑里；继续右找小、左找大，直到两个指针相遇，最后把 49 放进相遇处的坑。一趟划分后序列变成

$$\{27,38,13\}\ 49\ \{76,97,65,49'\},$$

49 落在下标 3，左边三个元素都小于它，右边四个元素都不小于它。注意左右两段内部还没有排好，只是被分到了枢轴两边。

```
int Partition(int A[], int low, int high) {
    int pivot = A[low];                    // 取第一个元素为枢轴
    while (low < high) {
        while (low < high && A[high] >= pivot) high--;
        A[low] = A[high];                  // 右边的小数填左坑
        while (low < high && A[low] <= pivot) low++;
        A[high] = A[low];                  // 左边的大数填右坑
    }
    A[low] = pivot;                        // 枢轴落位
    return low;
}
```

初始	49	38	65	97	76	13	27	49′	pivot= 49，low= 0，high= 7
右找小	27	38	65	97	76	13	坑	49′	$A[6]=27<49$，填入下标 0
左找大	27	38	坑	97	76	13	65	49′	$A[2]=65>49$，填入下标 6
右找小	27	38	13	97	76	坑	65	49′	$A[5]=13<49$，填入下标 2
左找大	27	38	13	坑	76	97	65	49′	$A[3]=97>49$，填入下标 5
相遇	27	38	13	49	76	97	65	49′	low=high= 3，49 落位

图里“坑”表示这个位置的值已经被移到别处，等待别处的元素填回来。每一行只移动一个

元素，指针向中间收缩；最后两个指针相遇的位置就是枢轴的最终位置。整个划分过程中，49 只在一开始被取出，最后才放回，中间一直以它的值参与比较，但不再被搬动。

```
void QuickSort(int A[], int low, int high) {
    if (low < high) {
        int p = Partition(A, low, high);
        QuickSort(A, low, p - 1);
        QuickSort(A, p + 1, high);
    }
}
```

划分过程从两端向中间扫描，每趟划分对长度为 m 的区间做 $O(m)$ 次比较和移动，因此总时间取决于递归树的样子。如果每次枢轴都落在中间，递归树近似平衡：

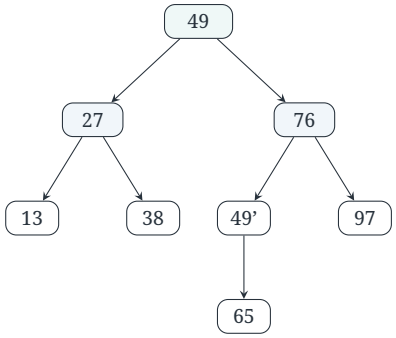
$$T(n) = 2T(n/2) + O(n) \quad \Longrightarrow \quad T(n) = O(n \log n).$$

如果每次枢轴都是当前区间的最小或最大值，例如初始序列已经有序、又固定取第一个元素作枢轴，那么每次划分只减少一个元素：

$$T(n) = T(n-1) + O(n) \quad \Longrightarrow \quad T(n) = O(n^2).$$

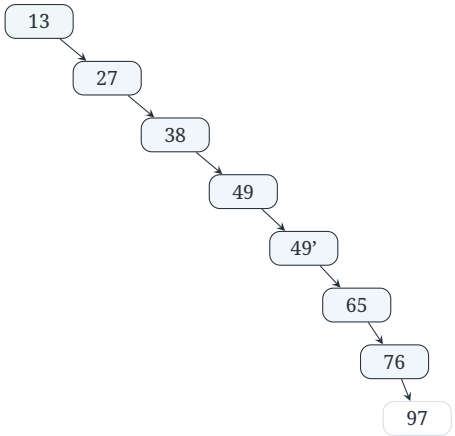
平均情况下，枢轴大致落在中间附近，可以证明 $T(n) = O(n \log n)$。

划分均匀



每层总代价 $O(n)$
深度 $O(\log n)$

划分极不均衡



每次只少一个元素
深度 $n-1$，时间 $O(n^2)$

图里每个结点是一次划分选出的枢轴。左边枢轴接近中点，树高约 $\log_2 n$，每层所有子区间的长度和不超过 n，所以每层代价 $O(n)$；右边初始有序时每次枢轴都是区间最小值，树退化成一条链，高度 $n-1$，总代价 $O(n^2)$。这就是“快排最怕初始基本有序”的原因。

快速排序的最好和平均时间复杂度为 $O(n \log n)$，最坏为 $O(n^2)$。空间开销来自递归工作栈：划分均匀时栈深 $O(\log n)$，最坏时栈深 $O(n)$，平均 $O(\log n)$。快速排序不稳定，因为划分时的填坑/交换是跳跃式的。一个最小的反例是 $\{5, 5', 2, 6, 1\}$，取首元素 5 为枢轴，一趟划分后得到 $\{1, 5', 2, 5, 6\}$，两个 5 的次序已经反了。

实际实现中常用随机选枢轴或“三数取中”来避免最坏情况；408 题目若没有特别说明，一般按“取第一个元素”处理。还有一点容易混：有的题把一次划分叫“一趟”，有的题把递归树的一层叫“一趟”，两种口径的趟数不同，做题时先看题干怎么定义。

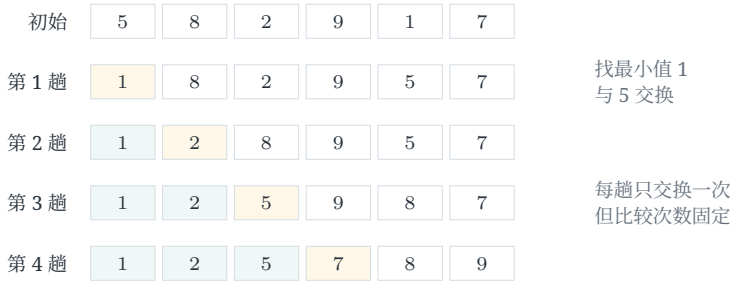
13.3 选择排序

选择排序每趟从尚未排序的部分中选出最小（或最大）的记录，放到这部分的前端。简单选择排序用线性扫描找最小元，堆排序则用堆来加速找最小元的过程。

简单选择排序

以 $\{5,8,2,9,1,7\}$ 为例。第一趟在全部 6 个元素中找最小值 1，与第一个元素 5 交换，得到 $\{1,8,2,9,5,7\}$；第二趟在剩下的 $\{8,2,9,5,7\}$ 中找最小值 2，与 8 交换，得到 $\{1,2,8,9,5,7\}$；继续下去，直到前 $n-1$ 个位置都确定。

```
void SelectSort(int A[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int min = i;
        for (int j = i + 1; j < n; j++)
            if (A[j] < A[min]) min = j;
        if (min != i) {
            int t = A[i]; A[i] = A[min]; A[min] = t;
        }
    }
}
```



图里浅色格子是已经确定的位置，深色格子是本趟找到的最小值。简单选择排序的交换次数很少，最多 $n-1$ 次；但无论初始序列如何，第 i 趟都要在剩下 $n-i$ 个元素中扫一遍，比较次数固定为

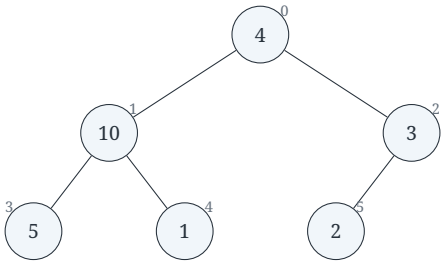
$$\sum_{i=0}^{n-2}(n-1-i)=\frac{n(n-1)}{2},$$

所以最好、平均、最坏时间都是 $O(n^2)$。移动次数最好为 0（已经有序，不需要交换），最坏为 $3(n-1)$ 次赋值。辅助空间 $O(1)$。

简单选择排序不稳定。反例是 $\{5,5',2\}$：第一趟找到最小值 2，与第一个 5 交换，得到 $\{2,5',5\}$，两个 5 的次序反了。

堆排序

堆是一棵**完全二叉树**，并且每个非叶结点都不小于它的孩子（**大根堆**），或者都不大于它的孩子（**小根堆**）。大根堆的根是整棵树的最大值，小根堆的根是最小值。完全二叉树适合顺序存储：把结点按层序放进数组，父子关系可以直接用下标算出来。



下标 i 的左孩子是 $2i+1$
右孩子是 $2i+2$
双亲是 $\lfloor (i-1)/2 \rfloor$

0	1	2	3	4	5
4	10	3	5	1	2

$A[1]=10$ 的孩子是
$A[3]=5$、$A[4]=1$

图里同一棵完全二叉树的两种看法：上面是树，下面是数组。树上的小数字是数组下标，按层序一一对应。下标从 0 开始时，i 的左孩子是 $2i+1$、右孩子是 $2i+2$、双亲是 $\lfloor (i-1)/2 \rfloor$；最后一个非叶结点是 $\lfloor n/2 \rfloor - 1$，叶子从 $\lfloor n/2 \rfloor$ 开始。408 教材也常用下标从 1 开始，那时左孩子是 $2i$、右孩子是 $2i+1$、双亲是 $\lfloor i/2 \rfloor$，最后一个非叶结点是 $\lfloor n/2 \rfloor$，做题前要先看清题目用的是哪一种。

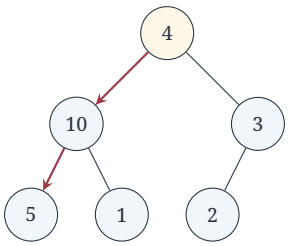
堆的核心操作是**向下调整**（下沉）。假设以 $A[k]$ 为根的左右子树都已经是堆，只有 $A[k]$ 可能违反堆序，那么把 $A[k]$ 与较大的孩子交换，再沿着这条路径继续向下，直到它不小于两个孩子或者到达叶子。对长度为 n 的堆，一次下沉最多走 $\lfloor \log_2 n \rfloor$ 层，每层最多比较两次（选较大的孩子、与根比较），所以是 $O(\log n)$。

```
// 大根堆的向下调整，堆的大小为 n
void SiftDown(int A[], int k, int n) {
    int temp = A[k];
    for (int i = 2 * k + 1; i < n; i = 2 * k + 1) {
        if (i + 1 < n && A[i] < A[i + 1]) i++;  // 选较大的孩子
        if (temp >= A[i]) break;                // 已满足堆序
        A[k] = A[i];
        k = i;
    }
    A[k] = temp;
}
```

建堆时，叶子本身可以看作大小为 1 的堆，所以从最后一个非叶结点开始，依次向前对每个结点做一次下沉。这样处理到根时，根的左右子树都已经是堆，正好满足下沉的前提。

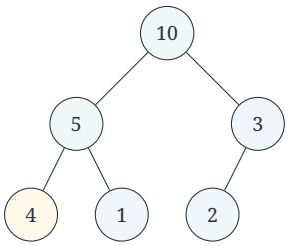
```
void BuildMaxHeap(int A[], int n) {
    for (int i = n / 2 - 1; i >= 0; i--)
        SiftDown(A, i, n);
}
```

初始完全二叉树



4 先与 10 交换
再与 5 交换

从根下沉后



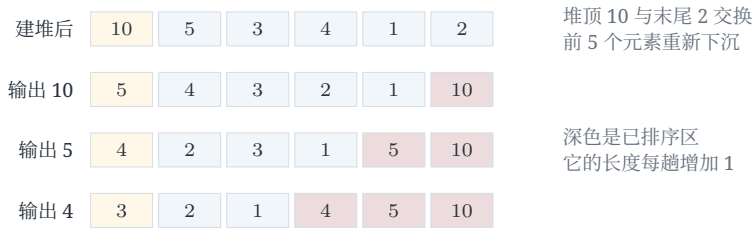
图里左边是初始数组 $\{4,10,3,5,1,2\}$ 对应的完全二叉树，红色箭头是 4 下沉的路径：4 比两个孩子 10、3 中的较大者 10 小，交换；到了 10 原来的位置，4 又比两个孩子 5、1 中的较大者 5 小，再交换；到叶子时停止。右边是调整后的树，根 10 已经是最大值。建堆从下标 2 的结点开始，但 3 已经大于它的孩子 2，不需要动；下标 1 的 10 也满足堆序；最后才处理根。

建堆的总时间复杂度是 $O(n)$，不是 $O(n\log n)$。直观上，越靠近叶子的结点越多，但它们下沉的距离越短；第 h 层的结点数约为 $n/2^{h+1}$，每个最多下沉 h 层，总和

$$\sum_{h\geqslant 0}\frac{n}{2^{h+1}}\cdot O(h)=O(n)\sum_{h\geqslant 0}\frac{h}{2^{h}}=O(n).$$

真正排序时，先把数组建成大根堆，然后重复“把堆顶与当前堆的最后一个元素交换，堆的大小减 1，再对新堆顶下沉”。每交换一次，最大值就落到数组末尾的已排序区。

```
void HeapSort(int A[], int n) {
    BuildMaxHeap(A, n);
    for (int i = n - 1; i > 0; i--) {
        int t = A[0]; A[0] = A[i]; A[i] = t;  // 堆顶换到末尾
        SiftDown(A, 0, i);                    // 剩余 i 个元素恢复成堆
    }
}
```



图里每一行是一次“取最大值”的结果：堆顶与当前堆的末尾交换后，末尾变成已排序区（深色），前 i 个元素重新下沉成堆。堆排序一共做 $n-1$ 次交换和下沉，每次 $O(\log n)$，加上建堆的 $O(n)$，总时间为 $O(n\log n)$，并且最好、平均、最坏都是这个量级。辅助空间只有几个临时变量，为 $O(1)$。

堆排序不稳定。反例是 $\{5,5',3\}$：它已经是大根堆，第一次把堆顶 5 与末尾 3 交换，得到 $\{3,5',5\}$；下沉后变成 $\{5',3,5\}$；最后得到 $\{3,5',5\}$，5’ 跑到了 5 前面。原因是堆顶和堆尾的交换是远距离的，相等关键字的相对次序无法保证。

堆本身还支持两种常用操作。**插入**：把新记录放到数组末尾，然后**向上调整**（上浮），与双亲比较，比双亲大就交换，直到满足堆序，代价 $O(\log n)$。**删除堆顶**：把最后一个记录移到堆顶，堆的大小减 1，再对堆顶下沉，代价也是 $O(\log n)$。这两种操作只影响一条从根到叶（或从叶到根）的路径，所以不需要重建整个堆。

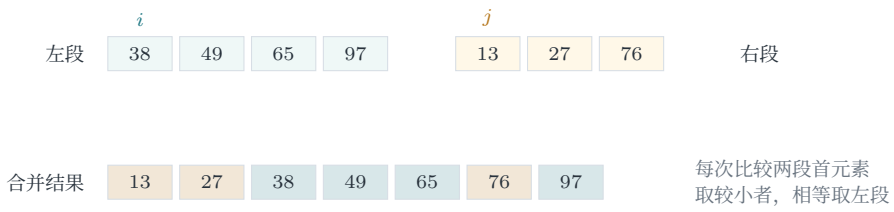
13.4 归并排序

归并排序把两个已经有序的段合并成一个有序段。二路归并是最基本的形式：两个指针分别指向两段的开头，每次取较小的那个放到辅助数组，直到一段取完，再把另一段剩下的元素直接复制过去。

```
void Merge(int A[], int low, int mid, int high) {
    int *B = (int *)malloc((high - low + 1) * sizeof(int));
    int i = low, j = mid + 1, k = 0;
```

```
    while (i <= mid && j <= high) {
        if (A[i] <= A[j]) B[k++] = A[i++];  // 相等时先取左段
        else              B[k++] = A[j++];
    }
    while (i <= mid)  B[k++] = A[i++];
    while (j <= high) B[k++] = A[j++];
    for (i = low, k = 0; i <= high; i++, k++) A[i] = B[k];
    free(B);
}

void MergeSort(int A[], int low, int high) {
    if (low < high) {
        int mid = (low + high) / 2;
        MergeSort(A, low, mid);
        MergeSort(A, mid + 1, high);
        Merge(A, low, mid, high);
    }
}
```

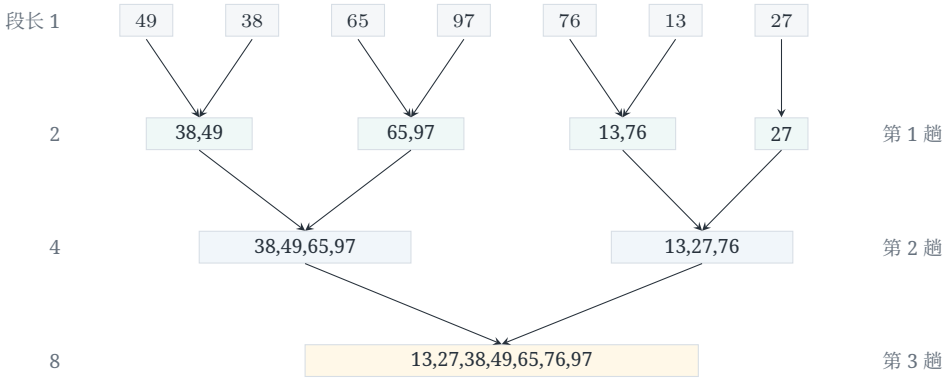


图里左右两段各自有序，两个指针从段首开始，比较 38 和 13，取 13；再比较 38 和 27，取 27；再比较 38 和 76，取 38……输出格的底色表示它来自哪一段。某一侧取完后，另一侧剩下的元素直接搬过去，不再比较。合并两个长度分别为 p, q 的段，最多比较 $p+q-1$ 次，移动 $p+q$ 个记录。

递归的归并排序把数组对半切开，分别排序，再合并。递归式

$$T(n) = 2T(n/2) + O(n)$$

的解是 $T(n) = O(n \log n)$。也可以自底向上实现：第一趟把相邻的两个长度为 1 的段合并成长度为 2 的段，第二趟合并成长度为 4 的段，依此类推，直到段长不小于 n。归并趟数为 $\lceil \log_2 n \rceil$，与初始序列无关；每一趟都要扫描并移动全部 n 个记录，所以每趟 $O(n)$。



图里同一层的方框是这一趟归并后的有序段，方框里的逗号分隔各元素。7 个记录经过 $1 \rightarrow$

$2 \to 4 \to 8$ 三次段长翻倍，最后合成一个有序段；每层所有方框的长度和都等于 7，所以每层的工作量是 $O(n)$。归并树的高度为 $\lceil \log_2 n \rceil$，这就是归并排序的趟数。

归并排序的时间复杂度在最好、平均、最坏情况下都是 $O(n \log n)$。最好情况下比较次数少一些（两段交错得少，一段很快取完）；最坏情况下比较次数为 $\Theta(n \log n)$（$n = 2^k$ 时恰为 $n \log_2 n - n + 1$），量级不变。辅助数组需要 n 个记录的空间，递归栈还需要 $O(\log n)$，总空间为 $O(n)$。合并时用 `A[i] <= A[j]` 保证相等时先取左段，所以归并排序稳定。它的代价是额外的 $O(n)$ 空间，这也是内部排序中“用空间换稳定”的典型。

13.5 基数排序

基数排序不比较关键字的大小，而是把关键字拆成若干位，按位分配和收集。以十进制整数为例，每个关键字可以看成 d 位数字，每一位有 $r = 10$ 种取值。最低位优先（LSD）的做法是：先按个位把记录分配到 10 个队列，再依次收集；然后按十位分配、收集；直到最高位。每趟分配和收集都必须稳定，否则前一趟排好的次序会被打乱。

以 $\{278, 109, 63, 930, 589, 184, 505, 269, 8, 83\}$ 为例，个位分配的结果是：0 号队列有 930，3 号有 63、83，4 号有 184，5 号有 505，8 号有 278、8，9 号有 109、589、269。按队列编号从小到大收集，得到

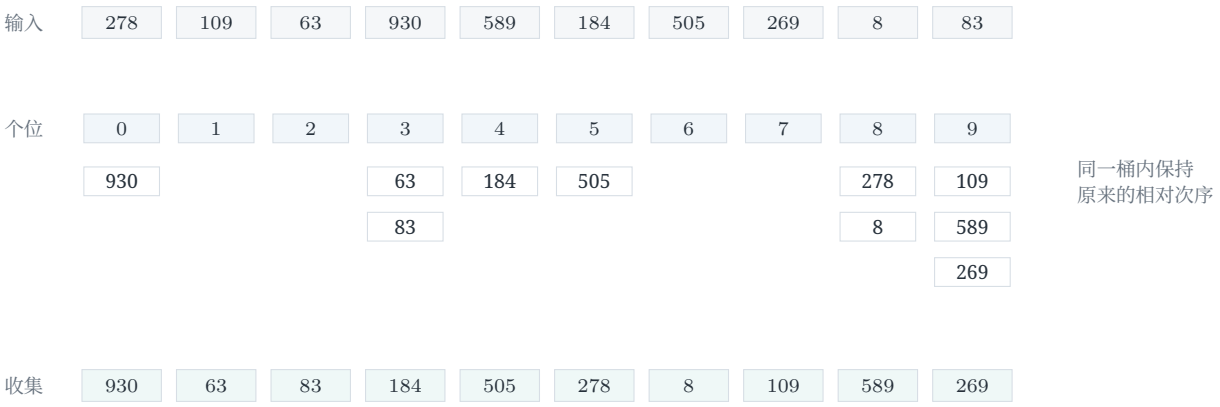
$$\{930, 63, 83, 184, 505, 278, 8, 109, 589, 269\}.$$

十位再分配、收集，得到

$$\{505, 8, 109, 930, 63, 269, 278, 83, 184, 589\}.$$

最后按百位分配、收集，得到

$$\{8, 63, 83, 109, 184, 269, 278, 505, 589, 930\}.$$



图里上方是输入序列，中间是按个位分成的 10 个队列，下方是按队列编号从小到大收集的结果。注意 3 号桶里 63 在 83 前面，8 号桶里 278 在 8 前面，9 号桶里 109、589、269 的次序也没有变——这正是“分配和收集稳定”的含义。若某一趟不稳定，比如把 83 放到了 63 前面，那么下一趟按十位排序后，个位的信息就被破坏了。

下面的数组版用计数排序完成每一趟“分配”和“收集”：先数出每个桶有多少个记录，算出每个桶在辅助数组中的起始位置，再按原序把记录放进对应的桶。链式版只是把辅助数组换成队列指针，思路相同。

```c
void RadixSort(int A[], int n, int d) {
    int *B = (int *)malloc(n * sizeof(int));
    int cnt[10], pos[10];
    int radix = 1;
    for (int p = 0; p < d; p++) {
        for (int r = 0; r < 10; r++) cnt[r] = 0;
        for (int i = 0; i < n; i++)          // 统计每个桶的记录数
            cnt[(A[i] / radix) % 10]++;
        pos[0] = 0;
        for (int r = 1; r < 10; r++)         // 每个桶的起始下标
            pos[r] = pos[r - 1] + cnt[r - 1];
        for (int i = 0; i < n; i++) {        // 按原序放入，保证稳定
            int r = (A[i] / radix) % 10;
            B[pos[r]++] = A[i];
        }
        for (int i = 0; i < n; i++) A[i] = B[i];
        radix *= 10;
    }
    free(B);
}
```

设关键字有 d 位、基数为 r、记录数为 n。每趟分配要扫描 n 个记录，收集要扫描 r 个队列并把 n 个记录接起来，所以每趟 $O(n+r)$，d 趟共

$$O\big(d(n+r)\big).$$

若 d 和 r 都可以看作常数，基数排序是线性的 $O(n)$。辅助空间包括 r 个队列的头尾指针；若记录用链结点串起来，还要 n 个指针域，合计 $O(n+r)$。基数排序稳定，前提是每一趟的分配和收集都稳定。

基数排序还有最高位优先（MSD）的版本：先按最高位分组，再对每组递归地排下一位。MSD 不要求全局稳定，但实现更复杂，408 主要考 LSD。基数排序不是基于比较的排序，所以不受“比较排序最坏 $\Omega(n\log n)$”下界的限制；它的代价是要把关键字拆成位，并且位的取值范围不能太大，否则 r 个队列的开销会超过比较排序。

13.6 内部排序方法的比较

把前面几种内部排序放在一起，可以看到它们在最好/平均/最坏时间、辅助空间和稳定性上的取舍。

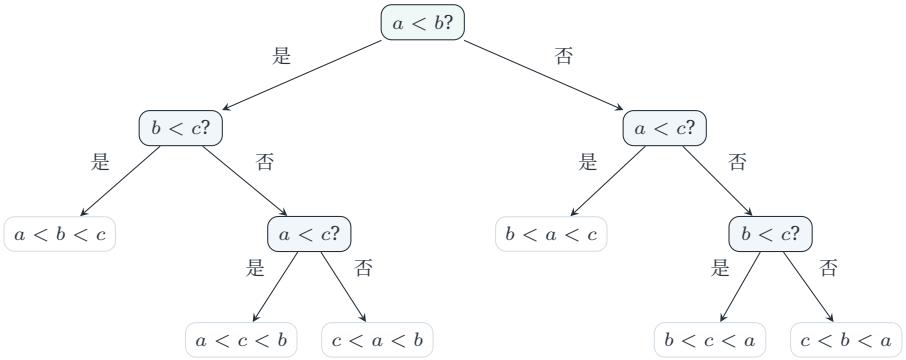
内部排序算法比较

算法	最好时间	平均时间	最坏时间	辅助空间	稳定性
直接插入排序	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	稳定
折半插入排序	$O(n\log n)$	$O(n^2)$	$O(n^2)$	$O(1)$	稳定
希尔排序	依赖增量	依赖增量	$O(n^2)$	$O(1)$	不稳定
冒泡排序	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	稳定
快速排序	$O(n\log n)$	$O(n\log n)$	$O(n^2)$	平均 $O(\log n)$，最坏 $O(n)$	不稳定
简单选择排序	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	不稳定
堆排序	$O(n\log n)$	$O(n\log n)$	$O(n\log n)$	$O(1)$	不稳定
归并排序	$O(n\log n)$	$O(n\log n)$	$O(n\log n)$	$O(n)$	稳定
基数排序	$O(d(n+r))$	$O(d(n+r))$	$O(d(n+r))$	$O(n+r)$	稳定

表里的“最好时间”按比较和移动的总量来算。折半插入排序即使初始有序，也要为每个元素做一次折半查找，比较次数是 $\Theta(n\log n)$，所以最好情况不是 $O(n)$；但它的移动次数为 0，实际运行仍然很快。希尔排序的最坏 $O(n^2)$ 是就 $d=\lfloor n/2\rfloor,\lfloor n/4\rfloor,\dots$ 这个增量序列而言的，换增量序列上界会变。

怎么选算法，可以先看数据规模和初始状态。n 很小时（例如几十个记录），直接插入排序和简单选择排序的常数小，反而比快排、归并快；若初始基本有序，直接插入或冒泡可以做到 $O(n)$，而简单选择排序仍要做满 $n(n-1)/2$ 次比较。n 较大时，快速排序的平均常数最小，是通用内部排序的首选；但它最坏会退化到 $O(n^2)$，对初始有序的序列要特别小心。堆排序的最坏时间有保证，辅助空间只有 $O(1)$，适合空间紧张又要求稳定性能的场合，代价是常数比快排大。归并排序稳定，时间也稳定在 $O(n\log n)$，但需要 $O(n)$ 的辅助空间，常用在外部排序和链表排序中。若关键字可以拆成位数不多的“位”，基数排序可以做到线性，但它依赖关键字的表示形式。

要求稳定时，可以选择直接插入、冒泡、归并、基数排序；希尔、简单选择、快速、堆排序都不稳定。这个结论不要只背表，最好能各举一个反例：希尔是远距离分组，简单选择是把最小值换到前面，快速是填坑时跨过相等元素，堆是堆顶与堆尾交换。



3 个元素有 $3!=6$ 种排列，判定树至少 6 个叶子
高度至少 $\lceil\log_2 6\rceil=3$

图里是一棵比较判定树：每个内部结点是一次两两比较，每个叶子是一种可能的排列。3 个元素有 6 种排列，所以树至少要有 6 个叶子；二叉树高度为 h 时最多有 2^h 个叶子，因此 $h\geqslant\lceil\log_2 6\rceil=3$。推广到 n 个元素，判定树至少有 $n!$ 个叶子，高度至少 $\lceil\log_2(n!)\rceil=\Theta(n\log n)$。这就是基于比较的排序最坏情况下界 $\Omega(n\log n)$ 的来源；堆排序和归并排序达到了这个量级，而基数排序因为不靠比较，可以突破这个下界。

还有几个和“趟数”有关的结论容易混。冒泡排序最多 $n-1$ 趟，最好 1 趟（第一趟无交换就结束）；简单选择排序和堆排序都是 $n-1$ 趟，每趟确定一个元素的最终位置；归并排序是 $\lceil \log_2 n \rceil$ 趟，与初始序列无关；基数排序是 d 趟，d 是关键字的位数；希尔排序的趟数等于增量序列的长度；快速排序的划分次数等于递归树的内部结点数，平衡或随机时约为 $\Theta(n)$，最坏为 $n-1$；递归深度最好/平均为 $O(\log n)$，最坏为 $O(n)$。比较这些数字时，先确认题目说的“一趟”是划分次数还是递归层数。

13.7 外部排序

内部排序要求所有记录同时放在内存里。如果文件有几十 GB，内存只有几百 MB，就不可能一次装下。外部排序的做法是：先把文件分成若干块，每块读入内存做内部排序，得到一个有序的**初始归并段**（run），写回外存；然后对这些归并段反复做多路归并，直到只剩一个有序文件。

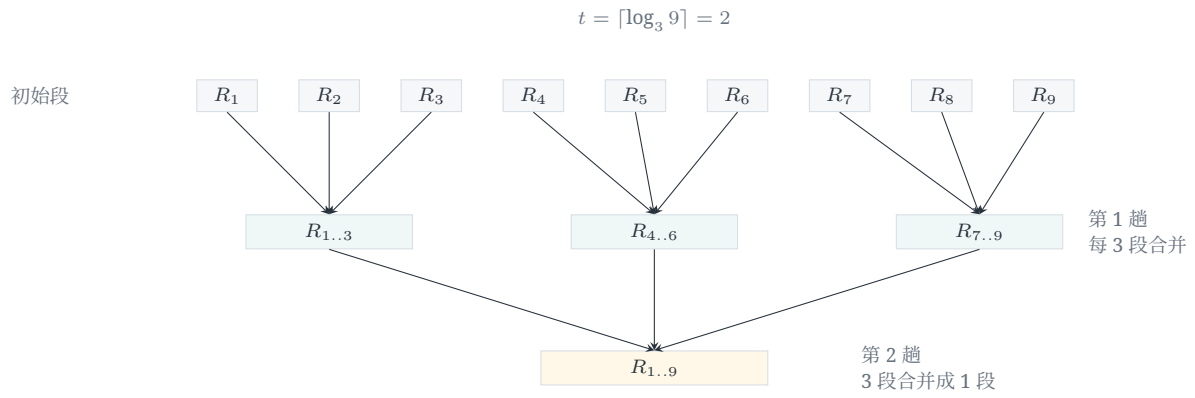
设文件有 N 个记录，内存缓冲区一次能容纳 w 个记录，则初始归并段的个数约为

$$m = \left\lceil \frac{N}{w} \right\rceil .$$

若采用 k 路平衡归并，每趟把 k 个段合并成一个，段数变为原来的约 $1/k$，需要的趟数为

$$t = \lceil \log_k m \rceil .$$

例如 $m = 100$、$k = 4$ 时，$4^3 = 64 < 100$，$4^4 = 256 \geqslant 100$，所以需要 4 趟，而不是 3 趟。

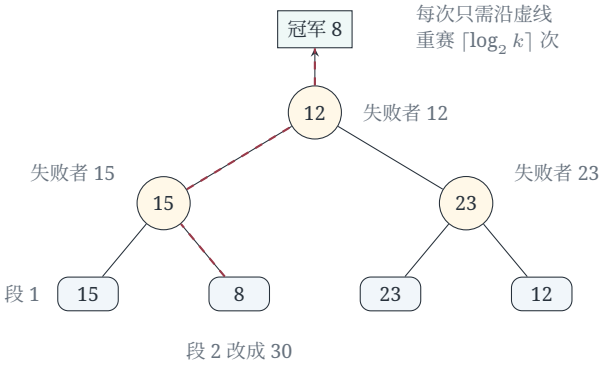


图里 9 个初始归并段经过两趟 3 路归并合成一个有序文件：第一趟每 3 个段合成一段，得到 3 个更长的段；第二趟把这 3 个段合成一个。每一趟都要把全部 N 个记录从外存读一遍、再写一遍，所以归并阶段的 I/O 次数为 $2Nt$；若把生成初始段时的一次读、一次写也算上，总 I/O 为

$$2N(t+1) = 2N\big(1 + \lceil \log_k m \rceil\big).$$

外部排序的总时间由三部分组成：生成初始段时的内部排序时间、外存读写时间、归并时的内部比较时间。三者中外存读写通常占主导，所以减少归并趟数比减少比较次数更重要。

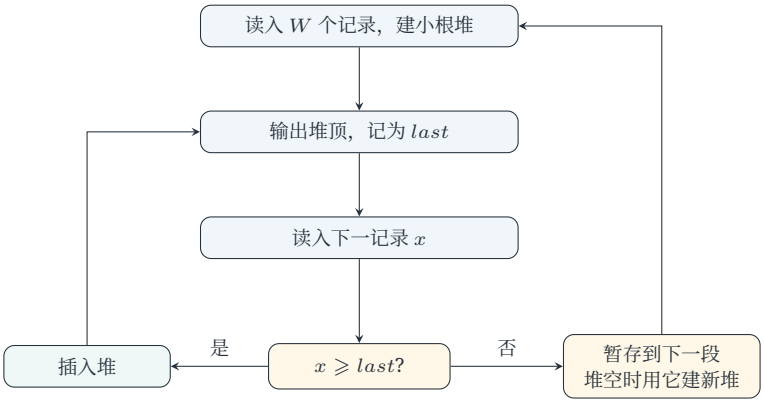
增大 k 可以减少趟数，但 k 路归并需要 k 个输入缓冲区和 1 个输出缓冲区，内存中至少要能放下 $k+1$ 个块；k 太大时每个缓冲区变小，I/O 的块数反而增加。另一个限制是内部归并的代价：如果每输出一个记录都线性扫描 k 个段的首元素，最多需要 $k-1$ 次比较，一整趟最多约 $(k-1)N$ 次比较，k 增大后这部分会变得很重。**败者树**就是用来把“选最小”的代价降到 $O(\log k)$ 的。



图里 $k = 4$ 个叶子是 4 个归并段的当前首记录，内部结点保存比较的失败者，冠军单独放在根的上方。初始时 15 与 8 比较，8 胜、15 留在左内部结点；23 与 12 比较，12 胜、23 留在右内部结点；8 与 12 比较，8 胜、12 留在根结点，冠军为 8。若段 2 的下一个记录变成 30，只需沿虚线从叶子走到根：30 与结点里的 15 比较，15 胜、30 留下；15 再与根里的 12 比较，12 胜、15 留下；冠军变为 12。整条路径只有 2 次比较，而不是重新扫描 4 个段。

败者树的叶结点数为 k，内部结点数为 $k-1$，树高为 $\lceil \log_2 k \rceil$。初始建树需要 $k-1$ 次比较；此后每输出一个记录，沿叶到根的路径重赛，最多 $\lceil \log_2 k \rceil$ 次比较。归并 N 个记录的总比较次数约为 $N\lceil \log_2 k \rceil$，比线性扫描的 $N(k-1)$ 小得多。**胜者树**也能做同样的事，但胜者树的内部结点保存胜者，重赛时要访问兄弟结点才能知道另一个候选者；败者树保存失败者，胜者可以直接和父结点中保存的失败者比较，访问的结点更少，这是它名字的由来。

初始归并段的长度不一定等于缓冲区大小。**置换-选择排序**可以在缓冲区只有 W 个记录时，生成比 W 长得多的归并段：先把 W 个记录建成小根堆，输出堆顶（当前段的最小值）；读入下一个记录，如果它不小于刚输出的记录，就插入堆中继续参与当前段，否则暂存起来留给下一段；堆空时当前段结束，用暂存的记录建新堆，开始下一段。



随机输入时，平均段长约为 $2W$；
输入已经有序时，整个文件可以成为一个段

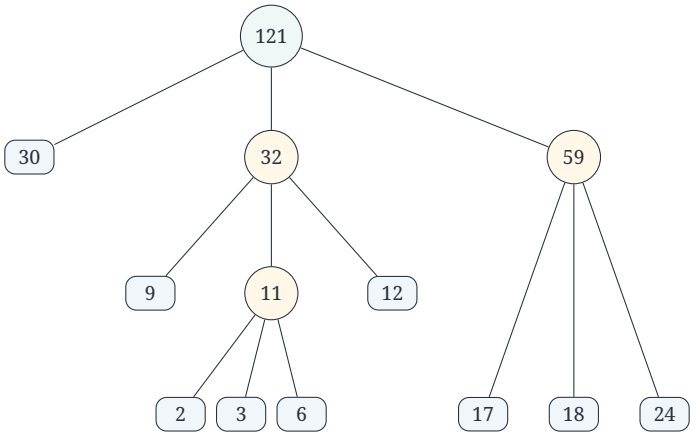
图里是置换-选择的流程：判断“新记录是否不小于上次输出”是核心，满足就继续留在当前段，不满足就推迟到下一段。因为推迟的记录不参与当前堆，堆中始终有记录可输出，所以一个段可以超过缓冲区容量。对随机排列的输入，段的平均长度约为 $2W$；输入已经升序时，所有记录都能留在同一个段中，段长为 N；输入逆序时，每读入一个记录都要推迟，段长退化为 W。

当各初始归并段的长度不相等时，归并的先后顺序会影响总 I/O。把归并段看成带权叶子，权

值是段长，构造一棵 k 叉 Huffman 树，使带权路径长度

$$\mathrm{WPL} = \sum_{i=1}^{m} w_i \, l_i$$

最小，其中 w_i 是第 i 段的长度，l_i 是它在树中的深度。这棵树叫**最佳归并树**，按它自底向上归并，总读写次数为 $2\,\mathrm{WPL}$。



图里是 9 个归并段、3 路归并的最佳归并树，叶子上的数字是段长，圆圈里的数字是合并后的段长。构造过程和 Huffman 树一样：每次取权值最小的 3 个结点合并，$2+3+6=11$，$9+11+12=32$，$17+18+24=59$，最后 $30+32+59=121$。按这棵树归并，WPL 为

$$30\times 1 + (9+12+17+18+24)\times 2 + (2+3+6)\times 3 = 30 + 160 + 33 = 223,$$

总读写次数为 $2\times 223 = 446$。

如果初始段数 m 不满足 $(m-1) \bmod (k-1) = 0$，这棵树就不是严格的 k 叉树，最后一层会有“缺孩子”的结点，Huffman 构造不能直接进行。这时要补若干个长度为 0 的虚拟段，把段数凑到满足条件为止。需要补的个数为

$$(k-1) - \big((m-1) \bmod (k-1)\big).$$

例如 $m=8$、$k=3$ 时，$(8-1) \bmod 2 = 1$，需要补 1 个长度为 0 的段，按 9 个段构造；虚拟段不贡献 WPL，只是把树补成满 3 叉树。$m=9$、$k=3$ 时 $(9-1) \bmod 2 = 0$，不需要补段，正好是图里的情况。

14 习题

14.1 基础过关题

1. （单项选择题）对同一组记录分别执行直接插入排序和折半插入排序，二者都采用本章给出的代码（升序）。与直接插入排序相比，折半插入排序

   (A) 比较次数与后移次数都减少

   (B) 只减少比较次数，后移次数不变

   (C) 只减少后移次数，比较次数不变

   (D) 总时间复杂度降为 $O(n\log n)$

2. （单项选择题）对序列 $\{5,1,4,2,8,3\}$ 执行一趟冒泡排序：从前向后依次比较相邻元素，逆序即交换，扫描到末尾为止，暂不启用“本趟无交换则提前结束”的优化。第一趟结束后的序列是

   (A) $\{1,4,2,5,3,8\}$　　(B) $\{1,2,4,5,3,8\}$

   (C) $\{1,4,2,3,5,8\}$　　(D) $\{1,4,2,5,8,3\}$

3. （单项选择题）对序列 $\{46,25,68,13,57,32,79,21\}$，按本章“取首元素为枢轴、先从右端找小、再从左端找大、交替填坑”的划分算法执行一趟划分，结果为

   (A) $\{21,25,32,13,46,57,79,68\}$　　(B) $\{21,25,32,13,46,57,68,79\}$

   (C) $\{13,25,32,21,46,57,79,68\}$　　(D) $\{21,25,13,32,46,57,79,68\}$

4. （单项选择题）已知大根堆按层序存放在数组 $A[0..5]$ 中，序列为 $\{20,15,18,8,9,12\}$。删除堆顶元素后，将堆尾元素移到堆顶并向下调整，得到的堆是

   (A) $\{18,15,12,8,9\}$　　(B) $\{15,12,18,8,9\}$

   (C) $\{18,15,9,8,12\}$　　(D) $\{18,12,15,8,9\}$

5. (填空题)对序列 $\{9,1,5,8,3,7,4,6,2\}$ 采用希尔排序,增量依次取 $4,2,1$($d_1=\lfloor 9/2\rfloor=4$,此后每次取前一个增量的一半并向下取整)。第一趟($d=4$)结束后的序列为 _____。

6. (填空题)将两个长度分别为 4 和 5 的有序表归并为一个有序表,最坏情况下需要比较 _____ 次;对 $n=1000$ 个记录做二路归并排序(自底向上,段长依次为 $1,2,4,\ldots$),共需归并 _____ 趟。

14.2 真题风格与综合训练

7. (真题风格·单项选择题)对序列 $\{329,457,657,839,436,720,355\}$ 按最低位优先(LSD)做十进制基数排序,第一趟按个位分配到 10 个队列后依次收集,所得序列是

(A) $\{720,355,436,457,657,329,839\}$ (B) $\{720,355,436,457,657,839,329\}$

(C) $\{720,355,436,657,457,329,839\}$ (D) $\{329,355,436,457,657,720,839\}$

8. (真题风格·单项选择题)某文件经内部排序后得到 100 个初始归并段,采用 4 路平衡归并,并用败者树选出当前最小记录。建立败者树后,每输出一个记录平均需要的比较次数与总归并趟数分别为

(A) 约 2 次,3 趟 (B) 约 2 次,4 趟

(C) 约 4 次,4 趟 (D) 约 100 次,25 趟

9. (真题风格·综合题)设关键字序列 $S=\{52,31,74,18,63,45,87,26\}$,下标从 0 开始,所有排序均按升序进行。约定:

- 直接插入排序采用本章代码,仅当 $A[j]>temp$ 时才后移;
- 简单选择排序每趟在未排序部分中选最小元素,与未排序部分的第一个元素交换;

- 二路归并排序采用自底向上方式，段长依次为 $1, 2, 4, 8$，归并时相等元素先取左段；
- 快速排序以区间首元素为枢轴，按本章“右找小、左找大、交替填坑”的划分算法。

回答：

(1) 直接插入排序执行完 $i = 1, 2, 3$ 三趟后的序列；

(2) 简单选择排序执行完前两趟后的序列；

(3) 二路归并排序执行完第一趟、第二趟后的序列；

(4) 以 52 为枢轴执行一趟划分后的序列；

(5) 上述四种算法中哪些稳定、哪些不稳定，并各给出判断理由。

10. **（真题风格 · 综合题）** 设计一个算法：从含 n 个整数的数组 $A[0..n-1]$ 中找出最小的 k 个元素（$1 \leqslant k \leqslant n$），存入 $B[0..k-1]$，不要求 B 内部有序。要求时间复杂度为 $O(n \log k)$，并回答：

(1) 说明算法思想与正确性依据；

(2) 写出 C 语言代码（可调用向下调整函数 `SiftDown`）；

(3) 分析时间、空间复杂度，并与“先整体排序再取前 k 个”“每趟线性扫描选最小”作比较。

15 算法设计专题

排课表、找零钱、走迷宫，这三件事在 408 的算法设计题里经常换一副面孔出现：给一堆区间问最多能选几个，给几种面额问最少凑多少枚，给一张图问有没有一条满足条件的路径。它们难的地方不在语法，而在于“从哪下手”。本章把常用的下手方式归纳成四条路线：分治、贪心、动态规划、回溯，最后统一讨论怎么给这些算法算时间与空间。

这四条路线不是互斥的标签。同一个最大子段和，可以分治，也可以动态规划；同一个 0-1 背包，可以动态规划，也可以回溯枚举。选哪条，取决于问题的结构：子问题能不能独立切开、局部最优能不能推出全局最优、子问题会不会重复、解空间能不能系统地枚举。下面的顺序也大致按这四个问题展开。

15.1 分治：把问题对半切开

先在有序数组里找一个数。从中间取一个元素比较，比它小就到左半边找，比它大到右半边找，每次丢掉一半。再看归并排序：把数组从中间切成两段，分别排好，再把两个有序段并成一个。两件事的动作不同，骨架相同：把原问题分成若干规模更小的同类子问题，分别求解，再把结果合起来。这个骨架叫分治。

分治能成立，需要三个条件凑齐。子问题必须与原问题同型，否则递归无法复用同一套逻辑；子问题的规模必须严格变小，否则递归不终止；子问题之间必须相互独立，否则分别求解后还得处理它们之间的耦合，合并会变得非常复杂。归并排序的两个半段互不影响，合并时只需比较大小；如果子问题共享数据（后面的动态规划就是这种情况），硬套分治就会重复计算。

二分查找每次只进入一个子问题，严格说属于“减治”（decrease-and-conquer），但它的分析方法和分治一样：写出规模缩小的递推式。

```
int BinarySearch(int A[], int n, int key) {
    int lo = 0, hi = n - 1;
    while (lo <= hi) {                /* 区间为空时 lo > hi */
        int mid = lo + (hi - lo) / 2;  /* 防止 lo + hi 溢出 */
        if (A[mid] == key) return mid;
        if (A[mid] < key) lo = mid + 1;
        else              hi = mid - 1;
    }
    return -1;
}
```

循环的区间是闭区间 $[lo, hi]$，所以条件写成 `lo <= hi`；如果在 `lo == hi` 时退出，就会漏掉最后一个元素。每次比较后区间长度至少减半，$T(n) = T(n/2) + \Theta(1)$，解得 $\Theta(\log n)$。迭代版本只用了几个变量，辅助空间 $O(1)$。

归并排序把“切”和“并”都写全了：

```
void Merge(int A[], int lo, int mid, int hi, int tmp[]) {
    int i = lo, j = mid + 1, k = lo;
    while (i <= mid && j <= hi) {
        if (A[i] <= A[j]) tmp[k++] = A[i++];
        else              tmp[k++] = A[j++];
    }
    while (i <= mid) tmp[k++] = A[i++];
```

```
    while (j <= hi)  tmp[k++] = A[j++];
    for (k = lo; k <= hi; k++) A[k] = tmp[k];
}

void MergeSort(int A[], int lo, int hi, int tmp[]) {
    if (lo >= hi) return;                /* 0 或 1 个元素，天然有序 */
    int mid = lo + (hi - lo) / 2;
    MergeSort(A, lo, mid, tmp);
    MergeSort(A, mid + 1, hi, tmp);
    Merge(A, lo, mid, hi, tmp);
}
```

合并时用 `<=` 而不是 `<`，是为了让左段的相等元素先落位，这样归并排序是稳定的。时间花在两个递归调用和一次合并上：合并要扫描两个有序段，代价与区间长度成正比，记作 cn，于是

$$T(n) = 2T\left(\frac{n}{2}\right) + cn, \qquad T(1) = \Theta(1).$$

把这个递推式展开成一棵树，就是图 15-1 的样子。

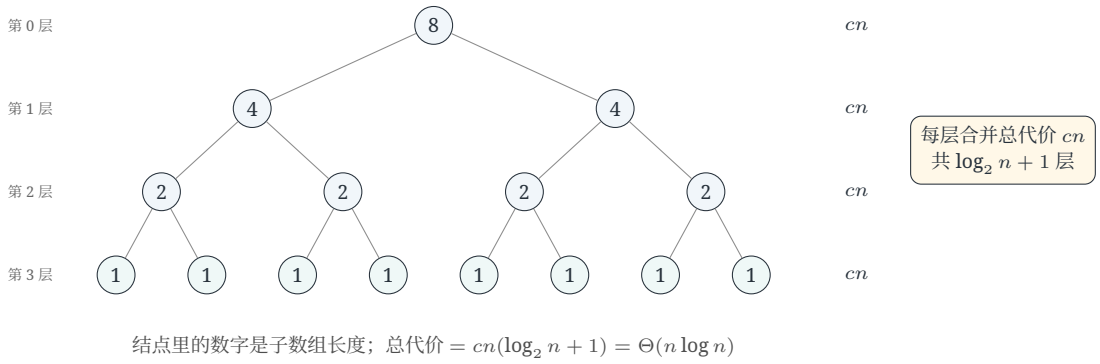


图 15-1　归并排序的递归树（$n = 8$）

图里每一层把所有元素各搬一遍，所以每层都是 cn；树高是 $\log_2 n$，加上根所在的最上面一层，共 $\log_2 n + 1$ 层。把每层加起来，得到 $cn(\log_2 n + 1)$，即 $\Theta(n \log n)$。注意不要只数结点：第 j 层有 2^j 个结点，但每个结点处理的区间只有 $n/2^j$ 长，乘起来仍是 cn。

不是每个递推式都能靠画树一眼看出解。把递归树每层的代价写成 $a^j f(n/b^j)$，再与叶层规模 $n^{\log_b a}$ 比较，就得到主定理。

要点 15.1: 主定理

设 $a \geq 1$，$b > 1$，$f(n) > 0$，递归式

$$T(n) = aT(n/b) + f(n)$$

（n/b 理解为 $\lfloor n/b \rfloor$ 或 $\lceil n/b \rceil$ 不影响渐近阶）。令 $n^{\log_b a}$ 为叶层规模，则：

1. 若 $f(n) = O\left(n^{\log_b a - \varepsilon}\right)$（存在某个 $\varepsilon > 0$），叶层主导，$T(n) = \Theta\left(n^{\log_b a}\right)$；
2. 若 $f(n) = \Theta\left(n^{\log_b a}\right)$，每层代价同阶，$T(n) = \Theta\left(n^{\log_b a} \log n\right)$；
3. 若 $f(n) = \Omega\left(n^{\log_b a + \varepsilon}\right)$（存在某个 $\varepsilon > 0$），且存在常数 $c < 1$，使所有足够大的 n 满足 $af(n/b) \leq cf(n)$，根层主导，$T(n) = \Theta(f(n))$。

三种情形对应递归树的三种形状：叶子那一层最贵、每层差不多贵、根那一层最贵。图 15-2 把每层的总代价画成横条，层 0 在最上面。

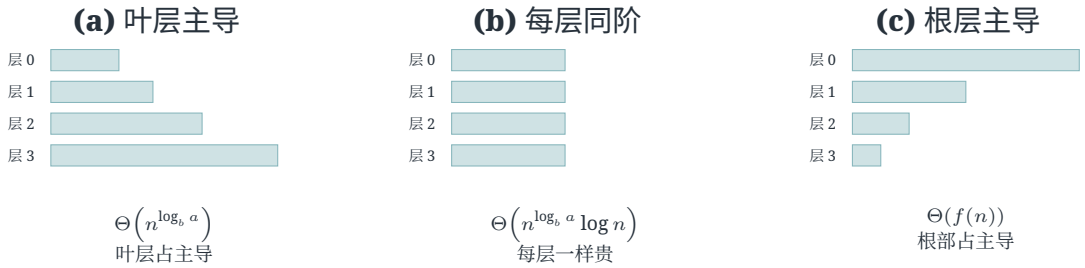


图 15-2　主定理三种情形下每层代价的变化

横条长度只表示该层总代价的相对大小，不表示具体数值。情形 (a) 的层代价按几何级数增长，总和由最后一行决定；情形 (b) 每层相同，总和是“每层代价 × 层数”；情形 (c) 每层按几何级数衰减，总和由第一行决定。

套几个例子：

- $T(n)=2T(n/2)+n$：$a=2$，$b=2$，$n^{\log_2 2}=n$，$f(n)=\Theta(n)$，属于情形 2，得 $\Theta(n\log n)$。这就是归并排序。
- $T(n)=3T(n/2)+n$：$n^{\log_2 3}\approx n^{1.585}$，$f(n)=n=O(n^{1.585-\varepsilon})$，属于情形 1，得 $\Theta(n^{\log_2 3})$。
- $T(n)=2T(n/2)+n^2$：$f(n)=n^2=\Omega(n^{1+\varepsilon})$，且 $2(n/2)^2=n^2/2\le cn^2$（取 $c=1/2$），属于情形 3，得 $\Theta(n^2)$。
- $T(n)=T(n/2)+1$：$a=1$，$n^{\log_2 1}=1$，$f(n)=\Theta(1)$，属于情形 2，得 $\Theta(\log n)$。这就是二分查找。

主定理只适用于 a、b 为常数、f 为正的递归式。$T(n)=2T(n/2)+n/\log n$ 落在情形 1 和情形 2 之间，主定理不适用，需要换别的方法；$T(n)=2T(n/2)+n\log n$ 的 f 比 n 大，但不是多项式级别的大，三种情形也都不覆盖。考试里给出标准形式时直接套；形式稍有变形时，先判断 f 与 $n^{\log_b a}$ 的阶关系，再决定能不能用。

跨中点合并：最大子段和

再看一个不能直接套二分的例子。给定数组 $A[0\ldots n-1]$，元素可正可负，求和最大的连续子段。取中点 mid，最优子段只有三种可能：完全在左半、完全在右半、跨过中点。前两种递归求解，第三种单独算。

以 $A=[2,-5,3,1,-1,4,-2]$ 为例，$mid=3$。左半最大是 $3+1=4$，右半最大是 4，跨中点的最大子段是 $3+1-1+4=7$，取三者最大，答案是 7。跨中点的那一段由两截拼成：以 mid 结尾的一段，和以 $mid+1$ 开头的一段。两截的选择互不影响，所以分别取最大后缀和最大前缀，再相加。

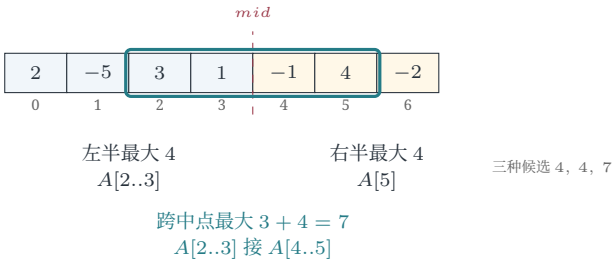


图 15-3　最大子段和的三种位置：左半、右半、跨中点

图里的青色框标出跨中点的最优段：从 mid 向左累加，$1, 4, -1, 1$ 中最大是 4；从 $mid+1$ 向右累加，$-1, 3, 1$ 中最大是 3。两段在中点处拼接，得到 7。

```c
int MaxCrossing(int A[], int lo, int mid, int hi) {
    int sum = 0, bestLeft = A[mid];
    for (int i = mid; i >= lo; i--) {      /* 以 mid 结尾的最大后缀 */
        sum += A[i];
        if (sum > bestLeft) bestLeft = sum;
    }
    sum = 0;
    int bestRight = A[mid + 1];
    for (int j = mid + 1; j <= hi; j++) {  /* 以 mid+1 开头的最大前缀 */
        sum += A[j];
        if (sum > bestRight) bestRight = sum;
    }
    return bestLeft + bestRight;
}

int MaxSubArray(int A[], int lo, int hi) {
    if (lo == hi) return A[lo];            /* 基例：单个元素 */
    int mid = lo + (hi - lo) / 2;
    int lmax = MaxSubArray(A, lo, mid);
    int rmax = MaxSubArray(A, mid + 1, hi);
    int cmax = MaxCrossing(A, lo, mid, hi);
    int best = (lmax > rmax) ? lmax : rmax;
    return (cmax > best) ? cmax : best;
}
```

基例返回单个元素，而不是 0，这样全负数时答案是最小的那个负数，不会把“空段的和 0”误当成合法答案。跨中点扫描两遍，每遍 $\Theta(n)$，加上两次递归：

$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n) \implies \Theta(n \log n).$$

递归深度 $\Theta(\log n)$，除输入数组外只用常数个变量。这道题有 $\Theta(n)$ 的 Kadane 算法（扫描一遍，前缀和为负就丢掉），分治版本的价值在于练习“切开后处理跨界情形”这个套路，而不是追求最快。

15.2 贪心：只保留一个候选

有 6 个活动要用同一间教室，每个活动有开始、结束时间（左闭右开），问最多能安排几个：

活动	a_1	a_2	a_3	a_4	a_5	a_6
时间段	$[1,3)$	$[2,5)$	$[4,6)$	$[6,8)$	$[5,9)$	$[8,10)$

一个直接的贪心规则：按结束时间从早到晚排序，依次检查，只要开始时间不早于上一个选中活动的结束时间就选。排序后是 $a_1(3), a_2(5), a_3(6), a_4(8), a_5(9), a_6(10)$，括号内为结束时间。$a_1$ 选；a_2 开始于 $2 < 3$，跳过；a_3 开始于 $4 \geq 3$，选；a_4 开始于 $6 \geq 6$，选；a_5 开始于 $5 < 8$，跳过；a_6 开始于 $8 \geq 8$，选。共 4 个。

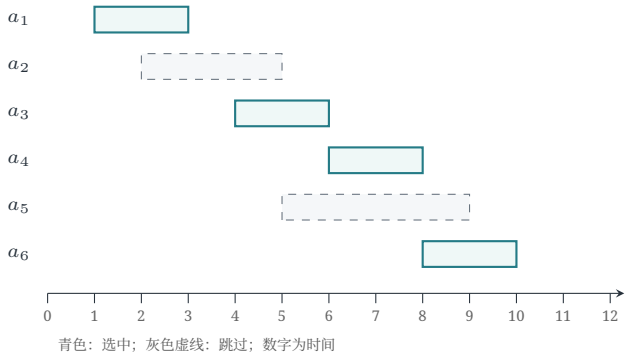


图 15-4　按结束时间最早的贪心选择（选出 a_1, a_3, a_4, a_6）

从图上看，a_2 开始得比 a_1 晚、结束得比 a_1 晚，选了 a_2 就会把 a_3 挡掉；a_5 横跨了 a_4 的整个区间，选了它反而少一个。结束时间最早的活动留给后面的时间最多，这是它胜出的原因。

这个直觉可以写成交换论证。把最优解记作 O，按结束时间排好。设 O 的第一个活动是 o，贪心选的第一个活动是 g。g 是所有活动里结束最早的，所以 $f(g) \le f(o)$。把 O 中的 o 换成 g：O 里剩下的活动都在 o 结束后才开始，而 g 结束得更早，所以它们仍然与 g 相容。换完之后活动数不变，仍是最优解。于是“选 g”不会错过最优解，这就是贪心选择性质。对 g 结束之后的时段重复同样的论证，整个贪心序列就都是安全的。

换成“开始时间最早”或“持续时间最短”就不成立。开始最早的活动可能一直占到很晚，挡住后面三个；持续最短的活动可能卡在中间，把两边都切断。贪心规则看起来越自然，越要用交换论证检查一遍。

贪心什么时候失效

先看找零。面额 $1, 3, 4$，凑 6。按“先拿面额最大”的贪心，$4+1+1$ 三枚；最优是 $3+3$ 两枚。人民币的 $1, 2, 5, 10$ 这套面额满足贪心最优的性质（可以用归纳证明），换成 $1, 3, 4$ 就不成立。面额体系本身决定了贪心能不能用，不能只看“面额大”。

再看背包。分数背包允许把物品切碎，按单位价值从高到低装，最后一件装不下就切一部分。0-1 背包每件要么整件拿、要么不拿，同样的贪心会失败。取容量 $W = 10$，三件物品：

物品	重量	价值	单位价值
A	6	31	$31/6 \approx 5.17$
B	5	25	5
C	5	25	5

按单位价值贪心先拿 A，剩 4 的容量，B、C 都放不下，总价值 31；最优是 $B + C = 50$。如果允许切分，拿 A 之后再装 B 的 $4/5$，总价值 $31 + 20 = 51$，贪心又对了。

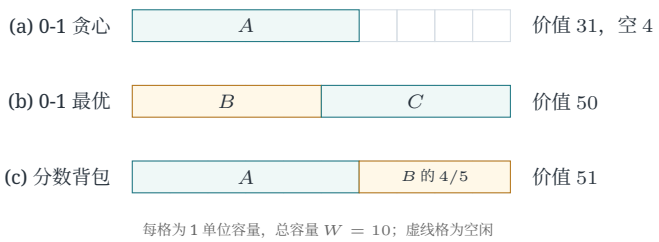


图 15-5　0-1 背包与分数背包：同一组物品，贪心的结果不同

图中三行是同一组物品的三种装法：第一行按单位价值先拿 A，剩下 4 格空着；第二行是 0-1 背包的最优解 $B+C$；第三行允许把 B 切开，装它的 $4/5$。每格代表 1 单位容量，总容量为 10。

分数背包的交换论证很直接：如果最优解没有优先装最高单位价值的物品，就用它替换等重量的低价值物品，总价值不会下降；最后一件装不下就切一部分。0-1 背包不允许切分，替换后重量对不上，这个论证就断了。

贪心法要证明两件事：贪心选择性质（存在一个最优解包含当前贪心选择）和最优子结构（做出该选择后，剩下的子问题的最优解与原问题的最优解拼得起来）。缺一不可。考试里如果只写“每次选最大的”而不说明为什么，通常拿不到证明分。

> **注意**
>
> 贪心题最常见的失分点不是代码，而是把“看起来合理”当成“已经证明”。0-1 背包按单位价值贪心、活动安排按开始时间贪心、找零按面额从大到小贪心，都是经典的失效例子。判断一个贪心策略是否成立，先试着构造反例；构造不出来，再用交换论证证明。

Huffman 编码每次合并权值最小的两棵树；Prim 每次选连接当前树与树外顶点的最便宜边，Kruskal 每次选全图最便宜的边；Dijkstra 每次取出距离最小的未确定顶点。它们都是贪心。这些算法在第 4、5、6 章已经出现过，证明思路同样是交换论证或割性质，可以对照看。贪心算法的时间通常花在排序上：活动安排 $O(n\log n)$，Huffman 用堆也是 $O(n\log n)$，其余是一次线性扫描。

15.3 动态规划：把子问题的答案存起来

计算 Fibonacci 数列的第 n 项。直接按定义递归，$F(n)=F(n-1)+F(n-2)$，会反复计算同一个 $F(k)$：$F(5)$ 要算 $F(4)$ 和 $F(3)$，$F(4)$ 又要算 $F(3)$ 和 $F(2)$，$F(3)$ 被算了两次。n 稍大，调用次数就接近指数。把算过的 $F(k)$ 记下来，每个 k 只算一次，复杂度立刻降到 $O(n)$。

这里出现了分治没有处理的情况：子问题重叠。分治要求子问题独立，各算各的；动态规划承认子问题会重复，于是用一张表保存答案，用空间换时间。能用动态规划的问题通常有三个特征：最优子结构（原问题的最优解由子问题的最优解拼成）、重叠子问题（递归过程中同一个子问题反复出现）、无后效性（状态确定后，后续决策只依赖状态，与怎么到达这个状态无关）。

把“走到某一步时还剩什么问题”压缩成一组参数，这组参数叫状态。状态必须包含决定后续决策的全部信息；如果漏了信息，不同历史会被误认为同一种状态，转移就不对。动态规划的设计可以按五件事检查：状态的含义、转移方程、边界、计算顺序、最终答案。顺序要保证算一个状态时，它依赖的状态已经算好。

最长公共子序列

给定两个串 $X[1\ldots m]$、$Y[1\ldots n]$，求它们的最长公共子序列（LCS）长度。子序列不要求连续，但字符的先后次序不能变。设 $dp[i][j]$ 为 X 的前 i 个字符与 Y 的前 j 个字符的 LCS 长度。看两个前缀的最后一个字符：

- 若 $X[i]=Y[j]$，这个字符可以放进公共子序列的末尾，$dp[i][j]=dp[i-1][j-1]+1$；
- 若 $X[i]\neq Y[j]$，LCS 不可能同时用上这两个字符，至少舍弃一个，$dp[i][j]=\max\{dp[i-1][j],\,dp[i][j-1]\}$。

边界是 $i=0$ 或 $j=0$ 时 $dp=0$，因为空串与任何串的 LCS 都是空串。第一行成立需要一点说明：两个末字符相等时，一定存在一个最优解以这个字符结尾。如果某个最优解的末尾不是它，可以把匹配挪到最后，长度不变，所以加 1 不会错过更优解。

取 $X=$ ABCBDAB，$Y=$ BDCABA，按行填表，得到图 15-6。单元格里的数字是 $dp[i][j]$，青色格子是回溯时字符相等的格子。

$X_i \backslash Y_j$	ε	B	D	C	A	B	A
ε	0	0	0	0	0	0	0
A	0	0	0	0	1	1	1
B	0	1	1	1	1	2	2
C	0	1	1	2	2	2	2
B	0	1	1	2	2	3	3
D	0	1	2	2	2	3	3
A	0	1	2	2	3	3	4
B	0	1	2	2	3	4	4

青色：$X[i]=Y[j]$ 的匹配格
右下角 $dp[7][6]=4$ 即答案

回溯：相等走左上，
否则走向值较大的一侧

图 15-6　$X=$ ABCBDAB 与 $Y=$ BDCABA 的 LCS 表

图里最大的数是右下角的 4。从右下角往回走：$X[7]=$ B 与 $Y[6]=$ A 不等，上方和左方都是 4，任选一个；走到 $(6,6)$ 时 A = A，走左上；如此反复，经过的青色格子对应字符 B, C, B, A，是一个 LCS。BCBA 和 BDAB 都合法，长度都是 4，表只记录长度，不唯一确定是哪一个。

```
/* dp[MAXM][MAXN] 为全局表，这里只写填表过程 */
int LCSLength(char X[], int m, char Y[], int n) {
    for (int i = 0; i <= m; i++) dp[i][0] = 0;
    for (int j = 0; j <= n; j++) dp[0][j] = 0;
    for (int i = 1; i <= m; i++)
        for (int j = 1; j <= n; j++)
            if (X[i-1] == Y[j-1])          /* 代码下标从 0 开始 */
                dp[i][j] = dp[i-1][j-1] + 1;
            else
                dp[i][j] = (dp[i-1][j] > dp[i][j-1])
                           ? dp[i-1][j] : dp[i][j-1];
    return dp[m][n];
}
```

表有 mn 个状态，每个状态 $O(1)$ 转移，时间 $O(mn)$，空间 $O(mn)$。如果只要长度，可以把二维表压成两行（或一行加一个临时变量），空间降到 $O(\min\{m,n\})$；如果要输出具体的子序列，回溯需要整张表，除非用分治式的 Hirschberg 算法。

0-1 背包与滚动数组

n 件物品，第 i 件重 w_i、价值 v_i，背包容量 W，每件最多选一次，求最大总价值。设 $f[i][j]$ 为“只考虑前 i 件、容量不超过 j”的最大价值。对第 i 件，只有两种选择：

$$f[i][j]=\begin{cases} f[i-1][j], & j<w_i,\\ \max\{f[i-1][j],\ f[i-1][j-w_i]+v_i\}, & j\ge w_i. \end{cases}$$

边界 $f[0][j]=0$，答案是 $f[n][W]$。选第 i 件时，剩下的容量 $j-w_i$ 只能从前 $i-1$ 件里选，所以用 $f[i-1][\cdot]$，不能写成 $f[i][\cdot]$，否则同一件物品会被重复考虑。

把 i 这一维压掉，转移变成

$$f[j]=\max\{f[j],\ f[j-w_i]+v_i\}.$$

一维写法能成立，靠的是内层循环的顺序：j 必须从 W 倒着走到 w_i。原因是 $f[j-w_i]$ 必须还是“前 $i-1$ 件”的旧值；倒序时 $j-w_i<j$，这个位置还没有被本轮更新。正序会先更新 $f[j-w_i]$，再拿它算 $f[j]$，同一件物品就被放了两次。

```
int Knapsack01(int w[], int v[], int n, int W) {
    int f[W + 1];
    for (int j = 0; j <= W; j++) f[j] = 0;
    for (int i = 1; i <= n; i++)           /* 物品下标 1..n */
        for (int j = W; j >= w[i]; j--) {  /* 必须倒序 */
            int t = f[j - w[i]] + v[i];
            if (t > f[j]) f[j] = t;
        }
    return f[W];
}
```



图 15-7　0-1 背包一维数组的更新方向

图 15-7(a) 是更新前的一行，全是 0。倒序先算 $j=10$，用到的 $f[6]$ 还没被本轮改过，仍是 0，得到 5；再算 $j=9,8,\ldots$，每次用到的 $f[j-4]$ 都在更左边，也还没更新。正序先算 $j=4$，把 $f[4]$ 改成 5；算到 $j=8$ 时用 $f[4]$，拿到的是刚写进去的 5，于是 $f[8]=10$，相当于把同一件物品放了两次。

完全背包允许每件物品选任意多次，内层改成 j 从 w_i 正着走到 W。正序用新值正是想要的：$f[j-w_i]$ 已经包含了“再选一件第 i 件”的可能。

0-1 背包的时间是 $O(nW)$，空间 $O(W)$。这里的 W 是容量的数值，不是输入的二进制定长；W 很大时 $O(nW)$ 可能比 $O(2^n)$ 还慢。所以这个算法叫伪多项式算法，不能因为写出了 nW 就认为它是多项式时间。

动态规划的复杂度有一套固定的算法：状态数乘每个状态的转移代价。LCS 有 mn 个状态，每个 $O(1)$，所以 $O(mn)$；0-1 背包有 nW 个状态，每个 $O(1)$，所以 $O(nW)$。空间是表的大小，能滚动就滚动。动态规划与贪心的区别也在这里：动态规划保留所有可能的状态，最后从表里取最优；贪心每一步只保留一个候选，赌它属于某个最优解。贪心更快，但需要证明；动态规划更稳，但表可能很大。

15.4　回溯：在解空间树上走到底再回头

4 皇后问题：在 4×4 棋盘上放 4 个皇后，任意两个不同行、不同列、不同对角线。按行放：第 1 行有 4 个位置可选；选定后，第 2 行只能放在不与第 1 行冲突的列；再选第 3 行……如果某一行没有任何位置可放，就退回上一行，换一个列再试。这个“走不通就退回”的过程叫回溯。

把每一步的选择画成一棵树：根是空棋盘，第 i 层结点表示前 i 行已经放好，边表示第 i 行放在哪一列，叶子是一个完整布局或死路。这棵树叫解空间树，也叫决策树。回溯就是在这棵树上

做深度优先搜索：发现当前结点不可能导出可行解（或更优解）时，剪掉它下面的整棵子树，返回上一层。

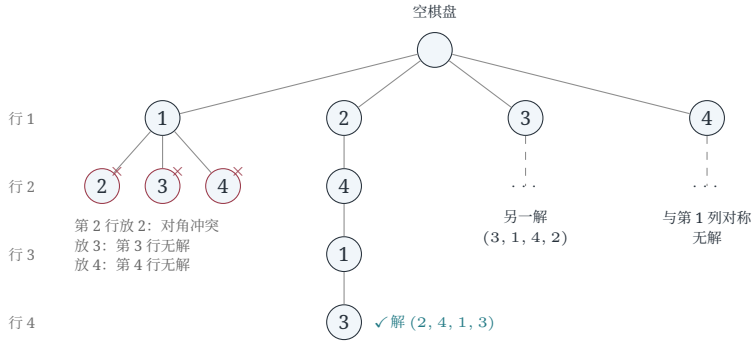


图 15-8　4 皇后的解空间树（只展开前两列的分支）

第 1 行放第 1 列后，第 2 行放第 2 列会与它同对角线；放第 3 列虽然暂时不冲突，但第 3 行没有任何列可放；放第 4 列则第 4 行无路可走。整棵子树被剪掉，回到根结点换第 2 列，得到 $(2,4,1,3)$。第 3 列是第 2 列的镜像，有解 $(3,1,4,2)$；第 4 列与第 1 列对称，无解。图中只画了必要的分支，未展开的用“$\cdots$”表示。

```
#define N 20                                    /* 最大棋盘规模 */
int col[N], d1[2 * N], d2[2 * N], sol[N], cnt;

void Queen(int r, int n) {                      /* 正在放第 r 行，行号从 0 开始 */
    if (r == n) { cnt++; return; }              /* 得到一个完整布局 */
    for (int c = 0; c < n; c++) {
        if (col[c] || d1[r + c] || d2[r - c + n - 1]) continue;
        col[c] = d1[r + c] = d2[r - c + n - 1] = 1;
        sol[r] = c;
        Queen(r + 1, n);
        col[c] = d1[r + c] = d2[r - c + n - 1] = 0;   /* 回溯：撤销选择 */
    }
}
```

`col[c]` 记录第 c 列是否被占，`d1[r+c]` 记录主对角线，`d2[r-c+n-1]` 记录副对角线；放置时置 1，递归返回后置 0，这就是回溯。两条对角线的编号分别用 $r+c$ 和 $r-c+n-1$，后者加 $n-1$ 是为了避免负下标。

最坏情况下，第 1 行 n 种、第 2 行最多 $n-1$ 种……总搜索量 $O(n!)$。实际剪枝后远小于这个上界，但复杂度分析仍写最坏界。空间是递归深度 $O(n)$ 加上三个标记数组和路径数组，仍是 $O(n)$。

子集树与排列树

子集和问题：n 个正整数，问是否存在一个子集，其和恰好为 S。每个元素只有选、不选两种决策，解空间树是一棵满二叉树，叫子集树：第 i 层决定第 i 个元素，左分支选、右分支不选，深度为 n。叶子有 2^n 个，整棵树（含根）有

$$1+2+4+\cdots+2^n=2^{n+1}-1$$

个结点。

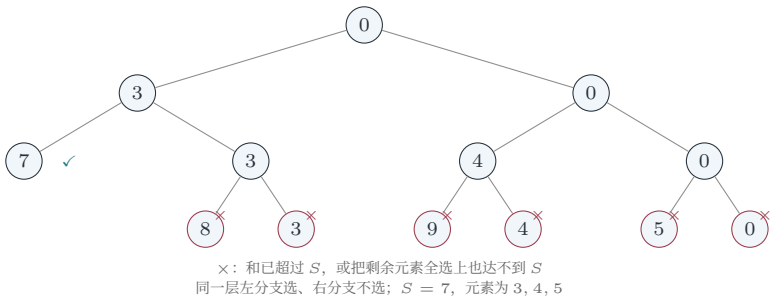


图 15-9　子集和问题的子集树与剪枝

图里只有一个成功结点 7（$3+4$）。其余分支被两类条件剪掉：当前和已经大于 S（元素都是正数，再加只会更大），或者当前和加上剩余所有元素仍小于 S（全选都不够）。这两个条件都只会剪掉不可能有解的子树，不会误伤可行解。

如果问题要求列出所有解，找到一个解后不能停，要继续回溯；如果只要求一个解，找到就可以一路返回。搜索顺序会影响找到解的快慢：把大的数放在前面，选入分支能更早触发“超过 S”的剪枝，不选分支的剩余和也会更小，往往更早被“全选也不够”截断。具体效果取决于数据分布，但正确性不受影响。

排列问题的解空间树是排列树：第 1 层 n 个分支，第 2 层 $n-1$ 个……叶子有 $n!$ 个。全排列、旅行商暴力搜索都属于这一类；n 皇后按行放时每行有 n 个列可选，但列不能重复，结构接近排列树。分析复杂度时先认清是哪一种：子集树 $O(2^n)$，排列树 $O(n!)$。

回溯与蛮力的区别只在剪枝。不剪枝时它就是系统枚举，最坏复杂度不变；剪枝改变的是平均情况和常数，不能把最坏界写成更小。如果要求最优解而不是任意解，剪枝条件还要加上“当前代价已经不可能优于已知最优解”，这类方法常叫分支限界（branch and bound）。408 以回溯为主，知道两者的区别即可。

15.5　复杂度：把账算清楚

算法写完之后，最后一步是回答“它有多快、占多少内存”。408 的算法设计题几乎都会单独留一问给复杂度，这一问写错，前面的代码再漂亮也会扣分。

渐近记号与增长阶

O 是上界：存在常数 $c>0$ 和 n_0，当 $n\ge n_0$ 时 $0\le T(n)\le cf(n)$。Ω 是下界，Θ 是上下界同阶。说归并排序 $O(n^2)$ 没有错，只是浪费了一个更紧的结论；考试问“时间复杂度”时，通常期望给紧界，写 Θ 或 O 都可以，但不要把最坏情况的 O 当成平均情况。对数底数不影响阶：$\log_2 n$ 与 $\ln n$ 只差常数倍，写成 $O(\log n)$ 即可。

常见增长阶与典型场景

阶	典型场景	例子
$O(1)$	与规模无关	顺序表按位取、散列查找（平均）
$O(\log n)$	每步规模折半	二分查找、平衡树查找
$O(n)$	扫描常数遍	顺序查找、Kadane、计数排序（值域 $O(n)$）
$O(n\log n)$	分治合并或用堆	归并排序、堆排序、活动安排排序
$O(n^2)$	双重枚举	冒泡/插入排序、LCS（$n=m$ 时）
$O(n^3)$	三重枚举	Floyd、朴素矩阵乘法
$O(2^n)$	子集枚举	子集和、0-1 背包的枚举解法
$O(n!)$	排列枚举	旅行商暴力、n 皇后最坏

这张表只给数量级。$n=10$ 时 $2^{10}=1024$，$10!\approx 3.6\times 10^6$，差了三个数量级；把 $O(2^n)$ 和 $O(n!)$ 混为一谈，在 n 稍大时就会得出完全错误的结论。

循环与递归的计数

循环的计数先看层数：相互独立的嵌套循环相乘；内层上界依赖外层变量时，写成求和。比如

```
for (int i = 1; i <= n; i++)
    for (int j = 1; j <= n; j += i)
        s++;
```

内层执行 n/i 次，总次数是

$$n\left(1+\frac{1}{2}+\frac{1}{3}+\cdots+\frac{1}{n}\right)=nH_n=\Theta(n\log n),$$

其中 H_n 是调和数，$\ln n<H_n<1+\ln n$。

再看等比型：

```
for (int i = 1; i <= n; i *= 3)
    for (int j = 1; j <= i; j++)
        s++;
```

i 取 $1,3,9,\ldots,3^k$，总次数

$$1+3+9+\cdots+3^k=\frac{3^{k+1}-1}{2}.$$

由 $3^k\le n<3^{k+1}$ 得总次数为 $\Theta(n)$。等比数列看末项，不是看项数：循环执行了 $\Theta(\log n)$ 次，但每次的工作量按几何级数增长，总工作量由最后一次决定。

递归先写 $T(n)$，再解。三种常用方法：递归树（展开看每层）、主定理（标准形式直接套）、代入展开（猜答案再归纳）。常见的递推式和解可以背下来，但要知道每个解是怎么来的：

递推式	解
$T(n)=T(n/2)+1$	$\Theta(\log n)$
$T(n)=T(n-1)+1$	$\Theta(n)$
$T(n)=2T(n/2)+n$	$\Theta(n\log n)$
$T(n)=2T(n/2)+1$	$\Theta(n)$
$T(n)=2T(n-1)+1$	$\Theta(2^n)$
$T(n)=T(n-1)+n$	$\Theta(n^2)$
$T(n)=3T(n/2)+n$	$\Theta(n^{\log_2 3})$

画递归树时注意：结点数按 a^j 增长，每个结点的代价是 $f(n/b^j)$，两者相乘才是该层总代价；不要只数结点。归并排序每层都是 cn，不是因为结点数不变，而是因为结点数翻倍、每个结点的区间减半，两者抵消。

空间与下界

空间复杂度只算辅助空间，不算输入和输出（题目另有约定除外）。递归栈要算：深度乘每帧的局部变量。归并排序的 `tmp` 是 $O(n)$，栈是 $O(\log n)$，合起来 $O(n)$；快速排序原地划分，但最坏递归深度 $O(n)$，栈就是 $O(n)$。如果递归是尾递归，编译器可能优化掉栈，但答题时按未优化算，除非题目说明。

有些问题的复杂度下界不来自某个算法，而来自问题本身。比较排序可以看成一棵决策树：每个内部结点是一次比较，叶子是一种排列。n 个元素有 $n!$ 种排列，所以叶子至少 $n!$ 个；高度为 h 的二叉树最多 2^h 个叶子，故

$$h \geq \log_2(n!) = \Theta(n\log n).$$

任何比较排序都逃不出这个下界，归并排序和堆排序已经达到，所以它们是最优的比较排序。这个论证和回溯里的解空间树不是一回事：前者对所有算法成立，后者只描述某一种搜索过程。

注意

复杂度这一问最常见的失分点：

- 把 $O(2^n)$ 和 $O(n!)$ 混为一谈，或把 $O(nW)$ 当成多项式时间（W 是数值，不是输入长度）。
- 递归栈漏算：归并排序的空间是 $O(n)$ 而不是 $O(1)$，快排最坏是 $O(n)$。
- 循环次数看错：`i *= 2` 的循环执行 $\log n$ 次，不是 n 次。
- 只给最坏情况却标成“平均”，或者反过来。
- 用 O 给松上界（如把 $O(n\log n)$ 写成 $O(n^2)$），虽不算错，但不够准确。

15.6 四条路线怎么选

把前面的内容压成一张表：

问题特征	优先尝试	典型问题
子问题独立、可合并	分治	归并排序、二分查找、最大子段和
每步有局部最优、可交换论证	贪心	活动安排、分数背包、Huffman
子问题重叠、最优子结构	动态规划	LCS、0-1 背包、编辑距离
需要枚举可行解、可剪枝	回溯	n 皇后、子集和、全排列

这张表只是起点。实际题目常常混用：先用贪心排序，再用动态规划；先用分治缩小范围，再用回溯枚举；图上的算法题可能同时用到 DFS 和 DP。判断标准始终是问题结构，而不是题目的外观。线性表、二叉树、图这些结构在第 1 至 5 章已经讲过，本章的路线要套在它们上面用。

答题时，不管用哪条路线，最后都要落到三件事：核心代码（或伪代码）、边界条件、时间与空间复杂度。缺一项，算法设计题就很难拿满分。

16 习题

16.1 基础过关题

1. （单项选择题）某分治算法将规模为 n 的问题划分为 3 个规模为 $n/2$ 的子问题，合并这些子问题的解需要 $O(n)$ 时间。该算法的时间复杂度为

(A) $\Theta(n\log n)$　(B) $\Theta\left(n^{\log_2 3}\right)$

(C) $\Theta(n^2)$　(D) $\Theta(n)$

2. （单项选择题）关于贪心算法，下列说法正确的是

(A) 只要每一步都选取当前看来最好的方案，贪心算法就一定能得到全局最优解

(B) 求解 0-1 背包问题时，按“单位重量价值最大”的顺序优先选取物品，一定能得到最优解

(C) 贪心算法通常需要证明问题具有贪心选择性质和最优子结构

(D) 贪心算法的时间复杂度一定低于动态规划

3. （单项选择题）用一维数组 $f[0\ldots W]$ 求 0-1 背包的最大价值，其中 $f[j]$ 表示背包容量不超过 j 时能获得的最大价值。外层按物品 $i=1,2,\ldots,n$ 枚举，内层枚举容量 j。为保证每件物品至多使用一次，内层应

(A) 令 j 从 0 递增到 W　(B) 令 j 从 W 递减到 w_i

(C) 令 j 从 w_i 递增到 W　(D) 任意顺序均可

4. （填空题）回溯法按深度优先策略搜索解空间树，若判定当前结点不可能导出可行解或最优解，就停止深入它的子树，这一操作称为 _____。用回溯法求解 n 个正整数的子集和问题时，解空间树是一棵 _____，在完全不剪枝的情况下最多有 _____ 个结点（含根结点）。

5.（填空题）阅读下面的程序段，其中 n 为正整数。

```
int s = 0;
for (int i = 1; i <= n; i *= 2)
    for (int j = 1; j <= i; j++)
        s++;
```

语句 `s++` 的执行次数为 _____。若把外层循环改成 `for (int i = 1; i <= n; i++)`，内层不变，则执行次数为 _____。（均用 Θ 记号表示。）

6.（单项选择题）下列问题中，通常不以分治策略作为主要求解框架的是

(A) 归并排序　(B) 有序数组上的二分查找

(C) 快速排序　(D) Dijkstra 单源最短路径算法

16.2 真题风格与综合训练

7.（真题风格・单项选择题）有 7 个活动，占用时间段（左闭右开）如下表所示。

活动	a_1	a_2	a_3	a_4	a_5	a_6	a_7
时间段	$[1,4)$	$[2,5)$	$[3,6)$	$[5,8)$	$[7,10)$	$[8,11)$	$[9,12)$

若按“结束时间最早，且开始时间不早于上一个已选活动的结束时间”的贪心策略安排，最多能安排的活动个数为

(A) 2　(B) 3

(C) 4　(D) 5

8.（真题风格・单项选择题）设 X = ABCB，Y = BDCA，用动态规划求两者最长公共子序列（LCS）的长度，结果为

(A) 1 (B) 2

(C) 3 (D) 4

9. （真题风格・综合题）给定长度为 n 的整数数组 $A[0\ldots n-1]$，其中 $n \geq 1$，元素可正可负。设计分治算法求连续子数组的最大和。假定所有可能的子段和都在 `int` 范围内。要求：**(1)** 说明如何划分问题，以及最大子段跨越中点时如何计算；**(2)** 给出 C 语言实现；**(3)** 写出时间复杂度的递推式并求解，同时说明递归栈空间。

10. （真题风格・综合题）给定 n 个正整数 $a[0\ldots n-1]$ 和目标值 S，数组已按非递增次序排列。设计回溯算法判断是否存在一个子集，其元素之和恰好为 S；若存在，输出其中一个子集。要求：**(1)** 说明解空间树的形态、搜索顺序以及你使用的剪枝条件；**(2)** 给出 C 语言实现，找到一个解即可；**(3)** 分析最坏情况下的时间复杂度和辅助空间。

拓展阅读：

- Cormen, Leiserson, Rivest, Stein《算法导论》第 4 章「分治策略」——递归树、主定理的完整证明与替换法；第 15 章「动态规划」——LCS、背包、最优二叉搜索树的推导；第 16 章「贪心算法」——活动选择与 Huffman 编码的交换论证。
- 邓俊辉《数据结构（C++ 语言版）》第 2 章「向量」与第 5 章「二叉树」——归并排序、二分查找、递归遍历的代码实现；第 6 章「图」——DFS/BFS 框架上的回溯与连通性算法。
- Sedgewick《算法》第 1.4 节「算法分析」——增长阶、倍率实验与内存模型；第 4.3 节「最小生成树」、第 4.4 节「最短路径」——贪心算法在图上的两个完整案例。

PART II　第二部分

计算机组成原理

1 计算机系统概述

先看一行 C 代码：`c = a + b;`。编译器把它变成几条机器指令，处理器从存储器中取出这些指令，译码，从寄存器或主存取操作数，做一次加法，再把结果写回。整个过程只涉及三样东西：指令、数据，以及存放它们的存储单元。本章要回答的是，这三样东西怎样被组织成一台机器，以及这种组织方式怎样决定程序跑多快。

1.1 一条指令在机器里走一遍

1946 年公布的 ENIAC 是第一台通用电子数字计算机，但它没有“存储程序”：计算步骤靠插线和开关设定，换一道题往往要重新接线。1945 年，冯·诺依曼在为 EDVAC 写的报告中提出了另一条路：把指令也当作数据存进存储器，机器按地址逐条取指、译码、执行。EDVAC 到 1951 年才投入运行，而 EDSAC 在 1949 年就已经跑起了存储程序；此后，“存储程序”成为通用计算机的基本模型。

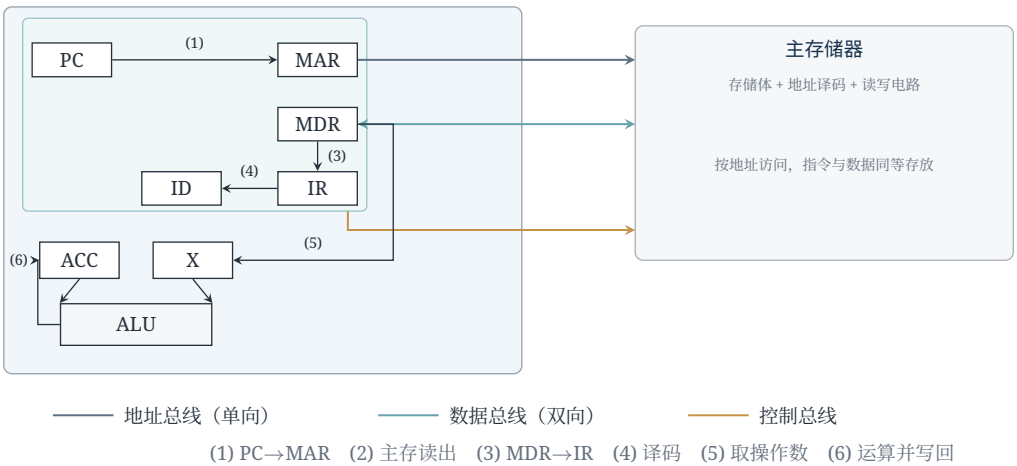
定义 1.1: 冯 • 诺依曼结构

冯·诺依曼结构的基本特征是：

1. 指令和数据都以二进制表示，存放在同一个存储器中，按地址访问；
2. 指令由操作码和地址码组成，指令序列由程序计数器（PC）引导，逐条取出执行；
3. 计算机由运算器、控制器、存储器、输入设备和输出设备五大部分组成；
4. 原始方案以运算器为中心，输入/输出设备与存储器之间的数据传送也要经过运算器。

这里最容易被误解的是“同一个存储器”。它指的是逻辑上统一编址、按地址访问的存储空间，不是说物理上只能有一个存储体。现代处理器把指令 Cache 和数据 Cache 分开，操作系统用虚拟存储把主存和磁盘连成一个地址空间，程序员看到的仍然是一个可以按地址访问的存储空间，所以这些优化并不推翻存储程序模型。

把机箱打开，五大部分的分工可以这样看。运算器负责算术和逻辑运算，核心是 ALU、累加器 ACC、乘商寄存器 MQ 和通用寄存器；控制器负责指挥，核心是 PC、指令寄存器 IR、指令译码器 ID 和时序/控制信号发生器；存储器存放程序和数据，核心是存储体、地址译码器和读写电路，CPU 通过 MAR 送地址、通过 MDR 收/发数据；输入设备把外界信息送进来，输出设备把结果送出去。运算器和控制器合称 CPU，CPU 与主存合称主机，外存和 I/O 设备属于外设。



图里标的是一条加法指令要走的路径。取指阶段，PC 把地址送到 MAR，经地址总线选中主存单元；主存读出内容，经数据总线送到 MDR，再送入 IR；控制器让 PC 指向下一条指令（指令长度可变时加上指令长度）。译码阶段，ID 分析 IR 中的操作码，产生控制信号。执行阶段，如果操作数在存储器中，IR 的地址码再走一次 MAR、主存、MDR，把操作数送到 X 或 ACC；ALU 完成运算，结果写回 ACC 或主存。图中的地址总线是单向的，数据总线是双向的，控制总线传递读/写、中断等信号；实际机器为了并行，往往给指令和数据配不同的总线或 Cache。

> **注意**
>
> 地址总线的位数决定 CPU 能直接编址多少个单元。n 位地址总线最多选择 2^n 个编址单元；若按字节编址，主存最大容量就是 2^n 字节。数据总线的宽度决定一次能并行传送多少位，64 位数据总线一次传 8 字节。MAR 的位数与地址总线位数一致，MDR 的位数通常与存储字长一致。

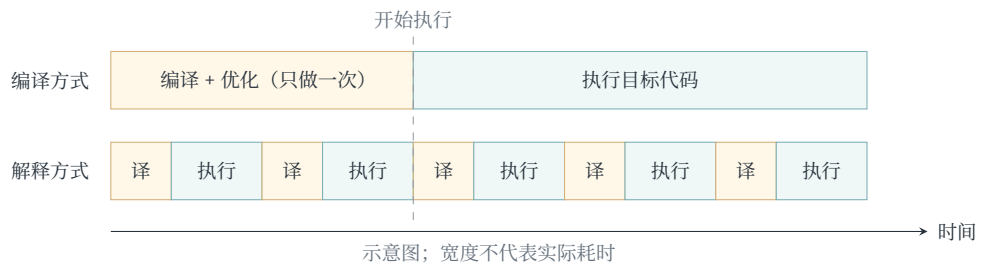
和原始方案相比，现代计算机改成了“以存储器为中心”：CPU 通过系统总线和存储器直接通信，I/O 设备通过 I/O 接口挂到总线上，DMA 方式下数据可以在存储器和 I/O 设备之间直接传送，不必经过运算器。但 CPU 和存储器之间的通路仍然是瓶颈，这就是常说的“冯·诺依曼瓶颈”。后来的 Cache、多级存储层次和更宽的内存总线，很大程度上都是在缓解这条通路的速度差。它说的是带宽限制，不是“内存容量不够”。

从所用器件看，计算机大致经历了电子管、晶体管、中小规模集成电路、大规模与超大规模集成电路四个阶段。1965 年，摩尔（Gordon Moore）提出，集成电路上可经济集成的晶体管数大约每 18–24 个月翻一番，这条经验规律后来被称为摩尔定律（Moore’s law）；它支撑了主频和存储容量的长期增长，但近年已经放缓，单核主频不再快速上升，性能提升更多依靠多核、向量指令和专用加速器。摩尔定律描述的是集成度趋势，不等于处理器性能自动按同一比例增长。

1.2 硬件之上：软件与层次结构

硬件只认识二进制机器指令。要让“$c = a + b$”这样的表达式跑起来，还需要软件把高层写法逐层落到机器指令上。操作系统、编译程序、汇编程序、链接程序、数据库管理系统、设备驱动，这些都属于系统软件：它们要么管理硬件资源，要么为其他程序提供运行环境。浏览器、办公软件、游戏属于应用软件。两者的边界不是绝对的，数据库系统既可以被看作系统软件，也可以被看作应用支撑平台；408 通常把它归入系统软件。

程序设计语言大致分三级。机器语言就是二进制指令，硬件能直接执行；汇编语言用助记符代替操作码和地址码，再由汇编程序翻译成机器语言；高级语言用接近自然语言和数学表达式的写法，由编译程序翻译，或者由解释程序逐条解释执行。编译和解释的差别不在“谁更高级”，而在翻译和执行的先后关系。

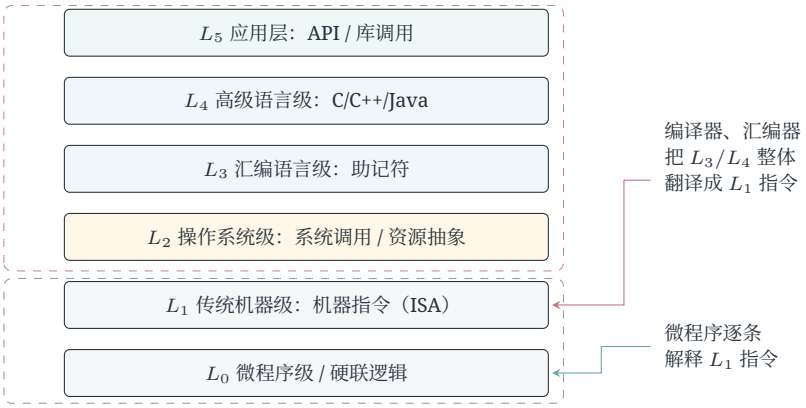


示意图；宽度不代表实际耗时

图的上半部分是编译方式：先把整个源程序翻译成目标程序，之后反复执行都不再付翻译开销，所以适合反复运行的大程序。下半部分是解释方式：读一条语句，翻译并执行，再读下一条，翻译和执行交替出现；启动快、跨平台方便，但每次执行都要重新翻译。Java 先编译成字节码，再

由 JVM 解释或即时编译（JIT），是两种方式的混合。图中把每条语句的翻译宽度画成一样，只是为了对比结构，不代表真实比例。

把“谁在翻译谁”排成层次，就得到常见的六级模型：



虚线框：L_2–L_5 为虚拟机器，L_0–L_1 为实际机器；
RISC 处理器常用硬联逻辑实现 L_0，此时没有微程序。

图里右侧的两条箭头表示上层程序怎样落到下层：编译器和汇编器做的是整体翻译，把一段源程序转换成等价的机器指令序列，再交给硬件；微程序做的是解释，每取到一条机器指令，就用一段微程序里的微指令序列去完成它。图中的虚线框表示层次的虚实：L_0 和 L_1 有对应的硬件，是实际机器；L_2 以上没有专门的硬件，是操作系统和语言处理程序在裸机上构造出来的虚拟机器。

这里要区分两个“层次”概念。一个是上面这种按翻译关系划分的机器层次；另一个是计算机组成里的“体系结构（ISA）与微架构”。指令格式、寻址方式、寄存器个数、数据类型这些对程序员可见的部分属于 ISA；流水线级数、Cache 容量、分支预测器、总线宽度属于微架构，对程序员透明。同一套 ISA 可以有完全不同的微架构，例如同样执行 x86 指令的处理器，流水线深度和 Cache 大小可能差很多。408 选择题常问“哪一项对程序员透明”，Cache 容量和流水线级数通常是透明的那一类，而寄存器个数、指令格式不是。

1.3 性能：指令、周期与时间

衡量性能有两个不同的问题：完成一个任务要多久，以及单位时间能完成多少任务。前者叫响应时间或执行时间，后者叫吞吐率。一台机器可能通过流水线让吞吐率很高，但单个任务从开始到结束的延迟并不短；反过来，降低延迟的措施通常也会改善吞吐率。408 题目里说“性能”，默认比较的是同一程序的执行时间。

CPU 执行时间只统计 CPU 真正花在程序上的时间，不包括等待 I/O 的时间；墙钟时间（elapsed time）则把 I/O 等待、操作系统调度都算进去。题目说“忽略 I/O”时，用 CPU 执行时间即可。把 CPU 执行时间拆开，就得到本章最重要的公式：

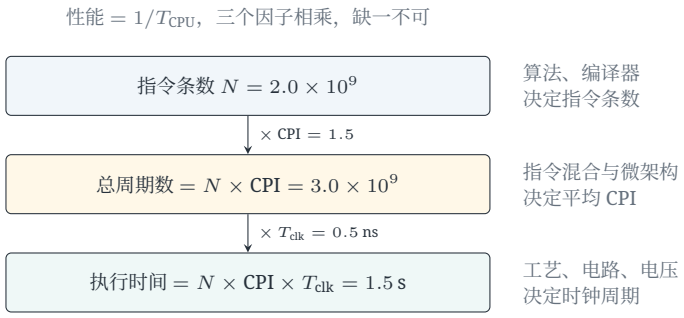
要点 1.1: CPU 执行时间

$$T_{\text{CPU}} = \frac{N \times \text{CPI}}{f} = N \times \text{CPI} \times T_{\text{clk}}, \qquad T_{\text{clk}} = \frac{1}{f}.$$

其中 N 是程序执行的机器指令条数，CPI 是平均每条指令占用的时钟周期数，f 是主频，T_{clk} 是时钟周期。

N 不是源程序的行数，也不是编译器生成的汇编语句数，而是真正被执行的机器指令条数；循环会把它放大，分支会改变它。CPI 也不是指令系统的固定常数，而是这个程序在这台机器上的

平均值。三个因子分别对应三类优化：更好的算法和编译器减少 N，更好的微架构降低 CPI，更好的工艺和电路提高 f。三者不是独立的，提高主频往往要加长流水线或提高电压，可能反过来增加 CPI。

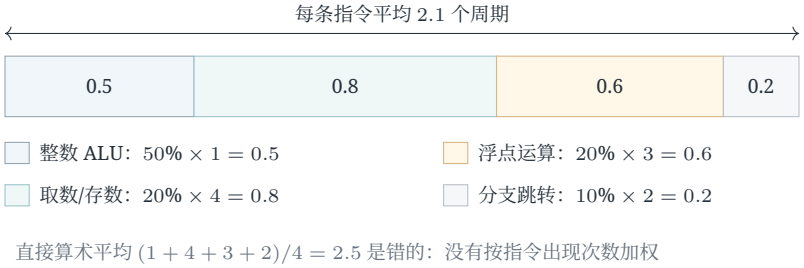


图把一次性能计算拆成两步单位换算：先把“多少条指令”乘上“每条多少周期”，得到总周期数；再乘上“每周期多少秒”，得到时间。以 $N = 2.0 \times 10^9$、CPI $= 1.5$、$f = 2.0$ GHz 为例，总周期数是 3.0×10^9，时钟周期是 0.5 ns，执行时间就是 1.5 s。如果只把主频提高一倍而 CPI 也翻倍，执行时间不会变；这就是不能只看主频的原因。

当程序里有多类指令时，平均 CPI 要按指令条数加权，而不是把各类 CPI 做算术平均。设第 i 类指令占全部指令的比例为 p_i，CPI 为 CPI_i，则

$$\text{CPI}_{\text{avg}} = \sum_{i=1}^{k} p_i \,\text{CPI}_i, \qquad \sum_{i=1}^{k} p_i = 1.$$

等价地，先算总周期数 $\sum_i N_i \,\text{CPI}_i$，再除以总条数 $\sum_i N_i$。两种写法结果相同，题目给“占比”就用前者，给“条数”就用后者。



图中每一段的宽度是 $p_i \times \text{CPI}_i$，也就是这类指令平均给“每条指令”贡献了多少个周期。四段首尾相接，总长度 $0.5 + 0.8 + 0.6 + 0.2 = 2.1$ 就是平均 CPI。取数/存数类指令只占 20%，但 CPI 高，贡献了 0.8 个周期，是四类里最大的一段；这说明优化时不能只盯着出现次数最多的指令。

知道执行时间和指令条数，就能算指令速率：

$$\text{MIPS} = \frac{N}{T_{\text{CPU}} \times 10^6} = \frac{f}{\text{CPI} \times 10^6}.$$

这里 f 用 Hz，分母的 10^6 是把“条/秒”换成“百万条/秒”。如果主频用 MHz 表示，则 MIPS $= f_{\text{MHz}}/\text{CPI}$，不再乘 10^6。浮点性能用 MFLOPS 表示，定义是每秒执行的百万次浮点运算；GFLOPS、TFLOPS 分别是 10^3、10^6 个 MFLOPS。

MIPS 有一个著名的陷阱：它把不同指令当成等价的工作量。RISC 机器的指令简单，完成同一任务往往需要更多指令，MIPS 可能更高，但程序不一定更快；CISC 机器一条指令做更多事，MIPS 可能更低，实际完成的任务却更多。所以 MIPS 只能在同一指令系统、同一程序的条件下比较，不

能跨机器直接比。MFLOPS 也只适合浮点密集型程序，拿它衡量整数或 I/O 密集型负载同样会失真。

实际评测用基准程序（benchmark）。SPEC CPU 让机器运行一组有代表性的程序，把每台机器的执行时间与参考机相比得到一个比值，最后用几何平均汇总。用几何平均而不是算术平均，是因为这些量是比值，几何平均不会被某一个特别快或特别慢的程序过度拉偏。基准程序也有局限：编译器可以针对它做特殊优化，测出来的分数不一定代表真实负载。

1.4 Amdahl 定律：一部分加速能换来多少

假设一个程序原来的执行时间是 T_0，其中比例为 α 的部分可以加速 k 倍，其余部分完全不能加速。改进后的时间是

$$T' = (1-\alpha)T_0 + \frac{\alpha T_0}{k} = T_0\left[(1-\alpha) + \frac{\alpha}{k}\right].$$

整体加速比是

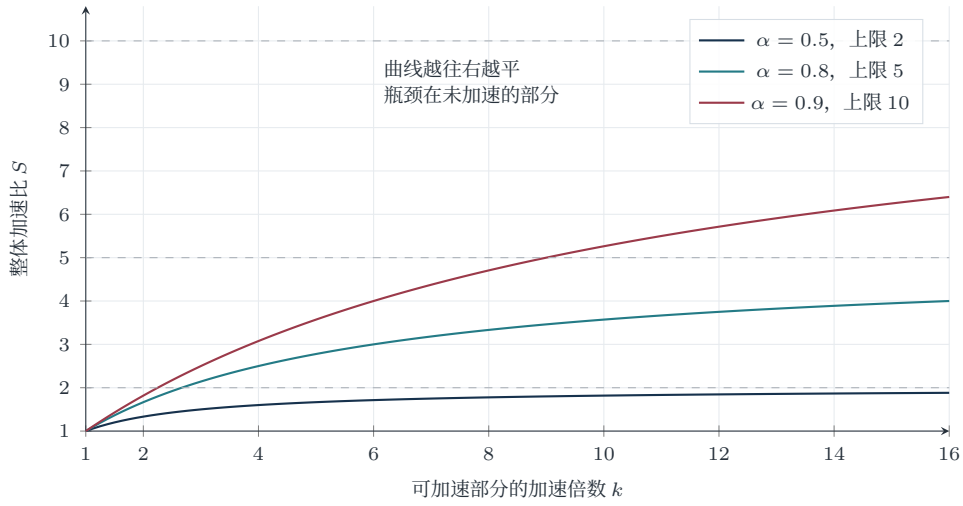
$$S = \frac{T_0}{T'} = \frac{1}{(1-\alpha) + \alpha/k}.$$

当 $k \to \infty$ 时，$\alpha/k \to 0$，加速比的上限是

$$S_{\max} = \frac{1}{1-\alpha}.$$

这个上限只由不能加速的部分决定，和可加速部分能提多快无关。

举个具体的例子：程序原来跑 100 s，其中 80 s 可以加速，$\alpha = 0.8$。把这部分提速 4 倍，改进后是 $20 + 80/4 = 40$ s，加速比 2.5；即使把这部分提速 100 倍，时间也只降到 20.8 s，加速比约 4.8，离上限 5 很近。剩下的 20 s 才是真正的瓶颈。



图里横轴是可加速部分的加速倍数，纵轴是整体加速比，每条曲线对应一个 α。三条虚线是各自的上限 $1/(1-\alpha)$。可以看到，$\alpha = 0.9$ 的曲线在 $k = 16$ 时到 6.4，离上限 10 还很远；$\alpha = 0.5$ 的曲线在同样的 k 下只有 1.88，已经贴近上限 2。曲线变平的速度说明：当可加速部分已经提得很快时，再继续加速它，收益主要被未加速部分吃掉。

用 Amdahl 定律时，α 是“原执行时间中可加速部分的比例”，不是指令条数的比例，也不是代码行数的比例。如果题目给出的是各部分原来的时间，先算它们占总时间的比例，再代入。若

同时改进多个部分，设第 i 部分原时间占比为 α_i、加速倍数为 k_i，则

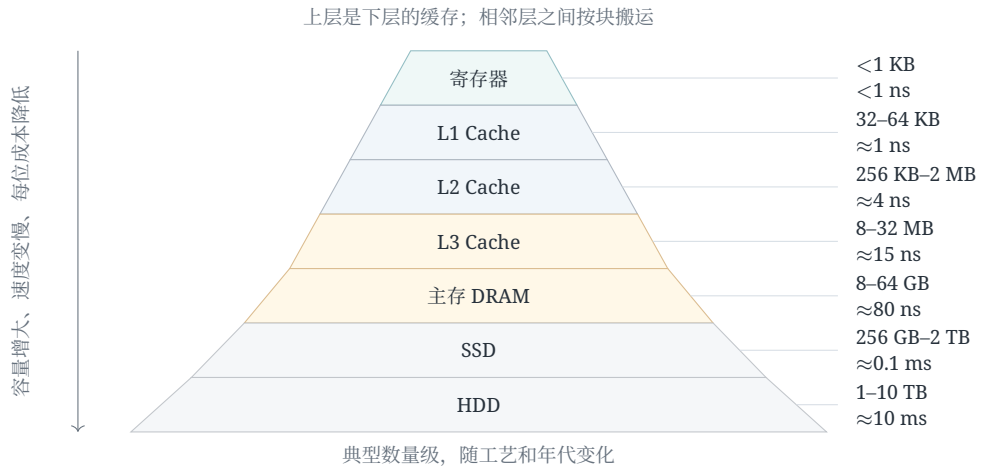
$$\frac{T'}{T_0} = \left(1 - \sum_i \alpha_i\right) + \sum_i \frac{\alpha_i}{k_i}.$$

Amdahl 定律假设问题规模固定。如果问题规模随着处理器数量一起变大（例如更大的矩阵、更多的数据），加速比可以接近线性增长，这时用的是 Gustafson 的观点；408 题目通常采用固定问题规模的版本。

1.5 存储层次：快、大、便宜不能同时要

遍历一个 10000×10000 的整型二维数组，按行访问和按列访问的指令条数差不多，耗时却可能差好几倍。原因不在 CPU 算得慢，而在数据所在的位置：按行访问时，相邻元素在内存中连续，第一次缺失会把一整块数据搬进 Cache，后续访问大多命中；按列访问时，每次跨过一整行，搬进来的块用不上，几乎每次都要访问主存。存储层次要解决的就是这个速度差。

程序访问有两个稳定的规律。时间局部性：刚被访问过的单元，很可能很快再被访问，比如循环变量、栈顶、反复调用的函数。空间局部性：一个单元被访问后，它附近的单元很可能接着被访问，比如数组的相邻元素、顺序执行的指令。存储层次把最近用过的和附近的数据放在快而小的层，把久不用的数据放在慢而大的层，用局部性把“平均访问时间”拉近到快的那一层。



图里越往上容量越小、访问越快、每位成本越高；越往下容量越大、访问越慢、每位成本越低。相邻两层之间不是按一个字节搬运，而是按块搬运：Cache 和主存之间通常以 64 字节左右的 Cache 行（块）为单位，主存和磁盘之间以页为单位。块的存在是为了利用空间局部性，一次把相邻数据都取上来；块太小浪费空间局部性，块太大则每次缺失要搬更多数据，缺失代价上升。

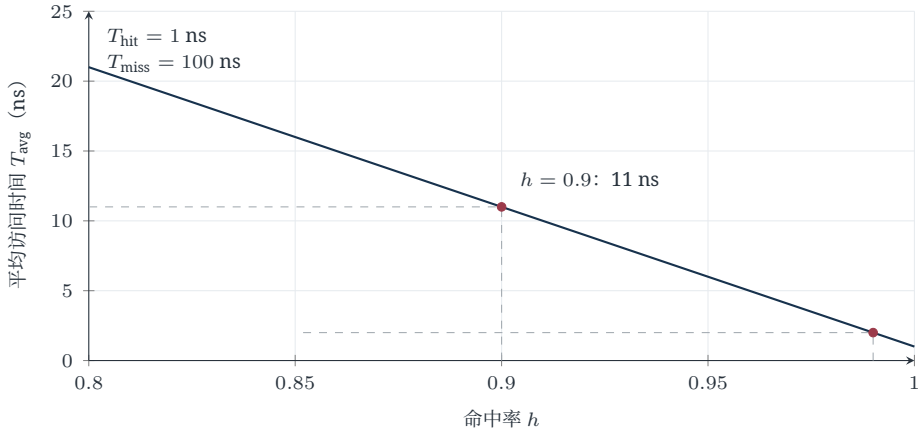
设上层命中率为 h，命中时间为 T_{hit}，未命中时额外付出的时间为 T_{miss}，则平均访问时间

$$T_{\text{avg}} = T_{\text{hit}} + (1 - h) T_{\text{miss}}.$$

如果题目给的是“下层完整访问时间”T_{lower}，则写成

$$T_{\text{avg}} = h\,T_{\text{hit}} + (1 - h) T_{\text{lower}}.$$

两种写法只差 T_{miss} 的定义：前者把 T_{miss} 看作“缺失代价”，即比命中多花的时间；后者把下层访问时间整个算进去。做题时先看题目给的是哪一个，不要混用。



图里假设命中一次只要 1 ns，缺失一次要多花 100 ns。命中率 0.9 时平均访问时间是 $1+0.1\times 100=11$ ns；命中率提高到 0.99，平均时间降到 $1+0.01\times 100=2$ ns，相差 5.5 倍。注意 1% 的缺失率仍然让平均时间翻了一倍：缺失代价比命中时间大两个数量级时，少量缺失就足以主导平均值，所以优化 Cache 命中率通常比单纯提高命中速度更有效。

存储层次对程序员基本透明：不用改程序，Cache 自动工作。但“透明”不等于“没有影响”。数组按行还是按列遍历、循环是否分块、数据结构是否紧凑，都会改变命中率。虚拟存储则是另一层：它把主存当作磁盘的 Cache，用页表完成地址转换，由硬件和操作系统共同管理；Cache 通常由硬件自动管理，操作系统只负责建立页表和在进程切换时维护地址空间。这些内容在第 3 章展开，这里先建立“上层快而小、下层慢而大、按块搬运、靠局部性生效”的整体图景。

1.6 容易混淆的几组概念

1. **CPU、主机与外设。**运算器加控制器是 CPU；CPU 加主存是主机（408 教材的常用口径）；外存和输入/输出设备属于外设。不要把外存算进主机，也不要把“主机”等同于整台计算机。
2. **机器字长、指令字长、存储字长。**机器字长是 CPU 一次能处理的二进制位数，通常等于通用寄存器位数和 ALU 宽度；指令字长是一条指令的位数，可以等于或多个存储字长；存储字长是一个编址单元的位数。三者可以不同，题目给“字长”时要看它修饰的是哪个对象。
3. **主频高不等于速度快。**执行时间由 N、CPI、T_{clk} 三者共同决定。主频提高但 CPI 或指令条数同步上升时，程序可能一点没快，甚至更慢。
4. **CPI 与 IPC。** CPI 是每条指令平均占用的周期数，IPC 是每周期平均完成的指令数，二者互为倒数。理想流水线的 IPC 接近 1；超标量处理器一个周期可以发射多条指令，IPC 可以大于 1，此时 CPI 小于 1。
5. **MIPS 不能跨指令系统比较。** MIPS 把指令当成等价的工作量，不同 ISA 的指令功能差别很大。同一台机器跑不同程序，MIPS 也不同，所以它不是一个机器的固有属性。
6. **Amdahl 定律中的 α。**α 是可加速部分占原执行时间的比例，不是指令条数或代码行数的比例。上限 $1/(1-\alpha)$ 来自不能加速的那部分，换更快的加速部件也突破不了。
7. **编译与解释、翻译与解释。**编译是“先整体翻译，再执行目标程序”；解释是“逐条翻译并执行”。在层次结构里，汇编器和编译器做翻译，微程序解释机器指令；操作系统提供的是系统调用和资源抽象，不是翻译。
8. **存储程序与 Cache 分离。**“指令和数据放在同一存储器”是逻辑模型。指令 Cache 和数据 Cache 分开、虚拟存储把磁盘并入地址空间，都是性能优化，不改变按地址访问、顺序取指的基本模型。
9. **地址总线与主存容量。** n 位地址总线给出 2^n 个编址单元；只有按字节编址时，最大容量才是 2^n 字节。若按字编址，还要乘存储字长。

10. **冯·诺依曼瓶颈**。它指 CPU 与存储器之间通路的带宽限制，不是容量不足。Cache、多级存储层次、更宽的总线和多通道内存都是在缓解这个瓶颈。

拓展阅读：

- Patterson & Hennessy《计算机组成与设计：硬件/软件接口》第 1 章——从程序性能、CPI、Amdahl 定律讲到存储层次，配有大量实测数据，适合把本章公式和真实机器对应起来。
- Bryant & O'Hallaron《深入理解计算机系统》第 1 章——从程序员视角走一遍“源程序到进程”的路径，第 1.9 节讨论 Amdahl 定律和并行性，第 6 章讲存储层次。
- 唐朔飞《计算机组成原理》第 1 章——冯·诺依曼结构、五大部件、指令周期和计算机性能指标的国内教材口径，与 408 的术语更接近。

本章符号速查： N 为执行的机器指令条数；CPI 为平均每条指令的时钟周期数；f 为主频，$T_{\text{clk}} = 1/f$；IPC $= 1/\text{CPI}$；α 为原执行时间中可加速部分的比例，k 为该部分的加速倍数；h 为命中率，T_{hit} 为命中时间，T_{miss} 为缺失代价。

2 习题

2.1 基础过关题

1. (单项选择题) 关于计算机系统的层次结构，下列说法中正确的是

(A) 汇编语言级（L_3）程序由汇编器翻译成机器语言程序后执行，属于解释执行

(B) 操作系统级（L_2）是软件扩充得到的虚拟机器，向上层提供系统调用等抽象

(C) 高级语言级（L_4）程序只能经编译器翻译后执行，不能解释执行

(D) 微程序级（L_0）是虚拟机器，传统机器级（L_1）是实际机器

2. (单项选择题) 某程序在计算机上运行。仅考虑下表中的单项措施，一定能使该程序 CPU 执行时间缩短的是

(A) 主频提高 20%，CPI 与指令条数均不变

(B) 指令条数减少 20%，CPI 增加 25%

(C) CPI 降低 20%，主频降低 25%

(D) 主频提高 10%，CPI 提高 10%

3. (填空题) 某计算机主频为 2.0 GHz，运行某程序共执行 3.2×10^9 条指令，平均 CPI 为 2.5。则该程序的 CPU 执行时间为 _____，该计算机运行此程序时的 MIPS 为 _____。

4. (填空题) 某程序的指令构成如下表。该程序的平均 CPI 为 _____；若处理器主频为 2.3 GHz，则运行该程序时的 MIPS 为 _____。

指令类型	整数 ALU	取数/存数	浮点运算	分支跳转
占比	40%	30%	20%	10%
CPI	1	3	2	6

5. （填空题）某程序在计算机上运行，总时间的 30% 用于可被加速的部分，其余部分不能加速。若把该可加速部分的速度提高到原来的 6 倍，则整个程序的加速比约为 _____；若该部分加速比趋于无穷大，则整体加速比的上限为 _____。

6. （单项选择题）下列关于计算机性能指标的说法中，正确的是

(A) 主频越高的计算机，运行同一程序的执行时间一定越短
(B) MIPS 是机器本身的固有属性，与所运行的程序无关
(C) 同一程序在两台机器上运行，CPU 执行时间较短的那台性能更高
(D) CPI 越小的计算机，运行同一程序的执行时间一定越短

2.2 真题风格与综合训练

1. （单项选择题）机器 A 的主频为 3.0 GHz，运行程序 P 时共执行 2.0×10^9 条指令，平均 CPI 为 1.5。机器 B 采用不同的指令系统，运行同一程序 P 时共执行 1.2×10^9 条指令，平均 CPI 为 2.5，主频为 2.0 GHz。忽略 I/O 等其它时间，则

(A) A 的性能是 B 的 1.2 倍
(B) A 的性能是 B 的 1.5 倍
(C) A 的性能是 B 的 1.8 倍
(D) 两者性能相同

2. （单项选择题）某程序在单处理器上运行的时间中，60% 的部分可以完美并行化，其余 40% 必须串行执行。忽略并行带来的额外开销和负载不均，使用 8 个处理器时该程序的加速比约为

(A) 1.60
(B) 2.11
(C) 2.50
(D) 3.20

3. （真题风格 · 综合题）某处理器的主频为 2.5 GHz，其指令系统包含四类指令，各类指令的 CPI 及在某基准程序中的占比见下表。假设以下各问中程序的总指令条数均为 2.0×10^9 条，且指令混合比例不变。

基准程序的指令构成

指令类型	整数 ALU	取数/存数	浮点运算	分支跳转
CPI	1	2	4	2
占比	40%	30%	20%	10%

(1) 求该程序的平均 CPI，以及在此处理器上运行时的 MIPS。

(2) 求该程序在此处理器上的 CPU 执行时间。

(3) 现做两项改进：把浮点运算的 CPI 由 4 降为 2，同时把主频提高到 3.0 GHz，其余指令的 CPI 和指令构成不变。求改进后的平均 CPI、CPU 执行时间，以及相对改进前的加速比。

4. **（真题风格・综合题）**某程序在当前机器上运行需 10.0 s，时间构成如下：访存停顿 4.0 s，浮点运算 2.0 s，整数运算及其它 4.0 s。假设各部分时间可以独立改进，且改进后程序执行的指令条数与访存次数不变。

(1) 若把访存停顿时间降为原来的一半，把浮点运算时间降为原来的 $1/4$，整数部分不变，求改进后的总执行时间和整体加速比。

(2) 在 (1) 的基础上，若希望整体加速比达到 2.0，整数部分需要加速多少倍？

(3) 若访存和浮点两部分都能无限加速（时间趋于 0），整体加速比的上限是多少？由此说明什么。

3 数据的表示与运算

同样一串 32 位 `1111 1111 1111 1111 1111 1111 1111 1111`，说明符写成 `int` 时它是 -1，写成 `float` 时却落在 NaN 上：阶码全为 1、尾数不全为 0。位串本身没有数值含义，含义来自编码约定。这一章把这些约定拆开来看：整数怎样用补码表示，加减到什么时候算越界，浮点数怎样分配符号、阶码和尾数，以及一个加法器怎样用同一套电路完成加减并给出标志位。

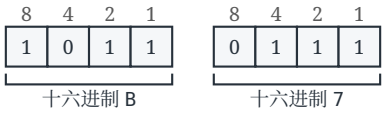
3.1 进位计数制与常见编码

先看一个具体的二进制数：

$$10110111.1011_2 = 1\cdot2^7+0\cdot2^6+1\cdot2^5+1\cdot2^4+0\cdot2^3+1\cdot2^2+1\cdot2^1+1\cdot2^0+1\cdot2^{-1}+0\cdot2^{-2}+1\cdot2^{-3}+1\cdot2^{-4}.$$

这就是“按权展开”：每一位的权是基数的整数次幂，小数点左边从 2^0 起往左递增，右边从 2^{-1} 起往右递减。任意 R 进制同理，第 i 位的权为 R^i。

十进制转 R 进制时，整数部分除 R 取余、余数逆序排列；小数部分乘 R 取整、整数顺序排列。前者先得到低位，所以最后要反过来读；后者先得到高位，顺着读即可。二进制、八进制、十六进制之间不必绕道十进制，按位分组最快：从小数点向两侧，每 3 位对应 1 个八进制位，每 4 位对应 1 个十六进制位，不足补 0。

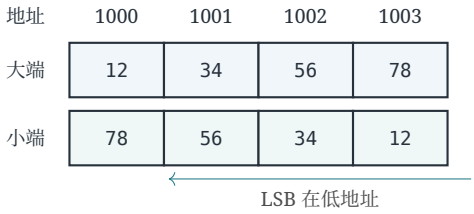


位串 *`1011 0111`* 按 *4* 位切开，两组分别读作 *B* 和 *7*，所以它等于 *B7H*。组内各位的权从左到右是 $8,4,2,1$；分组必须从小数点开始，向两端进行。

同一个位串按 3 位切，则 `010 110 111` 对应八进制 `267`；小数点后的 `1011` 要补成 `101 100`，得到 `.54`。补 0 只能补在两端，不能插在中间。

BCD 是另一套约定：用 4 位二进制表示一位十进制数，而不是表示这个十进制数的二进制真值。十进制 59 的 8421 BCD 是 `0101 1001`，而 59 的二进制是 `0011 1011`，两者不要混。8421 码中 `1010`–`1111` 这 6 个编码没有十进制数字对应，属于非法码。ASCII 则是 7 位字符编码，共 128 个码点，存进字节时最高位通常补 0；数字 `'0'`–`'9'` 和字母 `'A'`–`'Z'`、`'a'`–`'z'` 各自连续，这个连续性常用来做字符与数值的转换。

多字节数据放进内存时还有字节次序问题。设 32 位数 `0x12345678` 从地址 1000 开始存放：



大端把最高有效字节 *12* 放在最低地址 *1000*；小端把最低有效字节 *78* 放在 *1000*。两种方式只改变字节的排列，不改变一个字节内部 *8* 个比特的顺序。

边界对齐是另一条独立规则：一个 n 字节的数据通常要求起始地址是 n 的倍数，这样一次总线访问就能取完。结构体中因此可能出现填充字节；题目若明确说“紧凑存放”，就不能自行加填充。

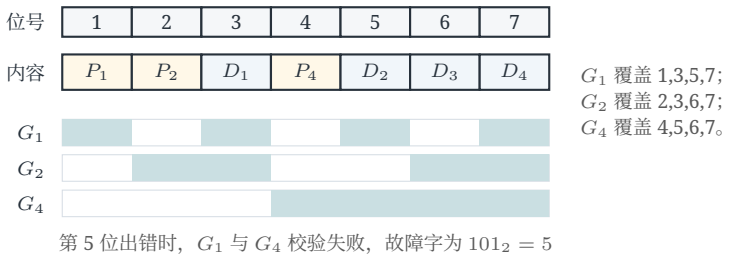
3.2 校验码

数据在存储或传输时可能有一位翻转，最省的办法是增加冗余位。奇偶校验只加 1 位：偶校验让整个码字中 1 的个数为偶数，奇校验让 1 的个数为奇数。接收方重新数一遍，发现奇偶性不对就知道出错了。它只能发现奇数个位错，不能定位，也无法发现两位同时出错。

要定位错误，冗余位必须携带“哪一位错了”的信息。海明码把校验位放在编号为 $1, 2, 4, 8, \ldots$ 的位置，其余位置放数据位。设数据位 k 个、校验位 r 个，故障字要能表示“无错”以及 $k+r$ 个可能出错的位置，所以需要

$$2^r \geq k + r + 1.$$

校验位 $P_1, P_2, P_4, \ldots$ 分别盯住编号二进制中的第 0、1、2、… 位：P_1 覆盖所有奇数编号，P_2 覆盖编号第 1 位为 1 的位置，P_4 覆盖编号第 2 位为 1 的位置。接收方对每组重新计算奇偶性，得到 r 位故障字；故障字为 0 表示没有检出错误，否则它的二进制值就是出错位号。



第 5 位出错时，G_1 与 G_4 校验失败，故障字为 $101_2 = 5$

以 $k = 4$、$r = 3$ 为例，$2^3 = 8 \geq 4 + 3 + 1 = 8$，*7* 个位置刚好够用。$P_1, P_2, P_4$ 放在 *1*、*2*、*4* 位，数据位 D_1–D_4 放在 *3*、*5*、*6*、*7* 位。浅色格表示该位置属于对应的校验组；每个位置被哪些组覆盖，由它的编号二进制决定。

上面 $k = 4, r = 3$ 的编码能纠正 1 位错：若第 5 位翻转，G_1 和 G_4 的奇偶性改变，G_2 不变，故障字为 $101_2 = 5$，直接把第 5 位取反即可。普通海明码的最小距离为 3，只能纠正 1 位错；再增加一位总校验位，就能区分“1 位错”和“2 位错”，构成 SEC-DED（纠一检二）码。

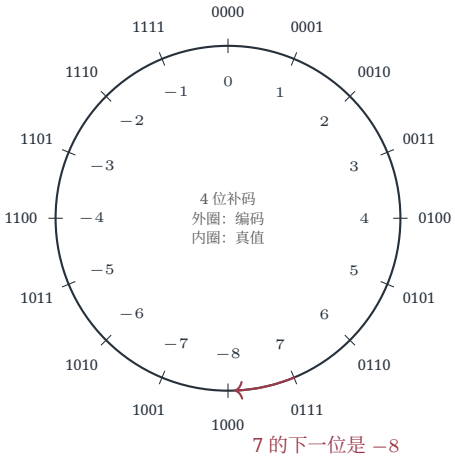
3.3 补码：把减法变成加法

墙上挂钟走到 10 点，往前拨 4 小时是 6 点，往后拨 8 小时也是 6 点。在模 12 的意义下，-4 和 $+8$ 是同一件事。计算机的 n 位寄存器就是一个模 2^n 的计数器，补码正是把这个现象固定成编码规则。

8 位寄存器里，-7 用 249 的二进制 `1111 1001` 表示：$249+7=256$，两者相加会自然溢出丢掉最高位，留下 0。一般地，n 位补码中

$$[x]_{\text{补}} = 2^n + x \qquad (-2^{n-1} \le x < 0).$$

这里的加法都在模 2^n 意义下进行，超出 n 位的进位直接丢弃。把 $2^n - |x|$ 拆成 $(2^n - 1 - |x|) + 1$，而 $2^n - 1 - |x|$ 恰好是把 $|x|$ 的 n 位逐位取反，所以负数补码的实用求法是“绝对值按位取反，末位加 1”，记作 $\sim |x| + 1$。



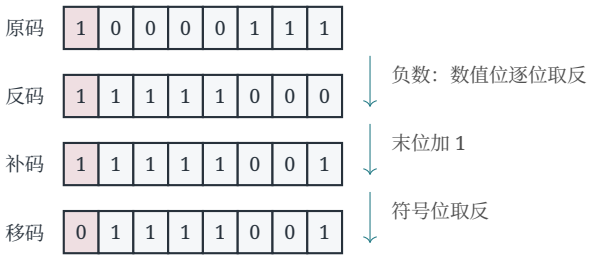
16 个编码按顺时针排列。外圈是 *4* 位二进制，内圈是按补码解释的真值：*0000–0111* 是 0 到 7，接下来 *1000* 不是 $+8$，而是 -8，然后 *1001–1111* 对应 -7 到 -1。顺时针加 *1* 在 *0111* 处跨过符号位，数值从最大正数跳到最小负数。

把位串 $b_{n-1}b_{n-2}\cdots b_0$ 按权展开，补码的数值公式可以写成

$$x = -b_{n-1}2^{n-1} + \sum_{i=0}^{n-2} b_i 2^i.$$

最高位的权是负的 -2^{n-1}，其余位仍是正的。这个式子比“符号位加绝对值”好用：它直接解释了为什么 `1000 0000` 表示 -128，也解释了为什么补码的零只有一种写法——全 0 的按权展开就是 0。

8 位补码的范围是 $[-128, 127]$，比原码和反码多出一个 -128。原码的 `1000 0000` 表示 -0，补码把这个编码征用为 -128；代价是它没有对应的正数，所以 `abs(INT_MIN)` 无法得到可表示的正值。下表把同一个真值 -7 在四种机器数中的写法放在一起：



真值 -7 的 8 位编码（首格为符号位）

红色格是符号位。原码到反码只改数值位；反码到补码是末位加 *1*，进位会向高位传播；补码到移码只把符号位翻转。移码的最高位为 *1* 表示非负，为 *0* 表示负，所以无符号比较移码的大小就等于比较真值的大小。

反码在现代通用计算机里基本不再直接使用，但它的“逐位取反”仍是补码求负的中间步骤；

移码则专门服务于浮点数的阶码，因为浮点运算经常需要比较两个阶码的大小，而带符号的补码不能直接按无符号数比较。

四种编码的对照如下（8 位）：

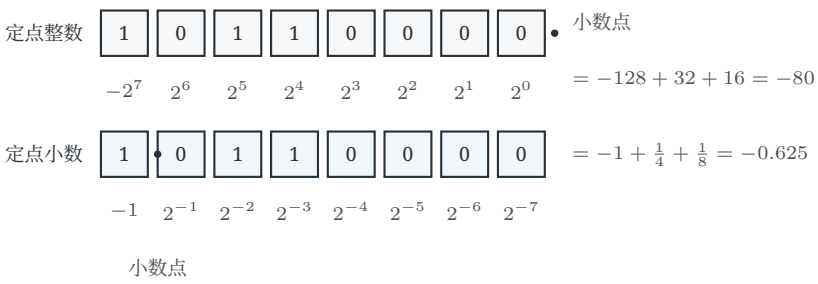
真值	原码	反码	补码	移码
$+7$	`0000 0111`	`0000 0111`	`0000 0111`	`1000 0111`
$+0$	`0000 0000`	`0000 0000`	`0000 0000`	`1000 0000`
-0	`1000 0000`	`1111 1111`	`0000 0000`	`1000 0000`
-7	`1000 0111`	`1111 1000`	`1111 1001`	`0111 1001`
-8	无	无	`1000 0000`	`0111 1000`

8 位移码的偏置量是 $2^7 = 128$，即 $[x]_{\text{移}} = x + 128$。它不是另一套独立的数值系统，只是把补码的符号位翻了过来：补码 `1111 1001`（-7）翻符号位得 `0111 1001`，正是 -7 的移码。

补码扩展位数时，正数在高位补 0，负数在高位补 1，称为符号扩展。8 位的 -100 是 `1001 1100`，扩展到 16 位为 `1111 1111 1001 1100`，即 `FF9CH`；直接在高位补 0 会把它变成 $+156$，这是常见的错误。

3.4 定点数与溢出

定点数就是小数点位置事先约定不变的数。约定小数点固定在最低位之后，就是定点整数；约定它固定在符号位之后、数值位之前，就是定点小数。硬件并不真的存一个小数点，它只是编程者对齐权重的约定。下图用同一串 8 位 `1011 0000` 说明这件事：



同一串 *`1011 0000`*，按定点整数读是 -80，按定点小数读是 -0.625。区别只在符号位和其余位的权：整数最低位权为 2^0，小数最低位权为 2^{-7}。

补码的加减法可以统一成一条式子。设字长为 n，则

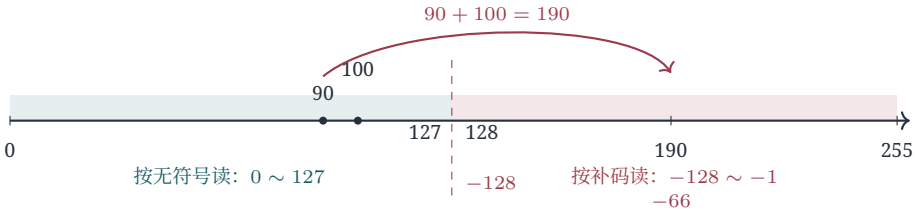
$$[x+y]_{\text{补}} = [x]_{\text{补}} + [y]_{\text{补}} \pmod{2^n}, \qquad [x-y]_{\text{补}} = [x]_{\text{补}} + [-y]_{\text{补}} \pmod{2^n},$$

其中 $[-y]_{\text{补}}$ 是把 $[y]_{\text{补}}$ 连同符号位一起取反再加 1。符号位不需要单独处理，最高位产生的进位直接丢掉。这正是补码比原码省事的地方：减法被改写成了加法。

溢出是另一回事。两个 8 位数 90 和 100 相加：

$$90 = \texttt{0101 1010}, \quad 100 = \texttt{0110 0100}, \quad \texttt{0101 1010} + \texttt{0110 0100} = \texttt{1011 1110}.$$

按无符号数是 190，没有超过 255，结果正确；按补码读，`1011 1110` 是 -66，显然不对，因为 $90 + 100$ 已经超出 $[-128, 127]$。同一个加法，对无符号数没溢出，对有符号数溢出了。



编码 190 落在补码负半区，按有符号数解释就是 $190 - 256 = -66$

横轴是 *8* 位编码 0 到 255。上半轴标无符号值，下半轴标同一编码的补码值；127 与 128 之间的虚线就是符号位翻转的位置。$90 + 100$ 的编码是 ***190***，它在无符号轴上合法，在补码轴上却已经越过边界，落到负半区。

判断溢出有三种等价说法，考场上按题目给的条件选用：

1. **进位判断。**记 C_n 为符号位向外的进位，C_{n-1} 为最高数值位向符号位的进位，则

$$\mathrm{OF} = C_n \oplus C_{n-1}.$$

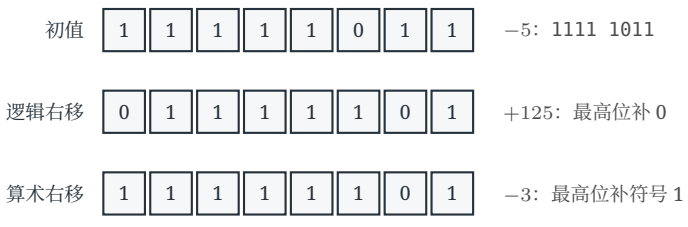
   两个进位不同，说明符号位被“顶”反了。注意 C_n 同时是无符号加法的进位标志 CF。
2. **双符号位。**运算时把符号位扩成两位，正数用 00、负数用 11。结果符号为 00 或 11 正常，01 是正溢出，10 是负溢出。两位符号不同就溢出。
3. **操作数符号。**两个同号数相加，若结果符号与它们不同，则溢出；异号数相加不可能溢出。减法 $x - y$ 看成 $x + (-y)$，再用同一条规则。

上面 $90 + 100$ 的例子中，$C_8 = 0$（没有无符号进位），$C_7 = 1$（最高数值位向符号位有进位），$0 \oplus 1 = 1$，所以 OF 为 1；而 CF 为 0。CF 和 OF 回答的是两个不同的问题：CF 看无符号数是否越界，OF 看补码是否越界。做减法时，加法器的 C_n 为 1 表示“没有借位”，所以借位标志是 $\overline{C_n}$。

四个常用标志位可以放在一张表里：

标志	含义	典型产生方式
CF	无符号进位/借位	加法取 C_n；减法取 $\overline{C_n}$
OF	有符号溢出	$C_n \oplus C_{n-1}$
SF	结果符号	结果最高位
ZF	结果为零	结果各位全 0

移位运算要区分逻辑移位和算术移位。逻辑移位不管符号，空位一律补 0；算术移位要保持符号，右移时空位补原来的符号位。左移两者相同，都是低位补 0。下图以 -5 为例：



最低位在移位中丢弃

初值 ***1111 1011*** 是 -5。逻辑右移一位，最高位补 *0*，得到 ***0111 1101***，即 $+125$；算术右移一位，最高位补符号 *1*，得到 ***1111 1101***，即 -3。$-5/2$ 的精确值是 -2.5，算术右移给出 -3，是向下取整而不是向 *0* 截断。

算术左移在不溢出的前提下等于乘 2；如果移位前后符号位发生变化，就说明结果超出了补码范围。例如 8 位的 $+100$ 左移一位得到 `1100 1000`，按补码是 -56，已经溢出。C 语言中有符号负数的右移在历史上由实现定义，常见平台采用算术右移，但写可移植代码时不应依赖这一点。

3.5 IEEE 754 浮点数

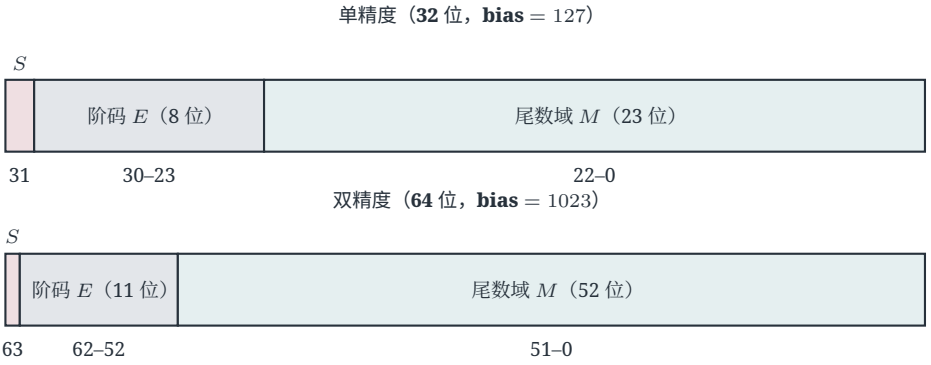
定点数的范围太窄。要同时表示 6.02×10^{23} 和 1.6×10^{-19}，必须让小数点浮动，这就是浮点表示。先看一个具体转换：把 -6.625 写成单精度浮点数。

$6.625 = 110.101_2$，规格化后是 $1.10101_2 \times 2^2$。符号位 $S = 1$；阶码真值 $e = 2$，加上偏置 127 得 $E = 129 = \texttt{1000 0001}_2$；尾数域只存小数点后的 23 位，即 `1010 1000 0000 0000 0000 000`。拼起来：

$$\underbrace{1}_{S}\ \underbrace{1000\,0001}_{E}\ \underbrace{1010\,1000\,0000\,0000\,0000\,000}_{M} = \texttt{C0D4 0000H}.$$

把十六进制展开核对：`1100 0000 1101 0100` 后接 16 个 0，前 9 位正是 S 和 E。

单精度和双精度的字段宽度不同，但含义一致：

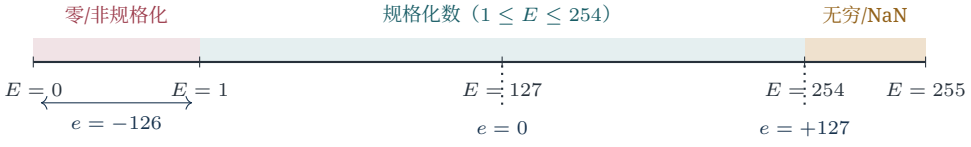


两行画成同样的总宽度，所以双精度的每一位更窄。S 占 *1* 位；E 用移码，单精度 *8* 位、双精度 *11* 位；M 只存规格化尾数的小数部分，小数点前的 *1* 不存，是隐藏位。

规格化数的值为

$$(-1)^S \times (1.M) \times 2^{E-\text{bias}}.$$

单精度的真指数范围是 $-126 \sim +127$，不是 $-127 \sim +128$。原因是 E 的两个端点被特殊值占用了：$E = 0$ 留给零和非规格化数，$E = 255$ 留给无穷和 NaN。于是 $E = 1$ 对应 $e = 1 - 127 = -126$，$E = 254$ 对应 $e = 254 - 127 = +127$。



E 的 *256* 个编码被分成三段。中间的 *1* 到 *254* 是正常规格化数，真指数从 -126 单调增加到 $+127$；两端各留一个编码给特殊值。注意 $E = 0$ 的非规格化数沿用 $e = -126$，只是尾数前不再补 *1*。

特殊编码的完整规定如下：

E	M	含义	数值
全 0	全 0	± 0	$(-1)^S \times 0$
全 0	非 0	非规格化数	$(-1)^S \times (0.M) \times 2^{1-\text{bias}}$
$1 \sim 254$	任意	规格化数	$(-1)^S \times (1.M) \times 2^{E-\text{bias}}$
全 1	全 0	$\pm\infty$	$(-1)^S \times \infty$
全 1	非 0	NaN	不是数

非规格化数填补 0 与最小规格化数之间的空隙。单精度最小规格化正数是 $2^{-126} \approx 1.18 \times 10^{-38}$，最小非规格化正数是 $2^{-149} \approx 1.40 \times 10^{-45}$，两者之间靠尾数域逐级细分。最大规格化数约为 $(2 - 2^{-23}) \times 2^{127} \approx 3.40 \times 10^{38}$。单精度有 24 位有效二进制位（1 位隐藏位加 23 位尾数），约相当于 7 位十进制有效数字；双精度 53 位，约 16 位十进制有效数字。

浮点数不能精确表示大多数十进制小数。0.1 的二进制是 $0.0001100110011\cdots_2$，规格化后为 $1.100110011\cdots_2 \times 2^{-4}$。单精度只能保留 24 位，舍入后尾数域为 `10011001100110011001101`，编码是 `3DCC CCCDH`。它并不等于 0.1，只是最接近 0.1 的可表示值。这就是 `0.1+0.2 != 0.3` 的来源，不是编译器算错，而是二进制浮点的固有精度限制。

舍入发生在尾数被截短的时候。IEEE 754 定义了四种模式：

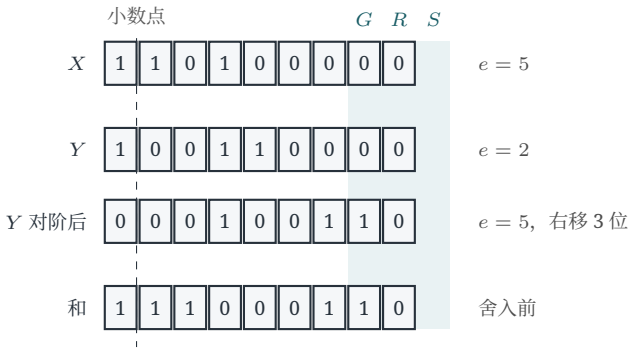
模式	规则
就近舍入到偶数	取最近的可表示值；正好在中间时取最低有效位为 0 的那个
向 $+\infty$ 舍入	总向上取
向 $-\infty$ 舍入	总向下取
向 0 舍入	截断，向绝对值小的方向取

默认是“就近舍入到偶数”。判断“中间”需要额外的位：保护位 G 保存要保留部分之后的第一位，舍入位 R 保存第二位，粘滞位 S 是其余所有位的或。$G = 1, R = 0, S = 0$ 表示恰好一半；$G = 1$ 且 R 或 S 非零表示比一半多；$G = 0$ 表示不到一半。硬件不必真的保存无限多位，三个附加位就足以决定一次舍入。

3.6 浮点加减运算

浮点加减不能直接把尾数相加，因为两个数的阶码通常不同。以 $X = 1.101000_2 \times 2^5$、$Y = 1.001100_2 \times 2^2$ 为例，二者分别是 52 和 4.75。计算分五步：

1. **对阶**。比较阶码，小阶向大阶看齐。这里 $\Delta E = 3$，把 Y 的尾数算术右移 3 位，阶码改成 5。右移时低位不能直接丢，要送进 G, R, S。
2. **尾数加减**。阶码相同后，对尾数做带符号加减。
3. **规格化**。结果可能大于等于 2（需要右规，阶码加 1）或小于 1（需要左规，阶码减 1，可能连续左移多位）。
4. **舍入**。按选定模式处理 G, R, S，舍入本身可能产生进位，若进位到整数部分，还要再做一次右规。
5. **溢出判断**。检查阶码是否超出范围：太大是上溢，结果置 $\pm\infty$；太小是下溢，结果变成非规格化数或 0。



每行前 7 格是“1 位隐藏位 + 6 位尾数”，后 3 格是保护位、舍入位、粘滞位。Y 的尾数 1.001100 右移 3 位变成 0.001001 100：前 6 位小数是 001001，后面 100 进入 G, R, S。X 的尾数后面补 3 个 0，不改变值。

把附加位一起相加：

$$1.101000\,000 + 0.001001\,100 = 1.110001\,100.$$

保留的 6 位小数是 110001，余下 100。$G = 1, R = 0, S = 0$ 表示恰好是一半；按“就近舍入到偶数”，保留部分末位为 1（奇数），要向上进 1，得到 1.110010×2^5，即 57.0。真实和是 56.75，正好夹在 56.5 和 57.0 中间，舍入误差为半个 ulp（0.25）。如果末位本来是 0，就会舍去余下的 100，结果保持偶数。

规格化的两个方向用两个小例子记住：

$$1.111 + 0.001 = 10.000 \;\longrightarrow\; 1.000 \times 2^{e+1},$$

这是右规，尾数和产生进位，右移一位、阶码加 1；

$$1.010 - 1.001 = 0.001 \;\longrightarrow\; 1.000 \times 2^{e-3},$$

这是左规，两个相近的数相减导致高位抵消，要连续左移直到最高位为 1，阶码相应减小。左规可能移很多位，所以浮点减法有时比加法慢。

上溢和下溢的界线要分清。规格化数的阶码超过最大允许值（单精度 $E > 254$）是上溢，结果置 $\pm\infty$；结果的真实指数小于 -126 时进入非规格化区，这是下溢，精度逐渐损失，直到变成 0。下溢不一定立刻变成 0，IEEE 754 的渐进下溢就是靠非规格化数保留这部分信息。

乘除法比对阶简单。乘法把阶码相加再减去一个偏置（$E_x + E_y - \text{bias}$），尾数相乘后规格化、舍入，符号为 $S_x \oplus S_y$；除法把阶码相减再加上偏置（$E_x - E_y + \text{bias}$），尾数相除。除数为 0 时，非零数除以 0 得到 $\pm\infty$，$0/0$ 得到 NaN。

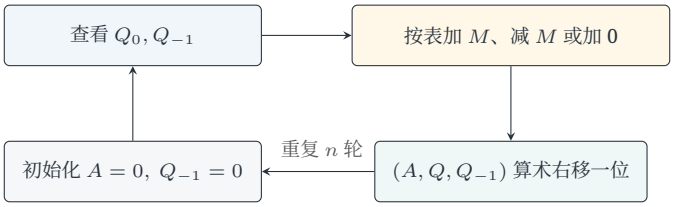
3.7 乘除运算

定点乘法可以用“逐位看乘数、加部分积、右移”实现。原码一位乘把符号位单独异或，数值部分做无符号乘法：乘数最低位为 1 就加被乘数，为 0 就加 0，然后部分积右移一位，重复 n 次，得到 $2n$ 位积。

补码乘法常用 Booth 算法，它不需要先把负数转成原码。做法是在乘数 Q 的最低位右边附加一位 Q_{-1}，初始为 0，每轮看 (Q_0, Q_{-1})：

Q_0	Q_{-1}	本轮操作
0	0	部分积加 0
0	1	部分积加 M
1	0	部分积减 M
1	1	部分积加 0

每轮操作后，把 A, Q, Q_{-1} 作为一个整体做算术右移。它的依据是把乘数中连续的 1 串 $011\cdots10$ 改写成 $100\cdots0-010\cdots0$：一串 k 个 1 的加法可以用一次减法和一次加法代替，所以 1 多的时候反而更快。



Booth 算法一轮的流程。先看 Q_0, Q_{-1} 决定加 M、减 M 还是加 *0*，再把 A, Q, Q_{-1} 联合算术右移一位；循环 n 轮后 $A{:}Q$ 就是 $2n$ 位积。

以 4 位补码 $M=-3=\texttt{1101}$、$Q=+3=\texttt{0011}$ 为例，A 初值 0，$Q_{-1}=0$：

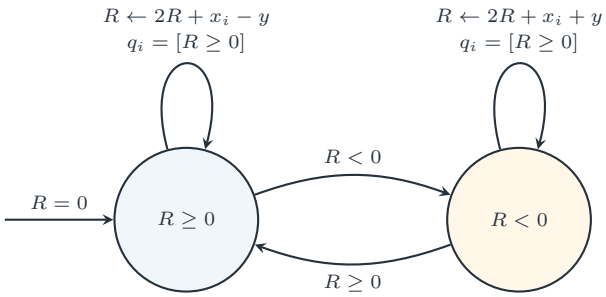
轮次	Q_0Q_{-1}	操作后 A	右移后 A	右移后 Q	说明
1	10	0011	0001	1001	$A \leftarrow A-M$
2	11	0001	0000	1100	不操作
3	01	1101	1110	1110	$A \leftarrow A+M$
4	00	1110	1111	0111	不操作

最终 $A{:}Q=\texttt{1111 0111}$，即 -9，与 $(-3)\times 3$ 相符。

除法也可以用补码或原码做。原码恢复余数法每轮先减除数，如果余数为负就加回来恢复，再上商 0；不恢复余数法（加减交替法）不恢复，而是让负余数继续参与下一轮，下一轮改成加除数。设当前余数为 R，本轮移入被除数的下一位 x_i，规则是：

$$R \leftarrow 2R + x_i, \qquad \begin{cases} R \leftarrow R - y, & \text{上一轮} R \geq 0, \\ R \leftarrow R + y, & \text{上一轮} R < 0, \end{cases}$$

然后根据更新后的 R 的符号上商：$R \geq 0$ 商位为 1，否则为 0。做完 n 位后若余数仍为负，再加一次 y 恢复。这个规则可以画成两个状态：



每轮移入一位被除数并上商一位；n 轮后若 $R < 0$，再执行 $R \leftarrow R + y$

左状态表示当前余数非负，下一轮做减法；右状态表示余数为负，下一轮做加法。两状态之间的转移由更新后的余数符号决定，商位 q_i 在每次更新后立即确定。

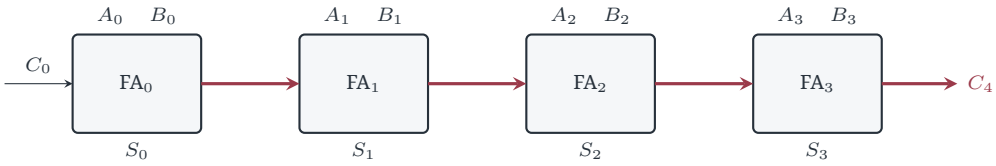
用 $7 \div 3$、3 位商手工走一遍：R 初值 0。第 1 位 $x_1 = 1$，$2R + x_1 = 1$，上一轮 $R \geq 0$ 所以减 3 得 -2，商位 0，转入负状态。第 2 位 $x_2 = 1$，$2(-2) + 1 = -3$，上一轮 $R < 0$ 所以加 3 得 0，商位 1，转回正状态。第 3 位 $x_3 = 1$，$2 \cdot 0 + 1 = 1$，减 3 得 -2，商位 0。最后余数为负，加 3 恢复为 1。商为 $010_2 = 2$，余数 1，与 $7 = 2 \times 3 + 1$ 一致。

3.8 算术逻辑单元

算术逻辑单元（ALU）的核心是加法器。一位全加器有三个输入 A_i, B_i, C_i，两个输出：

$$S_i = A_i \oplus B_i \oplus C_i, \qquad C_{i+1} = A_i B_i + (A_i \oplus B_i) C_i.$$

把 n 个全加器串起来，低位进位接高位进位，就是串行进位加法器。它结构最简单，但最坏情况下进位要从最低位一路传到最高位，延迟与位数成正比。



最坏路径：C_0 依次穿过 4 个全加器，位数越多延迟越大

每个 *FA* 的进位输出接下一个 *FA* 的进位输入。红色路径是 C_0 到 C_4 的最坏延迟路径；结果的 S_i 要等对应进位到达后才能稳定。

先行进位（CLA）把进位提前算出来。定义进位生成和传播

$$G_i = A_i B_i, \qquad P_i = A_i + B_i,$$

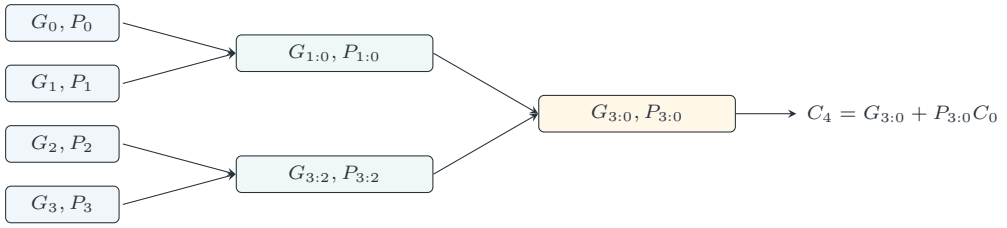
则

$$C_{i+1} = G_i + P_i C_i.$$

反复代入，得到只含 G, P, C_0 的展开式：

$$\begin{aligned}
C_1 &= G_0 + P_0 C_0, \\
C_2 &= G_1 + P_1 G_0 + P_1 P_0 C_0, \\
C_3 &= G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0, \\
C_4 &= G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_0.
\end{aligned}$$

所有进位都只经过两级门（与门算乘积项，或门求和），不再逐级等待。位数一多，展开式的项数会爆炸，所以实际芯片分组处理：先算 4 位一组的组生成 $G_{i:j}$ 和组传播 $P_{i:j}$，再用同样的公式在组间先行进位。



合并公式：$G_{i:j} = G_{i:k} + P_{i:k}G_{k-1:j}$，　$P_{i:j} = P_{i:k}P_{k-1:j}$

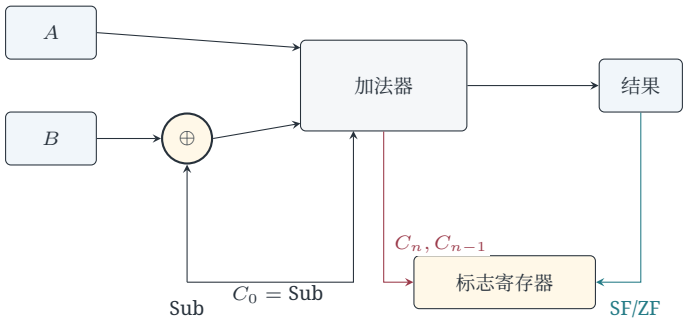
树高 $\lceil \log_2 n \rceil$，同时产生各级前缀可得到 C_1, C_2, C_3

4 位的组生成/组传播树。底层两两合并，得到 $G_{1:0}, P_{1:0}$ 和 $G_{3:2}, P_{3:2}$；再合并成 $G_{3:0}, P_{3:0}$，最后用 $C_4 = G_{3:0} + P_{3:0}C_0$ 得到最高位进位。每一层只做与、或，层数是对数级的。

ALU 还要完成减法、逻辑运算和标志位。减法不另造一套电路，而是把 B 取反后送进加法器，并令最低位进位 $C_0 = 1$：

$$A - B = A + \overline{B} + 1 \pmod{2^n}.$$

控制信号 Sub $= 1$ 时，B 的每一位先与 Sub 异或（等于取反），同时 $C_0 = 1$；Sub $= 0$ 时 B 原样送入，$C_0 = 0$，做加法。



减法数据通路。B 先与 *Sub* 异或：*Sub* $= 1$ 时逐位取反，同时加法器的 $C_0 = 1$；*Sub* $= 0$ 时 B 原样送入，$C_0 = 0$，执行加法。加法器的 C_n 与 C_{n-1} 送标志寄存器产生 *CF/OF*，结果送 *SF/ZF*。*ALU* 的逻辑单元与结果选择 *MUX* 未画出，它们与加法器并联。

标志位的产生逻辑可以写在 ALU 图旁边记：

标志	逻辑
CF	加法取 C_n；减法借位取 $\overline{C_n}$（合并写作 $C_n \oplus$ Sub）
OF	$C_n \oplus C_{n-1}$
SF	结果的最高位
ZF	结果各位或非

这里 C_n 是符号位向外的进位，C_{n-1} 是最高数值位向符号位的进位。减法时 OF 仍用同一个异或式，因为 $A + \overline{B} + 1$ 已经是一个普通的补码加法，溢出规则不变。CF 和 OF 的区别在上一节已经说过：CF 面向无符号数，OF 面向有符号数。

实际的 4 位 ALU 芯片（如 74181）内部带有先行进位，并向外提供组生成 G 和组传播 P 信号；把四片 74181 与一片 74182 先行进位部件级联，就能组成 16 位 ALU。408 考题常要求根据 74181 的功能表判断某组控制信号执行什么运算，或者计算多片级联时的进位延迟，抓住“组内先行、组间先行”这个结构即可。

4 习题

4.1 基础过关题

1. （单项选择题）某机字长 8 位，定点整数采用补码表示。下列叙述中正确的是

   (A) 最小负数为 -127，编码为 `1000 0000`　(B) 最小负数为 -128，编码为 `1000 0000`

   (C) 编码 `1111 1111` 表示 -0　(D) 最大正数为 $+128$，编码为 `0111 1111`

2. （填空题） 8 位补码 `1110 0110` 的真值为 _____；将它符号扩展为 16 位后，得到的 16 位机器数为 _____（用十六进制表示）。

3. （单项选择题） 8 位补码 $[x]_{\text{补}} = \texttt{7BH}$，$[y]_{\text{补}} = \texttt{0AH}$。执行 $x+y$ 后，结果的机器数以及进位标志 CF、溢出标志 OF 分别为

   (A) 85H，CF$=0$，OF$=1$　(B) 85H，CF$=1$，OF$=0$

   (C) 05H，CF$=0$，OF$=1$　(D) 85H，CF$=1$，OF$=1$

4. （单项选择题）下列关于 IEEE 754 单精度浮点数特殊编码的叙述中，正确的是

   (A) `7F80 0000H` 表示最大规格化正数　(B) `FF80 0000H` 表示 $-\infty$

   (C) `0000 0001H` 表示最小规格化正数　(D) `7FC0 0000H` 表示 $+0$

5. （填空题）十进制数 -0.15625 的 IEEE 754 单精度浮点数编码为 _____（用十六进制表示）。

6. （简答题）某 n 位 ALU 用补码加法器实现减法 $A-B$：把 B 的各位取反后送入加法器，并令最低位进位输入 $C_0=1$。说明这样做的原理，并指出此时加法器最高位进位输出 C_n 与无符号减法借位标志的关系。

4.2 真题风格与综合训练

7. （真题风格・单项选择题）在 32 位机器中，某变量的机器数为 `BF80 0000H`。若分别按 `int`（32 位补码）和 `float`（IEEE 754 单精度）解释，则对应的值依次为

(A) -1082130432，-1.0　(B) -1082130432，-1.5

(C) -1073741824，-1.0　(D) -1082130432，-2.0

8. （真题风格・单项选择题）设单精度浮点数 $a=1.0$，$b=2^{-24}$，二者均为规格化数。按 IEEE 754 默认舍入模式（就近舍入到偶数）计算 $a+b$，所得结果的机器数是

(A) `3F80 0000H`　(B) `3F80 0001H`

(C) `3F80 0002H`　(D) `4000 0000H`

9. （综合应用题）设单精度浮点数 X 的机器数为 C128 0000H，Y 的机器数为 4050 0000H。两数均为规格化数，运算采用就近舍入到偶数，并保留保护位、舍入位和粘滞位；本例不涉及上溢、下溢和特殊值。
请完成：(1) 分别写出 X、Y 的十进制真值；(2) 按“对阶—尾数加减—规格化—舍入—溢出判断”计算 $X+Y$，写出中间尾数形式；(3) 给出最终结果的 IEEE 754 单精度编码。

10. （综合应用题）某 8 位 ALU 用补码完成加、减运算。操作数 $A=$ 70H，$B=$ 90H。执行减法时，ALU 将 B 逐位取反后送入加法器，并令最低位进位输入 $C_0=1$。设 CF 为加法器最高位进位输出（减法时按“借位”解释，借位 $=\overline{C_7}$）；OF $=C_7\oplus C_6$，其中 C_6 是最高数值位向符号位的进位；SF 取结果最高位，ZF 在结果 8 位全 0 时为 1。(1) $A+B$ 的结果及 CF、OF、SF、ZF；(2) $A-B$ 的结果及 CF（借位）、OF、SF、ZF；(3) 解释为什么第 (1) 问中 CF$=1$ 而 OF$=0$。

5 存储系统

先看一组数量级。3 GHz 的处理器取一个寄存器操作数不到 1 ns，L1 Cache 命中约 4 个时钟周期，L2 约十几个周期，L3 约几十个周期，主存要上百个周期；到了 SSD，随机读变成几十微秒，机械磁盘的一次寻道则是几毫秒。处理器和存储器之间的速度差不是靠某一种器件就能填平的，实际做法是把不同速度、不同成本的存储器串成层次，让最近用到的数据待在最靠近 CPU 的那一层。

这一章围绕三件事展开：主存怎样由芯片组成、Cache 怎样把主存块留在身边、虚拟地址怎样经页表和 TLB 变成物理地址。三件事共用同一套“块、命中、缺失、替换”的语言，但管理方式不同：Cache 主要由硬件管理，虚拟存储由操作系统和 MMU 协同管理。

5.1 存储器的分类

存储器可以从不同角度分类，408 选择题常把“存取方式”和“器件类型”混在一起考。

存储器的常见分类

分类维度	类型	典型例子与说明
按存储介质	半导体	SRAM、DRAM、ROM、Flash；主存和 Cache 都属于这一类
	磁表面	磁盘、磁带；靠磁化方向记录
	光存储	光盘；靠激光读写
按存取方式	随机存取	存取时间与物理位置无关，如 SRAM、DRAM、ROM
	顺序存取	按物理顺序访问，如磁带
	直接存取	先定位到区域，再在区域内顺序找，如磁盘
	相联存取	按内容比较而不是按地址，如 TLB、Cache 的 Tag 比较
按易失性	易失	断电后信息丢失，如 SRAM、DRAM
	非易失	断电后信息保留，如 ROM、Flash、磁盘
按层次	寄存器 / Cache / 主存 / 辅存	越靠近 CPU 越快、越小、越贵

“随机存取”不等于“RAM”。ROM 也能按地址直接访问，存取时间与位置无关，所以它也是随机存取的；但 ROM 是非易失的只读存储器，不属于 RAM。磁盘是直接存取而不是随机存取：磁头先寻道，盘片再旋转到目标扇区，不同位置的访问时间不一样。相联存取按内容匹配，TLB 用 VPN 比较 Tag、Cache 用地址中的 Tag 比较行，都是相联查找。

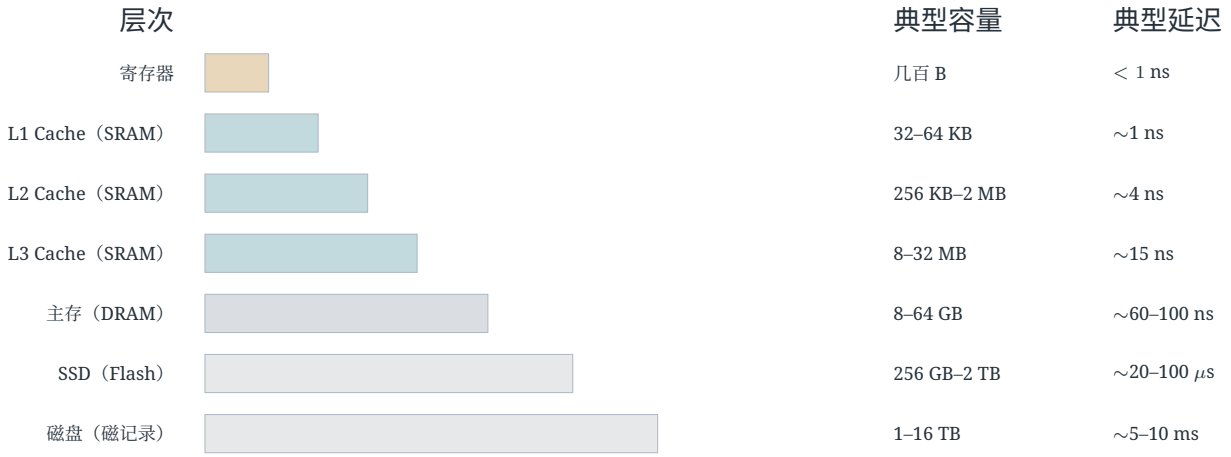
5.2 从一次循环看存储层次与局部性

考虑一段很普通的代码：

```
for (i = 0; i < 8; i++)
    sum += a[i];
```

数组 a 的 8 个 int 连续存放，sum 是一个单独的变量。CPU 执行这段循环时，取指令、读

`a[i]`、读写 `sum`，这些访问并不是均匀地落在整个主存里：指令在循环体内反复执行，`a[i]` 按地址递增，`sum` 每次迭代都访问同一个地址。程序对存储器的访问具有这种“聚集”倾向，存储层次才有意义。



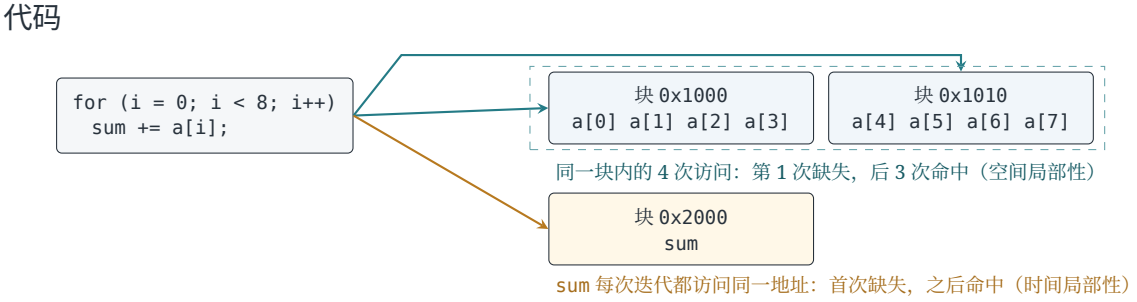
越往下：容量越大、延迟越高、每字节成本越低。数值只是常见量级，随工艺和配置变化。

图里每一层都保存着下一层的一部分内容：寄存器保存当前指令的操作数，L1 保存最近用过的指令和数据块，主存保存当前进程活跃的页，磁盘和 SSD 保存暂时不用的页和文件。上层容量小但快，下层容量大但慢；命中时只付上层的代价，缺失时才把块从下层搬上来。

时间局部性与空间局部性

把上面的循环拆开看，访问模式有两种不同的“聚集”。

时间局部性指一个地址被访问后，不久的将来很可能再次被访问。`sum` 在每次迭代中都被读写，循环体中的指令每次迭代都要重新取指，这些都是时间局部性。**空间局部性**指一个地址被访问后，它附近的地址很可能很快被访问。`a[0]`、`a[1]`、`a[2]` 地址相邻，若 Cache 块能装下 4 个整数，第一次读 `a[0]` 时把整块取进来，后三次就能直接命中。



图里把代码和它访问的块并排画出。左边两条箭头指向数组的两个块，虚线框表示“一个块服务了 4 次访问”；下面那条箭头指向 `sum`，表示同一个地址被反复使用。真实 Cache 的块通常为 32 或 64 字节，这里为了画图缩成 4 个整数。

把地址序列写成 $a_0, a_0 + k, a_0 + 2k, \ldots$，步长 k 越小，空间局部性通常越好。若 k 不小于块大小，每次访问都可能落在新块上，空间局部性就很差。C 语言按行存放二维数组，`a[i][j]` 的地址随 `j` 连续；按行遍历时一个块能服务多次访问，按列遍历时步长等于一行的大小，块内大部分字节用不上。

命中率与平均访问时间

设访问上层的命中率为 h，命中时间为 t_h，缺失后额外付出的时间为 p。这里 p 不含已经花掉的 t_h，只算“发现缺失之后，把块取上来并完成本次访问”的代价。平均访问时间（AMAT）为

$$T_{\text{avg}} = t_h + (1-h)\,p.$$

这个式子把命中检查和缺失处理分开，适合题目给出“命中时间”和“缺失惩罚”的情形。若题目把 t_m 定义为“缺失访问的总时间”，其中已经包含了命中检查，则应写成 $T_{\text{avg}} = h\,t_h + (1-h)\,t_m$。两种口径不能混用，否则会把命中时间重复加一次或漏掉。

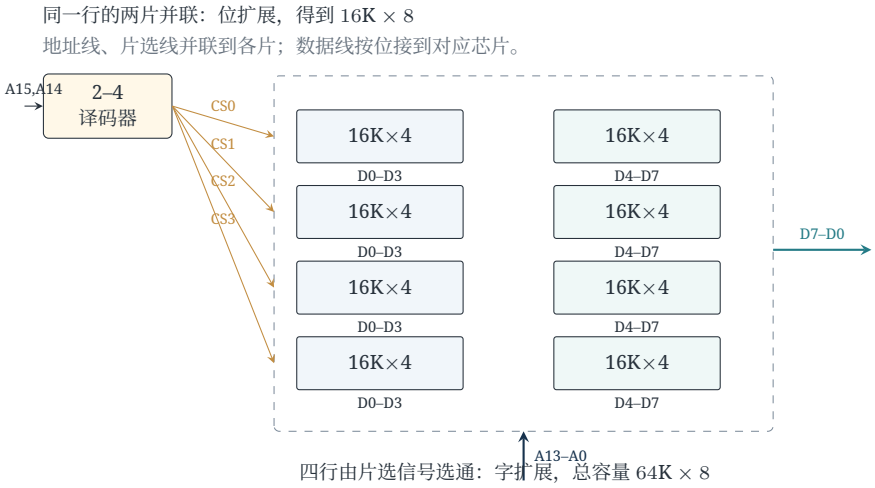
5.3 主存芯片的扩展与多体并行

主存芯片的容量由地址线和数据线决定。地址线有 a 根，片内就有 2^a 个单元；数据线有 d 根，每个单元就是 d 位；芯片容量为 $2^a \times d$ 位。实际系统需要的容量和字长往往与单片芯片不同，于是有位扩展、字扩展和字位同时扩展。

位扩展只增加数据线。用两片 $16\text{K} \times 4$ 的芯片组成 $16\text{K} \times 8$：两片的地址线、片选和读写控制线并联，数据线一片接 D_0–D_3，另一片接 D_4–D_7。地址线数量不变，数据线翻倍。

字扩展只增加地址线。用四片 $16\text{K} \times 8$ 的芯片组成 $64\text{K} \times 8$：数据线并联，高位地址 $A_{15}A_{14}$ 经 2–4 译码器产生四个片选，低位 A_{13}–A_0 送给片内地址。地址线增加 2 根，数据线不变。

两者同时做，就是用 $16\text{K}\times4$ 的芯片组成 $64\text{K}\times8$：共需 8 片，每两片并联做位扩展得到 $16\text{K}\times8$，四组之间再由高位地址选通做字扩展。



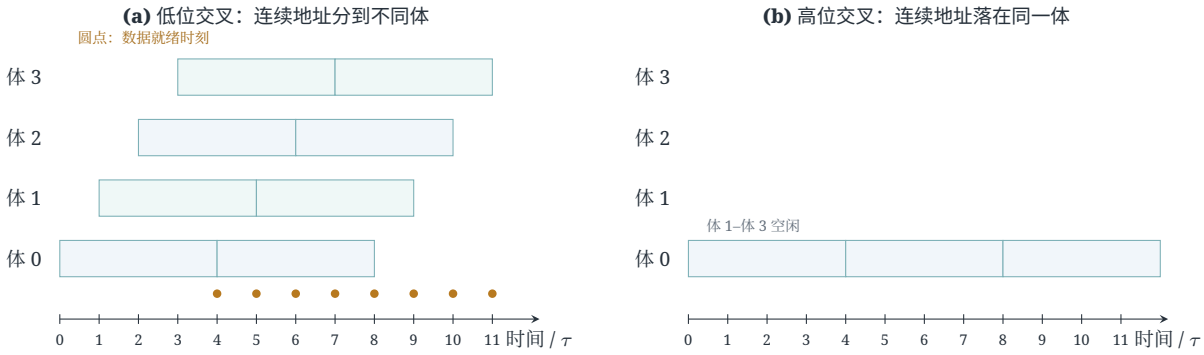
图里把 8 片芯片画成四行两列：每一行是一组，左右两片分别承担低 4 位和高 4 位数据；四行则由 $A_{15}A_{14}$ 译码后的片选信号选通。地址线 A_{13}–A_0 并联到所有芯片的片内地址端，数据线 D_7–D_0 按位连到对应芯片。实际电路还要接读写控制线，图中省略。

高位交叉与低位交叉

单体存储器的带宽受存取周期限制。把主存分成 m 个独立的体，每个体有自己的地址寄存器和数据寄存器，就可以让多个体重叠工作。连续地址怎样分到体，决定了这种并行能不能被顺序访问利用。

高位交叉编址用地址的高位选体，低位作体内地址。连续地址落在同一个体里，顺序程序只能一个体一个体地排队，不能流水；它的长处是不同地址区域可以分布到不同体，适合多个访问流并行。

低位交叉编址用地址的低位选体，高位作体内地址。连续地址依次落在不同的体上，顺序访问可以流水：第一个体还在存取，第二个体已经可以启动。



左图中，体 0 在 0 到 4τ 忙，体 1 在 τ 到 5τ 忙，依此类推；四个体错开 τ 启动。第一个字在 4τ 就绪，之后每隔 τ 出一个字，稳态带宽是单体的 4 倍。右图中，连续地址都落在体 0，体 0 只能一轮接一轮地工作，另外三个体闲着，顺序访问的带宽没有提高。

设体数为 m，体存取周期为 T，总线传输周期为 τ，连续读取 n 个字。第一个字要等 T，此后每隔 $\max(\tau, T/m)$ 出一个字：

$$t = T + (n-1)\max\left(\tau, \frac{T}{m}\right).$$

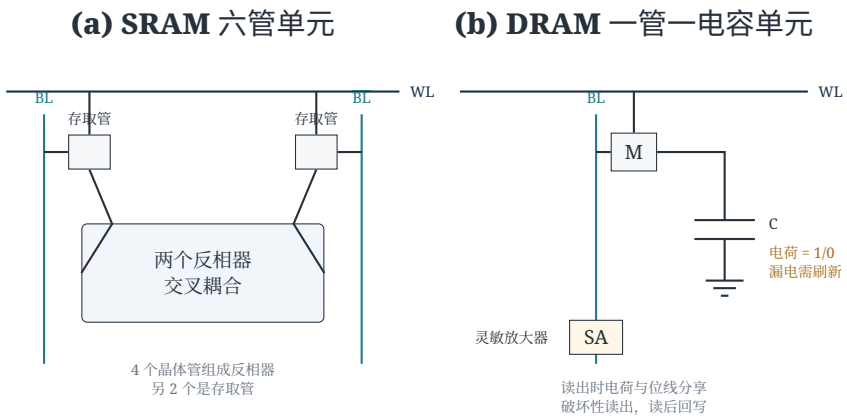
若 $\tau \le T/m$，瓶颈在体周期，稳态带宽为 mW/T；若 $\tau > T/m$，瓶颈在总线，稳态带宽为 W/τ。例如 $m=4$、$T=80\,\text{ns}$、$\tau=20\,\text{ns}$，连续读 6 个字：$t = 80 + 5\times 20 = 180\,\text{ns}$。若改用高位交叉，6 个字都在体 0，忽略总线也要 $6\times 80 = 480\,\text{ns}$。

另一种常见的并行结构是双口 RAM：它有两套地址线、数据线和控制线，两个端口可以同时访问不同的存储单元。若两个端口同时访问同一个单元，同时读是允许的；一读一写或同时写会产生冲突，需要仲裁逻辑决定谁先访问。

5.4　SRAM 与 DRAM 的存储单元

SRAM 的名字里虽然有“静态”，但它并不是只读的。它的存储单元由 6 个晶体管组成：4 个晶体管接成两个反相器，两个反相器的输出又分别接到对方的输入，形成交叉耦合的锁存结构；另外 2 个是存取管，由字线控制，把两个存储节点接到一对位线上。只要不断电，两个反相器就互相维持状态，不需要刷新。

DRAM 的单元简单得多：1 个晶体管加 1 个电容。电容上有电荷表示 1，没有电荷表示 0。电容会通过衬底和晶体管漏电，时间一长电荷就没了，所以必须在电荷漏光之前把每一行读出来、再写回去，这就是刷新。读出时，电容和位线分享电荷，位线电压变化很小，要靠灵敏放大器判断；读出后原来的电荷已经被破坏，必须回写。



图左边是 SRAM 的概念示意：两个反相器首尾相接，把状态锁住；两个存取管像开关，字线有效时把存储节点接到位线，读出或写入。右边是 DRAM：字线打开晶体管，电容和位线连通，灵敏放大器判断位线上的微小变化。DRAM 的读出会破坏电容上的电荷，所以读操作之后紧接着要回写。

SRAM 与 DRAM 的对比

特性	SRAM	DRAM
基本单元	6 个晶体管，两个反相器交叉耦合	1 个晶体管 + 1 个电容
刷新	不需要	需要，按行周期性刷新
读出	非破坏性	破坏性，读后回写
速度	通常较快	通常较慢
集成度与成本	密度低，每 bit 成本高	密度高，每 bit 成本低
典型用途	Cache、TLB、寄存器堆	主存

ROM 家族也常出现在选择题里：MROM 由厂家掩膜固化；PROM 允许用户编程一次；EPROM 用紫外线擦除；EEPROM 用电擦除；Flash 按块擦除、按页编程。NOR Flash 支持按字节随机读，读取快、擦除慢，适合存放代码；NAND Flash 按页读写、按块擦除，密度高，适合大容量数据存储。

SRAM 不需要刷新，但它仍然是易失的；断电以后触发器状态消失，数据一样会丢。DRAM 需要刷新，也不等于“断电后还能保存”，它同样需要持续供电，只是刷新解决的是电容漏电，而不是断电问题。

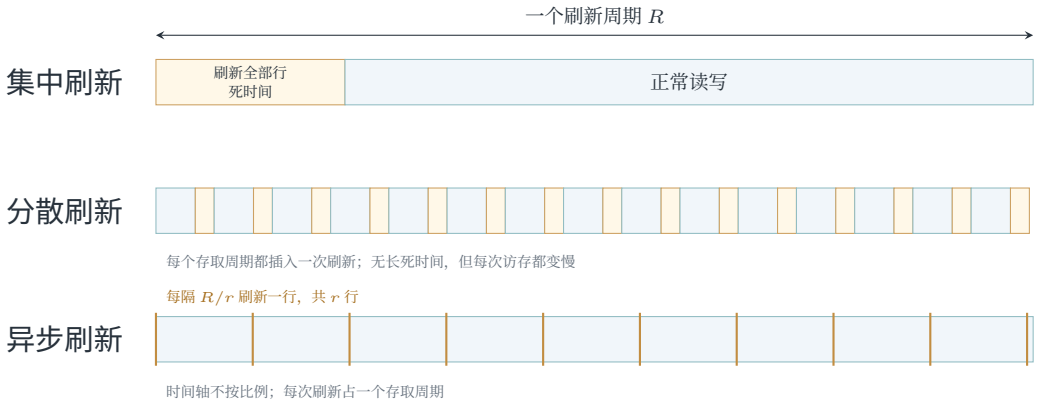
DRAM 的三种刷新方式

设芯片有 r 行，刷新周期为 R（数据保持时间），一次刷新占一个存取周期 t_R。刷新是按行进行的：每次选一行，读出并回写，刷新计数器指向下一行。

集中刷新把一个刷新周期内的一段时间专门留给刷新，逐行刷新 r 行，死时间为 $r\,t_R$。这段时间 CPU 不能访问存储器，优点是控制简单，缺点是有一段明显的长停顿。

分散刷新把刷新拆到每个存取周期里：每个周期先读写，再刷新一行。这样没有长死时间，但每个存取周期都被拉长，刷新开销均摊到每一次访问上。

异步刷新把 r 次刷新均匀铺在 R 内，每隔 R/r 刷新一行。它既避免了集中刷新的长停顿，也不像分散刷新那样每个周期都插一次刷新；如果某次刷新和 CPU 访问冲突，CPU 等一个存取周期即可。



图把三种方式画在同一条时间轴上，蓝色段是正常读写，橙色段是刷新。集中刷新在周期开头留下一块很长的橙色区域；分散刷新把橙色小块均匀塞进每个存取周期；异步刷新则把刷新点稀疏地铺满整个周期。图中的长度不代表真实比例，重点是三种方式的“停顿形态”不同。

举个数值例子：某芯片有 4096 行，刷新周期 $R = 64\,\text{ms}$，一次刷新占 $50\,\text{ns}$。集中刷新的死时间为 $4096 \times 50\,\text{ns} = 204.8\,\mu\text{s}$，占 $64\,\text{ms}$ 的 0.32%；异步刷新的平均间隔为 $64\,\text{ms}/4096 = 15.625\,\mu\text{s}$。集中刷新的死时间虽然占比不大，但它是一整段连续停顿；异步刷新把同样的开销拆成了很多个单周期停顿。

存取时间、存取周期与带宽

存取时间是从给出地址到数据在数据线上稳定的时间；**存取周期**是两次独立存储器操作之间必须间隔的最小时间。DRAM 读出后要恢复电荷、位线要预充，所以存取周期大于存取时间。SRAM 也有周期与时间的差别，只是差距通常小一些。

带宽是单位时间内传出的数据量。单体的连续访问中，若数据宽度为 W 字节、存取周期为 T，则 $\text{BW} = W/T$。多体低位交叉且总线跟得上时，稳态带宽可以提高到 mW/T，这一点在上一节已经算过。

5.5 Cache 的块、地址划分与映射

CPU 访问主存时，先看 Cache。Cache 与主存之间按块（block）交换，CPU 与 Cache 之间按字交换。Cache 中存放一个主存块的槽位叫行（line），行的大小等于块大小。一次访问要回答三个问题：这个块在不在 Cache 里？如果在，在哪个组、哪一行？块内的哪个字节是想要的？

把主存地址切成三段，这三个问题就分别有了着落：Tag 说明“是哪一个块”，Index 说明“落在哪一组”，Offset 说明“块内第几个字节”。

设主存地址为 n 位，块大小为 B 字节，Cache 数据区容量为 C 字节，行数为 $L = C/B$，相联度为 W，组数为 $S = L/W$。则

$$\text{Offset} = b = \log_2 B, \qquad \text{Index} = s = \log_2 S, \qquad \text{Tag} = n - s - b.$$

块号等于主存地址右移 b 位。直接映射时 $W = 1$、$S = L$，索引就是行号；全相联时 $W = L$、$S = 1$，没有索引；组相联介于两者之间。注意 C 是数据区容量，不是整个 Cache 的 SRAM 容量；后者还要加上 Tag、有效位等开销。



图里三行对应同一块 16 KB 数据区。直接映射有 256 行，索引 8 位，Tag 18 位；2 路组相联有 128 组，索引 7 位，Tag 19 位；全相联把所有行放进一个组，没有索引，Tag 26 位。块内偏移始终是 6 位，因为它只由块大小决定。相联度提高以后，索引少了一位，Tag 就多了一位，用于比较的位数和比较器数量也随之增加。

直接映射

直接映射规定每个主存块只能进入唯一的一行：

$$\text{行号} = (\text{块号}) \bmod L.$$

硬件只需要一个比较器：用索引选中一行，把该行保存的 Tag 和地址中的 Tag 比较，相等且有效位为 1 就命中。好处是简单、快；坏处是冲突缺失。若两个经常访问的块模 L 同余，它们会反复争同一行，即使 Cache 里还有空行也用不上，这种现象叫颠簸（thrashing）。

组相联映射

组相联先把 Cache 分成 S 个组，每组 W 行：

$$\text{组号} = (\text{块号}) \bmod S.$$

块只能进入唯一的一组，但可以放在该组的任意一行。查找时，索引选中组，组内 W 个 Tag 并行比较；命中后由多路选择器选出数据。$W = 1$ 退化为直接映射，$W = L$ 退化为全相联，所以组相联是两者的折中。

全相联映射

全相联没有索引，主存块可以进入任意一行。查找时所有行的 Tag 一起比较，冲突缺失最少，但比较器和选择逻辑最贵。大容量 Cache 很少做全相联，TLB 这类小容量结构则常用全相联或高组相联。

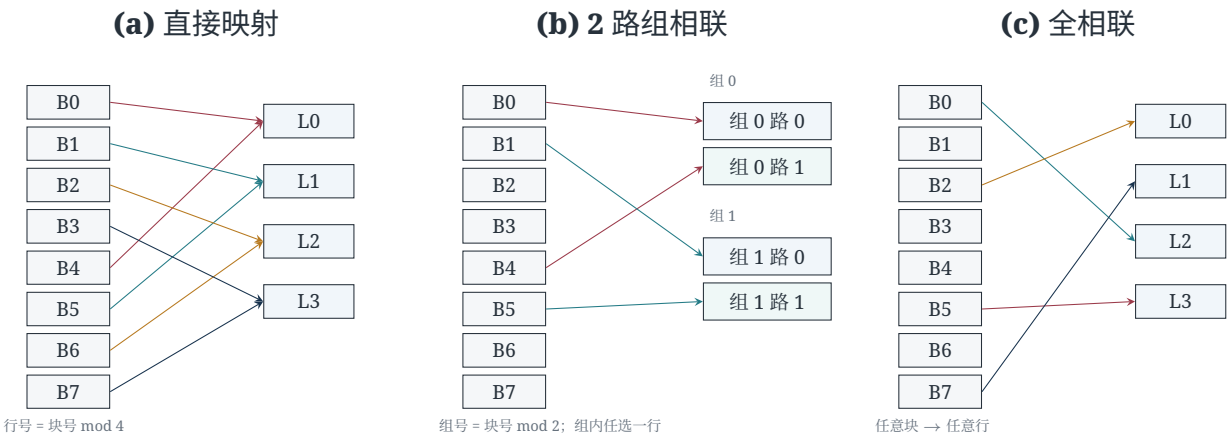


图 (a) 中同色的两个块（B0 与 B4、B1 与 B5 等）映射到同一行，交替访问就会互相替换；图 (b) 中 B0 和 B4 同属组 0，但组内有两行，可以同时留住；图 (c) 中任何块都能进任何行。三张图的主存块数相同，Cache 行数也相同，差别只在“块能落在哪些行”这一约束上。

三种映射方式的对比

	直接映射	组相联	全相联
索引位数	$\log_2 L$	$\log_2(L/W)$	0
Tag 位数	$n-\log_2 L-b$	$n-\log_2(L/W)-b$	$n-b$
比较器	1 个	W 个	L 个
冲突缺失	最多	较少	最少
硬件成本	最低	中等	最高
典型用途	大容量、低功耗 Cache	通用 Cache	TLB、小容量 Cache

Cache 的总容量

题目说“Cache 容量”时，通常指数据区容量；问“Cache 占多少 SRAM”时，要把额外信息也算进去。每行除数据外还要存 Tag 和有效位；写回法还要脏位；组相联还要替换状态。若每行数据 B 字节、Tag 为 t 位、有效位 1 位、脏位 1 位、替换状态 r 位，则

$$\text{总位数} = L \times (8B + t + 1 + \text{脏位} + \text{替换位}).$$

替换状态占多少位取决于实现。题目若说“采用每行 $\lceil \log_2 W \rceil$ 位的 LRU 计数器”，就按这个算；若题目不计替换位，就不要自己加。以 16 KB 数据区、64 B 块、2 路组相联、32 位地址为例：行数 256，Tag 19 位；若写回且每行 1 位 LRU，则每行 $512+19+1+1+1=534$ 位，总容量 $256\times 534 = 136\,704$ 位 $= 17\,088$ 字节 $= 16.6875$ KB，比 16 KB 数据区多出 0.6875 KB，约 4.3%。

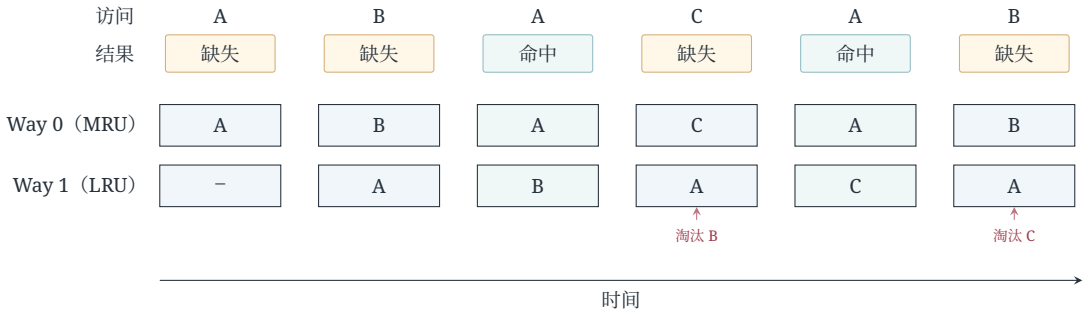
5.6 Cache 的替换算法

组内没有空行时，必须选一个牺牲行。替换算法要回答“谁最不可能再用”。它只影响缺失率，不影响命中时的查找速度。

LRU（Least Recently Used） 淘汰最久没有被访问的行。每访问一行，就把该行标记为“最近使用”；2 路组相联只需要 1 位就能区分两行的新旧，W 路可以用每行 $\lceil \log_2 W \rceil$ 位的年龄计数器，或者用栈式结构维护顺序。LRU 符合时间局部性，但精确实现的开销随相联度上升，实际硬件常用伪 LRU 或 Clock 近似。

FIFO（First-In First-Out）淘汰最早装入的行。命中不会改变装入顺序，所以一个很早装入但一直频繁使用的块也可能被换出。FIFO 实现简单，但缺失率通常高于 LRU，而且可能出现 Belady 异常：增加页框/行数，缺失次数反而变多。

随机替换不看历史，随机选一行。硬件最简单；在高相联度、大容量 Cache 中，它与 LRU 的差距往往不大。



同一访问序列：LRU 命中 2 次、缺失 4 次；若用 FIFO，A 命中后不调整顺序，C 会淘汰 A，最后只有 1 次命中。

图里每一列是一次访问，上格是最近使用的行，下格是最久未使用的行。访问 A 命中后，A 被提到 Way 0，B 退到 Way 1；访问 C 缺失时淘汰 Way 1 的 B；再访问 B 时又淘汰 Way 1 的 C。对比 FIFO 可以看出差别：FIFO 在 A 命中时不改变装入顺序，A 仍然是最早装入的，下一次缺失就被换出，尽管它刚刚被用过。

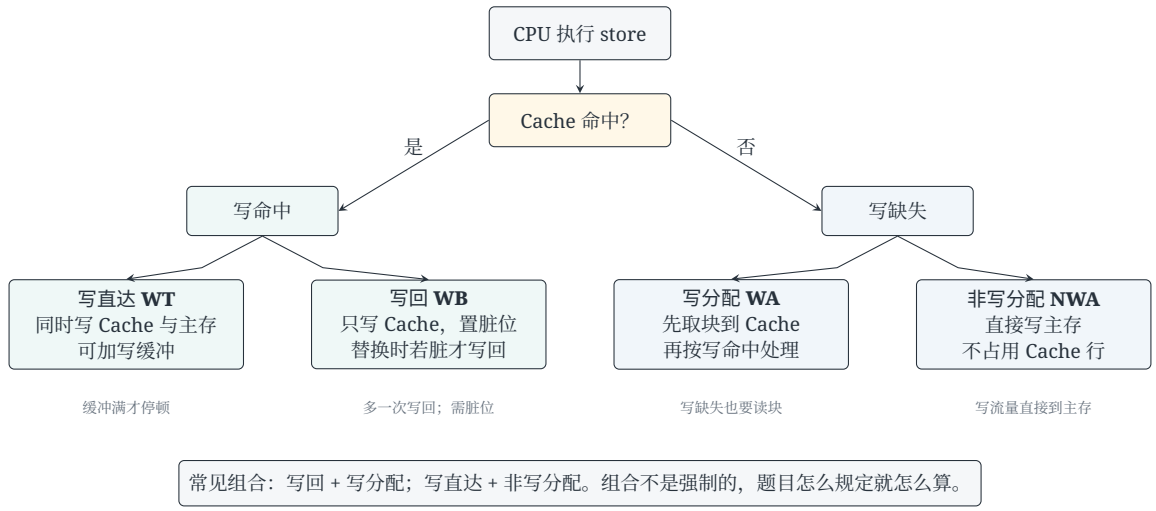
5.7 Cache 的写策略与性能

写操作比读操作多一层麻烦：Cache 里的副本改了，主存里的副本怎么办？如果写命中，有两种选择。

写直达（Write-Through，WT）同时更新 Cache 和主存。主存的副本会被更新，控制简单，不需要脏位；缺点是每次写都要访问主存，速度受主存限制。为了不让 CPU 干等，可以在 Cache 和主存之间加一个**写缓冲**（FIFO）：CPU 把数据交给缓冲就继续执行，缓冲满了才停顿。读缺失时要先查写缓冲，防止读到尚未写入主存的旧值。

写回（Write-Back，WB）只更新 Cache，并把该行标记为脏。主存暂时是旧的，等到这一行被替换时才写回。写回减少了主存写次数，适合写操作密集、局部性好的程序；代价是每行要多一个脏位，替换脏行时多一次写回，多核环境下还要处理缓存一致性。

如果写缺失，也有两种选择。**写分配（Write-Allocate，WA）**先把块从主存调入 Cache，再按写命中处理；它假设写入的地址附近还会被访问，利用了空间局部性。**非写分配（Write-No-Allocate，NWA）**不调入块，直接把字写入主存；它避免了“为了写一个字而读一整块”的开销。



图上半部分先判断命中，下半部分再选策略。写直达和写回是“命中时怎么更新主存”的选择；写分配和非写分配是“缺失时要不要把块搬进来”的选择，两组概念不要混在一起。

平均访问时间

设命中率为 h，命中时间为 t_h，缺失后额外付出的时间为 p，则

$$T_{\text{avg}} = t_h + (1-h)p.$$

两级 Cache 的情况，只有 L1 缺失才会进入 L2：

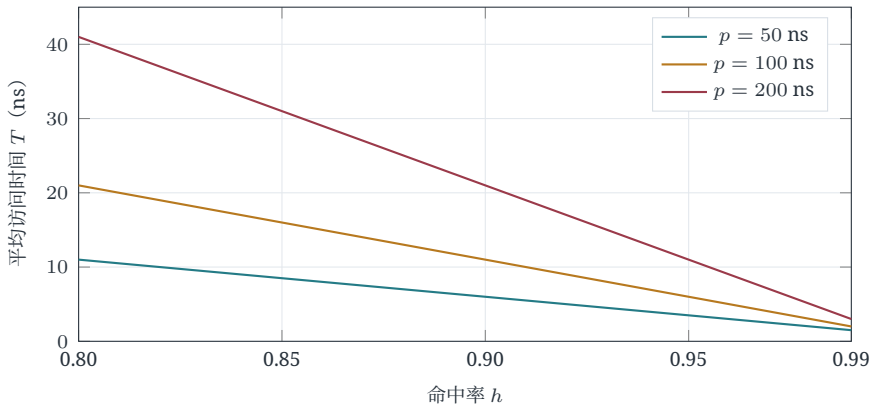
$$T_{\text{avg}} = t_1 + (1-h_1)\big(t_2 + (1-h_2)p\big).$$

这里 h_1 是 L1 命中率，h_2 是“在 L1 已缺失的条件下”L2 的局部命中率；t_2 是 L1 缺失后访问 L2 并回填的额外时间，p 是 L2 也缺失后访问主存的额外时间。若题目给的是 L2 的全局命中率 g_2，则 $g_2 = (1-h_1)h_2$，要先由它求出局部命中率 h_2，不能把 g_2 直接当成 h_2 代入。

例如 $t_1 = 1\,\text{ns}$、$h_1 = 0.97$、$t_2 = 8\,\text{ns}$、$h_2 = 0.85$、$p = 150\,\text{ns}$：

$$T_{\text{avg}} = 1 + 0.03\big(8 + 0.15 \times 150\big) = 1 + 0.03 \times 30.5 = 1.915\,\text{ns}.$$

若命中率降到 0.95，则 $T_{\text{avg}} = 1 + 0.05 \times 30.5 = 2.525\,\text{ns}$，增加了约 32%。缺失惩罚越大，命中率下降一点带来的代价就越明显。



图中三条曲线对应不同的缺失惩罚 p，命中时间固定为 1 ns。p 越大，曲线越陡：在 $h = 0.90$ 附近，$p = 200$ ns 的访问时间已经是 $p = 50$ ns 的约 3.5 倍。多级 Cache 的意义也在这里：L1 追求很低的 t_1，L2 用较大的容量把“L1 缺失后还要走多远”压下来。

5.8 虚拟存储器与页表

Cache 解决的是“主存太慢”，虚拟存储器解决的是另外几件事：每个进程都希望看到一块连续、独立、足够大的地址空间；主存装不下所有进程的全部代码和数据；程序不应该关心物理内存实际有多少、别的进程放在哪里。办法是把主存当作外存的缓存：进程使用虚拟地址，MMU 在运行时把它翻译成物理地址，只有当前活跃的页驻留主存，其余页留在可执行文件、映射文件或交换区里。

这套机制和 Cache 很像：页相当于块，主存相当于 Cache，外存相当于下一级；命中叫页命中，缺失叫缺页。区别在于缺页由操作系统处理，代价是外存 I/O；替换算法也由操作系统选择，而不像硬件 Cache 那样由硬件自动完成。

页式虚拟存储器

页式管理把虚拟地址空间和物理地址空间都切成固定大小的页。设页大小为 2^b 字节，虚拟地址 n 位，物理地址 m 位，则

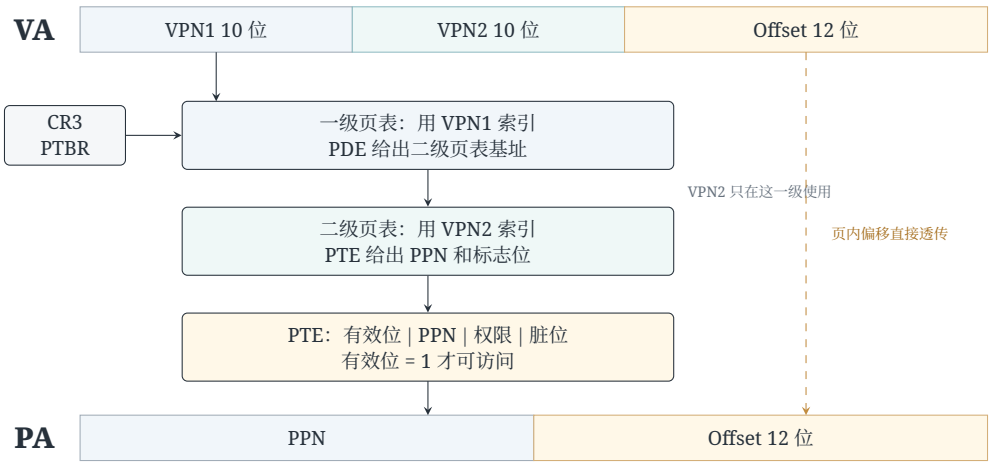
$$\text{VA} = \text{VPN} \,\|\, \text{VPO}, \qquad \text{PA} = \text{PPN} \,\|\, \text{PPO},$$

其中 $\text{VPO} = \text{PPO} = b$ 位，$\text{VPN} = n - b$ 位，$\text{PPN} = m - b$ 位。页内偏移在翻译前后不变，因为页是对齐的；变的只是页号。

页表是操作系统为每个进程维护的一张表，索引是 VPN，表项（PTE）里通常有：有效位（这一页在不在主存）、物理页框号 PPN、访问权限（读/写/执行）、脏位（是否被写过）、访问位（是否被访问过），以及不在主存时指向外存位置的字段。页表基址寄存器（PTBR，x86 中叫 CR3）指向当前进程的页表；进程切换时要换基址，TLB 也要相应失效或打上地址空间标识。

单级页表要为每个虚拟页准备一个表项，表的大小随虚拟地址空间线性增长。以 32 位虚拟地址、16 KB 页、4 B 表项为例：页内偏移 14 位，VPN 18 位，共 2^{18} 项，页表占 $2^{18} \times 4\,\text{B} = 1\,\text{MB}$。这还只是一个进程、一张表；如果页更小、地址空间更大，单级页表会迅速膨胀。更关键的是，大多数进程只用地址空间的一小部分，单级页表却要为未使用的区域也保留表项。

多级页表把 VPN 再切开，用一级表指向二级表，只给实际使用的区域分配二级表。以 32 位地址、4 KB 页、两级页表为例，VPN 的 20 位可以拆成 $10 + 10$：高 10 位索引一级表，低 10 位索引二级表。每张表 1024 项，每项 4 B，正好 4 KB，可以放在一页里。一级表的表项（PDE）给二级表的基址，二级表的表项（PTE）给物理页框号。未使用的二级表根本不用建，省下的空间就是多级页表的意义。



两级页表把 20 位 VPN 拆成 $10+10$；每级 1024 项 $\times$ 4 B = 4 KB，正好一页。

图从上往下是一次页表走查：VA 的高 10 位选中一级表项，得到二级表基址；中间 10 位选中二级表项，得到 PPN；最后把 PPN 和原来的 12 位页内偏移拼成 PA。虚线表示偏移不参与翻译。页表本身放在主存里，所以一次 TLB 缺失可能要多访问几次主存；这正是下一节 TLB 要解决的问题。

5.9 TLB 与 Cache 的协同

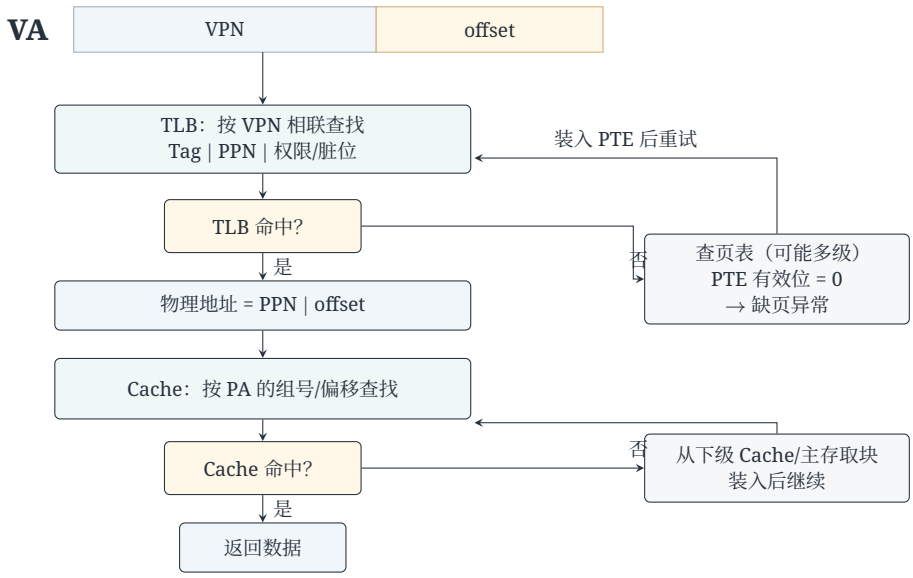
页表在主存里，如果每次访存都先查页表、再取数据，时间要翻倍甚至更多。TLB（Translation Lookaside Buffer）是页表的专用 Cache，缓存最近用过的页表项。它通常采用全相联或高组相联，容量很小，命中时间可以压到一两个时钟周期。

TLB 的每个表项包含：Tag（虚拟页号 VPN 的全部或高位）、PPN、权限位、脏位等。查找时用 VPN 去比较 Tag；命中就直接得到 PPN，和页内偏移拼成物理地址；缺失时查页表，把 PTE 装入 TLB，再重新执行这次访问。

TLB 的覆盖范围（reach）是它能同时映射的地址空间大小。全相联时

$$\text{TLB reach} = \text{表项数} \times \text{页大小}.$$

组相联时表项总数不变，reach 仍然是总表项数乘页大小，只是索引方式不同。例如 64 个表项、4 路组相联、页大小 4 KB：组数为 $64/4 = 16$，组号 4 位；若 VPN 为 20 位，则 Tag 为 $20 - 4 = 16$ 位。reach 为 $64 \times 4\,\text{KB} = 256\,\text{KB}$。



TLB 命中且 Cache 命中时，本次访问不访问主存；TLB 缺失不等于缺页，只有 PTE 有效位为 0 才触发缺页。

图的主路径是：VA 先查 TLB，命中得到 PA，再用 PA 查 Cache。右侧两条分支分别是 TLB 缺失和 Cache 缺失：TLB 缺失要查页表，可能走多级页表；Cache 缺失要从下级取块。两条分支都不要和“缺页”混为一谈：TLB 缺失只是页表项没缓存，查页表就能补上；缺页是页表项的有效位为 0，必须由操作系统把页从外存搬进来。

TLB 与 Cache 的四种组合

TLB	Cache	本次访问的主要动作
命中	命中	地址翻译和数据都在片上完成，最快
命中	缺失	用 PA 查 Cache 失败，从下级取块
缺失	命中	查页表得到 PA，再查 Cache；页表可能多级
缺失	缺失	查页表 + 取块，最慢

物理寻址、虚拟寻址与 VIPT

Cache 用虚拟地址还是物理地址来索引和比较，会影响 TLB 是否在关键路径上。

物理索引物理标记（PIPT）先用 TLB 把 VA 翻译成 PA，再用 PA 的索引选组、用 PA 的 Tag 比较。地址翻译必须在 Cache 查找之前完成，TLB 在关键路径上；好处是没有别名问题。

虚拟索引虚拟标记（VIVT）直接用 VA 的索引和 Tag，不需要等 TLB。问题是同一个物理页可能被多个虚拟地址映射（同义/别名），不同 VA 可能在 Cache 中留下两份副本；进程切换时也要处理整个 Cache。现代通用处理器很少用纯 VIVT 做大容量 L1。

虚拟索引物理标记（VIPT）用 VA 的低位做索引，用 PA 的高位做 Tag。因为页内偏移在 VA 和 PA 中相同，只要索引位完全落在页内偏移之内，索引就不依赖 VPN；TLB 翻译和 Cache 选组可以同时开始，翻译完成后再用 PPN 比较 Tag。这样既避开了 VIVT 的别名问题，又把 TLB 从索引的关键路径上挪开。条件是

$$\text{索引位数} \le \text{页内偏移位数}.$$

例如页大小 4 KB（偏移 12 位）、Cache 有 64 组（索引 6 位），索引完全在页内，VIPT 安全；

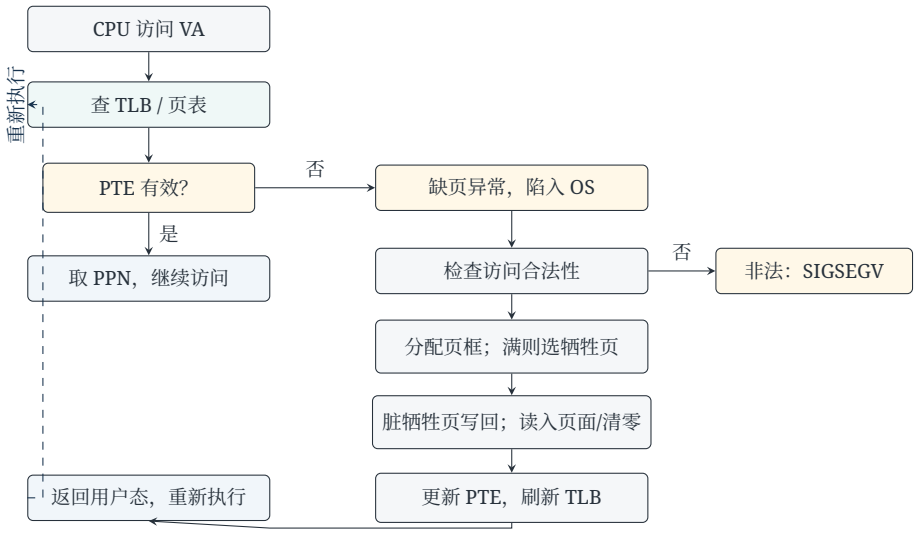
若 Cache 组数多到索引超过 12 位，索引就会依赖 VPN，可能出现同一物理页映射到不同组的情况，需要额外的别名处理。

5.10 缺页异常与页面置换

页表项的有效位为 0，说明这一页不在主存。CPU 访问它时触发缺页异常（page fault），转入操作系统。缺页不是“Cache 缺失”那种由硬件自动回填的小事，它要分配页框、可能写回脏页、读外存，代价高得多。

缺页处理的主要步骤：

1. 保存现场，进入内核；根据出错地址找到对应的虚拟页。
2. 检查这次访问是否合法。合法但页面不在主存，才继续处理；非法访问（越权、地址未映射）通常终止进程。
3. 在主存中找一个空闲页框。若没有空闲页框，按页面置换算法选一个牺牲页；若牺牲页是脏的，先写回外存。
4. 从可执行文件、映射文件或交换区读入所需页；对于按需清零的页（如未初始化的数据段），直接清零，不必读外存。
5. 更新页表项：填入 PPN，置有效位，按需要更新访问位/脏位；同时失效或更新 TLB 中对应的旧项。
6. 返回用户态，重新执行引起缺页的那条指令。因为缺页属于 fault 类异常，处理后同一条指令可以正常完成。



缺页是 fault，处理后重新执行同一条指令；Cache 缺失通常由硬件处理，不陷入 OS。

图把“命中路径”和“缺页路径”分开：左边是正常的地址翻译和数据访问，右边是缺页后操作系统的处理栈。非法访问从“检查合法性”分出去，不会走到分配页框。处理完以后，操作系统返回用户态，重新执行那条指令；此时页表已经更新，TLB 也已经刷新。

页面置换算法

主存没有空闲页框时，要选一个页换出。常见算法有四类：

- **OPT（最优置换）**：淘汰未来最长时间不会再被访问的页。它需要预知未来，无法真正实现，通常作为理论下界，用来评价其他算法。

- **FIFO**：淘汰最早进入主存的页。实现简单，但不管页最近有没有被访问；可能出现 Belady 异常——增加页框数，缺页次数反而增加。
- **LRU**：淘汰最久未被访问的页。它利用时间局部性，效果通常接近 OPT；精确实现需要记录访问顺序，硬件开销大，操作系统常用近似算法。
- **Clock（时钟/第二次机会）**：把页排成环形，每页有一个访问位。指针扫描时，访问位为 1 就清零并跳过，为 0 就淘汰。它是 LRU 的低开销近似。

Belady 异常的经典例子：引用串

$$1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5.$$

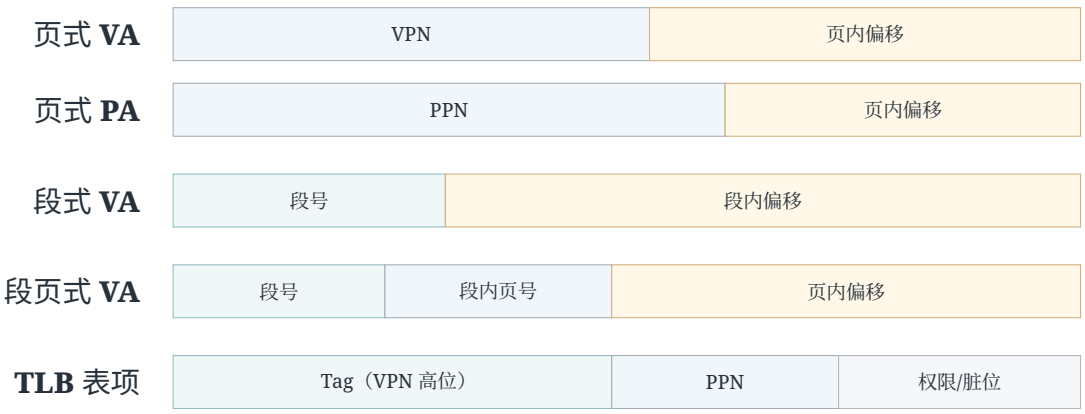
用 FIFO、3 个页框时缺页 9 次；换成 4 个页框，缺页反而变成 10 次。原因是 FIFO 只看装入时间，4 个页框时把一些即将再次使用的页留在了队列里，反而挤掉了更需要的页。LRU 属于栈算法，不会出现这种异常。

5.11 段式与段页式

页式按固定大小切分，对操作系统很方便，但对程序员来说，代码段、数据段、栈段这些逻辑单位被拆成了同样大小的页，共享和保护要按页来做。段式虚拟存储器换一种切法：按程序的逻辑结构划分成变长的段，每段从 0 开始编址。

段式地址是“段号 + 段内偏移”。段表项记录该段的基址和界限（长度）。访问时先查段表，检查段内偏移是否小于界限；合法就把基址加上偏移，得到物理地址。段式便于按逻辑单位共享和保护，但变长分配会在主存中留下外部碎片：段与段之间的空洞可能装不下一个新段，即使总空闲空间足够。

段页式把两者结合起来：先按逻辑分段，每段内再分页。虚拟地址是“段号 + 段内页号 + 页内偏移”。段表项指向该段的页表，页表项再给出物理页框号。这样段仍然保留逻辑意义，页又消除了外部碎片；代价是地址翻译要多查一级表。



页式：偏移在 VA/PA 中相同；段式：段表给基址和界限；段页式：段表指向页表；TLB 缓存的是页表项。

图把四种地址格式排在一起。页式的偏移在翻译前后不变；段式的段内偏移要和界限比较，越界就是非法访问；段页式在段号之后又多了一层页号；TLB 表项则是页表项的缓存形式，Tag 用来匹配 VPN，PPN 是翻译结果。

5.12 几组容易混淆的概念

存储系统常见辨析

概念	关键区别
随机存取 vs RAM	随机存取指存取时间与物理位置无关；ROM 也能按地址随机访问，但不属于 RAM。RAM 通常指可读可写的易失性半导体存储器。
SRAM vs DRAM	SRAM 用 6 管触发器，不需刷新，快而贵，做 Cache；DRAM 用 1 管 1 电容，需刷新、破坏性读出，慢而便宜，做主存。两者都是易失的。
Cache 缺失 vs 缺页	Cache 缺失由硬件回填，通常对 OS 透明；缺页是 PTE 有效位为 0，由 OS 处理，可能涉及外存 I/O。
写直达 vs 写回	写直达同时更新 Cache 和主存，通常不需要脏位；写回只更新 Cache，置脏位，替换时才写回。
写分配 vs 非写分配	写分配在写缺失时先把块调入 Cache；非写分配直接写主存。它们和写直达/写回是两个维度。
TLB 缺失 vs 缺页	TLB 缺失只是页表项没缓存，查页表即可；缺页是页表项有效位为 0，必须由 OS 调入页面。
高位交叉 vs 低位交叉	高位交叉用高位选体，连续地址在同一体，顺序访问不能流水；低位交叉用低位选体，连续地址分布到不同体，可以流水。
PIPT vs VIPT	PIPT 先翻译再查 Cache，TLB 在关键路径；VIPT 用 VA 低位索引、PA 高位比较 Tag，索引位必须落在页内偏移内。

把这些概念放回存储层次里看，关系会清楚很多：寄存器、Cache、主存、外存是层次；块、命中、缺失、替换是机制；虚拟地址、页表、TLB 是地址翻译；写策略和替换算法是具体策略。题目给出参数时，先确定“问的是哪一层、按什么粒度、计数口径是什么”，再套公式或模拟。

5.13 拓展阅读

- Patterson & Hennessy《计算机组成与设计》第 5 章：存储层次、Cache 映射与性能分析，适合配合本章的 AMAT 公式阅读。
- Bryant & O’Hallaron《深入理解计算机系统》第 6 章：局部性、Cache 组织和 Cache 友好代码；第 9 章：页表、TLB、缺页处理和地址翻译综合案例。
- 唐朔飞《计算机组成原理》存储器一章：SRAM/DRAM 单元、刷新方式、Cache 地址映射和替换算法的详细推导。

6 习题

6.1 基础过关题

1. (单项选择题) 下列关于 SRAM 与 DRAM 的叙述中，**错误**的是

(A) SRAM 用双稳态触发器保存信息，不需要刷新，常用来做 Cache
(B) DRAM 用电容上的电荷表示信息，读出是破坏性的，读后需要重写
(C) DRAM 的存取周期通常大于存取时间，原因之一是读后需要恢复
(D) SRAM 断电后信息不会丢失，因此适合做主存

2. (填空题) 某 DRAM 芯片内部有 8192 行，存取周期为 50 ns，器件的刷新周期（数据保持时间）为 64 ms。每次刷新占用一个存取周期。

(1) 采用**集中刷新**时，刷新全部行造成的“死时间”为 _____；
(2) 该死时间占刷新周期 64 ms 的百分比为 _____；
(3) 采用**异步刷新**时，相邻两次刷新操作的间隔约为 _____。

3. (单项选择题) 某计算机主存按字编址，字长 4 字节，由 4 个存储体构成并采用**低位交叉编址**。每个存储体的存取周期 $T = 60\,\text{ns}$，总线传输周期 $\tau = 15\,\text{ns}$。CPU 从字地址 0 开始连续读取 8 个字，忽略发地址等额外开销，所需时间为

(A) 105 ns (B) 165 ns
(C) 240 ns (D) 480 ns

4. (填空题) 某计算机按字节编址，主存地址 32 位。数据 Cache 的数据区容量为 64 KB，块大小 32 B，采用 4 路组相联映射。

(1) 主存地址中，块内偏移占 _____ 位，组号占 _____ 位，Tag 占 _____ 位。

(2) 若采用写回法，每行另有 1 位有效位和 1 位脏位（不计替换位等其他开销），则整个 Cache 的总容量为 _____ B。

5.（单项选择题）下列关于 Cache 写策略的叙述中，正确的是

(A) 写直达通常与非写分配搭配，写缓冲可减少 CPU 等待主存的时间
(B) 写回法在每次写命中时都要把整个块写回主存
(C) 写分配是指写缺失时绕过 Cache，直接写入主存
(D) 写直达不需要脏位，因此 Cache 与主存的内容总是相同

6.（单项选择题）某计算机虚拟地址 32 位，按字节编址，页大小 8 KB，采用单级页表覆盖整个虚拟地址空间，每个页表项 4 字节。则虚拟页号（VPN）位数、页表项数、页表大小分别为

(A) 19 位，2^{19} 项，2 MB　(B) 19 位，2^{19} 项，4 MB
(C) 21 位，2^{21} 项，8 MB　(D) 13 位，2^{13} 项，32 KB

6.2 真题风格与综合训练

1.（单项选择题）某 Cache 有 4 组，每组 2 行，块大小为 1 个字。主存块号 $0, 4, 8$ 都映射到第 0 组。初始 Cache 为空（其余组不参与本题），CPU 依次访问主存块

$$0,\ 4,\ 0,\ 8,\ 0,\ 4,\ 0.$$

分别采用 LRU 和 FIFO 替换，LRU 比 FIFO 多命中的次数是

(A) 0 次　(B) 1 次
(C) 2 次　(D) 3 次

2. （单项选择题）某处理器采用两级 Cache。L1 命中时间为 1 个时钟周期，L1 缺失率为 4%；L1 缺失后访问 L2，L2 命中需 10 个周期（已包含把块回填到 L1 并完成本次访问）。若 L2 也缺失，访问主存并回填 L2 需 120 个周期（已包含回填 L2，不含 L2 的 10 个周期检查）。L2 的局部缺失率为 25%。该处理器的平均访问时间（AMAT）为

(A) 2.6 个周期　(B) 6.2 个周期

(C) 2.5 个周期　(D) 1.4 个周期

3. （综合应用题）**Cache 访问序列与写策略分析。**某计算机按字编址，字长 4 字节，Cache 采用 2 路组相联映射，共 2 组（4 行），块大小为 1 个字，采用 LRU 替换。初始 Cache 为空。CPU 依次执行下列访存操作（数字为主存块号，读/写均为访问 1 个字）：

读0, 写0, 写0, 读1, 写2, 读1, 写4, 读0.

(1) 写出每个块号映射到的组号，并逐次判断命中/缺失。

(2) 设在 (1) 中采用**写回法 + 写分配**。统计命中次数、命中率、从主存读入的块数、因替换脏块而产生的写回次数，以及主存块传输总次数。计数口径：每次缺失（无论读缺失还是写缺失）都从主存读入 1 个块；每个被淘汰的脏块写回 1 次。

(3) 若改用**写直达 + 非写分配**，重新统计命中次数、主存读块数和主存写字数，并解释命中次数变化的原因。

4. （综合应用题）**TLB、页表与 Cache 的地址翻译。**某计算机按字节编址，虚拟地址 32 位，物理地址 28 位，页大小 4 KB，页表项大小 4 字节。TLB 采用 4 路组相联，共 16 项。L1 数据 Cache 采用物理寻址，数据区 32 KB，8 路组相联，块大小 64 B。

   (1) 求 VPN、VPO、PPN、PPO 的位数；单级页表的项数和页表大小各是多少？

   (2) 求 TLB 的组数、组号位数和 Tag 位数。

   (3) 求 L1 Cache 的组数，以及物理地址中 Tag、组号、块内偏移的位数。

   (4) 某进程访问虚拟地址 `0x12345678`，TLB 命中并得到 PPN = `0x0ABC`。求物理地址、Cache 组号、Tag 和块内偏移（用十六进制或十进制表示均可）。

   (5) 若该虚拟页的页表项有效位为 0，说明由谁处理、主要步骤是什么，以及处理完后如何继续。

7 指令系统

CPU 执行一条指令要经过取指、译码、取操作数、执行、写回。指令系统规定的是这条链路上两件事的格式：指令本身长什么样，操作数从哪里来。这两件事一定，数据通路和控制器的设计空间就大致定了，编译器能生成什么样的代码也大致定了。

先看一条具体的指令。RISC-V 的 `add x5, x6, x7` 编码成 32 位：

```
0000000 00111 00110 000 00101 0110011
```

即十六进制的 `0x007302B3`。最低 7 位 `0110011` 是主操作码，表示“这是一条寄存器—寄存器型的整数运算”；最高 7 位 `0000000` 与中间 3 位 `000` 合起来，才把“加”从同一主操作码下的减、与、或里区分出来；剩下的三组 5 位分别是目的寄存器 x5 和两个源寄存器 x6、x7。

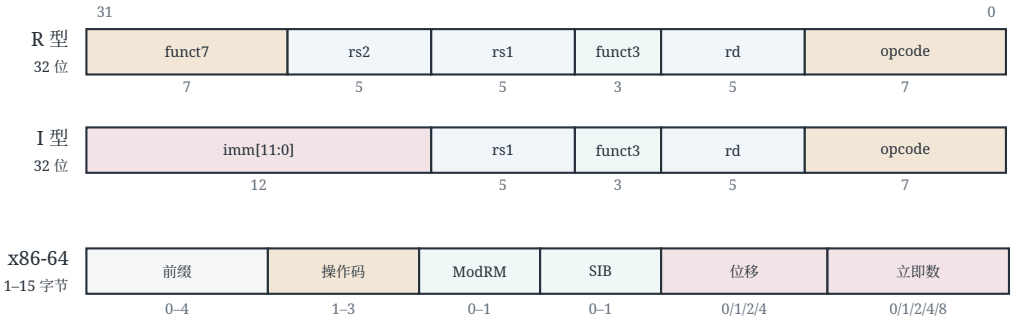
这段编码说明了一件常被忽略的事：操作码并不总是“一个字段决定一条指令”。当同一类操作的方式很多时，ISA 会把操作码拆成主操作码加若干辅助字段，让译码器先粗分大类，再细分具体操作。

7.1 指令格式：字段怎么放

经典教材把指令写成操作码加若干地址码，但真实的指令字里通常还有第三类字段——寻址方式字段。它告诉硬件地址码该怎么解释：是直接当地址，当偏移量，还是当寄存器号。于是多数指令字的划分是



指令字总位数是有限的：操作码长了，地址码就短。指令格式的设计就是在这两者之间分配位数，并决定字段放在哪几位。



图里两条 32 位指令的字段边界完全重合：`opcode` 总在最低 7 位，`rs1`、`funct3`、`rd` 的位置也不随指令类型变化。译码器取到指令的同时，寄存器号已经躺在固定位置上了。最下面一行的 x86 则没有固定边界，方框只表示“可能有哪些字段”，每个字段实际占几个字节由前面的字段内容决定，整条指令 1 到 15 字节不等。

变长带来的麻烦是真实的：不解析当前指令，就不知道下一条从哪里开始。x86 处理器因此要在流水线前端做指令长度预译码，把字节流先切成一条条指令。定长则相反，RISC-V 的基础指令每条 4 字节，PC 每次加 4 就能取到下一条，代价是一条本可以用 16 位表示的指令也占满 32 位。

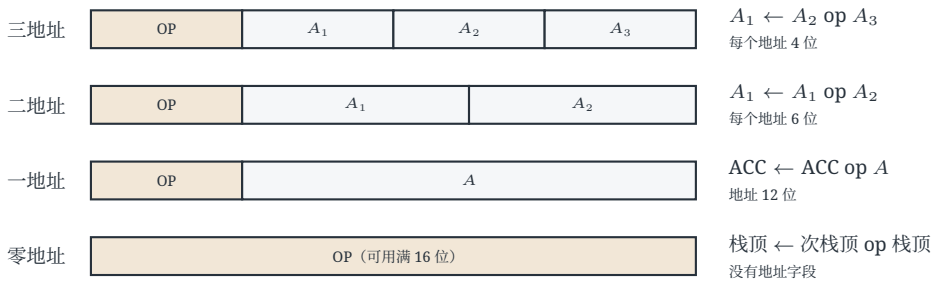
字段位置还会反过来影响硬件。RISC-V 的 I 型把 12 位立即数放在最高位，符号扩展后直接可用；B 型和 J 型为了不挪动 `rs1`、`rs2` 的位置，把立即数的各位打散重排，硬件只好多花几根线把它们拼回原位。格式设计始终在“字段位置固定、译码简单”和“字段够用、编码紧凑”之间取舍。

指令字长不一定等于机器字长，也不一定等于存储字长。它通常取存储字长的整数倍；如果一条指令比存储字长还长，取指就要多次访存，PC 的增量也必须按整条指令的字节数计算。

7.2 地址码数量与操作数来源

一条指令能显式写几个地址，决定了它属于哪种风格。下面的约定是目的操作数写在最左。

三地址指令写成 `OP A1, A2, A3`，语义是 $A_1 \leftarrow A_2$ op A_3。一次运算三个地址都写全，源操作数不被破坏，代价是指令长。二地址指令写成 `OP A1, A2`，语义是 $A_1 \leftarrow A_1$ op A_2，目的地址复用了源地址，省下字段但会覆盖一个操作数。一地址指令只剩一个地址，另一个操作数隐含在累加器里，即 ACC $\leftarrow$ ACC op A。零地址指令连地址都不写，常见于堆栈计算机：两个操作数默认取栈顶，运算结果压回栈顶；`NOP`、`RET` 这类指令本来也不需要地址。



指令字长都取 16 位：地址码越少，单个地址码越长，可寻址的单元越多

四条指令字一样长，差别全在位数怎么分。三地址要三个 4 位地址，只能覆盖 16 个单元；一地址把 12 位全给地址，能覆盖 4096 个单元。地址码数量与寻址范围是一对此消彼长的量，这也是早期机器偏爱一地址、二地址的现实原因——内存小，地址短，反而够用。

零地址指令把这种思路推到极端：既然操作数总在栈顶，地址就彻底不必写了。表达式 $(a+b)\times(c+d)$ 在堆栈机上写成

```
PUSH a; PUSH b; ADD; PUSH c; PUSH d; ADD; MUL; POP x
```

八条指令里只有 `PUSH`、`POP` 带地址字段，`ADD`、`MUL` 只占一个操作码。若指令允许变长，这几条零地址指令可以各占一个字节，代码总量甚至比三地址版本更短；但指令条数多了，压栈、弹栈还要访存。这就是“每条指令做多少事”与“一共要多少条指令”之间的交换，后面 CISC 与 RISC 的分歧，根子上也是这个交换。

还有一类一地址指令，地址码给的不是操作数而是转移目标，例如 `JMP A`。这类指令没有“取操作数”的问题，地址码的含义要结合操作码来判断。

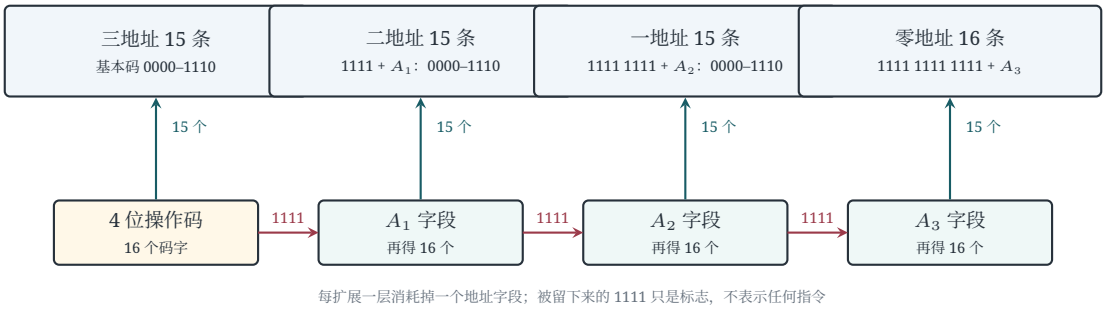
按操作数的存放位置，指令系统通常分成三类：堆栈型以零地址为主；累加器型以一地址为主，另一个操作数隐含在 ACC；通用寄存器型以二地址或三地址为主，配合 load/store 结构把指令做规整。x86 属于混合型，以二地址为主，且允许其中一个操作数直接来自主存。

7.3 扩展操作码

定长操作码的规则最简单：n 位操作码最多 2^n 条指令，译码时把操作码整段送进译码器就行。问题是各类指令的使用频率相差很大，传送、加减、比较占了程序的绝大部分，系统控制类指令寥寥无几，却要和它们平分编码空间。

扩展操作码让常用的指令用短码，少用的用长码。做法是：基本操作码取 $0000 \sim 1110$ 表示 15 条三地址指令，把 1111 留作标志；译码器看到 1111 时不把它当指令，而是继续读下一个 4 位字段 A_1，把它当操作码的延伸。于是 `1111 0000`～`1111 1110` 表示 15 条二地址指令，`1111 1111` 再往下扩展，依此类推。

这里有一个常被说错的约束。教材常说“短操作码不能是长操作码的前缀”，严格讲应该是：**被用作指令的码字之间不能有前缀关系**。1111 本身不是任何一条指令，它只是扩展标志，所以它当然可以作为 `1111 0000` 的前缀；真正的错误是把 1111 同时定义成一条三地址指令——那样译码器读到 1111 时就无法判断后面的 4 位是地址还是操作码。



每扩展一层消耗掉一个地址字段；被留下来的 1111 只是标志，不表示任何指令

横向的箭头是“扩展”：剩下的那 1 个码字连同下一个 4 位字段，变成 16 个新码字。纵向的箭头是“定型”：其中 15 个拿来当指令，留 1 个继续扩展。四层分别得到 15、15、15、16 条，一共 61 条；同样 16 位的字长，若操作码定长 4 位，只能有 16 条。

数指令条数有一个比逐层数前缀更稳的通式。把 L 位的指令字看成 2^L 个码点：一条三地址指令的 12 位地址字段可以任意取值而操作不变，所以它独占 2^{12} 个码点；二地址占 2^8 个，一地址占 2^4 个，零地址占 1 个。所有指令占的码点既不能重叠，也不能超过总数，于是

结论 7.1: 扩展操作码的容量约束

设指令字长 L 位，某类指令的地址字段共占 a_i 位，该类指令有 n_i 条，则

$$\sum_i n_i \, 2^{a_i} \;\leqslant\; 2^L.$$

对 16 位字长、4 位地址字段的例子，m 条三地址、n 条二地址、p 条一地址、q 条零地址满足

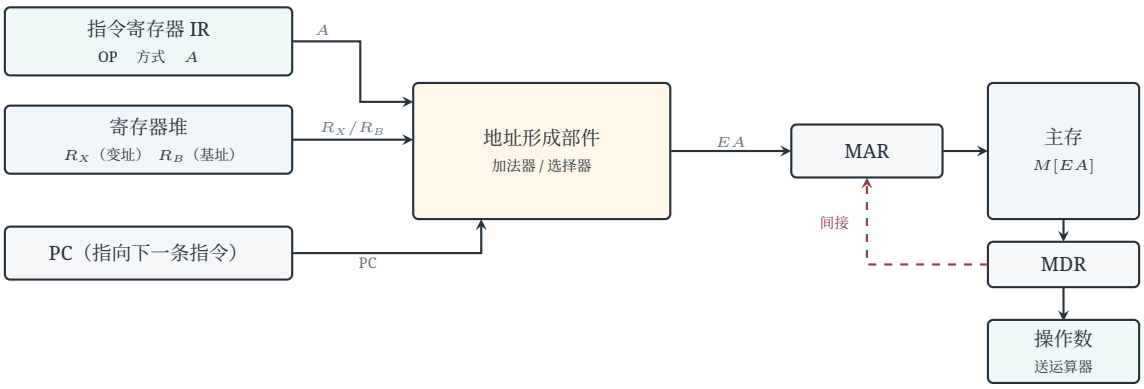
$$m \cdot 2^{12} + n \cdot 2^8 + p \cdot 2^4 + q \;\leqslant\; 2^{16}.$$

代入 $m = n = p = 15$，左端为 $61440 + 3840 + 240 = 65520$，剩下 16 个码点，所以零地址最多 16 条。这个不等式还有一个等价的读法：操作码集合是前缀无关的，短码占掉的编码空间正好等于它能扩展出的长码总数。

7.4 数据寻址方式

指令里的地址字段叫形式地址，记作 A。它往往不是操作数的真实地址：可能是偏移量，可能是寄存器号，甚至就是操作数本身。把这层解释规则叫寻址方式，指令里用几位寻址特征字段指明它。

无论哪种方式，硬件最终都要得到一个有效地址 EA，或者干脆绕过主存直接拿到操作数。下图是一条通用的取数通路。



实线：按 EA 访存取操作数；虚线：间接寻址多一次访存。
立即寻址与寄存器寻址不经过 MAR 和主存。

图中从左侧进入加法器的三条实线，是三种“基准”的来源：形式地址 A 来自指令寄存器，变址寄存器和基址寄存器来自寄存器堆，PC 来自取指部件。加法器把基准和 A 相加得到 EA；直接寻址时选择器让 A 直通，不做加法。虚线是间接寻址特有的回路：从 $M[A]$ 读出来的不是操作数而是地址，它要再送进 MAR 才行。

各种方式的差别可以列成一张表。表里的访存次数只算取操作数，不含取指令。

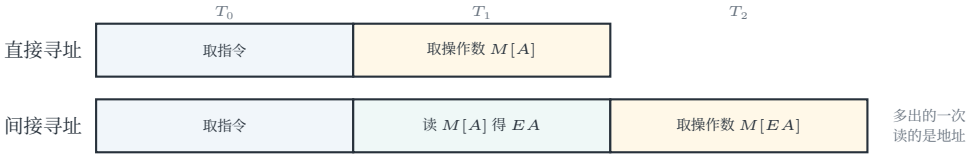
数据寻址方式一览

方式	有效地址 / 操作数	访存次数	典型用途
隐含寻址	地址隐含（如 ACC）	0	一地址指令
立即寻址	操作数 $= A$	0	常数、小整数
直接寻址	$EA = A$	1	全局变量
一次间接	$EA = (A)$	2	指针、函数参数
寄存器寻址	操作数 $= R_A$	0	局部变量
寄存器间接	$EA = (R_A)$	1	指针、数组
变址	$EA = (R_X) + A$	1	数组元素 `a[i]`
基址	$EA = (R_B) + A$	1	程序重定位
相对	$EA = (PC) + A$	1	转移、位置无关代码
堆栈	栈顶由 SP 隐含	0 或 1	表达式求值、函数调用

立即寻址的地址码字段就是操作数本身，通常按补码解释。取指时它已随指令进入 IR，取操作数不再访存，这是它比直接寻址快的原因。代价是位数：字段只有 12 位，常数就被限制在 $[-2048, 2047]$，更大的常数得先用 `lui` 之类的指令拼出来。

直接寻址的 $EA = A$，一次访存拿到操作数，含义最直白。寻址范围被地址字段的位数锁死：12 位地址只能指向 4096 个单元。

间接寻址的 $EA = (A)$，先按 A 读一次主存，读出来的字才是地址。一次间接要访存两次，换来两个好处：地址字段不必和机器地址一样宽，长地址可以存在主存里；以及天然的指针语义，改指针的值就能换操作数。



图里每个方格是一次访存。直接寻址两格就结束；间接寻址多出的第二格读回来的仍然是地

址，它必须再走一遍地址通路，所以取操作数的访存次数从 1 变成 2。两次间接就是 3 次，层数越多越慢，实际机器很少超过两层。

寄存器寻址与寄存器间接寻址只差一层括号，差别却是一次访存。寄存器寻址的操作数就在寄存器里，指令中只需 5 位寄存器号，取数不用访存；寄存器间接则把地址放在寄存器里，还要按那个地址访存一次。前者适合局部变量，后者适合指针和数组遍历。

变址、基址、相对寻址三者的计算式都是“寄存器加形式地址”，区别在谁提供基准、谁在程序运行中被修改。

结论 7.2: 变址与基址的区别

两者形式相同，$EA=(R)+A$，但谁变谁不变正好相反：

- **变址**：指令里的 A 是数组首地址，固定不变；变址寄存器 R_X 存放下标，由用户程序在循环中反复修改。一条 `LW R1, 0(R2)` 配合 R2 每次加 4，就能走遍整个数组。
- **基址**：基址寄存器 R_B 存放程序（或段）在内存中的起始地址，由操作系统装入，运行中不变；指令里的 A 是段内偏移。程序被换到别处，只要改 R_B，指令一条都不用重写，这就是重定位。

判断时看谁被反复修改：用户程序改的是变址，操作系统设定的是基址。

相对寻址以 PC 为基准，A 是补码位移。它的价值在下一小节展开，因为 PC 到底取哪个值，是这类题目最常见的失分点。

x86 的 SIB 寻址把基址、变址、比例因子和位移揉在了一起：$EA=$ 基址 $+$ 变址 $\times$ 比例因子 $+$ 位移。比例因子只取 1、2、4、8，正好是 `char`、`short`、`int`、`double` 的宽度，所以访问 `a[i]` 时一条指令就能算出元素地址，不必先做一次乘法。这种“把常用组合做进硬件”的思路，是 CISC 的典型手法。

堆栈寻址的操作数地址由栈指针 SP 隐含给出。软堆栈建在主存里，一次 push 要写数据、改 SP，至少一次访存；硬堆栈用一组寄存器实现，不访存但容量很小，通常只有十几层。

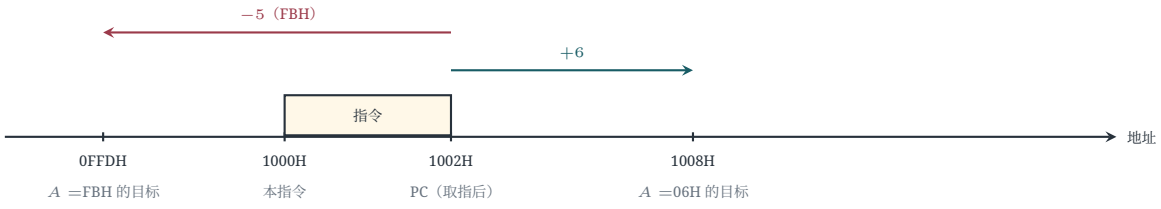
7.5 指令寻址与转移

数据寻址解决“操作数在哪”，指令寻址解决“下一条指令在哪”。顺序执行时 PC 加上一条指令的长度；遇到转移指令，PC 被改写成目标地址，称为跳跃。

按字节编址时 PC 的增量是指令的字节数，不是 1。16 位定长指令加 2，RISC-V 的 32 位指令加 4，x86 则要加实际长度。取指结束时，PC 已经指向下一条指令——这个细节决定了相对寻址的基准：

$$EA=(PC)+A,$$

其中 (PC) 是**下一条**指令的地址，不是当前正在执行的那条的地址。



指令占 1000H、1001H 两个单元，取指后 PC 变成 1002H。位移 $A=06\text{H}$ 时目标为 1008H；

A =FBH 是补码的 -5，目标为 0FFDH。箭头从 PC 出发而不是从 1000H 出发，这就是相对寻址与“直接拿指令地址加位移”的分水岭。

位移字段是 8 位补码，可正可负，范围 $-128 \sim +127$，对应目标地址在 $PC-128$ 到 $PC+127$ 之间。若想把范围扩大，可以让位移以指令长度为单位编码：指令总是对齐的，最低位不必存。MIPS 的分支偏移左移 2 位，RISC-V 的 B 型立即数以 2 字节为单位，都是这个道理，代价是目标必须对齐。

注意

算相对转移目标时，要用 **当前指令地址 + 指令长度 + 位移**，三项缺一不可。直接把位移加到当前指令地址上，是这类题最常见的错误；把补码位移当无符号数相加，是第二种常见错误。

用相对寻址还有一个工程上的理由：位置无关。程序被加载到内存的任何位置，指令与目标的相对距离都不变，代码不必修改；同时 8 位位移只占一个字节，比在指令里写完整的绝对地址省空间。代价是转移范围有限，跳得远就得换成绝对寻址或先装基址。

7.6 指令的几大类

指令按功能大致分几类：数据传送（MOV、LOAD、STORE）、算术逻辑（ADD、AND、CMP）、移位、转移（JMP、BEQ、CALL、RET）、输入输出、系统控制（停机、开中断、陷入）。

分类本身不难记，考试真正用得上的是每类指令对操作数和状态的影响：转移类只改 PC，不写数据；比较类只置标志位，不产生结果；调用类要把返回地址压栈，返回时再弹出。看懂一段汇编在做什么，靠的就是这些副作用，而不是助记符的字面意思。

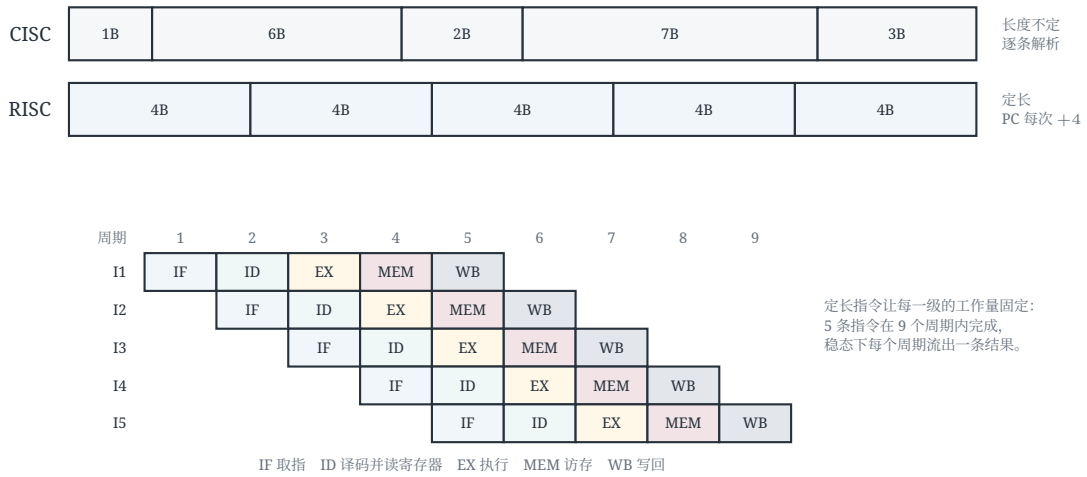
7.7 CISC 与 RISC

早期的存储器又贵又小，程序长度直接决定成本，于是设计者尽量让一条指令多做事：一条指令完成“取数—运算—存回”，甚至直接支持链表遍历、字符串比较。这类复杂指令集（CISC）用硬件的复杂度换程序员的方便，x86 是它的代表。

后来情况反了过来。编译器足够聪明，能把高级语言翻译成简单指令的组合；流水线要求每一级的工作量和时延都稳定，复杂指令反而成了障碍。于是出现了精简指令集（RISC）：指令少而规整，格式固定，只有 load/store 访存，寄存器多，控制器用硬布线实现。MIPS、ARM、RISC-V 都属于这一类。

CISC 与 RISC 的常见差别

特征	CISC	RISC
指令数量与格式	多，格式不规整	少，格式规整、种类有限
指令长度	可变（x86 为 1–15 字节）	通常定长（如 32 位）
寻址方式	多，且多数指令可直接访存	少，一般只有 load/store 访存
寄存器数量	传统设计较少	较多，减少访存压力
控制器	常用微程序，便于实现复杂指令	常用硬布线，译码快、延迟低
流水线	较难，需处理变长与不定长执行	容易，节拍均匀
代码密度	高	定长编码下偏低
编译负担	编译器较轻松	编译器要做更多寄存器分配与调度



上半部分对比两种取指流：CISC 的指令边界要逐条解析，长度从 1 字节到 15 字节；RISC 每条固定 4 字节，PC 加 4 就能取下一条。下半部分是定长指令的五级流水线，斜着看每一行表示一条指令依次经过五个阶段，任意一个周期里五个阶段都在忙。变长指令想进这样的流水线，必须先在前面加一级翻译，把长度不一、执行时间不一的指令变成节拍均匀的微操作。

实际产品早已不是非此即彼。x86 处理器前端把复杂指令译成一串类 RISC 的微操作，后端用乱序执行和深流水线跑这些微操作；RISC-V 的 C 扩展把常用指令压成 16 位，ARM 有 Thumb，都是为了提高代码密度。所以“RISC 一定定长”“CISC 一定用微程序”这类说法都不成立——微程序还是硬布线是具体实现的选择，不是指令集的定义特征。

至于性能，指令条数少不等于快，指令简单也不等于慢。同样一个程序在两台机器上，最终比的是执行时间，而执行时间取决于指令条数、平均 CPI 和主频三者的乘积关系。CISC 用更少的指令、更高的 CPI 完成工作，RISC 用更多的指令、更低的 CPI 完成工作，谁快要看具体数值。

注意

三个常见的绝对化说法要当心：

- “RISC 每条指令一个周期”指的是经典五级流水线中每条指令占用一个节拍，不是整个执行过程只花一个时钟周期；访存指令仍要经过 MEM 级。
- “只有 load/store 访存”是典型 RISC 的设计取向，不是所有 RISC 的硬性定义。
- “CISC 用微程序、RISC 用硬布线”只是统计意义上的常见搭配，两类都可以选另一种实现。

7.8　拓展阅读

- **Patterson & Hennessy**《计算机组成与设计（**RISC-V** 版）》第 2 章「指令：计算机的语言」——RISC-V 的 R/I/S/B/U/J 六种格式、过程调用与栈的使用，是本章最直接的补充。
- **Bryant & O’Hallaron**《深入理解计算机系统》第 3 章「程序的机器级表示」——从 x86-64 汇编出发，讲清一条 C 语句如何变成取数、运算、存回，以及变长指令的字节级编码。
- **唐朔飞**《计算机组成原理》第 7 章「指令系统」——机器指令、操作数类型、寻址方式、指令格式举例与 RISC 技术；不同版本的章节号可能略有差异，以所用教材为准。
- **延伸思考：**指令格式没有唯一解：RISC-V 打散立即数位以复用寄存器字段，x86 用前缀字节换向后兼容，ARM 用条件执行字段减少分支。遇到没见过的格式，先问三件事：操作码在哪几位、地址字段怎么解释、指令长度怎么确定。

8 习题

8.1 基础过关题

1. (单项选择题) 某计算机指令字长 16 位，采用扩展操作码，二地址指令的格式为「4 位基本操作码 + 两个 6 位地址码」。若该机已有 12 条二地址指令和 254 条一地址指令，则零地址指令最多可有（ ）条。

   (A) 64

   (B) 128

   (C) 256

   (D) 512

2. (单项选择题) 某机按字节编址，指令字长 2 字节。一条采用相对寻址的转移指令存放在 2000H、2001H 两个单元中，其 8 位形式地址字段为补码 F6H。取指完成后 PC 的内容为 2002H。该转移指令的转移目标地址是（ ）。

   (A) 1FF8H

   (B) 200CH

   (C) 1FF6H

   (D) 20F8H

3. (单项选择题) 下列关于寻址方式的叙述中，正确的是（ ）。

   (A) 立即寻址的操作数直接包含在指令中，取操作数时不需要访问主存

   (B) 直接寻址的有效地址等于形式地址所指主存单元的内容

   (C) 寄存器间接寻址中，操作数存放在地址码字段指定的寄存器中

   (D) 变址寻址中，变址寄存器的内容由指令的形式地址字段给出

4. (填空题) 某机按字节编址，采用一次间接寻址。某指令的形式地址字段为 3000H，主存 3000H 单

元的内容为 4000H，而 4000H 单元存放着该指令所需的操作数。则有效地址 $EA=$ _____，取该操作数需要访问主存 _____ 次（不计取指访存）。

5.（单项选择题）下列关于 CISC 与 RISC 的叙述中，错误的是（ ）。

(A) RISC 通常采用定长指令，指令译码和流水线控制较为简单

(B) CISC 的指令长度和寻址方式通常比 RISC 更多样

(C) 典型 load-store 型 RISC 中，只有取数/存数指令访问数据存储器

(D) CISC 一定采用微程序控制，RISC 一定采用硬布线控制

6.（填空题）零地址指令不显式给出操作数地址，其操作数通常隐含在 _____ 中；一地址指令通常把另一个操作数隐含在 _____ 中。

8.2 真题风格与综合训练

1.（单项选择题）某机指令字长 16 位，采用扩展操作码，每个地址码字段均为 4 位，可安排三地址、二地址、一地址和零地址指令。现已有 12 条三地址指令、40 条二地址指令和 200 条一地址指令，则零地址指令最多可有（ ）条。

(A) 184

(B) 2944

(C) 6144

(D) 384

2.（单项选择题）某 RISC 机采用 32 位定长指令，按字节编址，指令按字边界对齐。一条转移指令采用 PC 相对寻址：16 位偏移字段为补码，左移 2 位后与「本指令地址 $+4$」相加得到目标地址。该指令存放在从 2000H 开始的单元中，偏移字段为 FFF8H，则转移目标地址是（ ）。

(A) 1FE4H
(B) 1FFCH
(C) 1FE0H
(D) 2024H

3. (综合应用题) 某计算机指令字长 16 位，采用扩展操作码。二地址指令的格式为「4 位基本操作码 + 两个 6 位地址码」，一地址指令和零地址指令依次利用地址码字段继续扩展。设该机已有 14 条二地址指令。

(1) 若一地址指令有 120 条，零地址指令最多有多少条?
(2) 若要求零地址指令不少于 100 条，一地址指令最多有多少条?
(3) 给出一种具体的操作码分配方案，并说明为什么此时二地址操作码 1111 不能再单独表示一条二地址指令。

4. (综合应用题) 某程序分别在两台机器上运行。CISC 机 M1：编译后共 2.0×10^6 条指令，平均 CPI 为 4.0，主频 2.0 GHz。RISC 机 M2：同一程序编译后共 3.0×10^6 条指令，平均 CPI 为 1.5，主频 2.0 GHz。两机的 CPI 均已包含流水线停顿，忽略 I/O 等待。

(1) 分别求程序在 M1、M2 上的执行时间（单位：ms）。
(2) 求 M2 相对 M1 的加速比。
(3) 分别求两机的 MIPS。
(4) 若 M2 的主频提高到 2.5 GHz、CPI 不变，重算 M2 的执行时间与加速比。
(5) 结合 (1)～(3) 说明：为什么 M2 的指令条数更多，却仍然可能更快? 能否仅凭 MIPS 判断两台机器谁更快?

9 中央处理器（CPU）

一条 `add R1, R2, R3` 在 CPU 里要走完这样一串动作：PC 把地址送到 MAR，主存把指令送回 MDR、再送进 IR；译码器认出这是加法，控制器依次打开寄存器堆的读口，把 R2、R3 送到 ALU，最后把结果写回 R1。数据经过的部件和连线叫**数据通路**，每一步“谁开门、谁打入、谁驱动总线”的信号叫**控制信号**。这一章说到底只回答两个问题：路怎么铺，信号怎么发。

同一条路可以有不同的走法。每条指令走完再走下一条，是单周期或多周期实现；让多条指令的不同阶段在时间上错开，是流水线。路还是那条路，变的是节奏；节奏一变，吞吐率、冒险和控制器的复杂度都跟着变。

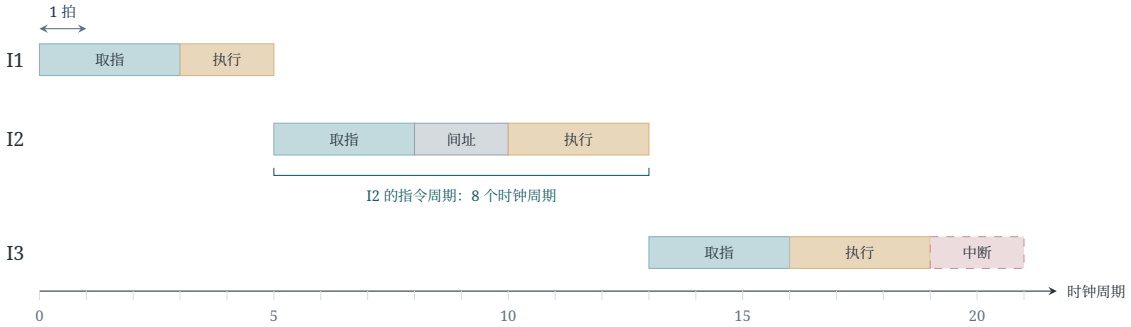
9.1 指令周期与定时

CPU 的时间分成三层。最细的一层是**时钟周期**（节拍），由主频决定：1 GHz 的 CPU，一个时钟周期就是 1 ns。若干时钟周期组成一个**机器周期**，通常对应一次总线事务或一次主存访问；比如一次访存要 4 个时钟周期，机器周期就是 4 ns。若干机器周期再组成一条指令的**指令周期**。三层是包含关系，不是同一个东西。

一条指令的指令周期一般分成四段：

- **取指**：PC→MAR，发读命令；主存把指令送到 MDR，再送入 IR；PC 加上本条指令的长度（定长指令就是 $+4$）。
- **间址**：只有间接寻址才需要。把指令中的形式地址送 MAR，再读一次主存，得到真正的有效地址。
- **执行**：按操作码做运算、访存或转移。不同指令在这里差别最大。
- **中断**：仅当本条指令执行完、且有可响应的中断请求时进入。保存断点、关中断、取中断服务程序入口。

注意间址段和中断段不是每条指令都有。寄存器型指令可能只有取指和执行，间接寻址的访存指令会多出一个间址段，响应中断时还要再加一段。所以“指令周期”是一个笼统的说法，具体长度随指令而变。



图上横轴是时钟周期，一条色块就是一段机器周期。I1 是一条简单的寄存器传送指令，只有“取指 + 执行”，占 5 拍；I2 多了间址，占 8 拍；I3 在最后接了中断周期。三个指令周期长度不同，这正是流水线要面对的第一件事：如果每一拍都按最长的指令来定，短指令就亏了；如果按短指令定，长指令又做不完。

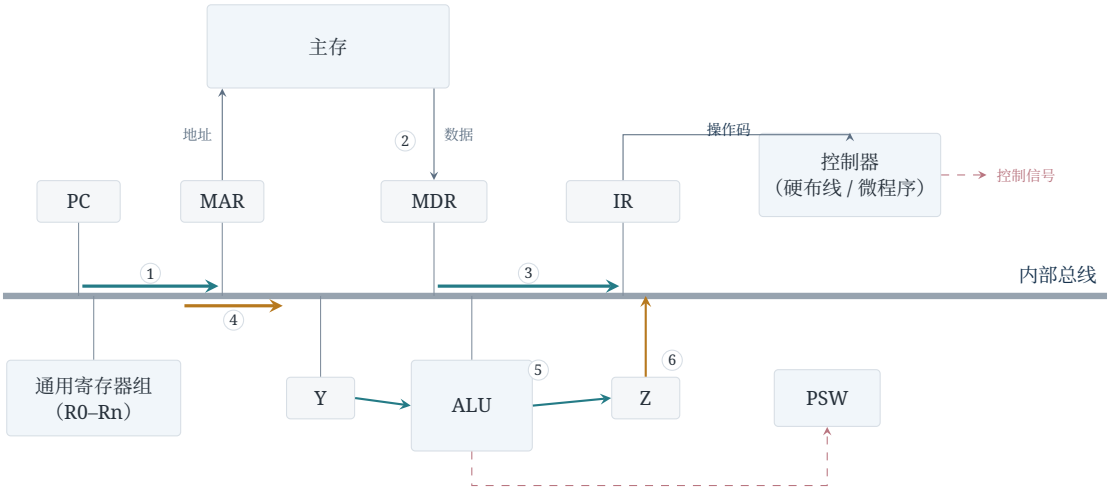
9.2 数据通路

从硬件看，CPU 分成运算器和控制器两块。运算器负责算，核心是 ALU、通用寄存器组、PSW 和暂存器 Y/Z；控制器负责发信号，核心是 PC、IR、指令译码器、时序部件和微操作信号发生器。PC、通用寄存器和 PSW 对程序员可见：通用寄存器可以直接读写，PC 通过转移指令改写，PSW 一般只有部分标志位可由程序读写；IR、MAR、MDR、Y、Z 对汇编程序透明，程序看不见它们。考试里问“哪些寄存器程序员可见”，考的就是这条分界线。

数据通路由三样东西构成：能存数的部件（寄存器、存储器）、能算数的部件（ALU、加法器、移位器）、以及把数据从一个部件送到另一个部件的连线与选择器（总线、多路选择器）。按互连方式，常见三种：

- **单总线**：所有部件挂在一条内部总线上。一个时钟周期只能有一个数据在总线上跑，因此一条指令要拆成很多拍。
- **多总线**：两条或三条总线并行传送，一拍能完成多个数据传送，速度快，但线多、驱动器多、芯片面积大。
- **专用通路**：寄存器堆做成多端口，ALU 输入直接连寄存器输出，不再绕公共总线。现代 CPU 基本是这种结构。

先看单总线。图里上半部分是取指要用的部件，下半部分是执行要用的部件，中间那条粗线就是内部总线。



(1) PC→MAR (2) M(MAR)→MDR，PC+Δ (3) MDR→IR
(4) R2→Y (5) (R1)+(Y)→Z (6) Z→ 总线 →R1

读图时先看编号：1 到 3 是取指，4 到 6 是执行；粗线是内部总线，虚线是控制信号。编号与下面的节拍一一对应。

以 `ADD R1, R2`（功能是 $R1 \leftarrow R1 + R2$）为例，单总线要拆成六拍（表中 Δ 为指令长度）：

节拍	微操作	有效控制信号	说明
T_0	PC→MAR	PCout, MARin	送出指令地址
T_1	M(MAR)→MDR，PC+Δ	MemRead, MDRin, PC+Δ	读指令，同时指向下一条
T_2	MDR→IR	MDRout, IRin	指令进入指令寄存器
T_3	R2→Y	R2out, Yin	先锁存一个操作数
T_4	(R1)+(Y)→Z	R1out, ALUadd, Zin	另一个操作数经总线到 ALU
T_5	Z→R1	Zout, R1in	结果写回

注意

不同教材对“取指占几拍”的约定并不一致。常见写法把 PC 加一和访存放在同一拍，前提是 PC 自带加一逻辑；如果题给的数据通路里 PC 只能经 ALU 加一，就必须单独占用一拍。做数据通路题时以题图为准，不要背拍数。

为什么非要 Y 和 Z？因为总线一拍只能送一个数，而 ALU 做加法必须同时拿到两个操作数。做法是先把 R2 锁进 Y，下一拍让 R1 上总线，ALU 的两个输入就都齐了。结果也不能直接写回寄存器堆：它要先锁进 Z，下一拍才能重新驱动总线。Y 是“操作数暂存”，Z 是“结果暂存”，两个都不是程序员能看见的寄存器。

访存指令更长。`lw R1, 0(R2)` 在执行段要先算地址：R2→Y，(Y)+0→Z，Z→MAR 并发读命令，M(MAR)→MDR，MDR→R1，这一串是五拍；加上取指的三拍，一共八拍。`sw` 还要先把要写的数据读出来，拍数只多不少。单总线 CPU 的 CPI 很大，主频再高也快不起来，这就是后来要改多总线、改专用通路、再上流水线的直接原因。

多总线的好处可以用一句话概括：把“一拍只能走一个数”的限制放宽。双总线下一拍能同时送两个操作数，三总线还能同时送回结果；如果寄存器堆本身有两个读口一个写口，一条寄存器型指令在 EX 段一拍就能算完。代价是每条总线都要一套驱动器和仲裁，芯片面积、功耗、布线都上去了。

把这些部件按“一条指令一拍走完”的方式连起来，就是**单周期数据通路**。取指、译码、执行、访存、写回全在一个时钟周期内完成，CPI 恒为 1。代价是时钟周期必须按最慢的指令（通常是 load）来定，简单的寄存器运算也得陪着等；而且取指和访存如果共用一个存储器，会在同一拍打架，所以单周期 CPU 通常把指令存储器和数据存储器分开。**多周期**实现把一条指令拆成几个短周期，部件可以复用，时钟周期短了，但 CPI 大于 1。408 的选择题常在这两种组织的 CPI、时钟周期和硬件复用上做对比。

9.3 控制器

控制器干两件事：一是决定“这一拍做哪个微操作”，二是把对应的控制信号发出去。前者是时序问题，后者是逻辑问题。按实现方式分硬布线和微程序两类。

硬布线控制器

硬布线控制器把控制信号写成指令译码信号、节拍信号和状态标志的组合逻辑函数。比如取指阶段：

$$PC_{out} = FE \cdot T_0, \quad MAR_{in} = FE \cdot T_0,$$
$$MDR_{out} = FE \cdot T_2, \quad IR_{in} = FE \cdot T_2.$$

其中 FE 是取指周期标志，T_i 是第 i 拍。执行阶段的信号再按操作码分叉，例如 $R1_{in} = ADD \cdot T_5$。这些乘积项用与门实现，同一信号的各个来源用或门汇总，所以叫“硬布线”。如果指令多、寻址方式多，逻辑表达式会非常庞杂，化简和改版都很痛苦；但它的速度快，信号只经过几级门延迟。RISC 处理器普遍采用硬布线，或者硬布线加少量微码。

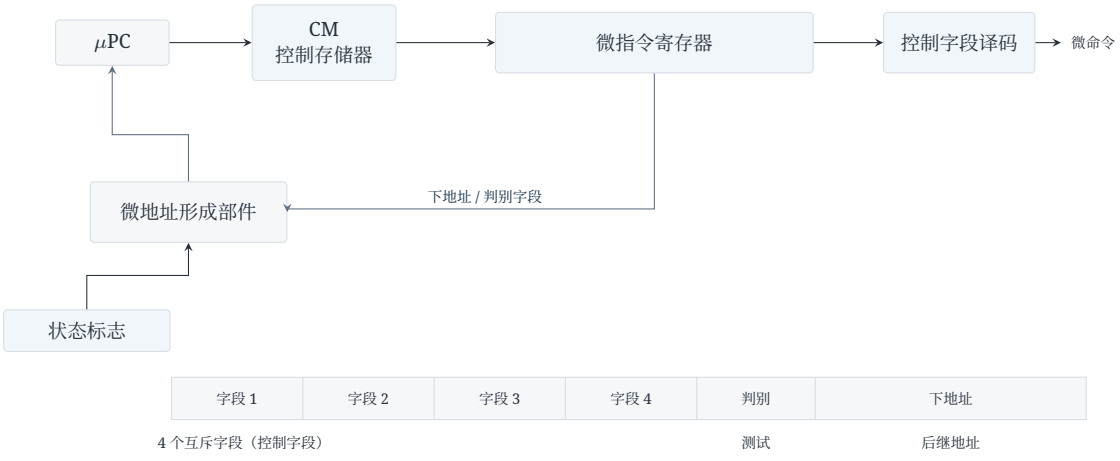
硬布线控制器还需要一个节拍发生器：从时钟出发，产生 $T_0, T_1, \ldots$，并按指令类型选择节拍序列。取指阶段所有指令共用同一套节拍；进入执行阶段后，不同指令走不同的分支。遇到中断请求，则在指令结束时插入中断周期。

微程序控制器

微程序控制器换了一个思路：把每条机器指令要做的微操作编成一段“微程序”，存进控制存储器（CM），执行时一条一条读出来。几个术语要分清：

- **微命令**：一个控制信号，比如“PC 打入 MAR”。
- **微操作**：一个不可再分的数据传送或运算，可能需要多个微命令同时有效。
- **微指令**：一个时钟周期内执行的一组微命令，外加下地址信息和判别信息。
- **微程序**：实现一条机器指令的微指令序列。

一条微指令通常分三个字段：控制字段（放微命令的编码）、判别测试字段（选择条件转移）、下地址字段（给出后继微指令地址）。控制字段可以再分成若干互斥的小字段，每个小字段经译码后产生一组微命令。



图的上半部分画的是执行一条微指令的路径：μPC 给出地址，从 CM 读出微指令送进微指令寄存器；控制字段经译码变成微命令，判别字段和下地址字段送回微地址形成部件，决定下一条微指令从哪里取。下半部分是微指令的位域划分（图中示例把控制字段切成四段）：每段内部的编码只在段内有效，段与段之间可以并行译码。

“互斥”是字段直接编码的关键。同一段里的微命令不会同时需要，所以可以用一个译码器从 m 位编码中选一个；如果字段有 m 位，最多能表示 2^m 个微命令，但通常要留一个编码表示“本

段不产生任何微命令”，因此实际最多 2^m-1 个。直接编码则是一位对一个微命令，速度最快，字长也最长；字段间接编码再省一些位，但要多一级译码，速度更慢。

微指令的格式还分水平型和垂直型。水平型一条微指令能并行控制很多微操作，字长大、微程序短；垂直型更像机器指令，一条只做一两个微操作，字长小、微程序长。实际处理器多用混合型。

控制存储器的容量计算有两个容易混的口径：CM 的字数由微程序地址位数决定，若 μPC 有 p 位，最多 2^p 条微指令；总容量再乘微指令字长。下地址字段有 n 位，只说明一条微指令能直接给出的地址范围是 2^n，不等于 CM 一定有那么大。取指微程序通常是所有机器指令共享的公共段，计算能容纳多少条机器指令时，要先把这段公共开销扣掉。

硬布线与微程序控制器的对比

	硬布线	微程序
控制信号来源	组合逻辑（门电路、PLA）直接产生	从 CM 读出微指令，译码后产生
速度	快，几级门延迟	较慢，受 CM 访问时间限制
修改与扩展	改指令就要改电路	改微程序即可，较规整
适用	RISC、指令少而规整	CISC、指令多而复杂

9.4 流水线

单周期 CPU 一条指令一个长周期，周期长度等于最慢那条路径的延迟；多周期 CPU 把指令拆成几段，每段一拍，但同一时刻只有一条指令在跑。流水线把这两件事结合起来：既把指令拆段，又让不同指令的不同段重叠。

按流水对象分，流水线可以是部件级（如浮点运算流水线）、处理机级（指令流水线）或处理机间（宏流水线）；按发射方式分，又有超标量、超流水线和超长指令字。这一节只讨论最基础的处理机级指令流水线。

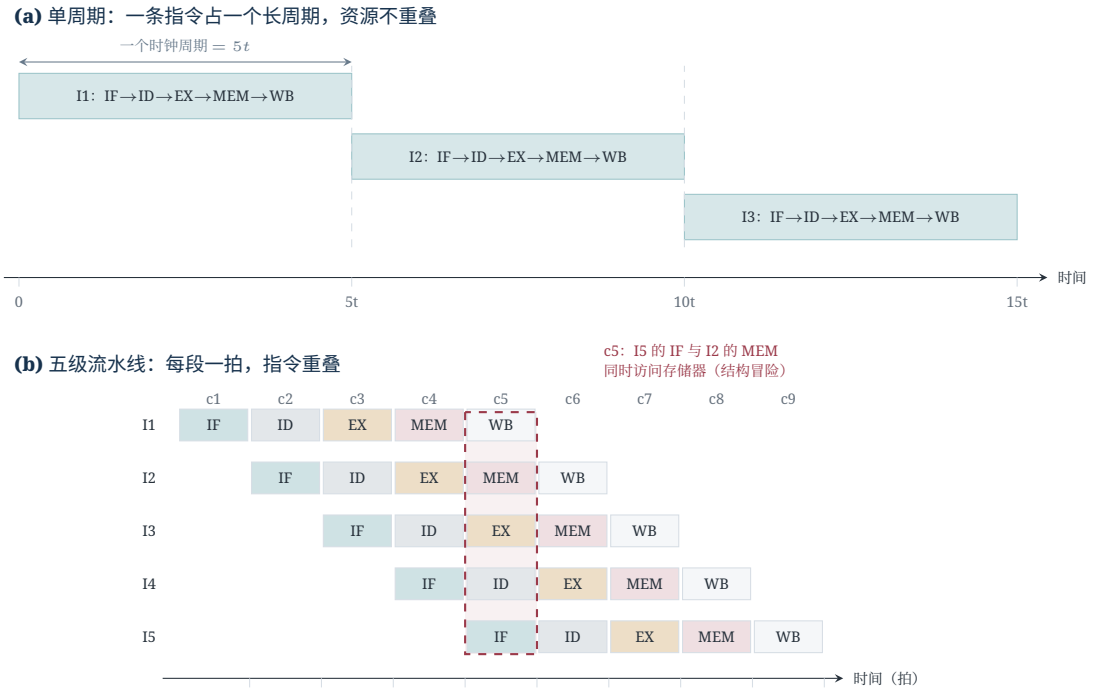
经典五级流水线把指令执行分成五段：

- **IF**：取指。PC 送指令存储器，指令进 IR，PC 更新。
- **ID**：译码，读寄存器堆，生成立即数；同时做冒险检测。
- **EX**：ALU 运算、计算访存地址、比较分支条件。
- **MEM**：访存。load 读数据，store 写数据；其他指令空过。
- **WB**：把结果写回寄存器堆。

每段之间放一个流水段寄存器（IF/ID、ID/EX、EX/MEM、MEM/WB），把上一段的输出锁住，供下一拍使用。时钟周期由最慢的一段加上寄存器开销决定：

$$T=\max_i\{d_i\}+t_{\text{reg}},$$

其中 d_i 是第 i 段组合逻辑延迟，t_{reg} 是段寄存器的建立时间、保持时间和时钟偏斜之和。各段延迟越均匀，T 越接近平均值，流水线越划算；某一段特别慢，其他段都得等它。



上半部分把“单周期”画成三个长条：I1 的五个阶段在一个时钟周期内依次走完，然后才轮到 I2，横向占 $5t$。下半部分把同样的五段拆到五个短拍里：I1 在第 1 拍取指，第 2 拍译码，此时 I2 正好进来取指。等到第 5 拍，五条指令分别处在五个不同的段，流水线满载，此后每拍完成一条。填满阶段要 5 拍，排空还要 4 拍，所以 n 条指令的总时间不是 n 拍，而是 $n+4$ 拍。

图里还标了一个结构冒险：第 5 拍 I5 在取指、I2 在访存，如果指令和数据共用一个存储器，两者就要抢端口。解决办法是把指令 Cache 和数据 Cache 分开，或者在取指端加指令缓冲。

设流水线有 k 段，各段等长，每段组合逻辑时间为 t，段寄存器开销为 t_r，时钟周期 $T = t + t_r$。执行 n 条指令：

$$T_{\text{pipe}} = (k+n-1)T, \qquad T_{\text{seq}} = nkt, \qquad S = \frac{T_{\text{seq}}}{T_{\text{pipe}}} = \frac{nkt}{(k+n-1)T}.$$

吞吐率是单位时间完成的指令数：

$$TP = \frac{n}{T_{\text{pipe}}} = \frac{n}{(k+n-1)T}, \qquad TP_{\max} = \frac{1}{T}.$$

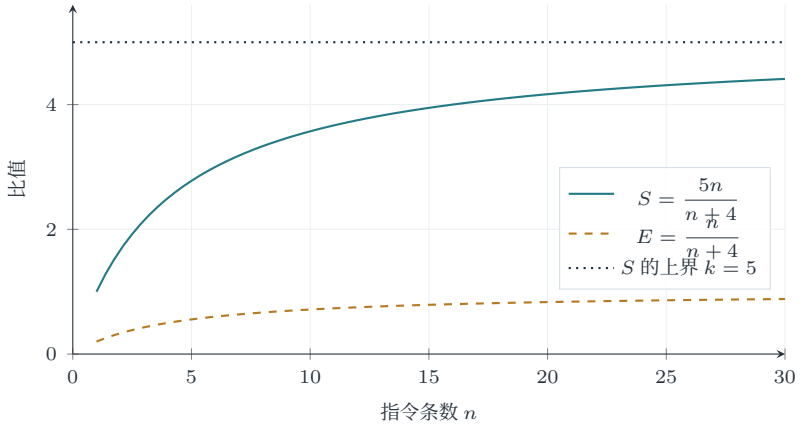
效率是时空图中“有效面积”占“总面积”的比例。每条指令真正干活的时间是 kt，占用的时空区是 kT，于是

$$E = \frac{nkt}{k(k+n-1)T} = \frac{nt}{(k+n-1)T} = \frac{S}{k}.$$

如果忽略段寄存器开销，即 $T = t$，三式简化为

$$S = \frac{nk}{k+n-1}, \qquad TP = \frac{n}{(k+n-1)t}, \qquad E = \frac{n}{k+n-1}.$$

这时 $n \to \infty$ 有 $S \to k$、$E \to 1$。考虑 t_r 后，加速比的上限变成 $kt/T = k/(1+t_r/t)$，效率的上限是 t/T，都达不到理想值。流水级数不是越深越好：段数增加，时钟可以更快，但寄存器开销占比变大，冒险的惩罚拍数也变多。



曲线按 $k = 5$、忽略段寄存器开销的理想情况画出。n 很小时加速比低，因为填满流水线的前 k 拍几乎没产出；n 增大后 S 从下方逼近 k，效率从下方逼近 1。图中 $n = 5$ 时 $S = 25/9 \approx 2.78$，只发挥了约一半的潜力，这说明流水线适合长指令流，不适合几条指令就结束的程序。

> **注意**
>
> 流水线提高的是吞吐率，不是单条指令的延迟。实际上因为多了段寄存器，单条指令从进入到离开的时间还略微变长。做题时先看清问的是“执行 n 条指令的总时间”“吞吐率”还是“单条指令延迟”，三者用的公式不同。

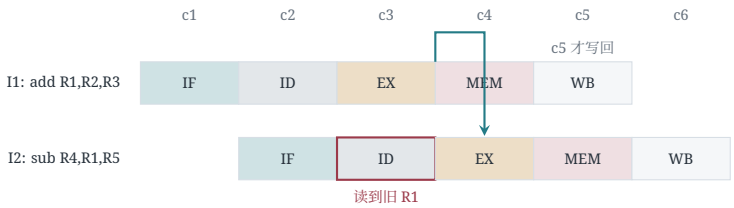
9.5 数据冒险与转发

流水线里，后面的指令可能要用前面指令还没写回的结果，这就是**数据冒险**。按依赖方向分三种：

- **RAW**（写后读，真依赖）：I1 写 R1，I2 读 R1。I2 必须等 I1 的结果。
- **WAR**（读后写，反依赖）：I1 读 R1，I2 写 R1。I1 必须先读到旧值。
- **WAW**（写后写，输出依赖）：I1、I2 都写 R1，最终必须是 I2 的值。

在按序流动的五级流水线里，读寄存器固定在 ID 段，写寄存器固定在 WB 段，指令又按序推进，所以 WAR 和 WAW 都不会发生：前面的读一定在后面的写之前，前面的写也一定在后面的写之前。只有 RAW 会出问题。WAR 和 WAW 出现在乱序执行、超标量或不同延迟的功能部件之间，靠寄存器重命名消除。

先看两条相邻的 ALU 指令：



I1 在 c3 的 EX 段末尾算出结果，放进 EX/MEM 寄存器；I2 在 c4 的 EX 段一开头就要用这个结果。转发通路把 EX/MEM 的值直接接到 ALU 的输入端，I2 就不必等 I1 到 c5 写回。注意 I2 在 c3 的 ID 段确实读到的是旧值，但这个旧值在 c4 进 ALU 之前会被多路选择器替换掉，所以不影响结果。

转发不是“把值写回寄存器堆再读出来”，而是在 EX 段入口加一组多路选择器，直接把流水段寄存器里的结果接过来。需要判断三种情况：

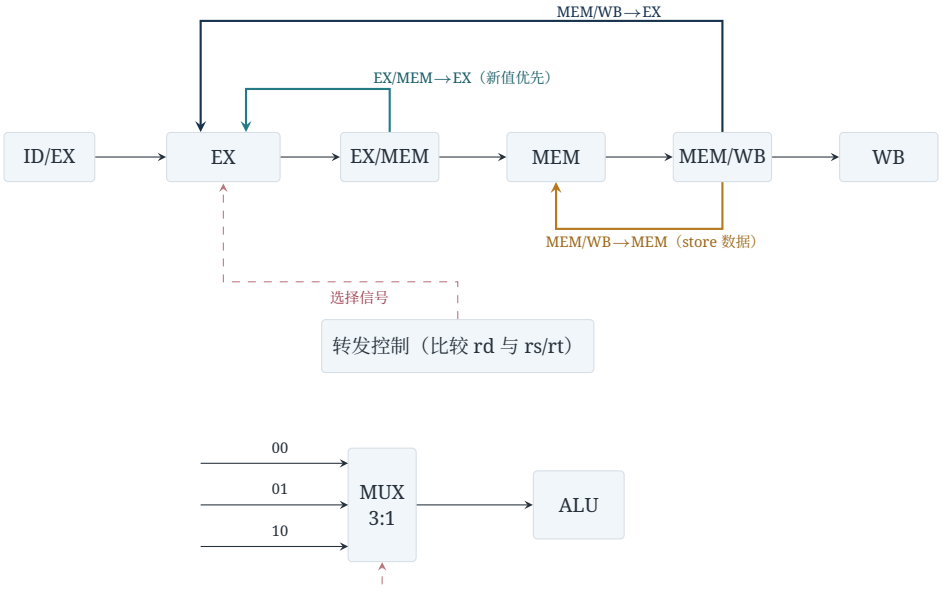
$$\text{转发 A} = \begin{cases} \text{EX/MEM}, & \text{EX/MEM.RegWrite} \wedge \neg\text{EX/MEM.MemRead} \wedge \text{EX/MEM.rd} \neq 0 \\ & \wedge \text{EX/MEM.rd} = \text{ID/EX.rs}, \\ \text{MEM/WB}, & \text{MEM/WB.RegWrite} \wedge \text{MEM/WB.rd} \neq 0 \wedge \text{MEM/WB.rd} = \text{ID/EX.rs}, \\ \text{不转发}, & \text{其他}. \end{cases}$$

B 操作数（`rt`）的判断完全对称。两个条件同时成立时，**EX/MEM 优先**：它是更近的那条指令的结果，MEM/WB 里的值更旧。比如

```
add R1, R2, R3
add R1, R1, R4
add R1, R1, R5
```

第三条指令要用的是第二条的结果，不是第一条的；如果按 MEM/WB 转发，就会拿到已经被覆盖的旧值。优先级就是为这种“连续写同一寄存器”准备的。条件里的 $\texttt{rd} \neq 0$ 也不能省：R0 恒为零，写 R0 没有效果，若不排除，寄存器号相同会误触发转发。

还有一点要注意：如果 EX/MEM 里装的是 load，它暂时只有算出的地址，真正的数据要到 MEM 段末才出来；这种情况不能从 EX/MEM 转发，只能靠下面的 load-use 停顿处理。

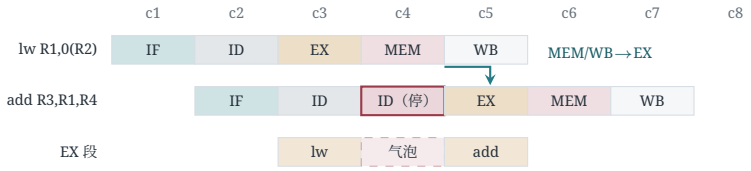


00：ID/EX 中的原值　01：EX/MEM 的结果　10：MEM/WB 的结果
01 与 10 同时命中时选 01，因为它来自更新的指令

上半部分是级间路径：EX/MEM 和 MEM/WB 各有一条回到 EX 入口的通路，store 的数据还可以从 MEM/WB 直接送到 MEM 段。下半部分把 EX 入口的选择器单独放大：三个输入分别是“寄存器堆读出的原值”“上一条指令的 ALU 结果”“再上一条指令的结果”，由转发控制部件给出选择码。通路越多，布线越复杂，所以真实处理器的转发条件都写在专门的控制逻辑里，不会临时用软件判断。

load-use 冒险

转发能解决 ALU 指令之间的 RAW，但有一类情况转发也来不及：load 之后紧接着用 load 的结果。



`lw` 的数据到 c4 的 MEM 段末尾才从存储器出来，而 `add` 本来要在 c4 的 EX 段开头就用它，中间差了一拍。转发通路再快，也不能把还没读出的数据送过去。解决办法是让 `add` 在 ID 段多停一拍：c4 时 PC 和 IF/ID 寄存器冻结，ID/EX 的控制信号清零，往 EX 段塞一个空操作（气泡）；到 c5，`add` 进入 EX，`lw` 的结果已经在 MEM/WB 里，从那里转发过来。

冒险检测在 ID 段做，条件是

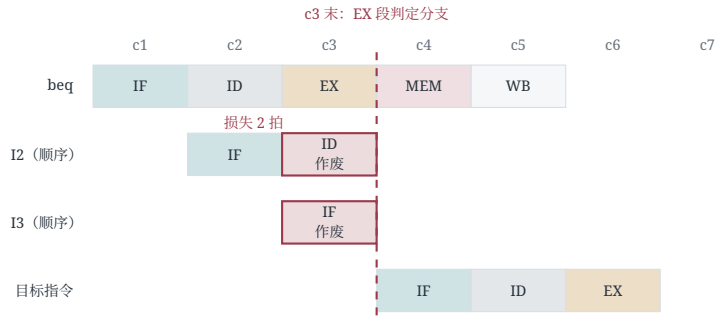
$$\text{ID/EX.MemRead} \wedge \big(\text{ID/EX.rt} = \text{IF/ID.rs} \ \vee \ \text{ID/EX.rt} = \text{IF/ID.rt}\big).$$

前一项说明上一条指令是 load，后一项说明当前指令的源寄存器和 load 的目的寄存器相同。检测到以后，硬件同时做三件事：PC 不更新、IF/ID 不写入、ID/EX 清空。三件事缺一不可，否则要么重复取指，要么把错误指令送进执行段。

转发也不是万能的。若消费者是 `sw` 且只把 load 的结果当“要写的数据”，数据在 MEM 段才需要，可以从 MEM/WB 直接送到 MEM 段，不必停；但如果 load 的结果被用作 `sw` 的基址寄存器，地址在 EX 段就要算，仍然要停一拍。判断的关键是“消费者到底在哪一段需要这个值”，而不是看指令的助记符。不过上面的检测条件为了简单，会把 store 数据这种情况也停一拍；只有数据通路额外支持 MEM 段转发时，才能把这一拍省掉。

9.6 控制冒险

取指令时 PC 已经指向下一条，但分支指令要等译码、甚至等 ALU 比较完，才知道该不该跳。在结果出来之前取进来的指令可能是错的，这就是**控制冒险**。



分支在 c3 的 EX 段末尾才判定。此时 c2 取进的 I2 已经进到 ID，c3 取进的 I3 在 IF，两条都得作废；如果分支成立，目标指令从 c4 才开始取。分支不成立时，I2、I3 正好是顺序路径上的指令，不用作废。所以“预测不跳转”在分支不跳时没有代价，跳转时损失 2 拍。

减少损失有三条路：

- **提前判定**：把比较器、目标地址加法器移到 ID 段，判定提前到 c2 末，只损失 1 拍。代价是 ID 段变长，可能拖慢时钟，还要额外的硬件。
- **预测**：静态预测固定“跳”或“不跳”；动态预测用分支历史表（BHT）记录最近几次的结果，用 2 位饱和计数器区分“强跳、弱跳、弱不跳、强不跳”，再配分支目标缓冲（BTB）缓存目标地址。预测对了没有损失，预测错了按上面的拍数冲刷。

- **延迟分支**：在指令集中留一个延迟槽，编译器把一条不受分支影响的指令填进去。硬件不冲刷，但指令集和编译器都变复杂了。

控制冒险的代价可以直接计入 CPI。若分支指令占 f_b，预测错误率为 p，每次错误损失 b 拍，在理想访存、无数据冒险的前提下：

$$\text{CPI} = 1 + f_b \cdot p \cdot b.$$

例如分支占 20%、错误率 10%、损失 2 拍，CPI 就是 $1 + 0.2 \times 0.1 \times 2 = 1.04$。这个式子只算控制冒险；如果还有 load-use 停顿，要再加上对应的项。不同来源的停顿不是简单相加，因为它们可能在同一拍重叠，精确值要看流水线时空图。

9.7 结构冒险与流水线控制

结构冒险是硬件资源不够用。除了图里画出的“取指和访存抢存储器”，还有两个常见位置：

- **寄存器堆端口**：ID 段要同时读两个源寄存器，WB 段要写一个目的寄存器。寄存器堆通常做成两读一写；如果同一拍读写同一个寄存器，可以约定“前半拍写、后半拍读”，让 ID 读到新值，否则就要额外停顿或加 WB→ID 转发。
- **功能部件**：经典五级流水线只有 EX 段用 ALU，不存在冲突；但如果把乘法、除法也放进 EX，或者多条指令同时进 EX，就需要多套功能部件或排队。

流水线控制器处理冒险的动作可以归结为三句话：**冻结**（PC、IF/ID 不更新）、**清空**（把 ID/EX 的控制信号清零，插入气泡）、**转发**（选择更近的结果）。数据冒险用转发加必要的冻结，控制冒险用冲刷加预测，结构冒险只能加资源或改变指令流。

9.8 从流水线到多核

单条流水线每拍最多完成一条指令，CPI 的下限是 1。要继续提高吞吐率，就得让一拍发射多条指令，这就是**超标量**。超标量处理器有多套取指、译码和执行部件，硬件要检查同一拍内多条指令之间的依赖；再配合乱序执行和寄存器重命名，可以把 WAR、WAW 也消掉，让 CPI 小于 1。

按 Flynn 分类，单核标量处理器是 SISD，向量处理器和 GPU 的数据并行部分是 SIMD，多核处理器和多机集群是 MIMD；MISD 在实际中很少见。

再往上就是**多核**：一个芯片上放多个处理器核，每个核有自己的寄存器和一级 Cache，共享最后一级 Cache 和主存，核间通过片内互连通信。多个核可以同时执行不同的指令流；共享数据时要靠 Cache 一致性协议（如 MESI）维护。

9.9 把这一章串起来

数据通路决定一拍能做什么，控制器决定每一拍做什么，流水线把“串行”改成“重叠”。冒险则要看“值什么时候需要、什么时候产生”：ALU 结果在 EX 末产生，消费者在 EX 初需要，差一拍，转发正好补上；load 结果在 MEM 末产生，消费者在 EX 初需要，差两拍，转发补不了，只能停一拍；分支结果在 EX 末产生，而取指在 IF 就要用，差得更远，只能预测或冲刷。

10 习题

10.1 基础过关题

1. （单项选择题）下列关于指令周期、机器周期和时钟周期的叙述中，正确的是

(A) 一条指令的指令周期可由若干机器周期组成，一个机器周期又由若干时钟周期组成，不同指令的指令周期可以不同

(B) 时钟周期是 CPU 完成一次主存访问所需的时间

(C) 机器周期与时钟周期是同一个概念，只是叫法不同

(D) 为保证流水线正常工作，所有指令的指令周期必须相同

2. （单项选择题）某 CPU 采用单总线数据通路，ALU 的两个输入端分别接暂存器 Y 和内部总线。执行 `ADD R1, R2`（功能 $R1 \leftarrow R1 + R2$）时，设置暂存器 Y 的主要原因是

(A) 提高 ALU 的运算速度

(B) 暂存一个操作数，解决单总线一次只能传送一个数据的矛盾

(C) 保存 ALU 产生的状态标志

(D) 在取指阶段暂存指令

3. （单项选择题）下列关于硬布线控制器和微程序控制器的叙述中，正确的是

(A) 微程序控制器的控制信号由组合逻辑电路直接产生，因此速度更快

(B) 硬布线控制器修改指令功能时，只需改写控制存储器中的微程序

(C) 微程序控制器把控制信号以微指令形式存放在控制存储器中，增加或修改指令只需改写相应微程序

(D) 微程序控制器不需要时钟，因而不存在时序问题

4. （单项选择题）在经典五级流水线（IF/ID/EX/MEM/WB）中，已配置 EX/MEM→EX 与 MEM/WB→EX 的转发通路，且寄存器堆在同一周期内先写后读。下列叙述中正确的是

(A) 相邻两条 ALU 指令之间的 RAW 冒险至少需要阻塞 1 个周期

(B) `lw` 指令与其后紧跟的使用该结果的 ALU 指令之间，仅靠转发不能消除冒险，仍需阻塞 1 个周期

(C) `lw` 的结果在 MEM 段末已经可用，可以直接转发到同一周期 EX 段的输入端，故 load-use 冒险无需阻塞

(D) 结构冒险可以通过编译器重排指令完全消除

5. （填空题）某微程序控制器的微指令字长为 32 位，格式为：控制字段 20 位、判别测试字段 3 位、下地址字段 9 位。控制字段分成 4 个互斥字段，每个 5 位，采用字段直接编码，每个字段留出一个编码表示“不产生微操作”。则：

   (1) 该微指令最多可控制 ____ 个微操作；

   (2) 控制存储器最多有 ____ 个字；

   (3) 若每条机器指令对应 4 条微指令，另有一段占 3 条微指令的公共取指微程序，则最多可容纳 ____ 条机器指令。

6. （填空题）某 5 段流水线各段的组合逻辑延迟分别为 $3,\ 2,\ 4,\ 2,\ 3$ ns，流水段寄存器延迟为 0.5 ns，忽略时钟偏斜。连续执行 200 条指令且不发生任何冒险。则：

   (1) 时钟周期为 ____ ns；

   (2) 执行这 200 条指令共需 ____ ns；

   (3) 相对非流水线实现（每条指令 $3+2+4+2+3=14$ ns，不计寄存器延迟）的加速比为 ____（保留两位小数）。

10.2 真题风格与综合训练

7. （真题风格 · 单项选择题）某 5 段流水线（IF/ID/EX/MEM/WB）中，分支指令要到 EX 段末才能确定是否跳转，处理器采用“总预测不跳转”的静态策略。程序中分支指令占 20%，其中 30% 实际发生跳转。忽略其他冒险与流水线建立开销，则该程序的 CPI 约为

   (A) 1.06　(B) 1.12

   (C) 1.20　(D) 1.40

8. （真题风格·单项选择题）某 5 段流水线处理器的取指和访存共用同一个单端口存储器，未分离指令 Cache 与数据 Cache。指令中 load/store 占 25%，其余指令不访问数据存储器；每条访存指令都要占用一个 MEM 段。发生争用时让访存指令优先，取指阻塞 1 个周期。忽略其他冒险，程序足够长，则 CPI 约为

(A) 1.00　(B) 1.13

(C) 1.25　(D) 1.50

9. （综合应用题·数据通路与指令周期）某 16 位 CPU 采用单总线数据通路，主要部件有：PC（带独立自增通路，自增不占用总线）、MAR、MDR、IR、通用寄存器 R0–R3、暂存器 Y 和 Z、ALU。约定：

- 单总线同一时钟周期只能传送一个数据，寄存器堆的读、写都在数据传送的同一周期内完成；
- 主存的读或写在一个时钟周期内完成；
- ALU 的组合输出必须经 Z 锁存后才能写回寄存器。

指令 `ADD R1, (R2)` 的功能为 $R1 \leftarrow R1 + \mathrm{Mem}[R2]$，源操作数采用寄存器间接寻址。回答：

(1) 写出取指周期的微操作序列及各步的有效控制信号；

(2) 写出执行周期的微操作序列及各步的有效控制信号，并说明为什么必须使用暂存器 Z；

(3) 该指令从取指到执行完毕共需多少个时钟周期？

(4) 若在该指令执行结束后响应中断，写出中断周期由硬件（中断隐指令）完成的三项操作及其先后次序。

10. **(综合应用题 · 流水线冒险与指令调度)** 某处理器采用经典五级流水线 (IF/ID/EX/MEM/WB), 配置完整的转发通路 (含向 EX 段两个输入端以及向 MEM 段存储数据的转发), 寄存器堆同一周期内先写后读; 紧邻的 load-use 冒险仍需阻塞 1 个周期。执行如下指令序列:

```
I1: lw   R1, 0(R2)     % R1 <- Mem[R2+0]
I2: add  R3, R1, R4    % R3 <- R1 + R4
I3: lw   R5, 8(R2)     % R5 <- Mem[R2+8]
I4: sub  R6, R5, R1    % R6 <- R5 - R1
I5: add  R7, R6, R3    % R7 <- R6 + R3
```

(1) 列出序列中所有的 RAW 相关, 标明相关寄存器, 并指出哪些会导致阻塞;

(2) 在完整转发下, 需要插入多少个阻塞周期? 用表格形式画出流水线时空图, 给出执行完该序列的总周期数;

(3) 仅通过重排指令消除全部阻塞, 给出新序列、总周期数, 并分别计算两种排列的 CPI (以 5 条指令、最后一条指令完成 WB 为口径)。

11 总线

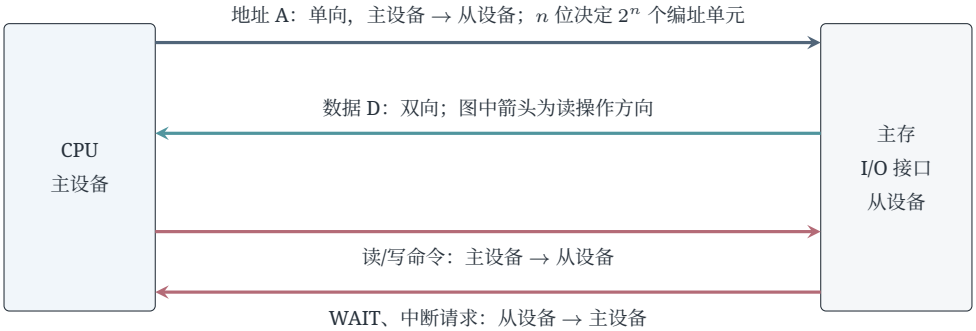
CPU 要从主存读一个 32 位字, 先得把地址送过去, 再发读命令, 等主存把数据放回总线。地址、数据、命令不能挤在同一根线上同时传, 于是分成三组信号线; 这三组线加上仲裁、定时等规则, 构成总线。

总线是部件之间传送信息的一组信号线。共享总线把多个部件挂在同一组线上，分时使用：同一时刻只能有一个主设备驱动总线，但可以有多个从设备同时接收。点对点总线则让每对部件独占一组线，后面的 PCIe 就是这种。主设备能主动发起事务，CPU、DMA 控制器都是；从设备被动响应，主存、I/O 接口通常是。

地址总线单向，从主设备指向从设备。n 根地址线能区分 2^n 个编址单元；按字节编址时，地址空间就是 2^n B。24 根地址线对应 16 MB。数据总线双向，读时从设备送到主设备，写时反过来；宽度决定一次能并行传多少位，常见 32 位、64 位。控制总线里每根线的方向固定，但不同线方向可以不同：读/写命令从主设备发往从设备，WAIT、中断请求则由从设备发回主设备。所以不能笼统地说控制总线是单向还是双向，要按每根线分别看。

按位置分，总线有片内总线、系统总线和通信总线。片内总线在芯片内部连接寄存器和 ALU；系统总线连接 CPU、主存和 I/O 接口；通信总线连接计算机与外设或另一台计算机，USB、PCIe、SATA 都属于这一类。408 主要考系统总线。

按传输方式还分串行和并行。串行总线在一根数据线上逐位传，线少，适合远距离；并行总线用多根数据线同时传多位，短距离内峰值高，但频率一高就受线间偏斜和串扰限制。后面的 PCI 与 PCIe 正好是这两种思路的代表。



同一时刻只有一个主设备驱动总线；多个从设备可以同时接收

看图时先看箭头：地址线只有朝右的箭头，数据线只有朝左的箭头，因为这里画的是读操作；控制线画了两根，一根朝右、一根朝左，说明控制总线里不同信号的方向可以相反。地址线下面那行字是常考的数量关系：地址位数决定能编址多少个单元，数据位数决定一次能搬多少位，两者不要混。

11.1 带宽：宽度、频率与每周期传送次数

总线快不快，不能只看时钟频率。32 位总线在 100 MHz 下每周期传一次，带宽是 $4\,\text{B}\times 100\times 10^6 = 400$ MB/s；同样 100 MHz，若改成每个时钟的上升沿和下降沿各传一次，带宽就翻到 800 MB/s。宽度、频率、每周期传送次数，三个量缺一不可。

要点 11.1: 峰值带宽与有效带宽

设数据总线宽度为 W 位，时钟频率为 f，每个时钟周期传送 N 次，则峰值带宽

$$B_{\text{peak}} = \frac{W}{8}\, f\, N \quad [\text{B/s}].$$

普通同步总线取 $N=1$；DDR 在上下沿都传，取 $N=2$。若一次传送占 k 个时钟周期，则把 f 换成 $1/(kT_{\text{clk}})$：

$$B = \frac{W}{8\,k\,T_{\text{clk}}}.$$

峰值带宽假设总线一直在传有效数据。实际事务里还有地址、仲裁、等待周期，真正有用的是有效带宽

$$B_{\text{eff}} = \frac{\text{一次事务传送的有效字节数}}{\text{该事务占用的总时间}}.$$

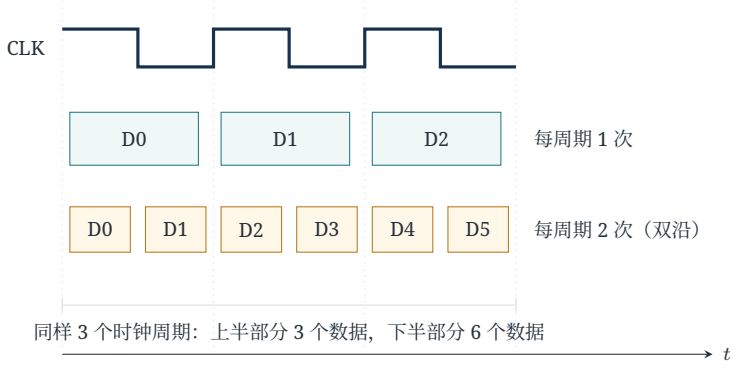
串行总线把“宽度”换成每通道 1 位，把频率换成符号率 R，再乘编码效率：

$$B = R \cdot \frac{k}{n} \cdot \frac{1}{8},$$

其中 k/n 是每 n 个传输符号里携带 k 位数据的比例，例如 8b/10b 编码取 $8/10$，128b/130b 编码取 $128/130$。

总线周期是完成一次总线操作所占的时间，它和时钟周期不是一回事。一个总线周期可以包含多个时钟周期，具体由协议规定；题目说“每个总线周期传送一次”时，要先把总线周期频率算出来，再乘每次传送的字节数。公式里的 k 就是起这个作用。

单位按十进制换算：1 MB/s $= 10^6$ B/s，1 GB/s $= 10^9$ B/s。408 题目没有特别说明时都按这个口径。

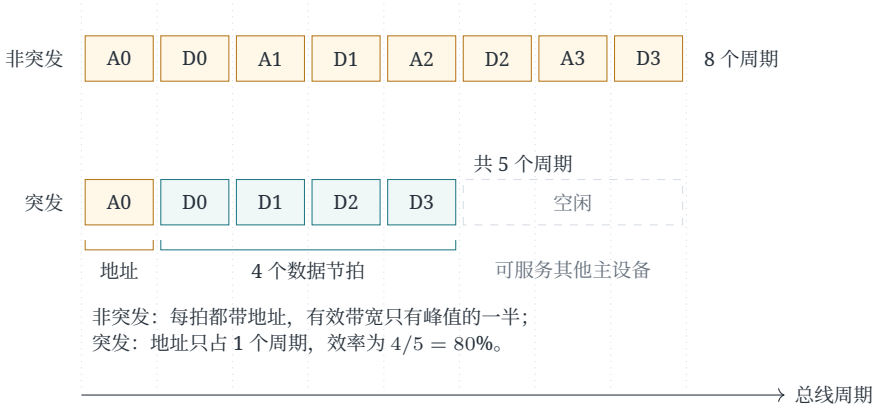


图的上面一行是普通同步总线：一个时钟周期一个数据块。下面一行是双沿传送：上升沿、下降沿各采一次，同样三个周期得到两倍数据。带宽公式里的 N 就是这里“每周期几个块”的倍数，不是时钟频率本身的倍数。

串行总线也能用同一套思路算。PCIe 3.0 每通道 8 GT/s，采用 128b/130b 编码，单向有效带宽为 $8 \times 10^9 \times \frac{128}{130} \times \frac{1}{8} \approx 0.985$ GB/s。$\times 16$ 就是 16 条通道并行，单向约 15.75 GB/s，双向合计约 31.5 GB/s。注意 GT/s 是每秒传输次数，不是每秒字节数，中间要乘编码效率再除以 8。

11.2 突发传输：一次地址，多个数据

Cache 块通常有 64 B。假设总线一次传 16 B，一次读块就要传 4 拍。如果每一拍都重新发地址，总线有一半时间在传地址，数据带宽白白浪费。突发传输就是针对这种情况：主设备只在开头发一次地址，后面连续传 n 个数据，地址由从设备自动递增。



图里同一个事务画了两遍：上面每传一个数据就重新发一次地址，8 个周期才传完 4 个数据；下面只在开头发一次地址，后面 4 个周期连续传数据，第 5 个周期就结束。总线周期固定时，省下来的 3 个周期可以给别的设备用。

设突发长度（数据拍数）为 n，地址阶段占 1 个总线周期，每个数据占 1 个周期，则

$$\eta = \frac{n}{n+1}, \qquad B_{\text{eff}} = \frac{nW/8}{(n+1)T_{\text{clk}}}.$$

若从设备慢，插入了 w 个等待周期，分母再加 w；若地址和数据复用同一组线，地址阶段无法与数据重叠，这 1 个周期的开销也躲不掉。突发越长，地址开销摊得越薄，但长度受从设备缓冲区、地址边界和最长占用时间限制；太长会让其他主设备等太久。

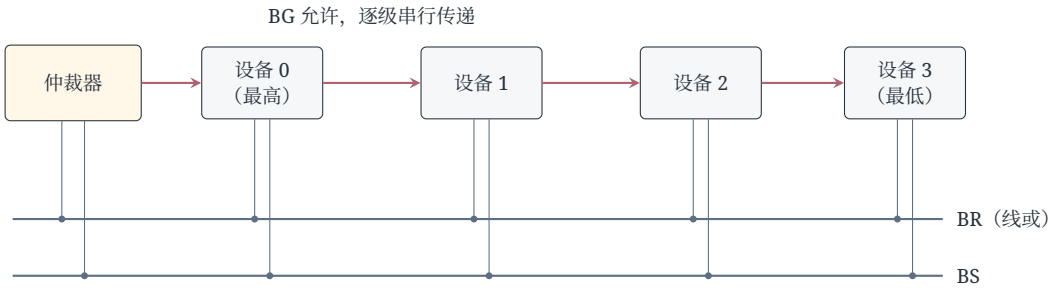
举个例子。128 位总线，100 MHz，突发传 8 个数据。地址 1 拍加数据 8 拍，共 9 拍，$T_{\text{clk}} = 10$ ns，耗时 90 ns。有效数据 $8 \times 16 = 128$ B，有效带宽 $128\ \text{B}/90\ \text{ns} \approx 1.42$ GB/s；峰值是 $16\ \text{B} \times 100\ \text{MHz} = 1.6$ GB/s，利用率 $8/9 \approx 88.9\%$。同样 8 个数据若每拍都发地址，需要 16 拍、160 ns，有效带宽只有 0.8 GB/s。

地址线和数据线分开的总线还能做流水线突发：当前数据在总线上传时，下一拍地址已经可以送出去。这样地址开销几乎被完全遮住。地址/数据复用的总线做不到这一点，因为同一组线不能同时传地址和数据。这种让地址和数据分时共用一组物理线的做法叫总线复用，它省引脚和走线，代价就是地址阶段无法与数据重叠，突发效率要低一些。

11.3　仲裁：多个主设备怎么排队

CPU 和 DMA 控制器可能同时想用总线。总线只能给一个，仲裁要回答三件事：谁先得到、多久能得到、得到后能占用多长时间。裁决逻辑集中在一个仲裁器里，叫集中式仲裁；没有仲裁器、各设备自己比较的，叫分布式仲裁。408 重点考集中式的三种：链式查询、计数器定时查询、独立请求。

链式查询只用三根线：总线请求 BR、总线忙 BS、总线允许 BG。所有设备的 BR 连在一起，形成线或：只要有一个设备拉有效，仲裁器就看到请求。仲裁器发出 BG 后，BG 从离它最近的设备开始，一级一级往下传。

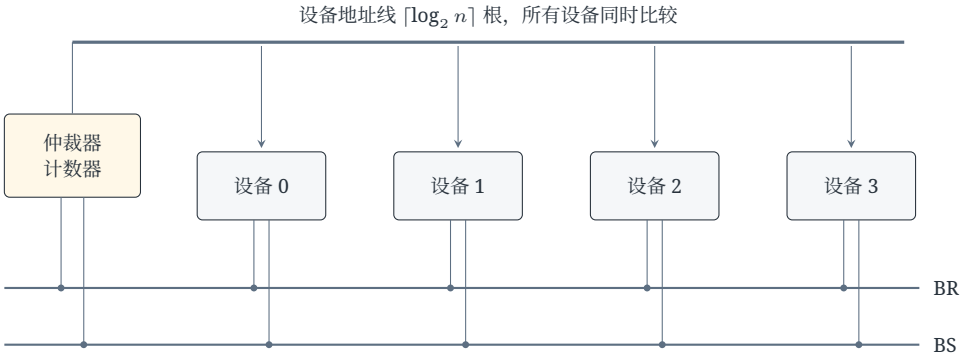


设备无请求时把 BG 原样右传；有请求时截获 BG、置 BS，并停止继续传递

看图时顺着 BG 箭头走：BG 不是每个设备一根，而是一根线从仲裁器出发，穿过每个设备。设备 0 离得最近，BG 先到它；它不请求，BG 才继续往右。因此优先级由物理位置固定死，设备 0 最高，越远越低。BR 是所有设备共用的一根线，任何设备都能拉有效；BS 表示总线正在被占用，防止别的设备在事务中途插进来。

链式查询的优点是线少，加设备只要把 BG 链续下去。缺点是 BG 每过一级都有门延迟，设备越多，最远端的等待越长；某一级链路断了，后面的设备就再也收不到 BG。优先级也没法改，远端设备在重负载下可能长期轮不到，也就是饥饿。

计数器定时查询把“位置”换成“编号”。仲裁器里有一个计数器，收到 BR 后把计数值送上设备地址线，所有设备同时拿自己的编号去比较；编号相同且有请求的设备获胜，置 BS，计数器随即停止。

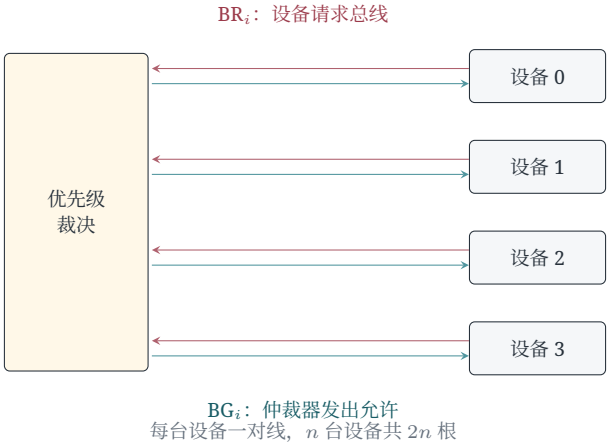


编号与计数值相同且发出请求的设备获胜；它置 BS，计数器停止

图里没有 BG 链，取而代之的是一组地址线。计数器从某个初值开始加 1，每个计数值同时送到所有设备；谁的编号等于计数值、并且它的 BR 有效，谁就拿到总线。BS 一旦置位，计数器停止，本轮仲裁结束。

计数器的初值决定优先级策略。每次从 0 开始，编号小的优先，是固定优先级；从上一次获胜者的下一个编号开始，各设备轮转，是循环优先级；初值由程序写入，就是可编程优先级。代价是地址线需要 $\lceil \log_2 n \rceil$ 根，加上 BR、BS 共 $2 + \lceil \log_2 n \rceil$ 根；最坏情况下计数器要扫到最后一个编号，响应时间随设备数增长。

独立请求方式更直接：每个设备都有自己的请求线 BR_i 和允许线 BG_i，仲裁器内部并行比较所有请求，直接选中优先级最高的那个。



图里每台设备都有两根独立的线，仲裁器一眼能看出谁在请求，不需要逐级传 BG，也不需要计数器扫描。所以独立请求响应最快，优先级也能由内部逻辑或程序灵活设定。代价是线数随设备数线性增长：n 台设备要 $2n$ 根请求/允许线。设备一多，布线和连接器都吃不消，因此它适合主设备数量不多的系统，PCI 就采用这种方式。

三种集中式仲裁方式的取舍

方式	信号线数	优先级	响应速度	主要问题
链式查询	3	由物理位置固定	逐级传递，慢	链路故障会切断后续设备
计数器查询	$2+\lceil\log_2 n\rceil$	可由计数初值设定	逐个比较，中等	设备多时地址线变宽
独立请求	$2n$	可编程	并行裁决，快	线数随设备数线性增长

分布式仲裁没有中央仲裁器。每个主设备有唯一的仲裁号，需要总线时把自己的号送到共享仲裁线上；线与之后，线上出现的是所有请求号中的最大值（或最小值，取决于约定），谁发现自己的号等于这个值，谁就获胜。CAN 总线用的就是这种思路：标识符小的帧优先，而且冲突时不会破坏已经在传的数据。

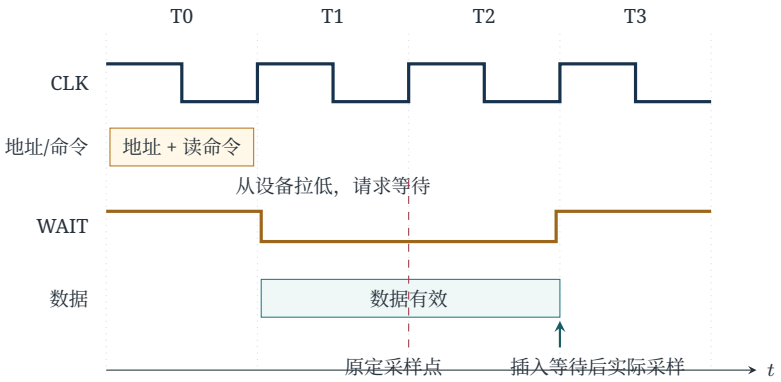
注意

链式查询的 BG 只有一根线，按物理顺序串行传递，不是每个设备一根。独立请求的 $2n$ 根线指 n 根 BR$_i$ 加 n 根 BG$_i$，每个设备一对，仲裁器不再需要逐级传递允许信号。优先级高也不等于响应快：链式查询里近端设备优先级最高，但 BG 仍要一级级传过去。

11.4 定时：按节拍走，还是按握手走

总线上的信号不会瞬间稳定。主设备什么时候放地址、从设备什么时候放数据、主设备在哪个时刻采样，需要一套规则，这就是总线定时。最常见的是同步和异步两种。

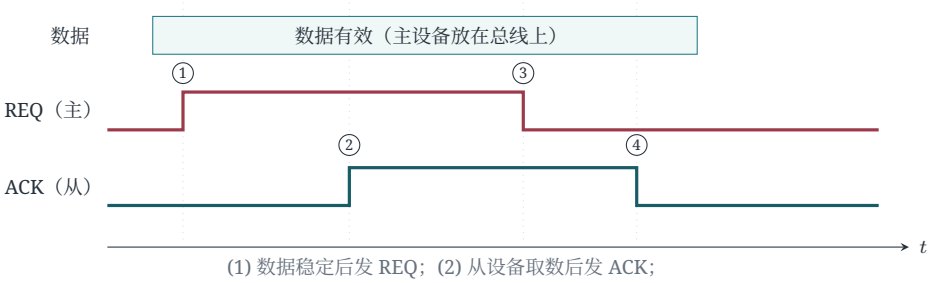
同步通信让所有设备共用一根时钟线。地址和命令在某个时钟沿发出，从设备在固定数量的周期内把数据放上线，主设备在约定的沿采样。整个事务占几个周期，协议事先规定死。



图里 T0 发地址和读命令，从设备在 T1 准备数据，正常应在 T2 的上升沿采样。如果从设备来不及，就在 T1 期间把 WAIT 拉低；主设备在 T2 沿看到 WAIT，插入一个等待周期，把采样推到 T3。等待周期里总线不传有效数据，但时间照样过去，所以算有效带宽时必须把它算进分母。

同步通信的优点是控制逻辑简单，所有设备按同一节拍走。问题是时钟偏斜：频率越高、总线越长，同一个时钟沿到达不同设备的时刻差越大，可能一个设备已经开始采样，另一个还没收到时钟。另外，快慢设备必须迁就同一个时钟，慢设备会拖住整个总线。半同步通信在同步基础上加 WAIT，让慢设备插入等待周期，兼顾简单和灵活；它仍然是同步总线，不是另一种独立的电气方式。

异步通信不传时钟，改用两根握手线：主设备发请求，从设备回应答。谁也不用猜对方多久能完成，每一步都等对方的信号。



(1) 数据稳定后发 REQ；(2) 从设备取数后发 ACK；
(3) 主设备见 ACK 撤 REQ；(4) 从设备见 REQ 撤销撤 ACK

图里数据先稳定，REQ 才拉高，这保证从设备看到请求时数据已经可用。从设备取走数据后拉高 ACK；主设备看到 ACK 才撤销 REQ；从设备看到 REQ 撤销后才撤销 ACK。四个动作互相等待，叫全互锁握手。它最可靠，也最慢，因为每一拍都要等一个来回。

握手方式按“谁等谁”分成三种。不互锁：主设备发完请求，过一段固定时间就认为传输完成，直接撤销；从设备也按固定延迟回应答，双方都不看对方。速度快，但设备一慢就出错。半互锁：主设备等到 ACK 才撤请求，从设备发完 ACK 后经过固定延迟就撤销，不再等请求撤销。全互锁：双方都等对方，就是图上这四条边。

异步通信没有时钟偏斜，快慢设备按各自速度握手，适合距离较远或速度差异大的场合。代价是每次传输多出几次信号跳变和等待，控制逻辑也比同步复杂；大批量连续传输时，握手开销会摊薄有效带宽。

四种定时方式的对比

方式	协调方式	优点	代价
同步	统一时钟	逻辑简单, 适合速度接近的设备	时钟偏斜；慢设备拖累整体
半同步	时钟 + WAIT	保留同步的简单性, 能兼容慢设备	等待周期占用总线时间
异步	请求/应答握手	快慢自适应，无时钟偏斜	握手开销，控制逻辑复杂
分离式	请求阶段与数据阶段分开	等待期间总线可服务其他事务	需要事务标识和缓冲, 读延迟可能变大

分离式通信把一次读事务拆成两段：主设备先发地址和命令，然后立刻释放总线；从设备准备好数据后，再申请总线把数据送回来。两段之间，总线可以去服务别的设备，吞吐率因此提高。代价是要记录这是哪个事务的响应，主设备也要能接受迟到的数据，控制比普通同步或异步都复杂。

11.5 从共享并行到点对点串行

总线标准规定机械尺寸、电气特性、时序和通信协议。符合同一标准的设备才能插在一起、按同样的节拍说话。408 常拿 PCI、PCIe、USB 做对比，重点不在背年份，而在传输方式和带宽口径。

PCI 是并行共享总线，32 位或 64 位数据线，33 MHz 或 66 MHz 时钟，地址和数据复用同一组 AD 线。32 位、33 MHz 时峰值带宽约 133 MB/s。它支持突发传输，一次地址后连续传多个数据；仲裁采用集中式独立请求，每个主设备一对 REQ#/GNT# 线。PCI 还支持即插即用，启动时由软件分配地址和中断号。

PCIe 换了架构：串行、点对点、全双工。每条通道（lane）有一对发送差分线和一对接收差分线，可以组合成 $\times1$、$\times4$、$\times8$、$\times16$。设备通过交换器连接，每条链路独占，不存在多个设备抢同一组线的问题，所以 PCIe 没有传统意义上的总线仲裁。编码上，PCIe 1.x/2.x 用 8b/10b，3.0 起用 128b/130b，计算带宽时要把编码开销扣掉。

USB 是主机控制的串行总线，树形拓扑，一个主机控制器最多带 127 个设备（含集线器）。所有通信由主机发起，设备不能主动往总线上发数据。它分控制、批量、中断、同步四种传输，分别照顾配置、大块数据、小延迟和实时流。USB 2.0 高速 480 Mbps；USB 3.0 超高速 5 Gbps，增加独立的发送和接收通道，实现全双工。

从并行共享走到串行点对点，原因很直接：时钟频率越高，并行线之间的偏斜和串扰越难控制，等长布线和屏蔽的成本迅速上升；差分串行信号抗干扰好，更容易提频。点对点连接又把共享总线的争用去掉了，靠交换器扩展端口。代价是协议更复杂，链路上传的是数据包，不再是简单的地址、数据、控制三组电平。

12 习题

12.1 基础过关题

1. (单项选择题) 下列关于系统总线三组信号线的叙述中，正确的是

(A) 地址总线是双向的，数据总线是单向的

(B) 中断类型号经数据总线传送，总线请求信号经控制总线传送

(C) 控制总线中每根信号线的方向相同，均由 CPU 指向 I/O 接口

(D) 数据总线的位数决定了主存的最大可寻址空间

2. (单项选择题) 采用链式查询方式进行总线仲裁时，下列叙述中**错误**的是

(A) 只需总线请求 BR、总线忙 BS、总线允许 BG 三根控制信号线

(B) 离仲裁器越近的设备优先级越高

(C) BG 信号逐级串行传递，某一级链路故障会使其后的设备都得不到总线

(D) 可通过改变计数器的初值灵活调整各设备的优先级

3. (填空题) 某总线有 16 个主设备。若采用计数器定时查询方式仲裁，除总线请求线 BR 和总线忙线 BS 外，还需要设备地址线，共需控制信号线_____ 根；若改用独立请求方式，共需控制信号线_____ 根。

4. (填空题) 某同步总线宽度为 64 位，总线时钟频率为 500 MHz，一个总线周期包含 2 个时钟周期，每个总线周期传送一次数据。该总线的带宽为_____MB/s。

5. (单项选择题) 异步总线通信中，主设备撤销请求信号必须以收到从设备的应答信号为条件，从设备撤销应答信号也必须以检测到请求信号已撤销为条件。这种握手方式称为

(A) 不互锁方式 (B) 半互锁方式

(C) 全互锁方式 (D) 同步方式

6. (填空题) 某同步总线宽度为 32 位，时钟频率为 100 MHz，每个时钟周期传送一次数据。采用突发传输方式从主存读出 4 个 32 位数据，只需在开始时发送 1 个地址（占 1 个时钟周期），随后连续传送数据，忽略其他开销。完成这次传输需要_____ns，有效数据传输带宽为_____MB/s。

12.2 真题风格与综合训练

7. (单项选择题) 某同步总线采用地址/数据线复用方式，地址线和数据线共 32 根，时钟频率为 200 MHz，每个时钟周期在上升沿和下降沿各传送一次数据。一次突发写事务传送 32 字节数据，其中地址阶段占 1 个时钟周期，数据阶段连续传送，忽略其他协议开销。该事务的有效数据传输带宽约为

(A) 1.07 GB/s (B) 1.28 GB/s

(C) 1.60 GB/s (D) 2.56 GB/s

8. (单项选择题) 某总线有 8 个主设备采用链式查询仲裁，设备按距仲裁器由近到远编号为 0～7。仲裁器收到请求后产生 BG 信号需 10 ns，BG 每经过一个不请求总线的设备需 5 ns，请求设备接收 BG 不再引入延迟。若只有设备 7 请求总线，从仲裁器收到请求到设备 7 收到 BG 至少需要

(A) 35 ns (B) 40 ns

(C) 45 ns (D) 50 ns

9. (综合应用题) 某同步总线宽度为 128 位，时钟频率为 200 MHz，每个时钟周期传送一次数据。CPU 通过该总线从主存突发读出一个 64 B 的数据块：地址阶段占 1 个时钟周期，随后连续传送 4 个数据，忽略总线申请与仲裁开销。由于主存较慢，从设备在数据阶段开始前通过 WAIT 信号插入了 1 个等待周期。回答下列问题：

(1) 该事务共占多少个时钟周期？总时间是多少纳秒？

(2) 该事务的有效数据传输带宽是多少？总线的理论峰值带宽是多少？带宽利用率为多少？

(3) 说明 WAIT 等待周期的作用，并解释为什么这种情况下不能靠继续提高总线时钟频率来提升有效带宽。

10. （综合应用题）某系统有 A、B、C 三个主设备，采用集中式独立请求仲裁，固定优先级为 A 最高、B 次之、C 最低，总线不可抢占：一个设备获得总线后必须传完整个事务才能释放。仲裁在事务结束的同一周期内完成，不额外占用总线周期；获得总线的设备从该周期起立即开始传输。各设备的事务长度与请求到达时刻如下（时刻以总线周期为单位，从 0 开始计）：

设备	请求到达时刻	事务占用周期数	优先级
A	0	4	高
C	1	2	低
B	2	6	中

回答下列问题：

(1) 按固定优先级调度，写出三个设备获得总线的顺序、各自的开始与完成时刻，以及各自的等待时间（从请求到达至开始传输的周期数）。

(2) 计算 $[0, 12)$ 区间内的总线利用率，并求三个设备的平均等待时间。

(3) 若把 C 的优先级提到 B 之上（仍低于 A），重新给出服务顺序与平均等待时间，说明优先级设置对平均等待的影响。

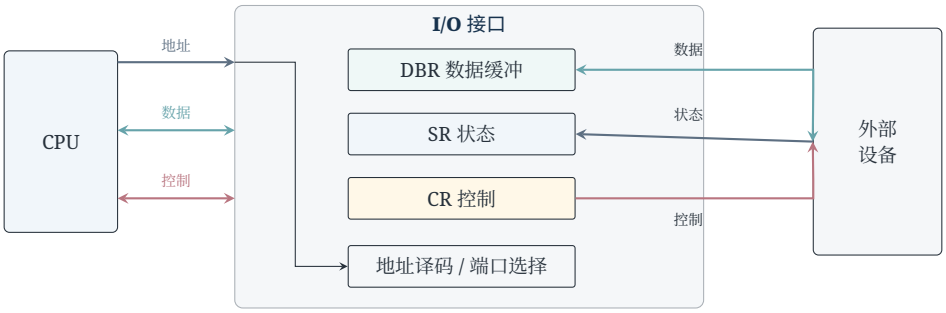
(4) 若 A 以很短的间隔不断发起长事务，会发生什么现象？给出两种在保持独立请求连线的前提下缓解该问题的措施。

13 输入输出系统

先看一个具体场景：程序要从磁盘读 4 KB 数据到内存。CPU 把起始地址和字节数写进磁盘控制器的寄存器，磁盘开始寻道、旋转；数据到达控制器后，DMA 把数据写进主存；整块传完，控制器向 CPU 发一个中断。短短一条读命令背后，既有接口里的端口寄存器，也有总线的争夺和中断现场的保存。本章就按“设备怎样接入—CPU 怎样等—设备怎样通知—数据怎样搬运—磁盘为什么慢”的顺序展开。

13.1 从一根导线开始：I/O 接口做什么

键盘按下一次键，产生的是一个或几个电脉冲；CPU 的总线却是并行、电平固定、时序严格的。两者不能直接连在一起。中间那块电路就是 **I/O 接口**（I/O interface），也叫 I/O 控制器。它的职责可以分成几类：把外设送来的数据锁存住，等 CPU 来取；把 CPU 写来的数据保持住，等外设慢慢消费；把串行信号转成并行、把非 TTL 电平转成 TTL 电平、把模拟量转成数字量；接收 CPU 的读/写命令，向设备发出启动、停止等控制信号；把设备的忙、闲、出错等状态登记下来供 CPU 查询；设备需要服务时，向 CPU 发出中断请求。

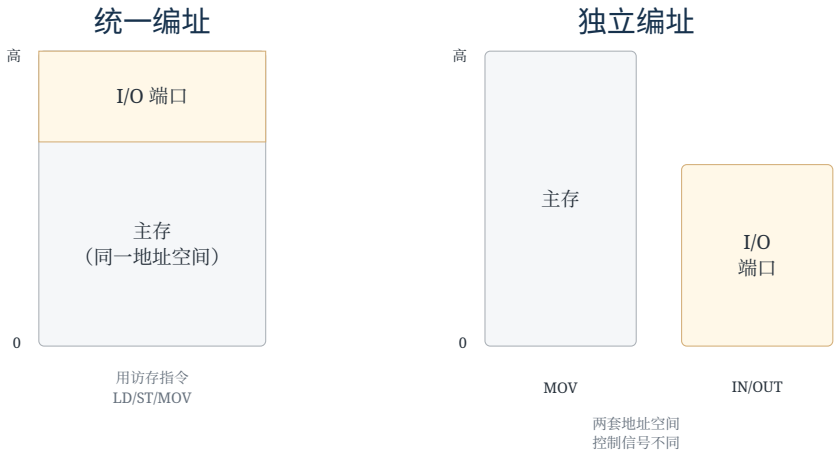


图里 CPU 一侧只有地址、数据、控制三组总线。地址送到译码器，译码器判断这次访问的是不是本接口、以及是接口里的哪一个寄存器；数据端口负责搬数据，状态端口只读，控制端口只写。设备一侧的时序可能和 CPU 完全不同，接口负责把两边的节奏对上。图中把数据、状态、控制画成三条独立的线，实际接口可能把它们复用在更少的引脚上，但“CPU 通过端口访问接口”这一点不变。

能被 CPU 用指令直接访问的寄存器叫 **I/O 端口**（I/O port）。一个接口通常有多个端口：数据端口、状态端口、控制端口，每个端口有自己的端口地址。接口是电路实体，端口是实体里“能被寻址的格子”。408 选择题常拿“一个接口就是一个端口”做干扰项，实际上一个接口往往包含三个以上的端口。

端口地址怎么放进地址空间，有两种做法。第一种是**统一编址**，也叫存储器映射 I/O：把端口地址安排在主存地址空间里，CPU 用普通的取数、存数指令访问端口，不需要专门的 I/O 指令。ARM、MIPS、RISC-V 通常采用这种方式。它的好处是指令多、寻址方式多，指针、数组、结构

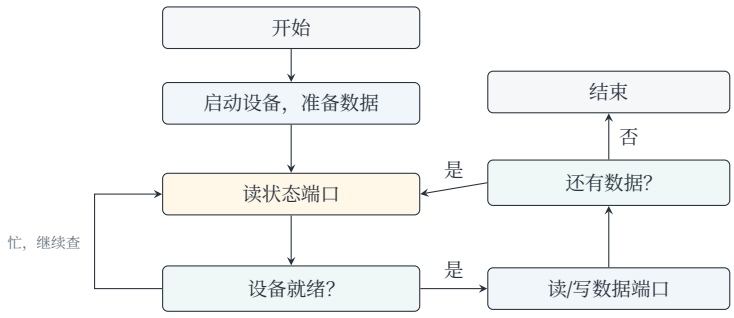
体都能直接指向端口；代价是端口占掉了一部分地址空间，可编址的主存范围相应减少。第二种是**独立编址**，也叫 I/O 映射 I/O：主存和端口各有一套地址空间，CPU 用专门的 I/O 指令（x86 的 `IN`、`OUT`）访问端口，用访存指令访问主存，并通过 `MEMR/MEMW` 与 `IOR/IOW` 等控制信号区分。它的好处是不占主存地址空间、端口地址位数少、译码简单；代价是需要专门的指令和引脚。



图左边是一套地址空间，I/O 端口只是其中一段，CPU 不需要知道自己访问的是内存还是端口，地址译码会把它送到正确的地方。图右边是两套地址空间，数值相同的地址在两边各有一个，靠指令和 `IOW/IOR` 这类信号区分。所以独立编址时，端口地址和某个主存地址数值相同并不会冲突；反过来，统一编址也不是“端口没有地址”，只是端口和主存共用一套地址。

13.2 程序查询：CPU 守着设备

最简单的一种 I/O 控制方式是程序查询。以打印机输出一个字符为例：接口里有一个“忙”位，CPU 先反复读状态端口；忙就继续读，直到状态变成“就绪”，再把字符写到数据端口。



查询循环由 CPU 执行；等待期间 CPU 不能转去运行别的程序

图里的循环就是程序查询的全部工作：读状态、判断、传送、再判断。设备慢的时候，CPU 大部分时间耗在“读状态—判断”这两步上，真正传送数据只是一瞬间。它不需要中断控制器，也不需要 DMA 控制器，硬件最省；代价是 CPU 和外设串行工作，CPU 的算力被白白占住。它适合单片机控制按键、LED 这类简单场合，或者设备速度本来就和 CPU 匹配的专用系统。

有些实现会在查询循环里插入其他任务，隔一段时间查一次，这叫定时查询；但它仍然是 CPU 主动去问设备，设备不能反过来通知 CPU。只要设备没有“主动敲门”的能力，CPU 就必须在某个时刻停下来检查。

13.3 中断：让设备主动找 CPU

程序查询的问题在于 CPU 必须守着设备。中断换了一个方向：设备准备好时，向 CPU 发一个中断请求；CPU 在完成当前指令后，暂停主程序，转去执行该设备的中断服务程序（ISR），处理完再返回。设备准备数据的那段时间，CPU 可以继续执行别的程序，因此 CPU 与外设是部分并行的。

中断源不只有 I/O 设备。时钟、电源故障、存储器校验错也会发中断；除零、溢出、缺页、非法指令、系统调用则来自 CPU 内部。408 通常把前者叫**中断**（外中断），后者叫**异常**（内中断）。外中断是异步的，和当前指令没有关系；异常是同步的，由当前指令触发。外中断里，可以被 IF 标志屏蔽的称为可屏蔽中断，键盘、磁盘等通常走这一类；电源故障、存储器校验错等用不可屏蔽中断（NMI），不受 IF 限制。

按 CPU 获得服务程序入口的方式，还可分为向量中断和非向量中断。向量中断由中断控制器在响应周期给出中断类型号，CPU 据此查表；非向量中断则由软件轮流查询各个设备，确定是谁发的请求。408 主要考向量中断。

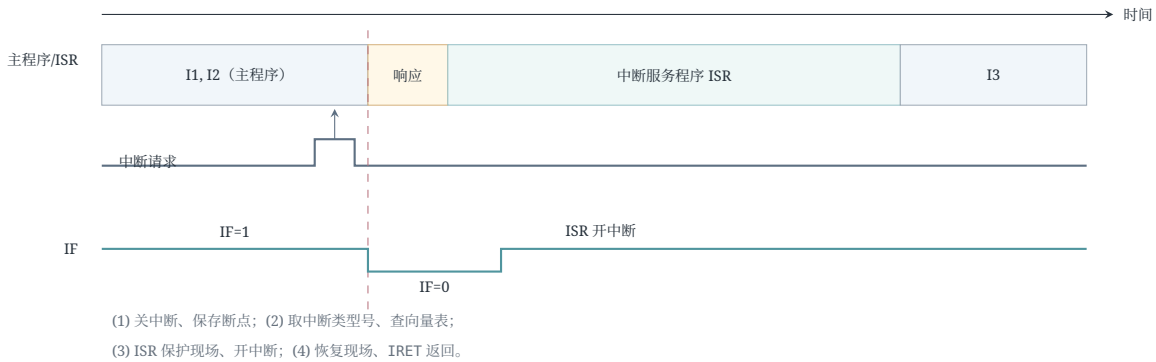
中断响应：CPU 在什么时候、做什么

CPU 不是一看到请求就立刻停下来。对可屏蔽中断，通常要同时满足几个条件：中断源有请求且未被屏蔽；CPU 处于开中断状态（IF=1）；当前指令已经执行完；没有复位等更紧急的事件。为什么不在指令中间响应？因为响应中断要保存一个能正确恢复的断点，指令中间的状态（比如写回还没完成、流水线里还有未提交的指令）很难用一个 PC 值表示。异常则不同，它由当前指令的执行结果触发，CPU 会在指令执行过程中或结束时按异常类型处理。

响应时，CPU 硬件会自动做一串操作，这段过程常称为**中断响应周期**；这串操作不是程序里的某条指令，所以又被称为**中断隐指令**：

1. **关中断**。对可屏蔽中断，硬件清 IF，防止响应过程本身又被另一个中断打断。
2. **保存断点**。把当前程序的 PC（指向下一条要执行的指令）压入堆栈；很多机器连同 PSW/FLAGS 一起保存。x86 在响应中断时依次压入 FLAGS、CS、IP，返回时再由 IRET 弹出。
3. **识别中断源**。有多个设备同时请求时，中断控制器按优先级选出最高的一个，在中断响应周期把它的中断类型号送给 CPU。
4. **查中断向量表**。以类型号为索引，从向量表中取出服务程序入口地址，装入 PC（以及代码段寄存器），CPU 随即转到 ISR。

进入 ISR 之后，还有两件事由软件完成：开头用 PUSH 保存本程序要用的通用寄存器，这叫**保护现场**；结尾用 POP 恢复，再执行 IRET 返回。保护现场是 ISR 的责任，不是硬件的责任。硬件保存的是“断点”，软件保存的是“现场”，两者层次不同，考试里经常被混在一起。

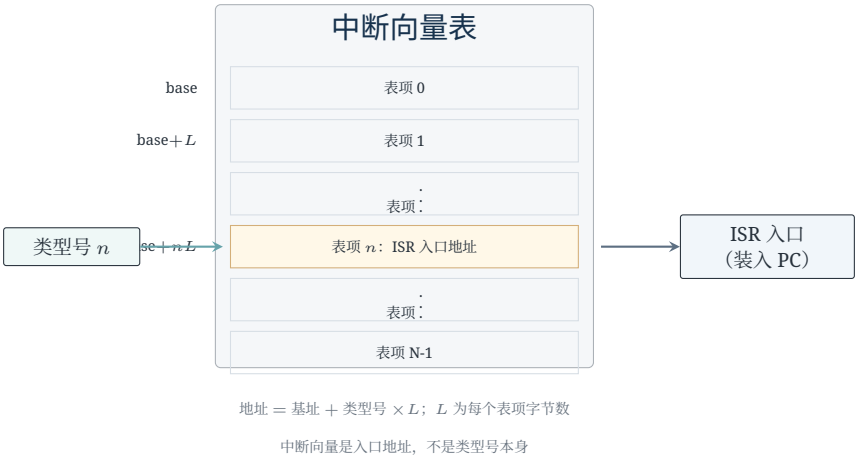


图里最关键的是那条竖直虚线：请求可以在 I2 执行到一半时到达，但 CPU 要等 I2 结束才响应，保存的返回地址指向 I3，而不是 I2 的中间。响应阶段 IF 被清 0；ISR 在保存好通用寄存器

后重新开中断，才能允许更高优先级的中断嵌套。IRET 返回时恢复的是中断前的 IF，不是无条件把它置 1；如果中断前 IF=0，返回后仍然是 0。

中断向量表：类型号到入口地址

中断类型号只是一个编号，真正要跳到哪里由中断向量表给出。表可以放在内存的固定区域，每个表项存放对应 ISR 的入口地址（或入口地址信息）。x86 的中断类型号是 8 位，实模式表最多 256 项；表固定在低地址 0～0x3FF，每个表项 4 字节，前 2 字节是 IP、后 2 字节是 CS。32 位保护模式下用的是中断描述符表 IDT，每个表项 8 字节，里面是门描述符，包含段选择子和偏移，不是直接的一个线性地址；64 位模式下每个表项为 16 字节。

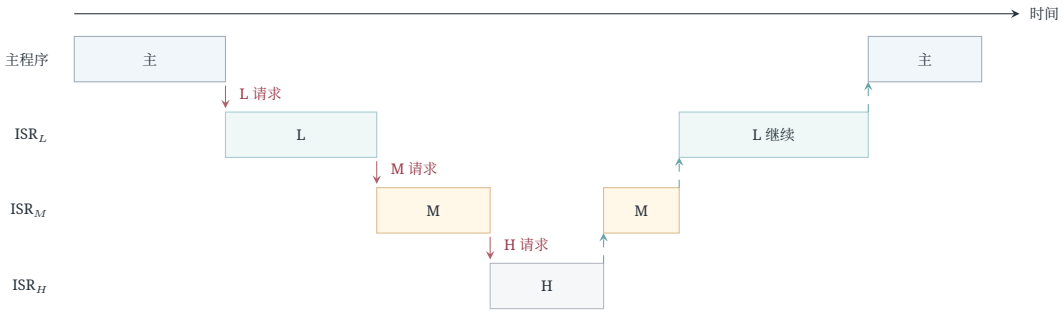


图里类型号 n 只是查表的索引，表项里放的是入口地址或入口地址信息。若表项长度为 L，第 n 项的首地址就是 基址 $+\,nL$；实模式 $L=4$，32 位保护模式 IDT 的 $L=8$，64 位模式为 16。408 计算题常把 L 直接写出来，按这个式子算即可。表本身由操作系统在启动时初始化，硬件只负责按类型号去查。

多重中断：谁能打断谁

如果 ISR 一直关着中断，高优先级的设备也只能等当前 ISR 做完。要支持多重中断（中断嵌套），需要两个条件：一是 ISR 在保护现场之后重新开中断；二是中断系统有优先级裁决，只有优先级更高的请求才能打断当前 ISR。优先级可以用硬件排队电路实现，也可以用中断控制器里的屏蔽寄存器设置；同级或更低级的请求只能挂起，等当前 ISR 返回后再处理。

为什么一定要先保护现场再开中断？假设 ISR 还没有保存寄存器就开了中断，嵌套进来的 ISR 可能改写同一组寄存器；等嵌套返回时，外层 ISR 原来的值已经丢了，恢复现场就会把错误的值写回主程序。堆栈在这里起关键作用：每进入一层中断，断点和现场依次压栈；返回时按后进先出弹出，所以嵌套层次越深，栈越深。



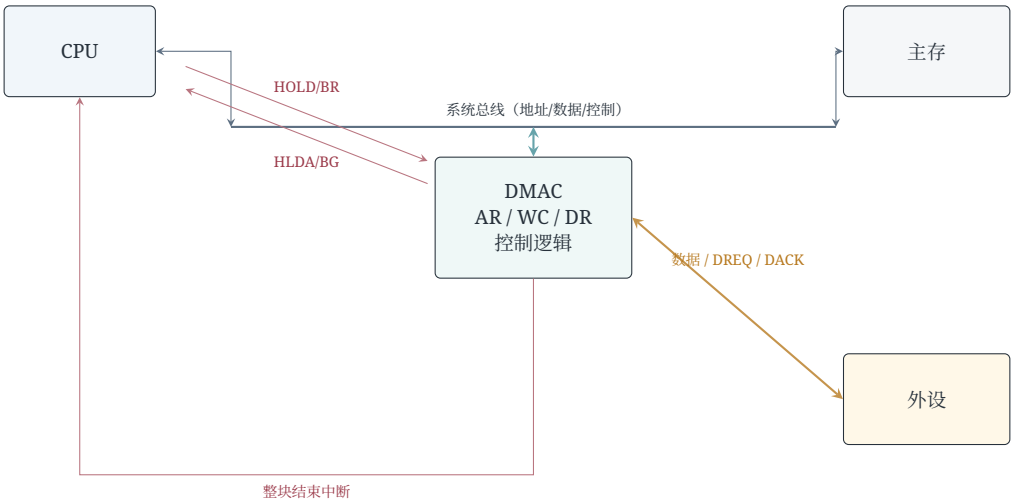
栈：L 断点 → M 断点 → H 断点；返回时逆序弹出

图里优先级是 $H > M > L$。L 先被响应；M 比 L 高，嵌套进 L；H 又比 M 高，嵌套进 M。H 执行期间如果有更低优先级的请求，它只能挂起，等 H 返回、M 也返回之后才有机会。虚线是返回路径：H 返回 M，M 返回 L，L 最后返回主程序。实际机器还会用屏蔽字临时改变某些源的优先级，但“高优先级可嵌套、低优先级挂起、同级不嵌套”这条规则不变。

13.4 DMA：把搬数据交给硬件

中断方式虽然让 CPU 不用守着设备，但每传送一个字/字节，CPU 仍要进一次 ISR，执行保护现场、取数、存数、恢复现场这一串指令。磁盘不是按字节慢慢来的：一块 4 KB 的数据可能在很短的时间内连续到达。如果按 4 B 一次中断，4 KB 就是 1024 次中断；每次哪怕只花 100 个时钟周期，也要十万多个周期。DMA 的思路是把“逐字搬运”从 CPU 的指令流里拿掉，交给专门的 DMA 控制器（DMAC）。

DMAC 里通常有三组关键寄存器：地址寄存器 AR 存放当前要访问的主存地址；字计数器 WC 存放还剩多少个字/字节要传；数据缓冲寄存器 DR 暂存一次传送的数据。此外还有控制/状态逻辑，用来接收 CPU 的命令、产生总线请求、在结束时发中断。设备侧用 DREQ（DMA 请求）和 DACK（DMA 应答）握手；系统侧用 HOLD（或 BR）向 CPU 申请总线，CPU 用 HLDA（或 BG）表示可以让出。

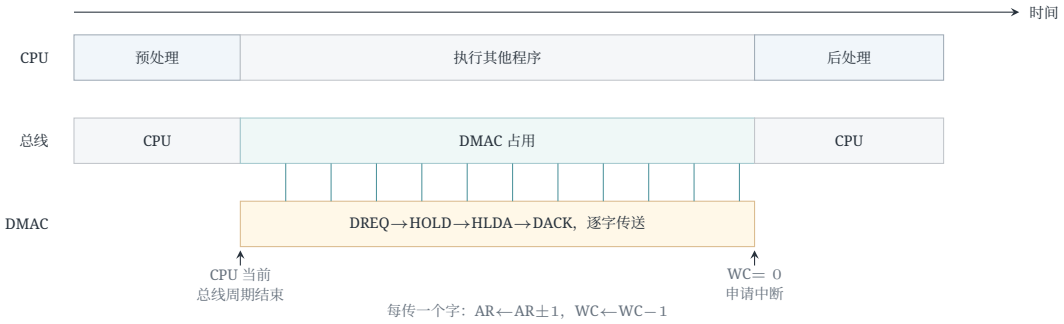


数据由 DMAC 在设备与主存之间搬运，不经过 CPU 通用寄存器；DMA 期间 CPU 让出总线

图里 CPU 和 DMAC 之间只有总线请求/允许和结束中断几根控制线。真正搬数据时，DMAC 把 AR 送到地址总线，发出 `MEMR/MEMW` 和 `IOR/IOW`，数据从设备经 DMAC 的数据缓冲（或直接经总线）写入主存。说“DMA 不经过 CPU”，准确含义是不靠 CPU 执行逐字搬运指令、不经过

CPU 通用寄存器；数据经过 DMAC 的缓冲寄存器是允许的，不要把它理解成“任何缓冲都不能经过”。

DMA 传送分三段。第一段是**预处理**，由 CPU 完成：把主存起始地址写入 AR，把传送字数写入 WC，设置方向，然后启动设备；之后 CPU 就可以去执行别的程序。第二段是**数据传送**，由 DMAC 完成：设备准备好一个数据，发 DREQ；DMAC 向 CPU 发 HOLD；CPU 在当前总线周期结束后让出总线，回 HLDA；DMAC 回 DACK，把 AR 送上地址总线，完成一次读/写；然后 AR 加 1（或减 1），WC 减 1。若 WC 不为 0，继续下一个数据；若 WC 为 0，进入第三段。第三段是**后处理**，DMAC 向 CPU 发中断，CPU 执行 ISR 检查状态、处理错误、决定是否启动下一块。



图里上排是 CPU 在做什么，中排是谁占着总线，下排是 DMAC 的动作。预处理和后处理由 CPU 执行指令完成；中间一长段数据传送，CPU 不执行搬运指令，但总线被 DMAC 按字逐个占用。注意边界：DMA 的总线请求（HOLD/BR）是在**总线周期**边界被采样并让出总线的，不必等一条指令执行完；可屏蔽中断才是在**指令周期**边界响应。这个粒度差别是 408 选择题的常见考点。

DMAC 与 CPU 争用总线有三种典型策略。**停止 CPU 访存**（成组传送）是 DMAC 一旦获得总线就一直占着，直到整块数据传完才释放；传送期间 CPU 不能访存，若当前指令和数据都在 Cache 中命中、且不需要写主存，还可以继续执行，一旦需要访存就被挡住。**周期挪用**（也叫周期窃取）是每传一个字（或几个字）申请一次总线，传完立即释放；CPU 的访存可能被推迟一个或几个周期，但在两次挪用之间仍可访存。**交替访存**把总线时间切成固定时隙，CPU 和 DMAC 轮流使用，不需要每次申请/释放，但两边都只能拿到一部分带宽。

周期挪用容易被误解成“只用 CPU 空闲的总线周期”。它不是：即使 CPU 正在使用总线，DMAC 也可以在当前总线周期结束后把下一个周期“借”走，CPU 的访存因此被推迟。它之所以叫“挪用”，正是因为它会打断 CPU 的正常访存节奏，只是每次只借一点点。

DMA 与程序中断方式的对比

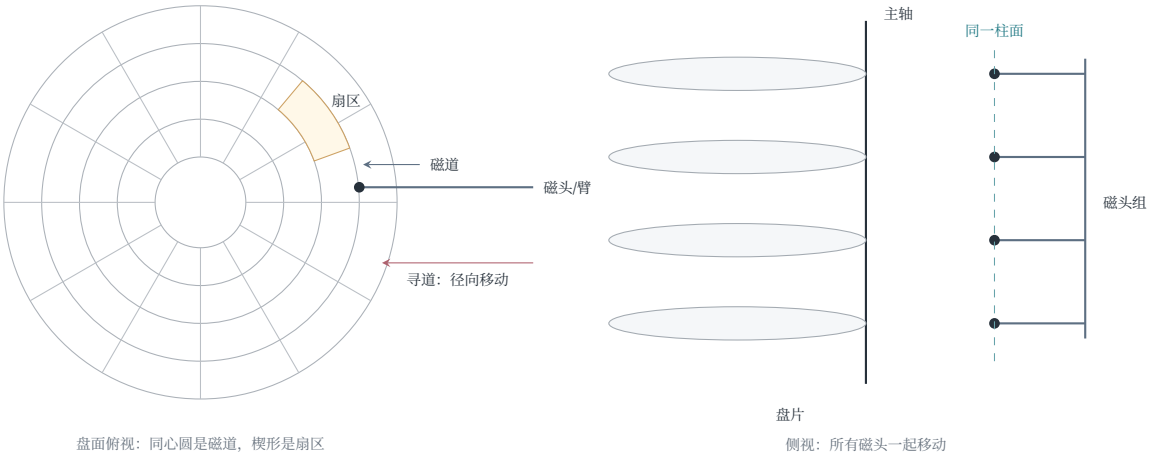
比较项	程序中断	DMA
数据传送者	CPU 执行 ISR 逐字搬运	DMAC 硬件搬运
传送单位	字/字节（有缓冲时可成块）	数据块
数据路径	设备 ↔CPU↔ 主存	设备 ↔ 主存（可经 DMAC 缓冲）
CPU 参与	每次事件都要进入 ISR	只在预处理和后处理参与
中断次数	通常每个字/字节一次	整块结束一次
响应边界	指令周期结束	总线周期结束
适用设备	键盘、打印机等慢速设备	磁盘、网卡等高速块设备

比 DMA 更进一步的方案是**通道**（channel），也叫 I/O 处理器（IOP）。通道有自己的指令系统，能执行存放在主存里的通道程序，一组通道命令字（CCW）可以描述“从哪个设备读、读到哪、传多少、下一步做什么”。CPU 只需发一条启动 I/O 的命令，通道就能独立完成一批数据块的传送，结束时再中断 CPU。DMA 只是一个硬件控制器，通道则是一个能执行程序的处理器；选择

通道一次服务一个高速设备，数组多路通道可以交错服务多个高速设备，字节多路通道则轮流为多个低速设备服务。

13.5　磁盘：机械运动的代价

磁盘和内存最大的不同是它有机械运动。若干盘片叠在同一根主轴上，每个盘面有一个磁头，所有磁头装在同一组臂上一起径向移动。盘面被分成许多同心圆，叫**磁道**；每条磁道又分成若干**扇区**；不同盘面上半径相同的磁道组成一个**柱面**。读写的最小单位是扇区，常见 512 B 或 4 KB。



图左边是盘面俯视：磁头沿半径方向移动，改变它所在的磁道，这就是寻道；盘片旋转，把目标扇区带到磁头下面。图右边是侧视：几片盘片共用主轴，所有磁头固定在同一个臂组上，所以它们同时位于同一柱面。切换磁头只需要电子选择，不需要机械移动；因此磁盘地址通常按“柱面号—磁头号—扇区号”排列，先把同一柱面的所有磁道用完，再移动到下一个柱面，减少寻道次数。

磁盘容量可以按几何参数算：

$$\text{容量} = \text{盘面数} \times \text{柱面数} \times \text{每道扇区数} \times \text{每扇区字节数}.$$

柱面数就是每个盘面的磁道数。408 题通常假设每条磁道的扇区数相同；真实磁盘常用区域记录（ZBR），外圈磁道扇区更多，但考试一般不涉及。

读一个扇区要等三件事：把磁头移到目标磁道，叫**寻道时间**；等目标扇区转到磁头下面，叫**旋转延迟**；把扇区内容读出来，叫**传输时间**。总存取时间是三者之和：

要点 13.1: 磁盘存取时间与传输率

$$T_{\text{access}} = T_{\text{seek}} + T_{\text{rot}}/2 + T_{\text{transfer}}, \qquad T_{\text{rot}} = \frac{60}{r}, \qquad T_{\text{transfer}} = \frac{k}{S} T_{\text{rot}}.$$

其中 r 是转速（r/min），S 是每道扇区数，k 是本次连续读取的扇区数，T_{rot} 是转一圈的时间。持续传输率为

$$R = S \cdot B \cdot \frac{r}{60},$$

B 是每扇区字节数。

平均旋转延迟取半圈，前提是目标扇区在圆周上的位置随机，平均要等半圈；最大旋转延迟是一整圈。传输时间按“本次读的扇区数占整道扇区数的比例”乘一圈时间，不是按数据总线宽度算。若连续读同一磁道的 k 个扇区，寻道和旋转延迟只付一次；若跨磁道，还要加上新的寻道

和旋转。平均寻道时间与请求分布、初始磁头位置和调度算法都有关，题目通常直接给出；不要把它当成最大寻道时间的一半。

举个数值例子。某磁盘转速 $5400\ \mathrm{r/min}$，平均寻道时间 $10\ \mathrm{ms}$，每道 200 个扇区，每扇区 512 B。转一圈的时间是

$$T_{\mathrm{rot}} = \frac{60}{5400}\ \mathrm{s} = 11.11\ \mathrm{ms},$$

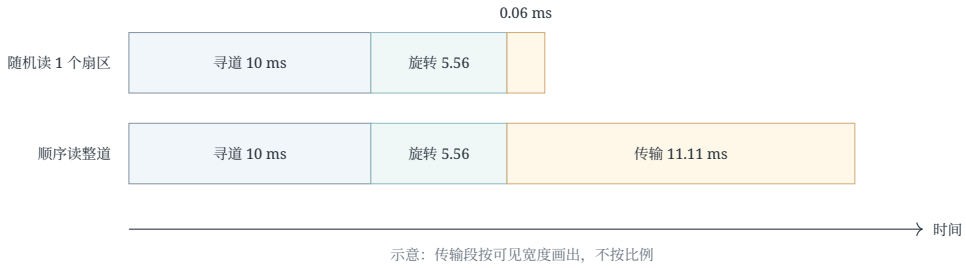
平均旋转延迟是 $5.56\ \mathrm{ms}$。读一个扇区的传输时间是

$$T_{\mathrm{transfer}} = \frac{1}{200} \times 11.11\ \mathrm{ms} = 0.0556\ \mathrm{ms},$$

所以平均存取时间约为 $10 + 5.56 + 0.056 = 15.61\ \mathrm{ms}$。持续传输率为

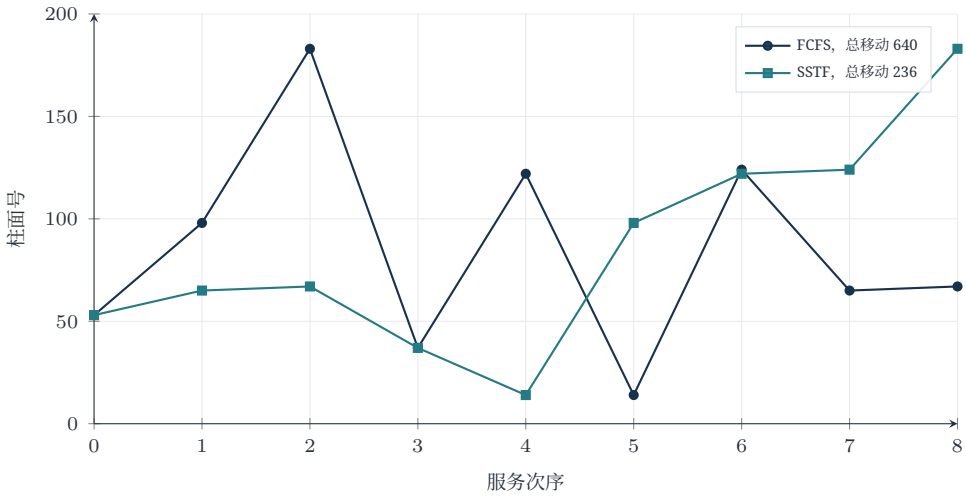
$$R = 200 \times 512 \times \frac{5400}{60} = 9.216 \times 10^{6}\ \mathrm{B/s} = 9.216\ \mathrm{MB/s}.$$

如果一次读完整条磁道（200 个扇区），传输时间变成 11.11 ms，寻道和旋转仍只付一次；单扇区读时传输只占很小一部分，这就是大块 I/O 能摊薄机械开销的原因。



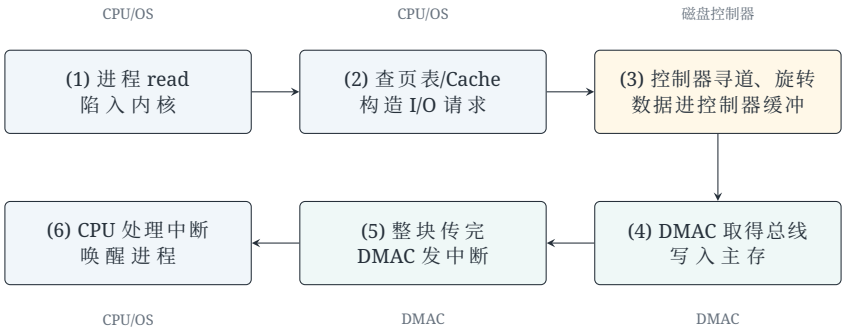
图里上条是随机读一个扇区，寻道和旋转占掉了几乎全部时间；下条是顺序读整道，传输段变长，但寻道和旋转没有重复支付。实际磁盘的传输时间比例取决于块大小和磁道容量，图中把极短的传输段画宽了，只为了让你看清三段结构。

当多个 I/O 请求排队时，先服务谁会影响总寻道距离。**FCFS** 按到达顺序服务，公平但磁头可能来回跑；**SSTF** 每次选离当前磁头最近的请求，寻道距离短，但远处的请求可能长期得不到服务；**SCAN** 像电梯，先沿一个方向服务沿途请求，到端点再反向；**C-SCAN** 只在一个方向服务，回程直接回到起点，等待时间更均匀；**LOOK** 是 SCAN 的变体，到了最远的请求就反向，不必走到盘边。



图里纵轴是柱面号，横轴是服务次序，相邻两点的垂直距离就是这次要移动的磁道数。FCFS 的折线来回跳，总移动 640 个柱面；SSTF 先做离当前磁头最近的请求，折线短得多，总移动 236 个柱面。SSTF 的代价是可能饿死远端请求，所以实际系统常用 SCAN/C-SCAN 一类有方向约束的算法。图只反映寻道，不含旋转延迟；真正的调度还要考虑请求到达时刻和目标扇区的旋转位置。

把前面的部件串起来，一次磁盘读的完整路径大致如下。



图里 CPU 只在第 (1)、(2)、(6) 步执行指令；第 (3) 步是磁盘控制器和机械运动；第 (4) 步是 DMAC 搬数据；第 (5) 步是一次中断。慢在机械，快在电子，所以磁盘 I/O 不适合让 CPU 逐字查询或逐字中断，而要用 DMA 把数据成块搬走。若数据已经在页缓存中，第 (3)～(5) 步不会发生，这里讨论的是缓存未命中的情况。

固态硬盘（SSD）没有磁头和盘片，随机读不需要寻道和旋转，延迟主要来自闪存读、地址映射（FTL）、垃圾回收和写放大。它以页为单位读写、以块为单位擦除，随机写比随机读更贵。408 里要区分：磁盘是直接存取，访问时间与位置有关；SSD 是随机存取，但仍有读写粒度和擦除块的限制。

13.6 几种 I/O 控制方式放在一起看

四种 I/O 控制方式的对比

	程序查询	程序中断	DMA	通道
数据传送控制	CPU 执行查询/传送指令	CPU 执行 ISR	DMAC 硬件	通道程序（IOP）
传送单位	字/字节	字/字节（或缓冲块）	数据块	一组数据块
CPU 干预	全程等待	每次事件	预处理、后处理	启动、结束
数据路径	设备 ↔CPU↔ 主存	同左	设备 ↔ 主存	设备 ↔ 主存
并行程度	无	部分	高	高
典型场景	单片机按键、LED	键盘、打印机	磁盘、网卡	大型机多设备

从查询到中断，变化的是“谁主动”：CPU 不用守着设备，设备可以发请求。从中断到 DMA，变化的是“谁搬数据”：CPU 不再逐字执行传送指令，DMAC 直接在主存和设备之间搬。从 DMA 到通道，变化的是“谁执行 I/O 程序”：DMAC 只会按固定参数搬一块，通道能执行通道程序、串联多个操作、管理多台设备。四者的硬件复杂度和 CPU 开销依次变化，适用场景也不同。

注意

本章最容易混淆的几处：

- 中断向量是服务程序的入口地址，不是中断类型号；类型号只是查表的索引。
- 保护断点（PC、PSW）由硬件在响应时完成；保护现场（通用寄存器）由 ISR 用 PUSH 完成。
- DMA 数据不经过 CPU 通用寄存器，但可以经过 DMAC 的数据缓冲；不要理解成“绝对不经过任何缓冲”。
- DMA 请求在总线周期边界被采样，可屏蔽中断在指令边界被响应；两者粒度不同。
- 周期挪用是“借一个或几个总线周期”，不是“只用总线空闲周期”；CPU 的访存可能被推迟。
- 统一编址不是“端口没有地址”，而是端口和主存共用地址空间；独立编址下地址数值相同也不会冲突。
- 平均旋转延迟是半圈；一个扇区的传输时间是整圈时间除以每道扇区数，不是按数据总线宽度算。
- IRET 恢复中断前的 IF，不是无条件开中断；中断前 IF=0 时，返回后仍为 0。

14 习题

14.1 基础过关题

1. （单项选择题）关于 I/O 接口与 I/O 端口，下列说法中错误的是

(A) 一个 I/O 接口中通常包含数据、状态、控制等多个端口

(B) I/O 端口是接口中能被 CPU 用指令直接访问的寄存器

(C) 统一编址时，I/O 端口与主存单元共用地址空间，可用访存指令访问

(D) 独立编址时，I/O 端口地址与主存地址数值相同时会产生冲突

2. （填空题）CPU 响应可屏蔽中断时，由硬件自动完成的主要操作包括：关中断、保存_____、识别中断源并取得_____，再据此查中断向量表转入中断服务程序。中断服务程序开头用压栈指令保存通用寄存器，这一操作称为_____。

3. （填空题）某机的中断向量表基址为 `0x00002000`，每个表项占 8 字节，表项中存放对应中断服务程序的入口地址。中断类型号 `0x40`～`0x4F` 共 16 个表项，则类型号 `0x40` 的表项首地址为_____，类型号 `0x4F` 的表项首地址为_____，这 16 个表项共占_____ 字节。

4. （单项选择题）某系统采用程序查询方式从外设输入数据。下列叙述中正确的是

(A) 外设就绪后主动向 CPU 发中断请求，CPU 再读数据

(B) CPU 在等待设备就绪期间可转去执行其他用户程序

(C) CPU 反复读状态端口，检测到就绪标志后再读数据端口

(D) 数据传送由 DMA 控制器控制，CPU 只负责启动

5. （单项选择题）下列关于 DMA 传送过程的叙述中，正确的是

(A) 预处理阶段由 DMAC 设置 CPU 中的地址寄存器和字计数器

(B) 数据传送阶段每传一个数据，CPU 都要执行一次中断服务程序

(C) 后处理阶段由 CPU 执行程序检查传送结果并决定是否继续

(D) 传送的数据必须经过 CPU 的通用寄存器

6. (填空题) 某磁盘转速为 6000 r/min，平均寻道时间为 6 ms，每磁道有 300 个扇区，每扇区 512 B。忽略控制器和排队开销，读一个扇区的平均存取时间为_____，磁盘持续传输率为_____MB/s（取 1 MB $= 10^6$ B，保留两位小数）。

14.2 真题风格与综合训练

1. (单项选择题) 某中断系统支持多重中断，中断源优先级为 $A > B > C$，各中断服务程序都在保护现场之后开中断。主程序运行期间 C 先请求并被响应；C 开中断后 A 请求到达并被响应；A 执行期间 B 请求到达。下列执行顺序中正确的是

(A) 主程序 $\rightarrow C \rightarrow A \rightarrow B \rightarrow C \rightarrow$ 主程序

(B) 主程序 $\rightarrow C \rightarrow A \rightarrow C \rightarrow B \rightarrow$ 主程序

(C) 主程序 $\rightarrow A \rightarrow B \rightarrow C \rightarrow$ 主程序

(D) 主程序 $\rightarrow C \rightarrow B \rightarrow A \rightarrow C \rightarrow$ 主程序

2. (单项选择题) 某系统采用周期挪用（周期窃取）DMA 方式从磁盘读入数据。下列叙述中正确的是

(A) DMAC 取得总线后一直占用到整个数据块传送结束

(B) DMA 请求与中断请求都在当前指令执行结束后才被采样

(C) DMAC 每传送一个数据占用一个（或几个）总线周期，随后归还总线

(D) DMA 传送结束后无需通知 CPU，CPU 也不需要做任何收尾工作

3. (综合应用题) 某计算机主频为 1 GHz，某外设以 4 MB/s 的速率持续输入数据（1 MB $= 10^6$ B）。

(1) 采用中断方式，每次中断传送 4 B，每次中断从响应到返回共执行 200 个时钟周期（含保护与恢复现场）。求每秒中断次数及 CPU 用于该外设的时间占比。

(2) 采用 DMA 方式，每块 8 KB，预处理需 500 个时钟周期，整块传完后的中断后处理需 300 个时钟周期。周期挪用 DMA 每传送 4 B 占用 1 个总线周期，总线周期为 $0.05\ \mu$s。求每秒 DMA 块数、CPU 用于预处理和后处理的时间占比，以及 DMA 占用总线的比例。

(3) 从 CPU 开销角度比较两种方式。

4. （综合应用题）某磁盘转速为 $7200\ \mathrm{r/min}$，平均寻道时间为 $8\ \mathrm{ms}$，每磁道 400 个扇区，每扇区 512 B。磁盘通过周期挪用 DMA 向主存传送数据，DMAC 每传送 4 B 占用一个总线周期，总线周期为 $0.08\ \mu$s，系统主频为 $1\ \mathrm{GHz}$。

(1) 求读一个扇区的平均存取时间（寻道时间 + 平均旋转延迟 + 传输时间），以及磁盘持续传输率。

(2) 连续传送时，求 DMA 占用总线的比例。

(3) 若每传送 4 KB 产生一次 DMA 结束中断，中断处理需 200 个时钟周期，求每秒中断次数及 CPU 开销占比。

(4) 若不用 DMA，而用中断方式每次传送 4 B，每次中断处理需 100 个时钟周期，求 CPU 开销占比，并说明磁盘为何采用 DMA。

PART III　第三部分

操作系统

1 操作系统概述

终端里敲下 `./a.out`，程序开始运行。它用 `fopen` 打开 `input.txt`，用 `malloc` 申请一块缓冲区，用 `printf` 把结果写到屏幕；按下 Ctrl-C，它又会立刻停下。整个过程里，程序没有一条指令直接操作磁盘、内存条或终端控制器。它看到的 `input.txt` 只是一个名字，`malloc` 返回的只是一段地址，`printf` 只是把字节交给了一个叫 stdout 的东西。把这些名字、地址和字节翻译成硬件动作的，是操作系统。

定义 1.1: 操作系统

操作系统（Operating System，OS）是控制和管理整个计算机系统的硬件与软件资源，合理组织、调度计算机的工作与资源分配，并为用户和其他软件提供方便接口与环境的程序集合。它是最基本的系统软件。

这个定义里有三个角色，考试和实际系统都绕不开。

第一，操作系统是资源管理者。处理器、主存、磁盘、键盘、网卡都归它分配和回收；多个程序同时要资源时，由它决定谁先拿、拿多少、拿多久。

第二，操作系统是用户与硬件之间的接口。用户通过命令或图形界面使用机器，程序通过系统调用使用机器，两者都不直接碰硬件。

第三，操作系统把裸机改造成一台更好用的“扩充机器”。进程、虚拟地址空间、文件、设备这些概念都不是硬件自带的，是操作系统在硬件之上造出来的抽象。

需要区分操作系统和内核。内核是操作系统最核心的部分，常驻内存，运行在内核态，负责中断处理、进程调度、内存管理和设备驱动等关键工作。shell、图形界面、编译器和各种系统工具通常运行在用户态，它们可能随系统一起安装，但不属于内核。408 的选择题说“操作系统”时，多数情况下指的是内核提供的功能；题目单独提到 shell 时，要看清问的是内核还是整个系统。

操作系统给用户和程序提供的接口分两类。命令接口面向人：联机命令接口像 shell，输入一条命令执行一条；脱机命令接口面向批处理，用户把一批作业写成脚本交给系统。程序接口面向程序，就是系统调用。图形用户界面可以看作命令接口的一种图形化实现，408 一般不在这里出难题。

1.1 一次运行里，操作系统做了什么

把刚才那次运行拆开，可以看到四类管理功能各自在什么时候出场。

一次文件读取涉及的管理功能

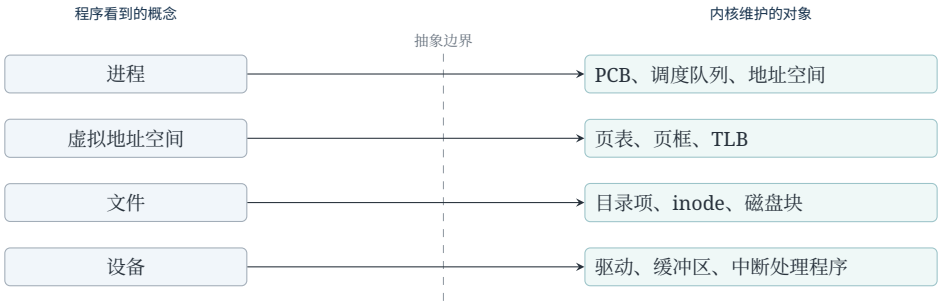
程序的动作	管理功能	操作系统在背后做什么
`fork`、`exec`	处理机管理	建立进程控制块，分配 PID，装入程序，把进程调入 CPU 运行
访问一个尚未装入的页	存储器管理	触发缺页异常，分配页框，从磁盘读入页面，更新页表
`read` 读文件	文件管理	按路径查目录，找到 inode，定位数据块，把数据复制到用户缓冲区
`printf` 输出到终端	设备管理	调用终端驱动，写入设备寄存器，等待中断确认完成

处理机管理也叫进程管理，管的是进程的创建、撤销、状态转换、调度、同步、通信和死锁处理。它的核心问题是：CPU 只有一个或几个，而可运行的进程有很多，下一时刻让谁运行。

存储器管理管的是内存的分配与回收、地址变换、内存保护和内存扩充。程序里用的是逻辑地址，硬件和操作系统要把它变成物理地址；进程之间不能互相踩内存；内存不够时，还要用虚拟存储把磁盘当成内存的延伸。

文件管理管的是文件的存储空间、目录结构、读写操作、共享和保护。用户看到的是“按名字打开一个文件”，操作系统看到的是目录项、inode、磁盘块和缓存页。

设备管理管的是设备的分配、缓冲、驱动和虚拟设备。设备的速度比 CPU 慢几个数量级，操作系统用缓冲和中断把这种速度差藏起来；SPOOLing 技术还能把一台独占的打印机虚拟成多台逻辑打印机。



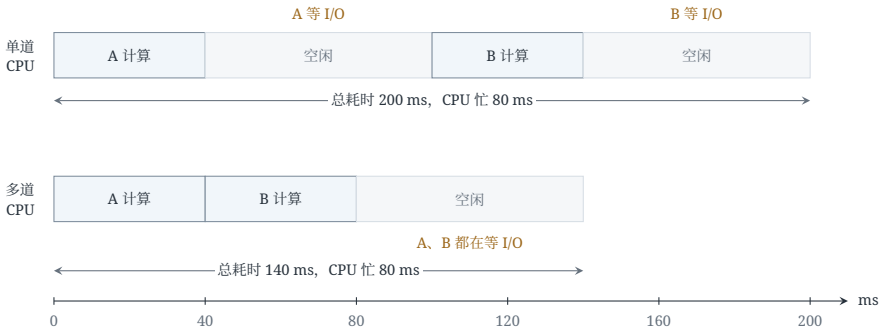
图里左边是程序看到的四个概念，右边是内核为它们维护的数据结构。进程对应 PCB 和地址空间，虚拟地址对应页表里的映射，文件对应目录项和 inode 指向的块，设备对应驱动、缓冲区和中断处理程序。抽象把右边这些细节藏起来，系统调用和中断负责在两边搬数据。

这四块在教材里分开讲，在实现里却互相调用。读文件要先经过文件系统，再经过块设备层和驱动，最后靠中断通知完成；缺页异常要文件系统把页面读进来，又要存储器管理分配页框；驱动收到中断后，可能唤醒一个阻塞的进程，这又回到进程管理。这里不展开这些调用关系，是因为它们在第 3 章到第 5 章才会展开；先记住四块的分工和它们共同的目标：把硬件管好，把接口做简单。

1.2　多道程序：让 CPU 不等 I/O

早期的单道批处理系统一次只让一道程序进内存。程序发起 I/O 后，CPU 就闲着等设备，等完了再继续算。磁盘、打印机、键盘的速度都远低于 CPU，这种等待浪费了大部分算力。

多道程序设计把多道程序同时放进内存。一道程序等待 I/O 时，操作系统把 CPU 切给另一道程序；等 I/O 完成的中断到来，再把原来的程序唤醒。这样，CPU 和设备可以同时忙。



图里假设两道程序各需要 40 ms 的 CPU 计算和 60 ms 的 I/O，I/O 可以重叠，忽略调度开销。单道执行时，A 计算 40 ms，等 I/O 60 ms，B 再计算 40 ms、等 I/O 60 ms，总共 200 ms，CPU

只忙了 80 ms，利用率 40%。多道执行时，A 在 40 ms 时开始等 I/O，B 立刻接上 CPU；A 的 I/O 在 100 ms 完成，B 的 I/O 在 140 ms 完成。两道程序总共用 140 ms，CPU 仍然忙 80 ms，利用率升到约 57%。

设每道程序的计算时间都是 c、I/O 时间都是 t，忽略开销，n 道程序错开启动后，最后一道的 I/O 在 $nc+t$ 时结束，CPU 忙 nc，利用率约为

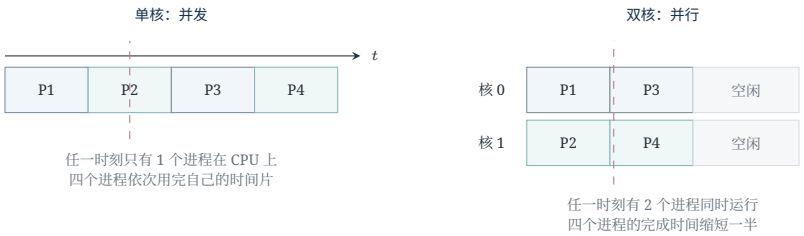
$$U = \frac{nc}{nc+t}.$$

n 越大，U 越接近 1。这个式子只说明趋势：实际的 I/O 设备有带宽上限，内存容量也有限，道数不可能无限增加；道数太多，调度和切换的开销又会吃掉收益。

多道程序提高的是资源利用率和吞吐量，不缩短每道程序自己的 CPU 时间。图中 A 在单道和多道下都在 100 ms 完成；B 的完成时间从 200 ms 提前到 140 ms。多道还带来了并发，进程之间会争抢资源，于是同步、互斥和死锁问题随之出现，这些是第 2 章的内容。

1.3 并发不是并行

并发（concurrency）指两个或多个事件在同一时间间隔内发生。单核 CPU 在 1 秒内轮流执行四个进程，宏观上四个进程都“在跑”，微观上任意时刻只有一个进程占着 CPU。并行（parallelism）指两个或多个事件在同一时刻发生，需要多个处理器或多个核同时执行。



图左边是一个核跑四个进程。P1 到 P4 各占一个时间片，任意画一条竖线，只有一个进程在运行。图右边是两个核，每个核跑两个进程；同一时刻 P1 和 P2 可以同时执行，之后 P3 和 P4 也可以同时执行。图中的长度只是示意，不代表真实的时间片长度。

> **注意**
>
> 单核 CPU 上通过时间片轮转得到的“同时运行”是并发，不是并行。并行要求同一时刻有多个处理器（核）在执行。408 选择题常把“单核也能实现并行”作为干扰项；反过来，“多核系统只有并行、没有并发”也不对，多核上多个进程仍可能分时复用同一个核。

1.4 共享、虚拟与异步

共享指系统资源可以被多个并发进程共同使用。共享有两种方式。互斥共享的资源一次只允许一个进程访问，比如打印机、共享变量、临界区；同时访问的资源可以在一段时间内被多个进程交替使用，比如磁盘上的文件、只读代码段。磁盘虽然是一个物理设备，但多个进程可以交替读写不同的文件，用户感觉不到排队，所以归入同时访问。

并发和共享互为存在条件。没有并发，进程一个一个串行执行，就不存在争抢资源的问题，共享机制没有意义；没有共享，进程无法交换数据，并发执行也失去了大部分价值。这两个特征是操作系统最基本的特征。

虚拟把物理实体映射成多个逻辑对应物。虚拟处理器让每个进程以为自己独占 CPU，靠的是分时复用；虚拟存储器让每个进程以为自己拥有一段连续的大地址空间，靠的是内存和外存的组合；SPOOLing 把一台独占打印机虚拟成多台逻辑打印机，靠的也是分时复用。408 把虚拟技术分为时分复用和空分复用：虚拟处理器、虚拟设备属于时分复用，虚拟存储器属于空分复用。

异步指进程以不可预知的速度向前推进。进程什么时候被调度、什么时候因 I/O 阻塞、什么时候被中断打断，都不确定，所以每个进程都是“走走停停”。操作系统要保证：只要同步机制写得对，无论执行顺序如何，结果都一致。异步来自并发、共享和中断，没有这三样，程序会一条接一条地确定性执行。

1.5 从批处理到实时系统

操作系统的发展阶段按“怎么让多道程序共享机器”来划分，比按年代背更有用。

操作系统发展阶段与几类系统的对比

类型	关键机制	优点	局限
手工操作	用户独占机器,插线或开关设定	无需操作系统	人机矛盾突出，资源浪费
单道批处理	监督程序自动衔接作业,内存中只放一道	减少人工干预	CPU 等 I/O，内存利用率低
多道批处理	多道程序驻留内存，等待 I/O 时切换	资源利用率高，吞吐量大	无交互，作业周转时间长
分时系统	时间片轮转,多个终端共享一台主机	交互性好，响应及时	不适合硬实时，切换有开销
实时系统	按优先级调度,强调截止时间	及时、可靠	资源利用率让位于确定性

批处理系统把作业成批送入，由监督程序自动调度，减少人工干预。单道批处理一次只放一道程序，CPU 仍然要等 I/O；多道批处理把多道程序同时放进内存，才真正把 CPU 和设备的重叠利用起来。批处理的缺点是用户提交作业后只能等结果，不能中途交互，作业周转时间也长。

分时系统把 CPU 时间切成很短的时间片，轮流分给多个终端用户。用户输入命令后，系统在一个时间片内响应，感觉像独占机器。分时系统有四个特点：多路性，一台主机连接多个终端；独立性，用户之间互不干扰；及时性，响应时间在可接受范围内；交互性，用户可以随时输入命令。若用户数为 n、时间片为 q，忽略切换开销，一个用户最坏要等其余 $n-1$ 个用户各运行一个时间片，响应时间约为 nq。用户越多或时间片越长，响应越慢；时间片太小，切换开销占比又会上升。

实时系统要求系统在规定的截止时间内完成响应。硬实时必须满足截止时间，错过可能造成严重后果，比如飞行控制、工业机器人；软实时偶尔错过可以接受，比如视频播放。实时系统更关心最坏情况下的响应时间，而不是平均吞吐量。

网络操作系统和分布式操作系统都管理多台机器，区别在用户看到什么。网络操作系统让每台机器运行自己的操作系统，用户显式地登录远程机器、访问共享资源；分布式操作系统把多台机器统一成一个系统，用户不需要知道资源在哪台机器上。个人计算机操作系统强调交互和易用，嵌入式操作系统强调资源受限下的实时性和可靠性。

1.6 内核态与用户态

CPU 至少有两种运行状态：用户态（目态）和内核态（管态，也叫核心态）。状态由程序状态字（PSW）中的模式位决定。用户态下，CPU 只能执行非特权指令，不能直接访问 I/O 端口、不能修

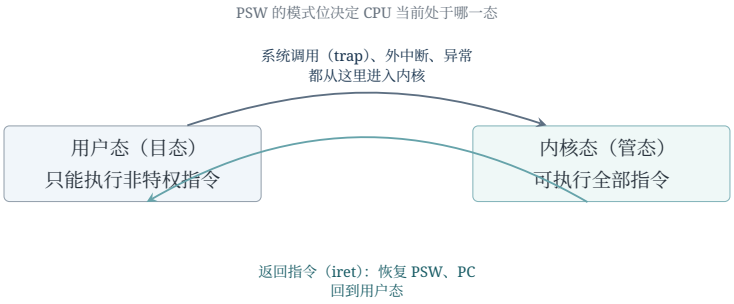
改页表、不能关中断；内核态下，CPU 可以执行全部指令，包括特权指令。

区分两种态的目的是保护。如果用户程序能直接关中断或改页表，一个错误就能让整个系统停摆，其他进程的数据也没有保障。把危险操作收进内核，由内核检查参数和权限，系统才能在多道程序下稳定运行。

特权指令与非特权指令

只能在内核态执行（特权指令）	用户态也能执行（非特权指令）
I/O 指令、关中断、开中断、设置时钟、修改 PSW、设置页表基址、清内存、停机	算术运算、逻辑运算、取数、存数、寄存器传送、跳转、压栈、系统调用（陷入）

表里最容易被忽略的是最后一项：系统调用指令本身在用户态执行，它不是特权指令。用户程序执行一条陷入指令，主动把 CPU 交给内核；CPU 进入内核态后执行的是内核的入口代码，用户程序的后续指令不会因此获得特权。把“系统调用在用户态执行”和“系统调用处理程序在内核态执行”分开看，就不会被“系统调用是特权指令”这种说法骗到。



图里只有两个状态和两条转移边。上面那条边是进入内核的三个常见入口：系统调用是程序主动陷入，外中断是设备或时钟异步打断，异常是当前指令执行出错或需要内核帮忙。下面那条边是内核返回用户态，必须用特权返回指令恢复 PSW 和 PC；用户程序不能自己把模式位改成内核态。

三条入口的性质不同。系统调用是同步的、主动的；外中断是异步的、被动的；异常是同步的、与当前指令有关的。它们共同的效果都是：CPU 保存现场，切换到内核态，跳到内核中对应的处理程序。

内核里放什么？408 常列四项：时钟管理，负责计时和时间片；中断处理，负责中断和异常的入口；原语，指执行期间不可中断的基本操作，比如进程切换、阻塞和唤醒的一部分，常用关中断来实现；系统控制的数据结构及处理，包括进程控制块、页表、文件表、设备表，以及建立在这些结构上的进程管理、存储管理和设备管理。

这里说的“原语不可中断”，指的是原语这段代码执行时不会被中断打断，不是说原语一定很长。关中断只是实现原语的一种手段，原语也可以靠硬件原子指令实现。

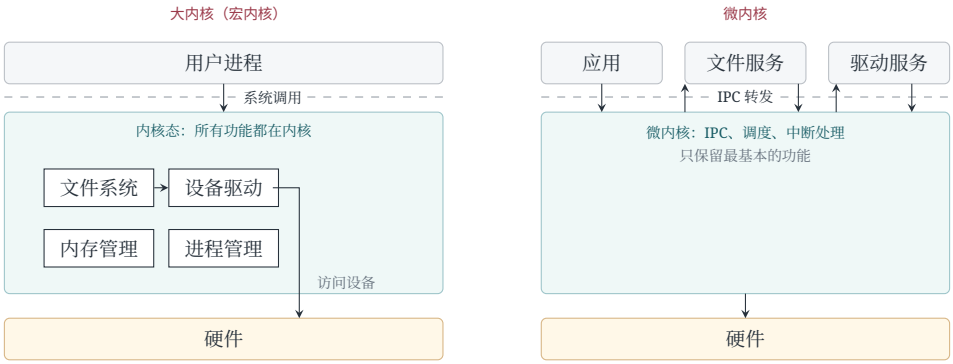
1.7 大内核与微内核

内核该放多少功能，有两种典型答案。

大内核（宏内核）把进程管理、内存管理、文件系统、设备驱动、网络协议都放进内核地址空间，模块之间直接函数调用。Linux 是典型的大内核。它的优点是性能高：一次文件读只在用户态和内核态之间切换两次，内核内部就是普通的函数调用，没有额外的消息传递。缺点是内核庞大、耦合强，一个驱动里的空指针就可能让整台机器崩溃。

微内核只把最基础的功能留在内核，通常包括中断处理、进程调度与通信、基本的存储管理；文件系统、设备驱动、网络协议作为用户态的服务进程运行，彼此通过进程间通信（IPC）交换消

息。微内核的优点是可靠性和可扩展性好：文件服务崩溃了可以重启，不会带崩内核；新增一个服务也不用改内核。缺点是性能开销大：每次服务转发都要经过内核，跨态和消息传递的次数明显增加。



左图里，用户进程一次系统调用进入内核，文件系统直接调用设备驱动，设备驱动直接访问硬件，跨态只发生在用户和内核之间。右图里，应用要读文件，先通过 IPC 把请求交给内核，内核再转给文件服务；文件服务要读磁盘，又通过内核转给驱动服务。每一次箭头穿过虚线，都是一次用户态和内核态的切换。微内核把模块解耦了，代价就是这些额外的穿越。

Windows NT 和 macOS 的 XNU 属于混合内核：保留了微内核的模块化思想，又把一部分服务放回内核，以折中性能和可靠性。外核（exokernel）走得更远，内核只负责物理资源的分配和保护，文件系统、网络协议等具体管理交给用户态的库操作系统，优点是灵活、抽象层少，缺点是资源管理复杂，应用之间难以统一。

1.8 中断与异常

中断是 CPU 对系统内外事件的一种响应：CPU 暂停当前程序，保存现场，转去执行处理程序，完成后再返回。它是操作系统取得控制权的基础。时钟中断让分时和抢占式调度成为可能；I/O 完成中断让 CPU 不必轮询设备；缺页异常让虚拟存储得以实现；系统调用则通过陷入指令主动进入内核。

408 里“中断”有广义和狭义两种用法。狭义的中断指外中断，来自 CPU 外部；广义的中断还包括异常（内中断），由 CPU 内部在当前指令执行过程中检测到。做题时看到“内中断”，按异常理解；看到“中断”单独出现，先看上下文是广义还是狭义。

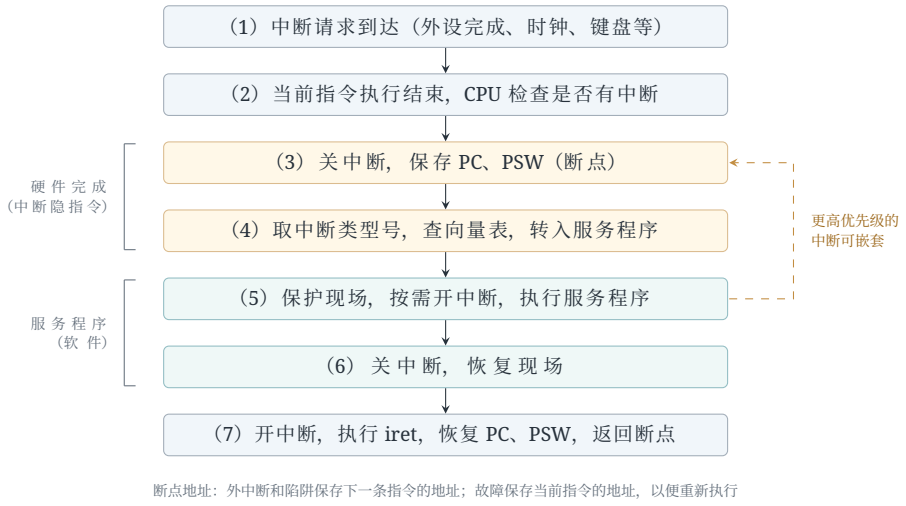
外中断与当前指令无关，是异步的。它分两类。可屏蔽中断通过 INTR 引脚请求，受 PSW 中的中断允许位 IF 控制：IF=1 时 CPU 响应，IF=0 时请求挂起，等开中断后再处理。键盘、打印机、时钟、网卡的中断通常属于这一类。不可屏蔽中断通过 NMI 引脚请求，不受 IF 影响，用于电源故障、内存校验错等紧急事件。

异常与当前指令有关，是同步的。按处理后的行为分三类。故障（fault）指由当前指令引发、处理后可以重新执行当前指令的异常，典型例子是缺页：页面装入后，重新执行那条访存指令就能继续。陷阱（trap）指有意触发的异常，处理后返回下一条指令，典型例子是系统调用和调试断点。终止（abort）指严重错误，程序无法继续，典型例子是硬件故障、总线错误。常见口径把缺页、除零、非法操作码、地址越界归入故障；除零虽然归为故障，但通常无法修复，内核会终止进程。不同体系结构对个别异常的归类可能不同，408 按教材口径答题。

外中断与异常的对比

	外中断（狭义中断）	异常（内中断）
来源	CPU 外部，外设或时钟	CPU 内部，指令执行过程
与当前指令的关系	异步，与当前指令无关	同步，由当前指令触发
检测时机	每条指令执行结束时检查	指令执行过程中检测
返回地址	下一条指令	故障返回当前指令，陷阱返回下一条指令；终止不返回
典型例子	时钟中断、I/O 完成、键盘	缺页、除零、非法操作码、系统调用、断点

CPU 响应外中断要满足几个条件：有中断请求；该请求没有被屏蔽（可屏蔽中断要求 IF=1）；当前指令已经执行结束。多个请求同时到达时，由硬件按优先级判优，不可屏蔽中断优先于可屏蔽中断，高优先级优先于低优先级。



图里的（1）到（7）是外中断处理的一般流程。异常在指令执行过程中检测，但保存现场、查表、执行处理程序、返回的骨架相同；故障返回当前指令，陷阱返回下一条指令。（1）是中断请求，（2）是 CPU 在指令边界检查中断，（3）和（4）由硬件完成，合称中断隐指令：关中断、保存断点（PC 和 PSW）、引出中断服务程序。注意“隐指令”不是程序里的一条指令，而是硬件在响应中断时自动完成的一串动作。（5）到（7）由软件完成：服务程序先保护通用寄存器，再按需要开中断，然后处理设备；结束时关中断、恢复现场，最后开中断并用 iret 返回。

断点地址决定返回后从哪里继续。外中断发生在指令边界，当前指令已经执行完，断点就是下一条指令的地址；陷阱（比如系统调用）也返回下一条指令；故障（比如缺页）要重新执行当前指令，断点保存的是当前指令的地址。这个区别在 408 选择题里经常出现。

中断向量表存放各类型中断的服务程序入口地址。CPU 取到中断类型号 n 后，用“向量表基址 $+\,n\times$ 表项大小”找到入口。若每个表项 4 字节、基址为 B，入口地址就放在 $B+4n$ 处。表中存的是入口地址，不是服务程序的代码。

如果服务程序在处理过程中开中断，并且新来的中断优先级更高，CPU 可以暂停当前服务程序，转去处理新的中断，这就是中断嵌套。嵌套的层次受优先级和屏蔽字控制：屏蔽字决定哪些中断源可以被响应，优先级决定同时到达时先响应谁。嵌套让高优先级事件得到及时处理，但层数太深会耗尽内核栈，所以实际系统会限制嵌套深度。

中断服务程序运行在内核态，被中断的程序可以是用户程序，也可以是内核程序。也就是说，中断可以在用户态发生，也可以在内核态发生；前者会触发一次从用户态到内核态的切换，后者

不会。

1.9 系统调用

系统调用是操作系统提供给应用程序的编程接口，是用户程序请求内核服务的规范入口。程序要读文件、创建进程、申请内存、发送网络包，都要通过系统调用把请求交给内核。用户程序不能直接执行 I/O 指令，也不能直接改页表，系统调用是它合法使用这些功能的通道。

以 `read(fd, buf, n)` 为例。C 库里的 `read` 是一个封装：它把系统调用号放进约定的寄存器，把文件描述符、缓冲区地址、字节数放进参数寄存器，然后执行一条陷入指令。CPU 收到陷入后，从用户态切换到内核态，跳到内核的系统调用入口；内核根据调用号查表，找到 `sys_read`，检查参数，访问文件系统，必要时经过设备驱动，把数据复制到用户缓冲区，最后把返回值放回寄存器，执行中断返回指令回到用户态。



图里把系统调用的路径分成用户态和内核态两段。用户态做三件事：调用库函数、准备调用号和参数、执行陷入指令。陷入指令是分界线，它让 CPU 从用户态进入内核态，同时保存返回地址和 PSW。内核态也做三件事：保存现场并按调用号查表、执行服务例程、把结果写回并恢复现场。最后一步执行中断返回指令回到用户态，库函数把内核的返回值交给应用程序。

参数从用户态传到内核态有三种常见方式。参数少时直接放寄存器，x86-64 用 `rdi`、`rsi`、`rdx`、`r10`、`r8`、`r9` 传前六个参数；参数多时压入用户栈，内核从栈上取；参数特别多或长度可变时，在用户内存里建一张参数表，寄存器只传表的首地址。第三种方式要注意：内核不能直接相信用户传来的指针，必须先检查地址范围，再用专门的复制函数把数据搬到内核空间，否则用户程序可以借内核之手读写任意内存。

系统调用号是服务的编号。内核维护一张系统调用表，表项是各服务例程的入口地址；入口程序用调用号做下标，取出地址并跳转。调用号由操作系统规定，应用程序通过库函数或汇编使用它，不能自己编一个号。返回值通常放在寄存器里；Linux 用负数表示错误，库函数再把它转成 `errno` 并返回 -1。

按功能分，系统调用主要有几类：

常见系统调用的分类

类别	作用	典型调用
进程控制	创建、装入、运行、等待、终止进程	`fork`、`execve`、`wait`、`exit`
文件管理	打开、读、写、定位、关闭文件	`open`、`read`、`write`、`lseek`、`close`
设备管理	请求设备、读写设备、控制设备	`ioctl`、设备文件的 `read/write`
进程通信	建立管道、共享内存、消息队列	`pipe`、`shmget`、`msgget`
内存管理	扩展堆、映射文件或匿名内存	`brk`、`mmap`、`munmap`
信息维护	取进程号、取时间、设置权限	`getpid`、`time`、`chmod`

系统调用和库函数经常被混为一谈。库函数是用户态的一段代码，可以在用户态完成计算，也可以封装系统调用。`strlen` 只扫描内存，不进入内核；`printf` 先把输出放进用户态缓冲区，输出到终端时，缓冲区满或遇到换行才调用 `write`；`malloc` 管理堆，堆不够时用 `brk` 扩展，对大块请求则可能直接用 `mmap`。因此“调用了库函数”不等于“发生了系统调用”，“系统调用一定通过库函数”也不对，程序可以用汇编直接执行陷入指令。

系统调用和普通函数调用的区别在特权级和栈。普通函数调用在同一特权级内跳转，压栈、跳转、返回都使用调用者的栈；系统调用要切换特权级、切换到内核栈、保存和恢复更多现场，开销明显更大。系统调用和中断的区别在触发方式：系统调用是程序主动执行的陷入指令，属于陷阱，同步；外中断由硬件异步触发，与当前指令无关。两者都会让 CPU 进入内核态，但入口和返回地址不同。

系统调用的开销主要来自保存和恢复现场、切换栈、参数检查和权限检查。频繁的小规模 I/O 会让这些开销超过实际工作，所以标准库用缓冲把多次小规模写合并成一次大规模写；操作系统也用页缓存、批量传输减少陷入次数。这些优化不改变系统调用的语义，只是减少穿越边界的次数。

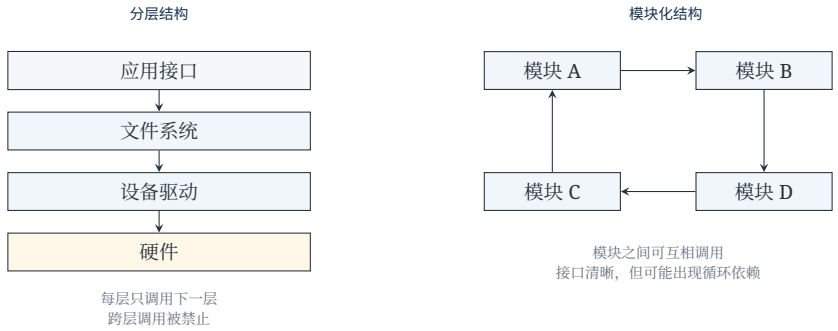
1.10 操作系统的体系结构

除了大内核和微内核，教材还列了几种组织方式。

简单结构没有明确的模块划分，应用甚至可以直接访问硬件。MS-DOS 是典型例子：用户程序可以绕过操作系统操作屏幕和磁盘，系统没有保护，一个程序出错就可能让整台机器死机。这种结构实现简单，但只适合早期的单用户机器。

模块化结构把功能拆成模块，每个模块有明确的接口，模块之间通过接口调用。优点是可以独立开发、独立替换，结构比简单结构清晰；缺点是接口不容易定义好，模块之间可能出现循环依赖，整体行为难以验证。

分层结构把系统排成若干层，每层只允许调用下一层，不能跨层调用。最底层是硬件，往上依次是设备驱动、文件系统、应用接口。分层的好处是每层可以单独调试和验证，替换一层不影响其他层；坏处是请求要穿过多层，开销大，而且层次的边界不容易划清，哪些功能放哪一层常有争议。



图左边是分层结构，箭头只连接相邻两层，请求从应用接口一路向下传到硬件，返回时再一路向上。图右边是模块化结构，四个模块之间可以按需要互相调用，不要求相邻。分层限制调用方向，换来可验证性；模块化放开调用方向，换来灵活性。两种结构都可以和前面的大内核、微内核组合，比如大内核内部常按模块划分，微内核整体上更像一组用户态模块加一个小内核。

外核结构把“资源分配”和“资源管理”分开。内核只负责把物理资源（页框、磁盘块、CPU 时间）分配给应用程序，并保证一个程序不能访问另一个程序的资源；具体怎么用这些资源，由用户态的库操作系统决定。外核减少了抽象层，应用可以按自己的需要组织文件系统和网络协议，适合专用服务器和科研原型。代价是每个应用都要自带一部分操作系统功能，资源管理的复杂度和一致性变差。

1.11 容易混淆的几组概念

1. **并发与并行。**并发是同一时间间隔内交替执行，单核就能做到；并行是同一时刻同时执行，需要多核或多处理器。单核上的时间片轮转是并发，不是并行。
2. **外中断与异常。**外中断来自 CPU 外部，异步，与当前指令无关；异常来自 CPU 内部，同步，由当前指令触发。广义的中断包括异常，狭义的中断只指外中断。
3. **系统调用与库函数。**系统调用进入内核，库函数通常在用户态；库函数可以封装系统调用，也可以完全不进入内核。`printf` 最终通过 `write` 输出，`strlen` 不调任何系统调用。
4. **用户态、内核态与内核程序、用户程序。**前两者是 CPU 的状态，后两者是程序的类别。内核程序运行在内核态，用户程序运行在用户态；用户程序执行系统调用后，CPU 进入内核态执行内核代码，用户程序本身并没有获得特权。
5. **特权指令与陷入指令。**I/O、关中断、改页表、停机属于特权指令，只能在内核态执行；陷入指令在用户态执行，用来主动进入内核，不是特权指令。
6. **中断隐指令与中断服务程序。**前者是硬件在响应中断时自动完成的一串动作：关中断、保存 PC 和 PSW、引出服务程序；后者是软件，负责保护通用寄存器、处理设备、恢复现场。保存 PSW 和 PC 由硬件做，保存通用寄存器由软件做。
7. **大内核与微内核。**大内核把所有功能放在内核态，性能高但耦合强；微内核只留最基本功能，服务放在用户态，可靠、易扩展，但 IPC 和跨态开销大。
8. **多道批处理与分时。**多道批处理追求资源利用率和吞吐量，用户不能交互；分时追求响应时间和交互性，用时间片轮转。两者都可能同时运行多道程序，但目标不同。
9. **操作系统与内核。**内核是操作系统的核心，负责中断、调度、内存管理等；shell、图形界面、编译器等是用户态程序，可能随系统安装，但不属于内核。

1.12 本章在 408 中的位置

本章通常出选择题，考概念辨析：并发与并行、外中断与异常、系统调用与库函数、特权指令与陷入指令、大内核与微内核。偶尔会在综合题里作为背景出现，比如给出一段系统调用路径，问某一步发生在用户态还是内核态。

本章的概念是后面五章的共同语言。第 2 章的进程状态转换要用中断和系统调用解释，第 3 章的缺页处理要用异常和中断的概念，第 5 章的设备驱动要用中断和缓冲。把本章的几条路径理顺，后面的图会好懂得多。

拓展阅读：

- OSTEP 第 1、2 章——用“虚拟化 CPU”和“虚拟化内存”两条主线解释操作系统在做什么，例子比教材更具体。
- 汤小丹《计算机操作系统》第 1 章——国内教材的术语体系，处理机管理、存储器管理、设备管理、文件管理的划分与 408 最接近。
- 陈海波《现代操作系统：原理与实现》第 1 章——从微内核、外核到现代 OS 架构的演变，适合把大内核和微内核的取舍放到真实系统里看。

2 习题

2.1 基础过关题

1. (单项选择题) 下列关于操作系统的叙述中，正确的是

(A) 操作系统是应用软件，与用户程序地位相同
(B) 操作系统管理处理器、存储器、I/O 设备和文件，并为用户和应用程序提供接口
(C) 并发是指同一时刻多个程序同时执行
(D) 虚拟只能把多个物理实体合并为一个逻辑实体

2. (单项选择题) 下列事件中，不可能引起 CPU 从用户态转入内核态的是

(A) 执行除法指令时除数为 0
(B) 用户程序调用 `read()` 读文件
(C) 时钟部件发出时钟中断
(D) 执行一条把两个寄存器内容相加的指令

3. (单项选择题) 下列指令中，只能在内核态执行的是

(A) 关中断（清除 PSW 中的中断允许位）
(B) 把两个通用寄存器的内容相加
(C) 读取本进程的程序计数器
(D) 转移到本进程代码段内的另一地址

4. (填空题) 用户程序执行陷入指令请求系统调用，按异常分类属于 _____ ①，处理结束后返回到陷入指令的 _____ ②。缺页由访存指令触发，按异常分类属于 _____ ③，缺页处理完成后 CPU _____ ④ 引发缺页的那条指令。

5. (单项选择题) 关于微内核结构，下列说法正确的是

(A) 它把文件系统、设备驱动等主要服务放在内核态，以减少态切换

(B) 它只在内核保留调度、进程间通信、中断处理等基本机制，可靠性较高，但态切换开销较大

(C) 它使系统不再发生用户态与内核态的切换

(D) 它允许两个用户态服务进程直接调用对方的函数，无需经过内核

6. (填空题) 某处理器主频为 2 GHz。程序 P 执行 8×10^6 条用户态指令，平均 CPI 为 1；执行过程中共发起 1000 次系统调用，每次从陷入到返回需要 2000 个时钟周期（已包含态切换、参数检查与服务例程本身，忽略其他开销）。P 的执行时间为 _____，系统调用开销所占比例为 _____。

2.2 真题风格与综合训练

7. (真题风格・综合题) 某单 CPU 系统采用多道程序设计，内存可同时容纳 3 道作业，忽略调度与进程切换开销。三道作业同时到达，各自依次执行「CPU 段—I/O 段—CPU 段」，时间如下（单位：ms）。

作业	第 1 段 CPU	I/O	第 2 段 CPU
J1	20	40	10
J2	10	30	20
J3	20	20	20

假设：各作业的 I/O 由独立设备完成，可彼此并行，也可与 CPU 重叠执行；CPU 采用非抢占调度，空闲时若有多道作业就绪，按 J1、J2、J3 的固定次序选择。
(1) 若按 J1→J2→J3 单道顺序执行，总时间是多少？(2) 多道执行时，画出 CPU 时间线，求总完成时间与 CPU 利用率（保留三位有效数字）。(3) 求多道执行时各作业的周转时间。

8. （真题风格·单项选择题）单 CPU 系统中，用户进程 A 正在用户态执行，此时磁盘控制器发出一个 I/O 完成中断，该 I/O 是此前进程 B 请求的。下列叙述中一定成立的是

(A) 进程 A 立即被置为阻塞态

(B) CPU 执行完当前指令后由用户态转入内核态，执行中断服务程序

(C) 操作系统立即切换到进程 B 运行

(D) 中断处理结束后，CPU 一定返回 A 的断点继续执行

9. （真题风格·综合题）用户进程 A 调用 `read(fd, buf, 100)` 从磁盘文件读取 100 字节，所读数据不在页缓存中，需要启动磁盘 I/O。题述事件如下：
① CPU 执行陷入指令，把断点（PC）与 PSW 等现场保存到内核栈，切换到内核态；
② 内核按系统调用号查系统调用表，找到 `read` 的服务例程；
③ 内核检查参数后启动磁盘读，把 A 置为阻塞态，调度进程 B 运行；
④ 磁盘控制器在数据就绪后发出 I/O 完成中断；
⑤ 中断服务程序把数据从磁盘缓冲区送入页缓存，并把 A 唤醒；
⑥ 内核把数据从页缓存拷入 A 的用户缓冲区，恢复 A 的现场，返回用户态，`read` 返回实际读到的字节数。
假设系统在中断处理结束后立即调度 A 运行，且进程切换本身在内核态内部完成、不额外产生用户态与内核态之间的切换。
（1）按发生顺序排列上述事件。（2）从 A 发起 `read` 到 A 收到返回值，A 自身经历了几次「用户态 → 内核态」、几次「内核态 → 用户态」？整个系统（含中断）共几次？（3）为什么 A 不能直接调用内核中的 `read` 服务例程，而必须通过系统调用？

10.（真题风格・单项选择题）下列关于操作系统体系结构的叙述中，正确的是

(A) 层次结构中每一层只使用其相邻下层提供的功能，因此可自底向上逐层调试

(B) 模块化结构中各模块通过接口相互调用，因此模块之间不存在依赖关系

(C) 简单结构（如 MS-DOS）划分了清晰的模块边界，用户程序不能直接访问硬件

(D) 外核把进程调度与内存分配的具体策略都实现在内核中，应用只能被动接受

3 进程管理

同一份 `ls` 程序，在三个终端里各运行一次，系统里就有三个进程。程序只是磁盘上的静态文件，进程则是它在某台机器上的一次执行：有自己的程序计数器、寄存器、栈和地址空间。操作系统管理 CPU，实际上就是管理这些进程——让谁上 CPU、什么时候换下来、多个进程怎样共享资源而不互相破坏。本章围绕这条主线展开。

3.1 程序与进程

定义 3.1: 进程与进程映像

程序（program）是存放在磁盘上的指令和数据集合，是被动的。进程（process）是程序的一次执行实例，是动态的，拥有运行现场。进程在内存中的那份实体称为**进程映像**（process image），通常包括代码段、数据段、堆、栈，以及内核中的进程控制块（PCB）。

同一份程序可以对应多个进程，每个进程的代码段可以共享同一份只读页面，但数据、堆、栈和寄存器现场各自独立。图 2-1 画出了一个进程映像的典型布局：代码段在低地址，栈在高地址并向低地址增长，堆向高地址增长；PCB 不在用户地址空间里，而是放在内核区，由操作系统直接管理。

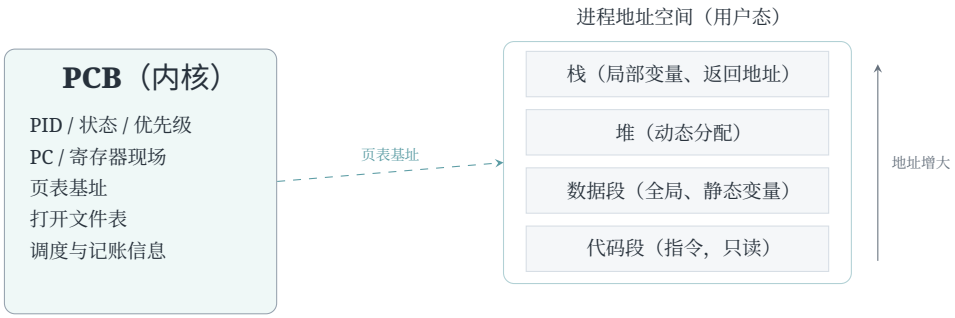


图 2-1　进程映像与 PCB

图里左边是内核中的 PCB，右边是用户态地址空间的四个主要区域。PCB 记录“这个进程是谁、现在什么状态、现场在哪里”，页表基址把它和右边的地址空间连起来。进程切换时，操作系

统把当前 CPU 的 PC、寄存器和栈指针写回 PCB，再读出下一个进程的 PCB；如果两个进程不属于同一地址空间，还要切换页表。

PCB 是操作系统感知进程存在的唯一标志。创建进程的本质是建立 PCB 并分配地址空间；撤销进程的本质是回收它占用的资源并删除 PCB。PCB 通常包含：进程标识（PID、父进程 PID、用户 ID）、处理机状态（通用寄存器、PC、PSW、栈指针）、调度信息（状态、优先级、等待时间）、内存管理信息（页表指针、基址/限长）、I/O 状态（打开文件表、设备、缓冲区）。

创建与终止。 类 UNIX 系统用 `fork()` 创建子进程：调用一次，返回两次——父进程得到子进程的 PID，子进程得到 0，两者从同一条语句继续执行。子进程先复制父进程的地址空间（写时复制），随后通常调用 `exec()` 用新程序替换整个映像。子进程先退出而父进程还没 `wait()` 时，子进程变成**僵尸**（zombie），PCB 和 PID 仍占着；父进程先退出时，子进程成为**孤儿**，由 `init` 或 `systemd` 收养。

进程控制操作（创建、终止、阻塞、唤醒、挂起、激活）必须是**原语**：执行过程中不能被中断，否则 PCB 队列或资源计数可能处于半更新状态。实现上常用关中断或内核锁保证原子性。

3.2 进程状态：PCB 在队列间移动

进程并不是一直都能运行。它可能正在等 CPU，也可能在等磁盘、等键盘、等另一个进程释放信号量。把“为什么现在不能运行”区分开，就有了进程状态。图 2-2 是常见的五状态模型。

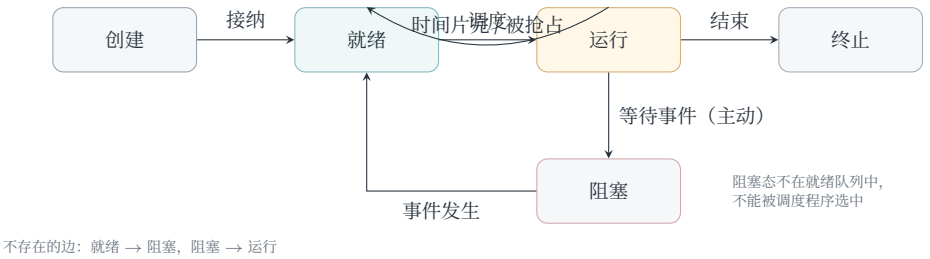


图 2-2　五状态模型与合法转换

每条边对应一个具体事件。创建完成后进程进入就绪态；调度程序从就绪队列选中它，才进入运行态。运行态的时间片用完，或者被更高优先级的进程抢占，退回就绪态——注意它并没有在等什么事件，只是暂时失去 CPU。运行态执行了 `read()` 之类的系统调用，或者执行 P 操作发现资源不够，就主动进入阻塞态。阻塞的原因消失（磁盘中断到达、信号量被 V），进程回到就绪态，而不是直接回到运行态。

两条“不存在的边”是考试常考点。就绪态没有 CPU，无法执行指令，所以不可能自己发起 I/O 或 P 操作进入阻塞态；阻塞态不在就绪队列里，调度程序根本不会选它，所以不可能直接进入运行态。运行 → 就绪与运行 → 阻塞也是两回事：前者是被剥夺 CPU，后者是主动等待事件。

内存紧张时，操作系统还可能把整个进程换到外存，形成**就绪挂起**和**阻塞挂起**两个状态，这就是七状态模型。挂起的进程即使等待的事件发生，也要先被“激活”换回内存，才能参与调度。中级调度负责换出和换入，下一节还会提到。

状态变化最终落实为 PCB 在队列之间的移动：就绪队列放着所有能运行的 PCB，每个设备（或每类事件）有一个等待队列。进程映像通常留在内存不动，除非被挂起。调度程序做的事，就是从就绪队列取下一个 PCB，恢复它的现场。

3.3　线程

一个进程里可以有多条执行流，这就是线程。进程仍然是资源分配和保护的基本单位，线程则是处理机调度的基本单位。图 2-3 左边画出了同一进程内线程共享和私有的部分，右边是三种用户线程到内核线程的映射。

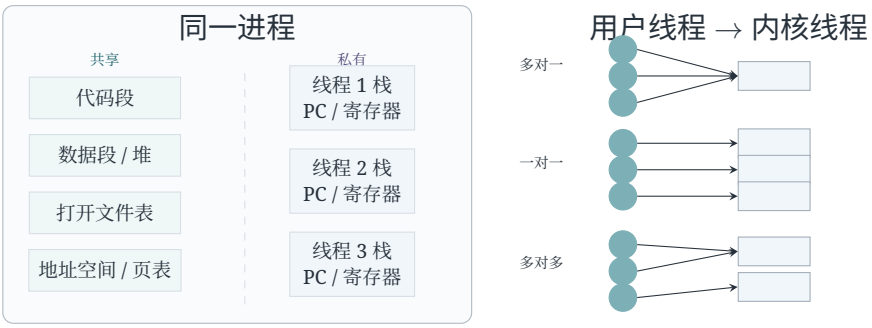


图 2-3　线程的共享/私有资源与三种映射模型

左图中，代码、全局数据、堆、打开文件表和地址空间属于整个进程，三个线程共用；每个线程单独拥有栈、程序计数器和寄存器组。正因为栈和寄存器私有，线程切换时只需保存/恢复这一小部分现场；又因为地址空间共享，线程之间传数据不需要经过内核。

右图是三种映射。多对一把多个用户线程映射到一个内核线程：切换在用户态完成，很快；但一个线程执行阻塞系统调用，内核看到的是整个进程阻塞，其他线程也动不了，而且同一时刻只能用一个 CPU 核。一对一把每个用户线程对应一个内核线程：一个线程阻塞不影响同进程其他线程，也能真正并行；代价是创建和切换都要陷入内核。多对多把多个用户线程复用到较少的内核线程上，兼顾并行和开销，但实现最复杂。

用户级线程由线程库管理，内核不可见；内核级线程由内核调度。判断“一个线程阻塞会不会拖住整个进程”“能不能在多核上并行”时，必须先看采用的是哪种映射，不能只背“线程比进程轻”这一句话。

3.4　进程通信

进程有独立的地址空间，不能像同一进程的线程那样直接读写对方的变量。操作系统提供三种常见方式，图 2-4 画出了它们的数据通路。



图 2-4　共享存储、消息队列与管道

共享存储最快，因为两个进程的页表指向同一组物理页，读写就是普通的内存访问；代价是没有任何天然的互斥，双方必须自己用信号量或互斥量保护共享数据。消息传递通过 `send/receive`

交换消息，数据要从发送方复制到内核、再复制到接收方，开销更大，但隔离性好，也适合跨机器通信。消息传递可以分直接通信（指定对方进程）和间接通信（通过邮箱），还可以分阻塞和非阻塞：阻塞发送要等消息被取走或放进缓冲区，非阻塞发送放得下就返回。

管道是内核中的字节流缓冲区，`pipe()` 返回读端和写端两个文件描述符。普通管道是半双工的，通常只能用于有亲缘关系的进程；命名管道（FIFO）有文件名，可以让无亲缘关系的进程打开。管道为空时，读操作阻塞（只要还有写端打开）；管道满时，写操作阻塞。管道不保留消息边界，写入的多个字节可能被读端分几次读走。

3.5 处理机调度

进程数量通常多于 CPU 数量，调度决定谁先上 CPU。按发生频率和对象，调度分成三级，图 2-5 把它们和队列画在一起。

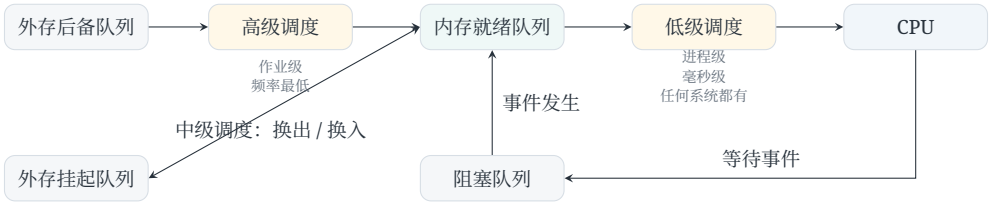


图 2-5　三级调度与进程队列

高级调度（作业调度）决定哪些外存作业被装入内存，频率最低，批处理系统才有；分时和实时系统通常没有这一级。中级调度（内存调度）在内存紧张时把进程换出到外存，条件合适再换入，对应挂起和激活。低级调度（进程调度）从就绪队列选一个进程上 CPU，频率最高，任何操作系统都必须有。图里还可以看到，阻塞队列和就绪队列不是一回事：进程等 I/O 时在阻塞队列，事件发生后 PCB 才被移到就绪队列。

需要调度的情况有：进程运行结束；进程主动阻塞；时间片用完；更高优先级进程到达；中断处理结束返回时发现有更高优先级进程就绪。中断处理程序内部通常不切换进程，调度发生在中断返回之前；内核临界区或原子操作过程中也不调度，否则队列和计数器可能被破坏。

调度指标。 设进程 i 的到达时间为 A_i，运行时间为 B_i，完成时间为 C_i，第一次获得 CPU 的时刻为 F_i。

要点 3.1: 调度指标

$$T_i = C_i - A_i, \qquad W_i = T_i - B_i, \qquad \text{带权周转} = \frac{T_i}{B_i}, \qquad \text{响应时间} = F_i - A_i.$$

平均周转时间为 $\frac{1}{n}\sum_i T_i$，平均带权周转为 $\frac{1}{n}\sum_i T_i/B_i$。等待时间只统计进程在就绪队列中等待 CPU 的时间，不包括阻塞等待 I/O 的时间；响应时间强调“第一次被服务”有多快，适合交互系统。

先来先服务（FCFS）。 按到达顺序服务，非抢占。实现简单，但会出现护航效应：一个长 CPU 作业排在前面，后面的短作业和 I/O 作业都要等，CPU 和 I/O 设备可能交替空闲。

短作业优先（SJF）。 每次选运行时间最短的作业，非抢占。在所有作业同时可用的前提下，SJF 使平均等待时间最短；但长作业可能一直排不上，发生饥饿。实际系统无法预知运行时间，只能用历史数据做指数平均来预测。

最短剩余时间优先（SRTF）。 SJF 的抢占版：新作业到达时，如果它的运行时间比当前进程的剩余时间还短，就抢占 CPU。它能使平均等待/周转时间更短，但调度更频繁，长作业同样可能饥饿。

高响应比优先（HRRN）。 非抢占，每次选响应比最大的作业：

$$R=\frac{\text{等待时间}+\text{要求服务时间}}{\text{要求服务时间}}=1+\frac{\text{等待时间}}{\text{要求服务时间}}.$$

短作业的 R 天然较大，长作业等待越久 R 也越大，因此兼顾了短作业优先和先来先服务，缓解饥饿。代价是每次调度都要重新计算所有等待作业的响应比。

时间片轮转（RR）。 就绪队列按 FIFO 排列，每个进程最多运行一个时间片 q，用完就放到队尾。q 太大，退化为 FCFS；q 太小，上下文切换的开销占比过高。RR 公平、响应快，适合分时系统，但不区分作业长短，平均周转时间不一定好。

优先级调度。 每个进程有优先级，可选抢占或非抢占。静态优先级实现简单，但低优先级进程可能饥饿；动态优先级可以根据等待时间调整，老化（aging）就是每等一段时间提高优先级。

多级队列与多级反馈队列。 多级队列把就绪进程分到几个固定队列，队列之间优先级固定，队列内部可以 RR；进程一般不跨队列，低优先级队列可能饥饿。多级反馈队列允许进程在队列之间移动：新进程进最高优先级队列，时间片用完就降级，低优先级队列的时间片更长；定期把所有进程提升回高优先级队列，防止饥饿。它不需要预知运行时间，能近似短作业优先，是通用操作系统中很常用的一类策略。

图 2-6 用同一组作业比较四种策略。作业为 P_1（0 ms 到达，运行 6 ms）、P_2（1 ms 到达，运行 4 ms）、P_3（2 ms 到达，运行 2 ms），RR 的时间片 $q=2$ ms。

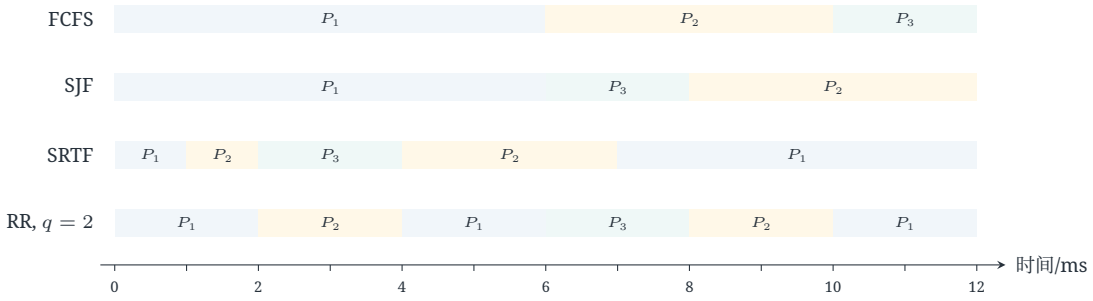


图 2-6　同一组作业在 FCFS、SJF、SRTF、RR 下的甘特图

横轴是时间，每一行是一种调度策略，色块表示当前运行的进程。FCFS 让 P_3 一直等到 10 ms 才开始；SJF 在 P_1 结束后先做较短的 P_3；SRTF 在 1 ms 和 2 ms 两次抢占，使 P_3 在 4 ms 就完成；RR 每 2 ms 换一次，公平但把长作业切得很碎。把完成时间代入指标公式，得到表 2-1。

表 2-1 同一组作业的调度结果

策略	完成时间 (C_1, C_2, C_3)	平均周转/ms	平均带权周转	特点
FCFS	$(6, 10, 12)$	8.33	2.75	简单，短作业吃亏
SJF	$(6, 12, 8)$	7.67	2.25	需预知运行时间
SRTF	$(12, 7, 4)$	6.67	1.50	抢占，长作业可能饥饿
RR, $q = 2$	$(12, 10, 8)$	9.00	2.42	公平，切换开销大

表中 C_1, C_2, C_3 分别是 P_1, P_2, P_3 的完成时间。

这张表说明，平均周转最短不等于“看起来最公平”。SRTF 的平均周转最好，但 P_1 被反复推迟到 12 ms；RR 每个进程都有机会，长作业的完成时间却变长了。选择算法要看系统目标：批处理追求吞吐和平均周转，分时追求响应和公平，实时追求截止时间。

计算 RR 时有一个细节：时间片用完的进程放到队尾；如果同一时刻有新进程到达，新进程入队和被抢占进程入队的先后会影响结果。题目一般会说明；没有说明时，先固定一种约定再算，不要中途改变。

3.6 同步与互斥

先看一个最简单的竞争例子。两个进程共享一个全局变量 `count`，都执行 `count++`。机器级上这不是一条指令，而是“读 `count` 到寄存器、加 1、写回内存”三步。如果进程 P_1 读到 0 后被切换，P_2 也读到 0，各自加 1 写回，最后 `count` 还是 1，一次更新丢了。这类错误取决于执行时序，称为竞态条件（race condition）。

访问共享资源的代码段称为**临界区**。进程进入临界区前要申请许可，离开时释放：

```
while (1) {
    entry_section();      // 申请进入
    critical_section();   // 访问共享资源
    exit_section();       // 释放
    remainder_section();  // 其余工作
}
```

一个好的临界区管理方案要满足四条：空闲让进（没有进程在临界区时，申请者应能进入）、忙则等待（已有进程在临界区时，其他进程必须等待）、有限等待（申请者不能无限期等待）、让权等待（进不去时应阻塞让出 CPU，而不是忙等）。前三条保证正确性，第四条是效率要求；自旋锁不满足让权等待，但仍然可以是正确的。

Peterson 算法。 两个进程的软件互斥方案，共享 `flag[2]` 和 `turn`：

```
bool flag[2] = {false, false};
int turn = 0;
// 进程 i，另一个进程 j = 1 - i
flag[i] = true;       // 我想进
turn = j;             // 如果对方也想进，先让对方进
while (flag[j] && turn == j) ;   // 忙等
/* critical section */
flag[i] = false;      // 我退出
```

`flag[i]` 表示进程 i 想进入，`turn` 用来在双方同时想进时决定谁先进。只要对方不想进，或者 `turn` 指向自己，就可以进入；离开时把自己的 `flag` 置假。它满足互斥、空闲让进和有限等

待，但等待方式是忙等，不满足让权等待。考试模型默认顺序一致性；真实多核上还需要内存屏障，这里不展开。

硬件原子指令。 TestAndSet 返回锁的旧值并把锁置为 true，整个过程不可分割：

```
bool TestAndSet(bool *lock) {
    bool old = *lock;
    *lock = true;
    return old;
}
// 加锁
while (TestAndSet(&lock)) ;
/* critical section */
*lock = false;
```

Swap 或 Exchange 把 lock 和局部变量 key 交换，也能构造自旋锁。原子指令适合多核、临界区很短的场景；等待时间长时，自旋会白烧 CPU，应该阻塞。关中断是单处理器内核保护极短临界区的办法：关中断期间不会被切换，但用户进程不能用它，多处理器上关一个核的中断也挡不住其他核。

信号量。 信号量是一个只能通过 P、V 操作访问的整数。记录型信号量还带一个等待队列，图 2-7 画出了 P、V 对值和队列的影响。

```
// P(S)：申请一个资源
P(S) {
    S.value--;
    if (S.value < 0) block(S.queue);   // 没有资源，阻塞并排队
}
// V(S)：释放一个资源
V(S) {
    S.value++;
    if (S.value <= 0) wakeup(S.queue); // 队列里还有人，唤醒队首
}
```

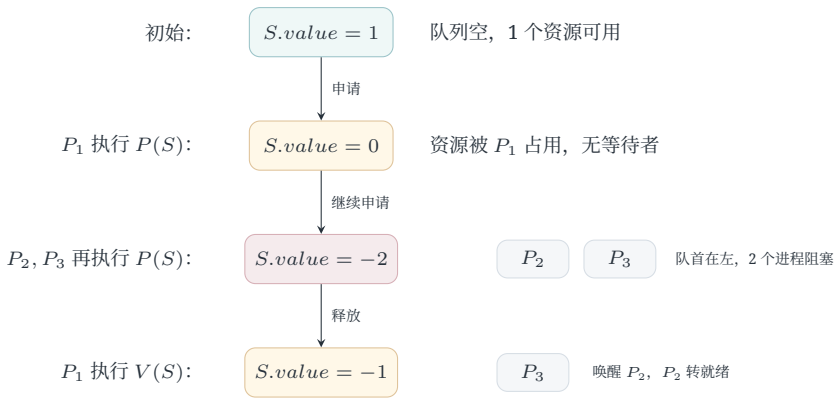


图 2-7　记录型信号量的值、等待队列与 P/V 的影响

初值 1 表示有一个资源。P_1 执行 P 后值变为 0，顺利进入；P_2、P_3 再执行 P，值依次变为 -1、-2，它们被阻塞并按顺序排队。此时 $|S.value| = 2$ 正好是等待进程数。P_1 执行 V 后值从 -2 变到 -1，仍然小于等于 0，说明队列里还有人，于是唤醒队首 P_2；P_3 继续等待。如果 V 后值变成正数，说明没有等待者，资源重新可用。

因此，记录型信号量满足 $S.value > 0$ 时表示可用资源数；$S.value = 0$ 时没有可用资源且没有等待者；$S.value < 0$ 时 $|S.value|$ 是等待队列长度。P 可能阻塞，V 不会阻塞；P、V 本身必须原子执行。

用信号量实现互斥与同步。 互斥信号量初值为 1，进入临界区前 P，离开后 V：

```
semaphore mutex = 1;
P(mutex);
/* critical section */
V(mutex);
```

同步信号量初值为 0，用来约束先后次序。若要求“A 做完某件事后 B 才能做”，就让 A 在事后执行 $V(S)$，B 在事前执行 $P(S)$。B 先跑到 $P(S)$ 时会阻塞，直到 A 发出 V。互斥量的 P/V 通常由同一进程成对执行；同步信号量的 P/V 则分属不同进程。

3.7 经典同步问题

生产者-消费者。 有界缓冲区容量为 N，生产者放数据，消费者取数据。需要三个信号量：`empty` 表示空位数，初值 N；`full` 表示数据个数，初值 0；`mutex` 保护缓冲区的读写指针和内容，初值 1。

```
semaphore empty = N, full = 0, mutex = 1;

Producer() {
    while (1) {
        item = produce();
        P(empty);          // 先确认有空位
        P(mutex);          // 再锁住缓冲区
        put(item);
        V(mutex);
        V(full);           // 数据数加 1
    }
}

Consumer() {
    while (1) {
        P(full);           // 先确认有数据
        P(mutex);
        item = get();
        V(mutex);
        V(empty);          // 空位数加 1
        consume(item);
    }
}
```

图 2-8 画出了环形缓冲区的状态：8 个槽中已有 3 个数据，`in` 指向下一个空槽，`out` 指向最早的数据，因此 `empty`=5、`full`=3。

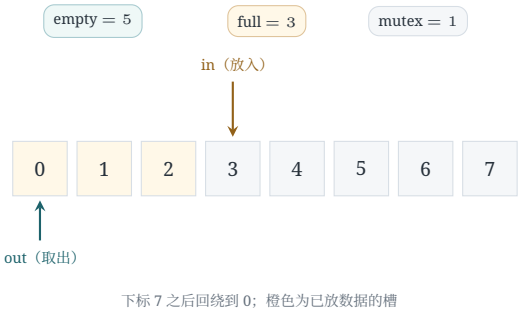


图 2-8 生产者-消费者的环形缓冲区

这段代码的关键是 P 的顺序：必须先 $P(empty)$ 或 $P(full)$，再 $P(mutex)$。如果生产者先拿到 `mutex`，再在 `empty` 上阻塞，那么缓冲区满时它占着锁睡觉；消费者要取数据也得先拿 `mutex`，于是消费者也被阻塞，双方互相等待，形成死锁。V 的顺序通常先 $V(mutex)$ 再 $V(full)$ 或 $V(empty)$：先释放锁，再唤醒对方，被唤醒的进程可以立刻进入临界区，减少一次无谓的阻塞。反过来写不影响正确性，但效率略差。

读者-写者。 多个读者可以同时读，读者和写者不能同时访问，写者和写者也不能同时访问。用 `readcount` 记录当前读者数，`mutex` 保护 `readcount`，`rw` 保护文件：

```
semaphore mutex = 1, rw = 1;
int readcount = 0;

Reader() {
    P(mutex);
    readcount++;
    if (readcount == 1) P(rw);   // 第一个读者锁住文件
    V(mutex);
    read();
    P(mutex);
    readcount--;
    if (readcount == 0) V(rw);   // 最后一个读者释放文件
    V(mutex);
}

Writer() {
    P(rw);
    write();
    V(rw);
}
```

第一个读者负责加锁，最后一个读者负责解锁，中间的读者只改 `readcount`。这样读者之间可以并行，写者必须等所有读者离开。这个版本是读者优先：只要不断有读者到达，写者可能一直等下去。写者优先需要额外的信号量，在写者到达时阻止新读者进入；实现更复杂，也可能让读者饥饿。

管程。 管程把共享变量和操作这些变量的过程封装在一起，保证同一时刻只有一个进程在管程内执行，互斥由编译器或运行时自动保证，不需要程序员手写 P/V。条件变量用于等待某个条件成立：`wait` 释放管程锁并阻塞，`signal` 唤醒一个等待者。条件变量不是信号量，它没有计数值；如果 `signal` 时没有等待者，这个信号就丢失了，不像 V 操作会把值加 1。Hoare 语义下 `signal`

立即把管程交给被唤醒者；Mesa 语义下被唤醒者只是转入就绪，等当前进程退出管程后再竞争。Java 的 `synchronized` 和 `wait/notify` 就是管程风格的同步。

3.8 死锁

死锁（deadlock）是指一组进程中的每个进程都在等待一个只能由组内其他进程释放的资源，没有外力介入，谁也无法继续。比如 P_1 占着打印机、等扫描仪，P_2 占着扫描仪、等打印机，双方都不肯先释放。

死锁同时满足四个必要条件：

1. **互斥**：资源一次只能被一个进程使用；
2. **请求并保持**：进程已经占有一些资源，又提出新的请求，且不释放已占有的资源；
3. **不可剥夺**：资源只能由占有者主动释放，不能被强行抢走；
4. **循环等待**：存在一个进程-资源的环形等待链。

这四个条件缺一不可，但同时成立也不一定真的死锁（多实例资源下尤其如此）。图 2-9 用资源分配图画出了两种情形。圆圈表示进程，方框表示资源，方框里的点表示实例；请求边从进程指向资源，分配边从资源指向进程（图中用虚线）。

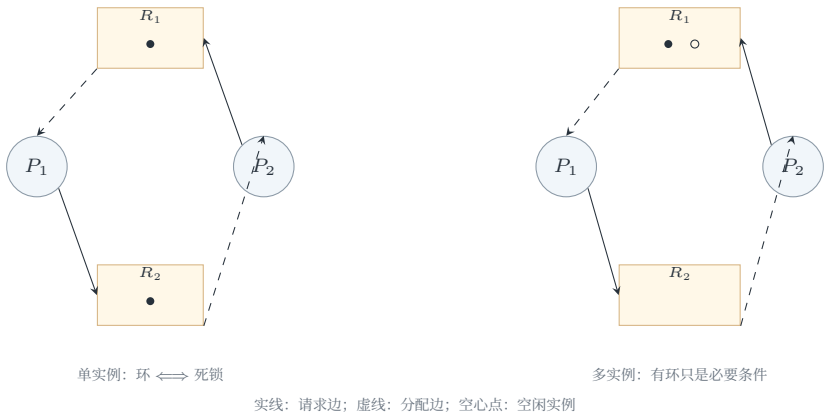


图 2-9　资源分配图中的环与死锁

左图每类资源只有一个实例，P_1 等 R_2、P_2 等 R_1，环上的每个请求都指向对方已经占着的唯一实例，无法化解，所以有环就是死锁。右图 R_1 有两个实例，P_1 占了一个，另一个空闲；P_2 请求 R_1 时可以直接拿到空闲实例，运行完后释放 R_2，P_1 就能继续。因此图里虽然有环，系统并没有死锁。

判断多实例资源是否死锁，要做**资源分配图化简**：反复找一个请求能够被当前可用资源满足的进程，假设它运行完成并释放全部资源，删去它的请求边和分配边；如果最终所有进程都能被消去，就没有死锁；剩下不能消去的进程集合就是死锁进程。化简的先后顺序不影响最终结论。

死锁处理策略。 表 2-2 概括了四类做法。

表 2-2 死锁的四种处理策略

策略	发生时机	典型做法
预防	设计阶段，静态	破坏四个必要条件之一
避免	每次分配前，动态	银行家算法，检查是否仍安全
检测	运行过程中	资源分配图化简、等待图找环
恢复	检测到死锁后	终止进程或剥夺资源

预防。 破坏互斥：把独占设备改造成共享服务，例如打印机假脱机，但很多资源无法这样改造。破坏请求并保持：进程运行前一次申请全部资源，或者申请新资源前先释放已占有的资源；前者资源利用率低，后者可能反复申请释放。破坏不可剥夺：允许抢占资源，被抢的进程回滚到安全点；CPU 和内存容易抢占，打印机这类设备很难。破坏循环等待：给资源编号，进程只能按编号递增的顺序申请，这样不会形成环；代价是资源申请顺序受限，可能产生不必要的等待。

避免。 不破坏条件，而是在每次分配前判断“分配后系统是否还安全”。安全状态是指存在一个安全序列，按这个顺序每个进程都能获得它声明的最大需求并完成。安全状态一定不会死锁；不安全状态只是可能死锁，因为进程实际请求的资源可能小于它的最大需求。银行家算法就是做这个判断。

设系统有 n 个进程、m 类资源，记录：

$$Available[j] \text{ 为当前可用的} j \text{ 类资源数}, \quad Need[i][j] = Max[i][j] - Allocation[i][j].$$

安全性检查的流程如图 2-10。

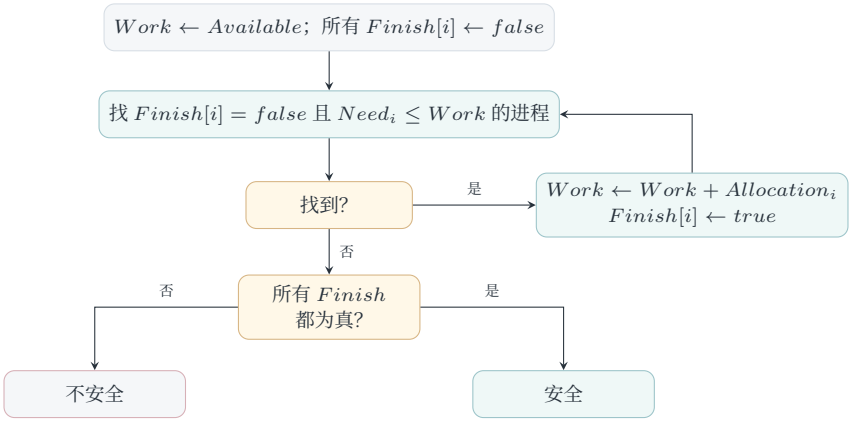


图 2-10 银行家算法的安全性检查

从 $Work = Available$ 开始，反复找需求不超过当前可用资源的未完成进程；找到就假设它完成，把已分配给它的资源归还到 $Work$，再继续找。如果所有进程都能完成，系统安全，这个顺序就是一个安全序列；如果还有进程未完成却找不到可推进的进程，系统不安全。安全序列通常不唯一，找到一个即可。

真正分配时，还要先检查请求是否合法：$Request_i \le Need_i$ 且 $Request_i \le Available$。然后做一次试分配，修改 $Available$、$Allocation_i$、$Need_i$，再跑一遍安全性检查；安全才确认分配，不安全就回滚，让进程等待。银行家算法要求每个进程预先声明最大需求，这个条件在实际系统中很难满足，所以它更多用于理论分析和批处理场景。

检测与恢复。 单实例资源可以用等待图：把资源结点去掉，若 P_i 等待 P_j 占有的资源，就画一条 $P_i \to P_j$ 的边；图中有环就是死锁。多实例资源仍用资源分配图化简。检测本身有开销：太频繁浪费 CPU，太稀疏则死锁进程长期占着资源。

恢复有两种思路。终止进程：可以一次终止所有死锁进程，简单但损失大；也可以逐个终止，直到死锁解除，代价是每次都要重新检测。剥夺资源：从某些进程手中抢走资源，分配给死锁进程，被抢的进程回滚到之前的安全点。选择牺牲者时要考虑已运行时间、还需要的资源、回滚代价，并防止同一个进程反复被选中而饥饿。

死锁、饥饿与活锁。 死锁一定有循环等待，组内所有进程都无法推进；饥饿没有环，进程只是长期得不到资源，系统其他部分仍在运行，理论上还有机会；活锁是进程不断重试、状态一直在变，但没有实际进展，例如两个进程反复检测到冲突后同时退让。三者都可能表现为“程序不动了”，但原因和处理方式不同。

3.9 几个容易混淆的判断

- **阻塞与挂起。**阻塞是进程在内存中等待某个事件，事件发生就转就绪；挂起是进程被换到外存，即使事件发生，也要先激活换回内存。就绪挂起的进程不参加调度。
- **死锁与饥饿。**死锁是循环等待，组内进程永远无法推进；饥饿没有循环，只是优先级或资源分配策略导致某个进程长期得不到服务。死锁必须破坏环或剥夺资源，饥饿可以用老化缓解。
- **互斥与同步。**互斥是“同一时刻只能有一个进程访问”，同步是“按某种先后次序执行”。互斥可以看作同步的一种特例；生产者-消费者既需要互斥（缓冲区），也需要同步（空位和数据个数）。
- **用户级线程的阻塞。**在多对一模型中，一个线程执行阻塞系统调用，内核按进程阻塞，整个进程都停下；一对一或多对多模型中，其他线程可能继续运行。判断这类题必须先看映射模型。
- **时间片 q 的取舍。** q 太大，RR 退化为 FCFS；q 太小，上下文切换开销占比过高。q 通常取略大于一次典型交互所需的时间。
- **P/V 的顺序。**互斥场景先 P 资源信号量、再 P 互斥量；颠倒后可能占着锁阻塞，形成死锁。V 的顺序通常先释放互斥量，再唤醒等待者。
- **不安全不等于死锁。**银行家算法判定不安全，只说明按最大需求继续分配可能进入死锁；进程实际请求较小，系统仍可能顺利完成。安全状态则一定不会死锁。
- **进程切换与线程切换。**线程切换要保存/恢复寄存器和栈，但同一进程的线程共享地址空间，不需要切换页表；进程切换通常还要换页表并处理 TLB，开销更大。

4 习题

4.1 基础过关题

1. （单项选择题）在单处理机系统中，进程 P 因时间片用完而被切换，进程 Q 因请求磁盘读操作而主动让出处理机。这两个进程本次状态转换后分别进入

(A) 就绪态、阻塞态　(B) 阻塞态、就绪态

(C) 就绪态、就绪态　(D) 阻塞态、阻塞态

2. （单项选择题）同一进程内的两个线程，下列资源中两者都私有的是

(A) 地址空间与打开文件表　(B) 栈与程序计数器

(C) 代码段与全局变量　(D) 堆与已打开文件

3. （填空题）某类资源共有 3 个，用记录型信号量 S 管理，初值 $S=3$。某时刻 S 的值为 -2，则此时该信号量的等待队列中有 ____ 个进程，已有 ____ 个进程获得该类资源。

4. （单项选择题）在生产者-消费者问题中，若生产者把 `P(empty)` 与 `P(mutex)` 的次序颠倒为先执行 `P(mutex)`、后执行 `P(empty)`，则当缓冲区已满时可能出现

(A) 新数据覆盖缓冲区中的旧数据　(B) 生产者与消费者互相等待，发生死锁

(C) 消费者重复取走同一个数据　(D) 信号量的初值被改写

5. （单项选择题）某系统的每类资源都有多个实例。若某时刻的资源分配图中存在环，则

(A) 系统一定处于死锁状态　(B) 系统一定未发生死锁

(C) 系统可能死锁，也可能未死锁　(D) 环上的每个进程都必然死锁

6. （填空题）某系统采用高响应比优先（HRRN）调度。当前有作业 A、B 等待调度：A 已等待 7 ms，要求服务时间 3 ms；B 已等待 2 ms，要求服务时间 1 ms。A 的响应比为 ____，B 的响应比为 ____，应优先调度 ____。

4.2 真题风格与综合训练

7. （单项选择题）某系统有 A、B 两类资源，总数分别为 4 和 2。当前分配情况如下表，此时可用资源为 $(1,0)$。下列序列中，可能成为安全序列的是

进程	Allocation		Max	
	A	B	A	B
P_1	1	1	2	1
P_2	0	1	0	2
P_3	2	0	3	1

(A) $P_2 \to P_1 \to P_3$　(B) $P_3 \to P_2 \to P_1$

(C) $P_1 \to P_3 \to P_2$　(D) 系统不安全，不存在安全序列

8. （单项选择题）某系统采用时间片轮转调度，时间片 $q = 2$ ms，忽略进程切换开销。三个进程的到达时间与运行时间如下表，同一时刻没有进程到达时间片结束点，就绪队列次序无歧义。它们的平均周转时间为

进程	P_1	P_2	P_3
到达时间/ms	0	1	3
运行时间/ms	5	3	1

(A) 6.0 ms (B) $\frac{20}{3}$ ms（约 6.67 ms）

(C) 7.0 ms (D) $\frac{22}{3}$ ms（约 7.33 ms）

9. （综合应用题）三个进程通过两个有界缓冲区流水作业：缓冲区 B_1 可放 4 个数据，B_2 可放 3 个数据。P_1 每次生成一个数据并放入 B_1；P_2 每次从 B_1 取出一个数据，加工后放入 B_2；P_3 每次从 B_2 取出一个数据并处理。缓冲区的读写指针是共享变量，读写操作不是原子操作，同一缓冲区的存、取不能交错。要求：

(a) 给出所有信号量的名称、含义与初值；

(b) 用 PV 操作写出三个进程的同步与互斥流程，并说明 P、V 的次序理由。

10.（综合应用题）某系统有 A、B、C 三类资源，总数分别为 $(5,5,4)$。当前分配情况如下表。

进程	Allocation			Max		
	A	B	C	A	B	C
P_0	1	0	2	3	2	2
P_1	2	1	0	3	3	1
P_2	0	1	1	2	1	2
P_3	1	1	0	2	2	1

(a) 求各进程的 Need 和当前 Available，判断系统是否安全，并给出一个安全序列；

(b) 若 P_0 请求 $(1,1,0)$，能否立即分配?

(c) 若 P_0 请求 $(0,1,0)$，能否立即分配?

其中 (2)(3) 相互独立，均以初始状态为出发点；只有请求同时不超过 Need 与 Available 时才进入试分配。

5 内存管理

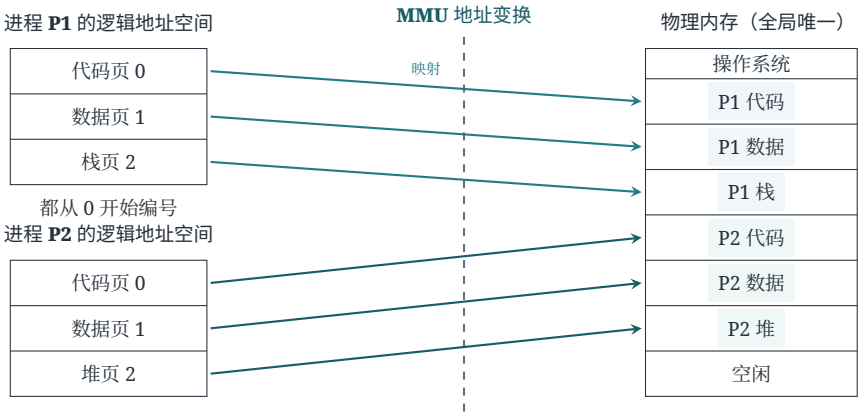
先看一个具体的麻烦。一个 32 位程序编译出来以后，代码里的地址从 0 开始，CPU 执行 `mov 0x12345, %eax` 时发出的也是 `0x12345`。但物理内存里同时还放着操作系统、这个进程、以及

别的进程，0x12345 这个物理单元未必属于它；即使碰巧属于，换一台机器或换一个进程，这个地址又对不上了。内存管理要解决的，就是让每个进程都以为自己独占一片从 0 开始的地址空间，而真正落到物理内存时又互不干扰。完成这件事的硬件叫 MMU（Memory Management Unit，内存管理单元），它把 CPU 发出的逻辑地址翻译成物理地址；操作系统则负责建立翻译用的表格、分配和回收物理空间，并在需要时把暂时不用的页面放到磁盘上。

本章沿着这条线展开：先看地址空间和连续分配，再进入分页、分段、页表与 TLB，最后把缺页处理和页面置换放进虚拟内存的框架里。

5.1 地址空间、重定位与内存保护

CPU 执行指令时发出的地址叫**逻辑地址**（Logical Address），也叫虚拟地址。它是程序视角的地址，每个进程都从 0 开始编址，所以不同进程可以有相同的逻辑地址。内存芯片真正接收的地址叫**物理地址**（Physical Address），整个系统只有一套物理地址空间，所有进程和操作系统共用。逻辑地址的集合叫进程的**地址空间**；32 位系统里通常是 $0 \sim 2^{32}-1$，共 4 GB。



图里左右两边都是“地址”，但含义不同：左边两个进程各自从 0 开始，页号可以重复；右边物理内存只有一份，P1 和 P2 的页面被放到不同的物理位置。中间虚线表示 MMU 在翻译，进程本身看不到右边的具体位置。

什么时候把逻辑地址变成物理地址

地址变换发生的时机有三种，差别不只是“早一点还是晚一点”，而是程序装入后还能不能移动。

编译时：如果编译器事先知道程序会装到物理内存的哪个位置，就可以直接生成绝对地址代码。这种代码装入后不能换位置，现代通用操作系统基本不用。

装入时（静态重定位）：编译器生成可重定位代码，装入程序在把程序放进内存时，把所有与地址有关的指令一次性加上装入基址。优点是不需要 MMU；缺点是程序一旦装入就不能移动，因此不支持内存紧凑，也不方便换入换出。

执行时（动态重定位）：指令里仍然保存逻辑地址，每次访存时由 MMU 的重定位寄存器（基址寄存器）加上基址，得到物理地址：

$$\text{物理地址} = \text{逻辑地址} + \text{基址寄存器}.$$

程序可以在内存中移动，只要更新基址寄存器；内存紧凑、交换、虚拟内存都依赖这一点。代价是每次访存多一次加法，并且需要硬件支持。

链接的时机容易和重定位混在一起，但它们是两件事。静态链接在程序运行前把目标模块拼成完整可执行文件；装入时动态链接在装入过程中解析并链接共享模块；运行时动态链接把链接

推迟到第一次调用，例如 Linux 的 `dlopen` 和 Windows 的 `LoadLibrary`。动态链接解决的是“模块什么时候被拼进来”，动态重定位解决的是“地址什么时候被换算”，不要混为一谈。

内存保护

只做加法还不够。进程 P1 如果把逻辑地址加到超出自己区域的位置，不能让它读到 P2 的数据。连续分配里通常用一对寄存器：基址寄存器保存进程的起始物理地址，界限寄存器保存进程的长度。MMU 先检查

$$0 \le \text{逻辑地址} < \text{界限寄存器},$$

越界就产生地址越界异常，由操作系统处理；检查通过后再加基址。这样基址决定“从哪里开始”，界限决定“能走多远”。

分页和分段系统把保护做得更细：页表项或段表项里带读/写/执行权限位，MMU 在翻译的同时检查本次访问是否被允许。后面讲页表项时会再看到这些位。

5.2 连续分配与碎片

连续分配要求一个进程占用一段连续的物理内存。它实现简单，也是理解碎片问题的起点。

单一连续分配与固定分区

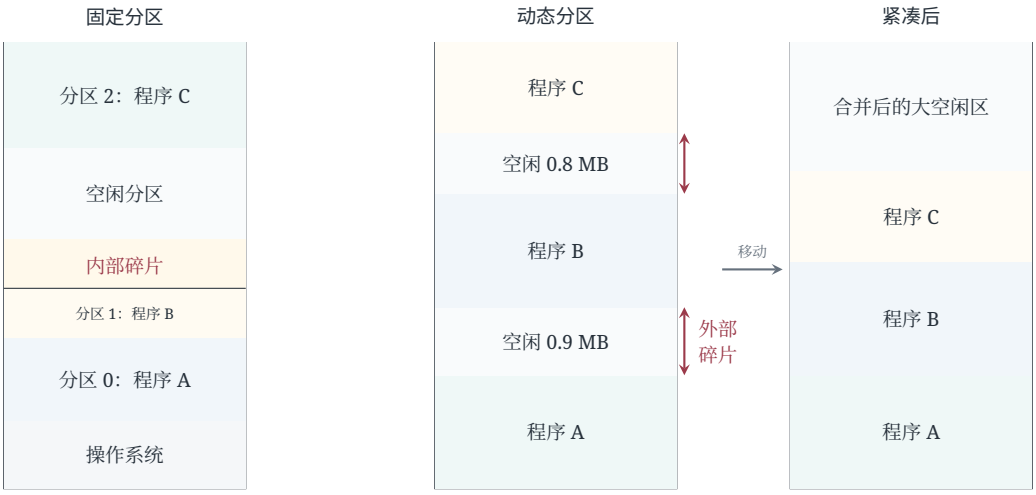
单一连续分配把内存分成系统区和用户区，用户区一次只放一道程序。它不需要复杂的分配算法，但内存利用率很低，只适合早期的单用户单任务系统。

固定分区分配把用户区预先切成若干大小固定的分区，每个分区放一道程序。分区说明表记录每个分区的起始地址、大小和状态。它的缺点是**内部碎片**：分区按固定大小切好，程序用不完的部分就浪费在分区内部。例如一个 8 MB 的分区装入 5 MB 的程序，剩下 3 MB 虽然空闲，却不能再分给别人。

动态分区与外部碎片

动态分区不预先切块。进程装入时，操作系统从空闲区里找一块足够大的连续空间；如果空闲块比请求大，就把剩下的部分留在空闲链里。进程退出后，它占用的空间重新变成空闲块。这样不会产生内部碎片，但会在内存中留下许多不连续的小空闲区，称为**外部碎片**：空闲总量也许够，却凑不出一段连续的请求。

解决外部碎片的办法是**紧凑**（Compaction）：把已分配的程序往一端移动，让空闲区合并成一大块。移动程序意味着指令里的地址不能是绝对地址，因此紧凑必须建立在动态重定位之上。



左图固定分区的浪费发生在分区内部，程序 B 用不完的那半块仍然属于它；中图动态分区的空闲块夹在程序之间，总量也许够，但无法满足一次连续的大请求；右图通过移动程序把空闲块合并到一端。紧凑需要动态重定位，否则程序移动后指令里的地址就错了。

空闲链的查找算法

动态分区通常把空闲块串成空闲链。新请求到来时，按某种规则挑一块：

常见空闲分区分配算法

算法	选择规则	特点
首次适应 FF	从链头（低地址）开始，选第一个足够大的块	查找快；低地址端容易积累小碎片，高地址端保留大块
邻近适应 NF	从上次分配位置继续向后找	不必每次从头扫描，空闲区分布较均匀；大块可能被较快消耗
最佳适应 BF	选所有足够大的块中最小的一块	本次剩余最小，但会留下大量难以利用的微小碎片，而且通常要遍历全链
最坏适应 WF	选最大的一块	剩余块仍然较大，短期内还能继续分配；大块很快被耗尽

没有一种算法在所有请求序列下都最好。考试里常比较“首次适应”和“最佳适应”：前者通常查找开销小，后者本次浪费小但碎片更碎。判断时要看题目强调的是查找时间、剩余空间还是碎片形态，不能只背一句“最佳适应最好”。

5.3 分页：把地址切成页号和页内偏移

连续分配要求整块空间，外部碎片很难避免。分页换了一个思路：把逻辑地址空间切成大小相等的**页**（Page），把物理内存切成同样大小的**页框**（Page Frame，也叫物理块）。进程的页可以离散地装进任意空闲页框，页与页之间不需要连续，于是外部碎片消失；代价是最后一页可能装不满，产生内部碎片。

地址结构

设页面大小为 2^k 字节。逻辑地址 A 的低 k 位就是页内偏移 d，高位是页号 p：

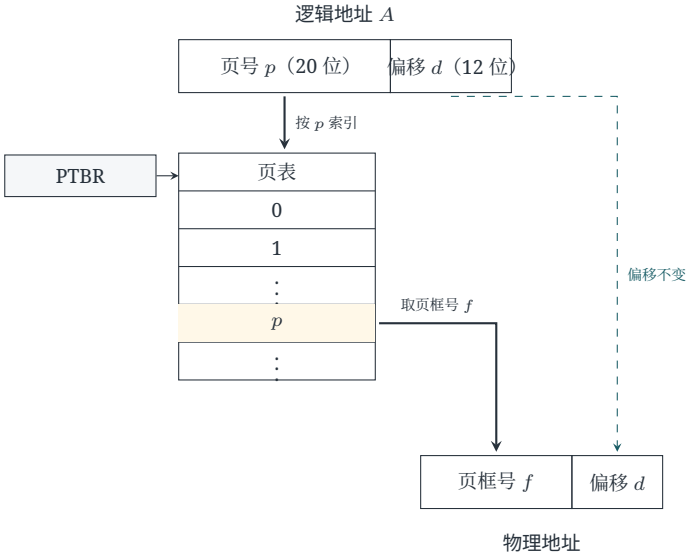
$$p = \left\lfloor \frac{A}{2^k} \right\rfloor, \qquad d = A \bmod 2^k, \qquad A = (p \ll k) \mid d.$$

物理地址由页框号 f 和同一个偏移拼成：

$$\text{物理地址} = (f \ll k) \mid d = f \cdot 2^k + d.$$

注意偏移在翻译前后**原样保留**，变的只有前面的页号。实际系统通常把页面大小取成 2 的幂，这样硬件可以直接按位切分，不必做除法。

例如页面大小 4 KB（$k = 12$），逻辑地址 `0x12345` 的页号是 `0x12`，偏移是 `0x345`。若页表把页 `0x12` 映射到页框 `0x7`，物理地址就是 `0x7345`。计算时先把地址写成二进制或十六进制，再按 12 位一组切开，比做十进制除法更不容易错。



图中页表项地址由 PTBR 加上“页号 × 页表项大小”得到；取出页框号后，把它和原来的偏移拼起来。页表本身存放在内存里，所以不查 TLB 时，一次数据访问至少要访问内存两次：一次读页表项，一次读数据。

页面大小是一个权衡。页大，页表项少、页表小，TLB 能覆盖更大的地址范围，但最后一页的内部碎片更大，换入换出一次搬的数据也更多；页小则相反。408 题目一般直接给出页面大小，实际系统常见 4 KB，也有 2 MB、1 GB 的大页。

页面大小的取舍

	页面较大	页面较小
页表	页表项少，页表占内存少	页表项多，页表占内存多
TLB	同样项数能覆盖更大范围	覆盖范围小，TLB 缺失更频繁
碎片	最后一页的内部碎片大	内部碎片小
I/O	一次换入换出数据多，传输时间长	传输时间短，但缺页次数可能更多

页表与页表项

每个进程有一张页表，页表基址寄存器 PTBR 指向它在内存中的起始位置。进程切换时，操作系统把 PTBR 换成新进程的页表基址，地址空间就跟着换了。页表项（PTE，Page Table Entry）通常包含：



图中的字段宽度只是示意，具体位数由系统决定。几个位的作用要分清：

- **有效位 V**：该页当前是否在内存。$V=0$ 时访问该页会触发缺页异常；它也可能表示这个页号根本没有映射，处理程序要区分“合法但不在内存”和“非法地址”。
- **页框号**：该页所在的物理页框。多级页表里，中间层页表项存放的是下一级页表的页框号。
- **保护位**：读、写、执行权限，以及用户态/内核态权限。越权访问同样由 MMU 当场发现。
- **访问位 A**：该页最近是否被访问过，供 CLOCK 等置换算法使用。
- **修改位 D**（脏位）：该页装入后是否被写过。换出时若 $D=1$，必须先写回磁盘，否则修改会丢失；$D=0$ 的页可以直接丢弃。

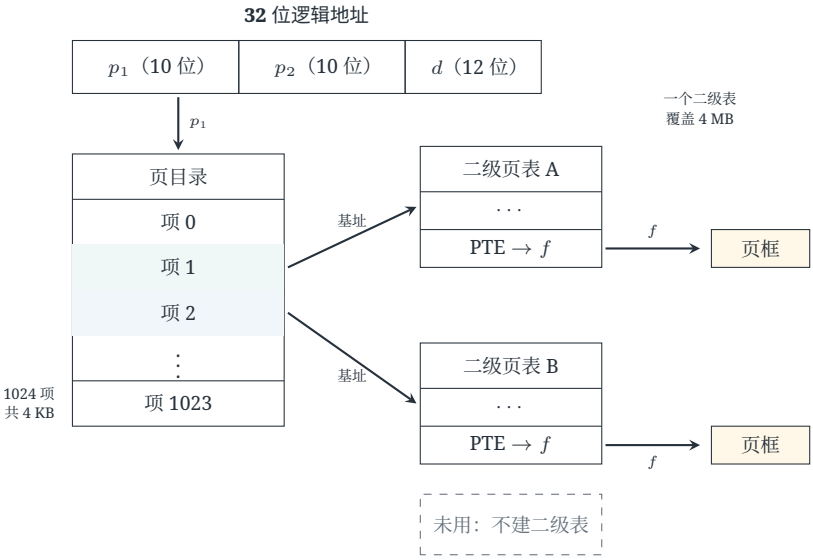
地址变换的过程可以写成：从逻辑地址取出页号 p；用 PTBR $+\,p\times$ PTE 大小 找到页表项；检查有效位和权限；取出页框号 f；最后拼成 $f\cdot 2^k+d$。如果页表有多级，中间层只负责找到下一级页表的基址，最后一级才给出真正的页框号。

页表本身也要占内存。32 位系统、4 KB 页、4 B 页表项时，页表有 2^{20} 项，共 $2^{20}\times 4\ \text{B}=4\ \text{MB}$。这还只是一个进程；如果每个进程都配一张完整的 4 MB 页表，内存很快会被页表吃光。更麻烦的是，单级页表要求这 4 MB 在物理内存中连续存放。多级页表就是为这两个问题设计的。

多级页表

把 32 位逻辑地址切成“一级页号 10 位、二级页号 10 位、页内偏移 12 位”。一级页表叫页目录，有 $2^{10}=1024$ 项，每项 4 B，正好 4 KB；每个页目录项指向一张二级页表，二级页表也有 1024 项，正好 4 KB。一个二级页表覆盖 2^{10} 个页，即 $2^{10}\times 4\ \text{KB}=4\ \text{MB}$ 的虚拟地址范围。

关键在“按需建立”：进程没有用到的 4 MB 区域，根本不需要分配对应的二级页表，页目录项的有效位保持 0 即可。假设一个进程只用了三个相距很远的 4 MB 区域，那么页目录 4 KB 加三张二级页表，共 16 KB；若用单级页表，则要 4 MB，后者约是前者的 256 倍（图中只画两个区域作示意）。



图中只画了页目录的几项。项 1 和项 2 指向两张二级页表，其他项可以是空的；空项不占二级页表的内存。查找时先由 p_1 找二级表基址，再由 p_2 找页框号，最后拼上偏移 d。

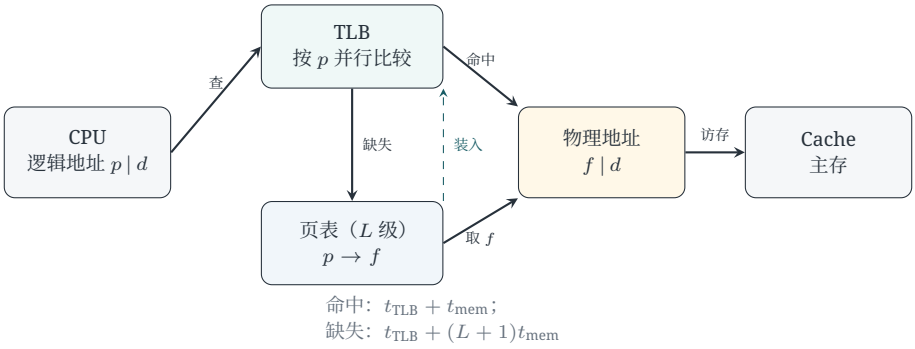
多级页表的代价是访存次数。二级页表且 TLB 缺失时，要读页目录项、读二级页表项、再读数据，共 3 次内存访问；L 级页表则是 $L+1$ 次。TLB 命中时这些访问都可以省掉，所以现代系统一边增加页表级数，一边把 TLB 做得更大、支持大页。

64 位系统的虚拟地址空间更大，常采用 4 级或 5 级页表，原理相同：每一级用若干位索引，最后一级给出页框号。408 题目以 32 位、二级页表为主，把位数拆分和访存次数算清楚即可。

5.4　TLB：把常用的页表项放到手边

页表放在内存里，每次访存都先读页表，等于把速度拖慢一倍。程序访问又具有局部性：一段时间内反复用的往往只是少数几个页。于是 MMU 里放一个小而快的 **TLB**（Translation Lookaside Buffer，快表），缓存最近用过的页表项。

TLB 通常采用全相联或组相联映射。全相联时，页号要和所有 TLB 表项并行比较；组相联时先按页号低位选组，再在组内比较。表项里除了页号（或虚拟页号）和页框号，还常带有效位、权限位和 ASID（地址空间标识）。ASID 用来区分不同进程的地址空间，避免每次进程切换都把整个 TLB 清空；如果没有 ASID，切换进程后旧表项可能把新进程的页号翻译错，必须刷新。



图里命中路径直接拿页框号，缺失路径才去读页表，读完再把表项装进 TLB。偏移 d 不参与 TLB 比较，命中后原样拼到页框号后面。若把 Cache 也算进来，地址翻译完成后才能用物理地址访问物理 Cache；408 题目通常按这个顺序处理。

设 TLB 命中率为 h，TLB 查找时间为 t_{TLB}，一次内存访问时间为 t_{mem}，页表为 L 级。命中时只需一次数据访存，缺失时要读 L 级页表再取数据：

$$\boxed{\text{EAT} = h\,(t_{\text{TLB}} + t_{\text{mem}}) + (1-h)\big(t_{\text{TLB}} + (L+1)t_{\text{mem}}\big)}.$$

若 t_{TLB} 很小可以忽略，则

$$\text{EAT} = t_{\text{mem}} + (1-h)\,L\,t_{\text{mem}}.$$

单级页表 $L=1$，退化成熟悉的 $(2-h)t_{\text{mem}}$。例如 $h=0.95$、$t_{\text{TLB}}=1$ ns、$t_{\text{mem}}=100$ ns、二级页表 $L=2$，则

$$\text{EAT} = 0.95\times 101 + 0.05\times 301 = 95.95 + 15.05 = 111 \text{ ns}.$$

命中率从 95% 再提高一点，缺失路径省下的是两次内存访问，收益很明显；这也是 TLB 必须做得又快又大的原因。TLB 能覆盖的地址范围叫 TLB reach，约等于“表项数 × 页面大小”，所以大页能显著扩大覆盖范围。

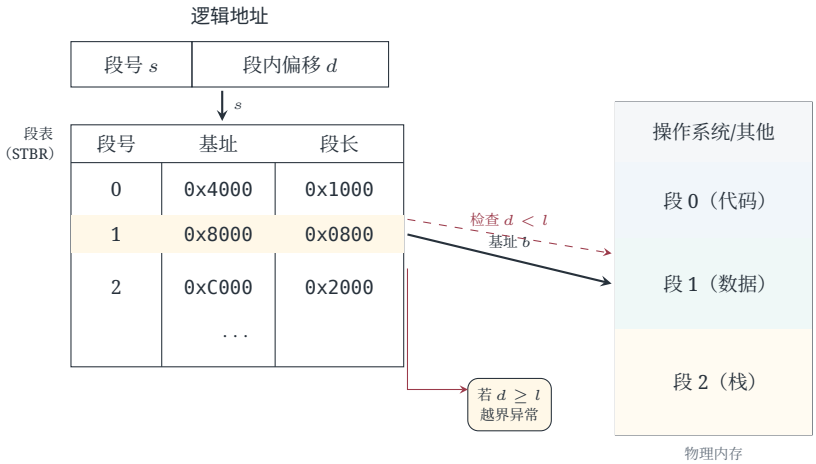
5.5 分段与段页式

分页按固定大小切分，程序员看不见页的边界；分段则按程序的逻辑结构切分。代码、数据、栈各自成段，每段长度可以不同，段号加段内偏移构成二维地址。程序员知道段的存在，链接器和编译器也按段组织内容。

每个进程有一张段表，段表项包含**段基址**、**段长**（界限）和保护位。地址变换时，先以段号 s 查段表，得到基址 b 和段长 l；然后检查偏移 $d<l$，越界就产生异常；通过后计算

$$\text{物理地址} = b + d.$$

注意这里是**相加**，不是像分页那样把页框号拼在偏移前面。分段必须显式比较 d 和 l，因为段长可变，偏移字段没有固定宽度来保证不越界。



图中的段 1 基址是 `0x8000`、段长是 `0x800`。逻辑地址 $(1, \texttt{0x300})$ 合法，物理地址是 `0x8300`；若偏移是 `0x900`，因为超过了段长，直接产生越界异常，不会去算加法。

分段的好处是共享和保护自然。两个进程可以各自在段表里指向同一个代码段，只读权限放在段表项里；分页要做到同样的事情，得让多个页表项指向同一批页框，而且一页里可能混着代码和数据，权限不好给。分段的麻烦是外部碎片：段长可变，段与段之间会留下不连续的空闲区，紧凑可以缓解，但需要动态重定位。

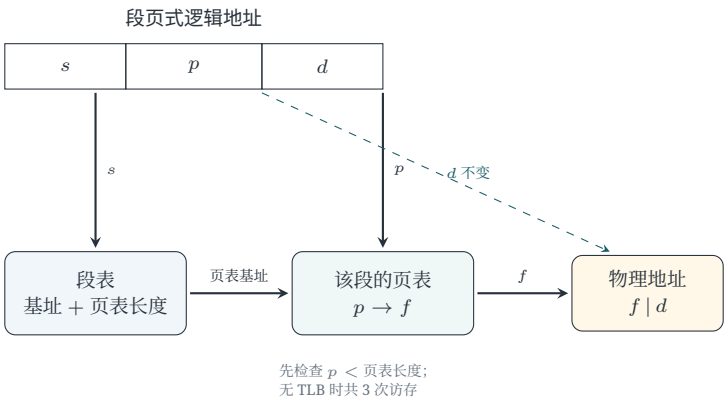
分页与分段的主要区别

维度	分页	分段
划分依据	物理单位，大小固定，由系统决定	逻辑单位，大小可变，由程序/编译器决定
地址结构	一维：页号 + 页内偏移	二维：段号 + 段内偏移
程序员可见性	透明，程序员看不到页边界	可见，程序员按逻辑模块划分
碎片	内部碎片（最后一页）	外部碎片（段间空闲区）
地址变换	页框号与偏移拼接	段基址与偏移相加，并检查 $d < l$
共享与保护	按页，粒度细但一页可能混杂内容	按段，逻辑意义清楚，便于共享和保护

段页式把两者结合起来：先按逻辑把程序分段，每段再分页；物理内存按页框分配。逻辑地址变成

$$(\text{段号}s,\ \text{页号}p,\ \text{页内偏移}d).$$

每个进程一张段表，每个段一张页表。段表项记录该段页表的基址和页表长度（该段有多少页）。变换时先查段表，检查 p 是否小于页表长度，再查页表得到页框号，最后拼上偏移。没有 TLB 时要访问段表、页表、数据三次内存；有 TLB 命中则只需一次。



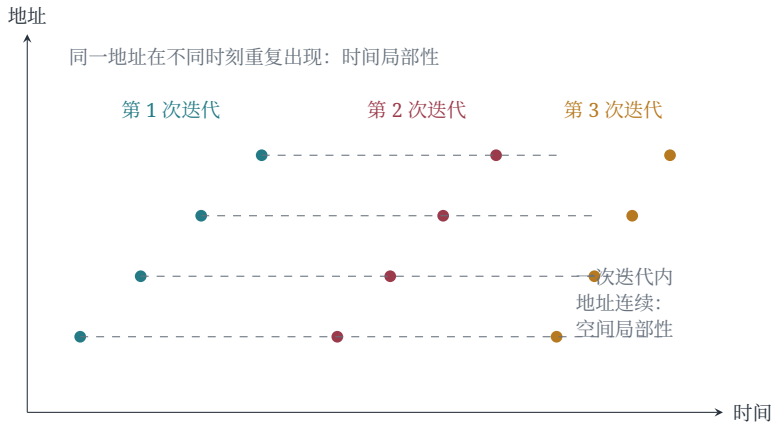
图中先由段号 s 找到该段的页表基址，检查页号 p 没有超过页表长度，再由 p 找到页框号 f，偏移 d 原样拼在最后。段页式既有分段的逻辑结构，又有分页没有外部碎片的优点；代价是地址变换更复杂，页表也更多。Linux 在 x86 上虽然保留了段寄存器，但把段基址设为 0、段限设为整个地址空间，实际上退化成以分页为主的模型；理解段页式的目的，是为了看懂硬件为什么同时提供段表和页表两套机制。

5.6 虚拟内存与请求分页

前面的分页仍然有一个前提：进程的页要全部在内存里。程序一大，或者并发进程一多，物理内存就不够。虚拟内存把这个前提去掉：进程的页只有一部分在内存，其余放在磁盘的交换区；访问到不在内存的页时，再由操作系统调入。这样每个进程都能看到一个很大的地址空间，而物理内存只装当前活跃的部分。这也带来四个常考特征：离散性、多次性、对换性和虚拟性——页面可以离散存放，作业可以分多次装入，页面可以在内存与外存之间换入换出，最终在逻辑上扩充了内存。

局部性原理

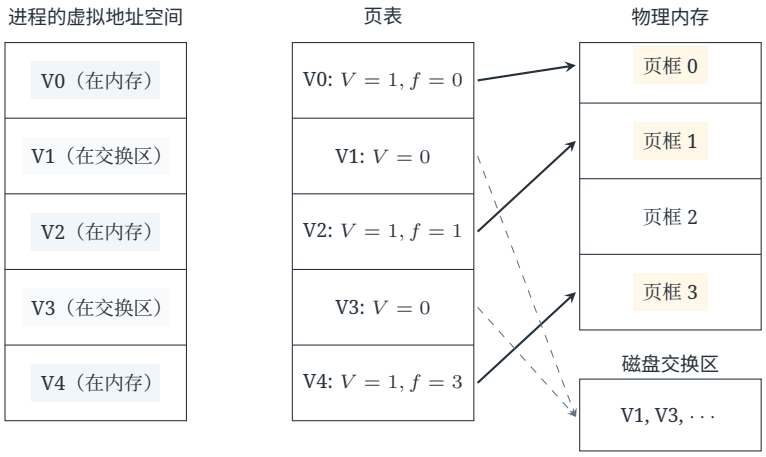
虚拟内存能工作，靠的是程序访问的局部性。时间局部性是指刚访问过的指令或数据很可能再次被访问，例如循环里的计数器和循环体；空间局部性是指刚访问过的地址附近的单元很可能被访问，例如顺序扫描数组、顺序执行指令。



图中每一轮循环都从低地址扫到高地址，所以同一条水平虚线上的点对应“同一个地址被反复访问”，这是时间局部性；每一轮内部点沿对角线上升，说明相邻地址连续被访问，这是空间局部性。局部性不是保证，只是统计规律；如果程序真的随机访问整个大数组，虚拟内存的效果会明显变差。

虚拟地址空间与交换区

每个进程有一个虚拟地址空间，页表项的有效位决定某一页当前在不在内存。有效位为 1 的页映射到某个物理页框；有效位为 0 的页可能还在磁盘交换区，也可能根本没有分配。访问前者会触发缺页，访问后者则是非法地址，处理方式不同。



进程看到的是左边连续的 V0 到 V4，实际只有 V0、V2、V4 在物理页框里，V1、V3 在磁盘上。页表是两边之间的桥：有效位为 1 的项给出页框号，有效位为 0 的项把访问引向缺页处理程序。

虚拟内存的**最大容量**由 CPU 的地址位数决定：32 位地址最多表示 2^{32} 字节，也就是 4 GB，不可能再大。**实际可用容量**还要受物理内存和交换区总量的限制，通常写成

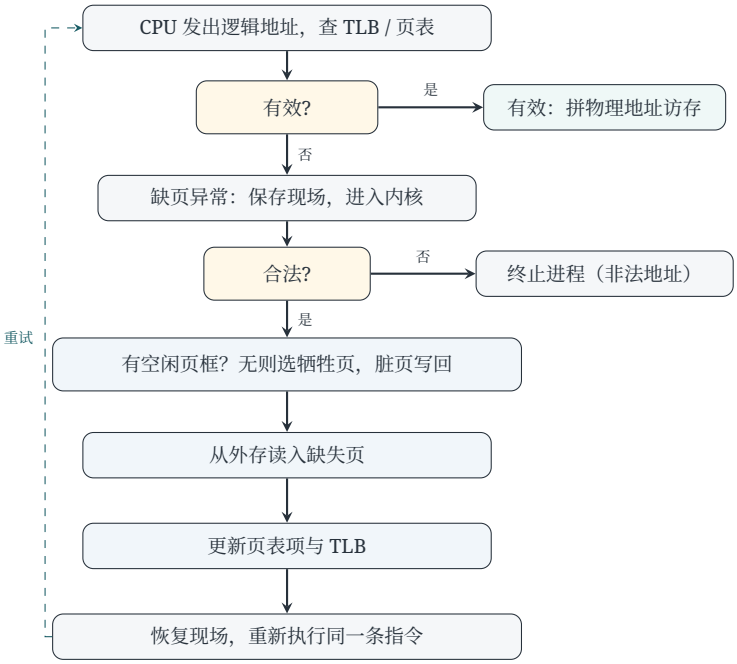
$$\min\left(2^{m},\ \text{物理内存}+\text{交换区}\right),$$

其中 m 是虚拟地址位数。考试里常把“最大容量”和“实际容量”分开问：前者只看地址结构，后者才看内存和外存。

请求分页与缺页处理

请求分页（Demand Paging）是最常见的实现：进程启动时只装入少数页，甚至一页也不装；访问到不在内存的页时，由缺页异常把页调入。也可用预调页（Prepaging），在缺页时顺便把相邻页也读进来，利用空间局部性减少后续缺页；但如果预读的页根本不用，就浪费了 I/O。

缺页处理和普通中断有几个关键区别。缺页是在指令执行过程中同步产生的异常（Fault），不是外设异步发来的中断；处理完成后要**重新执行**触发缺页的那条指令，而不是执行下一条；一条指令可能缺页多次，例如取指跨页、多个操作数分处不同的页。



流程图里最容易记错的是最后一步：缺页处理完不是“接着执行下一条”，而是回到同一条指令重新访存。另一个容易忽略的是牺牲页：如果没有空闲页框，要先按置换算法选一页；这页如果被修改过（脏位为 1），必须先写回磁盘，否则直接覆盖会丢数据。

设缺页率为 p，一次正常访存时间为 t_{mem}。若 t_{pf} 从本次访存开始计时，一直算到页面就绪、并包含重启后的那次成功访存（即把一次缺页的全过程都算进去），则

$$\boxed{\text{EAT} = (1-p)\,t_{\text{mem}} + p\,t_{\text{pf}}}.$$

若 t_{pf} 从本次访存开始计时，只算到页面就绪（包含首次访存和异常处理，但不含重启后的那次访存），则

$$\text{EAT} = t_{\text{mem}} + p\,t_{\text{pf}}.$$

两个公式看起来差 $p\,t_{\text{mem}}$，原因是两种口径下的 t_{pf} 本身就相差一次访存；做题时不要把同一个数套进两个公式。由于磁盘 I/O 比内存访问慢几个数量级，p 稍有上升，EAT 就会明显变差。例如 $t_{\text{mem}} = 100$ ns、按第二种口径取 $t_{\text{pf}} = 8$ ms、$p = 0.001\% = 10^{-5}$，则

$$\text{EAT} = 100 + 10^{-5} \times 8 \times 10^{6} = 100 + 80 = 180 \text{ ns},$$

几乎翻了一倍。这也是页面置换算法值得反复优化的原因。

5.7 页面置换算法

物理页框有限，缺页时如果没有空闲页框，就要从内存中选一页换出。选谁，直接影响缺页率。描述这类问题通常给一个**页面访问串**（引用串）和页框数，按顺序模拟每次访问，统计缺页次数。

OPT：知道未来时的下界

最优置换算法（OPT，Optimal）选择“未来最长时间内不再被访问”的页换出；如果某页以后不再用，就优先淘汰它。OPT 的缺页次数是所有算法中最少的，但需要预知未来，无法实现，只作为理论下界，用来衡量其他算法离最优有多远。

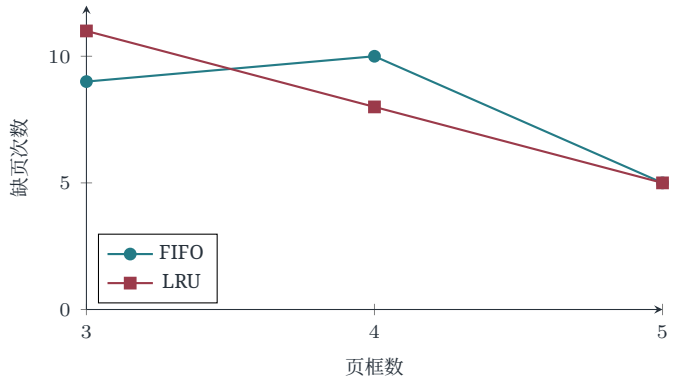
FIFO 与 Belady 异常

先进先出（FIFO）把驻留页排成一个队列，淘汰最早进入内存的页。实现只需要一个循环队列，非常简单；问题是它完全不看页面最近有没有被访问，可能把正在频繁使用的页换走。

FIFO 还有一个反直觉的现象：**增加页框数，缺页次数反而可能增加**，称为 Belady 异常。原因是增加页框改变了淘汰次序，可能把即将访问的页提前换出。下面这组数据来自访问串

$$2, 4, 1, 3, 2, 3, 4, 5, 2, 4, 1, 3, 5,$$

用 FIFO 模拟：



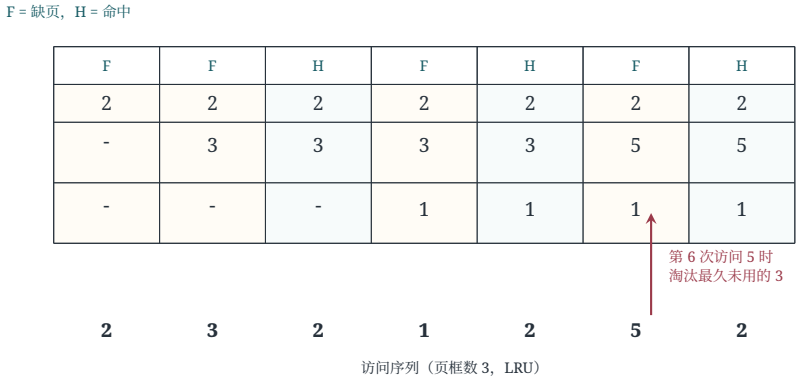
FIFO 从 3 个页框增加到 4 个，缺页次数从 9 次变成 10 次，确实出现了 Belady 异常；LRU 则是 $11 \to 8 \to 5$，页框越多缺页越少。LRU 和 OPT 属于**栈类算法**：n 个页框时的驻留集始终包含于 $n+1$ 个页框时的驻留集，所以缺页数不会随页框增加而上升。FIFO 不具备这个性质；基本 CLOCK 也不是栈类算法，理论上同样不能保证单调。

LRU：用过去预测未来

最近最久未使用（LRU，Least Recently Used）选择“最长时间没有被访问”的页换出。它基于局部性：过去很久不用的页，未来短时间内的可能性也较低。实现方式有两种：

- **时间戳法**：每个页表项配一个计数器，页面每被访问一次就把当前时钟值写进去；置换时选时间戳最小的页。
- **栈法**：维护一个按访问时间排序的栈，每次访问（包括命中）都把该页移到栈顶；栈底就是最久未使用的页。

LRU 的关键是**命中也要更新次序**。FIFO 只在页面进入内存时排队，命中不改变队列；LRU 只要访问就刷新“最近使用时间”，这是两者最容易被混淆的地方。



这张图把每次访问后的三个页框内容列出来。前两次是缺页装入，第 3 次访问 2 命中，但 LRU 会把 2 的“最近使用时间”刷新；第 6 次访问 5 时，3 已经是最久未用的页，于是被换出。命中列用浅色标出，可以看到 LRU 的命中也会改变后续的淘汰顺序。

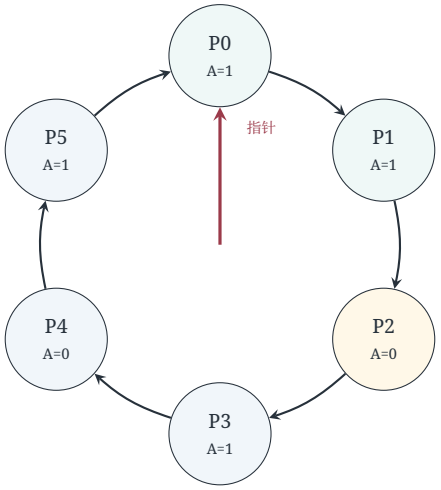
LRU 的性能接近 OPT，但精确实现代价高：每次访存都要更新时间戳或调整栈，纯硬件做起来很贵。实际系统常用 CLOCK 来近似。

CLOCK 与改进型 CLOCK

CLOCK 算法把驻留页组织成环形队列，每个页表项有一个访问位 A，页面被访问时由硬件置 1。发生置换时，指针从当前位置开始扫描：

1. 若当前页的 $A = 1$，说明它最近被访问过，给它“第二次机会”：把 A 清 0，指针前移；
2. 若当前页的 $A = 0$，说明它最近没被访问，选它换出，指针停在它的下一个位置。

最坏情况下指针会扫过一整圈，把所有 A 都清 0；第二轮一定能找到 $A = 0$ 的页。CLOCK 不需要记录精确的访问时间，只需要一个访问位和一个指针，开销比 LRU 小得多。



图中指针指向 P0。P0 和 P1 的访问位都是 1，先把它们清 0 并跳过；到 P2 时 $A = 0$，P2 被换出。等下一次扫描再经过 P0、P1 时，它们的 A 已经是 0，不会无限给机会。

基本的 CLOCK 只看访问位，换出时可能选中一个脏页，还要写回磁盘。**改进型 CLOCK**（Enhanced Second-Chance）把访问位 A 和修改位 M 组合起来，优先淘汰“最近没访问、也没修改”的页：

改进型 CLOCK 的优先顺序

优先级	A	M	含义
1（最先淘汰）	0	0	最近没访问，也没修改，可直接丢弃
2	0	1	最近没访问，但修改过，需要写回
3	1	0	最近访问过，未修改
4（最后考虑）	1	1	最近访问过且修改过，最不该淘汰

扫描规则要记清：第一轮找 $(0,0)$，不修改任何标志位；找不到时第二轮找 $(0,1)$，并把扫过的页的 A 清 0；如果还没找到，此时所有 A 都已是 0，再从头找 $(0,0)$ 或 $(0,1)$ 即可。第一轮不改 A，是为了不破坏“最近访问过”的信息；第二轮清 A，是为了让下一轮能继续推进。

把几种算法放在一起比较：

页面置换算法对比

算法	淘汰依据	Belady 异常	特点
OPT	未来最久不用	不会	理论最优，不可实现
FIFO	最早进入	可能	实现最简单，可能淘汰活跃页
LRU	最久未访问	不会	性能接近 OPT，精确实现开销大
CLOCK	访问位 $A=0$	理论可能	用环形扫描近似 LRU，开销小
改进 CLOCK	(A,M) 组合	理论可能	优先淘汰干净页，减少写回

5.8 页面分配、工作集与抖动

页面置换只回答“缺页时换谁”，还有一个前置问题：每个进程应该分到多少页框？分得太少，进程频繁缺页；分得太多，其他进程又不够用。

常见的分配策略有三种：

页面分配与置换范围的组合

策略	含义与特点
固定分配 + 局部置换	每个进程分到固定数量的页框，缺页时只从自己的页里换。隔离性好，但页框数难以适应程序阶段变化
可变分配 + 全局置换	不固定每个进程的页框数，缺页时可从所有进程的页中选牺牲页。灵活，但一个进程可能抢走另一个进程正在用的页
可变分配 + 局部置换	缺页时先从自己这里换，操作系统定期根据缺页情况调整页框数。兼顾灵活性和隔离性，实际系统常用

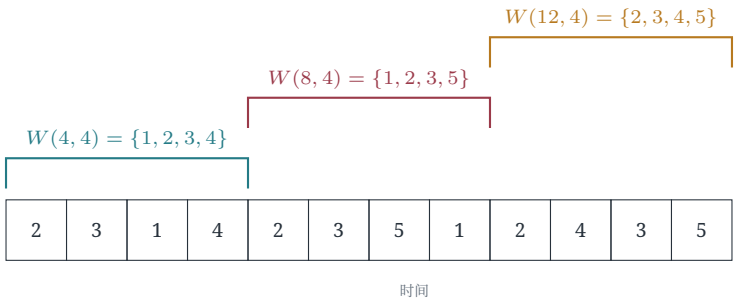
“固定分配 + 全局置换”通常不单独列出：既然允许从全局抢页框，分给每个进程的数量就不可能真正固定。调入页面的时机则有请求调页和预调页两种，前者只在缺页时调入，后者顺便多读几页。每个进程分到的页框数还有一个下限：至少要能同时容纳一条指令及其操作数所在的页面，具体由指令集决定，跨页取指或多个跨页操作数会抬高这个下限；上限则是该进程的页面总数。

工作集

工作集（Working Set）描述进程在最近一段时间内实际用到的页面集合：

$$W(t,\Delta)=\{\text{进程在时间区间}(t-\Delta,\,t]\text{ 内访问过的页面}\}.$$

Δ 是窗口大小。窗口越大，集合越稳定，但可能把已经不再使用的页也算进来；窗口越小，越能反映当前阶段，但波动更大。

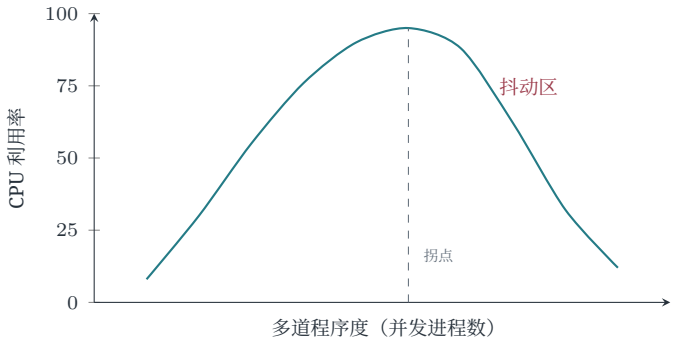


图中窗口 $\Delta=4$ 随时间滑动。时刻 1–4 内访问过 2、3、1、4，工作集是 $\{1,2,3,4\}$；滑到时刻 5–8 后变成 $\{1,2,3,5\}$；再滑到时刻 9–12 又变成 $\{2,3,4,5\}$。工作集变化说明进程在不同阶段需要的页不同，操作系统不能一次性把页框数定死。

抖动

如果分配给进程的页框数小于它当前工作集的大小，进程刚换入一页，很快又要换出另一页，缺页接连不断。此时进程大部分时间在等磁盘，真正执行指令的时间很少，CPU 利用率反而下降，这种现象叫**抖动**（Thrashing）。

抖动的典型表现是：缺页率很高、磁盘忙于换入换出、CPU 利用率低。此时**增加并发进程数会加重抖动**，因为每个进程分到的页框更少；正确的做法是减少并发度，或者按工作集模型、缺页频率（PFF）策略给进程增配页框。



曲线是示意：并发度较低时，增加进程能让 CPU 更忙；超过某个点后，页框被摊薄，缺页换入换出占满磁盘，CPU 利用率急剧下降。实际系统的拐点取决于工作集大小、页框总数和磁盘速度，不是一个固定数字。

防止抖动可以从两头入手：一是给每个进程足够的页框，至少覆盖当前工作集；二是控制并发度，当所有进程的工作集之和超过物理页框总数时，挂起部分进程。PFF 策略则直接看缺页率：高于上界就增配页框，低于下界就适当减少，把缺页率拉回可接受范围。

5.9 做内存管理题的顺序

地址转换题和置换题看起来公式很多，其实可以按固定顺序拆：

1. **先看页面大小**，把它写成 2^k，确定页内偏移占 k 位；逻辑地址总位数减去 k 就是页号位数。
2. **再看页表级数**。单级页表缺失时访存 2 次，二级 3 次，L 级 $L+1$ 次；TLB 命中则只算 1 次数据访存。
3. **把地址拆成各级索引和偏移**，按“页目录 $\rightarrow$ 二级页表 $\rightarrow$ 页框”的顺序查，别忘了检查有效位和权限。
4. **置换题先画帧表**，逐次访问、逐次更新；FIFO 只在装入时排队，LRU 命中也要刷新次序，CLOCK 看访问位和指针。
5. **最后核对单位**：页面大小是字节还是 KB，地址是十六进制还是十进制，缺页率是百分数还是小数。单位错一步，后面全错。

拓展阅读：

- OSTEP 第 18–22 章“分页、TLB、交换”，看地址变换和缺页处理的完整硬件流程。
- 陈海波《现代操作系统：原理与实现》第 5 章“内存管理”，看多级页表、大页和 ARM64 地址变换。

6 习题

6.1 基础过关题

1. （单项选择题）关于分页与分段存储管理，下列说法正确的是

   (A) 页长由系统固定且对用户透明；段长可变，访问时需检查段内偏移是否小于段长

   (B) 分段管理中段在内存中连续存放，因此不会产生外部碎片

   (C) 分页的逻辑地址是一维的，因此寻址时不需要页号，只需页内偏移

   (D) 段页式管理中，一次数据访问最多需要两次访存（查段表和页表）

2. （单项选择题）某分页系统采用二级页表，TLB 采用全相联映射。一次 TLB 查找耗时 2 ns，一次内存访问耗时 100 ns；TLB 缺失时需依次访问两级页表，再访问目标数据。忽略缺页与 Cache 的影响。若 TLB 命中率为 96%，则平均每次访存的有效访问时间为

   (A) 102 ns (B) 106 ns

   (C) 110 ns (D) 202 ns

3. （填空题）某 32 位系统按字节编址，采用一级页表，页面大小为 8 KiB，页表项大小为 4 B。则页内偏移占 ____ 位，页号占 ____ 位，每个进程的页表占用 ____ 内存。

4. （单项选择题）关于缺页异常的处理，下列说法正确的是

   (A) 缺页处理完成后，CPU 从触发缺页指令的下一条指令继续执行

   (B) 一条指令的执行过程中至多发生一次缺页

   (C) 缺页处理可能需要把被淘汰的脏页写回外存

   (D) 缺页由外部设备异步发出，属于外部中断

5.（单项选择题）关于抖动（thrashing）与工作集，下列说法正确的是

(A) 抖动由 CPU 利用率过低引起，增加并发进程数即可缓解

(B) 若所有进程工作集大小之和超过可用页框总数，系统可能因频繁换页而抖动

(C) 工作集窗口 Δ 越大，工作集中的页面数一定越少

(D) 采用全局置换可以从根本上消除抖动

6.（计算题）某分段系统按字节编址，段内偏移从 0 开始，段表如下：

段号	段长（B）	基址
0	1024	2000
1	500	6000
2	2048	9000

分别判断下列逻辑地址是否合法；合法的给出物理地址，非法的说明原因：(1) $(0, 800)$；(2) $(1, 600)$；(3) $(2, 1500)$。

6.2 真题风格与综合训练

7.（单项选择题）某 32 位系统按字节编址，页面大小为 4 KiB，页表项大小为 4 B，采用二级页表，逻辑地址按“一级页号 10 位、二级页号 10 位、页内偏移 12 位”划分。某进程只使用了两个虚页，其虚页号分别为 `0x00001` 和 `0x3FFFF`，这两个虚页的一级页号不同，且每个用到的一级页号下只建立一张二级页表。该进程的页目录与全部二级页表共占用多少内存？

(A) 4 KiB　(B) 8 KiB

(C) 12 KiB　(D) 4 MiB

8.（单项选择题）某系统采用改进型 CLOCK 算法，4 个页框按顺时针顺序存放页面 P_0, P_1, P_2, P_3，其（访问位 A，修改位 M）依次为 $(1,1)$、$(0,1)$、$(1,0)$、$(0,1)$，时钟指针当前指向 P_0。按“第一轮寻找 $(0,0)$ 且不修改标志位，第二轮寻找 $(0,1)$ 并把扫过的 A 清 0”的规则，下一个被换出的页面是

(A) P_0　(B) P_1

(C) P_2　(D) P_3

9. （真题风格・综合题）某请求分页系统采用固定分配、局部置换，进程的页面访问序列为

$$1,2,3,4,2,1,5,6,2,1,2,3,7,6,3,2,1,2,3,6,$$

共 20 次访问。规定：初始页框全空；缺页次数按“被访问页不在内存”计数，首次装入也算缺页；命中不改变 FIFO 队列的次序。

(1) 分配 3 个页框，分别用 FIFO、LRU、OPT 求缺页次数与缺页率。

(2) 将页框数增加到 4，求 FIFO 与 LRU 的缺页次数；与 (1) 比较，该序列是否出现 Belady 异常？

(3) 另取访问序列 $1,2,3,4,1,2,5,1,2,3,4,5$，用 FIFO 分别计算 3 个和 4 个页框时的缺页次数，并解释 Belady 异常的成因。

10. （真题风格・综合题）某 32 位系统按字节编址，页面大小 4 KiB，采用二级页表，逻辑地址按“一级页号 10 位、二级页号 10 位、页内偏移 12 位”划分，页表项与页目录项均为 4 B。页目录基址寄存器内容为 `0x00100000`。进程 P 的映射如下：

- 一级页号 1 的页目录项指向二级页表 `0x00200000`；
- 在 `0x00200000` 这张二级页表中，二级页号 3 对应页框号 `0x2A`，二级页号 5 对应页框号 `0x37`；
- 一级页号 2 的页目录项指向的二级页表中，二级页号 3 对应页框号 `0x11`。

(1) 访问逻辑地址 `0x00403ABC`，求一级页号、二级页号与页内偏移。

(2) 求读取该地址对应二级页表项时所用的物理地址，以及最终访问数据的物理地址。

(3) TLB 初始为空，依次访问 `0x00403ABC`、`0x00403AB8`、`0x00405ABC`、`0x00803ABC`，写出每次的 TLB 命中情况；并给出第一次访问在 TLB 缺失、页表命中时的访存次数。

(4) 若第 (1) 问地址的页表项有效位为 0，而该页合法且已分配在外存，说明缺页处理的主要步骤。再说明：若只给该进程 1 个页框，而它循环访问 `0x00403ABC` 与 `0x00405ABC`，会出现什么现象，应如何缓解。

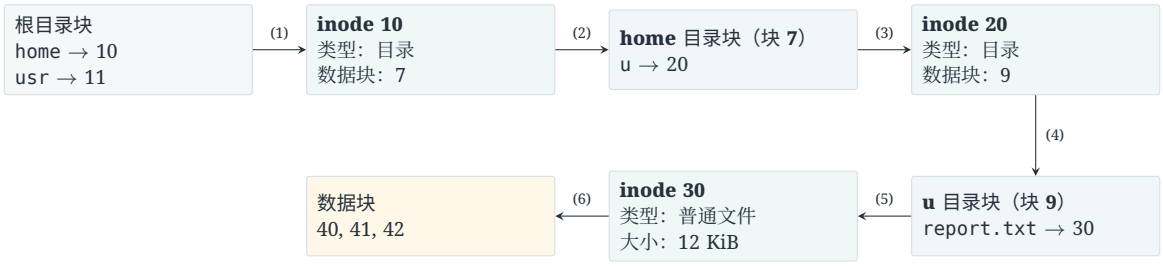
7 文件管理

在终端里敲下 `cat /home/u/report.txt`，内核实际做的事可以压缩成一句话：把路径逐段翻译成 inode 号，再顺着 inode 里的块指针把数据读出来。用户看到的是名字，磁盘上存的是块号，中间隔着目录、inode、空闲空间管理和磁盘调度几层。这一章就沿着这条链路走一遍。

7.1 路径名、目录项与 inode

文件系统对外只暴露名字。用户给出 `/home/u/report.txt`，内核不能直接拿这个名字去磁盘上找数据，因为磁盘不认识字符串。它需要一张“名字到文件实体”的表，这张表就是目录。

目录本身也是一个文件，只不过它的数据块里存放的不是用户数据，而是一项挨着一项的**目录项**（directory entry）。在 UNIX 类文件系统中，一个目录项通常只有两项核心内容：文件名和 inode 号。inode 号是文件实体的编号，真正的属性和数据块位置都放在 inode 里。老式文件系统（如 FAT）则把文件名、属性、起始簇号、大小等全部塞进目录项，这样的目录项就是一个完整的 FCB（文件控制块）。



图里 (1)(3)(5) 是查目录项，(2)(4) 是取 inode 中记录的数据块号，(6) 是沿 inode 的块指针读数据。每一段路径分量都要经历“读目录块找名字、读 inode 取下一跳”这两步。注意目录项中并没有文件大小、权限和数据块号，这些信息只在 inode 里；把文件名与 inode 分开，才可能让多个目录项指向同一个 inode，也就是后面要说的硬链接。实际系统还会缓存最近用过的目录项（dentry cache），否则每次打开文件都要从根目录一路读盘。

目录结构

最早的文件系统只有**单级目录**：整个磁盘一张目录表，所有文件平铺在里面。实现简单，但文件名不能重复，文件一多查找就退化成线性扫描，也无法让不同用户拥有各自的同名文件。

两级目录把目录分成主文件目录（MFD）和用户文件目录（UFD）：MFD 记录用户名和该用户目录的位置，UFD 存放这个用户的文件。不同用户可以有同名文件，隔离性也更好，但用户仍然只能把自己的文件平铺在一起，不能按项目再分层。

树形目录是现在最常见的结构。目录可以包含子目录，整个文件系统是一棵树，于是有了绝对路径和相对路径：绝对路径从根目录开始，相对路径从当前工作目录（current working directory）开始。树形结构不会出现环，删除目录时可以安全地递归回收；代价是共享一个文件不那么自然，因为一个文件在树中只能有一个父目录。

无环图目录允许一个文件出现在多个父目录下。UNIX 用硬链接实现文件共享：多个目录项指向同一个 inode，文件本身只有一份。硬链接不能跨文件系统，因为 inode 号只在本文件系统内唯一；它通常也不允许指向目录，因为那会制造环，让路径解析和引用计数陷入死循环。软链接则是另一种独立文件：它有自己的 inode，数据块里存目标路径，因此可以跨文件系统，也可以指向目录；目标删除后软链接仍然存在，但解析会失败，成为悬空链接。为了让共享安全，inode 中要维护链接计数：每增加一个目录项加一，删除一个目录项减一，减到零且没有进程打开时才真正释放数据块。

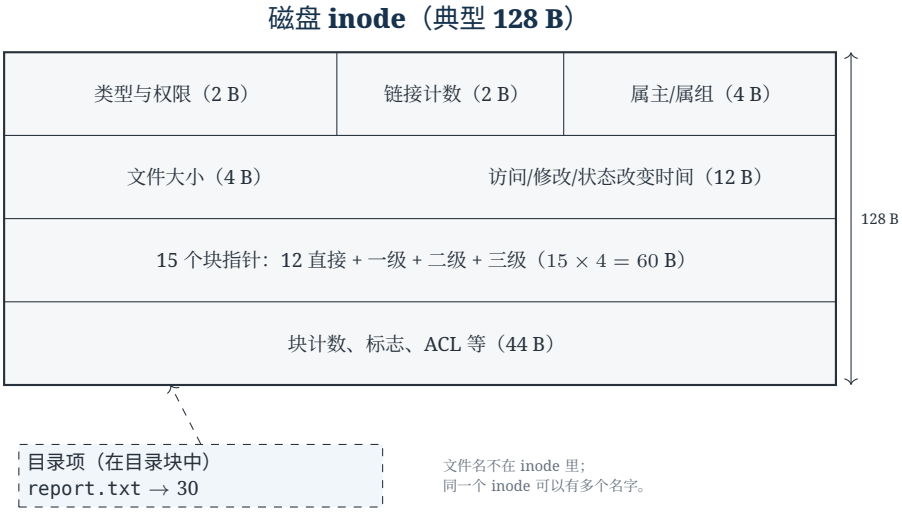
> **注意**
>
> **FCB 与 inode 的分工。**FCB 是“文件控制块”的统称，传统实现里它包含文件名和全部属性。UNIX 把文件名拿出来放进目录项，把其余属性放进 inode。这样目录项很小，目录块能装下更多项；更重要的是，文件名可以换、可以有两个名字，而文件实体只有一个 inode。

7.2 inode 与文件系统的磁盘布局

inode（索引节点）是定长的记录，集中放在磁盘的 inode 表里。给一个 inode 号（从 1 开始编号），内核用“表起始地址 + (编号 -1) $\times$ 记录长度”就能算出它的位置，不需要沿链查找。一个典型的 128 字节 inode 包含文件类型与权限、链接计数、属主和属组、文件大小、三个时间戳，以及 15 个块指针；文件名不在其中。



图中从左到右是文件系统在一个分区里的典型布局。超级块是总入口；两个位图分别记录 inode 和数据块谁被占用；inode 表是定长记录的数组；数据区里既有普通文件的数据块，也有目录的数据块。inode 号只在同一个文件系统内有意义，这也是硬链接不能跨文件系统的原因。



图上半部分是 inode 的字段划分，下半部分用虚线说明“目录项在 inode 之外”。链接计数记录有多少个目录项指向这个 inode；文件大小和块指针告诉内核数据在哪里。时间戳的具体个数和含义随系统而异，408 只需要知道 inode 保存的是“除文件名以外的文件属性”。

打开文件时，内核把 inode 从磁盘读入内存，再在系统打开文件表中登记偏移量、打开模式等信息，进程的文件描述符表指向打开文件表项。多个进程可以共享同一个内存 inode，但各自可以有独立的读写位置。

7.3 数据块怎么放：连续、链接与索引

inode 里的块指针最终要回答一个问题：文件的第 i 个逻辑块，落在磁盘的哪个物理块上？按回答方式的不同，文件物理结构分为连续分配、链接分配和索引分配三大类。



三幅图共用同一条编号的盘块行，方便对比同一组数据块在不同结构下的组织方式。连续分配把“块号”变成算术：已知起点和长度，第 i 块就是起点加 i，随机访问最快；但文件一旦变长，后面若被别的文件占住，就只能整体搬家，多次分配回收后还会留下外部碎片。链接分配把块分散开，每块尾部用指针串成链，没有外部碎片，文件也能随时长出新块；代价是随机访问要沿着链走，访问第 i 块一般需要 $i+1$ 次读盘，而且指针坏了会断链。索引分配把指针集中到一个索引块里，目录项只记索引块号，要读第 i 块时先读索引块、再读数据块，随机访问只需要两次读盘。

显式链接（FAT）是链接分配的一种变体：它把分散在各数据块尾部的指针集中到一张文件分配表（File Allocation Table）里，表项的下标是盘块号，表项的值是下一块号。目录项仍然只记起始块号，但沿着链走变成查内存中的表，不再每走一步读一次盘。

FAT 表（每个盘块一项）

块号	0	1	2	3	4	5	6	7	8	9
表项	空闲	空闲	7	5	9	8	空闲	4	结束	结束

文件 A 的目录项记起始块 2：



图里文件 A 的链是 $2 \to 7 \to 4 \to 9$，文件 B 的链是 $3 \to 5 \to 8$。目录项只保存起始块号，剩下全由 FAT 表项接力。FAT 若常驻内存，走链只是内存查表，随机访问的代价从“多次读盘”降为“多次访存”；但 FAT 本身要占内存，表项位数还限制了单个分区的最大块数——n 位表项最多表示 2^n 个盘块，这也是 FAT12、FAT16、FAT32 名字的由来。

三种物理结构的对比

结构	目录项/入口	随机访问	主要代价
连续分配	起始块号、长度	直接计算，最快	外部碎片；文件增长可能整体搬移
隐式链接	第一块块号	沿链逐块读，最慢	指针占块内空间；断链会丢失后续数据
显式链接（FAT）	第一块块号	查 FAT，内存命中时较快	FAT 占内存；表项位数限制分区大小
索引分配	索引块块号	先读索引块，再读数据块	索引块本身占空间；大文件要多级索引

混合索引与最大文件长度

单级索引的索引块只能装 $q = B/P$ 个块号（B 是盘块大小，P 是块号占用的字节数），文件最大也就 q 个数据块。要支持大文件，可以给索引块再建索引，形成多级索引；UNIX 的做法是把直接指针和不同级别的间接指针混在 inode 里，称为**混合索引**。

设 inode 有 n_0 个直接指针、1 个一级间接指针、1 个二级间接指针和 1 个三级间接指针，且 $q = B/P$，则该 inode 能寻址的数据块总数为

$$N_{\max} = n_0 + q + q^2 + q^3,$$

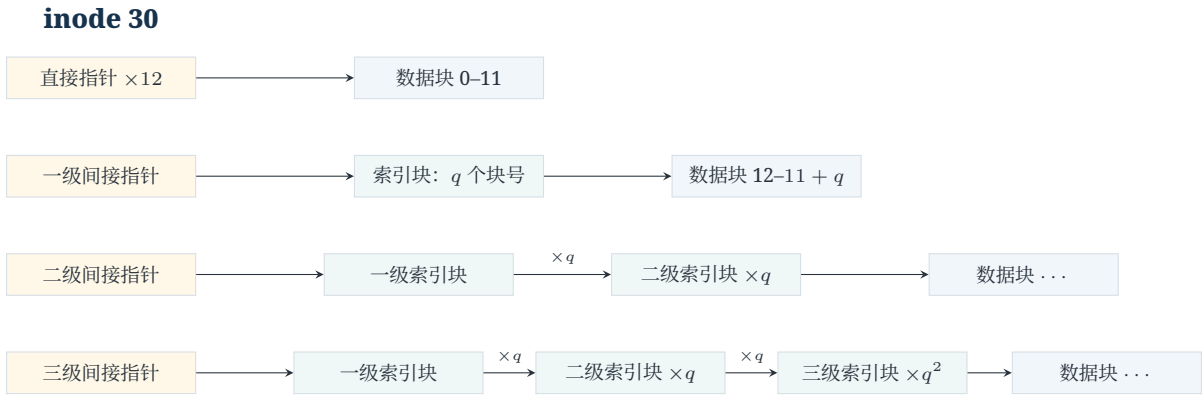
最大文件长度为

$$L_{\max} = N_{\max} B.$$

其中 q 个块由一级间接覆盖，q^2 个块由二级间接覆盖，q^3 个块由三级间接覆盖。以 $B = 4$ KiB、$P = 4$ B、$n_0 = 12$ 为例，$q = 1024$：

$$L_{\max} = (12 + 1024 + 1024^2 + 1024^3) \times 4 \text{ KiB} = 48 \text{ KiB} + 4 \text{ MiB} + 4 \text{ GiB} + 4 \text{ TiB}.$$

最后一项占绝对主导，所以常说这种配置支持约 4 TiB 的文件。若某本教材采用 10 个直接指针，只需把第一项换成 40 KiB，后面三项不变。



直接指针覆盖小文件；级数越深，能覆盖的块越多，但访问一个数据块要读的索引块也越多

图里每一层向右都乘一个 q。给定逻辑块号 x，先判断它落在哪一段：$0 \le x < n_0$ 走直接指针；接下来 q 个块走一级间接；再接下来 q^2 个块走二级间接；最后 q^3 个块走三级间接。判断出层级后，沿路径把相应的索引块读进来，最后读数据块。若 inode 已在内存、各级索引块都不在

内存，访问二级间接区的一个块需要先读二级索引块、再读一级索引块、最后读数据块，共三次磁盘访问；三级间接区则要四次。

7.4　空闲空间管理

文件被删除、被截短，或者磁盘刚格式化时，都会出现大量空闲盘块。文件系统必须记住哪些块能用，并且尽量让分配和回收都快。常见做法有四类。

空闲表法把连续的空闲区记成“起始块号、长度”的表项，分配时用首次适应、最佳适应等算法找一段够长的空闲区。它适合连续分配，回收时还能把相邻空闲区合并；缺点是空闲区一多，表就会变长，查找也变慢。

空闲链表法把空闲盘块或空闲盘区串成链表。分配时从链头摘一块，回收时挂回链头。它不需要额外的表，但每走一步都可能读盘，空闲块分散时效率不高。

位示图法为每个盘块分配一位：1 表示已分配，0 表示空闲（也可以反过来，做题时以题面约定为准）。位示图非常紧凑，而且能直接按块号定位。设字长为 w，盘块号、字号、位号都从 0 开始，则盘块号 b 与字号 i、位号 j 的关系是

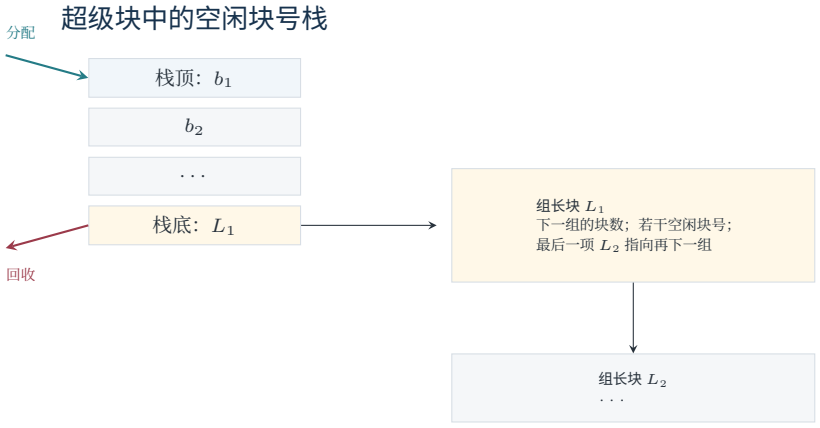
$$i = \left\lfloor \frac{b}{w} \right\rfloor, \qquad j = b \bmod w, \qquad b = i\,w + j.$$

N 个盘块需要 N 位，即 $N/8$ 字节。分配时从当前字号开始找第一个 0 位，回收时把对应位清 0。



图里每一行是一个字，格子里是位值。给定盘块号 b，用除法和取余就能找到它落在哪一位；反过来，分配时扫到第一个 0 位也能立刻算出盘块号。位示图的优点是空间小、定位快，缺点是整张图可能很大，不过按 408 的算法通常只要求会换算，不要求考虑它是否常驻内存。

成组链接法是 UNIX 采用的折中方案。它把空闲块按固定上限 N（经典实现中 $N = 100$）分成组，超级块里维护一个空闲块号栈，存放当前组的空闲块号；栈底那一项不是普通块号，而是下一组“组长块”的块号。组长块本身也是一个空闲块，它的数据区里记录下一组的块数以及最多 N 个块号，其中最后一项又指向再下一组的组长块。



栈空时，把 L_1 中记录的下一组信息读回栈，L_1 本身也作为空闲块分配出去；栈满再回收时，把整个栈写进刚回收的块，该块成为新的栈底。

图中 L_1 既在栈里，又是下一组组长块的块号；真正读它的时候，它已经要被分配出去了。分配走“弹栈”，回收走“压栈”，只有在栈空或栈满时才做一次磁盘读写，所以成组链接把空闲表/链表的磁盘操作集中成了批处理，既省内存又减少 I/O。

7.5　磁盘调度

磁盘的物理地址由柱面号、磁头号、扇区号三部分组成。读一个块要先把磁头移到目标柱面（寻道），再等目标扇区转到磁头下（旋转延迟），最后把数据传出来（传输时间）。平均访问时间可以写成

$$T_a = T_s + \frac{1}{2r} + \frac{b}{rN},$$

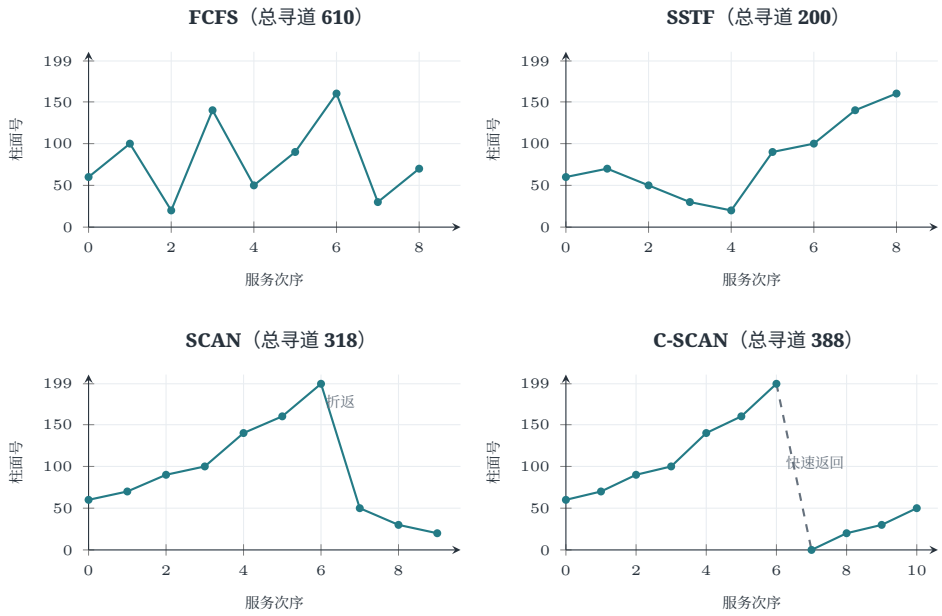
其中 T_s 是平均寻道时间，r 是转速（转/秒），N 是一条磁道的字节数，b 是要传输的字节数。$1/(2r)$ 是平均旋转延迟：目标扇区随机落在圆周上，平均要等半圈。若题目给的是每分钟转速 R，则平均旋转延迟为 $30/R$ 秒，即 $30000/R$ 毫秒。

三项中寻道时间通常最大，所以磁盘调度算法主要优化寻道顺序。请求队列是动态到达的，调度器要在“当前磁头位置”和“尚未服务的请求”之间做选择。

FCFS 按请求到达顺序服务，最公平也最简单。请求分散时，磁头会在盘面两端来回长距离移动，总寻道距离很大。

SSTF（最短寻道时间优先）每次选离当前磁头最近的请求。它的总寻道距离通常比 FCFS 小很多，但只顾眼前：如果新请求不断在磁头附近到达，远端请求可能长期得不到服务，发生饥饿。

SCAN（电梯算法）让磁头沿一个方向一路服务，到达磁盘端点后再折返。它不会饥饿，但刚被磁头越过的请求要等磁头走到端点、再折返回来，等待时间较长。**C-SCAN** 只在一个方向服务，到达端点后快速返回起点，返回途中不服务请求；这样各柱面的等待时间更均匀，代价是返回段白跑一趟。**LOOK** 和 **C-LOOK** 是它们的实用版本：磁头到最远的一个请求就折返，不必走到物理端点。



四幅图的纵轴是柱面号，横轴是服务次序，折线每往下或往上走一段，磁头就寻道一次；总寻道距离就是各段竖直线段长度之和（C-SCAN 的虚线返回段也计入）。这里用磁头初始在 60 号柱面、请求为 $100, 20, 140, 50, 90, 160, 30, 70$ 的一组数据做示例：FCFS 按到达顺序走，折线来回穿插；SSTF 每次挑最近的，折线局部很密、总距离最短；SCAN 先向大号柱面扫到端点 199 再折返；C-SCAN 到 199 后直接回到 0，再从 0 向大号方向服务。注意图中请求与习题中的请求不是同一组，不要把这里的总距离当成某道题的答案。

磁盘调度算法对比

算法	选择规则	优点	缺点/注意
FCFS	按到达顺序	公平、实现简单	寻道距离大
SSTF	每次选最近的请求	平均寻道距离小	可能饥饿；不一定全局最优
SCAN	单向服务到端点后折返	不会饥饿，像电梯	刚越过的请求等待久；端点附近不公平
C-SCAN	只在一个方向服务，到端点快速返回	等待时间更均匀	返回段空走，寻道距离可能更大
LOOK/C-LOOK	到最远请求就折返	比 SCAN 少走空路，实际常用	要区分“端点”和“最远请求”的题目约定

做题时先明确三件事：磁头初始位置、初始移动方向、以及“到端点折返”还是“到最远请求折返”。总寻道距离只与相邻两次服务位置的差有关，第一次从初始位置算起；平均寻道距离再除以请求个数。C-SCAN 的返回段是否计入，题目通常会说明，若没有说明就按“磁头实际移动的距离”计入。

7.6 把这一章串起来

文件管理考点与常见陷阱

问题	关键结论	容易错的地方
路径怎么变成盘块号	目录项存“名字 $\rightarrow$ inode 号”，inode 存属性和块指针	目录项不存文件大小、权限；硬链接共享 inode，软链接是独立文件
inode 最大文件	$q = B/P$，块数 $n_0 + q + q^2 + q^3$，再乘 B	直接指针个数按题目给；间接层数看逻辑块号落在哪个区间
三种分配方式	连续随机访问快但有外部碎片；链接无碎片但随机慢；索引用索引块换随机访问	隐式链接访问第 i 块要 $i+1$ 次读盘；FAT 是显式链接
位示图	$b = i\,w + j$，$i = \lfloor b/w \rfloor$，$j = b \bmod w$	字号、位号从 0 开始；位示图字节数为 $N/8$
成组链接	栈空读组长块，栈满把栈写进新回收块	组长块本身也是空闲块，最终会被分配
磁盘调度	总寻道 $= \sum$ |相邻位置差|，从初始位置算起	SSTF 会饥饿；SCAN 到端点，LOOK 到最远请求；C-SCAN 返回段是否计入看题面

这一章的主线其实只有一条：名字要经过目录找到 inode，inode 要通过块指针找到数据块，而块指针的组织方式、空闲块的记录方式、磁盘响应的顺序，决定了这条链路有多快、能支持多大的文件、以及多个请求下是否公平。把这条主线记住，零散的定义和公式就不容易混。

8 习题

8.1 基础过关题

1. （单项选择题）在 UNIX 类文件系统中，目录项（directory entry）与 i-node 的分工是

(A) 目录项保存文件名和文件的全部属性，i-node 只保存数据块号

(B) 目录项保存文件名和 i-node 号，文件类型、长度、物理块号等属性保存在 i-node 中

(C) 一个 i-node 只能被一个目录项引用，所以 i-node 中的链接计数恒为 1

(D) 删除文件时只要删掉目录项，其 i-node 和数据块就立即被回收

2. （单项选择题）关于文件的物理结构（分配方式），下列说法正确的是

(A) 连续分配支持随机访问，但文件增长时若后继盘块已被占用，可能要整体搬移

(B) 隐式链接分配中，访问文件的第 i 个逻辑块只需读一次盘

(C) 索引分配中，索引块里存放的是文件的数据内容

(D) 连续分配在多次分配与回收之后不会产生外部碎片

3. （填空题）某系统用位示图管理磁盘空闲空间：磁盘共有 2^{26} 个盘块，位示图每个字 64 位，约定“1 表示已分配、0 表示空闲”，盘块号、字号、位号均从 0 开始编号，位示图第 i 个字的第 j 位（自低位向高位计）对应盘块号 $b = i \times 64 + j$。（1）位示图共占用 _____ 字节；（2）盘块号 2 000 003 对应的字号 $i =$ _____，位号 $j =$ _____。

4. （单项选择题）关于磁盘调度算法 SCAN 与 C-SCAN，下列说法正确的是

(A) SCAN 对靠近磁盘两端的柱面比对中间柱面更公平

(B) C-SCAN 只在一个方向服务请求，返程快速移回起始端且不服务，因此各柱面的等待时间更均匀

(C) 对同一请求序列，C-SCAN 的总寻道长度一定小于 SCAN

(D) LOOK 与 SCAN 的唯一区别是 LOOK 必须移到磁盘最外端才折返

5. （填空题）某文件系统盘块大小为 4 KiB，每个盘块号占 4 字节，按 $1\ \text{KiB} = 2^{10}$ 字节计。(1) 只采用一级索引时，一个索引块能描述的最大文件长度为 _____；(2) 若改用 FAT16（FAT 表项 16 位，按 2^{16} 个表项计）管理同样 4 KiB 的盘块，则一个分区最大为 _____。

6. （单项选择题）某文件 `/home/u/a.txt` 被用户创建了两个硬链接和一个软链接，下列说法正确的是

(A) 删除其中一个硬链接后，文件数据立即被释放　(B) 软链接可以跨文件系统，硬链接不可以

(C) 硬链接可以指向目录，软链接不可以指向目录　(D) 每建立一个硬链接，系统就为文件新建一个 i-node

8.2 真题风格与综合训练

7. （单项选择题）某文件系统的 i-node 含 10 个直接块指针、1 个一级间接块指针、1 个二级间接块指针和 1 个三级间接块指针。盘块大小 4 KiB，每个盘块号占 4 字节。某文件已被打开（其 i-node 已在内存），现要读取该文件内偏移量 5 MiB 处的一个字节，且相关的间接块与数据块均不在内存。该字节所在区域及为此需要的磁盘访问次数是

(A) 一级间接区，2 次　(B) 二级间接区，2 次

(C) 二级间接区，3 次　(D) 三级间接区，4 次

8. （单项选择题）某系统用位示图管理磁盘空闲空间，字长 32 位，“1 表示已分配、0 表示空闲”，第 i 个字的第 j 位（j 自低位向高位计）对应盘块号 $i \times 32 + j$。分配时按字号递增、同一字内位号递增的顺序找第一个 0 位。已知所有字号小于 100 的字均已没有 0 位，而字号 100 的那个字的值为 `0xFFFF0000`，则本次分配的盘块号是

(A) 3200 (B) 3215

(C) 3216 (D) 3231

9. （真题风格・综合应用题）某 UNIX 类文件系统的 i-node 含 12 个直接块指针、1 个一级间接块指针、1 个二级间接块指针和 1 个三级间接块指针。盘块大小为 1 KiB，每个盘块号占 4 字节，一个指针恰好指向一个盘块。（1）该文件系统单个文件的最大长度是多少字节？用 GiB/MiB/KiB 的形式和字节数各表示一次。（2）文件已打开、i-node 在内存，要读取文件内偏移量 20 MiB 处的一个字节，该字节落在哪一级间接区？若相关的间接块与数据块都不在内存，需要几次磁盘访问？该字节对应的块号在二级间接块中处于什么位置？（3）若同一磁盘改用显式链接（FAT）管理，盘块仍为 1 KiB，FAT 表项 16 位，则最大分区容量和 FAT 表本身的大小各是多少？

10. （真题风格・综合应用题）某磁盘有 200 个柱面，编号 $0 \sim 199$，磁头当前位于 53 号柱面，并正朝柱面号增大的方向移动。柱面请求队列（按到达顺序）为

$$98,\ 183,\ 37,\ 122,\ 14,\ 124,\ 65,\ 67.$$

忽略旋转延迟与传输时间，只统计寻道距离。约定：SCAN 沿当前方向一直服务到物理端点 199 后才折返；C-SCAN 只在一个方向服务，到达 199 后立即移回 0 号柱面（这段返回移动计入寻道距离），再从 0 开始只向增大方向服务。（1）按 FCFS 服务，写出服务顺序并求总寻道距离与平均寻道距离。（2）按 SSTF 服务，写出服务顺序并求总寻道距离与平均寻道距离。（3）按 SCAN 和 C-SCAN 服务，分别写出服务顺序并求总寻道距离与平均寻道距离。（4）就本题数据说明四种算法在总寻道距离和公平性上的差别。

拓展阅读：

- OSTEP 第 39–40 章——文件系统实现与 inode 详解。
- 陈海波《现代操作系统：原理与实现》第 7 章——Ext4 与 F2FS 文件系统。

9 I/O 管理

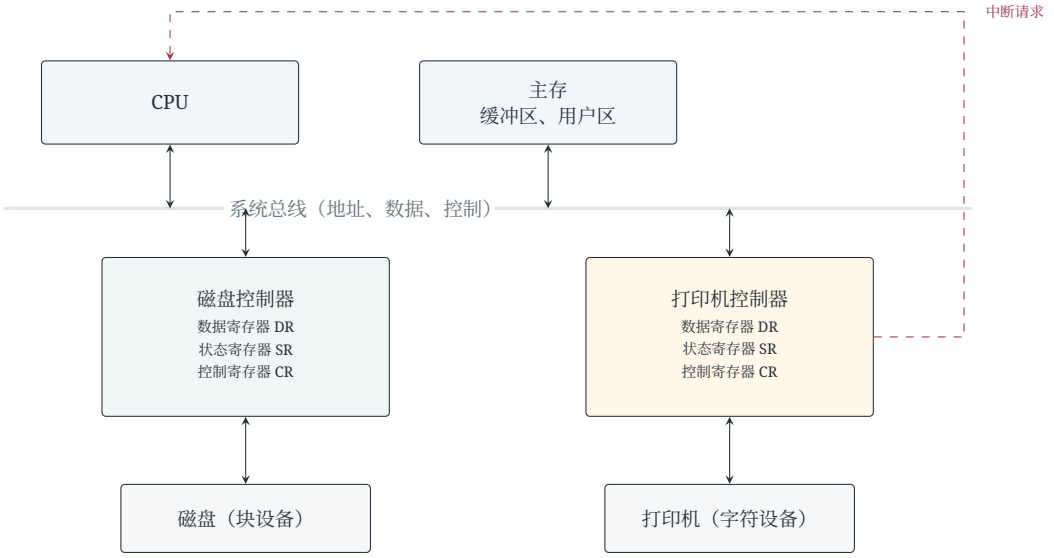
程序调用 `read(fd, buf, 4096)` 之后，CPU 并不会直接去拨动磁盘的磁头。它把请求交给文件系统，文件系统找到对应的设备，驱动程序再把“读第 n 块”翻译成一组寄存器操作。磁盘控制器收到命令，驱动电机、定位磁道，把数据读进自己的缓冲区，最后向 CPU 发一个中断。整条路径上，软件只负责发请求和收结果，真正的数据搬运由控制器、DMA 或通道完成。

本章沿着这条路径展开：先看设备和控制器长什么样，再看 CPU 用哪几种方式参与 I/O，然后自底向上看 I/O 软件的层次。后半部分讨论两类资源管理问题：内存中的缓冲区怎样安排，磁盘上的 SPOOLing 怎样把独占设备变成共享设备，以及多进程申请设备时怎样分配才不会死锁。

9.1 设备、控制器与总线

一台设备通常由两部分组成：机械或物理部分，以及插在总线上的控制器。磁盘的盘片、磁头是机械部分；磁盘控制器负责寻道、读写扇区、校验数据并发出中断。打印机也一样，控制器接收字符编码，控制打印头出字。CPU 看到的是控制器上的寄存器，而不是设备的机械结构。

控制器一般有三类寄存器。数据寄存器暂存要传输的数据；状态寄存器给出“忙”“就绪”“出错”等标志；控制寄存器接收 CPU 发出的命令，例如启动读、启动写、复位。CPU 可以用专门的 I/O 指令访问这些寄存器，例如 x86 的 `in`、`out`；也可以把寄存器编入物理地址空间，用普通的访存指令访问，后者称为内存映射 I/O。两种方式对上层软件的影响不同，但控制器内部的行为一样。



图中，CPU 和主存挂在总线上，控制器也挂在总线上。CPU 写控制寄存器发起命令，读状态寄存器判断是否完成，数据则通过数据寄存器或 DMA 通道在主存与设备之间流动。控制器完成一次操作后，通过中断线通知 CPU。

设备可以从几个不同角度分类，这些角度彼此独立。按数据单位分，块设备以固定大小的块传输，可以按块寻址，磁盘和光盘属于这一类；字符设备以字节流传输，通常不支持按块随机寻址，键盘、鼠标、打印机属于这一类。按共享属性分，独占设备一次只能给一个进程使用，打印机是典型例子；共享设备可以被多个进程交替访问，磁盘是典型例子；虚拟设备则是用软件把独占设备改造成逻辑上的共享设备，SPOOLing 就属于这一类。按速度分，还可以分成低速、中速和高速设备。

“块设备”和“独占设备”说的不是同一件事。磁盘是块设备，同时又是共享设备；打印机是字符设备，同时又是独占设备。考试里常把这两个维度混在一起，判断时先问清楚题目在按哪个维度分类。

9.2 I/O 控制方式

假设要把磁盘上的一块数据读进内存。CPU 可以用四种方式参与这件事，区别在于 CPU 花多少时间、数据由谁搬运、每传多少数据中断一次。

程序直接控制

CPU 向控制器发出读命令，然后反复读取状态寄存器。状态寄存器显示“忙”时，CPU 继续读；显示“就绪”时，CPU 把数据寄存器里的内容搬到内存。搬完一个字，再启动下一个字的传输。

这种方式叫程序直接控制，也叫轮询。它的缺点是 CPU 在等待期间只能空转，不能执行其他进程的有用工作。设备越慢，浪费越大。优点是实现简单，不需要中断硬件；在设备响应很快、或者系统很小的时候，轮询反而没有中断开销。

中断驱动

CPU 发出读命令后，把当前进程挂起，切换到其他进程。设备把一个字或一个字节准备好，向 CPU 发中断。CPU 响应中断，保存现场，进入中断处理程序，把数据寄存器里的内容搬到内存。如果还有数据，驱动程序再次启动设备，然后返回被中断的进程。

中断驱动把“等待”从 CPU 的忙等变成了设备的异步通知。CPU 在设备工作期间可以运行别

的进程。代价是每传一个字或一个字节就要中断一次。中断要保存现场、识别中断源、执行中断服务程序、恢复现场，开销不小。键盘、鼠标、串口这类低速字符设备适合这种方式；高速磁盘如果每字节都中断，CPU 会把大量时间花在中断处理上。

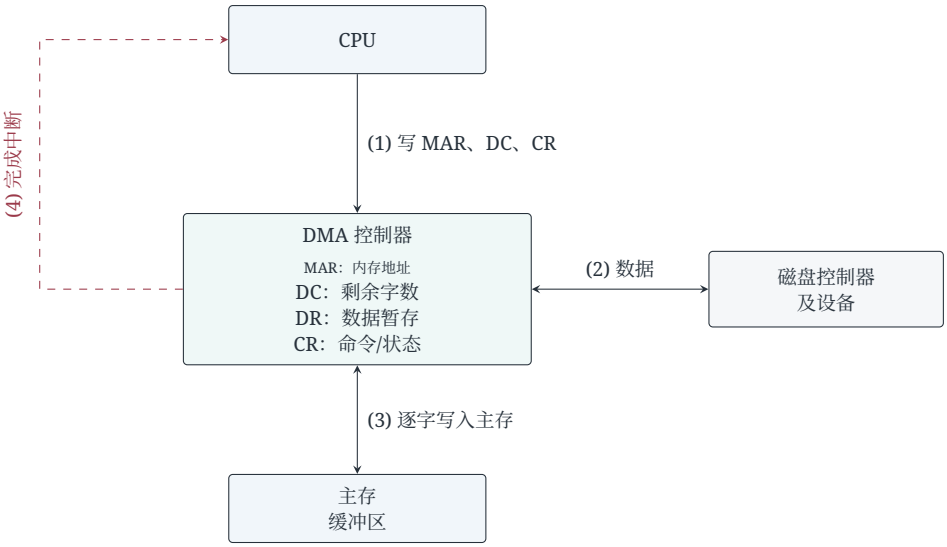
中断处理程序只做最必要的事：读状态、搬数据、唤醒等待的进程。耗时的工作可以推迟到后半部执行，避免长时间关中断。

DMA

DMA 方式在设备和主存之间加一个 DMA 控制器。CPU 不再逐字搬运数据，只负责初始化。以读磁盘为例，CPU 把内存起始地址写入 DMA 的地址寄存器 MAR，把要传的字数写入计数寄存器 DC，把“读”命令和方向写入命令寄存器 CR，然后启动 DMA。DMA 控制器向 CPU 申请总线，获得总线控制权后，控制设备把数据逐字写入主存。每传一个字，MAR 按传送单位递增，DC 减 1。DC 减到 0 时，DMA 控制器向 CPU 发一次中断，表示整块数据传完。

DMA 的关键变化有两个。数据传送不再经过 CPU，CPU 只参与开始和结束；中断频率从“每个字一次”降到“每块一次”。块越大，节省的 CPU 时间越多。

DMA 传送期间要占用总线。如果 CPU 也要访存，两者会争用内存周期。DMA 每传一个字，挪用一次总线周期，传完就把总线还给 CPU，这种方式叫周期挪用。另一种做法是一次占住总线，把整块数据传完再释放，叫成组传送。成组传送对 CPU 的影响更大，但传输效率更高。



图中四条路径对应一次 DMA 读操作。CPU 只做 (1) 和 (4)；(2)(3) 是 DMA 控制器在拿到总线后自己完成的。真实系统里，DMA 描述符可能不止一组，也可以支持分散聚集，把多个不连续的物理页拼成一次传输。

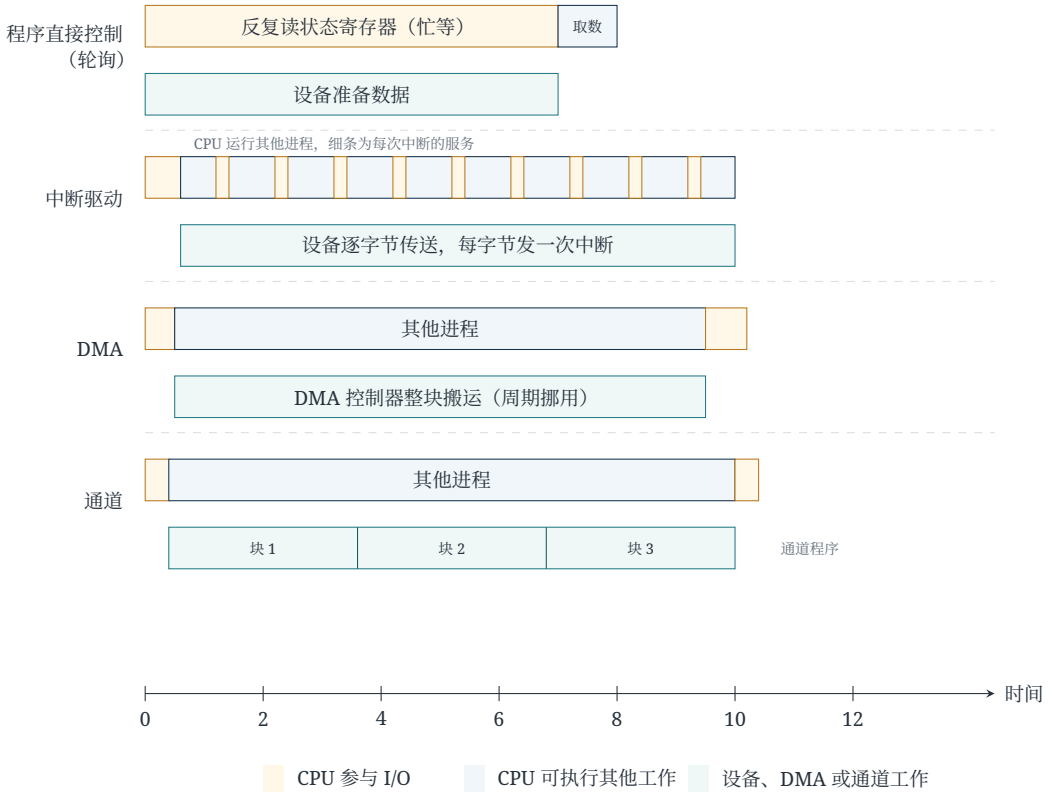
通道

通道比 DMA 更进一步。它是一个专门处理 I/O 的处理器，有自己的指令系统，能执行存放在主存里的通道程序。CPU 只要执行一条“启动 I/O”指令，把通道程序的地址告诉通道，通道就会自己取指令、控制多台设备、按程序里的条件跳转和循环，全部完成后才向 CPU 发一次中断。

DMA 只能做“把一块数据从设备搬到内存”这种固定动作，没有分支和循环能力。通道能执行程序，所以可以把“读多个磁道”“从多台设备收集数据”这样的任务写成通道程序，一次交给通道。CPU 的参与程度从“每块一次”降到“每个通道程序一次”。

通道有三种常见类型。字节多路通道按字节轮流为多台低速设备服务，适合终端、打印机这类设备。数组选择通道按块为高速设备服务，一次只服务一台设备，设备独占通道。数组多路通

道按块轮流为多台高速设备服务，兼顾了选择通道的效率和多路通道的并行性。



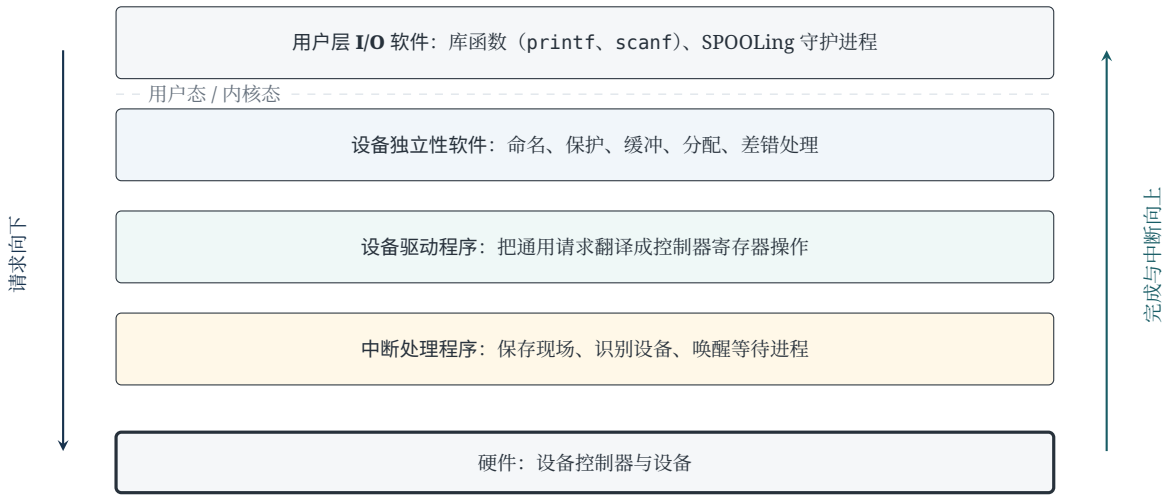
图中每一行是一条时间线。轮询方式下，CPU 在设备准备期间一直读状态寄存器；中断方式下，CPU 可以运行其他进程，但每传一个字节就被打断一次，细条表示中断服务；DMA 只在开始和整块结束时占用 CPU；通道把一组块交给通道程序，CPU 在整批数据传输期间都可以做别的事。越往下，CPU 被 I/O 打断的次数越少。

四种 I/O 控制方式

方式	传送单位	中断频率	CPU 的工作
程序直接控制	字/字节	不中断，靠轮询	反复读状态，设备就绪后搬数据
中断驱动	字/字节	每个单位一次	响应中断，在中断服务程序里搬数据
DMA	块	每块一次	初始化 DMA，处理整块完成中断
通道	一组块	每个通道程序一次	启动通道，处理通道结束中断

9.3 I/O 软件层次

用户程序调用 `printf` 时，并不知道数据最终写到磁盘还是屏幕。这个调用会经过库函数、系统调用、设备独立性软件、设备驱动程序，最后变成控制器寄存器上的一次写操作。I/O 软件通常分成四层，每一层只和相邻层打交道。



图中上方是用户态，下方是内核态和硬件。请求从用户层往下传，每一层只处理自己负责的那部分；设备完成操作后，中断从硬件往上走，最终唤醒等待的进程。

中断处理程序在最底层。设备发中断后，硬件保存程序计数器和程序状态字，根据中断向量跳到处理程序入口。处理程序先保存寄存器，再读控制器的状态寄存器，判断是哪台设备、出了什么错。如果是中断驱动的读操作，它把数据寄存器里的内容搬到内存；如果是 DMA，它只做完成后的收尾。最后唤醒等待该 I/O 的进程，恢复现场并返回。中断处理程序应该短，耗时的收尾工作可以放到后半部。

设备驱动程序在中断处理程序之上。它知道某种设备的具体细节：控制器有哪些寄存器、命令字怎么写、状态位怎么解释、出错后怎么重试。上层说“读第 n 块”，驱动程序把它变成“写柱面号、磁头号、扇区号，启动读命令”。驱动程序通常运行在内核态，设备制造商按操作系统的驱动接口编写。不同设备有不同驱动，同一类设备可以共用一个驱动。

设备独立性软件处理所有设备共有的问题。它把逻辑设备名映射到物理设备，提供统一的 `open`、`read`、`write`、`ioctl` 接口，管理缓冲区，分配设备，处理一般的 I/O 错误，向上层屏蔽不同设备的块大小和扇区大小。逻辑设备表 LUT 就放在这一层：进程用“打印机”这样的逻辑名申请设备，LUT 把它换成物理设备名和驱动程序入口。这样更换设备时，用户程序不用改。

用户层 I/O 软件包括库函数，SPOOLing 守护进程通常也以用户进程形式运行。`printf` 把格式化后的字符串交给 `write` 系统调用；打印守护进程在后台把输出井里的文件送到打印机。用户层软件运行在用户态，通过系统调用进入内核。

分层的好处是每一层只依赖下一层的接口。换一块网卡，只需要换驱动，文件系统和用户程序不受影响；换一种文件系统，缓冲区管理和驱动也不用改。代价是请求要穿过几层，函数调用和上下文切换有开销。

9.4 缓冲区管理

键盘每秒可能只产生几十个字节，CPU 每秒能执行几十亿条指令。如果 CPU 每读一个字节就等一次键盘，大部分时间都浪费在等待上。缓冲区的作用是在两者之间放一块内存：设备先把数据攒起来，CPU 一次取走一批；或者 CPU 先把输出攒起来，设备慢慢取。这样两边可以并行工作，中断次数也能减少。

缓冲区通常分配在内核空间，属于操作系统的内存，不是磁盘。这一点要和 SPOOLing 的“井”区分开：井在磁盘上，缓冲区在内存里。

单缓冲与双缓冲

先看输入方向。设备把一块数据读入缓冲区，耗时记为 T；操作系统把缓冲区里的数据搬到用户工作区，耗时记为 M；CPU 处理这块数据，耗时记为 C。在单缓冲下，设备、缓冲区、用户区只有一条通路，设备和 CPU 不能同时使用同一个缓冲区。

设备把缓冲区填满后，CPU 开始搬移。搬移完成，缓冲区空出来，设备才能开始读下一块。搬移之后的处理 C 可以和下一块的读入 T 重叠，但搬移 M 本身要占着缓冲区，不能和下一块的 T 重叠。因此，第一块要完整经历 $T+M+C$；此后每增加一块，所需时间是 $\max(C,T)+M$。处理 N 块的总时间为

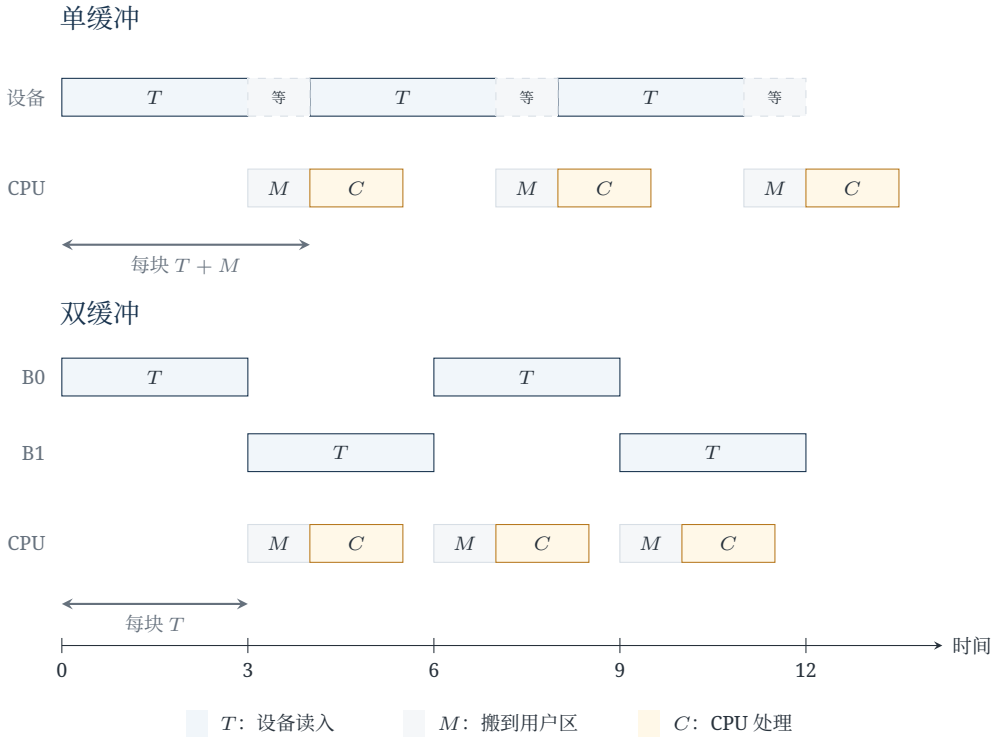
$$T_{\text{单}} = T + M + C + (N-1)\big[\max(C,T) + M\big].$$

双缓冲设置两个缓冲区。设备往缓冲区 0 填数据时，CPU 可以从缓冲区 1 取数据；缓冲区 0 填满后，设备转去填缓冲区 1，CPU 同时处理缓冲区 0。这样设备路径每块花 T，CPU 路径每块花 $M+C$，两条路径并行，稳态下每块的时间是两者中较慢的那个：

$$\max(T,\ M+C).$$

第一块仍然要经历 $T+M+C$，所以 N 块的总时间为

$$T_{\text{双}} = T + M + C + (N-1)\max(T,\ M+C).$$



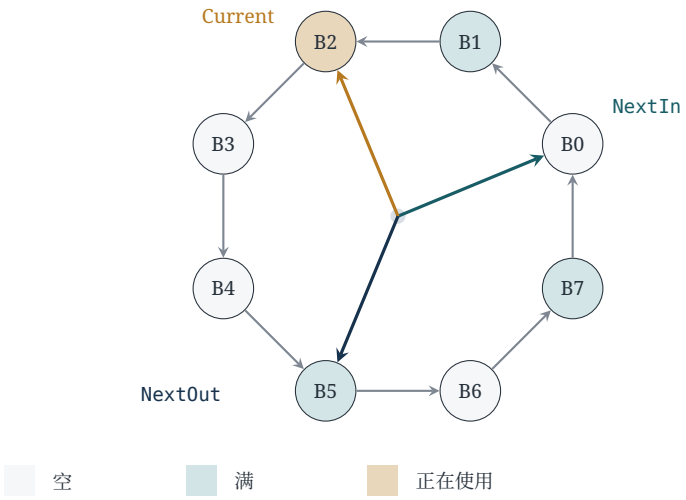
上半图是单缓冲。设备读完一块后，要等 CPU 把数据搬走（M），才能读下一块，所以设备在 M 期间空闲。下半图是双缓冲。设备填 B0 时，CPU 可以处理 B1；设备填 B1 时，CPU 处理 B0。设备几乎一直在工作，稳态每块只花 T。这张图画的是 $T > M+C$ 的情形，此时设备是瓶颈；如果 $M+C > T$，瓶颈变成 CPU 路径，每块花 $M+C$。

输出方向的分析完全对称。CPU 先把结果写进用户区，再搬到系统缓冲区（M），设备从缓冲区取走输出（T）。单缓冲下，设备取走之前缓冲区不能重用，所以每块仍是 $\max(C,T)+M$；双缓冲下，CPU 准备下一块和设备输出上一块可以并行，每块仍是 $\max(T, M+C)$。公式里的 T 换成输出时间，M 换成“用户区到系统缓冲区”的搬移时间即可。

有一个常见的错误：双缓冲的并行路径写成 $\max(T, C)$，漏掉 M。只有题目明确说 M 可以忽略，或者 M 已经包含在 C 里，才能把 M 当成 0。否则 CPU 路径是“搬移加处理”，即 $M + C$。

循环缓冲与缓冲池

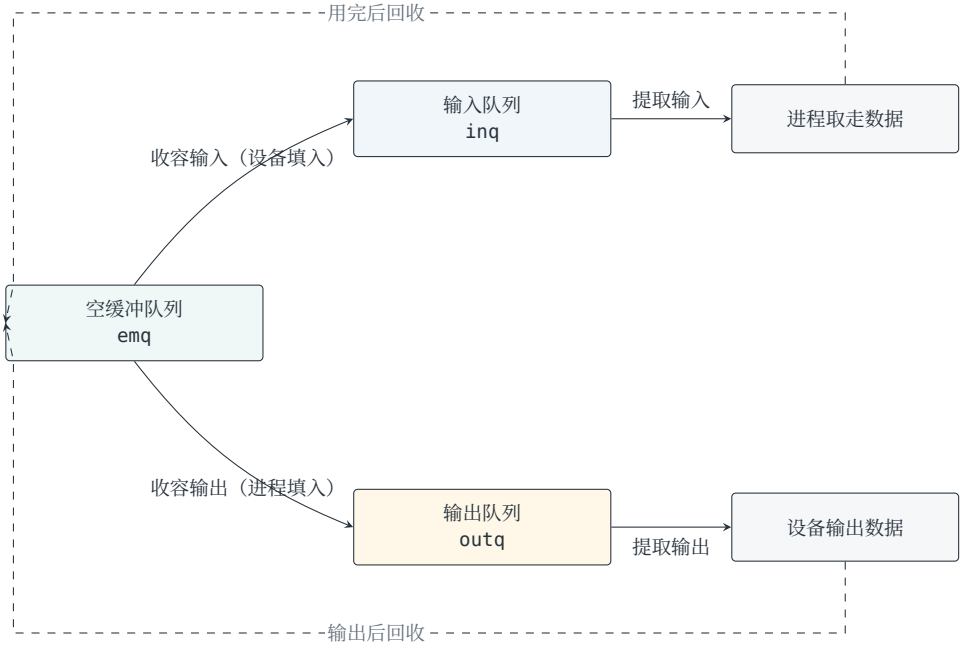
双缓冲只有两个缓冲区，遇到设备忽快忽慢、进程处理一阵停一阵的情况，两个缓冲区不够用。循环缓冲把多个缓冲区组织成一个环，每个缓冲区有一个状态：空、满、正在使用。设备按 `NextIn` 指针找下一个空缓冲区填入，CPU 按 `NextOut` 指针找下一个满缓冲区取出。填满后 `NextIn` 后移，取空后 `NextOut` 后移。环上的缓冲区首尾相接，指针走到末尾就回到开头。



图中浅灰表示空，浅青表示满，浅黄表示正在使用。`NextIn` 指向下一个空缓冲区，设备往这里填；`NextOut` 指向下一个满缓冲区，CPU 从这里取；Current 是 CPU 正在处理的缓冲区。环上的箭头表示缓冲区按固定顺序轮转。设备是生产者，CPU 是消费者，缓冲区的状态就是生产者与消费者之间的同步信息。环满时设备等待，环空时 CPU 等待。

循环缓冲能吸收突发：设备一次送来好几块，CPU 可以慢慢处理；CPU 一阵忙不过来，设备也有地方存。稳态吞吐率仍由较慢的一边决定，缓冲区数量不会让慢的一边变快，它只是把“等”的位置从设备或 CPU 挪到了队列上。

缓冲池是系统级的一组缓冲区，不再固定分给某台设备。池中的缓冲区按状态挂在三个队列上：空缓冲队列 `emq`、输入队列 `inq`、输出队列 `outq`。设备输入时，缓冲管理程序从 `emq` 取一个空缓冲区，设备把数据填进去，填满后挂到 `inq`；进程从 `inq` 取走数据，缓冲区变空，挂回 `emq`。输出方向对称：进程从 `emq` 取空缓冲区，填入输出数据后挂到 `outq`；设备从 `outq` 取走数据，输出完成后把空缓冲区挂回 `emq`。



图中，缓冲区在三个队列之间流动。收容输入和收容输出都要先从 emq 取空缓冲区；提取输入和提取输出把用过的缓冲区还给 emq。队列操作本身是临界区，多个进程或设备同时申请缓冲区时，必须互斥地修改队首指针和缓冲区状态。

四种缓冲方式

方式	数量	稳态每块时间	适用场景
单缓冲	1	$\max(C, T) + M$	实现简单，设备与 CPU 串行使用缓冲区
双缓冲	2	$\max(T, M + C)$	连续 I/O，设备与 CPU 可并行
循环缓冲	多个	由较慢的一边决定	吸收输入突发，需要同步
缓冲池	共享	由分配策略决定	多设备共享，利用率高

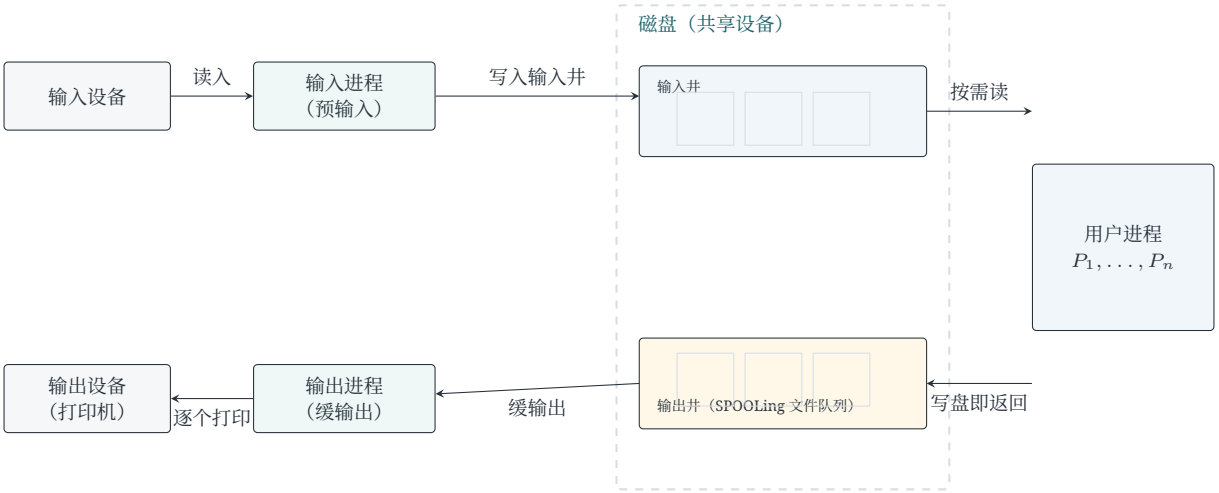
9.5 SPOOLing

三个用户几乎同时提交打印任务，而办公室只有一台打印机。没有 SPOOLing 时，第一个进程独占打印机，第二个和第三个只能等。有了 SPOOLing，三个进程都把要打印的数据写到磁盘上的输出井，写完就返回；后台的打印守护进程按顺序把文件送到打印机。三个进程都觉得自己有一台打印机，实际上它们只是在共享磁盘上的队列。

SPOOLing 是 Simultaneous Peripheral Operations On-Line 的缩写，中文叫假脱机。它由三部分组成。输入井和输出井是在磁盘上开辟的两块存储区域，输入井暂存从输入设备预先读入的数据，输出井暂存进程要输出的数据。输入进程和输出进程是后台进程，输入进程把物理输入设备的数据读进输入井，输出进程把输出井的数据送到物理设备。井管理程序负责分配和回收井中的空间，维护队列。

输出方向的流程如下。进程打开逻辑打印机，系统在输出井中为它建立一个 SPOOLing 文件。进程调用写操作，数据被追加到这个文件里；写完最后一块，进程返回，继续做别的事。此时数据还在磁盘上，没有送到打印机。输出进程按队列顺序取出文件，把内容逐块送到打印机。打印机打完一个文件，输出进程回收井空间，再处理下一个。

输入方向对称。输入进程提前把读卡器、终端等设备的数据读入输入井。进程要读数据时，从输入井取，不必直接和慢速输入设备打交道。输入井里没有数据时，进程阻塞，等输入进程读入后再唤醒。



图中上半部分走输入，下半部分走输出。用户进程只和磁盘上的井打交道：读数据时从输入井取，写数据时往输出井放。打印机由输出进程串行驱动，多个进程的打印请求在输出井里排队。输入井和输出井都在磁盘上，这点常被误记成内存。

SPOOLing 把独占设备改造成了共享设备。从进程的角度看，它独占了一台打印机；从系统的角度看，多台逻辑打印机对应同一台物理打印机，请求在队列里排队。它还把慢速设备的操作变成了磁盘操作：用户进程写磁盘比等打印机快得多，写完就能继续运行。这就是“虚拟设备”的含义。

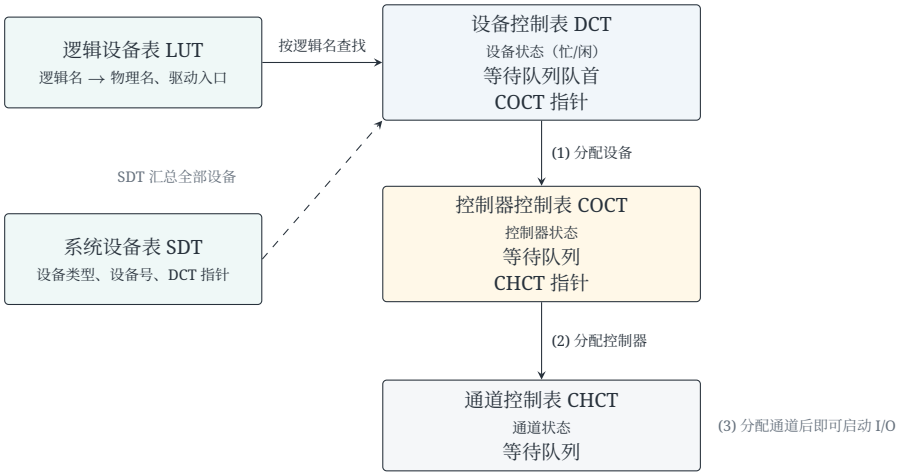
SPOOLing 不会让打印机本身的吞吐率变高。打印机每秒能打多少页是固定的，队列再长也不能改变这一点。它改善的是进程的等待时间和设备利用率：进程不必同步等待慢速设备，打印机也不必在两次任务之间空闲。要实现 SPOOLing，系统需要磁盘存放井，还需要多道程序支持，让输出进程能在用户进程运行的同时工作。

9.6 设备分配

设备是资源，多个进程可能同时申请。独占设备一次只能给一个进程，分配不当会让进程互相等待，甚至死锁。共享设备不需要独占，但要多进程的请求排队，按某种策略调度。虚拟设备则是用 SPOOLing 在逻辑上造出多台设备。

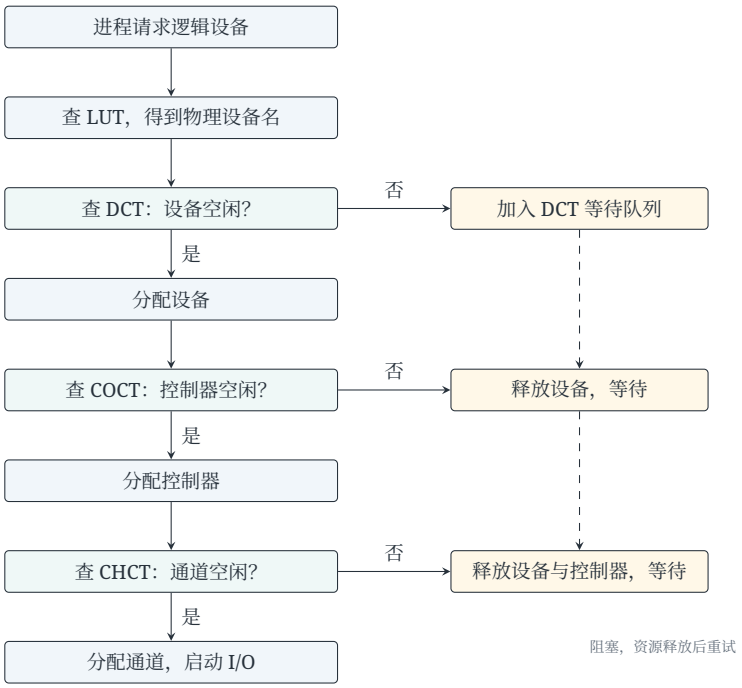
为了管理设备，系统维护一组表。系统设备表 SDT 是全局的，每台设备占一个表项，记录设备类型、设备号、指向设备控制表的指针和驱动程序入口。设备控制表 DCT 描述一台设备的状态：设备号、状态（忙或闲）、等待队列队首、指向控制器控制表的指针。控制器控制表 COCT 描述一个控制器：状态、等待队列、指向通道控制表的指针。通道控制表 CHCT 描述一个通道：状态和等待队列。

逻辑设备表 LUT 负责名字映射。进程申请设备时用的是逻辑名，例如“打印机”或 `/dev/lp0`；LUT 的表项记录逻辑名、物理设备名和驱动程序入口。LUT 可以整个系统一张，也可以每个进程一张。系统一张时，逻辑名必须全局唯一；每个进程一张时，不同进程可以用相同的逻辑名指不同的物理设备。



图中，LUT 把逻辑名变成物理设备，再找到 DCT；SDT 是全局目录，指向所有设备的 DCT。DCT、COCT、CHCT 串成一条链，对应“设备—控制器—通道”的层次。分配时先查设备，再查控制器，最后查通道；任何一层忙，请求都不能立即满足。

一次独占设备的分配通常按下面的顺序进行。进程给出逻辑设备名，系统查 LUT 得到物理设备名和驱动程序入口，再通过 SDT 找到该设备的 DCT，看 DCT 中的状态是否空闲。设备忙，就把进程挂到 DCT 的等待队列；设备空闲，就把设备分给它。接着查 COCT，看控制器能否接受；控制器忙，可以释放设备后再等，也可以带着设备排队，后者要小心死锁。再查 CHCT，看通道是否空闲。三层都分配成功，驱动程序才把请求变成寄存器操作，启动 I/O。I/O 完成后，中断处理程序唤醒等待的进程，系统逐层释放通道、控制器和设备，并唤醒相应队列的队首。



图中是一条分配路径。设备、控制器、通道三层依次检查，任何一层返回“否”，进程都不能启动 I/O。后面的检查失败时，前面已经分配的资源要释放，否则别的进程可能被永久挡住。实际系统里也可以让进程带着已分配的设备在控制器队列里等，但这样会增加死锁风险，需要额外的安全策略。

共享设备不采用这种独占分配。磁盘可以被多个进程同时打开，系统只把读写请求排成队列，按电梯算法之类的策略调度。虚拟设备用 SPOOLing 实现，进程申请的是输出井中的一个逻辑设备，物理打印机仍然由输出进程串行使用。

设备分配有两种时机。静态分配在进程运行前一次性分配它需要的全部设备，运行期间不再申请。这样做不会因为设备竞争而死锁，但设备利用率低，进程可能占着设备却长时间不用。动态分配在进程运行中按需申请，利用率高，但可能出现死锁。多类独占设备同时被多个进程申请时，系统可以用银行家算法做安全性检查：每次分配前先模拟分配，看是否存在安全序列；若分配后进入不安全状态，就拒绝本次分配，让进程等待。

9.7 几组容易混淆的概念

中断驱动和 DMA 的分界在“谁搬数据”。中断驱动由 CPU 在中断服务程序里把数据从设备寄存器搬到内存；DMA 由 DMA 控制器直接在主存和设备之间搬，CPU 只初始化和收尾。中断频率也不同：前者每字或每字节一次，后者每块一次。

块设备和独占设备是两个维度。磁盘是块设备，也是共享设备；打印机是字符设备，也是独占设备。判断时先看题目问的是数据单位还是共享属性。

缓冲和 SPOOLing 解决的不是同一个问题。缓冲区在内存里，用来缓和 CPU 与设备的速度差，减少中断次数；SPOOLing 的井在磁盘上，用来把独占设备改造成共享设备，让多个进程的请求排队。缓冲区通常属于某台设备或某个进程，缓冲池才是系统共享的；SPOOLing 的井从一开始就是共享的。

单缓冲和双缓冲的公式只差一个 M，但不能把 M 随手丢掉。单缓冲的稳态间隔是 $\max(C, T)+M$，双缓冲是 $\max(T, M+C)$。双缓冲的 CPU 路径是“搬移加处理”，不是单独的处理时间。

通道和 DMA 的区别在“有没有指令”。DMA 只能按预先设置好的地址和长度搬一块数据，不能分支、不能循环；通道有自己的指令系统，能执行通道程序，控制多台设备，完成后才中断一次。

LUT 和 SDT 都出现在设备分配的题目里，但职责不同。LUT 负责逻辑名到物理名的映射，按进程或按系统建立；SDT 是系统全部设备的清单，记录每台设备的类型、编号和 DCT 指针。题目问“按逻辑设备名找物理设备”时，答案是 LUT；问“系统中有哪些设备、到哪里找它们的控制表”时，答案是 SDT。

10 习题

10.1 基础过关题

1. （单项选择题）下列关于 I/O 控制方式的叙述中，正确的是

(A) 程序直接控制方式下，CPU 在等待设备就绪期间可切换去运行其他进程

(B) 中断驱动方式下，CPU 在中断服务程序中把设备数据寄存器的内容送入内存

(C) DMA 方式下，每传送一个字节，DMA 控制器就向 CPU 发出一次中断

(D) 通道方式下，通道程序由 CPU 逐条取指并执行

2. （填空题）用户程序调用 `read()` 读取磁盘文件。负责把逻辑设备名映射为物理设备、并完成缓冲管理与设备分配的是 ____；把“读第 n 块”这一通用请求翻译成对控制器寄存器的操作序列的是 ____；设备完成传送后处理中断、唤醒等待进程的是 ____；SPOOLing 打印守护进程位于 ____。

3. （单项选择题）进程以逻辑设备名请求一台设备时，系统依据哪张表取得对应的物理设备名和驱动程序入口？

(A) 系统设备表 SDT　　(B) 设备控制表 DCT

(C) 控制器控制表 COCT　　(D) 逻辑设备表 LUT

4. （单项选择题）下列关于 SPOOLing 技术的叙述中，**错误**的是

(A) 输出井位于磁盘，用于暂存尚未送到打印机的输出数据

(B) 进程调用打印函数后，必须等打印机把该任务全部输出完毕才能返回

(C) SPOOLing 可把打印机这类独占设备改造为多个进程共享的虚拟设备

(D) 输入进程负责把物理输入设备的数据预先读入输入井

5. （计算题）某系统采用单缓冲，设备把一块数据读入缓冲区的时间 $T = 50\,\mu s$，系统把缓冲区数据搬到用户区的时间 $M = 10\,\mu s$，CPU 处理一块数据的时间 $C = 40\,\mu s$。连续读入并处理 4 块数据，总时间是多少？

6. （填空题）缓冲池设置空缓冲队列 emq、输入队列 inq 和输出队列 outq。设备输入数据时，缓冲管理程序先从 ____ 取出一个空缓冲区，装满后把它挂到 ____；进程从该缓冲区取走数据后，缓冲区被回收并挂回 ____。

10.2 真题风格与综合训练

7. （单项选择题）某文件共 10 块，每块大小等于缓冲区大小。设备把一块读入缓冲区需 $T = 100\,\mu s$，系统把缓冲区数据搬到用户区需 $M = 20\,\mu s$，CPU 处理一块需 $C = 80\,\mu s$。与单缓冲相比，采用双缓冲读入并处理完整个文件可节省

(A) $0\,\mu s$　(B) $180\,\mu s$

(C) $200\,\mu s$　(D) $900\,\mu s$

8. （单项选择题）某磁盘以 1 KB 为一块。若采用中断驱动方式，每传送 1 字节中断一次，每次中断的响应、现场保存与数据搬移共需 50 个时钟周期；若采用 DMA，CPU 只在启动时花 200 个周期初始化，整块传完后花 800 个周期处理完成中断。CPU 主频为 1 GHz。只比较 CPU 用于初始化和中断处理的时间，不计 DMA 占用总线引起的访存等待，则传送一块数据时 DMA 比中断驱动方式节省的 CPU 时间约为

(A) $1.0\,\mu s$ (B) $50.2\,\mu s$

(C) $51.2\,\mu s$ (D) $40.96\,\mu s$

9. (综合应用题) 某办公室只有一台打印机，打印速度为 20 KB/s。每个打印任务的数据量均为 100 KB。系统采用 SPOOLing：进程把数据写入磁盘输出井的速率为 5 MB/s，写井请求按提交顺序串行执行，忽略寻道、调度等其他开销。现有 6 个进程在 $t = 0$ 几乎同时提交打印任务，此前打印机空闲。按 1 MB $=$ 1000 KB 计算。

(1) 采用 SPOOLing 时，第 6 个进程的打印请求从提交到写井返回需要多久？全部 6 个任务真正打印完成需要多久？

(2) 若不采用 SPOOLing，打印机被一个进程独占至该任务打印完毕，则第 6 个进程从提交到开始使用打印机需要等多久？全部任务打印完成需要多久？

(3) 比较两问结果，说明在这组参数下 SPOOLing 是否提高了打印机的吞吐率，它实际改善的是什么。

10. (综合应用题) 某系统有同类独占设备 4 台。进程 P_1, P_2, P_3 的最大需求量分别为 3、2、2 台，当前已分别占有 1、1、1 台，系统剩余 1 台。采用银行家算法的安全性检查进行设备分配。

(1) 当前状态是否安全？若安全，给出一个安全序列并写出检查过程。

(2) 若此时 P_1 再请求 1 台设备，能否立即分配？说明理由。

(3) 一次设备分配要依次检查哪些数据结构？分配成功后需要修改哪些表项？若分配会导致不安全状态，系统应如何处理？

11 综合专题

408 的操作系统综合题很少只考一章。题面给一个场景：三个进程同时到达，其中一个执行到第 3 ms 时发生缺页，缺页要从磁盘调页；另一个进程正在读文件，文件块散落在不同磁道。最后问某个进程的完成时刻、平均周转时间，或者这次访存的有效访问时间。

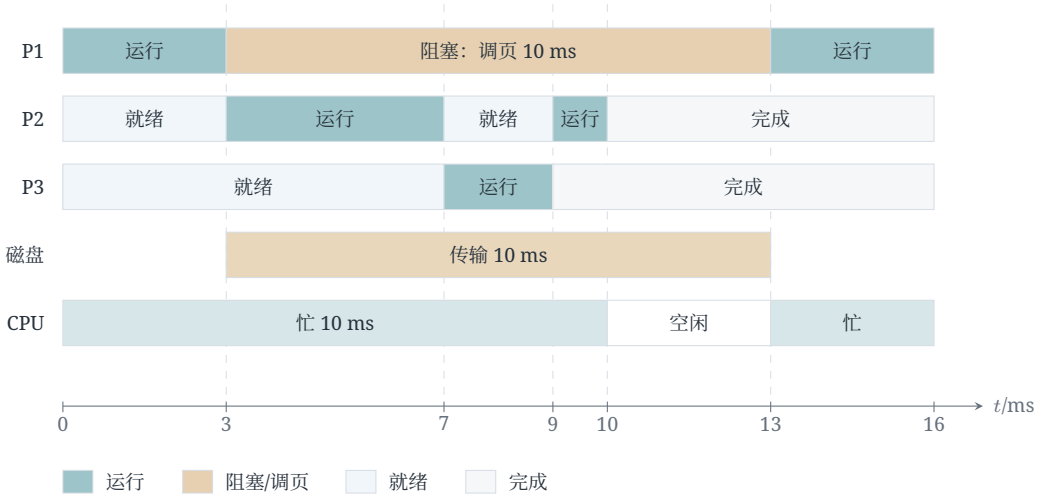
单章的公式都会用，仍然容易算错，原因是中间结果不能直接相加。调页的 10 ms 与 CPU 运行重叠；缺页返回的进程要重新排队；文件索引块本身也占用磁盘请求。下面按“时间线—资源—地址转换—文件到磁盘”的顺序，把这四本账拆开，再合起来。

11.1 时间线：CPU、磁盘和就绪队列

先看一个能手算完的例子。P1、P2、P3 在 $t=0$ 同时到达，按 P1、P2、P3 的顺序进入就绪队列，时间片 $q=4$ ms。P1 需要 6 ms CPU，执行到第 3 ms 时访问的页不在内存，缺页处理需要磁盘传输 10 ms；P2 需要 5 ms，P3 需要 2 ms。假定缺页处理程序本身的 CPU 时间可以忽略，调页期间 CPU 可以运行其他进程，P1 的页调回后排到就绪队列队尾。

时间线推演（$q=4$ ms）

区间/ms	占用 CPU	发生的事
0–3	P1	$t=3$ 时 P1 缺页，转入阻塞，磁盘开始传输（3–13）
3–7	P2	P2 用完 4 ms 时间片，剩余 1 ms，回就绪队列
7–9	P3	P3 完成，$C_3=9$
9–10	P2	P2 完成，$C_2=10$
10–13	空闲	就绪队列为空，P1 仍在等页
13–16	P1	P1 页已调入，运行剩余 3 ms，$C_1=16$



图里每一行是一个对象，横向看它什么时候处于什么状态。P1 的琥珀色段与 P2、P3 的青色段在时间上重叠，这就是调页 I/O 与 CPU 并行；磁盘行只有一段，说明同一个磁盘不能同时服务两个缺页。

完成时刻是 $C_1 = 16$、$C_2 = 10$、$C_3 = 9$ ms，周转时间分别为 16、10、9 ms，平均 $35/3 \approx 11.67$ ms。带权周转时间为 $16/6 \approx 2.67$、$10/5 = 2$、$9/2 = 4.5$，平均约 3.06。CPU 忙碌 13 ms，利用率 $13/16 = 81.25\%$；10–13 ms 的空闲是因为唯一没完成的 P1 还在等页，就绪队列里没有人。

若把 10 ms 直接加到 CPU 需求上，会得到 $6 + 5 + 2 + 10 = 23$ ms；实际是 16 ms。差的 7 ms 正是调页期间 P2、P3 占用的 CPU 时间。综合题的第一处失分点就在这里：设备时间与 CPU 时间可以重叠，不能无条件相加。

把这段推演变成规则，时间线上只有三种区间：进程占用 CPU、进程在就绪队列等待、进程因 I/O 或调页阻塞。设备是另一种资源，同一时刻只服务一个请求。

- 缺页只把当前进程从运行态摘下，CPU 立即从就绪队列选下一个；就绪队列为空，CPU 才空闲。
- 磁盘传输与 CPU 并行，同一磁盘上的多个缺页请求要排队。第 j 个请求的完成时刻为

$$f_j = \max(a_j, f_{j-1}) + t_{\text{trans},j},$$

  其中 a_j 是请求到达磁盘的时刻，$t_{\text{trans},j}$ 含该请求的寻道、旋转和传输。假设 P2 在 $t = 5$ 也缺页（其余条件不变），它的传输只能从 13 ms 开始，完成时刻是 23 ms，而不是 $5 + 10 = 15$ ms。
- 调页结束后进程回队首还是队尾、时间片是否续用，题面必须给。没给就按“回队尾、重新计时”处理，并把假设写进解答；这个选择会改变后续的调度顺序。
- 缺页指令没有执行完，调页后要重新执行。重试的访存通常不再另算一个完整访存周期，除非题目明确计入。
- 上下文切换、调度器、缺页处理程序的 CPU 时间默认忽略；题面给出时，把它们作为独立的小区间插入时间线。

PV 操作与时间线的关系也可以放进这套记账里：P 操作造成的阻塞就是一段阻塞区间，V 操作把等待者放回就绪队列。信号量的初值和 P/V 的先后决定谁先进入临界区，也就决定时间线上事件的顺序。

常用指标写成式子：

$$T_i = C_i - A_i, \qquad W_i = \frac{T_i}{S_i}, \qquad U = \frac{\sum_i S_i}{C_{\max}} \times 100\%.$$

A_i 是到达时刻，C_i 是完成时刻，S_i 是进程实际占用 CPU 的时间（不含阻塞），$C_{\max}$ 是最后一个进程的完成时刻。单 CPU 上 $\sum_i S_i$ 就是 CPU 的忙碌时间。

“等待时间”有两种口径：只统计进程在就绪队列里的等待，或者把阻塞时间也算进去。本题中 P1 的就绪等待为 0、阻塞 10 ms；P2 就绪等待 5 ms；P3 就绪等待 7 ms。若题目问作业等待时间，通常用“周转时间减运行时间”，得到的 P1 是 10 ms。解答里写明用的是哪一种，避免同一组数字被读成两个答案。

11.2 资源账本：谁占着，谁在等

调度题里的资源只有 CPU；综合题里还要同时数页框、磁盘缓冲区、打开文件表和设备。把每种资源的总量、已分配量和可用量列成一张表，进程的状态就可以读成“它持有哪些资源、还在等哪些资源”。

银行家算法在真正分配前先做一次假设分配，再检查剩下的状态能否让所有进程依次完成。检查从一个工作向量 Work 开始，初值等于当前可用资源 Available；每完成一个进程，就把它持有的资源加回 Work。

设系统有 A、B、C 三类资源，总量为 $(10, 6, 7)$，当前分配如下。

资源账本

进程	Allocation			Max			Need		
	A	B	C	A	B	C	A	B	C
P1	2	0	1	5	1	2	3	1	1
P2	1	1	0	3	2	2	2	1	2
P3	3	1	2	7	3	6	4	2	4
P4	0	1	1	2	2	2	2	1	1

已分配总量为 $(6, 3, 4)$，所以 Available $= (10, 6, 7) - (6, 3, 4) = (4, 3, 3)$。Need 一列由 Max − Allocation 得到，例如 P3 的 C 类还差 $6 - 2 = 4$ 个。

安全性检测从 Work $= (4, 3, 3)$ 开始。P1 的 Need $(3, 1, 1)$ 不超过 Work，假设它完成并释放 $(2, 0, 1)$，Work 变成 $(6, 3, 4)$。接着 P2、P4、P3 都能依次满足。

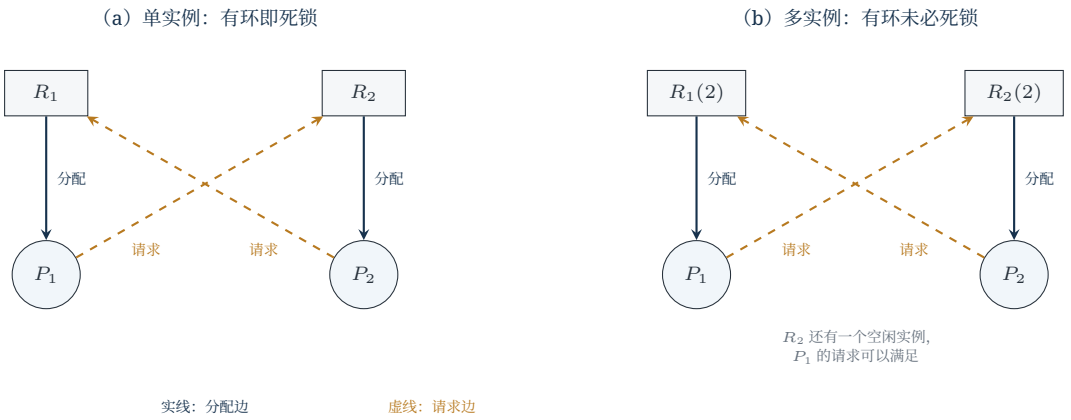
安全性检测

步骤	判断时的 Work	选中进程	该进程 Need	完成后的 Work
1	$(4, 3, 3)$	P1	$(3, 1, 1)$	$(6, 3, 4)$
2	$(6, 3, 4)$	P2	$(2, 1, 2)$	$(7, 4, 4)$
3	$(7, 4, 4)$	P4	$(2, 1, 1)$	$(7, 5, 5)$
4	$(7, 5, 5)$	P3	$(4, 2, 4)$	$(10, 6, 7)$

安全序列是 P1 → P2 → P4 → P3。注意 P3 一开始并不能完成，它的 C 类 Need 是 4，而可用量只有 3；等前面三个进程释放资源后才轮到它。这说明安全序列里的“依次”不等于“任意时刻都满足”，而是每一步在当前 Work 下满足。

若此时 P2 申请 $(1, 0, 1)$，先检查 Request $\le$ Need$_2$ $= (2, 1, 2)$ 且 Request $\le$ Available $=$

$(4,3,3)$，两项都成立。假设分配后 Available $=(3,3,2)$，P2 的 Need 变成 $(1,1,1)$。从 $(3,3,2)$ 出发仍能依次完成 P1、P2、P4、P3，所以这次申请可以立即满足。只要有一次试分配后找不到完成顺序，就应让申请者等待，而不是先分配再补救。



左图的四个节点首尾相接，形成 $P_1 \to R_2 \to P_2 \to R_1 \to P_1$ 的环。每类资源只有一个实例，环上的每个进程都在等别人手里的那一个，谁也动不了，这就是死锁。右图同样有环，但 R_2 有两个实例，一个分配给 P_2，另一个空闲；P_1 的请求可以立即满足，满足后 P_1 不再请求新资源，能够完成并释放 R_1。所以多实例资源的图里，有环只是死锁的必要条件，必须继续化简才能下结论。

资源分配图的化简规则：找一个所有请求边都能被当前可用资源满足的进程，假设它完成，删掉它的请求边和分配边，把资源还回可用池；重复这个过程。能删光，系统无死锁；删不动时剩下的进程就是死锁进程集合。单实例资源可以直接看环，多实例必须化简。

几个容易混淆的结论：

- 安全状态一定没有死锁；死锁状态一定不安全；不安全状态只是“可能将来死锁”，此刻未必已经死锁。
- 安全序列不唯一。上例也可以先完成 P2，再 P1、P4、P3。只要找到一条即可判定安全。
- 安全性检测中任选一个 Need $\le$ Work 的进程都不会把安全状态变成不安全，因为完成它只会让 Work 增大。所以不必回溯搜索。
- 银行家算法要求每个进程预先声明最大需求。题目只说“不会同时需要同种资源的最大量”而没有给出 Max 或 Need，信息不足，不能硬套安全性检测。
- 页框不足在 408 的模型里表现为缺页频繁（抖动），不作为死锁处理。除非题目把页框显式建模成可持有、可请求的资源，否则不要把它画进资源分配图。

11.3 一次访存：TLB、页表、Cache 与缺页

设虚拟地址 32 位，物理地址 30 位，页面大小 4 KiB，两级页表按 $10+10+12$ 位划分。TLB 全相联，共 16 项；Cache 容量 4 KiB，直接映射，共 128 行，每行 32 B。执行 `mov eax, [0x00403A5C]`，读 4 B。

先把地址拆开。页内偏移占低 12 位，`0x00403A5C` 的低 12 位是 `0xA5C`，虚页号是 `0x00403`。两级页号分别为

$$\text{L1 index} = \left\lfloor \texttt{0x00403}/2^{10} \right\rfloor = 1, \qquad \text{L2 index} = \texttt{0x00403} \bmod 2^{10} = 3.$$

若 TLB 未命中，硬件从 PDBR 指向的一级页表读出第 1 项，得到二级页表基址；再读二级页表的

第 3 项，得到页框号 `0x2F1C5`，有效位为 1。物理地址为

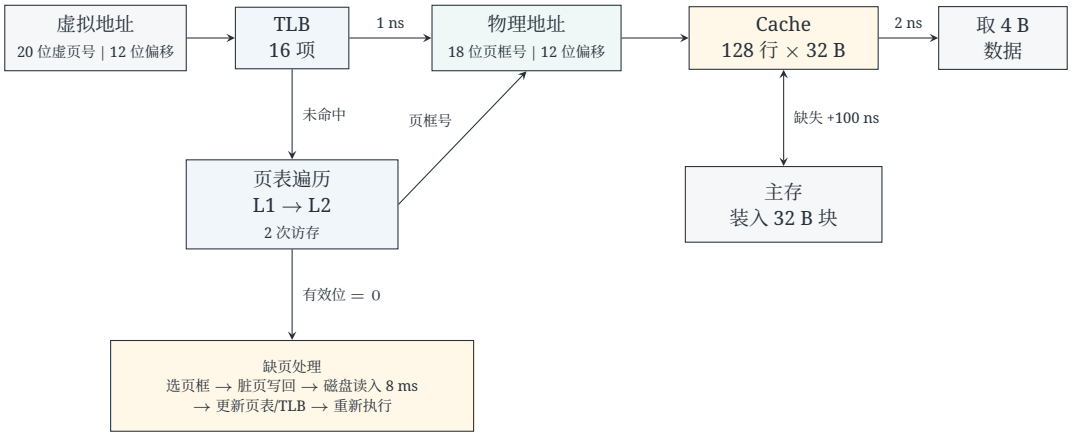
$$\texttt{0x2F1C5} \times 2^{12} + \texttt{0xA5C} = \texttt{0x2F1C5A5C}.$$

物理地址 30 位，页框号 18 位、页内偏移 12 位。Cache 的字段划分为：块内偏移 5 位、行号 7 位、标记 18 位。由 `0x2F1C5A5C` 得块内偏移 $\texttt{0x1C} = 28$、行号 $\texttt{0x52} = 82$、标记 `0x2F1C5`。读的 4 B 落在第 28–31 字节，正好在一个 32 B 块内，不会跨块。

地址与 Cache 的字段划分

层次	字段	位数	本例取值
虚拟地址	虚页号 / 页内偏移	20 / 12	`0x00403` / `0xA5C`
物理地址	页框号 / 页内偏移	18 / 12	`0x2F1C5` / `0xA5C`
Cache	标记 / 行号 / 块内偏移	18 / 7 / 5	`0x2F1C5` / `0x52` / `0x1C`

这里的 Cache 容量恰好等于一页，行号 7 位加块内偏移 5 位正好是 12 位页内偏移，所以标记字段与页框号数值相同。换成 8 KiB 的 Cache（仍为 128 行），块大小变成 64 B，块内偏移 6 位、行号 7 位，标记要按 $30 - 7 - 6 = 17$ 位重算，不能照抄页框号。



沿上排从左到右走，是一次全部命中的路径：虚拟地址查 TLB 得到页框号，拼成物理地址，再查 Cache。向下拐一次，就多付一次开销：TLB 未命中要读两级页表（两次主存访问），Cache 缺失要从主存装入一个块，页表项无效则进入缺页处理，代价从纳秒跳到毫秒。

设 TLB 访问 1 ns，Cache 访问 2 ns，主存一次 100 ns，缺页处理 8 ms。各种组合的耗时和主存访问次数如下。

一次访存的四条路径

情形	时间	主存访问次数
TLB 命中、Cache 命中	3 ns	0
TLB 命中、Cache 缺失	103 ns	1
TLB 未命中、Cache 命中	203 ns	2
TLB 未命中、Cache 缺失	303 ns	3
缺页	8 ms 起	2 次页表 + 磁盘

设 TLB 命中率 h、Cache 命中率 c、页表级数 k、缺页率 p，一次访问的平均时间为

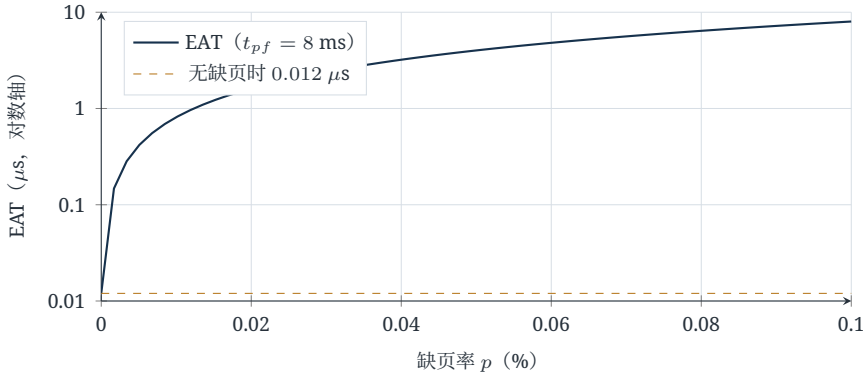
$$\mathrm{EAT} = t_{\mathrm{TLB}} + (1-h)k\,t_m + t_c + (1-c)t_m + p\,t_{pf}.$$

t_m 是一次主存访问，t_c 是 Cache 访问，t_{pf} 是一次缺页处理的总时间（含磁盘 I/O 和操作系统开销）。式子把缺页看成在正常路径上追加 t_{pf}，忽略了缺页时尚未完成的那几纳秒，误差不影响量级。

代入 $h = 0.98$、$c = 0.95$、$k = 2$、$p = 10^{-4}$：

$$1 + 0.02 \times 2 \times 100 + 2 + 0.05 \times 100 + 10^{-4} \times 8 \times 10^{6} = 1 + 4 + 2 + 5 + 800 = 812 \text{ ns}.$$

没有缺页时只要 12 ns；万分之一的缺页率就把它抬到 812 ns。若 $p = 1\%$，缺页项是 $80\ \mu$s，约是前四项的 6700 倍。



横轴是缺页率，纵轴取了对数。曲线在 p 很小时几乎是水平的，因为 12 ns 的固定开销还在；p 增大后，每 0.01 个百分点就抬高约 0.8 μs，很快把固定开销压得看不见。缺页率从 0.001% 涨到 0.1%，EAT 从约 0.09 μs 涨到 8 μs，相差近百倍。

如果题目不要求纳秒，只问访存次数，就把上表最后一列按 k 改写：TLB 命中不读页表；TLB 未命中读 k 次；Cache 命中不访问主存，缺失才读一次；缺页额外读一次磁盘，牺牲页是脏页时再写一次。多数 408 题默认 Cache 用物理地址索引，必须等地址转换完成；若题面说按虚拟地址索引，TLB 查找和 Cache 索引可以并行，但标记比较仍要等页框号。

缺页发生在页表遍历发现有效位为 0 时。CPU 陷入内核，操作系统选一个牺牲页框；若它是脏页，先写回磁盘，再读入所需页；更新页表项和 TLB，返回后重新执行刚才那条指令。缺页是故障（fault），不是终止异常，所以返回地址是当前指令，不是下一条。没有空闲页框时，读入之前必须先置换；脏页写回会让磁盘传输多一次，t_{pf} 要按“写回 + 读入”两次传输估算。

11.4　文件访问：从偏移量到磁头

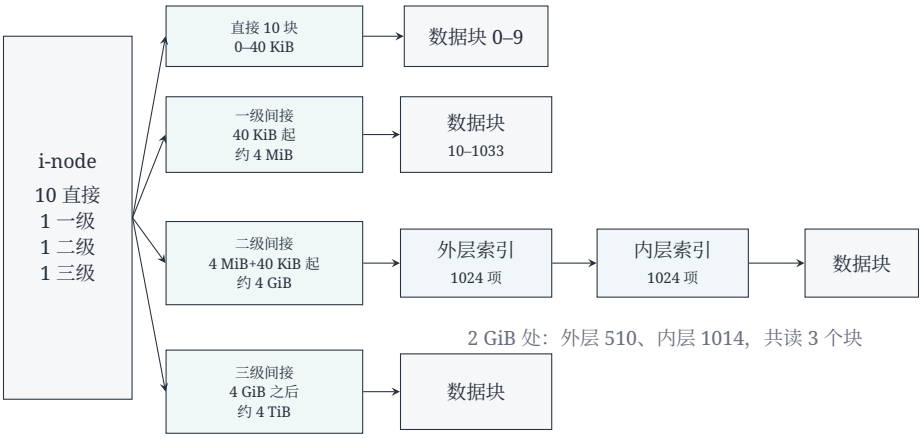
文件系统题给一个字节偏移量，第一步是换成逻辑块号，第二步查索引结构确定要读几个索引块，第三步才是磁盘时间。三步的单位分别是字节、块、毫秒，混在一起就会错。

设 i-node 有 10 个直接指针、1 个一级间接指针、1 个二级间接指针、1 个三级间接指针；盘块 4 KiB，指针 4 B。每个索引块能放 $4096/4 = 1024$ 个指针。

混合索引的覆盖范围

区域	逻辑块号	字节范围
直接（10 块）	0–9	0 到 40 KiB − 1
一级间接	10–1033	40 KiB 到 4 MiB + 40 KiB − 1
二级间接	1034–1 049 609	4 MiB + 40 KiB 到 4 GiB + 4 MiB + 40 KiB − 1
三级间接	1 049 610 起	4 GiB + 4 MiB + 40 KiB 起，再覆盖 4 TiB

最大文件为 $4\ \text{TiB}+4\ \text{GiB}+4\ \text{MiB}+40\ \text{KiB}=4\,402\,345\,713\,664\ \text{B}$，约 4.001 TiB。直接区覆盖 10 块，一级间接覆盖 1024 块，二级间接覆盖 1024^2 块，三级间接覆盖 1024^3 块；每一层只比上一层多一次间接寻址，容量却按 1024 倍增长。



图中每条横向路径对应一种索引深度。直接块只读数据本身；一级间接多读一个索引块；二级间接多读外层和内层两个索引块。访问越深，索引块占的 I/O 越多，但能覆盖的范围也越大。

访问偏移 2 GiB 处的一个字节。$2\ \text{GiB}=2^{31}\ \text{B}$，逻辑块号为 $2^{31}/2^{12}=2^{19}=524\,288$。直接区和一级间接区一共覆盖 $10+1024=1034$ 块，所以它在二级间接区内的相对块号是 $524\,288-1034=523\,254$。外层下标为 $\lfloor 523\,254/1024\rfloor=510$，内层下标为 $523\,254-510\times1024=1014$。

路径是：i-node（已在内存）→ 外层索引块 → 内层索引块 → 数据块，共读 3 个磁盘块。若 i-node 不在内存，再算一次读 i-node 的 I/O；若索引块已在缓冲区，只算未命中的那些。题目问“共几次磁盘 I/O”时，这三个假设必须写清楚。

若题目从 `open()` 开始，还要加上目录项的读入：按路径逐级查目录，每一级目录块可能都要读一次磁盘，找到目录项后才知道 i-node 号。通常题面说“i-node 已在内存”，就表示打开阶段已经完成，后续 `read()` 只算数据块和索引块。

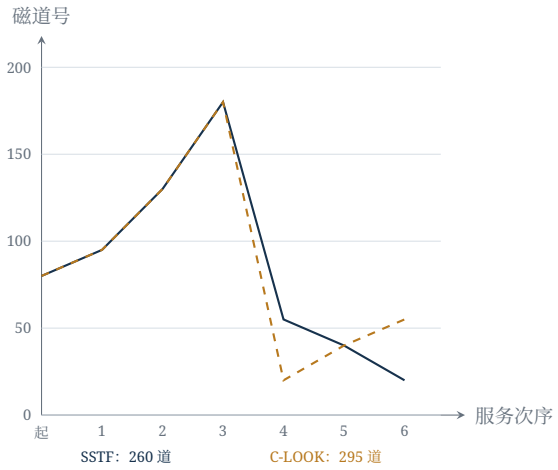
磁盘时间由寻道、旋转延迟和传输三部分组成：

$$T_{\text{rot}}=\frac{60}{r}\ \text{s},\qquad T_{\text{avg rot}}=\frac{T_{\text{rot}}}{2},\qquad T_{\text{trans}}=\frac{s}{S_{\text{track}}}\,T_{\text{rot}}.$$

r 是转速（rpm），s 是每块扇区数，S_{track} 是每道扇区数。设 $r=7200$、每道 400 个扇区、每扇区 512 B、平均寻道 8 ms。一转 $60/7200\ \text{s}=8.333\ \text{ms}$，平均旋转延迟 4.167 ms；4 KiB 是 8 个扇区，传输 $8/400\times8.333=0.167\ \text{ms}$。DMA 把整块传完后发一次中断，CPU 处理中断的时间通常忽略。一次随机块读约

$$8+4.167+0.167=12.33\ \text{ms}.$$

前面三个块若都随机分布，约 37 ms；若索引块命中缓冲区，只剩数据块的 12.3 ms；若三块在同一磁道相邻，寻道和旋转只付一次，传输时间叠加。



请求序列是 55、130、20、180、95、40，磁头在 80 号磁道，向磁道号增大的方向移动。SSTF 每次选最近的请求：$80 \to 95 \to 130 \to 180 \to 55 \to 40 \to 20$，移动 $15+35+50+125+15+20=260$ 道。C-LOOK 先沿增大方向服务 95、130、180，再回到最小的 20，依次服务 40、55：$15+35+50+160+20+15=295$ 道。

图中纵轴是磁道号，相邻两点的纵差就是这次寻道的距离。SSTF 在 180 之后直接跳到 55，形成一次 125 道的长跳；C-LOOK 的 180 到 20 是一次 160 道的回程，看起来更长，但它保持单向服务，不会在高低磁道之间反复折返。总寻道量不是唯一指标，题目若问平均等待时间或服务顺序，还要按时间线逐项累加。

11.5 把口径写进答案

综合题的最后一步不是再算一遍，而是检查题面给了哪些条件、自己用了哪些假设。下表左边是常见的题面条件，右边是它影响的记账位置。

条件与计算口径

题面条件	对计算的影响
缺页处理程序是否占 CPU	占则在时间线上插入一段内核处理区间；不占则 CPU 立即切换到就绪进程
调页磁盘能否并行	不能则多个缺页请求排队，后一个的完成时刻取 max，不能各自从缺页时刻加传输时间
调页后回队首还是队尾、时间片是否续用	改变后续调度顺序；题面未给时写明“回队尾、重新计时”的假设
页框固定还是可变、局部还是全局置换	改变缺页次数；全局置换下缺页率还取决于其他进程的行为
Cache 按虚拟地址还是物理地址索引	PIPT 必须等地址转换；VIPT 可与 TLB 并行查索引，但标记比较仍要物理页框号
索引块、目录块是否在缓冲区	直接改变文件访问的磁盘 I/O 次数，不能默认每次都读
磁盘调度是否把索引块也排进请求序列	改变请求顺序和总寻道量；只调度数据块时要说明
GiB 与 GB、ms 与 μs 与 ns	单位错是最常见的失分点；1 GiB $=2^{30}$ B，1 GB $=10^{9}$ B

做题时先画时间线或资源表，再代公式。时间线负责顺序和重叠，资源表负责可用量与安全性，地址字段表负责位数，块号表负责 I/O 次数。四张表对上了，最后的结果才有唯一的口径。

拓展阅读：

- OSTEP 第 18–22 章「分页」与第 26–28 章「并发」：地址转换、TLB、缺页处理和并发程序的完整讨论。
- OSTEP 第 39–45 章「文件系统实现」：i-node、目录、FAT 与 FFS 的磁盘布局和性能分析。
- 陈海波《现代操作系统：原理与实现》第 5–8 章：Linux 的缺页处理、页面回收、CFS 调度和设备驱动模型。

12 习题

12.1 基础过关题

1. （单项选择题）某系统用 TLB 和页表完成地址转换，数据访问经过 Cache。下列叙述中正确的是

   (A) TLB 未命中且页表项有效时，地址转换不需访问主存
   (B) Cache 按物理地址索引时，必须先完成地址转换
   (C) 缺页处理结束后，从触发缺页指令的下一条指令继续执行
   (D) 二级页表由操作系统查询，硬件只查一级页表

2. （填空题）某系统按字节编址，虚拟地址 32 位，物理地址 36 位，页面大小 8 KiB，页表项长 4 B，采用单级页表。

   (1) 页内偏移占 ____ 位，虚页号占 ____ 位，页框号占 ____ 位。
   (2) 该页表共有 ____ 个页表项，连续存放时占 ____ 主存。
   (3) 进程访问虚拟地址 `0x0001A2B3`，TLB 未命中，查得对应页框号为 `0x1F2C5`，则物理地址为 ____；若该次数据访问在 Cache 中命中，全程共需 ____ 次主存访问。

3. （单项选择题）系统有 A、B、C 三类资源，总量为 $(5, 2, 3)$，当前分配情况如下表。此时系统的一个安全序列是

进程	Allocation			Max		
	A	B	C	A	B	C
P0	1	0	1	2	2	2
P1	1	1	0	3	2	1
P2	2	0	1	2	1	2

(A) P0→P1→P2　(B) P1→P2→P0

(C) P2→P1→P0　(D) P2→P0→P1

4. (**填空题**) 某文件系统采用 UNIX 混合索引：i-node 含 10 个直接块指针、1 个一级间接块指针、1 个二级间接块指针；盘块 4 KiB，每个块指针 4 B，i-node 常驻内存。

   (1) 单个文件最大为 ____ B，约为 ____ GiB。

   (2) 访问文件内偏移 1 MiB 处的数据，需读 ____ 个索引块，共 ____ 次磁盘 I/O；该块在一级间接块中的下标为 ____。

   (3) 访问偏移 6 MiB 处的数据，需经过 ____ 间接索引，共 ____ 次磁盘 I/O。

5. (**填空题**) 某磁盘转速 10000 r/min，每道 400 个扇区，每扇区 512 B，平均寻道时间 8 ms。忽略控制器与排队开销，读取 8 KiB 数据：

   (1) 每转时间为 ____ ms，平均旋转延迟为 ____ ms。

   (2) 传输时间为 ____ ms。

   (3) 平均访问时间为 ____ ms。

6. (**单项选择题**) 某进程用 `mmap` 把文件映射到虚拟地址空间，采用请求分页。下列叙述中正确的是

(A) 采用 MAP_PRIVATE 映射的页被修改后，换出时必须写回原文件

(B) 文件映射页若未被修改，换出时可直接丢弃，不必写回文件

(C) 匿名页换出时写回其对应的可执行文件

(D) 映射区第一次被访问时，页面一定已在内存中

12.2 真题风格与综合训练

7. （真题风格 · 综合应用题）某单 CPU 系统采用时间片轮转调度，时间片 $q = 3$ 个时间单位；进程 P1 在 0 时刻到达，P2 在 2 时刻到达。两进程的页面引用序列（每个时间单位执行一条访存指令，引用一个页号）为

$$\text{P1}: 1, 2, 1, 3; \qquad \text{P2}: 2, 3, 2, 1.$$

每个进程固定分配 2 个页框，采用局部 LRU 置换，初始页框为空。缺页时该访存指令不完成，系统启动页面 I/O，服务时间为 3 个时间单位；系统只有一个磁盘，缺页请求按 FIFO 排队，一次服务一个。I/O 完成后进程排入就绪队列队尾，重新执行刚才缺页的指令。进程到达即排入就绪队尾；时间片用完且就绪队列非空时抢占并排尾；CPU 空闲时就调度队首进程。

(1) 分别给出 P1、P2 的缺页次数和运行结束时的驻留页集合。

(2) 画出 0–20 时间单位的 CPU 时间线（空闲区间也要标出）。

(3) 求两进程的完成时间、周转时间及平均周转时间，并计算 CPU 利用率（发生缺页的那一个时间单位也算 CPU 忙）。

(4) 若把时间片改为 1，完成时间和缺页次数是否改变？说明理由。

8. （真题风格·综合应用题）某系统按字节编址，虚拟地址 32 位，物理地址 28 位，页面大小 4 KiB，采用“一级页号 10 位 + 二级页号 10 位 + 页内偏移 12 位”的二级页表，页表项 4 B。TLB 全相联，共 8 项，当前有效项如下：

虚页号	页框号
0x00A38	0x02F10
0x00A3C	0x03A5C
0x00A40	0x01180

Cache 采用直接映射，共 64 行，每行 32 B。CPU 读取虚拟地址 0x00A3C5E7 处的 4 B 数据。

(1) 写出虚拟地址的字段划分，并给出该地址的一级页号、二级页号和页内偏移。

(2) 判断 TLB 是否命中；若命中，求物理地址。

(3) 若 TLB 未命中且页表项有效，在 Cache 命中和 Cache 缺失两种情况下，完成本次数据访问各需几次主存访问（不计 TLB 和 Cache 本身的访问）？

(4) 求该物理地址对应的 Cache 行号、标记字段和块内偏移；说明 4 B 数据是否跨块。

(5) 设 TLB 访问 2 ns，主存访问 100 ns，Cache 访问 5 ns；TLB 命中率 96%，Cache 命中率 95%，缺页率 10^{-4}，缺页处理时间 8 ms。求平均访问时间（从发出虚拟地址到取得数据）。

9. （真题风格·综合应用题）某文件系统采用 UNIX 混合索引：i-node 有 10 个直接块指针、1 个一级间接指针、1 个二级间接指针；盘块 4 KiB，块指针 4 B，i-node 常驻内存。磁盘有

200 个磁道（0–199），磁头当前在 100 号磁道并向磁道号增大的方向移动。现有 5 个读块请求，对应磁道号依次为

$$55,\ 90,\ 130,\ 160,\ 20.$$

(1) 求单个文件的最大长度（以字节和 GiB 表示）。

(2) 进程要读文件内偏移 10 MiB 处的一个字节：求逻辑块号、需要经过的索引块，以及不计 i-node 读取时的磁盘 I/O 次数。

(3) 分别按 FCFS、SSTF、SCAN、C-LOOK 写出服务顺序并计算磁头移动总磁道数；SCAN 和 C-LOOK 均按当前方向先服务，SCAN 到达端点 199 后才反向。

(4) 设每移动一个磁道用时 0.15 ms，磁盘转速 6000 r/min，每道 64 个扇区，每扇区 512 B。按 SSTF 顺序读取这 5 个 4 KiB 块，忽略排队和控制器开销，求总时间。

10. **（真题风格 · 综合应用题）** 某流水线由三个进程组成：P1 不断生产零件放入缓冲 A（容量 10）；P2 从 A 取出零件加工，把成品放入缓冲 B（容量 5）；P3 从 B 取出成品消费。A、B 均按 FIFO 使用，对同一缓冲区的存取必须互斥；生产、加工、消费本身可以并发。

(1) 给出所需的信号量及其含义、初值。

(2) 用 P、V 操作写出三个进程的程序。

(3) 若 P2 把 `P(mutexA)` 放在 `P(fullA)` 之前，分析可能出现的后果。

(4) 若同时有 3 个 P2 进程并行加工，信号量和程序是否需要修改？说明理由。

(5) 若把 B 的容量改为 1，程序是否仍然正确？对流水线性能有什么影响？

PART IV　第四部分

计算机网络

1 计算机网络体系结构

在浏览器地址栏输入一个网址，回车，几十毫秒后页面出现。这段时间里，浏览器只负责组织 HTTP 请求，操作系统里的 TCP 负责分段、编号和重传，网卡把比特变成电信号或光信号，交换机按 MAC 地址转发帧，路由器按 IP 地址选路。没有任何一个部件理解整个过程，但每一层都把自己的事情做完，把结果交给下一层。这种“各管一段、上下衔接”的组织方式就是分层。本章要建立的，就是这套分工的语言：协议、服务、封装、PDU，以及衡量网络快慢的几个基本指标。

按覆盖范围，网络分为广域网 WAN、城域网 MAN、局域网 LAN 和个域网 PAN；按交换方式，分为电路交换、报文交换和分组交换。408 的题目主要在这两个维度上展开，其中交换方式会直接影响时延的计算。

1.1 分层：一次网页访问拆开来看

假设主机 A 通过一个交换机和一个路由器访问主机 B。A 的应用层生成一条 HTTP 请求；传输层给它加上源端口、目的端口和序号，交给网络层；网络层加上源 IP、目的 IP，交给数据链路层；数据链路层加上帧头和帧尾，交给物理层；物理层把比特逐个送上介质。

中间设备并不是七层全做。交换机收到帧后，只看目的 MAC 地址，查表转发，根本不拆开 IP 首部；路由器收到帧后，剥掉帧头，取出 IP 分组，查路由表决定下一跳，再给分组套上新的帧头转发出去。也就是说，交换机工作在数据链路层，路由器工作到网络层，它们各自只理解自己那一层及以下的内容。

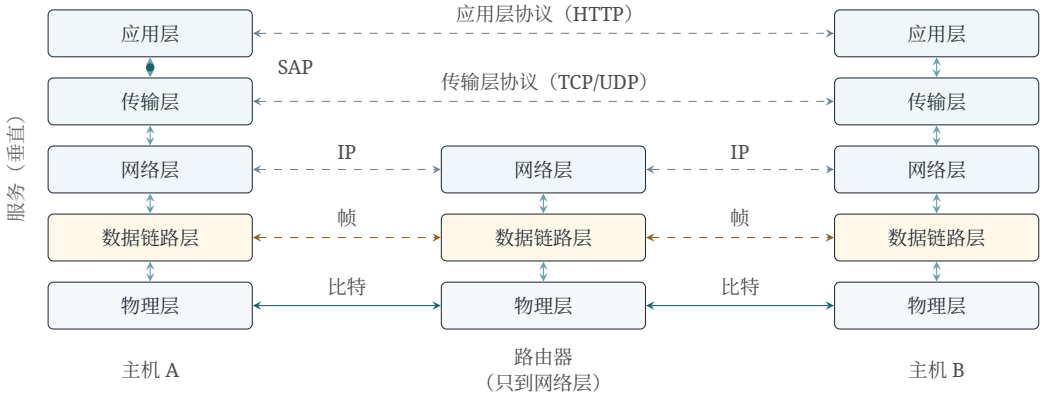


图 1-1　分层协议栈：水平虚线是对等层之间的协议，竖直箭头是相邻层之间的服务

图中横向的虚线只画在对等层之间：A 的应用层和 B 的应用层谈 HTTP，A 的传输层和 B 的传输层谈 TCP，依此类推。纵向的箭头表示下层向上层提供服务。二者方向不同，不能混着说。路由器只有下三层，所以它的应用层和传输层是空的；IP 分组在路由器上被拆到网络层，查完路由表再重新封装，这也是“路由器工作在网络层”的具体含义。

第 n 层向上层提供服务的逻辑接口叫**服务访问点**（SAP，Service Access Point）。在 408 的语境下常记：数据链路层的 SAP 是 MAC 地址，网络层的 SAP 是 IP 地址，传输层的 SAP 是端口号。SAP 是层间交往的“门牌号”，上层把数据送到这个门牌号，下层就知道该交给哪个实体。

分层不是免费的：每层都要加首部，层间调用也要花时间。但它带来的独立性很值钱。IPv4 升级到 IPv6，应用层的 HTTP 不用改；以太网从 10 Mb/s 升到 100 Gb/s，IP 也不用改。只要下层向上提供的服务不变，上层的实现就可以不动。所谓**网络体系结构**，就是分层方式和各层协议的集合；它是抽象定义，不是具体实现，同一种体系结构可以有多种实现。

协议是通信双方必须遵守的规则，包含三个要素：

- **语法**：数据和控制信息长什么样。例如 HTTP 请求行由方法、URL、版本三段组成，字段之间用空格分隔。
- **语义**：每个字段是什么意思、收到后要做什么。例如状态码 404 表示资源不存在，TCP 的 SYN 表示请求建立连接。
- **时序**：事件谁先谁后、什么时候重发。例如 TCP 三次握手的报文顺序，以及超时计时器的设置。

> **注意**
>
> 语法管“长什么样”，语义管“是什么意思、做什么”，时序管“什么时候做”。408 喜欢把这三者和“服务、接口、协议”混在一起出选择题；判断时先问：这句话描述的是格式、含义，还是顺序？

1.2 封装：首部是怎样一层层加上的

仍以上面的 HTTP 请求为例。假设应用层交给 TCP 的数据是 1000 B。TCP 加上 20 B 首部（源端口、目的端口、序号、确认号等），得到 1020 B 的报文段；IP 再加上 20 B 首部（源 IP、目的 IP、协议号、生存时间等），得到 1040 B 的分组；以太网加上 14 B 帧首部（目的 MAC、源 MAC、类型）和 4 B 尾部 FCS，得到 1058 B 的帧。首部开销是 58 B，约占原数据的 5.8%。数据越短，首部占比越大，这也是小包效率低的原因之一。

这个过程叫**封装**（encapsulation）：本层把上层交下来的 PDU 当作自己的服务数据单元（SDU），在前面加上本层首部（有时还在后面加尾部），形成本层的 PDU。首部里的地址、序号、长度、校验等控制信息，只对本层有意义；上层数据对下层来说只是一串字节，下层不关心它的内容。接收方的过程相反，叫**解封装**：每层检查并去掉本层首部，把剩下的数据交给上一层。

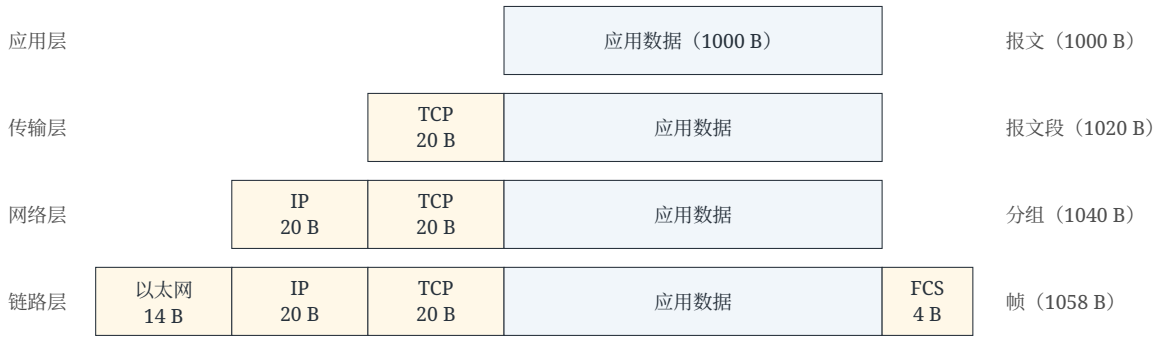


图 1-2　封装：上层 PDU 整体成为下层 SDU，逐层加首部；链路层还多一个尾部 FCS

从下往上看，每一行都比上一行多一个首部；从上往下看就是接收方的解封装。注意链路层是唯一通常既有首部又有尾部的层：FCS 放在帧尾，用来做差错检测。图中没有画前导码，以太网帧的 14 B 首部也不包含前导码。

各层 PDU 的名称要能对号入座：应用层叫**报文**（message），传输层 TCP 叫**报文段**（segment）、UDP 叫**用户数据报**（user datagram），网络层叫**分组**或**数据报**（packet/datagram），数据链路层叫**帧**（frame），物理层是**比特**（bit）。408 的选择题经常用“报文段/分组/帧”来定位层，看到名字就要能反应出它属于哪一层。

封装也不等于“把数据复制一遍”。实际实现中，协议栈往往在缓冲区头部预留空间，逐层向前移动指针填写首部，避免整块数据的搬移；但概念上，仍然是“上层 PDU 加本层控制信息”这个模型。

1.3　三种交换方式：电路、报文、分组

两台主机之间往往隔着多个交换结点。数据以什么单位、按什么方式通过这些结点，直接决定了时延和线路利用率。

电路交换先建立一条端到端的通路（拨号、信令），建立后双方独占这条通路，数据像流水一样连续通过，中间结点不做存储转发。传统电话网是典型例子。设建立时间为 s，报文长为 M bit，链路速率为 R，共 k 段链路，每段传播时延为 d_{prop}，忽略处理时延，则

$$T_{\text{c}} = s + \frac{M}{R} + k\,d_{\text{prop}}.$$

建立时间可能很长；但一旦建立，数据在中间结点没有“收完再发”的等待。

报文交换把整个报文作为一个单位。结点必须收完整个报文、做完校验，才能把它转发到下一跳。因此每个结点都需要能缓存整个报文的缓冲区，长报文会长时间占用链路。总时延为

$$T_{\text{m}} = k\frac{M}{R} + k\,d_{\text{prop}},$$

因为报文在 k 段链路上各被完整发送了一次。

分组交换把报文切成若干等长分组，每个分组独立地存储转发。关键变化是流水线：第 1 个分组在第 2 条链路上传输时，第 2 个分组已经在第 1 条链路上传输，不必等整个报文传完。设报文长 M bit，分成 n 个长为 $L = M/n$ 的分组，链路数仍为 k，则第 1 个分组到达终点需要 $k(L/R + d_{\text{prop}})$；此后每隔 L/R 到达一个分组，共 n 个，所以

$$T_{\text{p}} = k\left(\frac{L}{R} + d_{\text{prop}}\right) + (n-1)\frac{L}{R} = \frac{(n+k-1)L}{R} + k\,d_{\text{prop}}.$$

当 n 很大、L 很小时，$T_{\text{p}} \approx M/R + k d_{\text{prop}}$，与电路交换在数据传输阶段所需的时间基本相同，只多了 $(k-1)L/R$ 的流水线填充；而报文交换是 kM/R，差距随链路数 k 线性扩大。

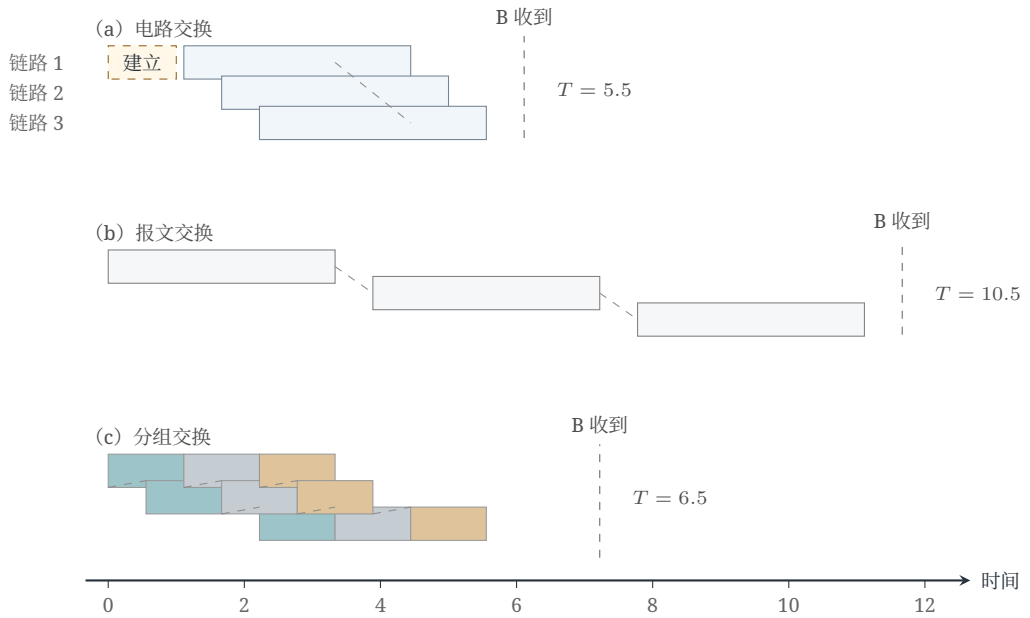


图 1-3　三种交换方式的时空图（$M/R = 3$，$d_{\text{prop}} = 0.5$，$k = 3$，$n = 3$；纵轴为链路，横轴为时间）

图里每一条斜向右下的虚线表示“一个比特从一条链路走到下一条链路”的传播过程。电路交换在建立后是一条连续的水流，三条链路上的蓝色块互相重叠，说明不同链路上的比特在同时

前进；报文交换必须等整个报文收完才转发，所以三个灰色块首尾相接、互不重叠；分组交换把报文切成三块，第 2、3 块依次跟进，形成阶梯状，总时延从 10.5 降到 6.5。这里的数值只是为了比较，单位统一取 M/R。

交换方式	传输单位	存储转发	总时延（k 段链路）	典型场景
电路交换	比特流	否	$s+\frac{M}{R}+kd_{\text{prop}}$	传统电话、实时语音
报文交换	整个报文	是	$k\frac{M}{R}+kd_{\text{prop}}$	电报、早期文件传输
分组交换	分组	是	$\frac{(n+k-1)L}{R}+kd_{\text{prop}}$	Internet、绝大多数数据网络

分组交换的代价是每个分组都要带首部，开销随分组数增加；结点排队还会带来排队时延和时延抖动，所以实时语音不一定适合分组交换。它的好处是线路可以统计复用，多个用户共享链路，空闲时不会像电路交换那样浪费带宽。

还要区分“分组交换”和“服务类型”。分组交换是交换方式；在网络层，它既可以提供无连接的**数据报服务**（IP），也可以提供面向连接的**虚电路服务**（X.25、帧中继）。虚电路不是电路交换：它仍然是分组交换，分组仍然要在结点存储转发，只是沿着预先建立的逻辑路径走，省去了每个分组的独立选路。

1.4 带宽、时延与“管道”

“带宽”这个词有两种用法。模拟通信里，带宽指信道能通过的频率范围，单位是 Hz；数字通信里，带宽指链路能达到的最高数据率，单位是 b/s。408 题目说“链路带宽为 100 Mb/s”通常指后者，说“电话线带宽为 3 kHz”指前者。看到单位就能判断说的是哪一种。

速率的进位是十进制：1 kb/s $=10^3$ b/s，1 Mb/s $=10^6$ b/s，1 Gb/s $=10^9$ b/s。存储容量却常用二进制：1 KB $=2^{10}$ B $=1024$ B。题目里“一个 1 MB 的文件在 1 Mb/s 的链路上传输”不是简单的 $1/1=1$ 秒，必须先把字节换成比特，再按十进制速率计算。

要点 1.1: 时延与两个常用乘积

- 发送时延：$d_{\text{trans}}=\dfrac{L}{R}$，$L$ 为数据长度（bit），R 为发送速率（b/s）。
- 传播时延：$d_{\text{prop}}=\dfrac{D}{v}$，$D$ 为链路长度（m），v 为信号在介质中的传播速率（m/s）。
- 结点总时延：$d=d_{\text{proc}}+d_{\text{queue}}+d_{\text{trans}}+d_{\text{prop}}$。
- 时延带宽积：$R\,d_{\text{prop}}$，单位是 bit，表示链路“管道”中最多能容纳的比特数。
- 利用率 U 与空载时延 D_0 的排队论近似：$D=\dfrac{D_0}{1-U}$。

处理时延 d_{proc} 是结点检查首部、查表、做差错检测的时间；排队时延 d_{queue} 取决于队列长度和当前负载，是最不确定的一项。考试里如果题目没有特别说明，通常忽略这两项，只算发送和传播。

这里最重要的是两个“与什么有关”。发送时延由数据量 L 和发送速率 R 决定，与距离无关；传播时延由距离 D 和传播速率 v 决定，与发送速率无关。把带宽从 100 Mb/s 提高到 1 Gb/s，只能缩短发送时延，不能改变信号在光纤里跑 1000 km 的时间。反过来，链路拉长也不会让网卡把数据推上介质的速度变慢。

举个具体的数：1000 B $=8000$ bit 的数据在 100 Mb/s 的链路上发送，

$$d_{\text{trans}}=\frac{8000}{10^8}=80\ \mu\text{s}.$$

若链路长 1000 km，信号速率为 2×10^8 m/s，则

$$d_{\text{prop}}=\frac{10^6}{2\times10^8}=5\ \text{ms}.$$

传播时延是发送时延的 60 多倍。长距离链路上，决定“快不快”的往往不是带宽，而是光跑得不够快。

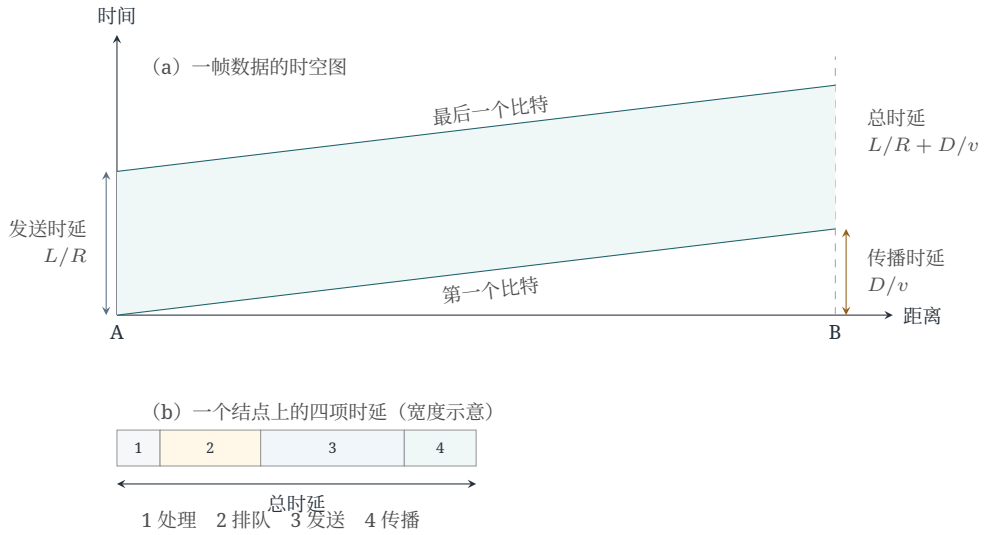


图 1-4　时延分解：（a）发送时延是把整帧推上链路的时间，传播时延是第一个比特在路上走的时间；（b）处理、排队、发送、传播四项

图（a）中，横轴是距离，纵轴是时间。数据在发送端从 $t=0$ 开始逐比特推上链路，用 L/R 推完；第一个比特以速度 v 斜向右上传播，经过 D/v 到达接收端；最后一个比特再晚 L/R 到达。两条斜线之间的水平宽度是帧长，垂直宽度是发送时延。图（b）把一个结点的四项时延拼在一起，其中处理和排队随负载变化，宽度只是示意。

把链路想成一根管子，可以更直观地理解时延带宽积：管子的长度是传播时延 d_{prop}，截面积是带宽 R，管子的“体积”就是 $R\,d_{\text{prop}}$ 个比特。它表示链路最多能容纳多少比特在途。若发送端持续发送，第一个比特到达接收端的那一刻，链路上正好有 $R\,d_{\text{prop}}$ 个比特；如果数据不够长，实际在途的比特数会小于这个容量。

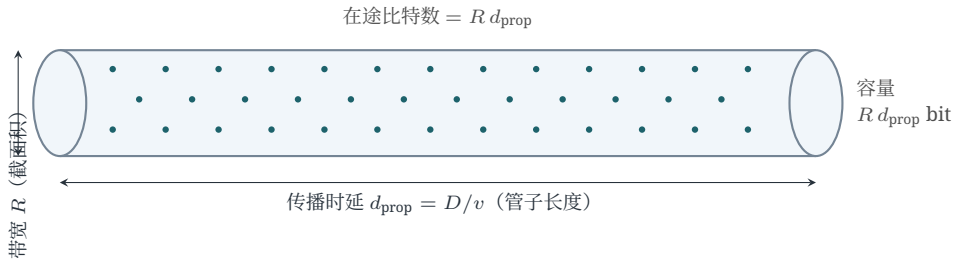


图 1-5　链路管道：长度是传播时延，截面积是带宽，体积就是在途比特数

用刚才的数：100 Mb/s 的链路，单向传播时延 5 ms，时延带宽积为

$$100\times10^6\times5\times10^{-3}=5\times10^5\ \text{bit},$$

也就是 500 kbit，约 62.5 kB。这条链路“装满”时，有 500 kbit 的数据正在路上，还没有到达接收端。

往返时间（RTT）是从发送端发出数据到收到确认的时间，包含往返的传播、发送、处理和排队。除非题目明确忽略其他项，不能只按 $2D/v$ 计算。RTT 在后面的可靠传输和拥塞控制里会反复出现。

吞吐量是单位时间内实际通过网络的数据量，受瓶颈链路限制：端到端吞吐量约为 $\min(R_1, R_2, \ldots, R_k)$。它可能远小于带宽，因为协议首部、确认、重传和等待都会占时间。**信道利用率** U 是有数据通过的时间占总时间的比例；排队论给出的近似 $D = D_0/(1-U)$ 说明，利用率接近 1 时分母趋近 0，时延会急剧上升。这就是网络“跑满”时体验明显变差的原因。

1.5 OSI 七层与 TCP/IP 四层

OSI 七层模型是 ISO 在 20 世纪 80 年代提出的参考模型，是法律上的国际标准，但实际部署并不成功。TCP/IP 是互联网真正使用的协议族，是事实标准。408 教学采用五层模型：把 OSI 的上三层合并为应用层，下两层保留，既保留了分层的清晰性，又贴近实际协议。

七层从下往上依次是：

- **物理层**：在传输介质上传输比特，规定接口的机械、电气、功能、规程特性。它不关心比特的含义，只关心“0 和 1 能不能可靠地送出去”。
- **数据链路层**：把比特流组织成帧，处理相邻结点之间的差错控制、流量控制和介质访问控制。它关心“这一跳”。
- **网络层**：把分组从源主机送到目的主机，负责路由选择、分组转发、拥塞控制和网际互连。它关心“走哪条路”。
- **传输层**：为两台主机的进程提供端到端通信，负责复用与分用、可靠传输和流量控制。它关心“哪个进程”。
- **会话层**：建立、管理和终止会话，在数据流中插入同步点，使长传输中断后可以从同步点继续。
- **表示层**：做数据格式转换、加密解密和压缩，解决不同主机表示方法不一致的问题。
- **应用层**：直接为用户应用提供网络服务。

网络层提供的是主机到主机的逻辑通信，传输层提供的是进程到进程的通信，数据链路层只负责相邻结点之间。三者范围不同，这是选择题的常见考点。

TCP/IP 四层把 OSI 的物理层和数据链路层合并为**网络接口层**，把会话层、表示层、应用层合并为**应用层**，中间保留**网际层**和**传输层**。网络接口层不规定具体协议，实际使用以太网、PPP、Wi-Fi 等；网际层的核心是 IP，辅以 ICMP、ARP、IGMP 和路由协议；传输层是 TCP 和 UDP；应用层是 HTTP、DNS、SMTP、FTP、DHCP 等。

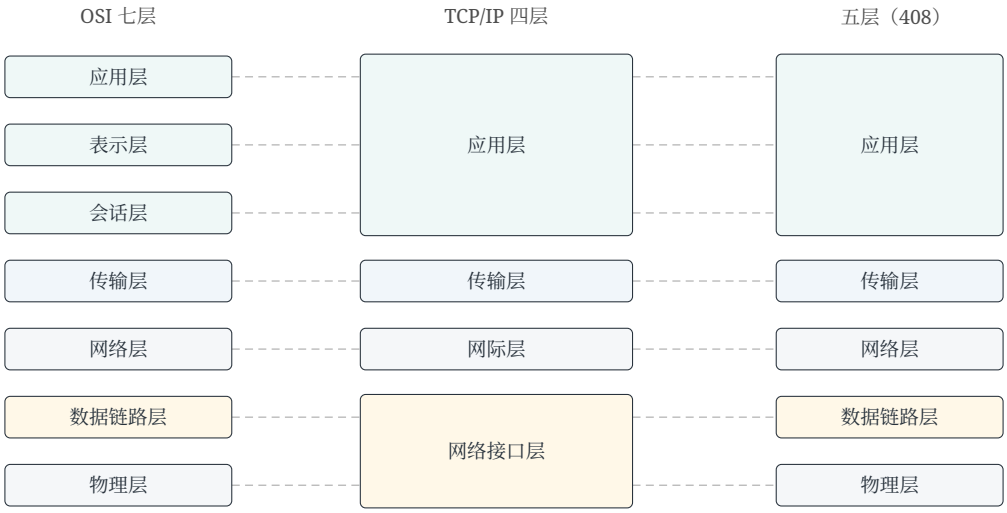


图 1-6　OSI 七层、TCP/IP 四层与五层模型的对应关系

图中虚线只表示功能上的对应，不表示协议完全相同。OSI 的会话层、表示层在 TCP/IP 中没有独立对应，功能并入应用层；TCP/IP 的网络接口层相当于 OSI 的物理层加数据链路层，但 TCP/IP 不规定这一层用什么协议。五层模型只是把 OSI 的上三层合并、下两层保留，所以它既不是 OSI，也不是 TCP/IP，而是 408 的教学约定。

五层模型的分工（408 采用）

层次	主要职责	数据单元	典型协议 / 设备
物理层	在介质上传输比特；规定接口的机械、电气、功能、规程特性	比特	中继器、集线器
数据链路层	相邻结点间的成帧、差错控制、流量控制、介质访问控制	帧	Ethernet、PPP；网桥、交换机
网络层	路由选择、分组转发、拥塞控制、网际互连	分组 / 数据报	IP、ICMP、ARP、OSPF；路由器
传输层	进程到进程通信、复用与分用、可靠传输、流量控制	报文段 / 用户数据报	TCP、UDP；网关
应用层	为用户应用提供网络服务	报文	HTTP、DNS、SMTP、FTP、DHCP、BGP

表中的设备对应关系要记牢：中继器和集线器在物理层，只放大和转发信号，不识别帧；网桥和二层交换机在数据链路层，按 MAC 地址转发帧，能隔离冲突域但不隔离广播域；路由器和三层交换机在网络层，按 IP 地址选路，能隔离广播域；网关在传输层及以上，做协议转换。

协议分类还有两个容易混的细节。ARP 解决 IP 地址到 MAC 地址的映射，408 通常把它归入网络层；BGP 虽然做路由选择，但它运行在 TCP 之上（端口 179），属于应用层。OSPF 直接封装在 IP 分组里（协议号 89），属于网络层。看到“路由协议”不要一律归到网络层，要看它被谁承载。

1.6　协议、服务与 SAP

协议和服务是分层体系里最容易说反的一对概念。协议是**水平**的：它只规定对等层之间如何通信，A 的 TCP 和 B 的 TCP 按 TCP 协议交换报文段，A 的 HTTP 和 B 的 HTTP 按 HTTP 协议交换请求和响应。服务是**垂直**的：下层通过层间接口向上层提供功能，上层只管调用，不关心下层怎么实现。

这个区别的实际意义是：第 n 层协议的实现可以换，只要它向上提供的服务不变，第 $n+1$ 层就不受影响。例如，底层从以太网换成 Wi-Fi，IP 层看到的仍然是一个能发送和接收分组的接口，IP 协议本身不需要改。

SAP 是服务访问点，是相邻层之间交换信息的逻辑接口。数据链路层的 SAP 是 MAC 地址，网络层的 SAP 是 IP 地址，传输层的 SAP 是端口号。上层把数据送到 SAP，下层就知道该交给哪个实体；这就像寄信时写清楚门牌号，邮递员不需要知道屋里的人是谁。

服务原语描述上层和下层如何交互，常见的有四类：

- **请求**（request）：服务用户要求服务提供者做某件事，方向是上层到下层的。
- **指示**（indication）：服务提供者通知服务用户有事件发生，方向是下层到上层的。
- **响应**（response）：服务用户对指示做出答复，方向是上层到下层的。
- **确认**（confirm）：服务提供者报告请求的结果，方向是下层到上层的。

有确认的服务四类原语都会出现；无确认的服务通常只有请求和指示。注意“请求”是上层发出的，“指示”才是下层通知上层，二者方向相反，选择题常在这里设陷阱。

服务还分为面向连接和无连接两类。面向连接服务在数据交换前建立连接、交换后释放连接，数据按顺序传送；无连接服务每个报文独立选路，不保证顺序。TCP 提供面向连接的传输服务，UDP 提供无连接的传输服务；IP 在网络层提供无连接的数据报服务，虚电路则在网络层提供面向连接的服务。面向连接不等于可靠：连接只是建立了路径和状态，可靠性还要靠确认、重传和序号来保证，这两件事要分开看。

1.7 本章在 408 中的位置

本章很少单独出大题，但它是后面五章的共同语言。选择题常考：分层职责、PDU 名称、设备所在层、协议三要素、发送时延与传播时延的辨析；计算题常考：分组交换总时延、时延带宽积、有效数据率、利用率和时延。把 $d_{\text{trans}} = L/R$ 和 $d_{\text{prop}} = D/v$ 的自变量记牢，后面学停等协议、滑动窗口和拥塞控制时会反复用到。

拓展阅读：

- Kurose & Ross《计算机网络：自顶向下方法》第 1 章——分组交换与电路交换的定量比较。
- 谢希仁《计算机网络》第 1 章——国内教材的术语体系，与 408 的表述最接近。

2 习题

2.1 基础过关题

1. （单项选择题）下列关于网络协议与服务的叙述中，正确的是

(A) 协议是相邻层之间的垂直约定，服务是对等层之间的水平约定
(B) 第 n 层通过服务访问点（SAP）向第 $n+1$ 层提供服务，对等层之间则遵循第 n 层协议
(C)「请求」原语由服务提供者发往服务用户
(D) 第 n 层实体向第 $n-1$ 层实体提供服务

2. （单项选择题）在 OSI 参考模型中，下列哪一组的全部功能都由数据链路层完成?

(A) 成帧、差错控制、介质访问控制
(B) 路由选择、拥塞控制、网际互连
(C) 数据格式转换、加密解密、数据压缩
(D) 会话建立、同步、断点续传

3. （填空题）主机甲向主机乙发送 10^8 bit 的数据。链路带宽为 1 Gbps，甲乙相距 2000 km，信号在介质中的传播速率为 2×10^8 m/s，忽略处理时延与排队时延。（1）发送时延为_____ s；（2）传播时延为_____ s；（3）二者中占主导地位的是_____。

4. （单项选择题）主机甲与主机乙相距 1000 km，链路带宽 10 Mbps，信号传播速率 2×10^8 m/s，忽略处理时延与排队时延。甲向乙发送一个长度为 5000 bit 的分组，从甲发出该分组的第一个比特算起，到乙完整收到该分组所需的时间是

(A) 0.5 ms
(B) 5.0 ms
(C) 5.5 ms
(D) 50.5 ms

5. （填空题）应用层产生 1000 B 的数据，自上而下依次经过传输层（加 20 B 首部）、网络层（加 20 B 首部）、数据链路层（加 18 B 首部与 4 B 尾部），物理层不再增加开销。以数据链路层帧长为分母计算传输效率。（1）数据为 1000 B 时，效率约为_____；（2）数据增大到 9000 B（各层开销不变）时，效率约为_____。

6. （单项选择题）下列关于 TCP/IP 模型的叙述中，错误的是

(A) 应用层对应 OSI 的应用层、表示层和会话层

(B) 网络接口层对应 OSI 的数据链路层和物理层

(C) 网际层提供无连接、不可靠的数据报服务

(D) 与 OSI 一样，TCP/IP 严格区分服务、接口和协议

2.2 真题风格与综合训练

7. （单项选择题）主机甲经一台路由器（存储转发）与主机乙相连，两段链路的数据传输速率均为 10 Mbps。甲分别用报文交换和分组交换向乙发送一个 2×10^6 bit 的报文，分组长度为 2000 bit。忽略链路传播时延、排队与处理时延，也忽略分组首部开销与拆装时间。两种方式完成该报文传输所需的时间分别为

(A) 200 ms，200 ms　(B) 400 ms，200.2 ms

(C) 400 ms，200 ms　(D) 400 ms，400.2 ms

8. （单项选择题）主机甲与主机乙之间经过一台二层交换机（不做路由）和一台路由器。数据从甲的应用层发出、由乙的应用层接收。下列叙述中正确的是

(A) 每经过一台路由器，数据链路层首部与尾部都要重新封装

(B) 每经过一台中间设备，网络层首部都要重新封装

(C) 传输层首部在每一跳都会被重新封装

(D) 物理层在转发时依据帧尾部的 FCS 决定是否丢弃该帧

9. （综合应用题）主机甲与主机乙相距 2000 km，链路带宽为 100 Mbps，信号传播速率为 2×10^8 m/s。甲采用停止-等待协议向乙发送数据，每个数据帧长 1000 B（1 B = 8 bit），确认帧长度可忽略；忽略处理时延与排队时延。

   (1) 求单程传播时延 τ 与往返时间 RTT；

   (2) 求一个数据帧的发送时延 T_d；

   (3) 求停止-等待协议下按有效数据计算的平均吞吐量与信道利用率；

   (4) 若改用滑动窗口协议连续发送，为使链路不因等待确认而空闲，发送窗口至少应有多少个帧？（写明计数口径）

10. （综合应用题）主机甲经两台路由器 R1、R2 与主机乙相连，三段链路速率均为 10 Mbps，每段链路的传播时延均为 2 ms。甲把数据拆成 500 个分组，每个分组长 1000 bit，其中首部占 40 bit。忽略处理时延、排队时延与确认开销。

   (1) 求这批分组携带的有效数据总比特数；

   (2) 求从甲发出第一个比特到乙收到最后一个比特所需的时间（写出流水计数口径）；

   (3) 以有效数据计算平均吞吐量（保留三位有效数字）；

   (4) 求首部开销占总传输比特的比例，并说明它与 (3) 中吞吐量的关系。

3 物理层

物理层在协议栈的最底下。它从数据链路层接过一串比特，把这些比特变成电信号、光信号或电磁波送进介质，再在另一端还原成同样的比特。至于这些比特组成什么帧、帧头里的地址是什么意思，物理层不解释。

定义 3.1: 物理层

物理层为数据链路层提供比特流传输服务：发送端把比特映射成适合介质传输的信号，接收端把信号还原成比特。它不识别帧边界，也不检查比特的含义。

物理层的接口标准通常从四个方面规定，考试里称为机械特性、电气特性、功能特性和规程特性。拿常见的 RJ-45 网口和 RS-232 串口对照，就很容易分清这四项。

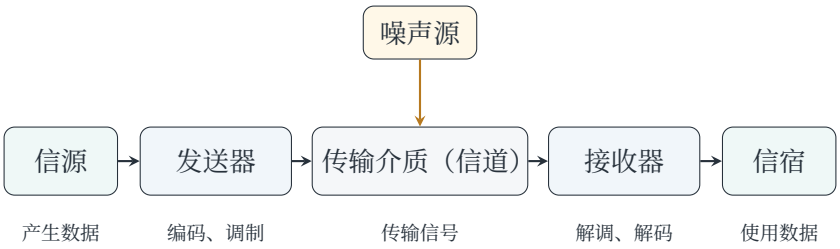
物理层接口的四个特性

特性	规定什么	例子
机械特性	连接器的形状、尺寸、引脚数、引脚排列、插拔和锁定方式	RJ-45 有 8 个引脚，水晶头有固定的卡扣形状
电气特性	电压范围、阻抗、传输速率、传输距离	RS-232 用 $-15 \sim -3$ V 表示 1，$+3 \sim +15$ V 表示 0
功能特性	每根引脚上信号的含义	RS-232 的 DTE 引脚 2 收数据，引脚 3 发数据
规程特性	各信号出现的先后顺序和握手过程	先发请求，收到允许后再发数据

物理层只认比特。集线器收到什么电平，就整形复制到其他端口，不会拆开帧看地址；交换机要读出 MAC 帧头，按目的地址转发，所以交换机工作在数据链路层。这个边界在后面讲设备时还会用到。

3.1 数据通信的基本量

两个人打电话，声音先被话筒变成模拟电信号，经过电话线送到对方听筒。把中间每一段拆开，就是数据通信系统的基本模型。



信源和信宿是数据的起点和终点。发送器把数据变成适合介质的信号，接收器做反变换。噪声在信道里加入，是接收端判错比特的主要原因。

数据是信息的实体，信号是数据在介质上的表现形式。同一个数据可以用不同信号传：数字 1 可以是一段高电平，也可以是一段高频振荡。模拟信号的取值连续，数字信号的取值离散。数字信号仍然有波形，只是它只取有限个电平，不能把它和“比特”画等号。

按信号的频谱位置，传输分两种。基带传输不搬移频谱，直接把数字波形送上介质，以太网双绞线就是这样。频带传输先把数字信号调制到高频载波上，再送上介质，电话线拨号上网、WiFi 都属于这一类。

按双方能否同时发送，通信方式分三种。

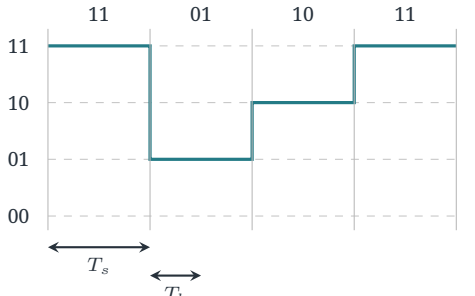
单工、半双工与全双工

方式	方向	典型例子
单工	只能固定一个方向发送	广播、电视
半双工	双方都能发，但同一时刻只能一方发	对讲机、共享式以太网
全双工	双方可以同时收发	电话、交换式以太网

描述传输速率有两个量。码元速率也叫波特率，指每秒传多少个码元，单位是 Baud。比特率指每秒传多少比特，单位是 bit/s。一个码元是持续时间为 T_s 的一个波形；如果这个波形有 V 种可区分的状态，一个码元就携带 $\log_2 V$ 比特。于是

$$B = \frac{1}{T_s}\ \text{(Baud)}, \qquad R = B\log_2 V\ \text{(bit/s)}.$$

比如 4 种电平，每种电平对应 2 bit，波特率和比特率就差一倍。



4 种电平，每个码元带 $\log_2 4 = 2$ bit，故 $R = 2B$

看这张图：一个码元占 T_s，一个比特只占 $T_b = T_s/2$。电平种类越多，同样波特率下比特率越高，但接收端要把相邻电平分得更开，对噪声和失真也更敏感。

3.2 信道能传多快：奈奎斯特与香农

一条理想低通信道，带宽为 W，最多每秒传多少个码元？奈奎斯特的回答是 $2W$。带宽限制的信号最快只能以大约半个周期的间隔变化；码元再密，相邻码元的波形会拖尾重叠，接收端就分不开了。这个上限与码元取几种电平无关。

结论 3.1: 奈奎斯特准则（理想无噪声低通信道）

$$B_{\max} = 2W \text{ (Baud)}, \qquad C_{\max} = 2W \log_2 V \text{ (bit/s)}.$$

其中 W 是信道带宽，单位 Hz；V 是一个码元可取的离散电平数。$2W$ 是码元速率上限，乘上每码元比特数才是数据率上限。

例如 $W = 6$ kHz、$V = 8$，则码元速率最多 12000 Baud，数据率最多 $12000 \times 3 = 36$ kbps。这个结果默认信道没有噪声、接收端能完美区分 8 种电平。实际信道做不到时，电平数要降下来。

有噪声时，电平不能取得太密，否则噪声会把相邻电平推到判决边界另一边。香农定理给出另一个上限：不管用什么编码、多少电平，信息速率都不能超过

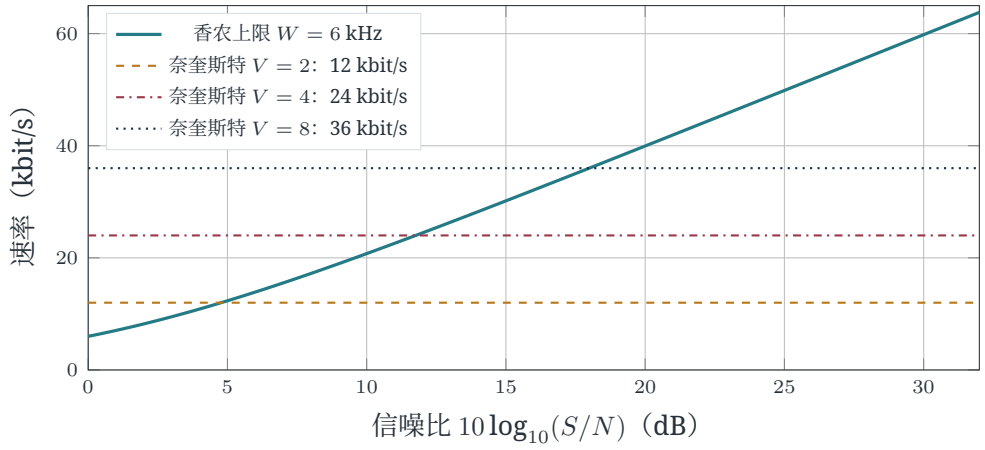
$$C = W \log_2 \left(1 + \frac{S}{N}\right) \text{ (bit/s)}.$$

S/N 是信号平均功率与噪声平均功率之比，是一个没有单位的比值。题目通常给分贝值，要先换算：

$$\left(\frac{S}{N}\right)_{\text{dB}} = 10 \log_{10} \frac{S}{N}, \qquad \frac{S}{N} = 10^{(S/N)_{\text{dB}}/10}.$$

香农上限是存在性上限。它说明“存在某种编码可以逼近这个速率”，但不说具体怎么编。实际系统用纠错码和更高阶的调制去逼近它。

两个上限同时存在时，实际速率取较小者。奈奎斯特管的是码元能排多密，香农管的是在噪声下每个码元能可靠携带多少信息。噪声大时香农先卡住，噪声小时奈奎斯特的 $2W$ 先卡住。

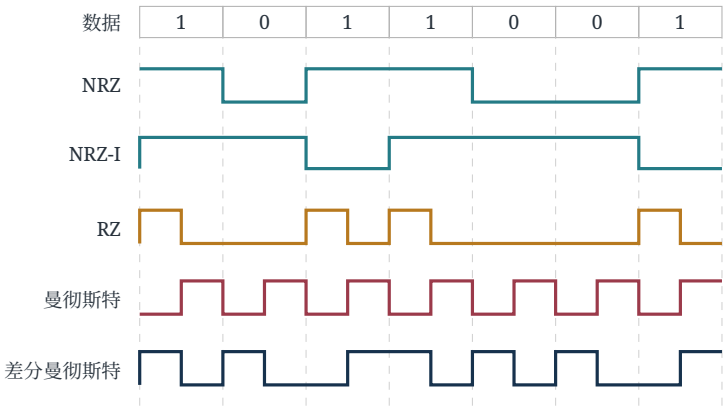


图中那条上升的曲线是香农上限，水平线是不同电平数下的奈奎斯特上限。以 $V = 8$ 为例，信噪比低于约 18 dB 时曲线在 36 kbit/s 下方，噪声是瓶颈；高于这个值时曲线抬到 36 kbit/s 以上，$2W$ 成了瓶颈。实际能用的速率就是曲线和对应水平线中较低的那条。

> **注意**
>
> 三个常见错误：把分贝值直接代进香农公式；把 $2W$ 当成比特率；把采样定理的 $2f_{\max}$ 和奈奎斯特准则的 $2W$ 混为一谈。前者用于模拟信号采样，后者用于信道码元速率，虽然都挂着奈奎斯特的名字，却不是一回事。

3.3 数字数据的编码

假设要发送比特串 1011001。最直接的办法是高电平表示 1、低电平表示 0，这就是不归零编码（NRZ）。接收端按约定的时钟逐位采样。问题是，如果连续出现很多个 1，电平一直不变，接收端的时钟稍微偏一点，就可能多算或少算一位。为了让接收端能自己从信号里提取时钟，有了带跳变的编码。



这五行对应同一条比特串 1011001。竖虚线是比特边界，每行只改变电平，不改变码元的起止时刻。看曼彻斯特和差分曼彻斯特两行，每一位的中间都有一次跳变；NRZ 和 NRZ-I 在连续相同比特处没有跳变。

不归零编码的规则最简单，一个比特周期内电平保持不变。它的频带利用率高，但长时间没有跳变，接收端无法从信号中提取时钟，而且连续相同比特会积累直流分量。NRZ-I 把数据放在“有没有跳变”上：每个比特开始时，遇到 1 就翻转，遇到 0 就保持。这样极性反接不影响译码，但连续多个 0 仍然没有跳变。

归零编码（RZ）让每个比特中间回到零。一种常见约定是：1 用前半比特的高电平表示，后半比特回到零；0 在整个比特内保持零。它自带时钟信息，但脉冲变窄，占用带宽约为 NRZ 的两倍，编码效率低。

曼彻斯特编码把跳变放在每个比特的正中间，这次跳变既当时钟又当数据。IEEE 802.3 的约定是“高到低表示 0，低到高表示 1”，有的教材反过来写，做题时以题干的约定为准。因为一个比特被拆成两个半比特的码元，曼彻斯特的波特率是数据率的两倍，编码效率只有 50%。10BASE-T 以太网用的就是它。

差分曼彻斯特编码也保证每个比特中间有跳变，但中间的跳变只负责同步；数据放在比特起始处有没有跳变上。常用约定是“起始处有跳变表示 0，无跳变表示 1”。它不依赖电平的绝对极性，抗干扰能力比曼彻斯特更强，令牌环网用过这种编码。

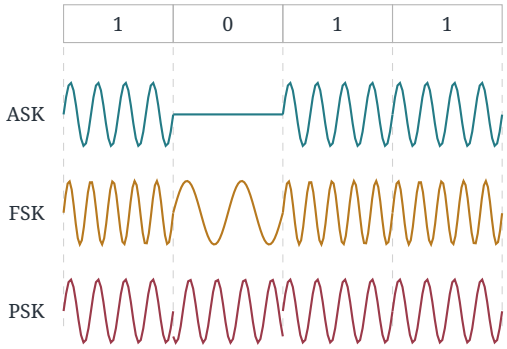
几种数字编码的对比

编码	同步方式	带宽或波特率	主要特点
NRZ	需外部时钟	最窄	规则简单，长连相同比特会失步
NRZ-I	靠跳变	较窄	遇 1 翻转、遇 0 保持，抗极性反转
RZ	自带时钟	约为 NRZ 的 2 倍	每位中间回零，利于同步，效率低
曼彻斯特	自带时钟	波特率 $=2\times$ 数据率	每位中间必跳变，10BASE-T
差分曼彻斯特	自带时钟	波特率 $=2\times$ 数据率	起始处跳变表示数据，抗干扰强

还有一类做法是把 m 位数据映射成 n 位码字，用冗余码字保证码流中有足够的跳变。比如 4B/5B 把 4 bit 映射成 5 bit，再用 NRZI 或 MLT-3 发送；100BASE-FX 用 4B/5B 加 NRZI，100BASE-TX 用 4B/5B 加 MLT-3。它不属于上面几种“逐位波形”编码，但目的相同：让接收端容易恢复时钟。

3.4 数字数据的调制

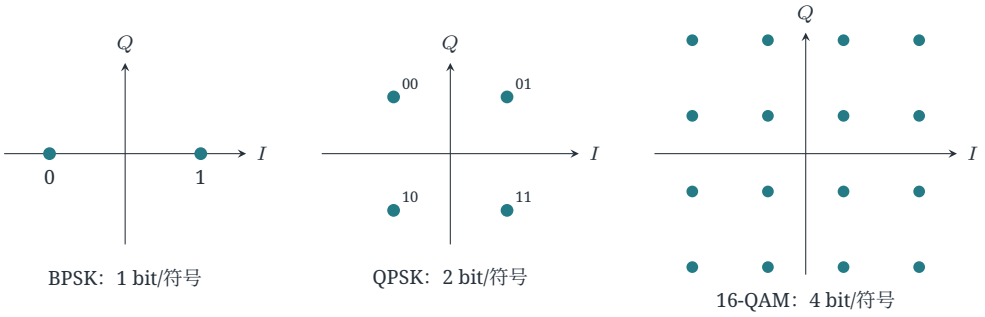
电话线只能传 300–3400 Hz 左右的模拟信号。要把数字比特送上去，就得把比特“骑”到一个高频载波上，这叫调制；接收端把载波上的变化还原成比特，叫解调。完成这两件事的设备是调制解调器（Modem）。



三行发送同样的比特 1011，载波都工作在同一个频段。ASK 改变振幅，常见的形式是有载波表示 1、没有载波表示 0，也叫 OOK。FSK 改变频率：1 用高频，0 用低频。PSK 改变相位：1 和 0 的波形相位差 $180°$，振幅和频率都不变。

ASK 实现简单，但抗噪能力差，衰落和干扰都容易被误判成幅度变化。FSK 的抗噪能力比 ASK 好，实现也不复杂，低速调制解调器用过。PSK 用相位携带信息，抗噪能力更强。QPSK 取 4 个相位，每个码元带 2 bit；8-PSK 取 8 个相位，每个码元带 3 bit。

QAM 同时改变振幅和相位，把星座点排在 I/Q 平面上。16-QAM 有 16 个点，每个码元带 4 bit；64-QAM 有 64 个点，每个码元带 6 bit。阶数越高，同样带宽下数据率越高，但星座点之间越挤，噪声稍大就会判错。



每个点是一种可以发送的载波状态。BPSK 只有两个点，判决时只需要看 I 的正负；QPSK 有四个点，每个点对应两位；16-QAM 排成 4×4 的网格，每个点对应四位。点与点之间越密，同样的噪声下越容易跨过判决边界，所以高阶 QAM 需要更好的信噪比。

3.5 模拟信号的数字化：PCM

声音、温度这类模拟量要进计算机，先得变成数字比特。电话网里用的是脉冲编码调制（PCM），分采样、量化、编码三步。

采样是每隔 T_s 取一个瞬时值。采样频率 $f_s=1/T_s$ 必须不低于信号最高频率 $f_{\max}$ 的两倍：

$$f_s \geq 2f_{\max}.$$

这叫采样定理。如果采样太慢，高频分量会折叠到低频，形成混叠，后面再怎么处理也恢复不回来。电话语音的最高频率按 4 kHz 算，所以采样频率取 8 kHz。

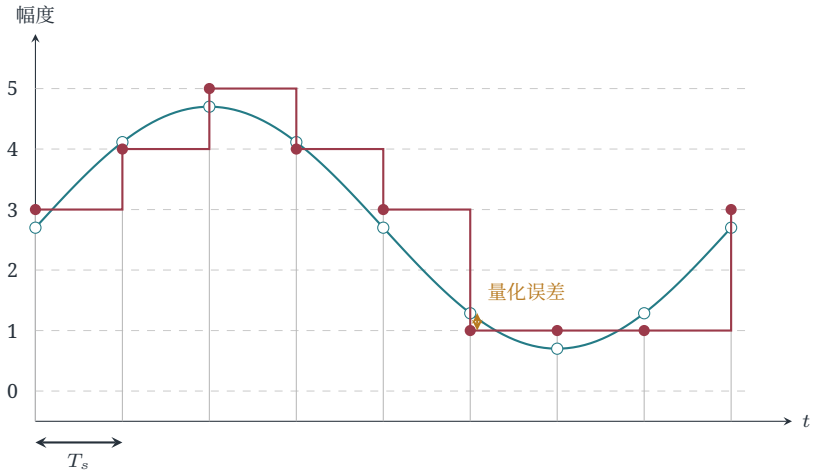
量化是把采样值的幅度范围分成有限个等级。若量化位数是 n，就有 2^n 个等级。采样值落到哪一级，就取哪一级的中心值。量化会带来误差，误差最大为半个量化间隔 $\Delta/2$。等级越多，误差越小，但每个样本占的比特也越多。

语音的小信号出现得比大信号频繁。如果对大信号和小信号用同样的步长，小信号的相对误差会很大。A-law 和 μ-law 用非均匀量化：小信号步长小，大信号步长大，相当于先对幅度做压缩再均匀量化。

编码是把每个量化级编成 n 位二进制码。一路 PCM 的速率是

$$R = f_s \times n \ \text{(bit/s)}.$$

电话语音用 8 kHz 采样、8 bit 量化，得到 $8000\times8=64$ kbps。



连续曲线是原始模拟信号，空心圆是采样点，水平虚线是量化级，实心点是量化后的值，阶梯线是量化器的输出。采样点落在两级之间时，量化器取最近的一级，差出来的那一小段就是量化误差。

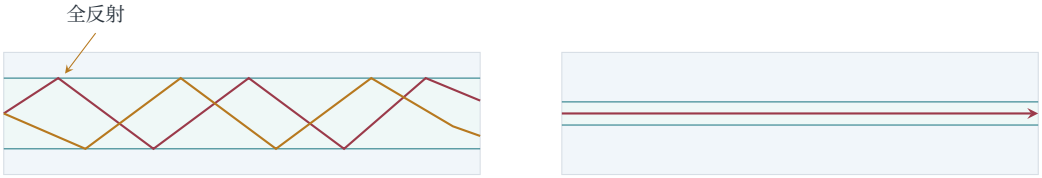
3.6 传输介质

介质决定信号以什么形式、跑多远、受什么干扰。局域网和广域网常用的有线介质有双绞线、同轴电缆和光纤，无线介质则靠电磁波在空间传播。

常用传输介质

介质	结构	优点	局限与场景
双绞线	两根绝缘铜线绞合，多对包在同一护套里	成本低、布线容易，绞合能抵消一部分干扰	距离短，典型网段 100 m；易受强电磁干扰；局域网接入
同轴电缆	内导体、绝缘层、金属屏蔽层、外护套	屏蔽好，抗干扰能力通常强于非屏蔽双绞线	笨重、成本较高；有线电视、早期总线以太网
光纤	纤芯、包层、涂覆层	带宽大、损耗低、抗电磁干扰、保密性好	接头和施工成本高，不能过度弯折；骨干网、长距离
无线	无线电波、微波或红外	无需布线，支持移动	受遮挡和干扰，频谱需要管理；无线局域网、移动通信

光纤利用全反射传输。光从折射率大的纤芯射向折射率小的包层，入射角大于临界角时，光全部反射回纤芯，不会漏到包层里。光纤的芯径和模式数决定了它的传输特性。



(a) 多模：多条路径，到达时间不同　　(b) 单模：近似一条路径，模间色散小

多模光纤的芯径较大，典型为 50 或 62.5 μm，光可以在里面走多条路径。路径长短不同，到达终点的时间就不同，形成模间色散。距离越长、速率越高，前后脉冲重叠得越厉害，所以多模光纤适合机房内和园区内的短距离连接。单模光纤的芯径只有 8–10 μm 左右，只允许一个模式传播，模间色散很小，配合激光器可以传几十公里，用于骨干网和长距离线路。

双绞线便宜、好布线，是局域网接入的主力。它的两根线绞在一起，外界干扰在两根线上感应出相近的电压，接收端取差值时就把干扰减掉了大半。同轴电缆的金属屏蔽层把内导体包起来，抗干扰能力更强，但线粗、接头麻烦，现在主要用有线电视。光纤不受电磁干扰，也不容易窃听，缺点是接头和光电转换器件贵，弯折半径不能太小。

3.7 信道复用

一条物理线路的带宽通常远大于一路信号需要的带宽。复用就是让多路信号共享同一条线路，同时不互相干扰。按划分资源的维度，常见的有频分、时分、波分和码分。

频分复用（FDM）把线路的频带切成若干段，每路占一段，段与段之间留保护带，防止相邻信道串扰。所有用户同时发送，但各占不同频率。广播、有线电视、ADSL 都是这个思路。

时分复用（TDM）把时间切成帧，每帧再切成时隙，每路固定占一个时隙，轮流发送。同步 TDM 的时隙是事先分好的，某一路没有数据时，它的时隙也空着，线路利用率不高。统计时分复用（STDM）按需分配时隙，有数据才占，代价是每个时隙要带地址或标识，接收端才能判断这一段属于哪一路。

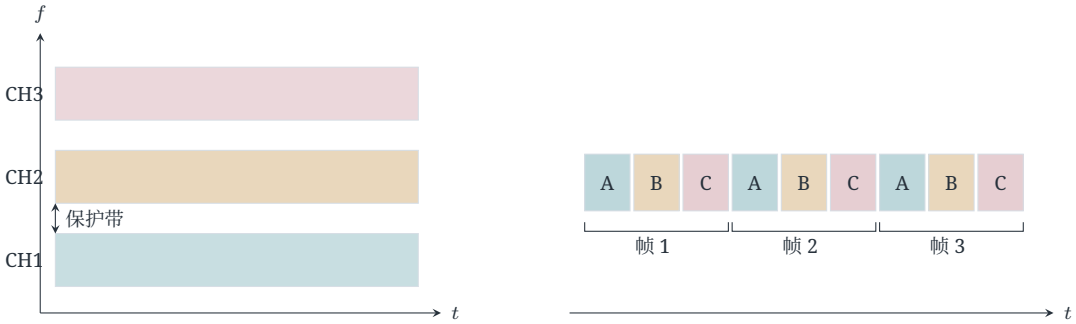
波分复用（WDM）是光纤上的频分复用。不同波长的光载波各自携带一路信号，用合波器合成一束送进光纤，末端再用分波器分开。密集波分复用（DWDM）把波长间隔做得很小，一根光纤可以同时传几十路甚至上百路。

码分复用（CDMA）让所有用户在同一时间、同一频带内发送，靠相互正交的码片序列区分。每个比特用 m 个码片表示，码片速率是比特率的 m 倍。发送比特 1 时发自己的码片序列，发送比特 0 时发反码。接收端用自己的码片序列做内积，再除以码片数 m：本用户的信号得到 $+1$ 或 -1，其他正交用户的贡献为 0。

比如四个站的码片序列取

$$A=(1,1,1,1),\quad B=(1,-1,1,-1),\quad C=(1,1,-1,-1),\quad D=(1,-1,-1,1).$$

任意两个不同序列的内积都是 0。把四路信号叠加后，接收端只要和某个站的码片序列做内积，就能把该站的数据分离出来，其他站的信号在运算中抵消。408 常给 8 位码片序列，做法一样，只是内积多算几项。



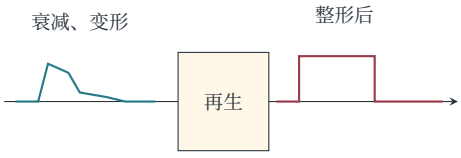
(a) FDM：按频率划分，各路同时　　(b) TDM：每路固定时隙，轮流发送



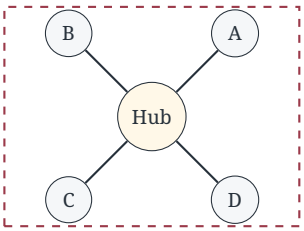
左图 (a) 是频分复用：三路信号同时存在，各占一段频率，中间的空隙是保护带。(b) 是时分复用：三路轮流占时隙，每帧的排列固定为 A、B、C。最下面一行是统计时分复用，时隙按需分配，哪一路有数据就占一个，所以排列不再固定。

3.8 中继器与集线器

信号在介质里传得越远越弱，波形也会变圆。中继器把收到的信号重新整形、放大，再发出去。注意它是“再生”，不是简单地把模拟波形放大：它按判决门限重新产生 0 和 1，所以能把已经变形的信号恢复干净。



(a) 中继器重新产生比特



(b) 所有端口在同一个冲突域

图 (a) 左边是经过长线后幅度变小、边沿变缓的脉冲，经过中继器后重新变成干净的方波。(b) 中四个站都连在同一个集线器上，集线器把一个端口收到的比特复制到其他所有端口，四个站处于同一个冲突域，同一时刻只能有一个站发送。

中继器只有两个端口，用来延长一个网段。集线器本质是多端口中继器：从一个端口收到的比特，整形后复制到所有其他端口。它不识别 MAC 地址，不缓存帧，也不隔离冲突域和广播域。所有端口共享同一段带宽，多个站同时发送就会冲突，所以共享式以太网要用 CSMA/CD。

交换机不一样。交换机读出 MAC 帧头，按目的地址把帧只转发到对应端口，每个端口是一个独立的冲突域。交换机仍然不隔离广播域，广播帧会从所有端口发出去，除非划分了 VLAN。

10 Mb/s 共享式以太网用集线器组网时，要遵守 5-4-3 规则：任意两个站之间最多经过 5 个网段、4 个中继器或集线器，其中最多 3 个网段可以连接计算机。中继器和集线器都不翻译协议，两端的数据率、编码方式必须一致。

最后把几个常考数字放在一起：电话语音按 4 kHz 带宽、8 kHz 采样、8 bit 量化，PCM 速率为 $8 \times 8 = 64$ kbps；10BASE-T 用曼彻斯特编码，数据率 10 Mb/s，波特率 20 MBaud；双绞线网段典型最大长度 100 m；单模光纤芯径约 8–10 μm，多模光纤约 50 或 62.5 μm。

4 习题

4.1 基础过关题

1. (填空题) 某无噪声低通信道的带宽为 4 kHz，采用 8-PSK 调制，即用 8 种不同相位表示数据。(1) 该信道的极限码元传输速率为_____ Baud；(2) 该信道的极限数据传输速率为_____ kbps。

2. (填空题)某有噪声信道的带宽为 5 kHz,信号平均功率与噪声平均功率之比 $S/N = 255$。(1)按香农定理，该信道的极限数据传输速率为_____ kbps；(2) 该信噪比约为_____ dB。

3. (单项选择题) 某 10BaseT 网卡采用曼彻斯特编码，并约定比特中间的跳变「高电平到低电平」表示 0、「低电平到高电平」表示 1（与某些教材的约定相反）。现收到连续 8 个码元，其比特中间的跳变方向依次为：高 → 低、低 → 高、低 → 高、高 → 低、高 → 低、低 → 高、高 → 低、低 → 高。该网卡收到的比特串是

(A) 01100101　(B) 10011010

(C) 01101001　(D) 01011010

4. (单项选择题) 关于传输介质，下列说法正确的是

(A) 单模光纤芯径较细，只传输一个模式，模间色散小，适合长距离传输

(B) 多模光纤只能使用激光器作光源，传输距离比单模光纤更远

(C) 同轴电缆的抗电磁干扰能力弱于非屏蔽双绞线

(D) 光纤依靠全反射传输，因此可弯曲任意角度而不产生损耗

5.（单项选择题）关于信道复用，下列说法正确的是

(A) FDM 中各子信道占用不同的时间片，适合传输突发性数据

(B) 同步 TDM 中每个子信道固定占用一帧中的某个时隙，用户空闲时该时隙仍被保留、不能给他人使用

(C) WDM 与 FDM 原理无关，是光纤中独有的复用方式

(D) CDM 中不同用户占用不同频带，接收端靠带通滤波器分离各路信号

6.（单项选择题）关于编码与调制，下列说法正确的是

(A) NRZ 编码在每个比特周期内电平都要回到零，因而具有自同步能力

(B) 曼彻斯特编码在每个比特中间都有跳变，该跳变兼作时钟信号，其码元速率是数据率的 2 倍

(C) 差分曼彻斯特编码用比特中间的跳变方向表示 0 和 1，抗干扰能力弱于曼彻斯特编码

(D) QPSK 用 4 种相位表示数据，每个码元携带 4 bit

4.2 真题风格与综合训练

1.（单项选择题）某信道带宽为 W，信噪比为 20 dB。若要求「无噪声条件下按奈奎斯特准则算出的极限数据速率」不低于「该信道的香农极限」，则每个码元至少应取多少个离散电平？

(A) 8　(B) 10

(C) 11　(D) 32

2.（单项选择题）某电缆的总可用频带为 108 kHz，用 FDM 划分为 8 个等宽子信道，相邻子信道之间留 4 kHz 保护带（共 7 段）。每个子信道采用 4 电平基带传输，且视为无噪声信道。每个子信道能提供的最大数据传输速率是

(A) 20 kbps (B) 27 kbps

(C) 40 kbps (D) 54 kbps

3. (综合应用题) 某电话信道带宽 $W = 3$ kHz，信噪比 30 dB，噪声为加性高斯白噪声。

   (1) 求该信道的香农极限（保留到 0.1 kbps），并说明该值是什么意义上的上限；

   (2) 若只用 16-QAM 达到这一速率，需要的码元传输速率是多少 Baud?

   (3) 判断 (2) 中的码元速率能否实现，若不能，求至少需要多少个离散电平才能逼近香农极限。

4. (综合应用题) 某同步 TDM 系统把 12 路 64 kbps 的数字话音复用到一条线路上：每帧依次为 12 路各安排 1 个时隙，每时隙 8 bit；帧尾再附加 8 bit 同步码。线路采用 4 电平基带传输，视为无噪声。

   (1) 求帧长（bit）与帧重复频率（帧/秒）；

   (2) 求线路的数据传输速率；

   (3) 求该复用系统的信道利用率；

   (4) 求线路所需的最小带宽。

拓展阅读：

- Kurose & Ross 第 1 章——奈奎斯特与香农公式的直观解释。
- 谢希仁第 2 章——物理层传输介质、编码与复用技术。

5 数据链路层

数据链路层把网络层交下来的分组封装成帧，再交给物理层发送。它只负责同一条链路两端的通信。这里的“链路”可以是一根电缆、一段光纤，也可以是一台主机到交换机、两台交换机之间的连接。中间经过交换机不算跨链路，经过路由器才算。

这一层要解决四件事：把比特流切成帧、发现传输错误、在共享介质上决定谁先发、在需要时提供可靠传输。以太网做帧定界、差错检测和介质访问控制，不保证可靠；PPP 做帧定界和差错检测，也不重传。408 的题目大多围绕以太网和 PPP 展开。

5.1 帧定界：从连续的比特里切出帧

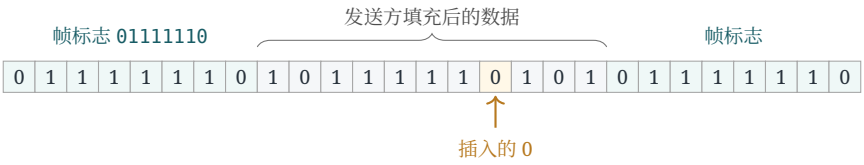
物理层送上来的是长长的 0/1 序列。接收方如果不知道一帧从哪里开始，就无法解释首部的地址、类型和尾部的校验字段。组帧要做的第一件事，就是让接收方能从比特流里认出帧边界。

最简单的办法是字符计数法：帧的第一个字节写入本帧的字符数。接收方先读这个数，再往后数相应多个字节。这个办法的弱点很直接：计数字段一旦出错，接收方就会按错误长度切分，后面的帧全部错位，而且很难重新同步。

更常用的是在帧前后放一个特殊标记。标记出现在数据里时，发送方要把它“转义”，接收方再还原。按处理单位不同，分为字节填充和零比特填充。

字节填充用于面向字节的链路。PPP 用 `0x7E` 作帧边界。数据中如果出现 `0x7E`，发送方发 `0x7D 0x5E`；如果出现转义字节 `0x7D`，发送方发 `0x7D 0x5D`。两个替换字节都等于原字节异或 `0x20`。PPP 默认还会转义小于 `0x20` 的控制字符，例如 `0x03` 变成 `0x7D 0x23`。接收方看到 `0x7D`，就把后一个字节异或 `0x20`，还原出原字节。

零比特填充用于面向比特的链路，HDLC 是典型例子。帧边界固定为 `01111110`，也就是六个连续的 1 两边各加一个 0。发送方在数据部分每遇到五个连续的 1，就插入一个 0；接收方每收到五个连续的 1，就删除后面的那个 0。这样数据里不可能出现六个连续的 1，边界标记就不会和普通数据混淆。



原数据有六个连续 1，发送方在第五个 1 后插入 0。
接收方看到五个连续 1 就删除后面的 0，数据还原为 `1011111101`。

图中左右两个青色块是帧标志，中间灰色块是数据，橙色位是发送方插入的 0。接收方每看到五个连续 1，就删掉后面的那个 0。

还有一种违规编码法。曼彻斯特编码用高低跳变表示 0 和 1，高-高、低-低属于非法波形。发送方可以用非法波形作帧边界，接收方看到它就认为一帧开始或结束。这个办法依赖物理层编码，通用性差，408 只需知道名字和适用条件。

5.2 差错检测与纠正

传输错误分两类。随机错误由噪声引起，表现为零星的比特翻转；突发错误由短暂干扰引起，表现为一串连续的比特出错。检错码只能发现错误，纠错码还能定位并改正错误。数据链路层常用的检错码是奇偶校验和 CRC，纠错码是汉明码。

奇偶校验。 一维奇偶校验在数据后加一位，使 1 的个数为奇数或偶数。它能发现奇数个比特错误，发现不了偶数个错误。二维奇偶校验把数据排成矩阵，每行每列都加校验位。某一位出错时，对应的行和列同时不满足校验，接收方就能定位这一位并翻转它。二维校验可以纠正一位错误，但两位错误可能互相掩盖。

CRC。 CRC 把数据看成一个二进制多项式，用生成多项式去除，把余数附在数据后面。接收方用同一个多项式去除整个码字，余数为 0 就认为没有检出错误。

要点 5.1: CRC 的发送码字

设数据比特串为 $M(x)$，生成多项式 $G(x)$ 的次数为 r。发送方先计算

$$R(x) = x^r M(x) \bmod G(x),$$

得到 r 位余数，再发送

$$T(x) = x^r M(x) + R(x).$$

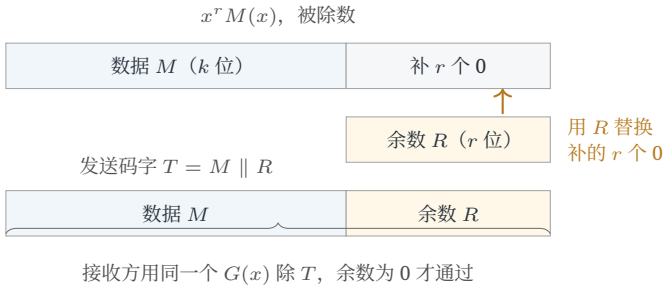
模 2 加法和减法都等于异或，不进位，也不借位。接收方计算 $T(x) \bmod G(x)$，结果为 0 表示没有检出错误。
$G(x)$ 的次数决定校验位数。例如 $G(x) = x^4 + x + 1$ 对应比特串 `10011`，次数为 4，余数就是 4 位。

下面用 408 常见的数据算一遍。取 $M =$ `1101011011`，$G =$ `10011`。G 的次数为 4，先在 M 后补四个 0，得到 `11010110110000`。用 `10011` 做模 2 除法，余数为 `1110`。发送方把余数替换掉补的四个 0，发送 `11010110111110`。接收方用 `10011` 除这个码字，余数为 0。

CRC 能检出什么错误，取决于 $G(x)$：

- $G(x)$ 至少有两项非零时，所有单比特错误都能检出。
- $G(x)$ 的常数项为 1，也就是 $G(0) \neq 0$ 时，所有长度不超过 r 的突发错误都能检出。

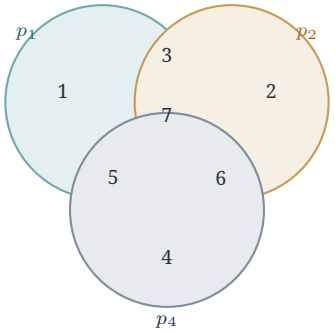
- $G(x)$ 含有因子 $x+1$ 时，所有奇数个比特错误都能检出。
- 如果错误图样恰好是 $G(x)$ 的倍数，余数仍为 0，这种错误检不出来。所以 CRC 不是“任何错误都能发现”。



图中上排是补了 r 个 0 的被除数，中间的余数替换掉这些 0，下排才是实际发送的码字。接收方用同一个 $G(x)$ 去除下排，余数为 0 就通过。

汉明码。 汉明码用多个校验位覆盖不同的数据位，使每个数据位至少被两个校验位覆盖。接收方重新计算各校验位，得到的故障字直接指出出错位置。以 (7,4) 汉明码为例，7 位码字中有 4 位数据、3 位校验。校验位放在第 1、2、4 位，数据位放在第 3、5、6、7 位。

三个校验圈分别记作 p_1、p_2、p_4。p_1 校验第 1、3、5、7 位，p_2 校验第 2、3、6、7 位，p_4 校验第 4、5、6、7 位。每个校验圈内所有位的异或规定为 0。



p_1 覆盖 1、3、5、7；p_2 覆盖 2、3、6、7；p_4 覆盖 4、5、6、7。
三个圈内各自异或为 0，数据位 3、5、6、7 被至少两个圈覆盖。

收到码字后，接收方分别计算三个圈的异或。三个结果组成故障字，为 0 表示没有检出错误；发生一位错误时，非 0 故障字的数值就是出错位的编号。例如故障字为 5，说明第 5 位出错，把它翻转即可。这个“故障字等于位置”的性质，来自校验位放在 1、2、4 这些 2 的幂的位置。

汉明码的纠错能力由最小汉明距离决定。码字之间最少有多少位不同，就记作 $d_{\min}$。要检出 e 位错误，需要 $d_{\min} \geq e+1$；要纠正 t 位错误，需要 $d_{\min} \geq 2t+1$。(7,4) 汉明码的 $d_{\min}=3$，可以纠正 1 位错误，检出 2 位错误。增加校验位可以提高纠错能力，但开销也更大。一般地，r 个校验位要覆盖 k 个数据位并指出错误位置，需要

$$2^r \geq k + r + 1.$$

这个不等式的右边多出的 1 表示“没有错误”这一种情况。

5.3 可靠传输：确认、超时与滑动窗口

差错检测只能发现问题。要让发送方知道帧有没有到达，还需要确认和重传。这类机制统称 ARQ（自动重传请求）。最基本的三个部件是：序号、确认帧、超时计时器。

停止-等待。 发送方发一帧，等确认；收到确认再发下一帧；超时没收到确认就重发。只用 1 位序号就能工作。序号为 0 的帧收到确认后，下一帧用 1；再下一帧又回到 0。接收方收到重复帧时丢弃，但必须再发一次确认，否则发送方的确认可能永远丢失。

设数据帧长度为 L，数据率为 R，单程传播时延为 τ。忽略确认帧的发送时间和处理时间，发送时延 $T_f = L/R$，停止-等待的信道利用率为

$$U = \frac{T_f}{T_f + 2\tau} = \frac{1}{1 + 2a}, \qquad a = \frac{\tau}{T_f}.$$

2τ 是往返传播时延。a 越大，等待确认浪费的时间越多，利用率越低。卫星链路时延大，停止-等待的效率很低，这也是滑动窗口出现的原因。

后退 N 帧（GBN）。 发送方可以连续发多个帧，不必每帧都停下来等确认。接收方只按序接收，收到失序帧就丢弃，并重复确认最后一个按序收到的帧。发送方为最早未确认的帧设置计时器，超时后从这一帧开始，把它和后面已经发过的帧全部重发。

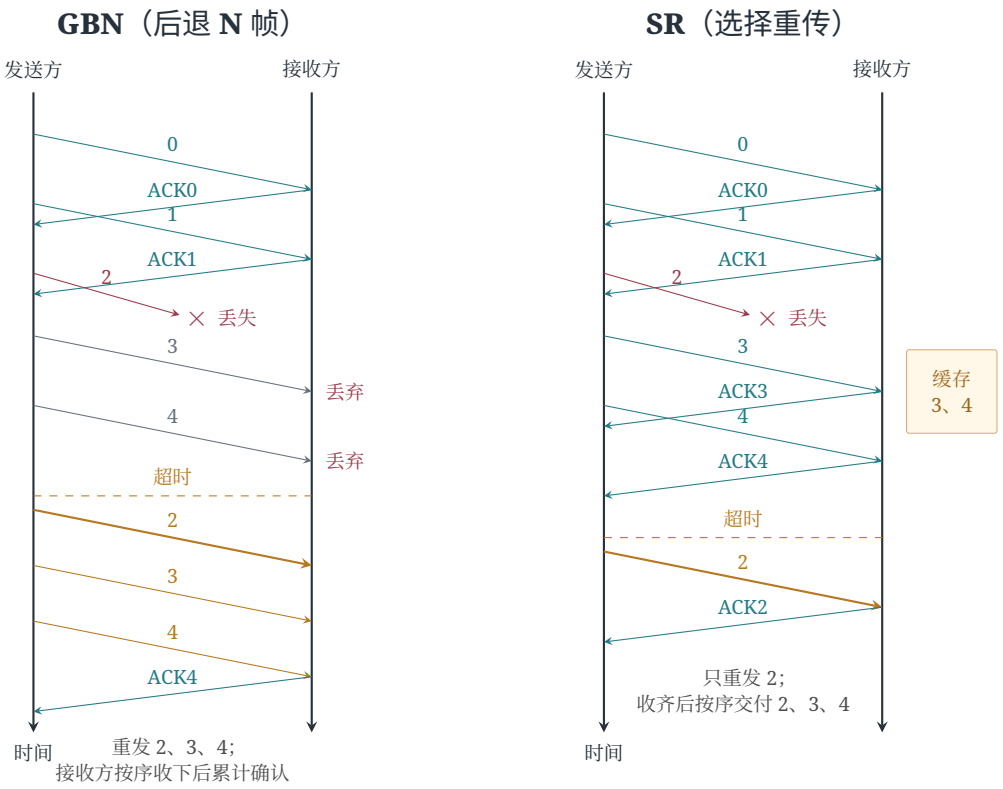
选择重传（SR）。 接收方设置缓存，把失序但正确的帧先存起来，并逐帧确认。发送方只重发超时或出错的帧，已经确认的帧不再重发。这样减少了重传量，代价是接收方要更多缓存，逻辑也更复杂。

三种可靠传输协议的窗口与重传

协议	发送窗口 W_T	接收窗口 W_R	超时后的动作
停止-等待	1	1	重发当前帧；1 位序号足以区分新旧帧
GBN	$1 \le W_T \le 2^m - 1$	1	从最早未确认帧开始，重发它及其后所有已发帧
SR	$W_T \le 2^{m-1}$	$W_R \le 2^{m-1}$	只重发超时或出错的帧，失序帧先缓存

表中 m 是序号位数，序号空间为 2^m。GBN 的上限是 $2^m - 1$，不能取 2^m。若取满 2^m，发送方把窗口内的帧全发完，确认又全部丢失，超时后重发第 0 帧；接收方此时正好也期待第 0 帧，就会把旧帧当成新帧。少一个序号，接收方就能分辨重复帧。

SR 通常取 $W_T = W_R = 2^{m-1}$。更一般的条件是 $W_T + W_R \le 2^m$。窗口超过一半序号空间时，新旧窗口会重叠，接收方无法判断一个帧是重传的旧帧还是新帧。



图里两条竖线分别是发送方和接收方，斜线表示帧或确认在链路上传播，向下表示时间推进。GBN 的接收方没有缓存，3、4 到得再对也会被丢掉；SR 的接收方先把它们存起来，所以发送方只需要补发 2。

5.4 介质访问控制

共享信道需要一套规则决定谁可以发。常见做法分三类：划分信道、随机接入、受控接入。

划分信道。 FDM 把频带切成几段，每个用户独占一段；TDM 把时间切成时隙，每个用户轮流使用；WDM 是光纤上的频分复用；CDMA 让所有用户同时同频发送，但每个用户使用不同的码片序列。CDMA 中，两个不同用户的码片序列内积为 0，同一序列与自身的内积为 1。接收方用某个用户的码片序列与收到的叠加信号做内积，就能提取该用户的数据。划分信道适合用户数固定、流量稳定的场景，用户空闲时隙会浪费。

随机接入。 所有站都可以随时发送,发生冲突后再想办法重发。ALOHA 是最早的方案：纯 ALOHA 想发就发，冲突后随机等待；时隙 ALOHA 只能在时隙边界开始发送，冲突窗口减半。设 G 为归一化负载，纯 ALOHA 的吞吐率为

$$S = Ge^{-2G},$$

最大值为 $1/(2e) \approx 0.184$。时隙 ALOHA 的吞吐率为

$$S = Ge^{-G},$$

最大值为 $1/e \approx 0.368$。时隙化把易冲突时间从两帧缩短到一帧，吞吐率翻倍。

CSMA 在发送前先监听信道，比 ALOHA 更礼貌。根据“信道忙时怎么办”，分为三种：

- 1-坚持：忙时继续听，一空闲就以概率 1 发送。冲突概率高，但信道利用率也高。

- 非坚持：忙时随机等一段时间再听。冲突少，但可能白白浪费空闲时间。
- p-坚持：空闲时以概率 p 发送，以 $1-p$ 推迟到下一个时隙。p 用来在冲突和空闲浪费之间折中。

载波监听不能消除冲突。信号在链路上传播需要时间，两个站可能同时听到“空闲”，然后同时开始发送。

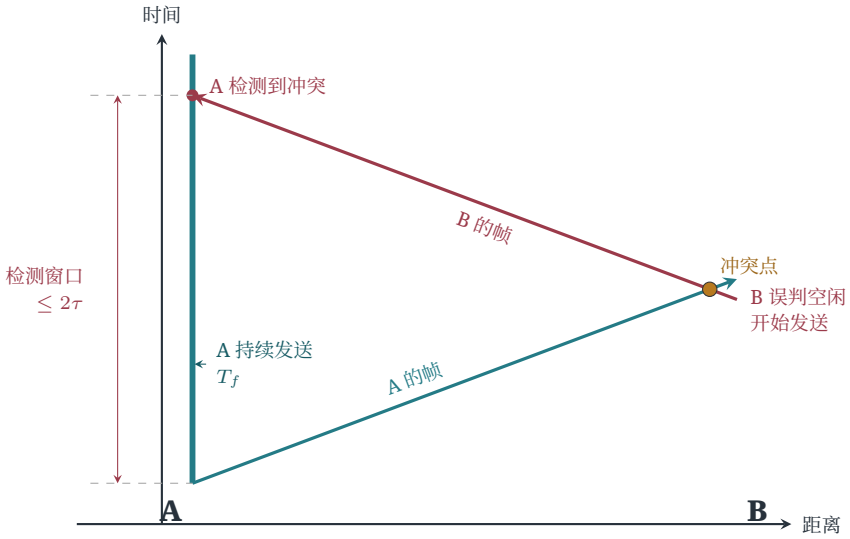
受控接入。 轮询由主站依次询问从站有没有数据；令牌传递让令牌在环上轮转，只有拿到令牌的站才能发送。受控接入不会冲突，但主站故障或令牌丢失会影响整个网络。令牌环网 802.5 和令牌总线 802.4 是典型例子。

5.5 CSMA/CD：共享以太网如何抢信道

早期的以太网是多台主机接在同一根同轴电缆上，所有站共享一个信道。一个站发送时，其他站也能收到信号。两个站同时发送，信号会叠加、互相破坏，这就是冲突。CSMA/CD 在 CSMA 的基础上增加了冲突检测：发送过程中继续听，发现冲突就立刻停发。

工作过程可以记成四步：先听后发，边发边听，冲突停发，随机重发。发送前监听信道，空闲就发；发送时继续检测；检测到冲突就停止发送，并发送一段强化碰撞信号，让所有站都知道发生了冲突；然后按二进制指数退避算法等待一段时间，再重新尝试。

冲突检测的关键是时间。信号从 A 传到 B 需要 τ。最坏的情况是：B 在 A 的信号到达前的一瞬间判断信道空闲，开始发送。两个信号在 B 附近相撞，碰撞信号再传回 A。A 从开始发送到收到碰撞信号，最多经过 2τ。如果 A 在这段时间内没有检测到冲突，就可以认为发送成功。



A 在 $t=0$ 开始发送；B 在 A 的信号到达前开始发送；两帧在 B 附近相撞。碰撞信号回传到 A 不超过 2τ。A 必须还在发送，才能听到这个冲突。

图里横轴是链路位置，纵轴是时间。A 的帧斜线表示第一个比特从 A 传到 B；B 的帧斜线表示 B 发出的信号。两条斜线相交的位置就是冲突点。A 在冲突信号回来时还没有发完，所以能检测到冲突。如果帧太短，A 已经发完并停止监听，就会误以为发送成功。

由此得到最短帧长的条件：

$$T_f = \frac{L}{R} \geq 2\tau \quad \Longrightarrow \quad L_{\min} = 2\tau \cdot R.$$

R 是数据率。10 Mb/s 以太网的争用期 2τ 规定为 51.2 μs，也就是 512 个比特时间。代入得 $L_{\min} = 512\ \text{bit} = 64\ \text{B}$。这个 64 B 从目的 MAC 算到 FCS，不包括前导码。100 Mb/s 以太网的时隙仍是 512 个比特时间，但时间缩短为 5.12 μs；为了在更短的时间里完成往返，网段的最大距离必须相应减小。

二进制指数退避的做法如下。设一个帧已经发生了 n 次冲突，令

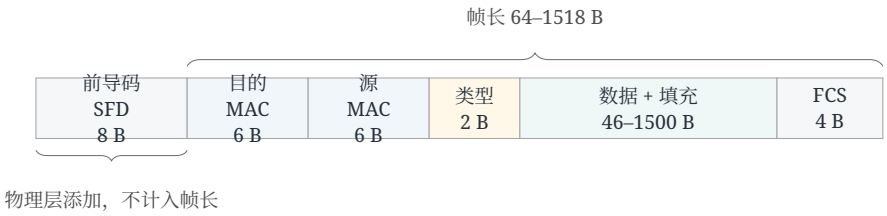
$$k = \min(n, 10),$$

从 $\{0, 1, \ldots, 2^k - 1\}$ 中随机选一个整数 r，等待 r 个争用期后再尝试发送。第一次冲突时 $r \in \{0, 1\}$，第二次冲突时 $r \in \{0, 1, 2, 3\}$。冲突次数越多，可选等待范围越大，各站错开重试时刻的概率越高。$n > 10$ 后范围不再增大，固定在 $[0, 2^{10} - 1]$。连续冲突 16 次仍不成功，就丢弃这个帧并向上层报告。

> **注意**
>
> **两个容易混的数**：争用期是 2τ，不是 τ；最短帧长是 64 B，也就是 512 bit，不是 512 B。计算时先把数据率换成 bit/s，把帧长换成 bit，再代入 $L_{\min} = 2\tau R$。另外，全双工交换式以太网的点对点链路不会冲突，不需要 CSMA/CD；但以太网帧的最小长度仍是 64 B。

5.6 以太网帧与 MAC 地址

以太网是目前最常用的局域网技术。它把数据链路层分成两个子层：上面的 LLC 子层处理逻辑链路控制，下面的 MAC 子层处理介质访问和帧的封装。408 常说的“以太网帧”一般指 DIX Ethernet V2 格式。



最短 64 B 来自 $6 + 6 + 2 + 46 + 4$；数据不足 46 B 时由填充字段补齐。
FCS 用 CRC-32，只检错，不重传。

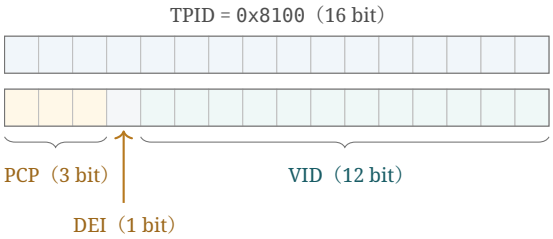
图中从左到右是前导码、目的 MAC、源 MAC、类型、数据和 FCS。括号标出的 64–1518 B 从目的 MAC 算到 FCS，前面的 8 B 不计入帧长。

目的 MAC 和源 MAC 各 6 B，共 48 位，通常写成 6 组十六进制数。全 1 地址 `FF-FF-FF-FF-FF-FF` 是广播地址。MAC 地址的第一位表示单播还是组播，第二位表示全球管理还是本地管理。网卡会接收发往自己 MAC 的帧、广播帧，以及自己加入的组播组的帧，其余帧丢弃。

类型字段说明上层协议。`0x0800` 表示 IP，`0x0806` 表示 ARP，`0x8100` 表示带 802.1Q 标签的帧。IEEE 802.3 的帧在这个位置放的是长度字段，并在数据部分放 LLC 首部。408 默认考 DIX V2 格式。

数据字段最少 46 B，最多 1500 B。最少 46 B 是为了让整个帧达到 64 B；数据不足时用填充字节补齐。最多 1500 B 就是常说的 MTU。超过 MTU 的 IP 分组要在网络层分片。

以太网提供无连接、不可靠的服务。发送前不建立连接，收到坏帧直接丢弃，不发送确认，也不重传。可靠性由上层协议负责。帧之间还要留出至少 96 个比特时间的帧间间隔，让接收方有时间处理上一帧。



4 B 标签插在源 MAC 之后、类型字段之前；
VID 取 1–4094，0 和 4095 保留。

图中上排是 16 位的 TPID，下排把 16 位分成 PCP、DEI 和 VID 三段。整个标签 4 B，插在源 MAC 与类型字段之间。

5.7　交换机与自学习

集线器工作在物理层，它把收到的比特直接复制到所有端口。所有端口在同一个冲突域里，同一时刻只能有一个站发送。交换机工作在数据链路层，它把帧收进来，检查目的 MAC，再决定从哪个端口转发。交换机的每个端口是一个独立的冲突域，端口之间可以同时通信。

交换机刚上电时，交换表是空的。它收到一个帧，先把帧的源 MAC 和入端口记下来，这就是自学习。然后查目的 MAC：

- 目的 MAC 在表里，且出端口不是入端口：只从该端口转发。
- 目的 MAC 在表里，且出端口就是入端口：说明目的主机和源主机在同一端口后面，丢弃这个帧。
- 目的 MAC 不在表里，或者是广播、组播地址：向除入端口外的所有端口泛洪。

表项有老化时间。主机长时间不发帧，表项会被删除，避免主机换端口后一直转发到旧端口。如果主机真的换了端口，交换机收到它从新端口发出的帧时，会用新的端口覆盖旧表项。



交换表快照
初始：空
收到 P→R：学习 P–1
收到 R→P：学习 R–3
收到 Q→P：学习 Q–2

端口号就是交换表里的出端口。
源 MAC 用来学习，目的 MAC 用来转发。

图中只画了一台交换机。P 第一次给 R 发帧时，交换机学到 P 在端口 1；R 的地址还不在表里，所以帧被泛洪到端口 2、3、4。R 回帧时，交换机学到 R 在端口 3，并查表知道 P 在端口 1，于是只从端口 1 转发。Q 给 P 发帧时，交换机学到 Q 在端口 2，同样只转到端口 1。

交换机有两种转发方式。存储转发把整个帧收完，检查 FCS 后再转发，坏帧不会进入下一段链路，但时延较大。直通转发只读到目的 MAC 就开始转发，时延小，但会把坏帧也传下去。现在的交换机大多默认存储转发。

交换机可以隔离冲突域，但不能隔离广播域。广播帧仍然会从所有端口发出去。要隔离广播域，需要 VLAN 或路由器。

不同设备隔离哪个域

设备	隔离冲突域	隔离广播域
中继器、集线器	否	否
网桥、交换机	是	否
路由器	是	是

交换机之间如果接成环路，广播帧会一直在环里打转，形成广播风暴。生成树协议（STP）通过阻塞部分端口，把有环的拓扑变成一棵无环的树。被阻塞的端口不转发数据帧，只保留在需要时接管的可能。

5.8 VLAN

VLAN 把一个物理交换网络划分成多个逻辑广播域。同一个 VLAN 内的主机可以直接二层通信；不同 VLAN 的主机即使接在同一台交换机上，也不能直接通信，必须经过路由器或三层交换机。

交换机用 802.1Q 标签区分 VLAN。标签 4 B，插在源 MAC 之后、类型字段之前。原来的帧加上标签后，最大长度从 1518 B 变成 1522 B。

端口分两种。Access 端口通常连接主机，只属于一个 VLAN，收发不带标签的普通以太网帧。Trunk 端口通常连接另一台交换机或路由器，承载多个 VLAN，帧在链路上带 802.1Q 标签。Trunk 上可以有一个本征 VLAN，它的帧不打标签，其余 VLAN 的帧都打标签。

VLAN 隔离的是广播域。一个 VLAN 内的广播帧不会转到另一个 VLAN。VLAN 不改变交换机每个端口是一个冲突域这一事实，也不提供路由功能。跨 VLAN 通信仍然要经过三层设备。

5.9 PPP

PPP 是点对点链路协议，常用于拨号上网、DSL 和 PPPoE。点对点链路只有两个端点，不需要 MAC 地址，也不需要介质访问控制。PPP 负责帧定界、差错检测、链路建立和网络层协议协商，但不提供可靠传输。



地址字段固定 `0xFF`，控制字段固定 `0x03`，两者可以协商压缩。
信息字段默认最大 1500 B，FCS 用 CRC-16。

图中从左到右依次是标志、地址、控制、协议、信息和 FCS。首尾两个 `0x7E` 之间是一帧，固定开销共 8 B。

PPP 的帧边界也是 `0x7E`。因为信息字段里可能出现 `0x7E`，PPP 在异步链路上使用字节填充：`0x7E` 变成 `0x7D 0x5E`，`0x7D` 变成 `0x7D 0x5D`。默认还会转义小于 `0x20` 的字符。接收方看到 `0x7D`，就把后一个字节异或 `0x20`，还原原字节。

PPP 的协议字段说明信息字段里装的是什么。`0x0021` 是 IP，`0xC021` 是 LCP，`0x8021` 是 IPCP。LCP 负责建立、配置、测试和关闭链路，还可以协商认证方式。PAP 把用户名和口令以明文发送；CHAP 用挑战-响应，不直接传口令。NCP 负责配置网络层协议，IP 对应 IPCP，用来协商 IP 地址、DNS 等参数。

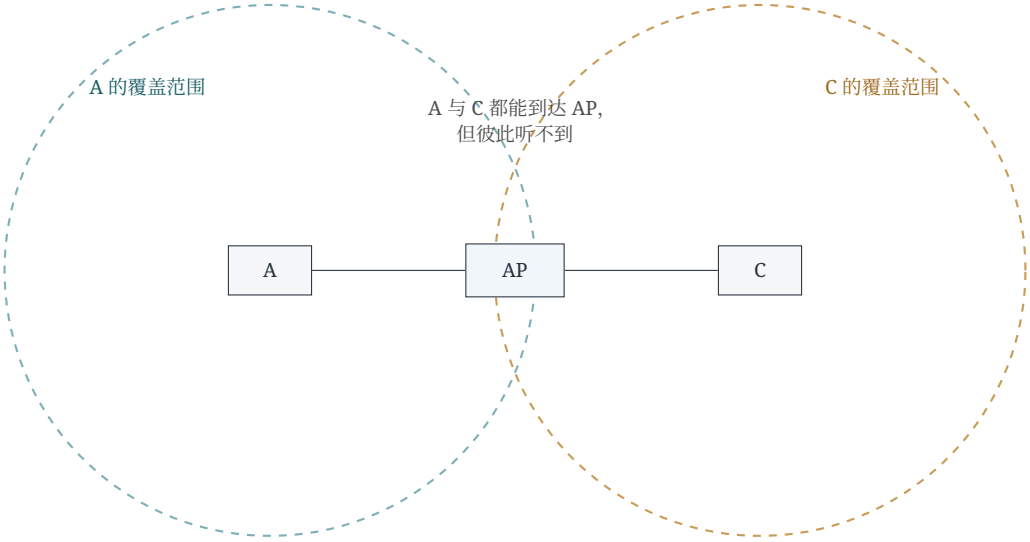
PPP 的数据帧不编号，不确认，也不重传。FCS 只用来发现错误，发现后丢弃帧，由上层协议决定要不要重发。PPP 也不做流量控制。它的“可靠”只体现在链路建立和认证过程，数据传输本身是不可靠的。

PPP 可以跑在以太网上，叫 PPPoE。PPPoE 在 PPP 帧前加 6 B 的 PPPoE 首部，再用以太网帧封装，常用于宽带拨号。这样既能使用以太网的多址链路，又能保留 PPP 的认证和会话管理。

5.10 无线局域网为什么不用 CSMA/CD

无线局域网使用 CSMA/CA，也就是冲突避免。原因有两个。第一，无线网卡通常半双工，发送时不能同时可靠地接收，边发边听很难做到。第二，无线信号的动态范围很大，发送时自己的信号会盖住远处微弱的冲突信号，检测不可靠。既然不能检测冲突，就只能尽量避开冲突。

无线还有一个有线网络没有的问题：隐藏站。两个站都能听到接入点 AP，但彼此听不到。它们可能同时判断信道空闲，然后同时向 AP 发送，在 AP 处发生冲突。发送方自己却检测不到这个冲突，因为冲突发生在 AP 附近。



A 先发 RTS，AP 回 CTS；C 收到 CTS 后推迟发送。
RTS/CTS 用短控制帧预约信道，降低隐藏站的冲突概率。

图中两个虚线圆是 A、C 的覆盖范围。它们在 AP 附近相交，但 A 和 C 都在对方圆外，所以彼此听不到。

CSMA/CA 的基本流程是：先监听信道，空闲持续 DIFS 后进入随机退避；退避计数到 0 再发送；接收方正确收到后，等 SIFS 发回 ACK。因为发送方听不到冲突，ACK 成了判断帧是否送达的主要手段。RTS/CTS 是可选项：发送方先发 RTS，AP 回 CTS，CTS 会被覆盖范围内的所有站听到，其他站据此推迟发送。

CSMA/CA 不能完全消除冲突。随机退避只能降低同时发送的概率，隐藏站和突发流量仍可能造成碰撞。它的目标是把冲突控制在可接受的范围，而不是像 CSMA/CD 那样检测到冲突就立刻停发。

5.11 408 高频考点速查

结论 5.1: 数据链路层常考点

1. 帧定界：字节填充的 `0x7E/0x7D` 替换规则；零比特填充“五个 1 插一个 0”。
2. CRC：补 r 个 0、模 2 除法、余数替换；生成多项式次数与校验位数的关系。
3. 汉明码：校验位放在 1、2、4 的位置；故障字指出出错位；$d_{\min}$ 与检错、纠错能力的关系。
4. 滑动窗口：停止-等待的利用率公式；GBN 的 2^m-1；SR 的 2^{m-1}；累计确认与逐帧确认的区别。
5. CSMA/CD：争用期 2τ；最短帧长 $L_{\min}=2\tau R$；二进制指数退避的范围；10 Mb/s 以太网的 51.2 μs 和 64 B。
6. 以太网帧：前导码不计入帧长；帧长 64–1518 B；数据字段 46–1500 B；FCS 用 CRC-32。
7. 交换机：源 MAC 学习、目的 MAC 转发、未知目的泛洪、同端口丢弃；每个端口一个冲突域，未划分 VLAN 时整个交换机一个广播域。
8. VLAN：802.1Q 标签 4 B；Access 与 Trunk 的区别；VLAN 隔离广播域，不隔离冲突域。
9. PPP：帧格式与字节填充；LCP/NCP；不提供可靠传输；FCS 只检错。
10. 无线：CSMA/CA 用随机退避降低冲突概率，用 ACK 确认单播帧是否送达；隐藏站问题；RTS/CTS 的作用。

拓展阅读：

- Kurose & Ross《计算机网络：自顶向下方法》第 6 章：CSMA/CD、交换机自学习和无线局域网的完整讨论。
- 谢希仁《计算机网络》第 3 章：PPP、CSMA/CD 和以太网的国内标准表述。

6 习题

6.1 基础过关题

1. （单项选择题）HDLC 采用零比特填充法组帧，帧定界符为 `01111110`。待发送的数据比特串为 `011111101111110`。发送方进行零比特填充后，实际发送的比特串是

(A) `01111101011111010` (B) `0111110101111101`

(C) `011111101111110` (D) `011111010111110010`

2. （单项选择题）下列关于组帧方法的说法中，正确的是

(A) 计数字段出错只影响本帧 (B) 数据中的定界字节须转义

(C) 零比特填充用于字符链路 (D) 违规编码法需增加校验位

3. （单项选择题）数据比特串 $M = 1101011011$，生成多项式 $G(x) = x^4 + x + 1$。按 CRC 计算规则得到的 4 位校验码是

(A) 1110 (B) 0111

(C) 1011 (D) 1101

4. （单项选择题）下列关于差错控制的说法中，正确的是

(A) CRC 能纠正所有单比特错 (B) 奇偶校验能检出偶数位错

(C) 海明距离 d 可检出 $d-1$ 位错 (D) 以太网用链路层确认重传

5. （填空题）某采用 CSMA/CD 的以太网，数据率为 100 Mb/s，最大端到端距离为 2 km，信号传播速率为 2×10^8 m/s，忽略中继器时延。为保证发送方能在发送期间检测到最坏情况下的冲突，最短帧长为 _____ bit，即 _____ B。

6. （单项选择题） PPP 工作在异步链路上，帧定界符为 `0x7E`，转义字节为 `0x7D`。转义规则为：`0x7E` 转义为 `0x7D 0x5E`，`0x7D` 转义为 `0x7D 0x5D`，小于 `0x20` 的字节转义为 `0x7D` 后跟“原字节异或 `0x20`”。若待发送的数据字节依次为 `7D 7E 03`，则填充后实际发送的字节序列是

(A) `7D 5D 7D 5E 7D 23`　(B) `7D 5E 7D 5D 7D 23`

(C) `7D 7E 03`　(D) `7D 5D 7E 03`

6.2 真题风格与综合训练

1. （单项选择题）某以太网采用二进制指数退避算法，争用期 $2\tau = 51.2\ \mu s$。某站发送一帧时连续发生冲突，则第 5 次重传时随机数 r 的取值范围和最大退避时间分别是

(A) $[0, 15]$，$768.0\ \mu s$　(B) $[0, 31]$，$1587.2\ \mu s$

(C) $[0, 31]$，$1638.4\ \mu s$　(D) $[0, 63]$，$3225.6\ \mu s$

2. （单项选择题）某 100 Mb/s 以太网交换机采用直通交换方式，输出端口无排队。一个以太网帧长 1000 B（从目的 MAC 地址到 FCS，不含前导码和帧开始定界符）。直通交换在收到目的 MAC 地址（6 B）后立即开始转发；若改用存储转发方式，则收完整帧后才开始转发。存储转发比直通交换多引入的转发时延是

(A) $0.48\ \mu s$　(B) $79.52\ \mu s$

(C) $80.00\ \mu s$　(D) $121.44\ \mu s$

3. （真题风格・综合题）某采用 CSMA/CD 的以太网，数据率 $R = 100$ Mb/s，任意两站间的最大距离为 2 km，信号传播速率为 $v = 2 \times 10^8$ m/s，忽略中继器和处理时延。

(1) 求单程传播时延 τ 和争用期 2τ。

(2) 求保证冲突检测的最短帧长，分别用 bit 和 B 表示。

(3) 若某帧的长度只有 800 bit，判断它能否在该网络上保证冲突检测；若不能，给出两种可行的修改方案，并分别算出对应的临界值。

(4) 若该帧连续发生 3 次冲突，求第 3 次重传时随机数 r 的取值范围及最大退避时间。

4. （真题风格・综合题）某 8 端口以太网交换机 S 的端口 1～4 分别连接主机 A、B、C、D，端口 5 连接路由器 R，端口 6～8 空闲。初始时交换表为空，忽略表项老化。

(1) 在未划分 VLAN 时，依次发生下列发送：1) A 向 B 发送单播帧；2) B 向 A 发送单播帧；3) C 向 D 发送单播帧；4) D 向 A 发送单播帧。写出每一步交换机新学到的表项，以及该帧被转发到的端口集合（不含输入端口）。

(2) 现将端口 1、2 划入 VLAN 10，端口 3、4 划入 VLAN 20，端口 5 配置为 Trunk 端口。判断 A 能否直接与 C 进行二层通信；若 A 直接把 C 的 MAC 地址作为目的地址发送帧，说明交换机的处理过程，以及二者通信必须经过什么设备。

(3) 在 (2) 的 VLAN 划分下，连接主机的 4 个端口共形成几个冲突域、几个广播域?

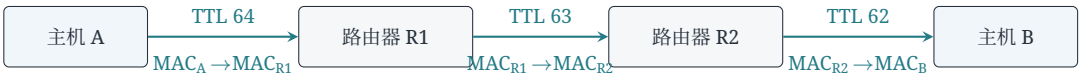
7 网络层

宿舍里的笔记本地址是 192.168.1.7，默认网关是 192.168.1.1。它要打开 93.184.216.34 上的网页，数据先交给网关，再由运营商的路由器一跳一跳送出去。链路层只负责同一条链路上的相邻结点，跨过多个网络把分组送到最终目的主机，要靠网络层。网络层做三件事：给主机编地址，按地址决定从哪个接口转发，让路由器互相交换到达各网络的路径。

7.1 网络层在整条路径上的位置

先看分组从主机 A 到主机 B 的过程。A 构造一个 IP 数据报，源地址写自己的地址，目的地址写 B 的地址。数据报要交给链路层封装成帧，帧首部的目的 MAC 地址是下一跳路由器的地址。路由器收到帧，剥掉帧头和帧尾，取出 IP 数据报，查路由表决定下一跳，再把 TTL 减 1、重新计算首部校验和，用下一跳的 MAC 地址封装成新帧发出去。每一跳都重复这个过程。

IP 首部：源 192.168.1.7，目的 93.184.216.34；这两个地址在整条路径上不变。



图里每个箭头是一段链路。IP 首部的源地址和目的地址从 A 到 B 基本不变，只有 TTL 每经过一个路由器减 1；帧首部和目的 MAC 地址逐跳改写。链路层地址只在本地链路有效，IP 地址才是端到端的标识。这也是为什么换一块网卡只改 MAC，不改 IP 的通信关系。

IP 采用数据报服务：每个分组自带完整的目的地址，独立选路，可能乱序、丢失，网络层不重传。可靠性交给传输层的 TCP。另一种做法是虚电路：先建立连接，分组只带虚电路号 VCI，沿途路由器保存连接状态，分组按序到达；某个结点故障，经过它的所有虚电路一起断开。Internet 选择了数据报，把复杂性放到端系统，路由器只做转发。

两者的区别可以记成一句话：数据报的每个分组都要重新查表，虚电路只在建链时选一次路。408 里说“网络层提供可靠交付”，只对虚电路成立；对 Internet 的 IP 不成立。

7.2 IPv4 地址与数据报

地址的写法与分类

192.168.1.7 写成二进制是 11000000.10101000.00000001.00000111，共 32 位。点分十进制只是方便人读。地址分两部分：网络前缀和主机号。同一前缀下的主机在同一条逻辑网络中，路由器只要知道前缀就能决定往哪个方向转发。

早期按前几位分类。A 类首位固定为 0，前缀 /8，首字节 1 到 126；B 类前两位 10，前缀 /16，首字节 128 到 191；C 类前三位 110，前缀 /24，首字节 192 到 223；D 类前四位 1110，用于组播，首字节 224 到 239；E 类 240 到 255 保留。0 和 127 有特殊用途，不算普通的 A 类网络。

分类地址的问题是粒度太粗。一个 B 类有 65534 个主机地址，很多单位用不完；C 类只有 254 个，又不够。子网划分和 CIDR 就是为解决这个问题出现的。

有几个地址必须记住：

- 主机号全 0 是网络地址，例如 192.168.1.0；主机号全 1 是直接广播地址，例如 192.168.1.255。
- 255.255.255.255 是受限广播，路由器不转发。
- 127.0.0.0/8 是环回地址，127.0.0.1 最常用。

- 169.254.0.0/16 是链路本地地址，主机拿不到 DHCP 地址时会自动配置。
- 10.0.0.0/8、172.16.0.0/12、192.168.0.0/16 是私有地址，不会出现在公网路由表中。
- 224.0.0.0/4 是组播地址，240.0.0.0/4 保留。

0.0.0.0 有两种用法：作为源地址，表示本机还不知道自己的地址，DHCP 发现阶段会这样发；在路由表里写成 0.0.0.0/0，表示默认路由，匹配所有目的地址。

主机地址可以手工配置，也可以由 DHCP 分配。DHCP 客户端广播 DISCOVER，服务器回 OFFER，客户端再广播 REQUEST，服务器回 ACK。DHCP 用 UDP，服务器端口 67，客户端端口 68。它分配的不只是 IP，还包括子网掩码、默认网关和 DNS 服务器。

首部里的字段

看一个具体的数据报：从 192.168.1.7 发往 93.184.216.34，数据部分 3980 字节，首部 20 字节，总长度字段写 4000。IPv4 首部固定部分 20 字节，加上选项最多 60 字节。



图中每一行是 32 位，也就是 4 字节。固定首部占 5 行，所以没有选项时 IHL 等于 5。IHL 的单位是 4 字节，不是字节；抓包看到 IHL 大于 5，就说明有选项。总长度以字节为单位，16 位最大 65535，实际还要受链路 MTU 限制。

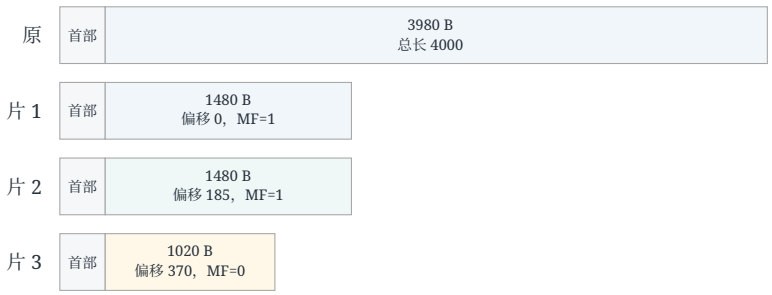
标识、标志、片偏移三个字段只在分片时有用。标识把同一数据报的所有分片归为一组；标志字段里的 DF 表示不许分片，MF 表示后面还有分片；片偏移记录本片数据在原数据报中的位置，单位是 8 字节。

TTL 每经过一个路由器减 1，减到 0 就丢弃，并通常返回 ICMP 超时报文。协议字段告诉目的主机把数据交给谁：1 是 ICMP，6 是 TCP，17 是 UDP，89 是 OSPF。首部校验和只校验首部，不校验数据；每跳 TTL 改变后必须重新计算。算法是把首部按 16 位分组做反码求和，再取反，接收方用同样的方法验证。

源地址和目的地址在普通转发中不变，NAT 是例外。区分服务字段的前 6 位是 DSCP，用来标记优先级；后 2 位是 ECN，用来做显式拥塞通知。

分片

以太网 MTU 是 1500 字节。一个 4000 字节的 IP 数据报，首部 20 字节、数据 3980 字节，要从以太网发出，必须分片。每个分片都要带完整的 20 字节首部，所以每片最多装 1480 字节数据。1480 是 8 的倍数，符合片偏移的单位。



片偏移 = 数据起点 ÷8。前两片取 1480 B：$20 + 1480 = 1500$，且 1480 是 8 的倍数。

图里的长度按比例画。片偏移记录的是数据部分在原数据报中的起点，单位是 8 字节：1480 除以 8 得 185，2960 除以 8 得 370。最后一片的数据长度可以不是 8 的倍数，前面各片必须是。

更换 MTU 时，通用算法如下。设原数据部分长度为 L 字节，每片数据上限

$$D = \left\lfloor \frac{\text{MTU} - \text{首部长度}}{8} \right\rfloor \times 8,$$

片数 $n = \lceil L/D \rceil$。前 $n-1$ 片各装 D 字节，第 n 片装 $L - (n-1)D$ 字节；数据总长正好是 D 的整数倍时，最后一片也是 D 字节。第 i 片的偏移等于前面所有片的数据长度之和除以 8。每个分片的总长度字段等于自己的首部加数据，不是原数据报的总长。

注意

片偏移的单位是 8 字节，不是 1 字节；总长度字段是首部加数据，不是只有数据；除最后一片外，每片数据长度必须是 8 的倍数。这三点是分片计算最常见的错误。

DF=1 时路由器不能分片，会丢弃数据报并返回 ICMP 终点不可达（类型 3，代码 4，需要分片），报文里可以带上下一条链路的 MTU。源端据此调小报文，这叫路径 MTU 发现。重组只在目的主机进行，中间路由器不重组；任何一个分片丢失，目的主机最终都会丢弃整个数据报。

7.3 子网、掩码与 CIDR

从 /24 里借位

一个 /24 有 256 个地址，减去网络地址和广播地址，可分配 254 个。四个部门各要 20 台主机，直接给四个 /24 浪费太多。做法是从主机号借 3 位当子网号：/24 变成 /27，一个 /24 切成 8 个 /27，每个 32 个地址，可分配 30 个。



192.168.1.32/27：网络地址.32，广播地址.63，可用地址.33 到.62。
块大小 $2^5 = 32$，每个子网可分配 $32 - 2 = 30$ 个主机地址。

图中末字节被分成 3 位子网号和 5 位主机号。块大小是 $2^5 = 32$，所以 8 个子网的网络地址是.0、.32、.64、……、.224。以.32/27 为例，网络地址是.32，广播地址是.63，可用地址是.33 到.62。

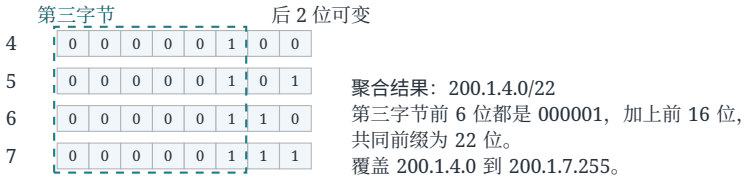
子网掩码把网络前缀和子网号对应的位置 1，主机号位置 0。/27 的掩码是 255.255.255.224。网络地址等于 IP 地址与掩码按位与；广播地址等于网络地址加上主机位全 1 的值。

计算时先看块大小：块大小等于 2^{32-n}，其中 n 是前缀长度。网络地址一定是块大小的整数倍。给定 192.168.1.100/27，100 落在 96 到 127 这一块，网络地址是 192.168.1.96，广播地址是 192.168.1.127，可用地址是 97 到 126。

地址块大小是 2^{32-n}，可用主机数是 $2^{32-n}-2$（$n \le 30$）。/31 用于点对点链路，两个地址都可以分配给两端；/32 表示单个主机。408 题目没有特别说明时，仍按减 2 计算。有的教材按早期规定排除子网号全 0 和全 1 的子网，做题时看题目采用哪种口径。

CIDR 与路由聚合

CIDR 把网络前缀长度写成 /n，n 可以是 0 到 32 的任意值，不再要求 8、16、24。路由表里一条 200.1.4.0/22 可以覆盖四个 /24：200.1.4.0/24 到 200.1.7.0/24。



图中第三字节写成 8 位。四个网络的前 6 位都是 000001，只有后 2 位不同，所以共同前缀是 $16+6=22$ 位，聚合为 200.1.4.0/22。这个 /22 覆盖 200.1.4.0 到 200.1.7.255，正好是四个 /24 的并集。

聚合的条件是地址连续且按前缀对齐。200.1.4.0/24 和 200.1.6.0/24 不能聚合成 /23 或 /22 而不覆盖 200.1.5.0/24；如果强行通告 /22，会把未使用的地址也包含进来。聚合减少路由表项，但前缀变短后，匹配范围变大，需要最长前缀匹配来保证具体路由优先。

7.4 ARP 与 ICMP

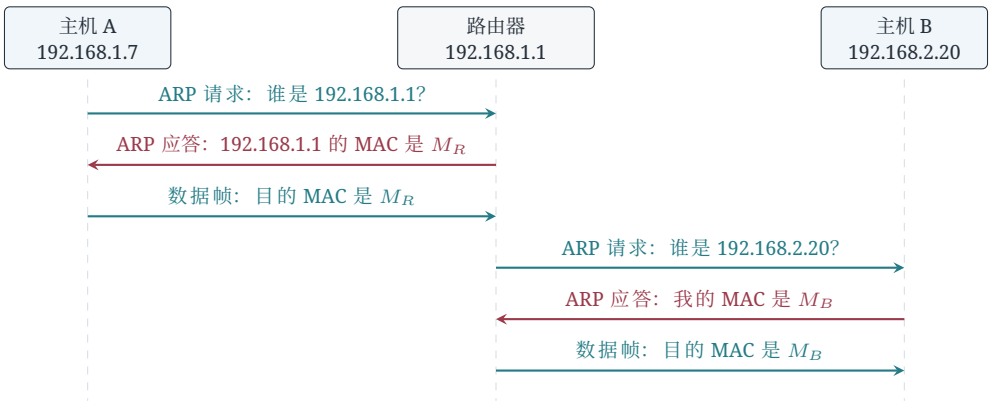
ARP：把 IP 地址换成 MAC 地址

主机 A(192.168.1.7)要给同网段的主机 C(192.168.1.20)发数据。A 知道 C 的 IP，不知道 C 的 MAC。A 在本地广播一个 ARP 请求，目的 MAC 填 FF-FF-FF-FF-FF-FF，内容问“谁是 192.168.1.20”。同网段所有主机都收到，只有 C 回复，回复是单播。A 把 IP 和 MAC 的映射放进 ARP 缓存，之后直接查缓存。

ARP 报文直接封装在以太网帧里，以太网类型是 0x0806，不走 IP。IPv4 的 ARP 报文 28 字节：硬件类型 2 字节、协议类型 2 字节、硬件地址长度 1 字节、协议地址长度 1 字节、操作码 2 字节、发送方 MAC 6 字节、发送方 IP 4 字节、目标 MAC 6 字节、目标 IP 4 字节。请求的操作码是 1，应答是 2。请求报文里的目标 MAC 字段通常填 0，因为发送方还不知道它。

ARP 缓存有老化时间，表项过期后要重新解析。主机换网卡、IP 冲突或虚拟机迁移时，缓存里的旧映射会造成短时通信异常。主机上线或地址变化时可以发免费 ARP，主动通告自己的映射，让同网段主机更新缓存。

跨网段时，A 解析的是网关的 MAC，如图所示。



图中 A 要发给 192.168.2.20 的 B。目的 IP 不在本地子网，A 先问网关 192.168.1.1 的 MAC，不是 B 的 MAC。数据帧到达路由器后，路由器查表找到下一跳，在右侧网段再发 ARP 解析 B 的 MAC。IP 首部的目的地址始终是 192.168.2.20。

ICMP：差错报告和探测

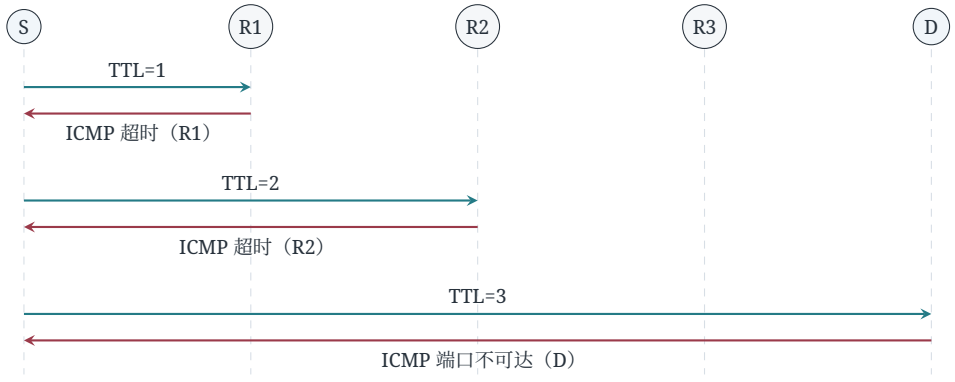
Ping 不通时，主机可能收到 ICMP 差错报文，也可能什么都没收到；这些都与 TCP 无关。ICMP 是 IP 的辅助协议，封装在 IP 数据报里，协议号 1。它传递两类信息：差错报告和查询。

常见类型有：终点不可达 3、重定向 5、超时 11、参数问题 12；回送请求 8、回送回答 0、时间戳请求 13、时间戳回答 14。差错报文里带原始 IP 首部和数据区前 8 字节，源主机据此知道是哪个数据报出的错。

有几种情况不发 ICMP 差错报文：对 ICMP 差错报文本身、对非第一片分片、对组播或广播地址、源地址为 0 或环回地址。这样做是为了避免差错报文互相触发，形成风暴。

Ping 发回送请求（类型 8），目的主机回送回答（类型 0），用标识和序号匹配请求与应答，统计往返时间和丢包率。Ping 不经过传输层，没有端口。

Traceroute 利用 TTL。源端先发 TTL=1 的探测报文，第一跳路由器把 TTL 减到 0，丢弃并回 ICMP 超时（类型 11），源端就得到第一跳的地址。然后发 TTL=2，得到第二跳，依此类推。到达目的主机时，如果用 UDP 探测，目的主机回 ICMP 端口不可达（类型 3，代码 3）；如果用 ICMP Echo 探测，则回回送回答。



实线是源端发出的探测，虚线是路由器或目的主机返回的 ICMP。源端按返回报文的源地址记录第 i 跳。

图中每一行是一次探测。TTL 逐次加 1，返回报文的源地址依次给出路径上的路由器。Linux 的 traceroute 默认用 UDP 高端口，Windows 的 tracert 默认用 ICMP Echo，两者原理相同。

7.5 路由表、转发与路由协议

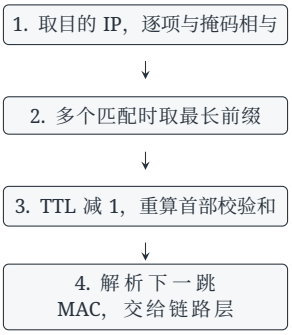
查表与最长前缀匹配

一台路由器可能同时连着本地子网、下一台路由器和默认网关，它的路由表大致如下。

目的网络	掩码	下一跳	接口
192.168.1.0	255.255.255.0	直连	eth0
192.168.2.0	255.255.255.0	10.0.0.2	eth1
0.0.0.0	0.0.0.0	10.0.0.1	eth1

目的地址 192.168.2.130 到来时，路由器把地址分别与每条掩码按位与：与第一条得 192.168.2.0，不等于 192.168.1.0，不匹配；与第二条得 192.168.2.0，匹配；与默认路由也匹配。多条匹配时选前缀最长的，所以走第二条，下一跳 10.0.0.2。





左边四个框是嵌套的：/0 包含所有地址，/8 包含 192 开头的地址，/16 包含 192.168 开头的地址，/24 只包含 192.168.2.0 这一块。目的地址落在最内层，就选 /24。最长前缀匹配只看前缀长度，不看度量值；度量值只在同一条前缀有多条路径时比较。

右边的四步是转发过程：查表、取最长前缀、TTL 减 1 并更新校验和、解析下一跳 MAC 后交给链路层。TTL 减到 0 时丢弃并回 ICMP 超时；下一跳链路 MTU 小于数据报且 DF=1 时，回 ICMP 需要分片。

默认路由 0.0.0.0/0 能匹配所有地址，前缀长度是 0，所以总是最后才被选中。直连路由的下一跳就是接口本身，不需要再经过路由器。路由表可以由管理员静态配置，也可以由路由协议动态生成。静态路由简单，不会自适应拓扑变化；动态路由收敛慢一些，但能自动绕开故障。

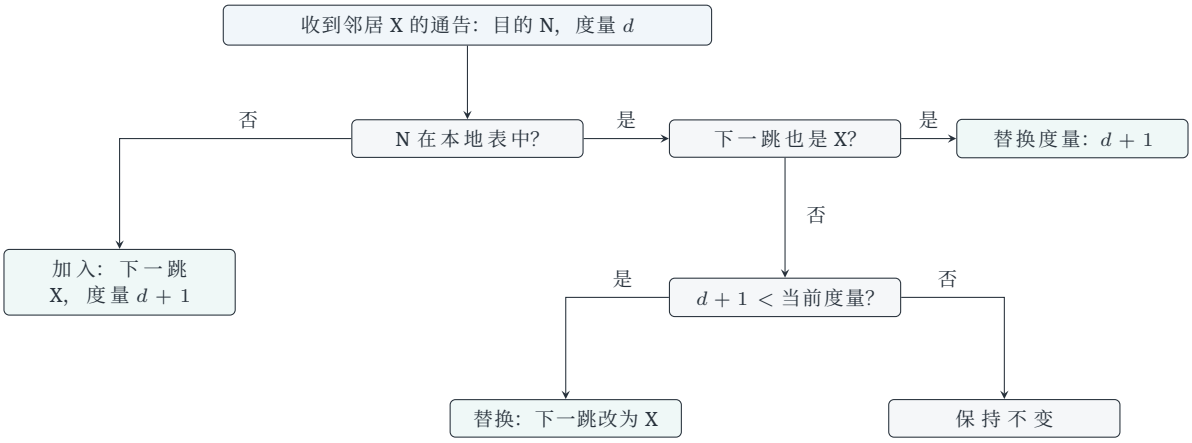
距离向量与链路状态

RIP 是最简单的动态路由协议。每个路由器只和邻居说话，每 30 秒把自己整张路由表发一遍。收到邻居 X 的表项（目的 N，度量 d）后，候选度量是 d+1，然后按三条规则决定是否更新：

1. N 不在本地表：加入，下一跳 X，度量 d+1。
2. N 在表中且下一跳就是 X：无条件替换度量。邻居的新度量可能变大，也要替换，否则会保留过期信息。
3. N 在表中但下一跳不是 X：只有 d+1 小于当前度量才替换，下一跳改为 X。

RIP 用跳数做度量，最大 15，16 表示不可达。它跑在 UDP 上，端口 520。跳数不区分带宽，所以 RIP 可能选一条跳数少但很慢的路径。

链路断开后，坏消息传播慢：邻居可能互相学到对方的路由，度量一点点加 1，直到 16。这叫计数到无穷。水平分割规定：从某个接口学到的路由，不再从该接口发回去。毒性逆转把从某个接口学到的路由以度量 16 从该接口发回。触发更新在拓扑变化时立即发送，不等 30 秒。这三种方法都在缓解同一个问题。



这张图就是 RIP 更新的判断顺序。考试时先把邻居表每项加 1，再逐项对照本地表，特别注意“下一跳相同”这一条。

OSPF 走另一条路。每个路由器发现邻居，生成链路状态通告 LSA，洪泛到整个区域；每台路由器收集 LSA 得到链路状态数据库 LSDB，也就是本区域的拓扑图；然后以自己为根跑 Dijkstra，算出到每个网络的最短路径。

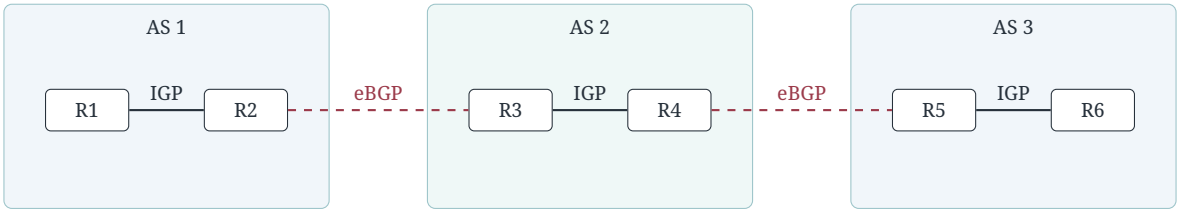
OSPF 直接封装在 IP 里，协议号 89，不用 TCP 或 UDP。度量是开销，通常与带宽成反比。网络大时划分区域，区域 0 是骨干区域，跨区域路由由区域边界路由器汇总。OSPF 支持等价多路径，两条开销相同的路径可以一起用。

RIP、OSPF、BGP 的分工

协议	算法	度量	封装	使用范围
RIP	距离向量	跳数，16 表示无穷	UDP，端口 520	AS 内部，小型网络
OSPF	链路状态	开销（带宽反比）	IP，协议号 89	AS 内部，大型网络
BGP	路径向量	AS 路径与策略	TCP，端口 179	AS 之间

AS 之间

一个校园网、一个运营商网络，各自是一个自治系统 AS。AS 内部的路由器用同一个 IGP（RIP 或 OSPF）交换路径；AS 的边界路由器之间用 BGP 交换可达性。BGP 不追求最短路径，它按策略选路：哪个邻居更便宜、哪条路径不经过竞争对手，都会影响选择。



实线：AS 内部使用 IGP（RIP 或 OSPF）。虚线：AS 之间使用 BGP，会话建立在 TCP 179 上。

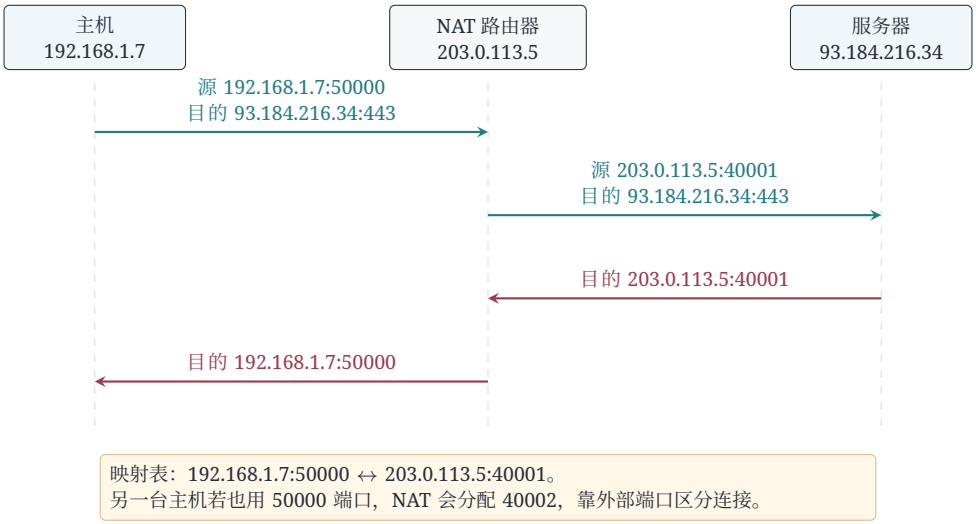
图中实线是 AS 内部的 IGP，虚线是 AS 之间的 eBGP。BGP 会话建立在 TCP 上，端口 179，

用 TCP 的可靠传输保证路由更新不丢失。BGP 通告的路径带有 AS 号序列，路由器如果发现序列里有自己的 AS 号，就知道出现了环路，丢弃该路由。

iBGP 用于把从外部学到的路由传给同一 AS 内的其他边界路由器，它不改变 AS 路径。eBGP 负责在 AS 边界之间交换路径，iBGP 负责把路径带进 AS 内部。两者合起来，公网上的每个 AS 才知道哪些前缀可达、要经过哪些 AS。

7.6 NAT

家里有三台设备，运营商只给了一个公网地址 203.0.113.5。笔记本 192.168.1.7 访问 93.184.216.34:443 时，源地址和源端口是 192.168.1.7:50000。NAT 路由器把源地址改成 203.0.113.5，并分配一个外部端口 40001，把映射记下来。服务器看到的连接来自 203.0.113.5:40001。回包到达时，路由器查表把目的地址和端口改回 192.168.1.7:50000。



图中四步是：内部主机发出原始报文，NAT 改写源地址和端口，服务器回包，NAT 按映射改回目的地址和端口。映射表的关键字通常是协议、内部 IP、内部端口，值是公网 IP 和外部端口。

只改 IP 地址、不改端口叫基本 NAT；同时改端口叫 NAPT 或 PAT，日常家用路由器都是 NAPT。多个内部主机可以共享一个公网地址，靠端口区分。

改地址后要更新 IP 首部校验和；改地址或端口后要更新 TCP 或 UDP 校验和，因为校验和覆盖伪首部中的 IP 地址，也覆盖含端口号的传输层首部。外部主机不能主动连接内部主机，除非 NAT 上配置了静态映射或端口转发。NAT 破坏了端到端可达性，也给主动发起的语音、P2P 应用带来麻烦。

私有地址范围是 10.0.0.0/8、172.16.0.0/12、192.168.0.0/16。它们不会出现在公网路由表中，只能在私网内部使用。

7.7 一次访问的完整路径

访问 93.184.216.34 时，主机查路由表得到默认网关 192.168.1.1，用 ARP 解析网关的 MAC。封装成帧后，源 MAC 是主机，目的 MAC 是网关；IP 首部源地址 192.168.1.7，目的地址 93.184.216.34，TTL 为 64。

网关查表转发，兼做 NAT 时改写源地址。沿途路由器逐跳减 TTL、重算校验和、按下一跳 MAC 重新封装；MTU 不够且 DF=0 时在本地分片。目的主机按协议字段交给 TCP，再由端口交给应用；回包按 NAT 映射改回内部地址。

8 习题

8.1 基础过关题

1. (单项选择题) 某单位分配到地址块 172.16.8.0/22，现将其等长划分为前缀长度为 /26 的子网。可划分出的子网个数与每个子网中可分配给主机的地址个数分别是

(A) 16，62　(B) 16，64

(C) 4，62　(D) 64，62

2. (单项选择题) 一个 IPv4 数据报总长度为 3000 B，其中首部为固定的 20 B；DF=0，途经某链路的 MTU=1500 B。该数据报在该链路上被分片后，分片个数及各分片的片偏移值依次为

(A) 2 片，0、1480　(B) 3 片，0、185、370

(C) 3 片，0、1480、2960　(D) 4 片，0、185、370、555

3. (单项选择题) 下列关于 ARP 的叙述中，正确的是

(A) ARP 请求报文封装在 IP 数据报中发送

(B) 同一局域网内，ARP 请求以广播帧发送，ARP 应答通常以单播帧发送

(C) ARP 高速缓存中的表项一旦建立就永久有效

(D) 目的主机位于另一个网络时，源主机应通过 ARP 解析目的主机的 MAC 地址

4. (填空题) ICMP 报文封装在 IPv4 数据报中，IPv4 首部协议字段的值为 _____；ping 使用的回送请求与回送回答的 ICMP 类型值分别为 _____ 和 _____；traceroute 逐跳发现路径时，依赖路由器返回的 ICMP _____ 报文（类型值为 11）。

5. （单项选择题）路由器 R1 与 R2 直连。R1 原有路由表中与本题相关的表项如下表所示；某时刻 R1 收到 R2 发来的距离向量，其中 R2 到达 Net1、Net2、Net3、Net4 的距离分别为 2、3、1、4。按 RIP 的距离向量算法更新后，R1 到达 Net3、Net4 的距离与下一跳分别是

目的网络	距离	下一跳
Net1	3	R2
Net2	2	R3
Net3	5	R2

(A) Net3：2，R2；Net4：5，R2　(B) Net3：6，R2；Net4：5，R2

(C) Net3：2，R2；Net4：4，R2　(D) Net3：5，R2；Net4：5，R2

6. （填空题）某 NAT 路由器执行 NAPT，内部主机 A、B 同时用源端口 3000 访问同一台 Web 服务器，转换表如下。服务器返回一个目的地址为 `203.0.113.5`、目的端口为 62001 的 TCP 报文。

内部端点	转换后的外部端点
`192.168.1.10:3000`	`203.0.113.5:62000`
`192.168.1.11:3000`	`203.0.113.5:62001`

（1）NAT 应将该报文转发给哪个内部端点？

（2）完成上述转发时，必须重新计算哪些校验和？

8.2 真题风格与综合训练

7. （单项选择题）某路由器的转发表如下。现收到目的地址分别为 `10.1.2.130`、`10.1.2.77`、`10.1.3.9` 的三个 IP 数据报，按最长前缀匹配，它们对应的下一跳依次是

目的前缀	下一跳
0.0.0.0/0	R0
10.0.0.0/8	R1
10.1.0.0/16	R2
10.1.2.0/24	R3
10.1.2.128/25	R4

(A) R4、R3、R2　(B) R4、R2、R2

(C) R3、R3、R2　(D) R4、R3、R1

8. (**单项选择题**) 主机 H1 的地址为 192.168.1.10/24，默认网关为 192.168.1.1；主机 H2 位于另一网络，地址为 10.0.0.20/24。H1 向 H2 发送 IP 数据报，路径为 H1—R1—R2—H2，途中未使用 NAT。下列说法正确的是

(A) H1 发出的帧的目的 MAC 地址是 H2 的 MAC 地址

(B) H1 发送前用 ARP 解析 10.0.0.20，得到 H2 的 MAC 地址

(C) H1 用 ARP 解析默认网关 192.168.1.1 的 MAC 地址，发出的 IP 数据报目的地址仍为 10.0.0.20

(D) R1 转发该数据报时，会同时改写 IP 首部中的源地址和目的地址

9. (**真题风格·综合应用题**) 某单位分配到地址块 192.168.10.0/24，需要为四个子网分配地址：研发部 60 台主机、市场部 28 台主机、财务部 12 台主机，以及两台路由器之间的点对点链路（需要 2 个可分配地址）。采用变长子网划分（VLSM），按需求从大到小分配，并假定每个子网都保留网络地址和广播地址。(1) 写出每个子网的网络地址、子网掩码（分别用点分十进制和 /n 表示）、可用地址范围和广播地址。(2) 全部划分完成后，还剩多少个未分配的地址？(3) 若还要新增一个需要 25 台主机的子网，能否从剩余地址中划出？给出一个满足要求的地址块。

10. （真题风格·综合应用题）某网络运行链路状态路由算法，各路由器之间的链路代价如下表所示，所有链路均为双向且两个方向代价相同。以路由器 A 为源点。

链路	代价	链路	代价	链路	代价
A–B	4	B–C	1	C–F	8
A–C	2	B–E	5	D–E	1
A–D	6	C–D	2	E–F	2
		C–E	6		

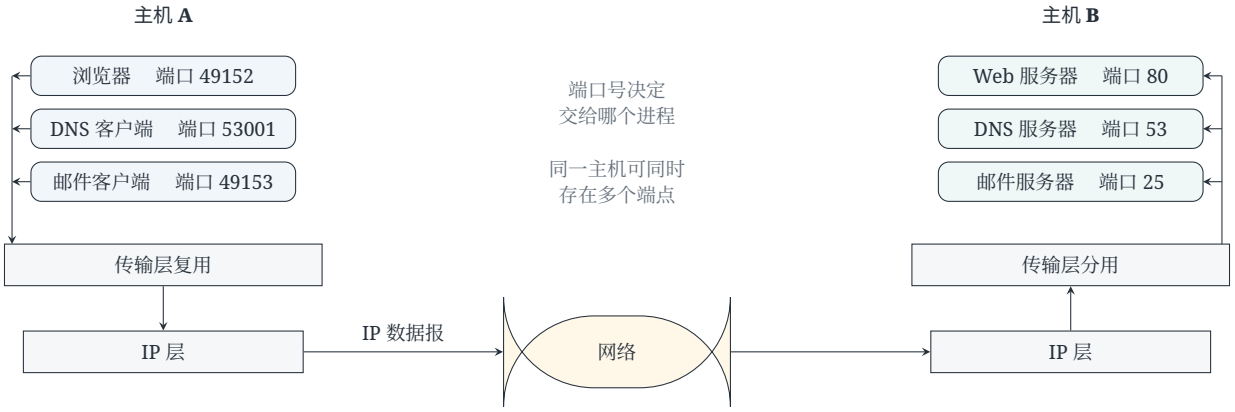
（1）用 Dijkstra 算法求 A 到 B、C、D、E、F 的最短距离与最短路径（写出结点序列），并列出算法每次选出的结点及相应的松弛结果。（2）若在 A、E 之间新增一条代价为 4 的链路，重新求 A 到 E、F 的最短距离与路径，并指出 A 的转发表中哪些目的地的下一跳发生变化。

9 传输层

网络层把分组送到目的主机，但主机上往往同时运行着浏览器、邮件客户端、DNS 解析器等多个进程。一个 IP 分组到达后，应该交给哪一个进程？这是传输层要回答的第一个问题。传输层把“主机到主机”的通信细化成“进程到进程”的通信，用端口号标识通信端点；在这个基础上，它还可以选择只做很薄的一层封装（UDP），或者用序号、确认、重传、窗口和拥塞控制搭出一套可靠的字节流服务（TCP）。本章沿着“端口与复用分用 → UDP → 可靠传输的通用机制 → TCP 的可靠传输、流量控制、拥塞控制与连接管理”这条线展开。

9.1 端口、套接字与复用分用

一台主机只有一个（或几个）IP 地址，却可能同时运行几十个网络进程。传输层给每个进程分配一个 16 位的**端口号**（0–65535），IP 地址加端口号就构成一个**套接字地址**。发送数据时，多个进程的数据被传输层汇聚后交给 IP 层，这叫**复用**（multiplexing）；接收数据时，传输层根据端口号把数据交给正确的进程，这叫**分用**（demultiplexing）。



图里最值得注意的是：IP 层只负责把数据报送到主机 B，真正的“分拣”发生在传输层。接收方看到目的端口 80，就把数据交给 Web 服务器；看到 53，就交给 DNS 服务器。端口号只在本地有意义，不同主机上的同一个端口号互不冲突。

端口号分为三段：0–1023 是**知名端口**，由 IANA 分配给标准服务，例如 HTTP 80、HTTPS 443、DNS 53、SMTP 25、POP3 110、FTP 21 和 20、Telnet 23；1024–49151 是**注册端口**；49152–65535 是**动态端口**，客户端通常临时申请一个这样的端口。408 选择题里常见的陷阱是“客户端端口必须大于 1024”之类的绝对化说法——标准服务也可以使用动态端口，关键看进程是否绑定。

仅有“IP 地址 + 端口”还不够区分 TCP 连接。一条 TCP 连接由**源 IP、源端口、目的 IP、目的端口**四元组唯一标识。因此，同一台 Web 服务器可以在本地 80 端口上同时服务成千上万个客户：它们的四元组不同。UDP 的分用通常只看目的端口（以及绑定时的本地 IP），所以一个 UDP 端口上的数据报可能来自很多不同客户。

9.2 UDP：只加一层薄薄的信封

UDP（User Datagram Protocol）的设计目标是把事情做少。它在 IP 之上只增加了端口号和校验和，提供**无连接、不可靠、面向报文**的传输服务。发送前不需要握手，发送后不等待确认；数据报可能丢失、重复、乱序，UDP 本身不负责恢复。也正因为如此，UDP 首部只有 8 字节，发送方不需要维护连接状态，适合 DNS、DHCP、TFTP、SNMP、实时音视频这类“宁可丢一点，也不要等”的应用。

“面向报文”是 UDP 最容易被误解的性质。应用层每调用一次 `sendto`，UDP 就把它封装成一个独立的数据报，既不合并也不拆分；接收方每调用一次 `recvfrom`，最多取走一个完整的数据报。若接收缓冲区小于数据报长度，多余部分会被截断丢弃，而不会像 TCP 那样留在缓冲区里等下一次读取。IP 层在下面可能会分片，但那是 IP 层的事，分片在到达目的主机后由 IP 层重组，UDP 看到的仍然是一个完整报文。

UDP 首部（8 B）

16	31
源端口号（16 位）	目的端口号（16 位）
UDP 长度（16 位）	校验和（16 位）

伪首部（12 B，只参与校验和计算，不随数据报发送）

源 IP 地址（32 位）		
目的 IP 地址（32 位）		
全 0（8 位）	协议号 17（8 位）	UDP 长度（16 位）

校验时把伪首部、UDP 首部和数据拼在一起，按 16 位字求和；伪首部本身不出现在链路上。

首部四个字段各 16 位。**长度**字段包含 8 字节首部和数据部分，最小值是 8（没有数据）。16 位长度字段理论上允许 65535 字节的 UDP 数据报，但 IPv4 总长度字段也只有 16 位，减去至少 20 字节 IP 首部和 8 字节 UDP 首部后，UDP 数据部分最大为 65507 字节；若 IP 首部带选项，还要更小。

校验和覆盖三部分：伪首部、UDP 首部和 UDP 数据。伪首部包含源 IP、目的 IP、一个全 0 字节、协议号 17 和 UDP 长度，共 12 字节。计算时先把校验和字段置 0，若数据部分为奇数字节，末尾临时补一个全 0 字节（补的字节不传输、也不计入长度），然后按 16 位字做**二进制反码求和**：相加时若最高位产生进位，就把进位加回最低位；最后把结果取反，填入校验和字段。若计算结果为 0，则填入全 1（`0xFFFF`），因为反码表示中全 0 与全 1 都表示“零”。

IPv4 中 UDP 校验和是**可选**的：校验和字段为 0 表示发送方没有计算校验和。IPv6 中 UDP 校验和是强制性的。408 题目若给出“校验和字段为 0”，要按“未计算”处理，而不是按“校验结果为 0”处理。

UDP 不提供流量控制，也不提供拥塞控制。发送方可以按应用产生的速率直接往网络里灌数据；网络拥塞时，UDP 不会自动减速。可靠性只能由应用层自己补，例如 DNS 客户端在超时后重发查询，实时音视频则常常直接丢弃迟到数据。

9.3 可靠传输：从停等到滑动窗口

IP 层提供的是“尽最大努力”服务，分组可能出错、丢失、重复或乱序。要在这样的基础上实现可靠传输，需要一套基本工具：**校验和**发现比特错误，**确认**（ACK）告诉发送方哪些数据已经收到，**定时器**处理确认丢失，**序号**处理重复和乱序。停等、后退 N 帧（GBN）和选择重传（SR）是三种最典型的协议，它们的关系是“窗口从 1 到 N、接收方从只收到缓存”的逐步放松。

停等协议与链路利用率

停等协议（Stop-and-Wait）最简单：发送方发出一帧后停下来，等收到该帧的确认再发下一帧。如果定时器超时还没收到确认，就重传。接收方收到正确帧后回 ACK，收到重复帧则丢弃并重发

ACK。用 1 位序号就足以让接收方区分“新帧”和“上一帧的重复”：发送方在 0、1 之间交替，接收方期望的序号也随之交替。

停等协议的问题是**链路利用率低**。设帧的发送时延为 T_D，往返时延为 RTT，确认帧的发送时延为 T_A，则

$$U = \frac{T_D}{T_D + \mathrm{RTT} + T_A}.$$

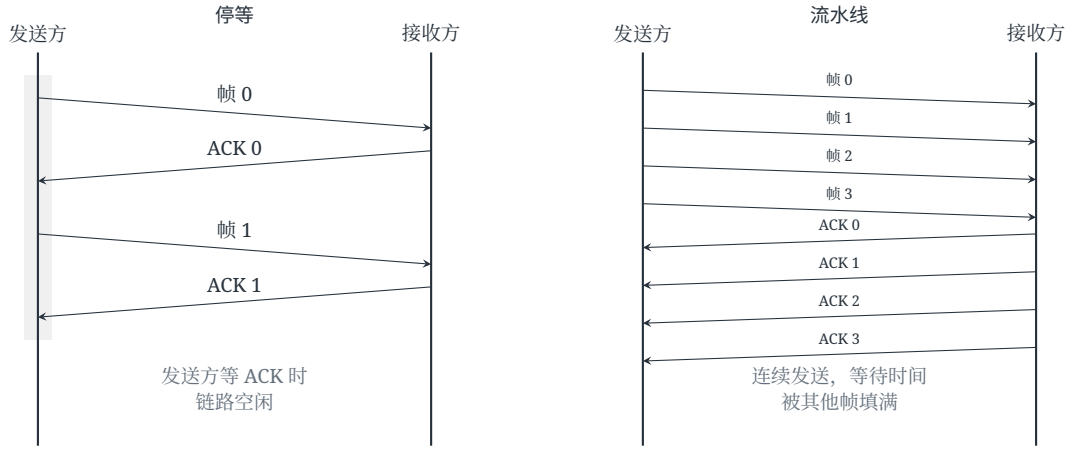
分母里的 RTT 主要由传播时延和中间节点处理时延构成。举个例子：链路速率 1 Gbps，帧长 1000 B，RTT 为 10 ms。帧的发送时延

$$T_D = \frac{1000 \times 8}{10^9} = 8\ \mu\mathrm{s},$$

而 RTT 是 10 ms，约是 T_D 的 1250 倍，于是

$$U \approx \frac{8\ \mu\mathrm{s}}{10\ \mathrm{ms} + 8\ \mu\mathrm{s}} \approx 0.08\%.$$

也就是说，发送方 99.9% 的时间在等确认，链路几乎空闲。

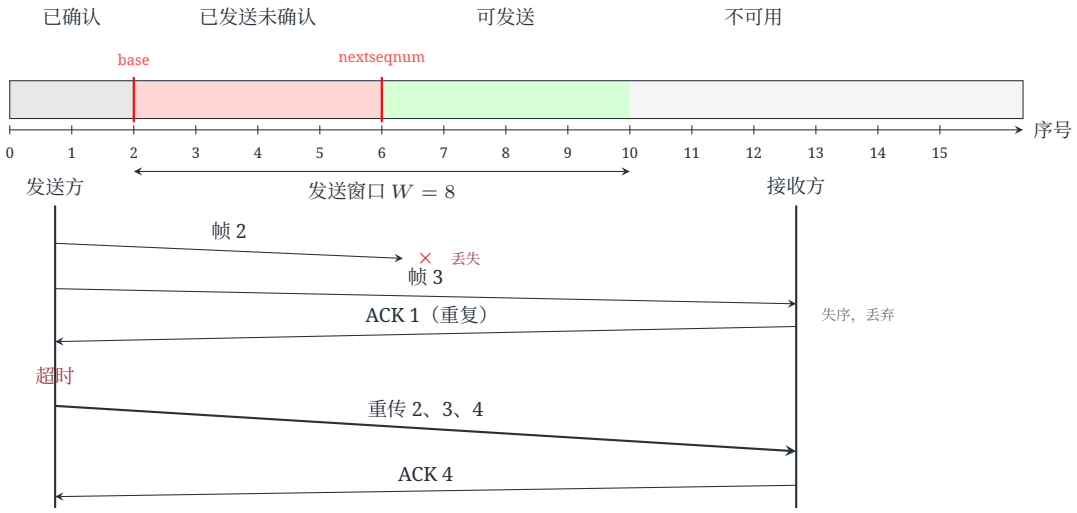


图左边是停等：每发一帧都要等一个 RTT，发送方的“空闲”段很长。右边是流水线：发送方不等确认就连续发出多帧，确认像流水一样回来，链路利用率可以接近 100%。当然，流水线也带来新问题：如果中间某一帧丢了，后面已经到达的帧怎么办？这就是 GBN 和 SR 要解决的事。

后退 N 帧（GBN）

GBN 的发送方维护一个长度为 W 的**发送窗口**，窗口内的序号是“已发送但还没被确认”和“还可以发送”的范围。接收方的窗口长度只有 1：它只按序接收，期望的序号是 `rcv_base`。收到序号等于 `rcv_base` 的帧，就交付给上层、把 `rcv_base` 加 1，并回一个累计确认；收到序号不等于 `rcv_base` 的帧，即使内容正确也直接丢弃，并重复回上一次的确认。

发送方只给窗口内最老的未确认帧设置一个定时器。收到确认号 n 时，表示 n 及之前的帧都已被正确接收，窗口可以滑到 $n+1$。如果最老帧超时，发送方把窗口内所有已发送但未确认的帧全部重传——这就是“后退 N 帧”名字的来源。



图的上半部分画出 GBN 的发送窗口：0、1 已确认，2 是窗口左端（base），2–5 已发送但未确认，6–9 是窗口内还可以发送的序号，10 以后在窗口外。下半部分是一条时间线：帧 2 丢失，帧 3 虽然到达，但接收方期望的是 2，于是把它丢弃并重复回 ACK 1；发送方超时后，把 2、3、4 一起重传。注意帧 3 之前已经正确到达过一次，GBN 仍然要重传它，这就是“后退 N 帧”的代价。

GBN 的接收窗口为 1，所以实现简单、接收方不需要缓存乱序帧；代价是一次丢失可能导致整个窗口重传。链路误码率低时，这种浪费可以接受；误码率高时，GBN 的效率会明显下降。

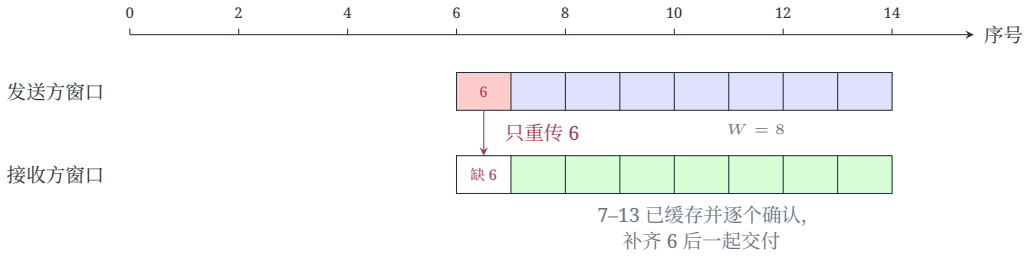
GBN 的序号位数 n 和窗口长度 W 必须满足

$$W \le 2^n - 1.$$

为什么不能取 $W = 2^n$？设 $n = 2$，序号为 0–3，窗口取 4。发送方连续发出 0、1、2、3，四帧都正确到达，接收方回 ACK 0–ACK 3。假如所有 ACK 都丢失，发送方超时后重传 0。此时接收方已经收完 0–3，期望的序号回到下一轮的 0；重传的 0 正好等于期望值，于是被当成新帧接收，上层收到重复数据。取 $W = 2^n - 1$ 时，例如 $n = 2$、$W = 3$，接收方收完 0、1、2 后期望 3；重传的 0 不等于 3，会被丢弃，不会产生重复。

选择重传（SR）

SR 把接收窗口也扩大到 W：接收方可以接收并缓存窗口内的乱序帧，对每一帧单独回确认；当缺失的帧到达、把空洞补上后，再把缓存中连续的帧一起交付给上层。发送方为每个未确认帧维护逻辑定时器，哪一帧超时就只重传哪一帧，而不是重传整个窗口。



图中发送方窗口从 6 开始，7–13 已发送且被接收方缓存，只有 6 缺失。发送方超时后只重传 6；接收方收到 6 后，把 6–13 一起交付，窗口滑到 14。与 GBN 相比，SR 避免了重传已经正确到达的 7–13，但接收方要维护缓存和乱序帧的确认信息，发送方也要为每帧维护定时器，复杂度更高。

SR 的窗口长度必须满足

$$W \le 2^{n-1}.$$

原因是发送窗口和接收窗口不能“重叠”到让旧帧和新帧混淆。设 $n=3$，序号 0–7，若取 $W=5>4$。发送方发出 0–4，接收方全部收到并回 ACK，但 ACK 全丢；接收方窗口滑到 5、6、7、0、1，期望下一轮的 5。发送方超时后重传 0–4，其中 0、1 正好落在接收方的新窗口内，接收方会把它们当成新数据接收，造成重复。取 $W \le 2^{n-1}$ 时，发送方最多重传 W 个旧帧，而接收方窗口已经滑过至少 W 个序号，旧帧不会落入新窗口。

把 GBN 和 SR 放在一起比较：

GBN 与 SR 的对比

项目	GBN	SR
接收窗口	1，只按序接收	W，可缓存乱序帧
确认方式	累计确认	通常逐帧确认
超时处理	重传窗口内所有未确认帧	只重传超时的那一帧
接收方缓存	不需要	需要
发送方定时器	最老未确认帧一个	每个未确认帧一个
序号空间	$W \le 2^n - 1$	$W \le 2^{n-1}$
适用场景	误码率低、实现简单	误码率高、带宽浪费敏感

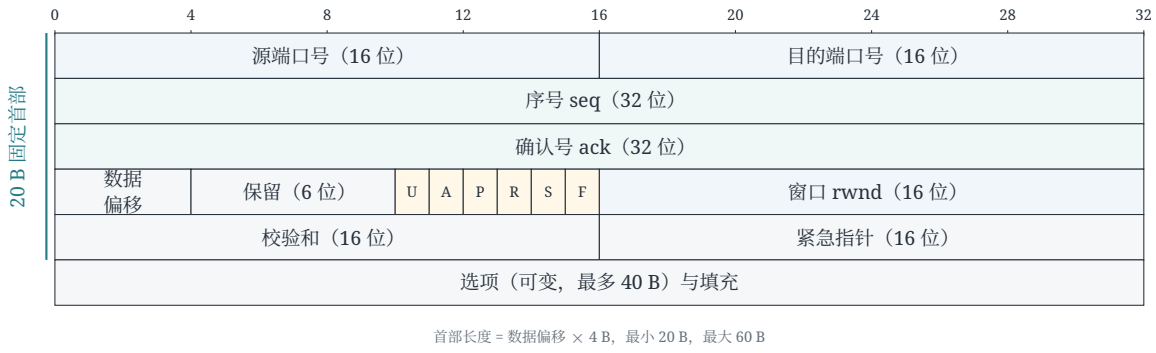
TCP 并不是纯粹的 GBN 或 SR。它使用累计确认，正常情况下像 GBN；但它有快速重传机制，收到 3 个重复 ACK 就立即重传缺失段，而不必等超时；还可以通过 SACK 选项告诉发送方哪些段已经收到，从而只重传真正缺失的部分。可以把它看成“以 GBN 为主、在丢包时借用 SR 思想”的混合方案。

9.4 TCP 报文段与可靠传输

TCP（Transmission Control Protocol）提供面向连接、可靠、全双工、面向字节流的传输服务。它把应用层交下来的数据看成没有边界的字节流，为每个字节编号，用确认号告诉对方“我下一个想收哪个字节”。为了在 IP 之上实现可靠传输，TCP 使用校验和、序号、确认、超时重传和滑动窗口；为了不让发送方压垮接收方或网络，它又加入流量控制和拥塞控制。

首部格式

TCP 报文段首部固定部分 20 字节，加上最多 40 字节选项，最长 60 字节。



首部长度 = 数据偏移 × 4 B，最小 20 B，最大 60 B

几个字段的含义需要记牢。**数据偏移**占 4 位，单位是 4 字节，所以最小值为 5（$5\times4=20$ 字节），最大值为 15（60 字节）。**保留**字段目前一般置 0。六个标志位中，URG 表示紧急指针有效，

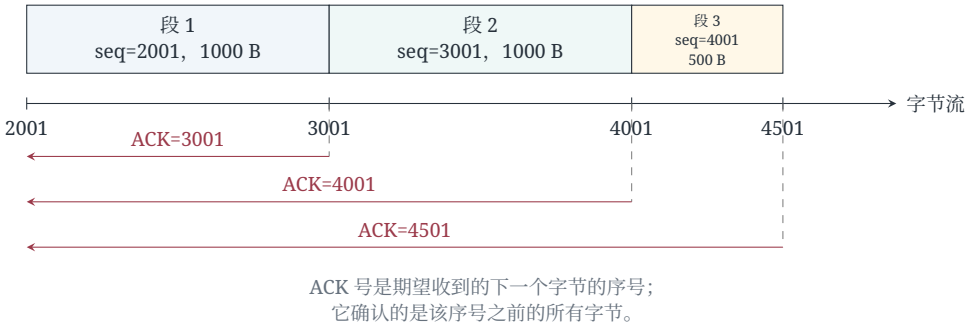
ACK 表示确认号有效，PSH 表示请求接收方尽快交付，RST 用于复位连接，SYN 用于建立连接，FIN 用于释放连接。**窗口**字段是接收窗口 rwnd，单位字节，16 位最大 65535；若使用窗口扩大选项，实际窗口可以按协商的移位值左移。

校验和的计算方法与 UDP 类似，伪首部中的协议号改为 6，长度改为 TCP 长度（首部 + 数据）。TCP 校验和是强制性的。

TCP 的 **MSS**（Maximum Segment Size）指数据部分的最大长度，不包括 TCP 首部。以太网 MTU 为 1500 B 时，减去 20 B IP 首部和 20 B TCP 首部，MSS 通常取 1460 B；MSS 在连接建立时由双方协商。

序号、确认号与字节流

TCP 的序号不是“第几个报文段”，而是“数据部分第一个字节在字节流中的编号”。假设初始序号 ISN 为 2001，发送方发送 1000 字节，则本报文段的序号是 2001，下一个报文段的序号是 3001；接收方正确收到后，回确认号 3001，表示“2001–3000 都收到了，下一个想收 3001”。



ACK 号是期望收到的下一个字节的序号；
它确认的是该序号之前的所有字节。

图里段 1、段 2、段 3 的序号分别是 2001、3001、4001。接收方收到段 1 后回 ACK=3001，收到段 2 后回 ACK=4001，收到段 3 后回 ACK=4501。确认号是**累计**的：ACK=4001 表示 4001 之前的所有字节都已正确收到，而不只是段 2。若段 2 丢失、段 3 先到，接收方仍然回 ACK=3001，表示“我还在等 3001”；发送方连续收到多个这样的重复 ACK，就可以判断 3001 附近丢失，触发快速重传。

TCP 是全双工的，两个方向各自维护序号和确认号。连接建立时，双方通过 SYN 交换初始序号；SYN 本身消耗一个序号，所以客户端发 SYN（seq=x）后，服务器的确认号是 $x+1$。FIN 也消耗一个序号。纯粹的 ACK 报文段如果不携带数据，不消耗序号。

TCP 是面向字节流的，这意味着它不保留应用层的报文边界。应用层三次写入 100 字节，接收方可能一次读到 300 字节，也可能分三次读到 100 字节；应用层必须自己按协议格式解析边界。这一点与 UDP 的“一次发送对应一个数据报”正好相反。

超时重传与 RTT 估计

TCP 为发送出去但尚未确认的数据设置定时器。超时时间 RTO 不能拍脑袋定：太小会频繁误重传，太大会在真正丢包时等待过久。TCP 用往返时延的指数加权移动平均来估计 RTT。记 E 为平滑后的 RTT 估计，D 为 RTT 偏差的估计，S 为本次测量得到的 SampleRTT，则

$$E_{\text{new}} = (1-\alpha)E_{\text{old}} + \alpha S, \qquad \alpha = \frac{1}{8}, \tag{2}$$

$$D_{\text{new}} = (1-\beta)D_{\text{old}} + \beta|S - E_{\text{old}}|, \qquad \beta = \frac{1}{4}, \tag{3}$$

$$\text{RTO} = E_{\text{new}} + 4D_{\text{new}}. \tag{4}$$

第一式让 E 平滑地跟随新样本，第二式估计 RTT 的波动；RTO 在平均值上再加 4 倍偏差，给波动留出余量。举个数值例子：设 $E_{\text{old}} = 100$ ms，$D_{\text{old}} = 10$ ms，本次 $S = 120$ ms，则

$$E_{\text{new}} = 0.875 \times 100 + 0.125 \times 120 = 102.5 \text{ ms},$$

$$D_{\text{new}} = 0.75 \times 10 + 0.25 \times |120 - 100| = 12.5 \text{ ms},$$

$$\text{RTO} = 102.5 + 4 \times 12.5 = 152.5 \text{ ms}.$$

工程实现还会给 RTO 设置上下限，初始 RTO 通常取 1 秒左右；408 计算题一般按题目给出的初值和公式计算。

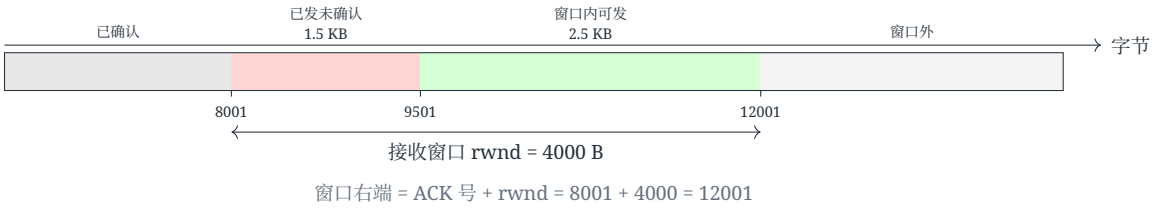
这里有一个容易被忽略的问题：如果报文段被重传过，那么收到的 ACK 到底是对原始发送的确认，还是对重传副本的确认？发送方无法区分。因此 **Karn 算法**规定：**重传过的报文段不参与 RTT 样本的计算**；同时，超时后 RTO 按指数退避（每超时一次翻倍），直到收到一个没有歧义的新数据确认，再恢复正常的 RTT 估计。这样避免把“重传后的 ACK”误当成“原始发送的 ACK”，把 RTT 估计得过大或过小。

9.5 TCP 的流量控制

流量控制解决的是“发送方太快，接收方缓冲区放不下”的问题。接收方在 TCP 首部的窗口字段中告诉发送方：从确认号开始，我还能接收多少字节。这个值叫**接收窗口** rwnd。发送方的实际发送窗口不能超过 rwnd，也不能超过拥塞窗口 cwnd，因此

$$\text{发送窗口} = \min(\text{rwnd}, \text{cwnd}).$$

流量控制是端到端的，只考虑接收方的处理能力；拥塞控制考虑的是整个网络，二者共同限制发送速率。



图中的确认号是 8001，接收方通告 rwnd=4000 B，所以窗口覆盖 8001–12000。发送方已经有 1.5 KB 在途（8001–9500），因此还能发送 2.5 KB（9501–12000）。如果发送方还有更多数据，只能等新的 ACK 把窗口往前推。

当接收方缓冲区满时，它通告 rwnd=0。发送方暂停发送，但不能就此认为连接死了：它会启动**坚持定时器**（persist timer），定期发送**零窗口探测报文**，接收方收到后回复当前窗口大小。这样即使窗口更新报文丢失，双方也不会互相等待而死锁。

窗口字段只有 16 位，最大 65535 字节。在高带宽时延积较大的链路上，这个窗口太小，会限制吞吐量。TCP 的**窗口扩大选项**允许双方在建立连接时协商一个移位值，把窗口字段左移若干位，从而表示更大的窗口。

还有一个细节叫**糊涂窗口综合征**（Silly Window Syndrome）：接收方每次只腾出几个字节就通告一个很小的窗口，发送方于是发出大量首部占主导的小段，效率极低。解决办法分两头：接收方采用 Clark 算法，等到缓冲区有足够空间（例如能放下一个 MSS 或缓冲区的一半）再通告非零窗口；发送方采用 Nagle 算法，在还有未确认数据时先把小段攒起来，等到凑满 MSS 或收到 ACK 再发。Nagle 算法和延迟确认一起使用时，可能让交互式应用多等几十毫秒，所以有些实时应用会关闭 Nagle。

9.6 TCP 的拥塞控制

流量控制防止压垮接收方，拥塞控制防止压垮网络。发送方维护一个**拥塞窗口** cwnd，表示在不知道网络当前能承受多少之前，自己允许有多少数据在途。cwnd 是发送方根据丢包事件推测出来的，不是接收方通告的。真正能发多少仍然取 $\min(\text{rwnd}, \text{cwnd})$。cwnd 和 ssthresh 的单位可以是 MSS 或字节，同一道题里要保持一致；408 题目常用 MSS 为单位。

TCP 拥塞控制有四个经典算法：慢开始、拥塞避免、快重传、快恢复。它们围绕两个变量工作：cwnd 和**慢开始门限** ssthresh。

慢开始从很小的 cwnd 开始（教材通常取 1 MSS）。每收到一个对新数据的 ACK，cwnd 增加 1 MSS；如果所有 ACK 都按时到达，那么每经过一个 RTT，cwnd 翻一倍：1、2、4、8、16……所以“慢开始”其实不慢，它是指数增长，只是起点小。当 cwnd 达到 ssthresh 时，转入拥塞避免。

拥塞避免把指数增长改成线性增长：每经过一个 RTT，cwnd 增加 1 MSS。按单个 ACK 计算，增量约为 MSS^2/cwnd，累加一个 RTT 正好约 1 MSS。这个阶段增长慢，但更稳，通常称为“加法增大”（Additive Increase）。

发生丢包时，TCP 用两种方式判断，处理方式不同：

- **超时**：发送方等不到确认，说明情况可能很严重。此时令

$$\text{ssthresh} = \max\left(\frac{\text{cwnd}}{2},\ 2\ \text{MSS}\right), \qquad \text{cwnd} = 1\ \text{MSS},$$

  然后重新进入慢开始。这就是“乘法减小”（Multiplicative Decrease）。
- **3 个重复 ACK**：发送方连续收到 3 个重复 ACK，说明缺失段之后的报文段还在到达，网络并没有完全堵死。此时执行**快重传**，立即重传缺失段，不必等超时；同时令

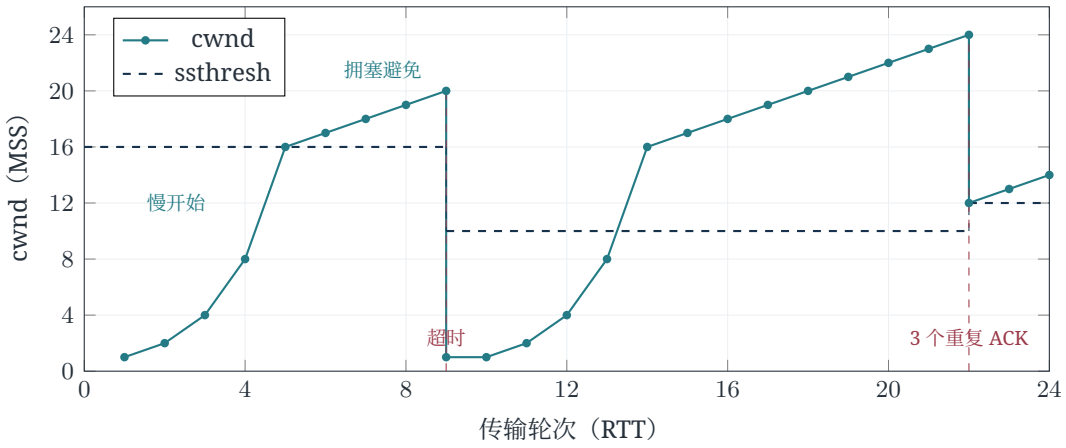
$$\text{ssthresh} = \frac{\text{cwnd}}{2}, \qquad \text{cwnd} = \text{ssthresh},$$

  然后直接进入拥塞避免。这个过程叫**快恢复**：窗口减半，但不回到 1，因为网络显然还在交付数据。

这里的“3 个重复 ACK”是指：在原本的 ACK 之后，又收到 3 个确认号相同、且没有确认新数据的 ACK。发送方据此推断，缺失段之后至少有三个报文段到达了接收方。

> **注意**
>
> **本章统一采用的教材化规则。**408 教材和多数题目对快恢复做了简化：收到 3 个重复 ACK 时，直接把 cwnd 置为减半后的 ssthresh，然后进入拥塞避免。真正的 TCP Reno 会把 cwnd 临时置为 ssthresh $+ 3$ MSS，每多一个重复 ACK 再膨胀 1 MSS，直到收到确认新数据的 ACK 才收缩回 ssthresh。两种写法在考试中可能给出不同数值。本章习题统一使用简化规则；只有题目明确写“TCP Reno 快恢复”时，才使用 $+3$ 的细节。超时处理则统一取 $\text{ssthresh} = \max(\text{cwnd}/2, 2\ \text{MSS})$，除法按题目要求取整，未说明时向下取整到整数 MSS。

还有一个容易出错的细节：慢开始阶段每个 RTT 翻倍，如果翻倍后超过了 ssthresh，**不截断**。例如 ssthresh=10 MSS，cwnd 从 8 MSS 翻到 16 MSS，这一轮仍然按慢开始完成；下一轮才转入拥塞避免。408 的填表题经常在这里设坑。



这张图把一次完整的拥塞事件画了出来。初始 ssthresh=16 MSS：第 1–5 轮 cwnd 按 1、2、4、8、16 指数上升，第 5 轮到达门限；第 6–9 轮转入拥塞避免，每轮加 1，到 20 MSS 时发生超时。超时后 ssthresh 减半为 10 MSS，cwnd 跌回 1 MSS，重新慢开始。第 10–14 轮又按 1、2、4、8、16 上升；注意第 14 轮从 8 翻到 16，越过了新的门限 10，但没有被截断。第 15 轮起进入拥塞避免，每轮加 1，直到 24 MSS 时收到 3 个重复 ACK。此时 ssthresh 减半为 12 MSS，cwnd 置为 12 MSS，直接进入拥塞避免，所以曲线上只下降到 12，而不是跌到 1。

对比超时和 3 个重复 ACK 这两处下落，可以看到：超时是“跌到 1 再指数爬升”，3 个重复 ACK 是“减半后线性爬升”。这就是 TCP 拥塞控制最典型的锯齿形。图中的 ssthresh 用阶梯虚线表示：$16 \to 10 \to 12$，每次都在丢包事件发生时更新。

拥塞控制的效果是整个网络收益：所有 TCP 连接在丢包时都减半，在正常时慢慢加一，总体呈现“加法增大、乘法减小”的 AIMD 行为。它不能保证公平，但能在没有集中调度的情况下让网络大致稳定运行。

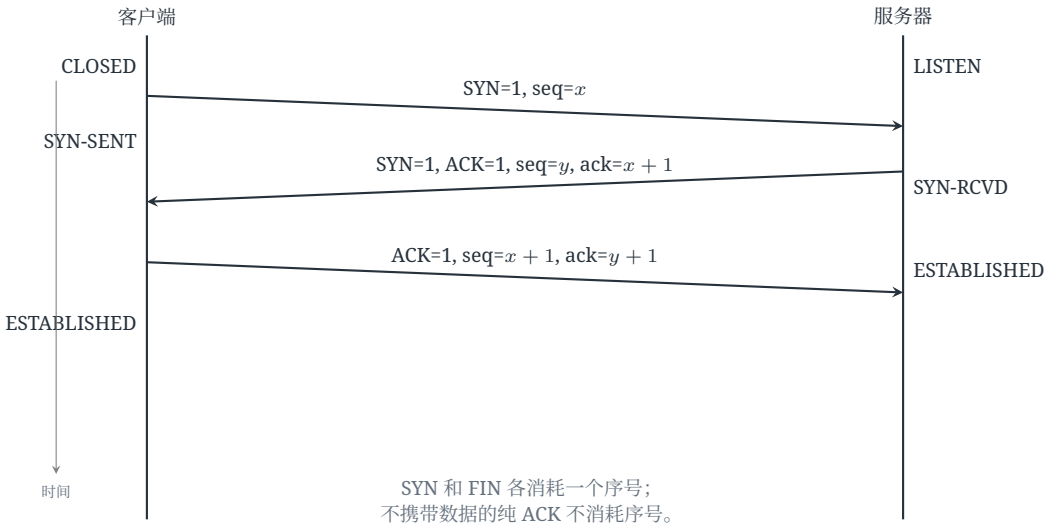
9.7 连接管理

TCP 是面向连接的，通信前要建立连接，通信后要释放连接。连接建立和释放的过程不仅是“打招呼”，还要交换初始序号、确认双方的收发能力，并处理旧报文段和异常情况。

三次握手

三次握手的过程如下：

1. 客户端发送 `SYN=1, seq=x`，进入 `SYN-SENT` 状态。这个报文段不携带数据，但消耗一个序号 x。
2. 服务器收到后，回复 `SYN=1, ACK=1, seq=y, ack=x+1`，进入 `SYN-RCVD` 状态。它确认了客户端的 x，同时告诉客户端自己的初始序号是 y。
3. 客户端收到后，发送 `ACK=1, seq=x+1, ack=y+1`，进入 `ESTABLISHED` 状态。服务器收到这个 ACK 后也进入 `ESTABLISHED`。



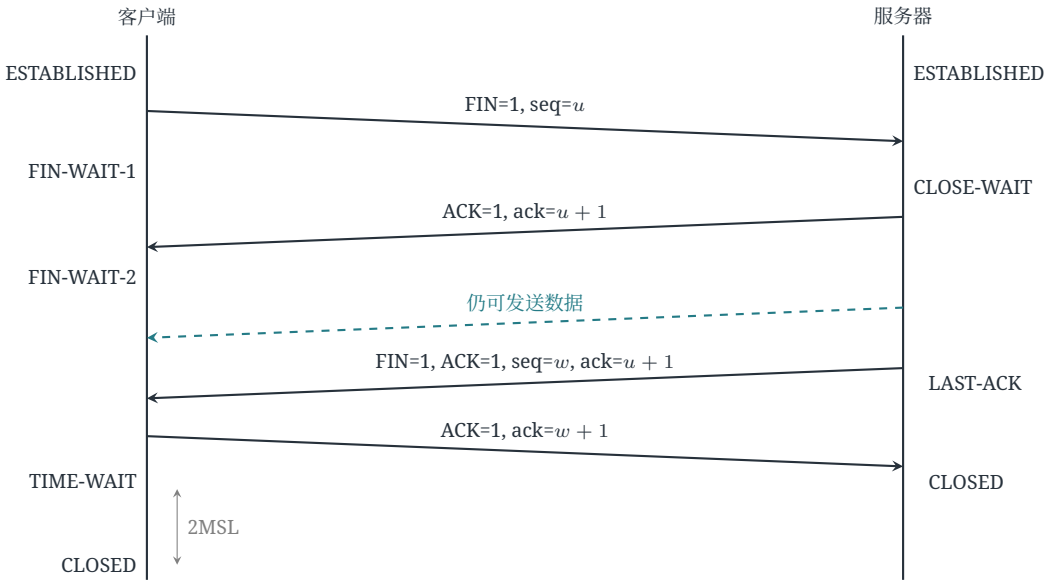
为什么需要三次，而不是两次？核心原因是防止**已失效的连接请求**突然到达服务器。假设客户端很早以前发出的一个 SYN 在网络中滞留，客户端已经放弃并重新发起连接；如果只有两次握手，这个迟到的 SYN 到达服务器后，服务器会直接认为连接建立并分配资源，而客户端根本不会使用它，资源就被浪费了。三次握手中，服务器收到旧 SYN 后回 SYN+ACK，客户端看到不认识的确认号，会回 RST 拒绝，服务器就不会建立连接。

三次握手还让双方都确认了对方的发送和接收能力：客户端从 SYN+ACK 知道服务器能收也能发，服务器从第三次 ACK 知道客户端能收。同时，双方交换了初始序号，为后续的可靠传输打下基础。

四次挥手与 TIME-WAIT

TCP 是全双工的，两个方向要分别关闭，所以释放连接通常需要四次报文交换：

1. 客户端发送 `FIN=1, seq=u`，进入 `FIN-WAIT-1`。FIN 消耗一个序号。
2. 服务器回复 `ACK=1, ack=u+1`，进入 `CLOSE-WAIT`。客户端收到后进入 `FIN-WAIT-2`。此时客户端到服务器方向已经关闭，但服务器还可以继续发送数据，这叫**半关闭**。
3. 服务器把剩余数据发完后，发送 `FIN=1, ACK=1, seq=w, ack=u+1`，进入 `LAST-ACK`。
4. 客户端回复 `ACK=1, ack=w+1`，进入 `TIME-WAIT`，等待 2MSL 后进入 `CLOSED`。服务器收到这个 ACK 后立即进入 `CLOSED`。



图中服务器收到 FIN 后进入 CLOSE-WAIT，它还可以把剩余数据发完（虚线箭头），然后才发自己的 FIN。客户端回复最后一个 ACK 后进入 TIME-WAIT，等待 2MSL 才真正关闭。

TIME-WAIT 要等 2MSL，原因有两个。第一，最后一个 ACK 可能丢失；如果服务器没收到 ACK，会重发 FIN，客户端在 TIME-WAIT 期间还能再回一次 ACK。如果客户端发完 ACK 就立即关闭，服务器重发的 FIN 就没人理，服务器会一直停在 LAST-ACK。第二，等待 2MSL 可以让本次连接产生的所有旧报文段在网络中消失，避免它们干扰下一个使用相同四元组的新连接。MSL 是报文段最大生存时间，教材常取 2 分钟，因此 2MSL 约为 4 分钟；具体实现可能更短。

TCP 的完整状态比上面两条路径多。建立连接涉及 CLOSED、LISTEN、SYN-SENT、SYN-RCVD、ESTABLISHED；关闭连接涉及 FIN-WAIT-1、FIN-WAIT-2、CLOSE-WAIT、LAST-ACK、TIME-WAIT。如果双方同时关闭，还会出现 CLOSING 状态；408 选择题偶尔考状态名称，但重点是主动关闭方和被动关闭方各自经过哪些状态。

TCP 连接状态速查

状态	所处位置	含义与典型转换
CLOSED	起点/终点	无连接；被动打开进入 LISTEN，主动打开进入 SYN-SENT
LISTEN	服务器	等待 SYN；收到 SYN 后回 SYN+ACK，进入 SYN-RCVD
SYN-SENT	客户端	已发 SYN；收到 SYN+ACK 后回 ACK，进入 ESTABLISHED
SYN-RCVD	服务器	已收 SYN 并回 SYN+ACK；收到 ACK 后进入 ESTABLISHED
ESTABLISHED	双方	正常数据传输；收到 FIN 进入 CLOSE-WAIT，主动关闭进入 FIN-WAIT-1
FIN-WAIT-1	主动关闭方	已发 FIN；收到 ACK 进入 FIN-WAIT-2，收到 FIN+ACK 可直接进入 TIME-WAIT
FIN-WAIT-2	主动关闭方	对方已确认本方的 FIN；收到对方 FIN 后进入 TIME-WAIT
CLOSE-WAIT	被动关闭方	已收到 FIN；应用关闭后发 FIN，进入 LAST-ACK
LAST-ACK	被动关闭方	已发 FIN；收到最后 ACK 后进入 CLOSED
TIME-WAIT	主动关闭方	等待 2MSL，超时后进入 CLOSED

连接管理还要注意异常情况。如果收到不匹配的报文段，或者端口没有进程监听，主机可能回 RST 复位连接。SYN 洪泛攻击利用大量伪造源地址的 SYN 把服务器的半连接队列占满，使正常客户无法完成三次握手；常见的防御手段包括 SYN cookie、缩短半连接超时、限制半连接数量等。

9.8 把本章串起来

传输层的核心问题是“进程到进程”的交付，端口号负责复用和分用。UDP 只做端口和校验，保留报文边界，不保证可靠，也不做流量控制和拥塞控制；它的简单换来了低开销和低时延。TCP 在 IP 之上用序号、确认、定时器和滑动窗口实现可靠字节流：序号按字节编号，确认号是期望收到的下一个字节；超时重传用 RTT 估计和 Karn 算法确定时机；流量控制用 rwnd 防止压垮接收方；拥塞控制用 cwnd 和 ssthresh 防止压垮网络；连接管理用三次握手和四次挥手建立、释放连接。

可靠传输的通用机制可以单独拿出来比较：停等协议窗口为 1，链路利用率低；GBN 用接收窗口 1 和累计确认换取简单，但一次丢失会重传整个窗口；SR 用接收缓存和逐帧确认换取效率，但实现复杂。GBN 的窗口上限是 2^n-1，SR 是 2^{n-1}，这两个不等式来自“序号空间必须大于窗口，否则旧帧会冒充新帧”。

本章关键数字与结论

项目	结论
端口与首部	端口 16 位；UDP 首部 8 B；TCP 固定首部 20 B、最长 60 B，数据偏移单位 4 B
TCP 编号与窗口	seq/ack 各 32 位；rwnd 16 位；发送窗口 $=\min(\text{rwnd},\text{cwnd})$
GBN / SR	GBN 接收窗口 1、累计确认、$W\le 2^n-1$；SR 逐帧确认、$W\le 2^{n-1}$
拥塞控制	慢开始指数增长；拥塞避免每 RTT 加 1 MSS；超时 cwnd=1；3 个重复 ACK 减半

10 习题

10.1 基础过关题

1. （单项选择题）主机甲的应用进程使用 UDP 连续三次调用 `sendto`，每次交付 100 B 数据；主机乙的应用进程每次调用 `recvfrom` 时都提供一个 400 B 的接收缓冲区，且三次数据报均正确到达。下列叙述正确的是

(A) 乙调用 1 次即可收到 300 B　　(B) 乙必须调用 3 次，每次收到 100 B

(C) 乙可能 1 次收到 300 B，也可能分 3 次收到　　(D) 乙调用 2 次，分别收到 100 B 和 200 B

2. （单项选择题）某 UDP 数据报的源端口号为 `0x1234`，目的端口号为 `0x5678`，数据部分为 4 B：`0x01 0x02 0x03 0x04`。其 IP 首部中源 IP 地址为 192.168.1.1，目的 IP 地址为 192.168.1.2。发送方计算校验和时先把校验和字段置 0。该 UDP 数据报的校验和为

(A) `0x0FD0`　(B) `0xF02F`

(C) `0x9341`　(D) `0xFFFF`

3. （填空题）主机甲、乙已建立 TCP 连接，此前收发的数据均已被确认。甲当前待发送数据的第一个字节序号为 3001。甲发送一个报文段：携带 500 B 数据，同时捎带确认乙已按序送达的 8000 号字节及其之前的全部数据。若乙按序正确收到该报文段，则该报文段首部的序号为 _____，确认号为 _____；乙随后发回的确认号为 _____。

4. （单项选择题）关于 TCP 连接建立与释放过程中的状态和标志位，下列叙述正确的是

(A) 服务器收到第一个 SYN 后立即进入 ESTABLISHED 状态

(B) 主动关闭方发送 FIN 后立即进入 FIN-WAIT-2 状态

(C) 第三次握手的 ACK 报文段可以携带数据，所携带的数据要消耗序号

(D) TIME-WAIT 状态由被动关闭方在发出 FIN 后进入

5. （单项选择题）主机甲与乙的 TCP 连接中 MSS$=1$ KB，某时刻甲的 cwnd$=18$ KB、ssthresh$=16$ KB。甲连续收到 3 个重复 ACK，于是执行快重传。本题采用教材化规则：3 个重复 ACK 后 ssthresh 取当时 cwnd 的一半，cwnd 置为该 ssthresh 并进入拥塞避免。若随后 1 个 RTT 内发送的数据全部被确认，且无新的拥塞事件，则该 RTT 结束时 cwnd 为

(A) 9 KB　(B) 10 KB

(C) 12 KB　(D) 18 KB

6. （单项选择题）某 TCP 连接当前的 RTT 估计值为 EstimatedRTT $=100$ ms，偏差为 DevRTT $=10$ ms。发送方发送一个报文段后超时并重传，随后收到该报文段的确认（无法判断该确认对应原始报文还是重传副本）。之后发送方发送一个新报文段，测得无歧义的 SampleRTT $=140$ ms。按 Jacobson/Karels 算法（$\alpha=0.125$，$\beta=0.25$），更新后的 RTO 为

(A) 105 ms　(B) 140 ms

(C) 175 ms　(D) 180 ms

10.2 真题风格与综合训练

1. （单项选择题）主机甲与乙使用滑动窗口协议通信，帧序号用 3 位二进制表示，即序号为 0～7。甲连续发送了序号 0～6 共 7 帧，其中 2 号帧丢失，其余 6 帧均正确到达乙，随后甲发生超时（所有确认均未丢失）。若分别采用后退 N 帧（GBN）和选择重传（SR），则甲需要重传的帧数分别为

   (A) 5 帧和 1 帧　(B) 7 帧和 1 帧

   (C) 5 帧和 5 帧　(D) 7 帧和 5 帧

2. （单项选择题）主机甲与乙已建立 TCP 连接，MSS$=1$ KB。某时刻甲收到乙的一个确认报文段，其确认号为 4001、窗口字段为 6 KB（6144 B）。此时甲已发送但尚未收到确认的数据恰好为 1 KB（序号 4001～5024），甲的 cwnd $=8$ KB。在收到下一个确认之前，甲最多还能发送多少字节的新数据

   (A) 5 KB　(B) 6 KB

   (C) 7 KB　(D) 8 KB

3. （综合应用题）主机甲与乙建立 TCP 连接，MSS$=1$ KB。初始 cwnd $=1$ KB、ssthresh $=8$ KB，甲始终有数据要发送。传输过程依次发生：

   (i) cwnd 按慢开始、拥塞避免规则增长，直至 cwnd 达到 12 KB 时发生**超时**；

   (ii) 超时后 cwnd 重新增长，到 cwnd 达到 10 KB 时收到 **3 个重复 ACK**；

   (iii) 此后又成功传输 2 个 RTT，未再发生任何拥塞事件。

   本题统一采用教材化规则：超时令 ssthresh $=\max(\text{cwnd}/2,\ 2\ \text{MSS})$、cwnd $=1$ MSS；3 个重复 ACK 令 ssthresh $=\text{cwnd}/2$、cwnd $=$ ssthresh 并进入拥塞避免；慢开始阶段每经过 1 个 RTT cwnd 翻倍，拥塞避免阶段每经过 1 个 RTT cwnd 加 1 MSS；慢开始翻倍后即使超过 ssthresh 也不截断；所有除法向下取整到整数 MSS。

   (1) 写出全过程每个 RTT 结束时的 cwnd，标出各阶段名称及两次拥塞事件的位置。

   (2) 求最终的 cwnd 与 ssthresh。

   (3) 按“每个 RTT 发送 cwnd 个 MSS 的新数据（不含重传）”计数，求全过程共发送多少数据（KB）；并求从超时到 3 个重复 ACK 之间发送了多少数据、经历了几个 RTT。

4. (综合应用题) 主机甲(客户端)与主机乙(服务器)建立 TCP 连接, 甲选定的初始序号 $\mathrm{ISN}_A = 1000$, 乙选定的初始序号 $\mathrm{ISN}_B = 2000$。所有报文段均正确按序到达, 未发生重传。连接建立后依次发生:

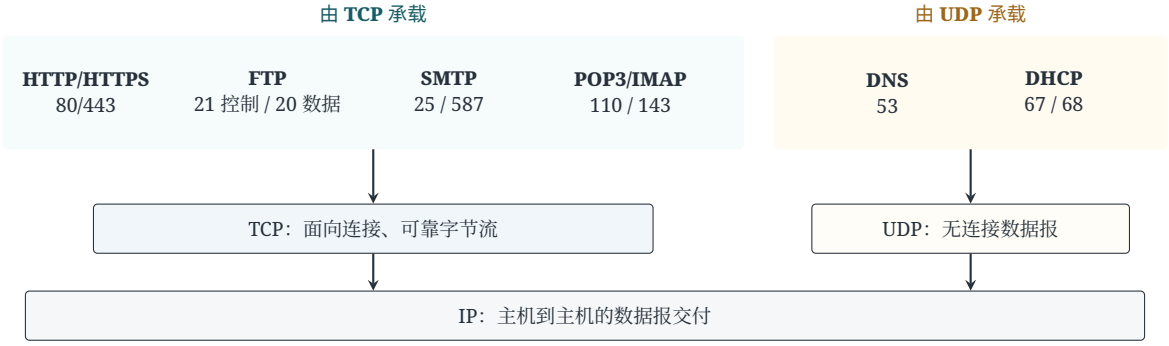
   (1) 甲发送 SYN;
   (2) 乙回复 SYN+ACK;
   (3) 甲发送 ACK;
   (4) 甲发送 600 B 数据(捎带确认);
   (5) 乙发送 200 B 数据(捎带确认);
   (6) 甲发送 ACK;
   (7) 甲发送 FIN(此时甲已无数据待发);
   (8) 乙发送 ACK;
   (9) 乙发送 400 B 数据;
   (10) 甲发送 ACK;
   (11) 乙发送 FIN;
   (12) 甲发送 ACK。

   (1) 写出 (1)~(12) 每个报文段的序号字段与确认号字段的取值, 并注明 SYN、FIN 标志位。
   (2) 乙收到 (12) 后进入什么状态? 甲发送 (12) 之后还要停留多久才能进入 CLOSED? 设 $\mathrm{MSL} = 2$ min。
   (3) 若甲在 (7) 时还有 100 B 数据未发完, 能否把数据与 FIN 放在同一报文段中发送? 若能, 写出该报文段的序号与确认号, 并给出甲下一个报文段应使用的序号。

11 应用层

应用层是协议栈里离用户最近的一层。浏览器地址栏里的一次回车、邮件客户端的一次发送、文件传输工具的一次下载，背后都是应用层协议在约定报文长什么样、谁先发、收到什么算成功。应用层不负责把比特送到对端，那是下面几层的事；它只定义通信双方交换的消息和动作。传输层用端口号把数据交给正确的进程，所以每种应用协议都要选定 TCP 或 UDP，并占用一个（或一组）端口。

应用层协议可以直接架在 TCP 上，也可以直接用 UDP。图里把常见的几种协议按传输层分了组，端口号写在协议名下面。DNS 的普通查询走 UDP 53，只有在响应被截断或者做区域传送时才改用 TCP 53；DHCP 则完全依赖 UDP 广播，因为客户端开口问的时候还没有 IP 地址。HTTP、FTP、SMTP、POP3/IMAP 都要求可靠传输，所以用 TCP。



图中上半部分是常见应用协议及默认端口，下半部分说明它们架在哪种传输层协议上。端口号是默认约定，服务器可以改，但考题和抓包通常按默认值认协议。

大多数应用协议采用客户/服务器模型：客户主动发起请求，服务器被动等待。DNS 存根解析器、浏览器、FTP 客户端、邮件用户代理都是客户；DNS 权限服务器、Web 服务器、FTP 服务器、邮件服务器都是服务器。P2P 应用则把每个节点同时当作客户和服务器，文件分发和区块链是典型例子。

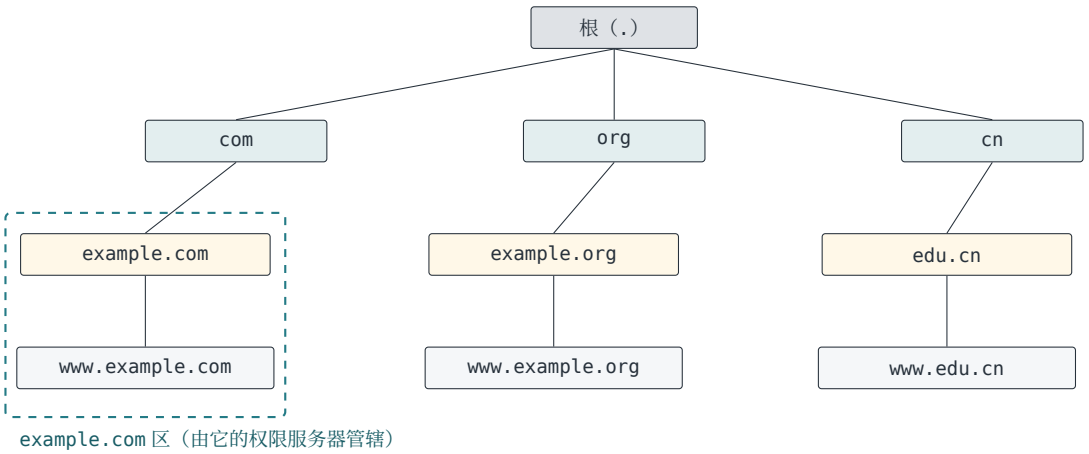
11.1 DNS：把域名换成 IP 地址

在浏览器里输入 `www.example.com`，浏览器并不知道它的 IP。操作系统里的存根解析器（stub resolver）会把这个问题交给本地域名服务器，本地服务器可能还要去问根、顶级域和权限服务器。整个系统要做的事情不止“域名换 IP”：它还要保存邮件路由（MX）、别名（CNAME）、反向解析（PTR）等信息。DNS 就是这些记录的分布式数据库。

域名空间是一棵树

域名从右往左读。`www.example.com.` 最后那个点代表根，`com` 是顶级域，`example` 是二级域，`www` 是主机名。每个标号（label）不超过 63 个字符；整个域名的文本表示不超过 253 个字符，DNS 线上编码不超过 255 字节。域名不区分大小写，但保留原始大小写方便显示。

域和区不是一回事。域是名字空间里的一棵子树；区（zone）是某台权限域名服务器实际管辖的那一部分。一个域可以划分成多个区，把子域委派给下级服务器；一个区只属于一个域，不能跨域。图里虚线框标出的就是 `example.com` 区：它包含 `example.com` 本身和下面的主机 `www`，而 `com` 区只保留指向它的 NS 委派记录和必要的 glue 记录。



图中根在最上面，名字从上往下逐级变长；实际书写时从左往右写，解析时从右往左读。虚线框说明“区”可以小于“域”：com 区把 example.com 委派出去后，只保留 NS 委派记录和必要的 glue 记录。

DNS 的常见资源记录如下。考试最常考 A、CNAME、MX、NS 和 PTR 的区别：A 是最终地址，CNAME 是别名，MX 指向邮件服务器并带优先级，NS 指向某个区的权限服务器，PTR 用于反向解析。

DNS 常见资源记录

类型	作用
A	域名到 IPv4 地址的映射
AAAA	域名到 IPv6 地址的映射
CNAME	把一个域名指向另一个域名（别名）
NS	某个区的权限域名服务器
MX	邮件交换服务器，带优先级，数值越小越优先
PTR	IP 地址到域名的反向映射
SOA	区的起始授权记录，描述区的主服务器和管理参数
TXT	任意文本，常用于 SPF、DKIM 等验证

四类服务器各管一段

根域名服务器不保存所有域名。它只保存根区：每个顶级域的 NS 记录，以及必要的 glue A/AAAA 记录（比如 com 的服务器地址）。根服务器有 13 个标识（a.root-servers.net 到 m.root-servers.net），通过任播在全球部署大量实例，所以“13 台机器”是误解。

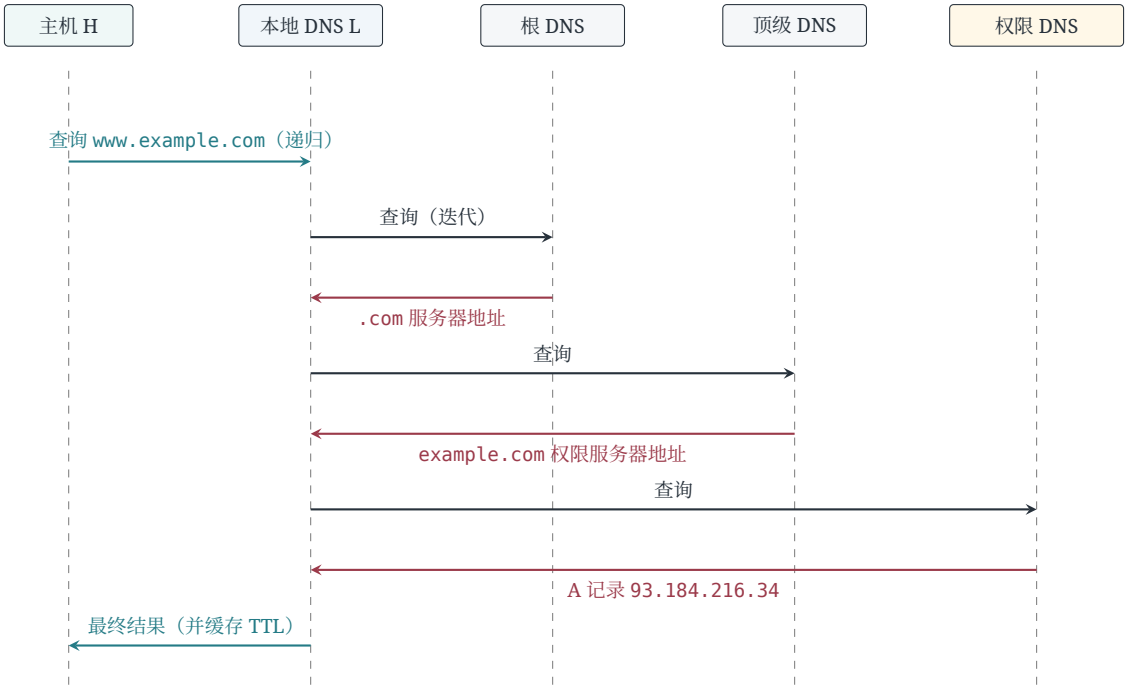
顶级域服务器管 .com、.org、.cn 这些顶级域，返回下一级区的权限服务器地址。权限域名服务器保存某个区的完整数据，能给出最终答案。本地域名服务器（也叫递归解析器）是主机访问 DNS 的第一站，它替主机完成后续查询并缓存结果；它不属于“根—顶级—权限”的层次结构，但通常离主机很近。

递归查询和迭代查询

递归查询是“你替我查到底”：被问的服务器如果不知道答案，必须自己再去问别的服务器，最后把最终结果或者失败返回给请求者，不能只给一个“去问某某”的转介。迭代查询是“你告诉我下一步问谁”：被问的服务器只返回下一台服务器的地址（转介），由请求者自己继续问。

主机到本地域名服务器通常用递归，因为主机不想承担逐级查询的负担；本地域名服务器到根、顶级、权限服务器通常用迭代，因为递归解析器自己有能力继续问，而且这样根服务器不用

替每个客户跑完全程。这不是协议强制的：服务器可以在响应里把递归可用（RA）标志置 0，拒绝递归请求；实际使用哪种方式取决于配置。



图中实线是请求，深色回箭头是应答，时间从上往下。根和顶级服务器只给“下一站”地址，真正给出最终 A 记录的是权限服务器；本地 DNS 把结果缓存后，TTL 内再有主机问同一名字就不用再往上跑。

这条路径一共 8 条报文：4 次请求、4 次应答。如果本地 DNS 的缓存里已经有未过期的记录，第一步就能直接回答，后面的查询都不会发生；如果某台服务器返回了 glue 记录，也可能少问一次。考试里常见的“共发出几条查询报文”要按题设的缓存状态和查询方式数，不能死记 8。

缓存、报文格式与传输层选择

DNS 的每条记录都有 TTL（生存时间）。本地服务器拿到记录后缓存一段时间，TTL 到期就删除；它也可以缓存否定答案（域名不存在），但否定缓存的 TTL 通常更短。缓存能加快解析、减轻根服务器压力，代价是记录变更后旧值可能在一段时间内仍然生效。主机的 `hosts` 文件是本地静态配置，查询时先于 DNS 使用，但它不是 DNS 缓存。

DNS 查询和响应共用一种报文格式：12 字节首部（事务 ID、标志、问题数、回答数、授权数、附加数），后面是问题区、回答区、授权区和附加区。事务 ID 用来把响应和请求配对。普通查询走 UDP 53；响应超过 UDP 允许的长度被截断（TC 标志置 1）时，客户端改用 TCP 53 重查；区域传送（AXFR/IXFR）也走 TCP 53。EDNS0 允许更大的 UDP 载荷，所以“超过 512 字节一定用 TCP”是旧模型的简化说法。

11.2　HTTP：一次请求到底发生了什么

在地址栏输入 `http://www.example.com/index.html`，浏览器先做 DNS，再和服务器建立 TCP 连接，然后发一条 HTTP 请求，服务器返回响应。HTTP 只规定报文格式和交互规则，不规定浏览器怎么渲染页面，也不规定服务器用什么编程语言。

URL 的一般形式是 `scheme://host:port/path?query#fragment`。HTTP 默认端口 80，HTTPS 默认 443，省略端口就用默认值；片段（fragment）只在浏览器内部使用，不会发给服务器。URL 里的非 ASCII 字符和保留字符要做百分号编码。

请求报文和响应报文

请求报文和响应报文都是四段：起始行、首部行、空行、实体主体。空行不能省；没有主体时，空行后面直接结束。请求行是“方法 + 请求目标 + 版本”，响应状态行是“版本 + 状态码 + 原因短语”。首部行是“字段名: 值”，字段名大小写不敏感。图里左边是一个请求，右边是一个响应，颜色区分了四段。



图中左右两列的结构完全对应：起始行、若干首部行、一个空行、可选的主体。请求的起始行是“做什么”，响应的起始行是“结果如何”。

常见方法里，GET 取资源，参数放在 URL 里；POST 提交数据，参数放在主体里；HEAD 只要首部，不返回主体；PUT 上传或替换资源；DELETE 删除资源；OPTIONS 查询支持的方法。GET、HEAD、PUT、DELETE 都是幂等的：重复执行同样的请求，服务器状态的效果一样。POST 不保证幂等，所以不能随意重放。

HTTP 常见方法与状态码

方法	典型用途
GET	获取资源，参数在 URL 中，安全且幂等
POST	提交数据，参数在主体中，不保证幂等
HEAD	与 GET 相同，但只返回首部
PUT	上传或替换指定资源，幂等
DELETE	删除指定资源，幂等
OPTIONS	查询服务器支持的方法或跨域策略

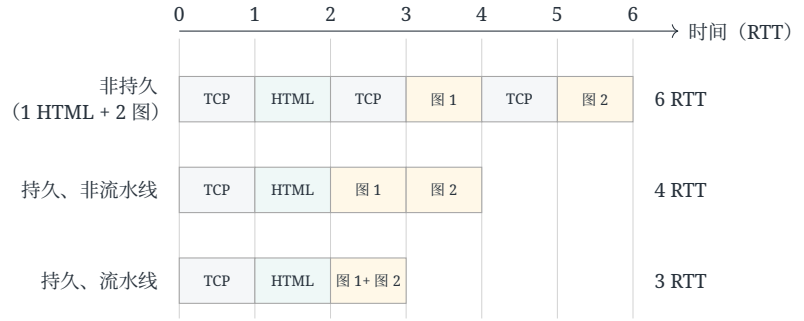
状态码	含义
200 / 204 / 206	成功 / 成功但无主体 / 部分内容
301 / 302 / 304	永久重定向 / 临时重定向 / 缓存仍然有效
400 / 401 / 403 / 404	请求有误 / 需要认证 / 拒绝访问 / 资源不存在
500 / 502 / 503 / 504	服务器内部错误 / 网关收到无效响应 / 服务不可用 / 网关超时

状态码的第一位决定大类：2xx 成功，3xx 重定向，4xx 客户端错误，5xx 服务器错误。301 表示资源永久搬家，客户端以后应该直接用新地址；302 是临时搬家，原地址仍然有效；304 不是错误，它表示条件请求里的缓存副本还没过期，服务器不传主体。401 是“你没认证”，403 是“认证了但没权限”，这两个容易混。

非持久连接、持久连接与流水线

HTTP/1.0 默认非持久连接：每个对象一条 TCP 连接，传完就关。HTTP/1.1 默认持久连接：一条 TCP 连接可以依次传多个对象，除非请求或响应里写了 `Connection: close`。持久连接省掉了反复建立和释放 TCP 连接的开销，是 HTTP/1.1 最重要的改进之一。

图里用三个时间轴对比三种方式。假设页面有 1 个基本 HTML 文件和 2 幅图像，忽略传输时间，每个小格是 1 RTT。非持久方式下每个对象都要“1 RTT 建连接 + 1 RTT 请求响应”，3 个对象共 6 RTT；持久非流水线方式只建一次连接，然后 HTML、图 1、图 2 依次各用 1 RTT，共 4 RTT；流水线方式在收到 HTML、知道图像 URL 后，一次性把两个图像请求都发出去，图像阶段只等 1 RTT，共 3 RTT。



图中灰色格是 TCP 连接建立，青色格是基本 HTML，橙色格是内嵌对象。非持久方式每个对象都要重新建连接；持久方式复用同一条连接；流水线把多个请求一次性发出，所以图像阶段从 2 RTT 压成 1 RTT。

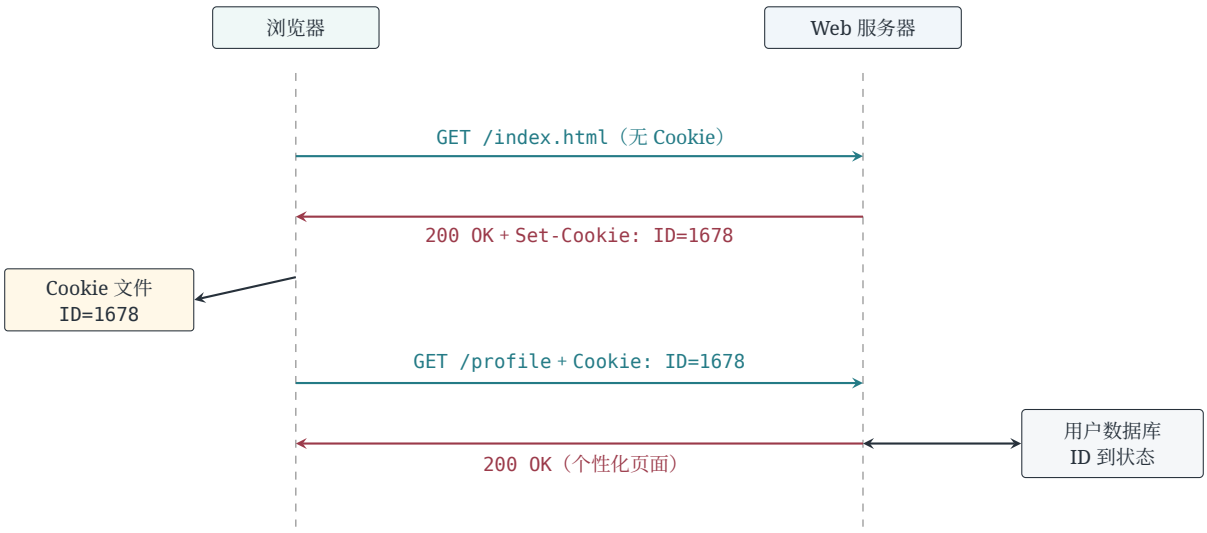
把对象数推广一下。设页面共有 M 个对象（1 个基本 HTML 加 n 个内嵌对象，$M = n + 1$），忽略传输时间：

- 非持久、串行：每个对象 2 RTT，共 $2M = 2(n + 1)$ RTT。
- 持久、非流水线，且必须先取到 HTML 才知道对象 URL：建连接 1 RTT，取 HTML 1 RTT，再逐个取 n 个对象 n RTT，共 $n + 2$ RTT。
- 持久、流水线：建连接 1 RTT，取 HTML 1 RTT，所有内嵌对象一起请求、一起响应 1 RTT，共 3 RTT（$n \geq 1$）。
- 如果客户端在建连接前就已经知道所有对象 URL，持久非流水线是 $1 + M$ RTT，流水线是 2 RTT（建连接 1 RTT，所有请求-响应 1 RTT）。

这些式子都假设服务器按序响应、每个请求-响应恰好 1 RTT、传输时间可以忽略。题目如果给出并行连接数 k，对象阶段要按并行度分摊；如果给出带宽和对象大小，就把传输时间单独加在 RTT 之外。实际浏览器通常开多个并行连接，所以“非持久串行”是理论上的最坏情况。HTTP/2 在一条 TCP 连接上多路复用多个请求，HTTP/3 则改用基于 UDP 的 QUIC，但 408 的时间计算仍以 HTTP/1.1 的模型为主。

Cookie 与 Web 缓存

HTTP 本身是无状态协议：服务器处理完一个请求就忘了是谁来过。Cookie 是在应用层补上会话状态的机制。图里第一次访问时，服务器在响应里用 `Set-Cookie` 下发一个标识；浏览器把它存进 Cookie 文件；之后每次访问同一站点，浏览器自动在请求里用 `Cookie` 首部带回这个标识；服务器拿它去数据库里查用户状态。



图中第一次响应下发 Set-Cookie，浏览器保存；第二次请求自动带回 Cookie，服务器据此查数据库。Cookie 本身通常只是一个随机标识，真正的用户状态在服务器端。

Cookie 的常用属性有：Expires 或 Max-Age 控制过期时间，不设就是会话 Cookie，关浏览器即失效；Domain 和 Path 控制作用域；Secure 表示只在 HTTPS 下发送；HttpOnly 表示 JavaScript 读不到，能降低 XSS 窃取风险；SameSite 限制跨站请求是否携带 Cookie，用于缓解 CSRF。Cookie 不是认证协议本身，它只是携带标识，服务器仍要校验这个标识对应的会话是否有效。

Web 缓存（代理缓存）放在客户端和源服务器之间。缓存命中就直接返回；未命中就替客户端向源服务器取，再存一份。客户端可以用条件 GET 检查缓存是否还新鲜：请求里带 If-Modified-Since 或 If-None-Match，服务器判断没变就回 304，不传主体。Cache-Control: max-age=... 给出新鲜期，no-store 表示不允许缓存。HTTPS 就是 HTTP over TLS，默认端口 443；TLS 握手会增加 RTT，但 HTTP 报文本身的格式不变。

11.3 FTP：两条连接分工

用户用 FTP 登录服务器，先看到目录列表，再下载一个文件。目录列表和文件内容都走数据连接，登录、切换目录、请求文件的命令和应答走控制连接。FTP 的特别之处就在这里：它同时使用两条 TCP 连接，一条传命令，一条传数据，这种把控制信息带外（out-of-band）传送的方式，和 HTTP 把命令和数据放在同一条连接里的做法形成对比。

控制连接由客户端主动发起，连到服务器的 21 端口，在整个会话期间保持。数据连接每传一个文件或一份目录列表时单独建立，传完就关闭；下一次传输再建一条新的。控制连接上传的是 NVT ASCII 命令和三位数字应答：1xx 表示准备就绪，2xx 表示成功，3xx 表示需要更多信息，4xx 表示暂时失败，5xx 表示永久失败。



图中上半段是控制连接的命令与应答，下半段是单独的数据连接。数据连接的方向取决于主动/被动模式，但控制连接永远由客户端发起。

数据连接由谁发起，决定了 FTP 的两种模式。主动模式（PORT）下，客户端告诉服务器自己的 IP 和监听端口 N，服务器从默认的 20 端口主动连到 N。问题在于客户端常常在 NAT 或防火墙后面，外来的连接会被挡住。被动模式（PASV）下，客户端发 PASV，服务器返回一个临时端口 M，由客户端主动连到服务器的 M 端口；所有连接都由客户端发起，防火墙友好，所以现代客户端默认用被动模式。

区分两种模式的关键是“数据连接由谁发起”，不是“用哪个端口”。主动模式下服务器用 20 端口作为源端口；被动模式下服务器用临时端口，客户端用临时端口。控制连接始终是客户端连服务器的 21 端口。

FTP 是有状态的：服务器记得当前登录用户、当前目录、传输类型。TYPE A 是文本模式，会做行尾转换；TYPE I 是二进制模式，不做转换。FTP 用明文传用户名、密码和文件，不安全；FTPS 是 FTP over TLS，给控制连接和数据连接加密；SFTP 是 SSH 的文件传输子系统，完全是另一套协议，默认走 22 端口，不是 FTP 的扩展。

11.4 电子邮件：SMTP 发，POP3/IMAP 收

Alice 给 Bob 发一封邮件。Alice 的邮件客户端（用户代理，UA）把邮件提交给她的邮件服务器；她的服务器再转发给 Bob 的邮件服务器；Bob 用客户端把邮件从自己的服务器拉下来。图里四台实体和三条链路，前两条都是“推”，最后一条是“拉”。



发送方服务器把邮件排队，按 MX 记录找到接收方服务器后重试投递；
接收方服务器把邮件放进 Bob 的邮箱，等 Bob 主动来取。

图中 SMTP 负责前两段，都是把邮件推给下一台服务器；POP3/IMAP 负责最后一段，由收件人主动拉取。SMTP 不能用来读取邮箱里的邮件。

发送方服务器在转发前会查 DNS 的 MX 记录，找到接收方域的邮件服务器；如果该域没有 MX 记录，才回退到 A 记录。邮件服务器之间可能经过多个中继，但每一跳仍然用 SMTP。

SMTP 的命令与应答

SMTP 是命令/应答协议。一个会话大致是：服务器先回 220；客户端用 EHLO（或老式的 HELO）打招呼；用 MAIL FROM 给出信封发件人；用 RCPT TO 给出信封收件人，可以有多个；用 DATA 进入邮件内容，内容以单独一行句点结束；最后 QUIT。下面是一个简化会话，省略了扩展协商的多行应答。

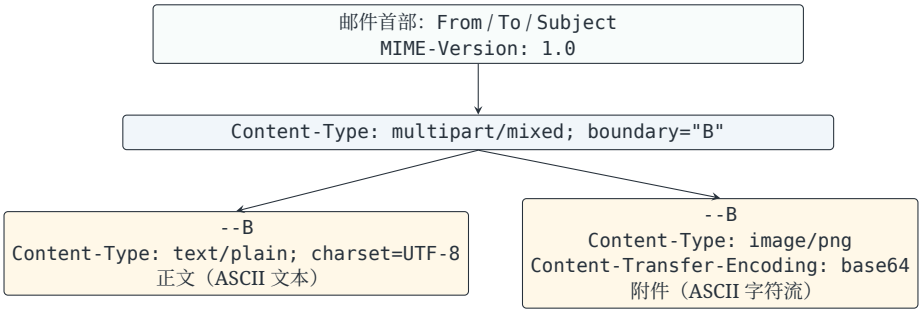
```
S: 220 mail.example.com ESMTP
C: EHLO client.example.com
S: 250 OK
C: MAIL FROM:<alice@example.com>
S: 250 OK
C: RCPT TO:<bob@example.net>
S: 250 OK
C: DATA
S: 354 End data with CRLF.CRLF
C: From: Alice <alice@example.com>
C: To: Bob <bob@example.net>
C: Subject: Hello
C:
C: This is the message body.
C: .
S: 250 OK
C: QUIT
S: 221 Bye
```

信封和首部是两回事。`MAIL FROM` 和 `RCPT TO` 是投递用的信封，决定邮件送到哪里、退信退给谁；`From:` 和 `To:` 是显示在邮件里的首部，可以和信封不同。密送（Bcc）只出现在 `RCPT TO` 里，不会写进首部。SMTP 允许在同一条 TCP 连接上连续发多封邮件，不必每封都重新建连接；扩展 SMTP 还支持流水线，把多个命令一次发出。

传统 SMTP 只传 7 位 ASCII，行尾是 CRLF，行长也有限制。二进制附件和中文不能直接塞进去，需要 MIME 扩展。MIME 不改 SMTP 的命令，只在邮件首部增加字段，把内容重新编码成 ASCII。

MIME：让邮件装下二进制和中文

MIME 主要增加三类首部。`MIME-Version: 1.0` 声明使用 MIME。`Content-Type` 说明内容的类型和参数，例如 `text/plain`、`image/png`、`multipart/mixed`。`Content-Transfer-Encoding` 说明内容如何编码成 ASCII，常见的是 `base64` 和 `quoted-printable`。图里是一封“正文 + 图片附件”的邮件结构。



接收方按 boundary 切分 part，再按每个 part 的 Content-Type 还原正文和附件。

图中整封邮件仍然是一个 7 位 ASCII 文本：boundary 是分隔线，base64 把二进制图片变成可打印字符。MIME 只扩展邮件内容的表示，不改变 SMTP 的传输过程。

Base64 把每 3 个字节编成 4 个 ASCII 字符，体积大约增加三分之一；quoted-printable 适合大部分是 ASCII、少量非 ASCII 的文本，可读性更好。`multipart/alternative` 表示“同一内容的不同格式”（比如纯文本和 HTML），接收方选一个显示；`multipart/mixed` 表示“正文加附件”。扩展 SMTP 的 8BITMIME 和 SMTPUTF8 可以放宽 7 位限制，但考试默认模型仍是 7 位 ASCII 加 MIME 编码。

POP3 与 IMAP

POP3 把邮件从服务器拉到本地，分三个阶段：特许阶段用 USER/PASS 登录；事务阶段用 STAT、LIST、RETR 取邮件，用 DELE 做删除标记；更新阶段在 QUIT 之后真正删除被标记的邮件。图里画出了三个阶段和关键命令。



DELE 只在本次会话里做删除标记，RSET 可以取消；
只有进入更新阶段，服务器才真正删掉这些邮件。

图中三个阶段是顺序进行的：没有登录不能取信，没有 QUIT 不会真正删除。POP3 默认下载后删除，也可以配置保留副本，但它没有服务器端文件夹和多设备状态同步。

IMAP 则以服务器端邮箱为准：支持文件夹、邮件标志（已读、已删、已标记）、只取首部或部分内容、服务器端搜索、多客户端同步。POP3 适合单设备收信，IMAP 适合手机、平板、电脑同时看同一个邮箱。

POP3 与 IMAP 的分工

	POP3	IMAP
默认端口	110（明文），995（TLS）	143（明文），993（TLS）
存储模型	下载到本地为主	邮件留在服务器，客户端缓存
文件夹	协议本身不提供服务器端多文件夹	支持服务器端文件夹和层次结构
多设备	状态难同步	标志和文件夹在服务器端同步
典型命令	USER、PASS、LIST、RETR、DELE	LOGIN、SELECT、FETCH、STORE、SEARCH
适用场景	单设备、离线阅读	多设备、在线协作

网页邮箱是另一回事：浏览器和 Web 服务器之间用 HTTP，Web 服务器背后再用 SMTP、IMAP 和邮件系统交互。所以“网页邮箱用 HTTP 打开”不等于 SMTP 被 HTTP 取代，邮件服务器之间仍然说 SMTP。

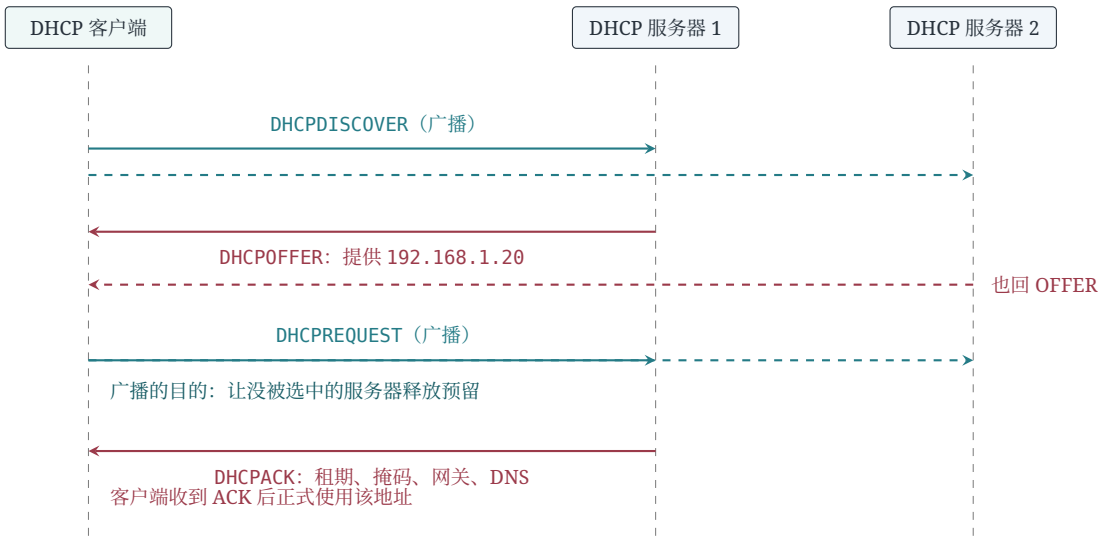
11.5 DHCP：主机怎样拿到地址

一台新笔记本连上 Wi-Fi，没有 IP 地址，不能上网。它需要四样东西：自己的 IP 地址、子网掩码、默认网关、DNS 服务器地址。DHCP 自动把这些参数发给它，并且只租一段时间，这就是“动态主机配置”。

DHCP 用 UDP：客户端 68 端口，服务器 67 端口。客户端开口问的时候还没有 IP，所以源地址填 `0.0.0.0`，目的地址填受限广播地址 `255.255.255.255`。DHCP 是应用层协议，但它配置的是网络层参数；配置完成后，主机才能正常发 IP 数据报。

四步交互：D-O-R-A

DHCP 的典型四步是 DISCOVER、OFFER、REQUEST、ACK，简称 D-O-R-A。图里画了一个客户端和两台服务器：客户端广播发现，两台服务器都可能给 offer，客户端选一个并广播请求，被选中的服务器回 ACK。



图中 DISCOVER 和 REQUEST 用广播，因为客户端此时还没有可用 IP；DISCOVER 的源地址是 `0.0.0.0:68`，目的地址是 `255.255.255.255:67`。OFFER 和 ACK 可以单播也可以广播。第二个服务器的 OFFER 不是多余的：REQUEST 广播就是为了告诉它“我没选你”，让它释放预留。

各报文的含义如下。DHCP 服务器不一定对每个 DISCOVER 都回应，所以客户端可能收到零个、一个或多个 OFFER。

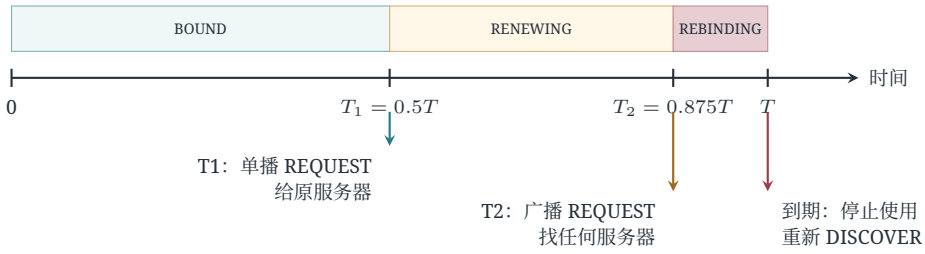
DHCP 报文

报文	作用
DHCPDISCOVER	客户端广播寻找可用的 DHCP 服务器
DHCPOFFER	服务器提供一个可用地址、租期和配置参数
DHCPREQUEST	客户端选择一个 OFFER，广播确认并通知其他服务器
DHCPACK	被选中的服务器确认分配，客户端开始使用
DHCPNAK	服务器拒绝请求，客户端需要重新 DISCOVER
DHCPDECLINE	客户端发现地址冲突，拒绝使用该地址
DHCPRELEASE	客户端提前释放地址
DHCPINFORM	客户端已有地址，只请求其他配置参数

如果 DHCP 服务器和客户端不在同一个广播域，广播到不了服务器，需要 DHCP 中继代理（relay agent）。中继通常由路由器担任：它把 DISCOVER 转发给服务器，并在 `giaddr` 字段里填上客户端所在子网的地址；服务器根据 `giaddr` 选择地址池，把 OFFER/ACK 发回中继，由中继转给客户端。没有中继，跨网段的客户端就拿不到地址。

租约、续租与地址冲突

DHCP 分配的地址有租期 T，不是永久的。图里画出了租约的时间轴：$T_1 = 0.5T$ 时客户端进入续租状态（RENEWING），单播 DHCPREQUEST 给原服务器；续租成功，租期重新计时。如果到 $T_2 = 0.875T$ 还没成功，客户端进入重绑定状态（REBINDING），广播 DHCPREQUEST 找任何一台服务器；成功就继续用，失败就等到 T 到期。如果服务器回 DHCPNAK，客户端必须立即停止使用该地址，重新走 D-O-R-A。



图中越往右时间越晚：先在原服务器处续租，失败后再广播找别的服务器，最后到期才放弃地址。租期过半和八分之七这两个时刻是 DHCP 的默认值。

DHCP 不保证地址一定唯一。客户端拿到地址后可以用免费 ARP（gratuitous ARP）探测一下，发现冲突就发 DHCPDECLINE 拒绝这个地址，重新申请。恶意 DHCP 服务器可以给客户端发错误的网关或 DNS，这叫 DHCP 欺骗；防范手段包括 DHCP snooping、端口安全和静态绑定。

DHCPv6 的思路相同，但用 UDP 546（客户端）和 547（服务器），报文名是 SOLICIT、ADVERTISE、REQUEST、REPLY，不走广播而是用组播地址 `ff02::1:2`。408 主要考 DHCPv4，知道这个区别即可。

11.6 本章小结：把协议放到一条时间线上

把本章的协议放在一起看，各自的角色很清楚：DNS 把名字解析成地址，DHCP 让主机拿到地址和网关，HTTP 取网页，FTP 传文件，SMTP 发邮件，POP3/IMAP 收邮件。它们使用的传输层和端口如下。

应用层协议速查

协议	传输层 / 端口	谁发起	关键点
DNS	UDP 53，TCP 53 备用	解析器	层次命名、递归/迭代、TTL 缓存
HTTP	TCP 80 / 443	客户端	无状态、持久连接、Cookie、缓存
FTP	TCP 21 控制；20 或临时端口数据	客户端控制；数据看模式	双连接、带外控制、主动/被动
SMTP	TCP 25 / 587	发送方	推邮件、命令应答、7 位 ASCII + MIME
POP3	TCP 110 / 995	接收方	拉邮件、三阶段、下载后删除
IMAP	TCP 143 / 993	接收方	服务器端邮箱、多设备同步
DHCP	UDP 67 / 68	客户端广播	D-O-R-A、租约、中继

一台主机刚接入网络时，顺序大致是：先用 DHCP 拿到 IP、掩码、网关和 DNS；访问 `www.example.com` 时，先用 DNS 解析出 IP；如果目的地址不在本网段，主机用 ARP 找到默认网关的 MAC 地址（ARP 属于网络层/链路层，不是应用层）；然后 TCP 三次握手，最后发 HTTP 请求。DNS 和 DHCP 用 UDP，HTTP 用 TCP；FTP、SMTP、POP3/IMAP 各自在自己的连接上工作，互不替代。

延伸阅读：Kurose & Ross《计算机网络：自顶向下方法》第 2 章有 DNS、HTTP、FTP、SMTP/POP3 的抓包示例；谢希仁《计算机网络》第 6 章使用国内常见术语，可对照阅读。

12 习题

12.1 基础过关题

1. （单项选择题）主机 H 通过本地域名服务器 L 解析域名 `www.example.com`，L 的缓存为空。下列叙述中正确的是

(A) H 对 L 采用递归查询时，L 必须返回最终结果或失败，不能只给出下一台服务器地址

(B) H 对 L 采用迭代查询时，L 必须代 H 完成后续全部查询

(C) 迭代查询中，根服务器会代替 L 继续向 `.com` 服务器查询

(D) 递归查询中，根服务器若没有该记录，可以只返回 `.com` 服务器地址

2. （单项选择题）下列关于 DNS 层次结构和区的说法中，正确的是

(A) 一个域只能划分为一个区

(B) 本地域名服务器属于 DNS 层次结构的一级

(C) 区可以小于或等于它所属的域

(D) 根域名服务器保存所有二级域名的 IP 地址

3. （单项选择题）关于 FTP 的两条 TCP 连接，下列说法中正确的是

(A) 控制连接由服务器主动发起

(B) 主动模式下，服务器从端口 20 向客户端告知的端口发起数据连接

(C) 被动模式下，服务器从端口 21 向客户端发起数据连接

(D) 数据连接在整个会话期间保持，控制连接每传完一个文件就关闭

4. （单项选择题）用户 A 把一封邮件发送给用户 B：(1) A 的用户代理把邮件提交给 A 的邮件服务器；(2) A 的邮件服务器把邮件转发给 B 的邮件服务器；(3) B 用 IMAP 客户端从自己的邮件服务器读取邮件。第 (1)(2)(3) 步所用协议依次是

(A) SMTP、SMTP、IMAP　(B) SMTP、IMAP、SMTP

(C) IMAP、SMTP、SMTP　(D) SMTP、SMTP、SMTP

5. (单项选择题) 关于 DHCP 的工作过程，下列说法中正确的是

(A) 客户端以 0.0.0.0 为源、255.255.255.255 为目的广播 DHCPDISCOVER

(B) DHCPREQUEST 总是单播发送，以便其他服务器保留已预留的地址

(C) 所有收到 DHCPDISCOVER 的服务器都必须发送 DHCPOFFER

(D) 分配到的地址永久有效，续租时必须重新执行完整四步

6. (填空题) 在 HTTP/1.1 中，(1) 请求首部 _____ 用于指定目标主机，使一台服务器可以区分多个虚拟主机；(2) 客户端缓存副本仍然有效时，服务器返回状态码 _____；(3) HTTP/1.1 默认使用 _____ 连接。

12.2 真题风格与综合训练

1. (单项选择题) 某网页包含 1 个基本 HTML 文件和 5 个图像，全部位于同一服务器。忽略 DNS 时间和对象传输时间，且必须先收到基本 HTML 才能知道 5 个图像的 URL。非持久串行、持久非流水线、持久流水线三种方式所需的 RTT 数依次为

(A) 12、7、3　(B) 12、8、3

(C) 10、7、3　(D) 12、7、2

2. (单项选择题) 关于 FTP 与电子邮件协议，下列说法中正确的是

(A) FTP 的控制连接和数据连接都贯穿整个会话

(B) FTP 数据连接可传二进制文件；SMTP 传二进制附件需 MIME 编码

(C) POP3 提供服务器端多文件夹和多设备同步，IMAP 不提供

(D) SMTP 的 MAIL FROM 与邮件首部 From 都表示邮件作者

3. **（真题风格，综合题）**某网页由 1 个基本 HTML 文件（20 KB）和 4 幅 JPEG 图像（每幅 100 KB）组成，全部放在同一台服务器上。设往返时间 RTT = 100 ms，链路带宽为 10 Mb/s，为便于计算取 1 KB = 1000 B；忽略 DNS 解析时间、报文首部开销和连接释放时间，对象的传输时间按“对象大小/带宽”在 RTT 之外另行计算。必须先收到基本 HTML 才能知道 4 幅图像的 URL；建立 TCP 连接用 1 RTT，连接建立后一次请求—响应用 1 RTT。分别计算：(1) 非持久连接、串行方式的总时间；(2) 持久连接、非流水线方式的总时间；(3) 持久连接、流水线方式的总时间；(4) 若客户端在建立 TCP 连接前就已获得 4 幅图像的 URL，可在连接建立后立即把 5 个请求一起流水线发出，总时间为多少？与 (3) 相差多少？四种方式中哪种最短？

4. **（真题风格，综合题）**某主机新接入一个局域网，尚未配置 IP 地址。该局域网内有 DHCP 服务器和本地域名服务器，出口路由器是该局域网的默认网关；本地域名服务器无缓存，对上级服务器采用迭代查询。主机随后访问 `http://www.example.com/index.html`，该 Web 服务器位于另一个网络。回答下列问题：(1) 写出 DHCP 获取地址的四个报文名称及顺序；指出其中由客户端广播发送的报文；写出 DHCPDISCOVER 的源 IP 地址、目的 IP 地址，以及服务器和客户端使用的 UDP 端口。(2) 写出本地域名服务器完成解析的迭代查询过程（向谁查询、得到什么），它共发送几条查询报文？主机向本地域名服务器发送几条查询报文？若在 TTL 内另一台主机查询同一域名，本地域名服务器如何处理？(3) 主机向 Web 服务器发送 IP 数据报前，需要用 ARP 获得哪个设备的 MAC 地址？为什么不是 Web 服务器的 MAC 地址？该 ARP 请求的作用范围如何？(4) 从主机发出 TCP SYN 到收到 HTTP 响应，最少需要多少 RTT（不计 DNS 时间和传输时间）？HTTP/1.1 默认采用哪种连接方式？

PART V　第五部分

408 模拟试卷

1 模拟卷一

考试说明：考试时间 180 分钟，满分 150 分。

一、单项选择题：1–40 小题，每小题 2 分，共 80 分

数据结构（1–11 题）

1. 若线性表最常用的操作是存取第 i 个元素及其前驱和后继的值，则采用（ ）存储方式最节省时间。

(A) 单链表　(B) 双链表

(C) 循环单链表　(D) 顺序表

2. 栈和队列的共同点是

(A) 都是先进先出　(B) 都是先进后出

(C) 只允许在端点处进行插入和删除　(D) 没有共同点

3. 设模式串 T =”abcabca”，采用本书的 next 定义：下标从 1 开始，$\texttt{next}[1]=0$，失配时令 $j=\texttt{next}[j]$。则 next[4] 和 next[7] 的值分别为

(A) 1 和 4　(B) 1 和 2

(C) 2 和 4　(D) 1 和 3

4. 一棵完全二叉树有 1001 个结点，其中叶子结点的个数是

(A) 500　(B) 251

(C) 250　(D) 501

5. 具有 n 个顶点的有向完全图的边数是

(A) $n(n-1)/2$ (B) $n(n-1)$

(C) n^2 (D) $n(n+1)$

6. 在一棵 5 阶 B-树中，除根结点外的每个非终端结点至少包含的关键字个数为

(A) 1 (B) 4

(C) 3 (D) 2

7. 下列排序算法中，不稳定的是

(A) 直接插入排序 (B) 冒泡排序

(C) 快速排序 (D) 归并排序

8. 用不带头结点的单链表存储队列，队头在链表头，队尾在链表尾。在进行出队操作时

(A) 仅修改头指针 (B) 仅修改尾指针

(C) 头尾指针都可能修改 (D) 头尾指针都不修改

9. 一棵二叉排序树中，关键字最小的结点

(A) 一定是叶结点　(B) 一定无左孩子

(C) 一定无右孩子　(D) 可能是任何结点

10. 采用“牺牲一个存储单元”判别队满/队空的循环队列约定：`front` 指向队头元素，`rear` 指向队尾元素的下一空位，数组下标按模 6 循环。若用一个大小为 6 的数组实现该循环队列，当前 `rear=0`、`front=3`。从队列中删除一个元素，再加入两个元素后，`rear` 和 `front` 的值分别为

(A) 1 和 4　(B) 2 和 4

(C) 2 和 3　(D) 1 和 3

11. 对权值集合 $\{5, 7, 10, 15, 20, 45\}$ 构造 Huffman 树，其最小带权路径长度 WPL 为

(A) 102　(B) 183

(C) 228　(D) 330

计算机组成原理（12–22 题）

12. 计算机中，运算器的主要功能是进行

(A) 算术运算　(B) 逻辑运算

(C) 算术和逻辑运算　(D) 初等函数运算

13. 8 位补码表示的整数范围是

(A) $-128 \sim 127$ (B) $-127 \sim 127$

(C) $-128 \sim 128$ (D) $-127 \sim 128$

14. IEEE 754 单精度浮点数中，阶码的偏置值（bias）为

(A) 128 (B) 255

(C) 126 (D) 127

15. 某 32 位按字节编址的计算机采用 8 KiB 直接映射 Cache，块大小为 64 B。主存地址按 Tag、行号、块内偏移划分，三者位数依次为

(A) 19、7、6 (B) 18、8、6

(C) 20、6、6 (D) 19、6、7

16. 指令系统中采用不同寻址方式的主要目的是

(A) 缩短指令字长，扩大寻址空间 (B) 增加指令的条数

(C) 简化指令译码 (D) 实现程序控制

17. 在经典五级流水线中，Load-use 数据冒险

(A) 可以通过转发完全解决 (B) 属于控制冒险

(C) 不会发生 (D) 必须阻塞至少 1 个时钟周期

18. 微程序控制器中，存放微程序的是

(A) 主存储器　(B) Cache

(C) 控制存储器 (CM)　(D) 通用寄存器

19. 下列总线仲裁方式中，响应速度最快但控制线最多的是

(A) 链式查询　(B) 计数器定时查询

(C) 独立请求方式　(D) 分布式仲裁

20. 在 DMA 方式下，数据传送是在（　）之间直接进行的

(A) CPU 与外设　(B) 外设与外设

(C) CPU 与主存　(D) 主存与外设

21. 某计算机主频为 1 GHz，CPI 为 2。其 MIPS（每秒百万条指令）约为

(A) 2000　(B) 1000

(C) 500　(D) 250

22. 某 DRAM 芯片采用地址复用技术，行地址和列地址分时传送，且行、列地址位数相同。若存储容量为 16M × 8 位，则芯片的地址引脚数为

(A) 12 (B) 14

(C) 24 (D) 28

操作系统（23–32 题）

23. 用户态程序请求操作系统内核服务的规范入口是

(A) 系统调用 (B) 库函数

(C) 中断 (D) 命令行

24. 在单 CPU 系统中，并发进程之间

(A) 可以并行执行 (B) 以上都不对

(C) 微观上同时执行 (D) 宏观上同时、微观上交替执行

25. 信号量 S 的当前值为 −2，则表示

(A) 有 2 个可用资源 (B) 有 2 个进程在等待

(C) 有 2 个进程在临界区 (D) 系统即将死锁

26. 某进程具有 32 位虚拟地址空间，页面大小为 4 KiB，采用单级页表，每个页表项占 4 B。若为整个虚拟地址空间配置完整页表，则页表大小为

(A) 1 MiB (B) 2 MiB

(C) 4 MiB (D) 8 MiB

27. 在请求分页系统中，若采用 FIFO 页面置换算法，可能发生

(A) LRU 异常 (B) Belady 异常

(C) OPT 异常 (D) 时钟异常

28. SPOOLing 系统的输入井和输出井位于

(A) 内存 (B) 输入输出设备

(C) 寄存器 (D) 磁盘

29. 在下面的 I/O 控制方式中，需要 CPU 干预最少的是

(A) 程序查询 (B) 中断方式

(C) DMA 方式 (D) 通道方式

30. 设磁盘的磁头当前在 50 号磁道，正向磁道号增大的方向移动。请求序列为：30, 180, 70, 100, 40。用 SSTF 算法，磁头移动的总磁道数为

(A) 170 (B) 210

(C) 240 (D) 290

31. 在文件系统中，文件目录的主要作用是

(A) 实现文件的逻辑结构　(B) 实现文件的物理结构

(C) 实现按名存取　(D) 管理外存空闲空间

32. 死锁的四个必要条件中，系统无法通过设计破坏的是

(A) 互斥条件　(B) 持有并等待

(C) 不可剥夺条件　(D) 循环等待条件

计算机网络（33–40 题）

33. TCP/IP 参考模型中，提供端到端可靠传输的是

(A) 网络层　(B) 传输层

(C) 应用层　(D) 数据链路层

34. 若信道带宽为 4 kHz，信噪比为 30 dB，根据香农定理，最大数据率约为

(A) 8 kbps　(B) 40 kbps

(C) 80 kbps　(D) 120 kbps

35. 以太网采用 CSMA/CD 协议时，争用期为

(A) 传播时延 τ　(B) 处理时延

(C) 发送时延　(D) 2τ

36. 某 IPv4 数据报总长度 1500 B（首部 20 B），DF=0，要通过 MTU=620 B 的链路，则分片数为

(A) 2　(B) 3

(C) 4　(D) 5

37. TCP 连接释放过程中，主动关闭方发送 FIN 后进入的状态是

(A) TIME_WAIT　(B) FIN_WAIT_1

(C) CLOSE_WAIT　(D) LAST_ACK

38. DNS 解析中，本地域名服务器代替查询主机完成全部域名解析工作的方式称为

(A) 递归查询　(B) 迭代查询

(C) 反向查询　(D) 缓存查询

39. 某网页含 1 个基本 HTML 文件和 4 个同服务器图像。忽略 DNS 与对象传输时间，必须先收到 HTML 才知道图像 URL；HTTP/1.1 使用一个持久连接，并对图像请求采用流水线。完成全部获取至少需要

(A) 2 RTT (B) 3 RTT

(C) 6 RTT (D) 10 RTT

40. 在数据链路层，交换机通过（ ）学习 MAC 地址

(A) 目的 MAC 地址 (B) 源 MAC 地址

(C) ARP 协议获得的地址 (D) 管理员配置的地址

二、综合应用题：41–47 小题，共 70 分

41.（13 分，数据结构）已知两个带头结点的单链表 A 和 B 分别表示两个集合，其元素递增排列。编写算法求 A 和 B 的交集，结果存放在 A 链表中（利用原结点），并分析算法的时间复杂度。

42.（10 分，数据结构）一棵二叉树采用二叉链表存储。编写算法判断该二叉树是否为完全二叉树，要求辅助队列不能依赖固定的任意上限。

43.（12 分，组成原理）将十进制数 -58.75 表示为 IEEE 754 单精度浮点数格式（十六进制），并写出其二进制编码的详细推导过程。

44.（11 分，组成原理）某 16 位计算机指令格式如下：操作码 4 位，两个操作数各 6 位（均为寄存器-寄存器型，每个操作数 3 位寄存器号 +3 位寻址方式）。问：(1) 该指令系统最多有多少条指令？(2) 若改用扩展操作码，保留 12 条两操作数指令，一操作数指令最多有多少条？

45.（8 分，操作系统）面包师问题：面包店有一个最多放 N 个面包的柜台。面包师不断烤面包放入柜台，顾客不断从柜台取面包。用信号量写出同步伪代码并说明各信号量的含义。

46.（7 分，操作系统）某请求分页系统的页面访问序列（页号）为

$$1,2,3,4,2,1,5,6,2,1,$$
$$2,3,7,6,3,2,1,2,3,6$$

。假设进程分得 4 个物理块，初始为空，采用 LRU 页面置换算法。
(1) 写出每次访问后的页框内容；(2) 列出缺页发生的访问及被淘汰页；(3) 计算缺页次数和缺页率。

47.（**9 分，计算机网络**）主机 A 的 IP 地址为 192.168.1.37/28，主机 B 的 IP 地址为 192.168.1.49/28。(1) 主机 A 和 B 是否在同一子网？(2) 若子网掩码改为 255.255.255.224(/27)，A 和 B 是否在同一子网？(3) 写出两种情况下的子网网络地址。

2 模拟卷二

考试说明：考试时间 180 分钟，满分 150 分。

一、单项选择题：1–40 小题，每小题 2 分，共 80 分

数据结构（1–11 题）

1. 对 n 个元素的顺序表进行顺序查找，等概率下查找成功的平均比较次数为

(A) n　　(B) $n/2$

(C) $(n+1)/2$　　(D) $(n-1)/2$

2. 已知一个栈的入栈序列为 $1,2,3,\ldots,n$，出栈序列为 $p_1,p_2,\ldots,p_n$。若 $p_1=n$，则 p_i $(1<i\leqslant n)$

(A) 一定是 $n-i+1$　　(B) 可能是任意值

(C) 一定是 i　　(D) 无法确定

3. 设模式串 T =”ababaab”，采用本书的 next 定义：下标从 1 开始，$\texttt{next}[1]=0$，失配时令 $j=\texttt{next}[j]$。该模式串的 next 数组为

(A) 0,1,1,2,3,1,2　　(B) 0,1,1,2,3,2,3

(C) 0,1,1,2,2,1,2　　(D) 0,1,1,2,3,4,2

4. 设高度为 h（按层数计，根为第 1 层）的二叉树只有度为 0 和 2 的结点，则此类二叉树中所包含的结点数至少为

(A) $2h-1$　　(B) $2h$

(C) $2h+1$　　(D) $h+1$

5. 下列关于 AOE 网的叙述中，错误的是

(A) 关键活动不按期完成会影响整个工程工期

(B) 任何一个关键活动提前完成，整个工程也会提前完成

(C) 一个 AOE 网的关键路径可以有多条

(D) 缩短某条关键路径上的非公共关键活动不一定缩短总工期

6. 在一棵 3 阶 B-树中插入关键字，下列情况中一定会导致结点分裂的是

(A) 插入到叶结点　(B) 插入到根结点

(C) 插入后结点有 2 个关键字　(D) 插入后结点有 3 个关键字

7. 对一组数据 (2, 12, 16, 88, 5, 10) 进行排序，若前三趟排序结果如下：第一趟: 2,12,16,5,10,88；第二趟: 2,12,5,10,16,88；第三趟: 2,5,10,12,16,88。则采用的排序算法可能是

(A) 冒泡排序　(B) 选择排序

(C) 归并排序　(D) 基数排序

8. 假设通信电文由字符集 {a,b,c,d,e} 组成，各字符在电文中出现的频率分别为 7,5,2,4,9。构造 Huffman 树，字符 c 的 Huffman 编码长度为

(A) 2　(B) 1

(C) 4　(D) 3

9. 设散列表长 $m = 13$，散列函数 $H(k) = k \bmod 13$，用线性探测法处理冲突。已插入关键字 16,29,42。若再插入关键字 55，其地址为

(A) 3 (B) 4

(C) 5 (D) 6

10. 一个连通无向图有 8 个顶点，所有顶点度数之和为 24。要保留其一棵生成树，需要从原图中删除的边数为

(A) 3 (B) 4

(C) 5 (D) 6

11. 下列算法中，时间复杂度为 $O(n\log n)$ 且空间复杂度为 $O(1)$ 的是

(A) 归并排序 (B) 快速排序

(C) 堆排序 (D) 基数排序

计算机组成原理（12–22 题）

12. 冯 · 诺依曼计算机中指令和数据均以二进制形式存放在存储器中，CPU 区分它们的依据是

(A) 指令操作码的译码结果 (B) 指令和数据的寻址方式

(C) 指令周期的不同阶段 (D) 指令和数据所在的存储单元

13. 将十进制数 $+286$ 表示为 16 位补码（十六进制）是

(A) 011EH (B) 811EH

(C) 011DH (D) FE2H

14. IEEE 754 单精度浮点数 `C0A00000H` 对应的十进制真值是

(A) -5.0 (B) -4.0

(C) -6.0 (D) -8.0

15. 某计算机的 Cache 共有 16 块，采用 2 路组相联映射方式。每个主存块大小为 32 B，按字节编址。主存 129 号单元所在的主存块应装入的 Cache 组号是

(A) 0 (B) 1

(C) 2 (D) 4

16. 某指令系统采用扩展操作码，指令字长 16 位。二地址指令的两个地址码各占 6 位，一地址指令的地址码占 6 位。若二地址指令有 15 条、一地址指令有 62 条，则零地址指令最多有

(A) 64 条 (B) 128 条

(C) 256 条 (D) 512 条

17. 在采用常规 EX → EX 转发、且不考虑编译器重排的五级流水线中，若相邻两条 ALU 指令之间存在 RAW 数据冒险，以下正确的处理方式是

(A) 插入 1 个气泡　(B) 通过转发解决，无需阻塞

(C) 插入 2 个气泡　(D) 编译器调度重新排列指令

18. 某微程序控制器采用字段编码法。4 个互斥控制字段分别需要表示 3、5、4、2 个微命令，并且每个字段还需保留一种“不发出微命令”的编码。控制字段至少需要多少位?

(A) 8　(B) 9

(C) 10　(D) 14

19. 某总线数据宽度为 64 位，总线时钟频率为 200 MHz，在时钟上、下沿各传输一次数据。忽略协议开销，其理论最大数据传输率为

(A) 1.6 GB/s　(B) 3.2 GB/s

(C) 6.4 GB/s　(D) 12.8 GB/s

20. DMA 方式中，DMA 控制器与 CPU 争夺总线使用权的方式称为

(A) 周期挪用（窃取）　(B) 中断请求

(C) 无条件传送　(D) 程序查询

21. 某 CPU 主频为 2 GHz，平均 CPI 为 1.5。执行一个包含 3×10^9 条指令的程序需要的时间约为

(A) 1.0 秒 (B) 1.5 秒

(C) 2.25 秒 (D) 3.0 秒

22. 若存储芯片容量为 256K × 8 位，用若干片构成 4MB 的存储器（按字节编址），则所需的芯片数为

(A) 8 (B) 16

(C) 32 (D) 64

操作系统（23–32 题）

23. 在 Unix/Linux 的进程控制语义中，下列说法正确的是

(A) `exec` 成功后一定创建一个新的进程

(B) `fork` 创建新进程；`exec` 用新程序替换当前进程的地址空间，但不因此创建新进程

(C) 每次处理中断都会创建一个新进程

(D) 创建进程只需分配程序代码，无需建立 PCB 等管理信息

24. 时间片轮转调度算法中，如果时间片取得太大，该算法将退化为

(A) SJF 调度算法 (B) 多级反馈队列

(C) 优先级调度算法 (D) FCFS 调度算法

25. 某系统有 3 个并发进程都需要 4 个同类资源，系统至少应提供多少个资源才能保证不发生死锁

(A) 9 (B) 10

(C) 11 (D) 12

26. 在页式存储管理中，地址变换由（ ）完成

(A) 操作系统 (B) MMU（硬件）

(C) 编译程序 (D) 装入程序

27. 在请求分页系统中，哪种页面置换算法理论上最优但无法实现

(A) FIFO (B) LRU

(C) OPT (D) CLOCK

28. 某分页系统的 TLB 查询时间为 10 ns，主存访问时间为 100 ns，TLB 命中率为 90%。假定页表在主存中，TLB 与主存串行访问，忽略缺页和 Cache；TLB 未命中时需先访问页表再访问数据。有效访问时间为

(A) 110 ns (B) 210 ns

(C) 200 ns (D) 120 ns

29. SPOOLing 技术实现了设备的（ ）共享

(A) 独占设备的虚拟共享 (B) 共享设备的独占使用

(C) 设备的物理共享 (D) 设备的并行使用

30. 磁盘的访问时间不包括

(A) 寻道时间 (B) 旋转延迟

(C) 传输时间 (D) 处理时间

31. I/O 软件层次结构中，层次最低、负责首先响应设备中断的是

(A) 用户层 I/O 软件 (B) 设备独立性软件

(C) 设备驱动程序 (D) 中断处理程序

32. 预防死锁的方法中，破坏” 持有并等待” 条件的方法是

(A) 资源一次性分配 (B) 资源有序分配

(C) 允许剥夺资源 (D) SPOOLing 技术

计算机网络（33–40 题）

33. 某链路数据率为 1 Mb/s，数据帧长 1000 bit，单程传播时延为 10 ms，确认帧发送时延和处理时间忽略。GBN 发送窗口为 4，在无差错、发送方持续有数据时，信道利用率约为

(A) 4.76% (B) 9.52%

(C) 19.05% (D) 80.00%

34. 一个信道的带宽为 3 kHz，无噪声，采用 16 种不同相位的调制技术。最大数据传输速率约为

(A) 12 kbps (B) 96 kbps

(C) 48 kbps (D) 24 kbps

35. CRC 校验码的生成多项式为 $G(x) = x^3 + x + 1$（即 1011），数据为 10110，则发送的码字为

(A) 10110010 (B) 10110000

(C) 10110110 (D) 10110101

36. 设某路由器建立了如下路由表。现收到目的地址为 128.96.39.10 的 IP 数据报，下一跳应为

目的网络	下一跳
128.96.39.0/25	接口 0
128.96.39.128/25	接口 1
128.96.40.0/25	R2
0.0.0.0/0	R3

(A) 接口 0 (B) 接口 1

(C) R2 (D) R3

37. 在 TCP 拥塞控制中，当发送方收到 3 个重复的 ACK 时，会触发

(A) 慢开始 (B) 快重传

(C) 拥塞避免 (D) 连接释放

38. 以下协议中，用于从邮件服务器获取邮件的是

(A) SMTP (B) POP3

(C) SNMP (D) DHCP

39. DNS 协议使用的传输层协议是

(A) 仅 TCP (B) 仅 UDP

(C) TCP 和 UDP 均可 (D) ICMP

40. 以下不属于内部网关协议（IGP）的是

(A) RIP (B) OSPF

(C) BGP (D) 以上都是

二、综合应用题：41–47 小题，共 70 分

41.（13 分，数据结构）设一棵二叉树采用二叉链表存储。编写算法求二叉树中任意两个结点的最近公共祖先（LCA）。设 p、q 是树中两个不同且确实存在的结点，并分析时间复杂度。

42.（10 分，数据结构）已知关键字序列为 (30,15,25,40,10,20,35)，依次插入一棵初始为空的二叉排序树。(1) 画出最终的 BST；(2) 计算等概率下查找成功的 ASL；(3) 删除结点 25（用前驱替代），画出删除后的 BST。

43.（12 分，组成原理）某计算机的 Cache 采用 2 路组相联映射，共有 8 组。主存地址 32 位，按字节编址，Cache 块大小为 16 B。(1) 给出主存地址的划分（Tag、组索引、块内偏移各占多少位）；(2)Cache 的总容量是多少（含 Tag 和有效位）？(3) 若 CPU 依次访问地址 0,8,16,32,0,8,16,32，计算 Cache 命中率。

44.（11 分，组成原理）某 16 位、按字编址的计算机采用单总线 CPU。PC、MAR、MDR、IR、通用寄存器 R0–R3、暂存器 Y 和 Z 均连接到内部总线；ALU 的 A 端取自 Y，B 端取自总线，结果锁存到 Z。PC 有不占用内部总线的自增通路；主存通过 MAR 和 MDR 读写。假定一次主存读在一个时钟周期内完成，且每周期至多有一个部件向内部总线送数。写出指令 `ADD R1, (R2)`（R1 $\leftarrow$ R1 + Mem[R2]）的取指周期和执行周期微操作，并列出主要控制信号。

45.（8 分，操作系统）系统中有 5 个进程 P0-P4 和 3 类资源 (A,B,C)，资源数量为 (10,5,7)。当前时刻的 Allocation 和 Max 矩阵如下。问：(1)Need 矩阵；(2) 系统当前是否安全？若安全给出安全序列；(3) 若 P1 发出请求 Request(1,0,2)，能否分配?

	Allocation			Max		
	A	B	C	A	B	C
$P0$	0	1	0	7	5	3
$P1$	2	0	0	3	2	2
$P2$	3	0	2	9	0	2
$P3$	2	1	1	2	2	2
$P4$	0	0	2	4	3	3

46.（7 分，操作系统）某请求分页系统，页面大小 4 KB。进程的逻辑地址空间为 64 KB，分得 4 个物理块。页表内容：页 0→ 块 2, 页 1→ 块 5, 页 2→ 块 1, 页 3→ 块 8。求：(1) 逻辑地址 5000 的物理地址；(2) 若访问逻辑地址 20000 时发生缺页（页表无映射），OS 应如何处理?

47.（9 分，计算机网络）一个 TCP 连接以 1 KB 的 MSS 发送数据。初始 ssthresh=16 KB，cwnd=1 KB。按时间顺序：连续成功传输使 cwnd 变为 2,4,8,16,17,18,19，随后发生超时。采用 408 教材化简化模型：超时时将 ssthresh 取当前 cwnd 的一半（以整数 MSS 向下取整），并将 cwnd 置为 1 MSS。(1) 画出 cwnd 的变化折线图；(2) 发生超时后，ssthresh 和 cwnd 的新值是多少？(3) 之后 3 个 RTT 的传输均成功，第 3 次成功后 cwnd 是多少?

3 模拟卷三

考试说明：考试时间 180 分钟，满分 150 分。

一、单项选择题：1–40 小题，每小题 2 分，共 80 分

数据结构（1–11 题）

1. 若长度为 n 的线性表采用顺序存储结构，在第 i 个位置（$1 \leqslant i \leqslant n+1$）插入一个新元素，需要移动的元素个数为

(A) i　　(B) $n-i$

(C) $n-i+1$　　(D) $n-i-1$

2. 循环队列用数组 $A[0..m-1]$ 存放元素，front 指向队头元素，rear 指向队尾元素下一位置。则队列中元素个数为

(A) $(\text{rear}-\text{front})\%m$　　(B) $\text{rear}-\text{front}$

(C) $(\text{rear}-\text{front}+m)\%m$　　(D) $(\text{front}-\text{rear}+m)\%m$

3. 一棵有 124 个叶子结点的完全二叉树，最多有（　）个结点

(A) 248　　(B) 247

(C) 249　　(D) 250

4. 无向带权图的边为 A–$B(2)$、A–$C(5)$、B–$C(1)$、B–$D(4)$、C–$D(1)$。用 Dijkstra 算法从 A 出发，A 到 D 的最短路径长度为

(A) 3 (B) 6

(C) 5 (D) 4

5. 在散列查找中，装填因子 α 越大

(A) 发生冲突的概率越大 (B) 平均查找长度越小

(C) 查找效率越高 (D) 存储空间利用率越低

6. 若需在 $O(n\log n)$ 时间内完成排序且要求稳定，应选择

(A) 快速排序 (B) 堆排序

(C) 归并排序 (D) 希尔排序

7. 对关键字序列 (10,15,20,25,30) 进行折半查找，查找关键字 25 的过程中，依次与哪些关键字进行了比较

(A) 20,25 (B) 10,15,20,25

(C) 20,30,25 (D) 10,20,25

8. 一棵 AVL 树插入结点后导致不平衡，需要进行旋转调整。以下哪种旋转可以处理” 在左子树的右子树上插入” 导致的不平衡

(A) LL 旋转 (B) RR 旋转

(C) LR 旋转 (D) RL 旋转

9. 拓扑排序算法的时间复杂度（用邻接表存储）为

(A) $O(n)$ (B) $O(e\log e)$

(C) $O(n^2)$ (D) $O(n+e)$

10. 以下关于 B+ 树的叙述，正确的是

(A) 非叶结点保存全部数据记录

(B) B+ 树不支持顺序访问

(C) 叶结点之间没有顺序链接

(D) B+ 树的所有关键字都出现在叶结点中，非叶结点仅起索引作用

11. 对含 n 个关键字的序列，关键字比较次数始终为 $n(n-1)/2$ 的算法是

(A) 冒泡排序 (B) 快速排序

(C) 简单选择排序 (D) 插入排序

计算机组成原理（12–22 题）

12. 计算机系统的经典层次结构中，操作系统直接建立在下列哪一层之上？

(A) 机器语言层 (B) 汇编语言层

(C) 微程序层 (D) 高级语言层

13. $[-0]_{补}$ 的 8 位二进制表示为

(A) 00000000 (B) 10000000

(C) 11111111 (D) 01111111

14. 以下关于浮点数规格化的目的，错误的是

(A) 使浮点数的表示唯一 (B) 充分利用尾数的有效位数

(C) 提高运算结果的精度 (D) 简化浮点数的加法和减法

15. 某计算机的 Cache 容量为 4 KB，块大小为 64 B，主存容量为 256 MB，按字节编址。若采用直接映射，主存地址中 Tag 字段的位数为

(A) 8 (B) 22

(C) 18 (D) 16

16. 基址寻址方式中，操作数的有效地址为

(A) 基址寄存器内容加形式地址 (B) 变址寄存器内容加形式地址

(C) 程序计数器 PC 加形式地址 (D) 形式地址本身

17. 流水线中，结构冒险的产生原因是

(A) 硬件资源的冲突 (B) 相邻指令的数据依赖

(C) 转移指令的出现 (D) 流水线段数过多

18. 某 5 段流水线各段耗时均为 2 ns，连续执行 100 条无相关、无停顿的指令。从第一条指令进入流水线到最后一条完成共需

(A) 200 ns (B) 208 ns

(C) 1000 ns (D) 1040 ns

19. 中断响应过程中，由硬件自动完成的操作不包括

(A) 保存 PC（程序计数器） (B) 保存通用寄存器

(C) 加载中断向量 (D) 关闭中断

20. DMA 方式与中断方式的主要区别是

(A) DMA 方式不需要 CPU 干预 (B) DMA 方式可处理复杂 I/O

(C) DMA 方式仅在数据块传送的初始化和完成阶段需要 CPU 干预 (D) DMA 方式比中断方式慢

21. 某 8 位微机中，$x = +46$，$y = -38$。用补码计算 $x + y$，结果及溢出标志 OF 为

(A) 8, OF=1 (B) 8, OF=0

(C) -8, OF=1 (D) 84, OF=1

22. 采用低位交叉编址的多体并行存储器中，地址的划分方式是

(A) 高位表示体号，低位表示体内地址 (B) 低位表示体号，高位表示体内地址

(C) 奇数地址和偶数地址分开 (D) 按块划分

操作系统（23–32 题）

23. 用户程序通过（ ）从用户态切换到内核态

(A) 系统调用 (B) 函数调用

(C) 库函数 (D) 以上都可以

24. 在进程调度算法中，对短进程不利的是

(A) FCFS (B) SJF

(C) RR (D) 多级反馈队列

25. 对记录型信号量执行 P 操作后，若信号量值小于 0，则当前进程将

(A) 被唤醒 (B) 终止

(C) 进入就绪态 (D) 阻塞

26. 某页式存储系统按字节编址，页面大小为 1 KiB。逻辑地址 2050 所在页映射到物理页框 5，则该逻辑地址对应的物理地址为

(A) 2050　(B) 4098

(C) 5122　(D) 7170

27. 在虚拟存储器中，当程序访问的页面不在主存时，将触发

(A) 越界中断　(B) 缺页中断

(C) I/O 中断　(D) 时钟中断

28. 文件的逻辑结构中，通过索引表把关键字映射到记录地址、最便于按关键字随机存取的是

(A) 顺序文件　(B) 索引文件

(C) 流式文件　(D) 无结构字节流

29. 采用 SPOOLing 技术后，进程对打印机发出的打印请求

(A) 直接发送给打印机　(B) 写入磁盘上的输出井

(C) 排队等待直到打印机空闲　(D) 由中断处理程序处理

30. 在磁盘调度中，SSTF 算法的缺点是

(A) 平均寻道时间长　(B) 需要预知未来请求

(C) 实现复杂　(D) 可能导致饥饿

31. 在 I/O 控制方式中，使用专门的 I/O 处理器的是

(A) 程序查询　(B) 中断方式

(C) DMA 方式　(D) 通道方式

32. 在每类资源可能有多个实例的一般资源分配图中，有环是死锁的

(A) 充分必要条件　(B) 充分非必要条件

(C) 必要非充分条件　(D) 既非充分也非必要条件

计算机网络（33–40 题）

33. TCP/IP 参考模型中，与 OSI 参考模型网络层对应的是

(A) 网络接口层　(B) 网际层

(C) 传输层　(D) 应用层

34. 将数字信号调制为模拟信号时，同时改变载波的振幅和相位的调制技术称为

(A) ASK (B) FSK

(C) PSK (D) QAM

35. 100BaseT 以太网中，”100” 表示

(A) 最大传输距离 100 米 (B) 最多连接 100 个结点

(C) 数据传输率 100 Mbps (D) 编码效率为 100%

36. 若子网掩码为 255.255.255.224，则每个子网最多可容纳的主机数为

(A) 30 (B) 32

(C) 62 (D) 64

37. 主机甲向主机乙发送 TCP 报文段，序号 seq=200，数据长度 100 B。乙收到后发回的确认号 ack 是

(A) 200 (B) 299

(C) 300 (D) 301

38. DHCP 协议的作用是

(A) 域名解析 (B) 电子邮件发送

(C) 文件传输 (D) 动态 IP 地址分配

39. 主机 A 的地址为 192.168.1.62/27，主机 B 的地址为 192.168.1.65/27。A 向 B 发送 IP 数据报时，下列说法正确的是

(A) A、B 在同一子网，A 应 ARP 查询 B 的 MAC 地址

(B) A、B 不在同一子网，A 应 ARP 查询默认网关在本链路上的 MAC 地址

(C) A、B 不在同一子网，因此 A 无需解析任何 MAC 地址

(D) A、B 在同一子网，A 应把数据报直接交给 DNS 服务器

40. 在 CSMA/CD 协议中，发生冲突后采用的重传策略是

(A) 立即重传　　(B) 二进制指数退避

(C) 固定时间后重传　　(D) 优先级高的先重传

二、综合应用题：41–47 小题，共 70 分

41.（**13 分，数据结构**）已知一个带有头结点的单链表 L，设计一个尽可能高效的算法，将链表中的所有结点按数据域的值从小到大重新链接（就地排序），并分析时间复杂度。

42.（**10 分，数据结构**）已知关键字序列 (12,24,8,37,15,30,45,20)，画出依次插入生成的二叉排序树，并判断该树是否为平衡二叉树（AVL）。若不是，指出不平衡结点并说明如何调整。

43.（**12 分，组成原理**）某计算机的指令流水线分为 IF、ID、EX、MEM、WB 五个阶段，各段时间均为 2 ns。执行以下指令序列：

```
I1: lw  R1, 0(R2)
I2: lw  R3, 0(R4)
I3: add R5, R1, R3
I4: sub R6, R5, R7
```

(1) 若无转发机制，需要插入多少个阻塞周期？画出流水线时空图。(2) 若有转发机制（但不能消除 Load-use 冒险），需要插入多少个阻塞周期？(3) 计算两种情况的总执行时间。

44.（11 分，组成原理）在 8 位字长计算机中，$[x]_{补}=6\mathrm{AH}$，$[y]_{补}=9\mathrm{FH}$。(1) 写出 x, y 的十进制真值；(2) 用补码完成 $x+y$，给出 8 位结果并判断有符号溢出 OF；(3) 完成 $x-y$，给出 8 位结果并判断 OF。

45.（8 分，操作系统）读者-写者问题：一个文件可被多个读者同时读，但写者必须独占访问。用信号量实现读者优先的同步算法（写伪代码），并说明是否可能导致写者饥饿。

46.（7 分，操作系统）某请求分页系统为进程分配了 3 个物理块，页面访问序列为

$$1,2,3,4,2,1,5,6,2,1,2,3,7,6,3,2,1,2,3,6.$$

(1) 分别用 OPT、FIFO、LRU 算法计算缺页次数；(2) 哪种算法在本序列中表现最好？为什么 OPT 无法在实际系统中实现？

47.（9 分，计算机网络） 主机 A 的 IP 地址为 192.168.10.66/27，主机 B 的 IP 地址为 192.168.10.92/27。(1) 写出/27 的子网掩码；(2)A 和 B 是否在同一子网？(3) 若 A 向 B 发送数据，A 需要 ARP 解析 B 的 MAC 地址吗？说明原因。

PART VI　第六部分

附录

第一章

真题索引使用说明

本书原有的 `appendix-exams.tex` 汇总了按年份和题号标注的历史题目，但其中部分年份、题号和题面尚未逐项对照教育部考试中心发布的原卷完成来源核验。为避免把未经核验的元数据当作事实，正式讲义暂不收录该索引。

> **注意**
>
> 章节正文和模拟卷中的题目可用于知识训练；凡标注具体年份与题号的题目，在来源核验完成前，应视为“真题风格题”，不据此判断某一知识点在某年是否实际出现。刷历年真题时请以合法取得的原卷及官方答案为准。

建议的核验字段

每一道拟收入索引的真题至少应核对：考试年份、题号、学科归属、题型、分值、题面、选项、官方答案，以及题图是否完整。只有这些字段全部与原卷一致后，才恢复到可发布索引中。